

Linux Fun

Paul Cobbaut

Linux Fun

Paul Cobbaut

It-2.0

Publication date Thu 16 Apr 2015 01:02:51 AM CEST

Abstract

This book is meant to be used in an instructor-led training. For self-study, the intent is to read this book next to a working Linux computer so you can immediately do every subject, practicing each command.

This book is aimed at novice Linux system administrators (and might be interesting and useful for home users that want to know a bit more about their Linux system). However, this book is not meant as an introduction to Linux desktop applications like text editors, browsers, mail clients, multimedia or office applications.

More information and free .pdf available at <http://linux-training.be> .

Feel free to contact the author:

- Paul Cobbaut: paul.cobbaut@gmail.com, <http://www.linkedin.com/in/cobbaut>

Contributors to the Linux Training project are:

- Serge van Ginderachter: serge@ginsys.be, build scripts; infrastructure setup; minor stuff
- Hendrik De Vloed: hendrik.devloed@ugent.be, buildheader.pl script

We'd also like to thank our reviewers:

- Wouter Verhelst: wouter@grep.be, <http://grep.be>
- Geert Goossens: mail.goossens.geert@gmail.com, <http://www.linkedin.com/in/geertgoossens>
- Elie De Brauer: elie@de-brauer.be, <http://www.de-brauer.be>
- Christophe Vandeplas: christophe@vandeplas.com, <http://christophe.vandeplas.com>
- Bert Desmet: bert@devnox.be, <http://bdesmet.be>
- Rich Yonts: richyonts@gmail.com,

Copyright 2007-2015 Paul Cobbaut

Permission is granted to copy, distribute and/or modify this document under the terms of the **GNU Free Documentation License**, Version 1.3 or any later version published by the Free

Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled 'GNU Free Documentation License'.

Table of Contents

I. introduction to Linux	1
1. Linux history	3
1.1. Linux history	4
2. distributions	5
2.1. Red Hat	6
2.2. Ubuntu	6
2.3. Debian	6
2.4. Other	6
2.5. Which to choose ?	7
3. licensing	8
3.1. about software licenses	9
3.2. public domain software and freeware	9
3.3. Free Software or Open Source Software	9
3.4. GNU General Public License	10
3.5. using GPLv3 software	10
3.6. BSD license	11
3.7. other licenses	11
3.8. combination of software licenses	11
II. installing Linux	12
4. installing Debian 8	14
4.1. Debian	15
4.2. Downloading	15
4.3. virtualbox networking	31
4.4. setting the hostname	33
4.5. adding a static ip address	33
4.6. Debian package management	34
5. installing CentOS 7	35
5.1. download a CentOS 7 image	36
5.2. Virtualbox	38
5.3. CentOS 7 installing	43
5.4. CentOS 7 first logon	51
5.5. Virtualbox network interface	52
5.6. configuring the network	53
5.7. adding one static ip address	53
5.8. package management	54
5.9. logon from Linux and MacOSX	55
5.10. logon from MS Windows	55
6. getting Linux at home	57
6.1. download a Linux CD image	58
6.2. download Virtualbox	58
6.3. create a virtual machine	59
6.4. attach the CD image	64
6.5. install Linux	67
III. first steps on the command line	68
7. man pages	70
7.1. man \$command	71
7.2. man \$configfile	71
7.3. man \$daemon	71
7.4. man -k (apropos)	71
7.5. whatis	71
7.6. whereis	71
7.7. man sections	72
7.8. man \$section \$file	72
7.9. man man	72
7.10. mandb	72

8. working with directories	73
8.1. pwd	74
8.2. cd	74
8.3. absolute and relative paths	75
8.4. path completion	76
8.5. ls	76
8.6. mkdir	78
8.7. rmdir	78
8.8. practice: working with directories	80
8.9. solution: working with directories	81
9. working with files	83
9.1. all files are case sensitive	84
9.2. everything is a file	84
9.3. file	84
9.4. touch	85
9.5. rm	86
9.6. cp	87
9.7. mv	88
9.8. rename	89
9.9. practice: working with files	90
9.10. solution: working with files	91
10. working with file contents	93
10.1. head	94
10.2. tail	94
10.3. cat	95
10.4. tac	96
10.5. more and less	97
10.6. strings	97
10.7. practice: file contents	98
10.8. solution: file contents	99
11. the Linux file tree	100
11.1. filesystem hierarchy standard	101
11.2. man hier	101
11.3. the root directory /	101
11.4. binary directories	102
11.5. configuration directories	104
11.6. data directories	106
11.7. in memory directories	108
11.8. /usr Unix System Resources	113
11.9. /var variable data	115
11.10. practice: file system tree	117
11.11. solution: file system tree	119
IV. shell expansion	121
12. commands and arguments	124
12.1. arguments	125
12.2. white space removal	125
12.3. single quotes	126
12.4. double quotes	126
12.5. echo and quotes	126
12.6. commands	127
12.7. aliases	128
12.8. displaying shell expansion	129
12.9. practice: commands and arguments	130
12.10. solution: commands and arguments	132
13. control operators	134
13.1. ; semicolon	135
13.2. & ampersand	135
13.3. \$? dollar question mark	135

13.4. && double ampersand	136
13.5. double vertical bar	136
13.6. combining && and 	136
13.7. # pound sign	137
13.8. \ escaping special characters	137
13.9. practice: control operators	138
13.10. solution: control operators	139
14. shell variables	140
14.1. \$ dollar sign	141
14.2. case sensitive	141
14.3. creating variables	141
14.4. quotes	142
14.5. set	142
14.6. unset	142
14.7. \$PS1	143
14.8. \$PATH	144
14.9. env	145
14.10. export	145
14.11. delineate variables	146
14.12. unbound variables	146
14.13. practice: shell variables	147
14.14. solution: shell variables	148
15. shell embedding and options	149
15.1. shell embedding	150
15.2. shell options	151
15.3. practice: shell embedding	152
15.4. solution: shell embedding	153
16. shell history	154
16.1. repeating the last command	155
16.2. repeating other commands	155
16.3. history	155
16.4. !n	155
16.5. Ctrl-r	156
16.6. \$HISTSIZE	156
16.7. \$HISTFILE	156
16.8. \$HISTFILESIZE	156
16.9. prevent recording a command	157
16.10. (optional)regular expressions	157
16.11. (optional) Korn shell history	157
16.12. practice: shell history	158
16.13. solution: shell history	159
17. file globbing	160
17.1. * asterisk	161
17.2. ? question mark	161
17.3. [] square brackets	162
17.4. a-z and 0-9 ranges	163
17.5. \$LANG and square brackets	163
17.6. preventing file globbing	164
17.7. practice: shell globbing	165
17.8. solution: shell globbing	166
V. pipes and commands	168
18. I/O redirection	170
18.1. stdin, stdout, and stderr	171
18.2. output redirection	172
18.3. error redirection	174
18.4. output redirection and pipes	175
18.5. joining stdout and stderr	175
18.6. input redirection	176

18.7. confusing redirection	177
18.8. quick file clear	177
18.9. practice: input/output redirection	178
18.10. solution: input/output redirection	179
19. filters	180
19.1. cat	181
19.2. tee	181
19.3. grep	181
19.4. cut	183
19.5. tr	183
19.6. wc	184
19.7. sort	185
19.8. uniq	186
19.9. comm	187
19.10. od	188
19.11. sed	189
19.12. pipe examples	190
19.13. practice: filters	191
19.14. solution: filters	192
20. basic Unix tools	194
20.1. find	195
20.2. locate	196
20.3. date	196
20.4. cal	197
20.5. sleep	197
20.6. time	198
20.7. gzip - gunzip	199
20.8. zcat - zmore	199
20.9. bzip2 - bunzip2	200
20.10. bzip2 - bunzip2	200
20.11. practice: basic Unix tools	201
20.12. solution: basic Unix tools	202
21. regular expressions	204
21.1. regex versions	205
21.2. grep	206
21.3. rename	211
21.4. sed	214
21.5. bash history	218
VI. vi	219
22. Introduction to vi	221
22.1. command mode and insert mode	222
22.2. start typing (a A i l o O)	222
22.3. replace and delete a character (r x X)	223
22.4. undo and repeat (u .)	223
22.5. cut, copy and paste a line (dd yy p P)	223
22.6. cut, copy and paste lines (3dd 2yy)	224
22.7. start and end of a line (0 or ^ and \$)	224
22.8. join two lines (J) and more	224
22.9. words (w b)	225
22.10. save (or not) and exit (:w :q :q!)	225
22.11. Searching (/ ?)	225
22.12. replace all (:1,\$ s/foo/bar/g)	226
22.13. reading files (:r :r !cmd)	226
22.14. text buffers	226
22.15. multiple files	226
22.16. abbreviations	227
22.17. key mappings	228
22.18. setting options	228

22.19. practice: vi(m)	229
22.20. solution: vi(m)	230
VII. scripting	231
23. scripting introduction	233
23.1. prerequisites	234
23.2. hello world	234
23.3. she-bang	234
23.4. comment	235
23.5. variables	235
23.6. sourcing a script	235
23.7. troubleshooting a script	236
23.8. prevent setuid root spoofing	236
23.9. practice: introduction to scripting	237
23.10. solution: introduction to scripting	238
24. scripting loops	239
24.1. test []	240
24.2. if then else	241
24.3. if then elif	241
24.4. for loop	241
24.5. while loop	242
24.6. until loop	242
24.7. practice: scripting tests and loops	243
24.8. solution: scripting tests and loops	244
25. scripting parameters	246
25.1. script parameters	247
25.2. shift through parameters	248
25.3. runtime input	248
25.4. sourcing a config file	249
25.5. get script options with getopts	250
25.6. get shell options with shopt	251
25.7. practice: parameters and options	252
25.8. solution: parameters and options	253
26. more scripting	254
26.1. eval	255
26.2. (())	255
26.3. let	256
26.4. case	257
26.5. shell functions	258
26.6. practice : more scripting	259
26.7. solution : more scripting	260
VIII. local user management	262
27. introduction to users	265
27.1. whoami	266
27.2. who	266
27.3. who am i	266
27.4. w	266
27.5. id	266
27.6. su to another user	267
27.7. su to root	267
27.8. su as root	267
27.9. su - \$username	267
27.10. su -	267
27.11. run a program as another user	268
27.12. visudo	268
27.13. sudo su -	269
27.14. sudo logging	269
27.15. practice: introduction to users	270
27.16. solution: introduction to users	271

28. user management	273
28.1. user management	274
28.2. /etc/passwd	274
28.3. root	274
28.4. useradd	275
28.5. /etc/default/useradd	275
28.6. userdel	275
28.7. usermod	275
28.8. creating home directories	276
28.9. /etc/skel/	276
28.10. deleting home directories	276
28.11. login shell	277
28.12. chsh	277
28.13. practice: user management	278
28.14. solution: user management	279
29. user passwords	281
29.1. passwd	282
29.2. shadow file	282
29.3. encryption with passwd	283
29.4. encryption with openssl	283
29.5. encryption with crypt	284
29.6. /etc/login.defs	285
29.7. chage	285
29.8. disabling a password	286
29.9. editing local files	286
29.10. practice: user passwords	287
29.11. solution: user passwords	288
30. user profiles	290
30.1. system profile	291
30.2. ~/.bash_profile	291
30.3. ~/.bash_login	292
30.4. ~/.profile	292
30.5. ~/.bashrc	292
30.6. ~/.bash_logout	293
30.7. Debian overview	294
30.8. RHEL5 overview	294
30.9. practice: user profiles	295
30.10. solution: user profiles	296
31. groups	297
31.1. groupadd	298
31.2. group file	298
31.3. groups	298
31.4. usermod	299
31.5. groupmod	299
31.6. groupdel	299
31.7. gpasswd	300
31.8. newgrp	301
31.9. vigr	301
31.10. practice: groups	302
31.11. solution: groups	303
IX. file security	304
32. standard file permissions	306
32.1. file ownership	307
32.2. list of special files	309
32.3. permissions	310
32.4. practice: standard file permissions	315
32.5. solution: standard file permissions	316
33. advanced file permissions	318

33.1. sticky bit on directory	319
33.2. setgid bit on directory	319
33.3. setgid and setuid on regular files	320
33.4. setuid on sudo	320
33.5. practice: sticky, setuid and setgid bits	321
33.6. solution: sticky, setuid and setgid bits	322
34. access control lists	324
34.1. acl in /etc/fstab	325
34.2. getfacl	325
34.3. setfacl	325
34.4. remove an acl entry	326
34.5. remove the complete acl	326
34.6. the acl mask	326
34.7. eiciel	327
35. file links	328
35.1. inodes	329
35.2. about directories	330
35.3. hard links	331
35.4. symbolic links	332
35.5. removing links	332
35.6. practice : links	333
35.7. solution : links	334
X. process management	335
36. introduction to processes	337
36.1. terminology	338
36.2. basic process management	339
36.3. signalling processes	343
36.4. practice : basic process management	346
36.5. solution : basic process management	347
37. process priorities	349
37.1. priority and nice values	350
37.2. practice : process priorities	353
37.3. solution : process priorities	354
38. background jobs	356
38.1. background processes	357
38.2. practice : background processes	359
38.3. solution : background processes	360
XI. disk management	362
39. disk devices	365
39.1. terminology	366
39.2. device naming	368
39.3. discovering disk devices	369
39.4. erasing a hard disk	374
39.5. advanced hard disk settings	375
39.6. practice: hard disk devices	376
39.7. solution: hard disk devices	377
40. disk partitions	379
40.1. about partitions	380
40.2. discovering partitions	381
40.3. partitioning new disks	383
40.4. about the partition table	385
40.5. GUID partition table	386
40.6. labeling with parted	386
40.7. practice: partitions	388
40.8. solution: partitions	389
41. file systems	390
41.1. about file systems	391
41.2. common file systems	392

41.3. putting a file system on a partition	395
41.4. tuning a file system	396
41.5. checking a file system	397
41.6. practice: file systems	398
41.7. solution: file systems	399
42. mounting	400
42.1. mounting local file systems	401
42.2. displaying mounted file systems	402
42.3. from start to finish	404
42.4. permanent mounts	405
42.5. securing mounts	406
42.6. mounting remote file systems	407
42.7. practice: mounting file systems	408
42.8. solution: mounting file systems	409
43. troubleshooting tools	411
43.1. lsof	412
43.2. fuser	413
43.3. chroot	414
43.4. iostat	415
43.5. iotop	416
43.6. vmstat	417
43.7. practice: troubleshooting tools	418
43.8. solution: troubleshooting tools	419
44. introduction to uuid's	420
44.1. about unique objects	421
44.2. tune2fs	421
44.3. uuid	421
44.4. uuid in /etc/fstab	422
44.5. uuid as a boot device	423
44.6. practice: uuid and filesystems	424
44.7. solution: uuid and filesystems	425
45. introduction to raid	426
45.1. hardware or software	426
45.2. raid levels	427
45.3. building a software raid5 array	429
45.4. practice: raid	432
45.5. solution: raid	433
46. logical volume management	434
46.1. introduction to lvm	435
46.2. lvm terminology	436
46.3. example: using lvm	437
46.4. example: extend a logical volume	439
46.5. example: resize a physical Volume	441
46.6. example: mirror a logical volume	443
46.7. example: snapshot a logical volume	444
46.8. verifying existing physical volumes	445
46.9. verifying existing volume groups	447
46.10. verifying existing logical volumes	448
46.11. manage physical volumes	449
46.12. manage volume groups	451
46.13. manage logical volumes	453
46.14. practice : lvm	455
46.15. solution : lvm	456
47. iSCSI devices	460
47.1. iSCSI terminology	461
47.2. iSCSI Target in RHEL/CentOS	461
47.3. iSCSI Initiator in RHEL/CentOS	463
47.4. iSCSI target on Debian	465

47.5. iSCSI target setup with dd files	466
47.6. iSCSI initiator on ubuntu	468
47.7. using iSCSI devices	470
47.8. practice: iSCSI devices	471
47.9. solution: iSCSI devices	472
48. introduction to multipathing	473
48.1. install multipath	474
48.2. configure multipath	474
48.3. network	475
48.4. start multipathd and iscsi	475
48.5. multipath list	477
48.6. using the device	478
48.7. practice: multipathing	479
48.8. solution: multipathing	480
XII. boot management	482
49. bootloader	484
49.1. boot terminology	485
49.2. grub	488
49.3. grub2	493
49.4. lilo	494
49.5. practice: bootloader	495
49.6. solution: bootloader	496
50. init and runlevels	497
50.1. system init(ialization)	498
50.2. daemon or demon ?	503
50.3. starting and stopping daemons	503
50.4. chkconfig	504
50.5. update-rc.d	506
50.6. bum	507
50.7. runlevels	508
50.8. systemd	510
50.9. practice: init	516
50.10. solution : init	517
XIII. system management	519
51. scheduling	521
51.1. one time jobs with at	522
51.2. cron	524
51.3. practice : scheduling	526
51.4. solution : scheduling	527
52. logging	528
52.1. login logging	529
52.2. syslogd	532
52.3. logger	535
52.4. watching logs	535
52.5. rotating logs	536
52.6. practice : logging	537
52.7. solution : logging	538
53. memory management	540
53.1. displaying memory and cache	541
53.2. managing swap space	542
53.3. monitoring memory with vmstat	544
53.4. practice : memory	545
53.5. solution : memory	546
54. resource monitoring	547
54.1. four basic resources	548
54.2. top	548
54.3. free	548
54.4. watch	549

54.5. vmstat	549
54.6. iostat	550
54.7. mpstat	551
54.8. sadc and sar	551
54.9. ntop	552
54.10. iftop	552
54.11. iptraf	552
54.12. nmon	553
54.13. htop	553
55. package management	554
55.1. package terminology	555
55.2. deb package management	557
55.3. apt-get	559
55.4. aptitude	562
55.5. apt	563
55.6. rpm	564
55.7. yum	566
55.8. alien	573
55.9. downloading software outside the repository	574
55.10. compiling software	574
55.11. practice: package management	575
55.12. solution: package management	576
XIV. network management	577
56. general networking	580
56.1. network layers	581
56.2. unicast, multicast, broadcast, anycast	584
56.3. lan-wan-man	586
56.4. internet - intranet - extranet	588
56.5. tcp/ip	589
57. interface configuration	590
57.1. to gui or not to gui	591
57.2. Debian nic configuration	592
57.3. RHEL nic configuration	594
57.4. ifconfig	596
57.5. ip	598
57.6. dhclient	599
57.7. hostname	599
57.8. arp	600
57.9. route	601
57.10. ping	601
57.11. optional: ethtool	602
57.12. practice: interface configuration	603
57.13. solution: interface configuration	604
58. network sniffing	606
58.1. wireshark	607
58.2. tcpdump	611
58.3. practice: network sniffing	612
58.4. solution: network sniffing	613
59. binding and bonding	614
59.1. binding on Redhat/Fedora	615
59.2. binding on Debian/Ubuntu	616
59.3. bonding on Redhat/Fedora	617
59.4. bonding on Debian/Ubuntu	619
59.5. practice: binding and bonding	621
59.6. solution: binding and bonding	622
60. ssh client and server	623
60.1. about ssh	624
60.2. log on to a remote server	626

60.3. executing a command in remote	626
60.4. scp	627
60.5. setting up passwordless ssh	628
60.6. X forwarding via ssh	629
60.7. troubleshooting ssh	630
60.8. sshd	631
60.9. sshd keys	631
60.10. ssh-agent	631
60.11. practice: ssh	632
60.12. solution: ssh	633
61. introduction to nfs	635
61.1. nfs protocol versions	636
61.2. rpcinfo	636
61.3. server configuration	637
61.4. /etc/exports	637
61.5. exportfs	637
61.6. client configuration	638
61.7. practice: introduction to nfs	639
62. introduction to networking	640
62.1. introduction to iptables	641
62.2. practice : iptables	642
62.3. solution : iptables	643
62.4. xinetd and inetd	644
62.5. practice : inetd and xinetd	646
62.6. network file system	647
62.7. practice : network file system	649
XV. kernel management	650
63. the Linux kernel	652
63.1. about the Linux kernel	653
63.2. Linux kernel source	656
63.3. kernel boot files	660
63.4. Linux kernel modules	662
63.5. compiling a kernel	667
63.6. compiling one module	670
64. library management	672
64.1. introduction	673
64.2. /lib and /usr/lib	673
64.3. ldd	673
64.4. ltrace	674
64.5. dpkg -S and debsums	674
64.6. rpm -qf and rpm -V	675
64.7. tracing with strace	676
XVI. backup management	677
65. backup	679
65.1. About tape devices	679
65.2. Compression	680
65.3. tar	680
65.4. Backup Types	682
65.5. dump and restore	683
65.6. cpio	683
65.7. dd	684
65.8. split	685
65.9. practice: backup	685
XVII. apache and squid	687
66. apache web server	689
66.1. introduction to apache	690
66.2. port virtual hosts on Debian	697
66.3. named virtual hosts on Debian	701

66.4. password protected website on Debian	703
66.5. port virtual hosts on CentOS	704
66.6. named virtual hosts on CentOS	708
66.7. password protected website on CentOS	710
66.8. troubleshooting apache	712
66.9. virtual hosts example	713
66.10. aliases and redirects	713
66.11. more on .htaccess	713
66.12. traffic	713
66.13. self signed cert on Debian	714
66.14. self signed cert on RHEL/CentOS	716
66.15. practice: apache	718
67. introduction to squid	719
67.1. about proxy servers	719
67.2. installing squid	720
67.3. port 3128	720
67.4. starting and stopping	720
67.5. client proxy settings	721
67.6. upside down images	723
67.7. /var/log/squid	725
67.8. access control	725
67.9. testing squid	725
67.10. name resolution	725
XVIII. mysql database	727
68. introduction to sql using mysql	729
68.1. installing mysql	730
68.2. accessing mysql	731
68.3. mysql databases	733
68.4. mysql tables	735
68.5. mysql records	737
68.6. joining two tables	740
68.7. mysql triggers	741
XIX. dns server	743
69. introduction to DNS	745
69.1. about dns	746
69.2. dns namespace	749
69.3. caching only servers	754
69.4. authoritative dns servers	757
69.5. primary and secondary	757
69.6. zone transfers	757
69.7. master and slave	759
69.8. SOA record	759
69.9. full or incremental zone transfers	760
69.10. DNS cache	761
69.11. forward lookup zone example	762
69.12. example: caching only DNS server	763
69.13. example: caching only with forwarder	765
69.14. example: primary authoritative server	767
69.15. example: a DNS slave server	771
69.16. practice: dns	773
69.17. solution: dns	774
70. advanced DNS	775
70.1. example: DNS round robin	776
70.2. DNS delegation	777
70.3. example: DNS delegation	778
70.4. example: split-horizon dns	780
70.5. old dns topics	782
XX. dhcp server	786

71. introduction to dhcp	788
71.1. four broadcasts	789
71.2. picturing dhcp	790
71.3. installing a dhcp server	791
71.4. dhcp server on Red Hat	791
71.5. dhcp options	791
71.6. client reservations	791
71.7. example config files	792
71.8. older example config files	792
71.9. advanced dhcp	794
71.10. Practice: dhcp	795
XXI. iptables firewall	796
72. introduction to routers	798
72.1. router or firewall	799
72.2. packet forwarding	799
72.3. packet filtering	799
72.4. stateful	799
72.5. nat (network address translation)	800
72.6. pat (port address translation)	800
72.7. snat (source nat)	800
72.8. masquerading	800
72.9. dnat (destination nat)	800
72.10. port forwarding	800
72.11. /proc/sys/net/ipv4/ip_forward	801
72.12. /etc/sysctl.conf	801
72.13. sysctl	801
72.14. practice: packet forwarding	802
72.15. solution: packet forwarding	804
73. iptables firewall	807
73.1. iptables tables	808
73.2. starting and stopping iptables	808
73.3. the filter table	809
73.4. practice: packet filtering	814
73.5. solution: packet filtering	815
73.6. network address translation	816
XXII. selinux	819
74. introduction to SELinux	821
74.1. selinux modes	822
74.2. logging	822
74.3. activating selinux	822
74.4. getenforce	823
74.5. setenforce	823
74.6. sestatus	824
74.7. policy	824
74.8. /etc/selinux/config	824
74.9. DAC or MAC	825
74.10. ls -Z	825
74.11. -Z	825
74.12. /selinux	826
74.13. identity	826
74.14. role	826
74.15. type (or domain)	827
74.16. security context	828
74.17. transition	828
74.18. extended attributes	829
74.19. process security context	829
74.20. chcon	829
74.21. an example	830

74.22. setroubleshoot	832
74.23. booleans	834
XXIII. Introduction to Samba	835
75. introduction to samba	838
75.1. verify installed version	839
75.2. installing samba	840
75.3. documentation	841
75.4. starting and stopping samba	842
75.5. samba daemons	843
75.6. the SMB protocol	844
75.7. practice: introduction to samba	845
76. getting started with samba	846
76.1. /etc/samba/smb.conf	847
76.2. /usr/bin/testparm	848
76.3. /usr/bin/smbclient	849
76.4. /usr/bin/smbtree	851
76.5. server string	852
76.6. Samba Web Administration Tool (SWAT)	853
76.7. practice: getting started with samba	854
76.8. solution: getting started with samba	855
77. a read only file server	857
77.1. Setting up a directory to share	858
77.2. configure the share	858
77.3. restart the server	859
77.4. verify the share	859
77.5. a note on netcat	861
77.6. practice: read only file server	862
77.7. solution: read only file server	863
78. a writable file server	864
78.1. set up a directory to share	865
78.2. share section in smb.conf	865
78.3. configure the share	865
78.4. test connection with windows	865
78.5. test writing with windows	866
78.6. How is this possible ?	866
78.7. practice: writable file server	867
78.8. solution: writable file server	868
79. samba first user account	869
79.1. creating a samba user	870
79.2. ownership of files	870
79.3. /usr/bin/smbpasswd	870
79.4. /etc/samba/smbpasswd	870
79.5. passdb backend	871
79.6. forcing this user	871
79.7. practice: first samba user account	872
79.8. solution: first samba user account	873
80. samba authentication	874
80.1. creating the users on Linux	875
80.2. creating the users on samba	875
80.3. security = user	875
80.4. configuring the share	876
80.5. testing access with net use	876
80.6. testing access with smbclient	876
80.7. verify ownership	877
80.8. common problems	877
80.9. practice : samba authentication	879
80.10. solution: samba authentication	880
81. samba securing shares	881

81.1. security based on user name	882
81.2. security based on ip-address	882
81.3. security through obscurity	883
81.4. file system security	883
81.5. practice: securing shares	885
81.6. solution: securing shares	886
82. samba domain member	888
82.1. changes in smb.conf	889
82.2. joining an Active Directory domain	890
82.3. winbind	891
82.4. wbinfos	891
82.5. getent	892
82.6. file ownership	893
82.7. practice : samba domain member	894
83. samba domain controller	895
83.1. about Domain Controllers	896
83.2. About security modes	896
83.3. About password backends	897
83.4. [global] section in smb.conf	897
83.5. netlogon share	898
83.6. other [share] sections	898
83.7. Users and Groups	899
83.8. tdbsam	899
83.9. about computer accounts	900
83.10. local or roaming profiles	900
83.11. Groups in NTFS acls	901
83.12. logon scripts	902
83.13. practice: samba domain controller	903
84. a brief look at samba 4	904
84.1. Samba 4 alpha 6	906
XXIV. introducing git	908
85. git	910
85.1. git	911
85.2. installing git	912
85.3. starting a project	912
85.4. git branches	915
85.5. to be continued... ..	916
85.6. github.com	917
85.7. add your public key to github	917
85.8. practice: git	918
XXV. ipv6	919
86. Introduction to ipv6	921
86.1. about ipv6	922
86.2. network id and host id	922
86.3. host part generation	922
86.4. ipv4 mapped ipv6 address	923
86.5. link local addresses	923
86.6. unique local addresses	923
86.7. globally unique unicast addresses	923
86.8. 6to4	923
86.9. ISP	924
86.10. non routable addresses	924
86.11. ping6	924
86.12. Belgium and ipv6	925
86.13. other websites	925
86.14. 6to4 gateways	927
86.15. ping6 and dns	927
86.16. ipv6 and tcp/http	927

86.17. ipv6 PTR record	927
86.18. 6to4 setup on Linux	927
XXVI. Appendices	930
A. certifications	932
A.1. Certification	932
B. Raspberry Pi Linux	934
B.1. buying the hardware	935
B.2. downloading the images	936
B.3. writing the SD cards	937
B.4. adjusting network settings	938
B.5. connecting the hardware	939
B.6. connecting with ssh	940
B.7. camera module	941
B.8.	942
B.9. building a stack of pi's	943
B.10. sending mail from the pi	943
B.11. disable ipv6	944
B.12.	945
B.13.	946
B.14.	947
B.15. raspi camera	947
C. keyboard settings	948
C.1. about keyboard layout	948
C.2. X Keyboard Layout	948
C.3. shell keyboard layout	948
D. hardware	950
D.1. buses	950
D.2. interrupts	951
D.3. io ports	952
D.4. dma	952
E. installing Linux	954
E.1. about	954
E.2. legacy content	954
F. disk quotas	957
F.1. About Disk Quotas	957
F.2. Practice Disk quotas	957
G. introduction to vnc	958
G.1. About VNC	958
G.2. VNC Server	958
G.3. VNC Client	958
G.4. Practice VNC	959
H. cloning	960
H.1. About cloning	960
H.2. About offline cloning	960
H.3. Offline cloning example	960
I. License	962
Index	969

List of Tables

2.1. choosing a Linux distro	7
4.1. Debian releases	15
22.1. getting to command mode	222
22.2. switch to insert mode	222
22.3. replace and delete	223
22.4. undo and repeat	223
22.5. cut, copy and paste a line	223
22.6. cut, copy and paste lines	224
22.7. start and end of line	224
22.8. join two lines	224
22.9. words	225
22.10. save and exit vi	225
22.11. searching	225
22.12. replace	226
22.13. read files and input	226
22.14. text buffers	226
22.15. multiple files	227
22.16. abbreviations	227
30.1. Debian User Environment	294
30.2. Red Hat User Environment	294
32.1. Unix special files	309
32.2. standard Unix file permissions	310
32.3. Unix file permissions position	310
32.4. Octal permissions	313
39.1. ide device naming	368
39.2. scsi device naming	368
40.1. primary, extended and logical partitions	380
40.2. Partition naming	380
50.1. systemd power management	514
69.1. the first top level domains	751
69.2. new general purpose tld's	751
72.1. Packet Forwarding Exercise	802
72.2. Packet Forwarding Solution	804

Part I. introduction to Linux

Table of Contents

1. Linux history	3
1.1. Linux history	4
2. distributions	5
2.1. Red Hat	6
2.2. Ubuntu	6
2.3. Debian	6
2.4. Other	6
2.5. Which to choose ?	7
3. licensing	8
3.1. about software licenses	9
3.2. public domain software and freeware	9
3.3. Free Software or Open Source Software	9
3.4. GNU General Public License	10
3.5. using GPLv3 software	10
3.6. BSD license	11
3.7. other licenses	11
3.8. combination of software licenses	11

Chapter 1. Linux history

This chapter briefly tells the history of Unix and where Linux fits in.

If you are eager to start working with Linux without this blah, blah, blah over history, distributions, and licensing then jump straight to **Part II - Chapter 6. Working with Directories** page 28.

1.1. Linux history

All modern operating systems have their roots in 1969 when **Dennis Ritchie** and **Ken Thompson** developed the C language and the **Unix** operating system at AT&T Bell Labs. They shared their source code (yes, there was open source back in the Seventies) with the rest of the world, including the hippies in Berkeley California. By 1975, when AT&T started selling Unix commercially, about half of the source code was written by others. The hippies were not happy that a commercial company sold software that they had written; the resulting (legal) battle ended in there being two versions of **Unix** in the Seventies : the official AT&T Unix, and the free **BSD** Unix.

In the Eighties many companies started developing their own Unix: IBM created AIX, Sun SunOS (later Solaris), HP HP-UX and about a dozen other companies did the same. The result was a mess of Unix dialects and a dozen different ways to do the same thing. And here is the first real root of **Linux**, when **Richard Stallman** aimed to end this era of Unix separation and everybody re-inventing the wheel by starting the **GNU** project (GNU is Not Unix). His goal was to make an operating system that was freely available to everyone, and where everyone could work together (like in the Seventies). Many of the command line tools that you use today on **Linux** or Solaris are GNU tools.

The Nineties started with **Linus Torvalds**, a Swedish speaking Finnish student, buying a 386 computer and writing a brand new POSIX compliant kernel. He put the source code online, thinking it would never support anything but 386 hardware. Many people embraced the combination of this kernel with the GNU tools, and the rest, as they say, is history.

Today more than 90 percent of supercomputers (including the complete top 10), more than half of all smartphones, many millions of desktop computers, around 70 percent of all web servers, a large chunk of tablet computers, and several appliances (dvd-players, washing machines, dsl modems, routers, ...) run **Linux**. It is by far the most commonly used operating system in the world.

Linux kernel version 3.2 was released in January 2012. Its source code grew by almost two hundred thousand lines (compared to version 3.1) thanks to contributions of over 4000 developers paid by about 200 commercial companies including Red Hat, Intel, Broadcom, Texas Instruments, IBM, Novell, Qualcomm, Samsung, Nokia, Oracle, Google and even Microsoft.

```
http://en.wikipedia.org/wiki/Dennis_Ritchie
http://en.wikipedia.org/wiki/Richard_Stallman
http://en.wikipedia.org/wiki/Linus_Torvalds
http://kernel.org
http://lwn.net/Articles/472852/
http://www.linuxfoundation.org/
http://en.wikipedia.org/wiki/Linux
http://www.levenez.com/unix/ (a huge Unix history poster)
```

Chapter 2. distributions

This chapter gives a short overview of current Linux distributions.

A Linux **distribution** is a collection of (usually open source) software on top of a Linux kernel. A distribution (or short, distro) can bundle server software, system management tools, documentation and many desktop applications in a **central secure software repository**. A distro aims to provide a common look and feel, secure and easy software management and often a specific operational purpose.

Let's take a look at some popular distributions.

2.1. Red Hat

Red Hat is a billion dollar commercial Linux company that puts a lot of effort in developing Linux. They have hundreds of Linux specialists and are known for their excellent support. They give their products (Red Hat Enterprise Linux and Fedora) away for free. While **Red Hat Enterprise Linux** (RHEL) is well tested before release and supported for up to seven years after release, **Fedora** is a distro with faster updates but without support.

2.2. Ubuntu

Canonical started sending out free compact discs with **Ubuntu** Linux in 2004 and quickly became popular for home users (many switching from Microsoft Windows). Canonical wants Ubuntu to be an easy to use graphical Linux desktop without need to ever see a command line. Of course they also want to make a profit by selling support for Ubuntu.

2.3. Debian

There is no company behind **Debian**. Instead there are thousands of well organised developers that elect a **Debian Project Leader** every two years. Debian is seen as one of the most stable Linux distributions. It is also the basis of every release of Ubuntu. Debian comes in three versions: stable, testing and unstable. Every Debian release is named after a character in the movie Toy Story.

2.4. Other

Distributions like CentOS, Oracle Enterprise Linux and Scientific Linux are based on Red Hat Enterprise Linux and share many of the same principles, directories and system administration techniques. **Linux Mint**, Edubuntu and many other *buntu named distributions are based on Ubuntu and thus share a lot with Debian. There are hundreds of other Linux distributions.

2.5. Which to choose ?

Below are some very personal opinions on some of the most popular Linux Distributions. Keep in mind that any of the below Linux distributions can be a stable server and a nice graphical desktop client.

Table 2.1. choosing a Linux distro

distribution name	reason(s) for using
Red Hat Enterprise (RHEL)	You are a manager and you want a good support contract.
CentOS	You want Red Hat without the support contract from Red Hat.
Fedora	You want Red Hat on your laptop/desktop.
Linux Mint	You want a personal graphical desktop to play movies, music and games.
Debian	My personal favorite for servers, laptops, and any other device.
Ubuntu	Very popular, based on Debian, not my favorite.
Kali	You want a pointy-clicky hacking interface.
others	Advanced users may prefer Arch, Gentoo, OpenSUSE, Scientific, ...

When you are new to Linux in 2015, go for the latest Mint or Fedora. If you only want to practice the Linux command line then install one Debian server and/or one CentOS server (without graphical interface).

Here are some links to help you choose:

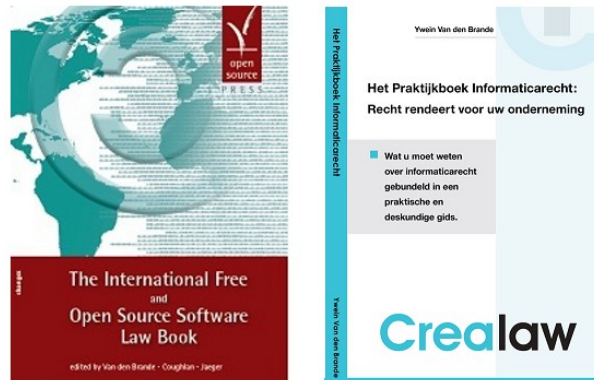
```
distrowatch.com
redhat.com
centos.org
debian.org
www.linuxmint.com
ubuntu.com
```

Chapter 3. licensing

This chapter briefly explains the different licenses used for distributing operating systems software.

Many thanks go to **Ywein Van den Brande** for writing most of this chapter.

Ywein is an attorney at law, co-author of **The International FOSS Law Book** and author of **Praktijkboek Informatierecht** (in Dutch).



<http://ifossilawbook.org>
<http://www.crealaw.eu>

3.1. about software licenses

There are two predominant software paradigms: **Free and Open Source Software** (FOSS) and **proprietary software**. The criteria for differentiation between these two approaches is based on control over the software. With **proprietary software**, control tends to lie more with the vendor, while with **Free and Open Source Software** it tends to be more weighted towards the end user. But even though the paradigms differ, they use the same **copyright laws** to reach and enforce their goals. From a legal perspective, **Free and Open Source Software** can be considered as software to which users generally receive more rights via their license agreement than they would have with a **proprietary software license**, yet the underlying license mechanisms are the same.

Legal theory states that the author of FOSS, contrary to the author of **public domain** software, has in no way whatsoever given up his rights on his work. FOSS supports on the rights of the author (the **copyright**) to impose FOSS license conditions. The FOSS license conditions need to be respected by the user in the same way as proprietary license conditions. Always check your license carefully before you use third party software.

Examples of proprietary software are **AIX** from IBM, **HP-UX** from HP and **Oracle Database 11g**. You are not authorised to install or use this software without paying a licensing fee. You are not authorised to distribute copies and you are not authorised to modify the closed source code.

3.2. public domain software and freeware

Software that is original in the sense that it is an intellectual creation of the author benefits **copyright** protection. Non-original software does not come into consideration for **copyright** protection and can, in principle, be used freely.

Public domain software is considered as software to which the author has given up all rights and on which nobody is able to enforce any rights. This software can be used, reproduced or executed freely, without permission or the payment of a fee. Public domain software can in certain cases even be presented by third parties as own work, and by modifying the original work, third parties can take certain versions of the public domain software out of the public domain again.

Freeware is not public domain software or FOSS. It is proprietary software that you can use without paying a license cost. However, the often strict license terms need to be respected.

Examples of freeware are **Adobe Reader**, **Skype** and **Command and Conquer: Tiberian Sun** (this game was sold as proprietary in 1999 and is since 2011 available as freeware).

3.3. Free Software or Open Source Software

Both the **Free Software** (translates to **vrije software** in Dutch and to **Logiciel Libre** in French) and the **Open Source Software** movement largely pursue similar goals and endorse similar software licenses. But historically, there has been some perception of differentiation due to different emphases. Where the **Free Software** movement focuses on the rights (the

four freedoms) which Free Software provides to its users, the **Open Source Software** movement points to its Open Source Definition and the advantages of peer-to-peer software development.

Recently, the term free and open source software or FOSS has arisen as a neutral alternative. A lesser-used variant is free/libre/open source software (FLOSS), which uses **libre** to clarify the meaning of free as in **freedom** rather than as in **at no charge**.

Examples of **free software** are **gcc**, **MySQL** and **gimp**.

Detailed information about the **four freedoms** can be found here:

<http://www.gnu.org/philosophy/free-sw.html>

The **open source definition** can be found at:

<http://www.opensource.org/docs/osd>

The above definition is based on the **Debian Free Software Guidelines** available here:

http://www.debian.org/social_contract#guidelines

3.4. GNU General Public License

More and more software is being released under the **GNU GPL** (in 2006 Java was released under the GPL). This license (v2 and v3) is the main license endorsed by the Free Software Foundation. It's main characteristic is the **copyleft** principle. This means that everyone in the chain of consecutive users, in return for the right of use that is assigned, needs to distribute the improvements he makes to the software and his derivative works under the same conditions to other users, if he chooses to distribute such improvements or derivative works. In other words, software which incorporates GNU GPL software, needs to be distributed in turn as GNU GPL software (or compatible, see below). It is not possible to incorporate copyright protected parts of GNU GPL software in a proprietary licensed work. The GPL has been upheld in court.

3.5. using GPLv3 software

You can use **GPLv3 software** almost without any conditions. If you solely run the software you even don't have to accept the terms of the GPLv3. However, any other use - such as modifying or distributing the software - implies acceptance.

In case you use the software internally (including over a network), you may modify the software without being obliged to distribute your modification. You may hire third parties to work on the software exclusively for you and under your direction and control. But if you modify the software and use it otherwise than merely internally, this will be considered as distribution. You must distribute your modifications under GPLv3 (the copyleft principle). Several more obligations apply if you distribute GPLv3 software. Check the GPLv3 license carefully.

You create output with GPLv3 software: The GPLv3 does not automatically apply to the output.

3.6. BSD license

There are several versions of the original Berkeley Distribution License. The most common one is the 3-clause license ("New BSD License" or "Modified BSD License").

This is a permissive free software license. The license places minimal restrictions on how the software can be redistributed. This is in contrast to copyleft licenses such as the GPLv. 3 discussed above, which have a copyleft mechanism.

This difference is of less importance when you merely use the software, but kicks in when you start redistributing verbatim copies of the software or your own modified versions.

3.7. other licenses

FOSS or not, there are many kind of licenses on software. You should read and understand them before using any software.

3.8. combination of software licenses

When you use several sources or wishes to redistribute your software under a different license, you need to verify whether all licenses are compatible. Some FOSS licenses (such as BSD) are compatible with proprietary licenses, but most are not. If you detect a license incompatibility, you must contact the author to negotiate different license conditions or refrain from using the incompatible software.

Part II. installing Linux

Table of Contents

4. installing Debian 8	14
4.1. Debian	15
4.2. Downloading	15
4.3. virtualbox networking	31
4.4. setting the hostname	33
4.5. adding a static ip address	33
4.6. Debian package management	34
5. installing CentOS 7	35
5.1. download a CentOS 7 image	36
5.2. Virtualbox	38
5.3. CentOS 7 installing	43
5.4. CentOS 7 first logon	51
5.5. Virtualbox network interface	52
5.6. configuring the network	53
5.7. adding one static ip address	53
5.8. package management	54
5.9. logon from Linux and MacOSX	55
5.10. logon from MS Windows	55
6. getting Linux at home	57
6.1. download a Linux CD image	58
6.2. download Virtualbox	58
6.3. create a virtual machine	59
6.4. attach the CD image	64
6.5. install Linux	67

Chapter 4. installing Debian 8

This module is a step by step demonstration of an actual installation of **Debian 8** (also known as **Jessie**).

We start by downloading an image from the internet and install **Debian 8** as a virtual machine in **Virtualbox**. We will also do some basic configuration of this new machine like setting an **ip address** and fixing a **hostname**.

This procedure should be very similar for other versions of **Debian**, and also for distributions like **Linux Mint**, **xubuntu/ubuntu/kubuntu** or **Mepis**. This procedure can also be helpful if you are using another virtualization solution.

Go to the next chapter if you want to install **CentOS**, **Fedora**, **Red Hat Enterprise Linux**,

4.1. Debian

Debian is one of the oldest Linux distributions. I use Debian myself on almost every computer that I own (including **raspbian** on the **Raspberry Pi**).

Debian comes in **releases** named after characters in the movie **Toy Story**. The **Jessie** release contains about 36000 packages.

Table 4.1. Debian releases

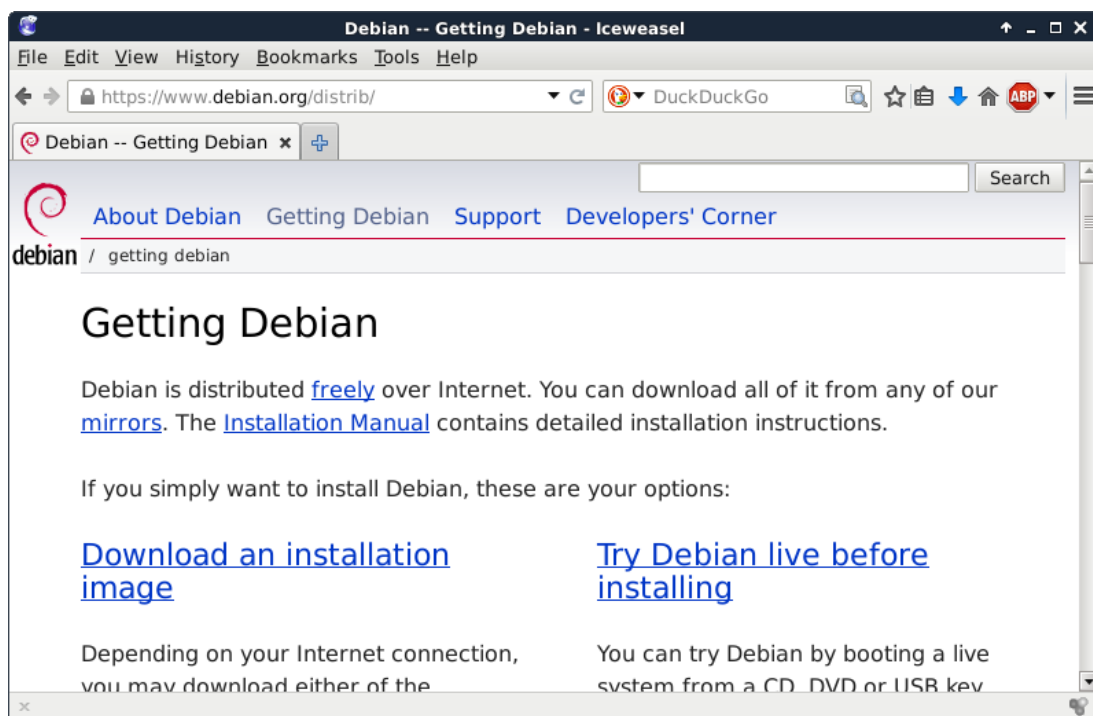
name	number	year
Woody	3.0	2002
Sarge	3.1	2005
Etch	4.0	2007
Lenny	5.0	2009
Squeeze	6.0	2011
Wheezy	7	2013
Jessie	8	2015

There is never a fixed date for the next **Debian** release. The next version is released when it is ready.

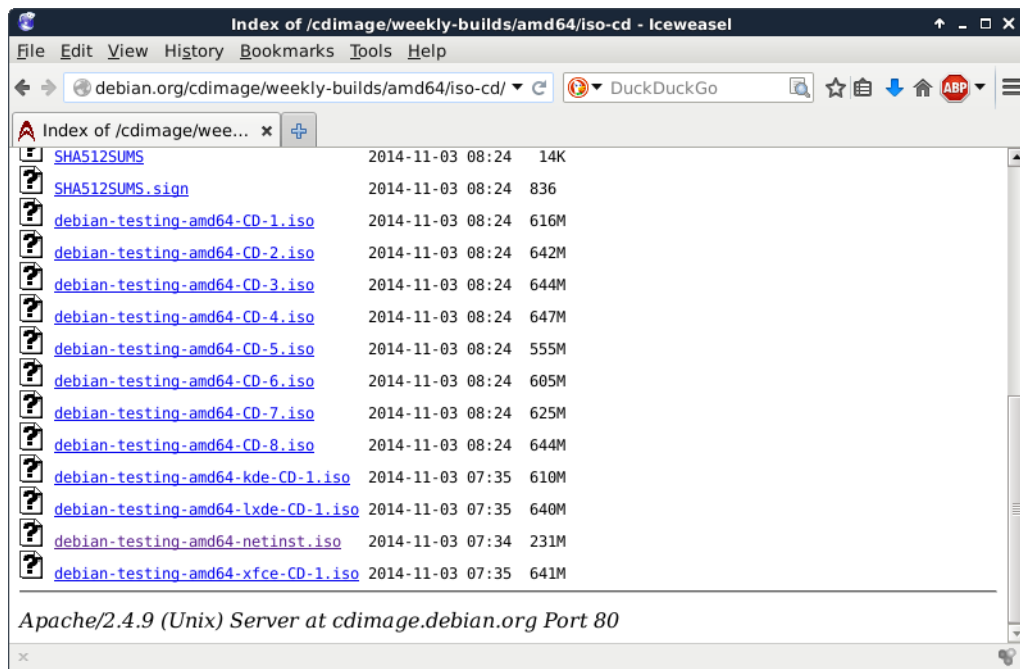
4.2. Downloading

All these screenshots were made in November 2014, which means **Debian 8** was still in 'testing' (but in 'freeze', so there will be no major changes when it is released).

Download Debian here:

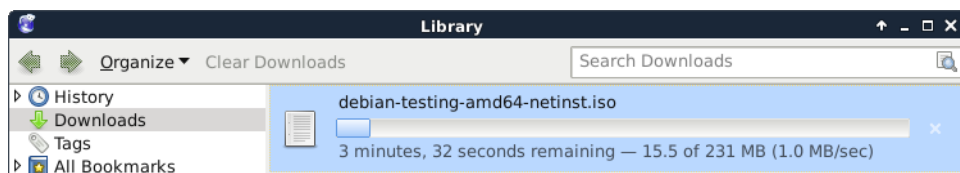


After a couple of clicks on that website, I ended up downloading **Debian 8** (testing) here. It should be only one click once **Debian 8** is released (somewhere in 2015).



You have many other options to download and install **Debian**. We will discuss them much later.

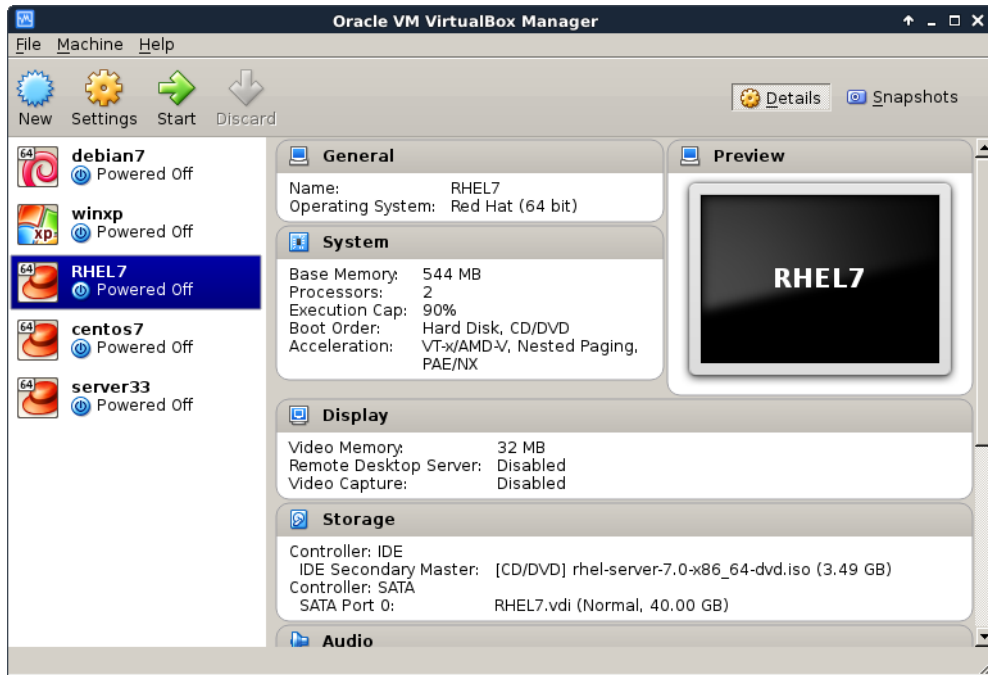
This small screenshot shows the downloading of a **netinst** .iso file. Most of the software will be downloaded during the installation. This also means that you will have the most recent version of all packages when the install is finished.



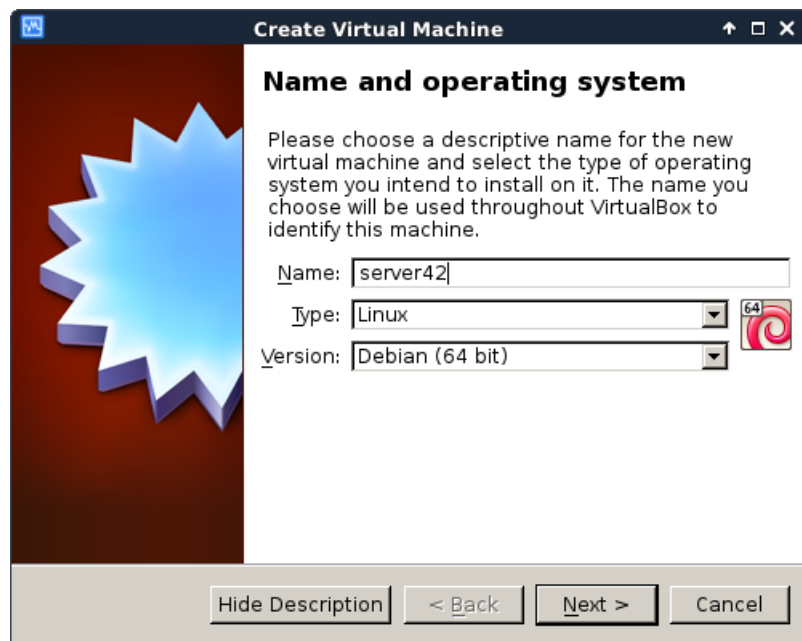
I already have Debian 8 installed on my laptop (hence the **paul@debian8** prompt). Anyway, this is the downloaded file just before starting the installation.

```
paul@debian8:~$ ls -hl debian-testing-amd64-netinst.iso
-rw-r--r-- 1 paul paul 231M Nov 10 17:59 debian-testing-amd64-netinst.iso
```


Create a new virtualbox machine (I already have five, you might have zero for now). Click the **New** button to start a wizard that will help you create a virtual machine.

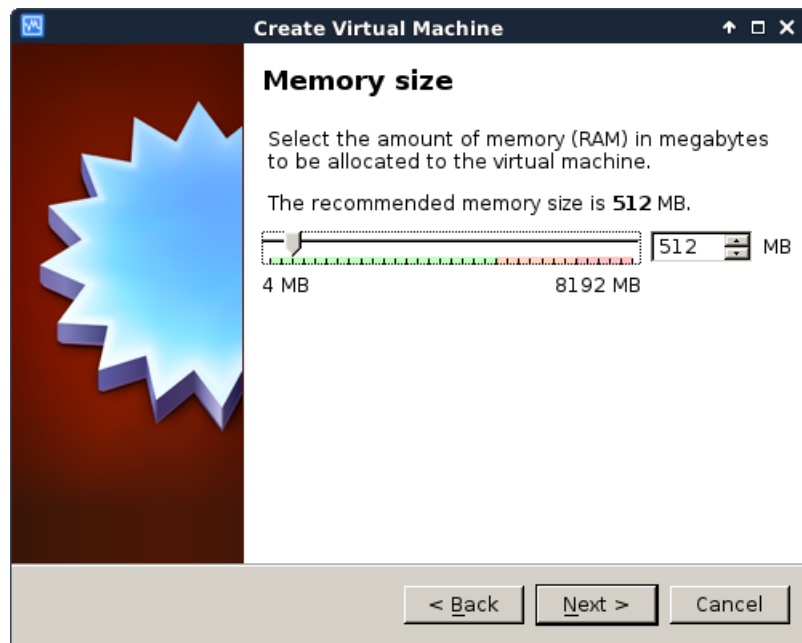


The machine needs a name, this screenshot shows that I named it **server42**.

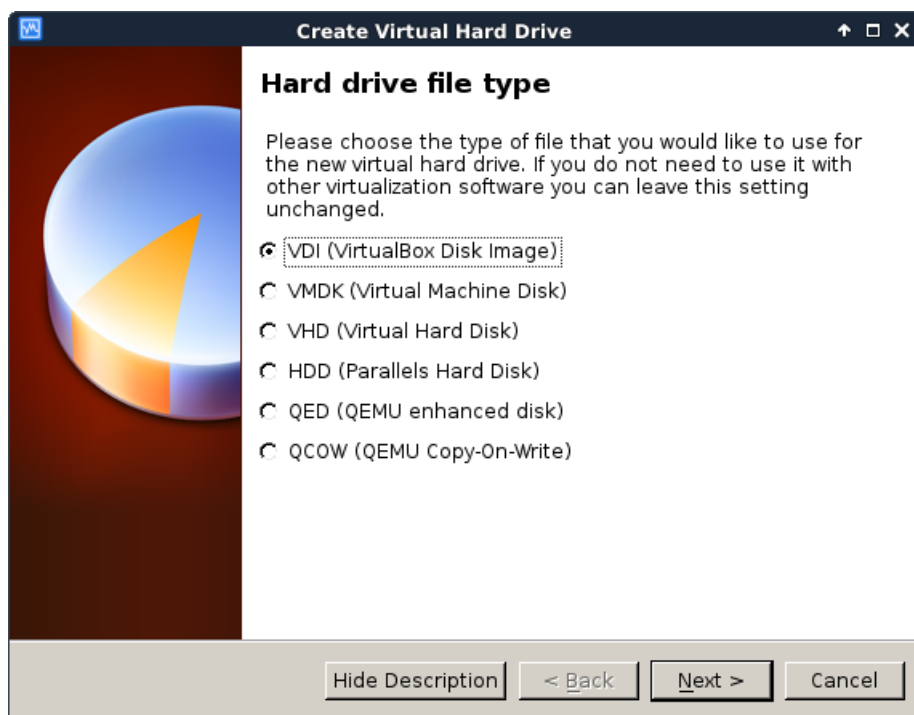


Most of the defaults in Virtualbox are ok.

512MB of RAM is enough to practice all the topics in this book.



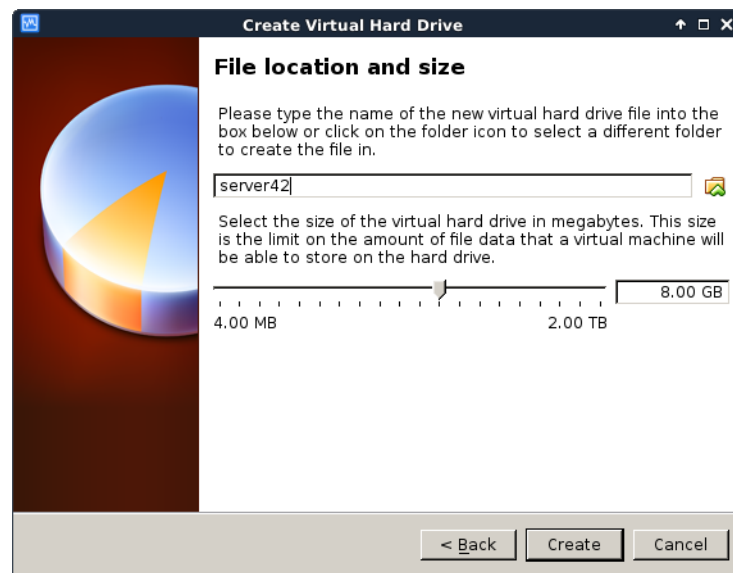
We do not care about the virtual disk format.



Choosing **dynamically allocated** will save you some disk space (for a small performance hit).

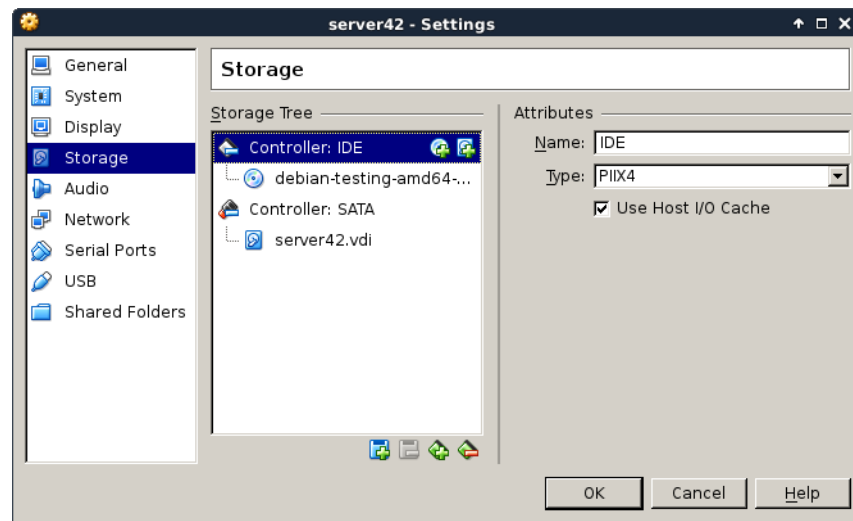


8GB should be plenty for learning about Linux servers.



This finishes the wizard. Your virtual machine is almost ready to begin the installation.

First, make sure that you attach the downloaded .iso image to the virtual CD drive. (by opening **Settings**, **Storage** followed by a mouse click on the round CD icon)

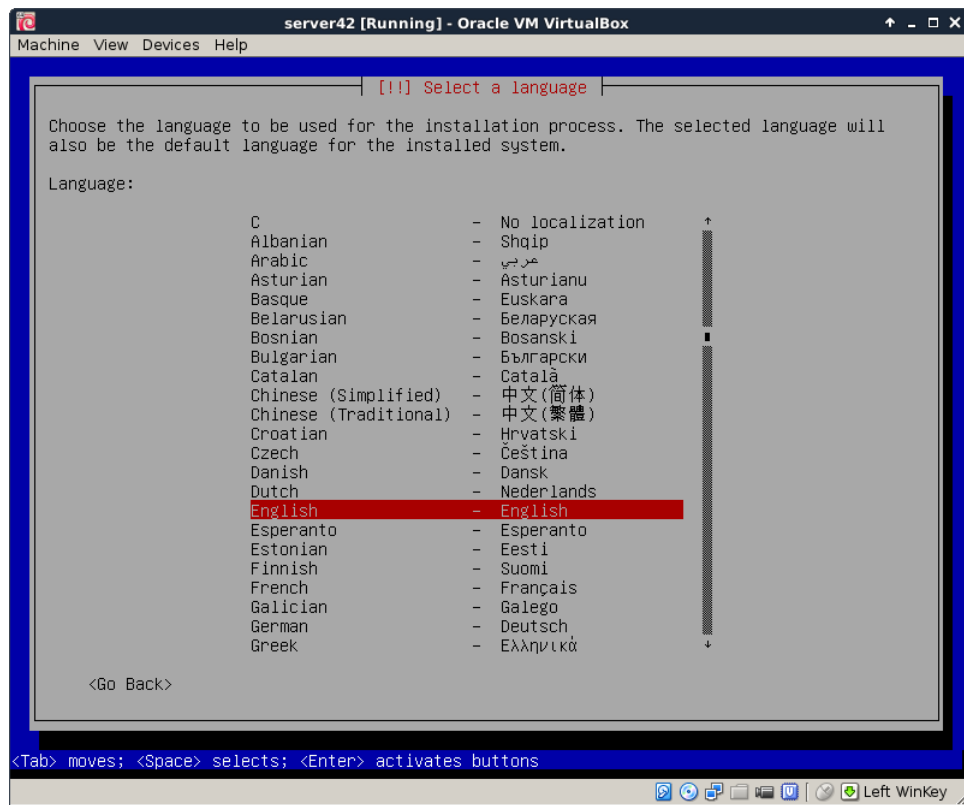


Personally I also disable sound and usb, because I never use these features. I also remove the floppy disk and use a PS/2 mouse pointer. This is probably not very important, but I like the idea that it saves some resources.

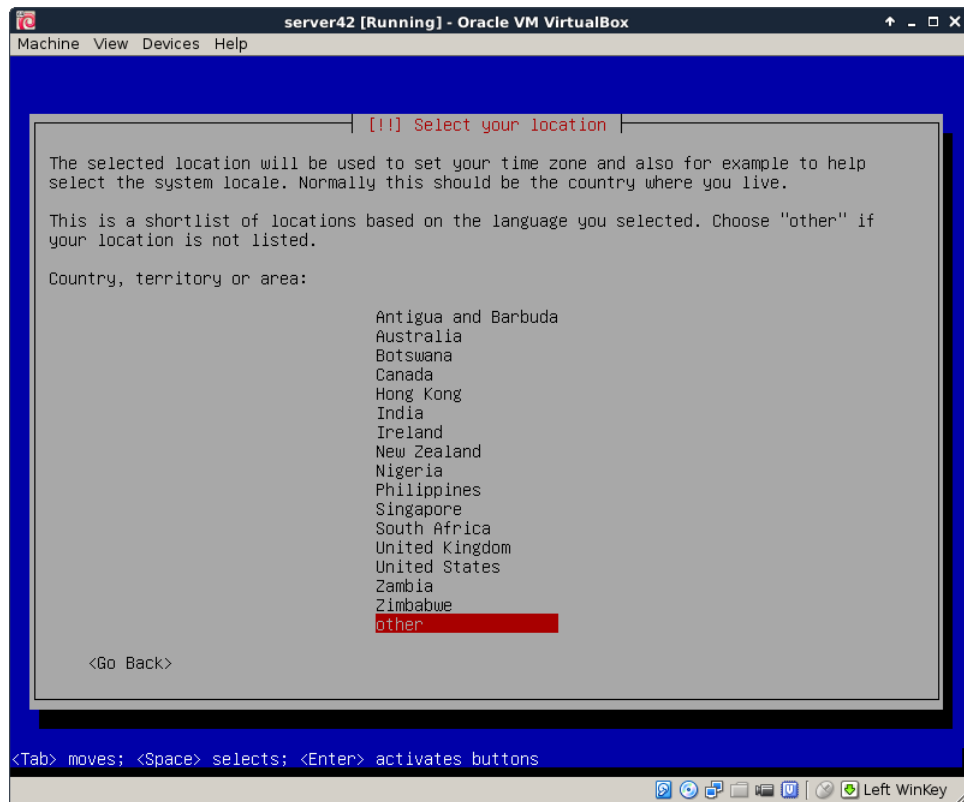
Now boot the virtual machine and begin the actual installation. After a couple of seconds you should see a screen similar to this. Choose **Install** to begin the installation of Debian.



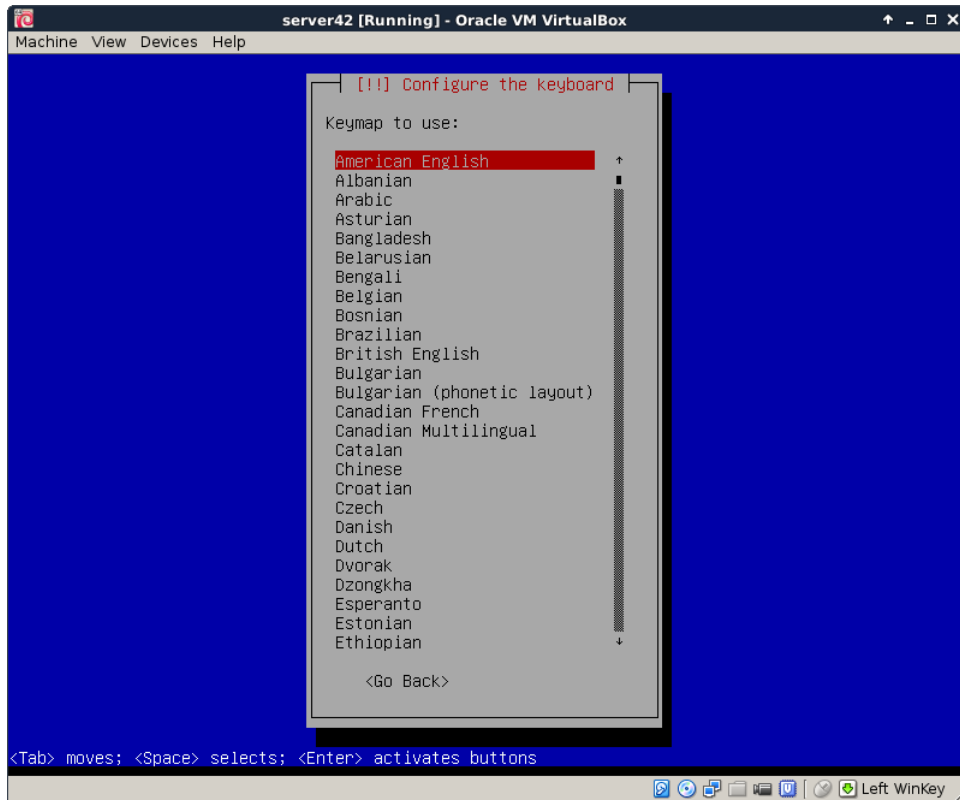
First select the language you want to use.



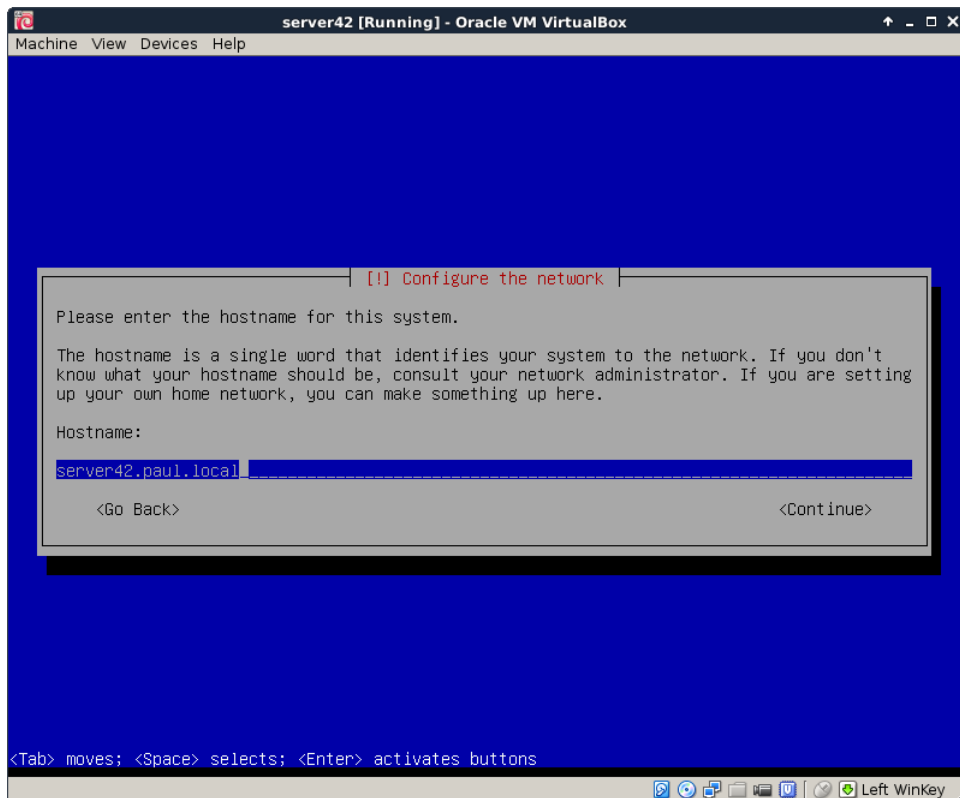
Choose your country. This information will be used to suggest a download mirror.



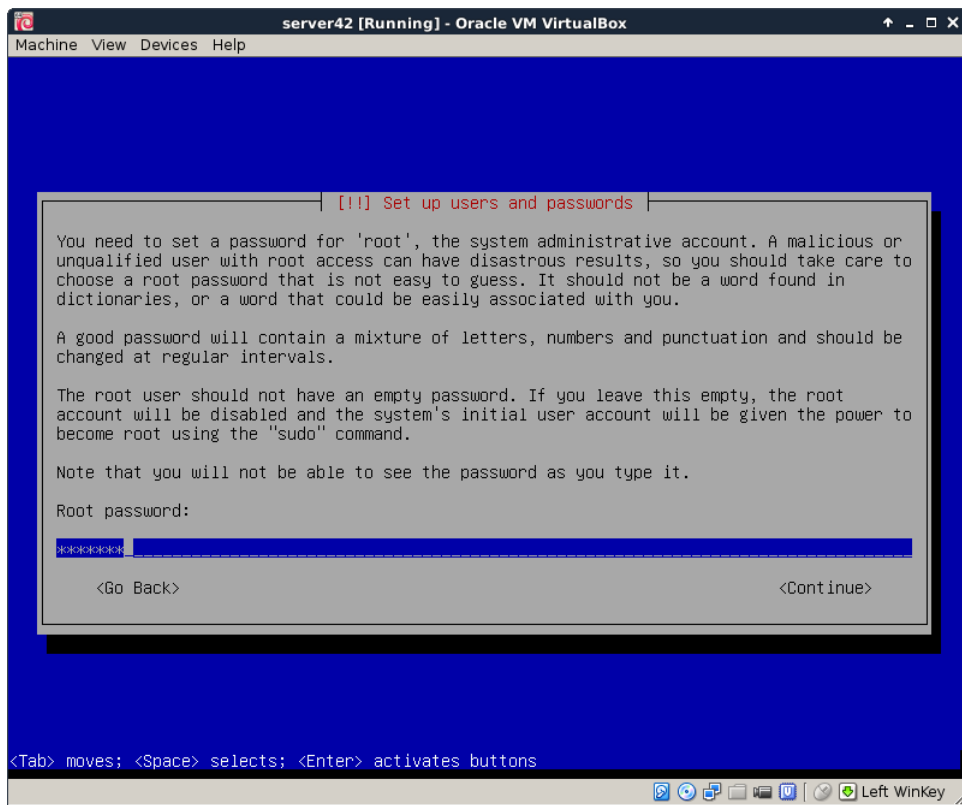
Choose the correct keyboard. On servers this is of no importance since most servers are remotely managed via **ssh**.



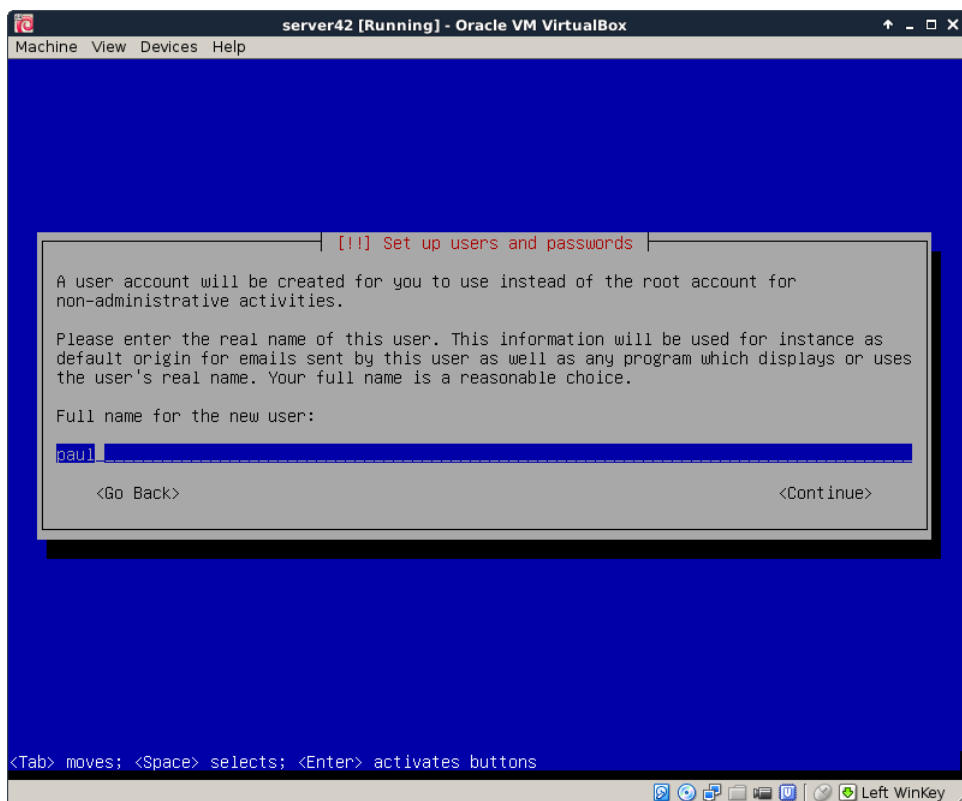
Enter a **hostname** (with **fqdn** to set a **dnsdomainname**).



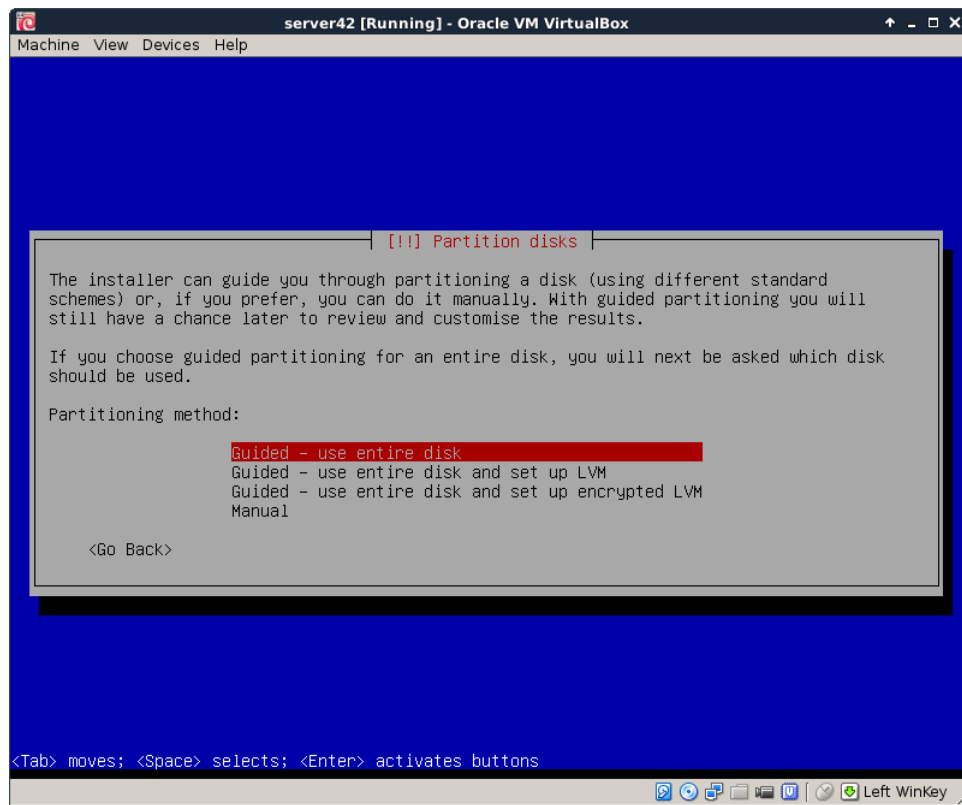
Give the **root** user a password. Remember this password (or use **hunter2**).



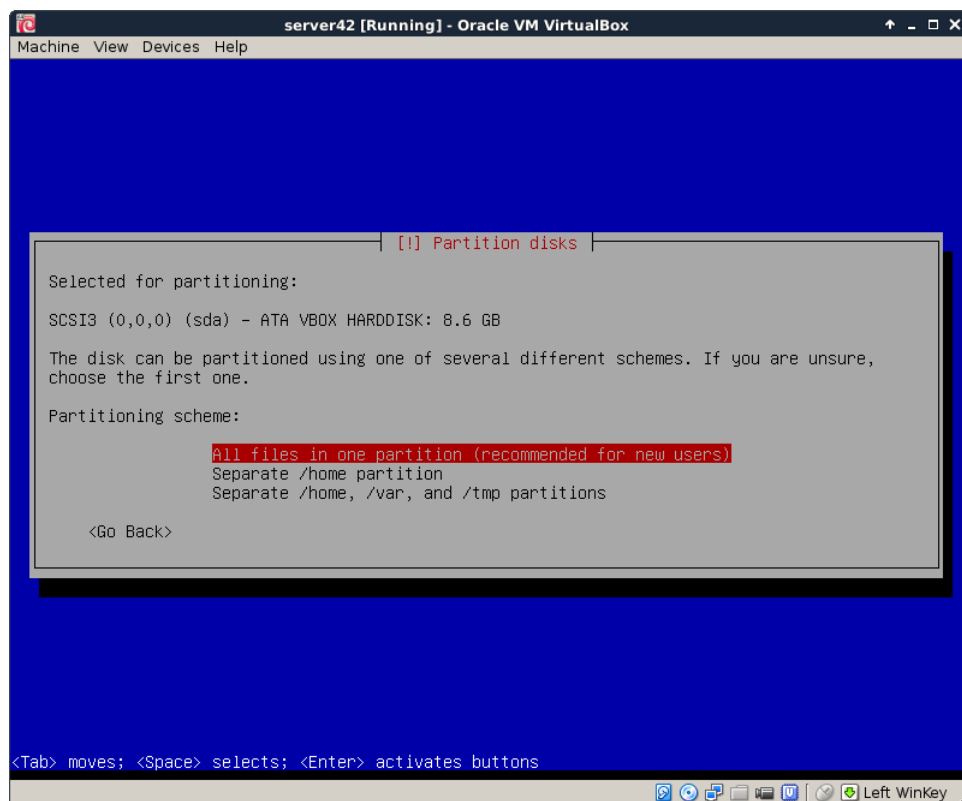
It is advised to also create a normal user account. I don't give my full name, Debian 8 accepts an identical username and full name **paul**.



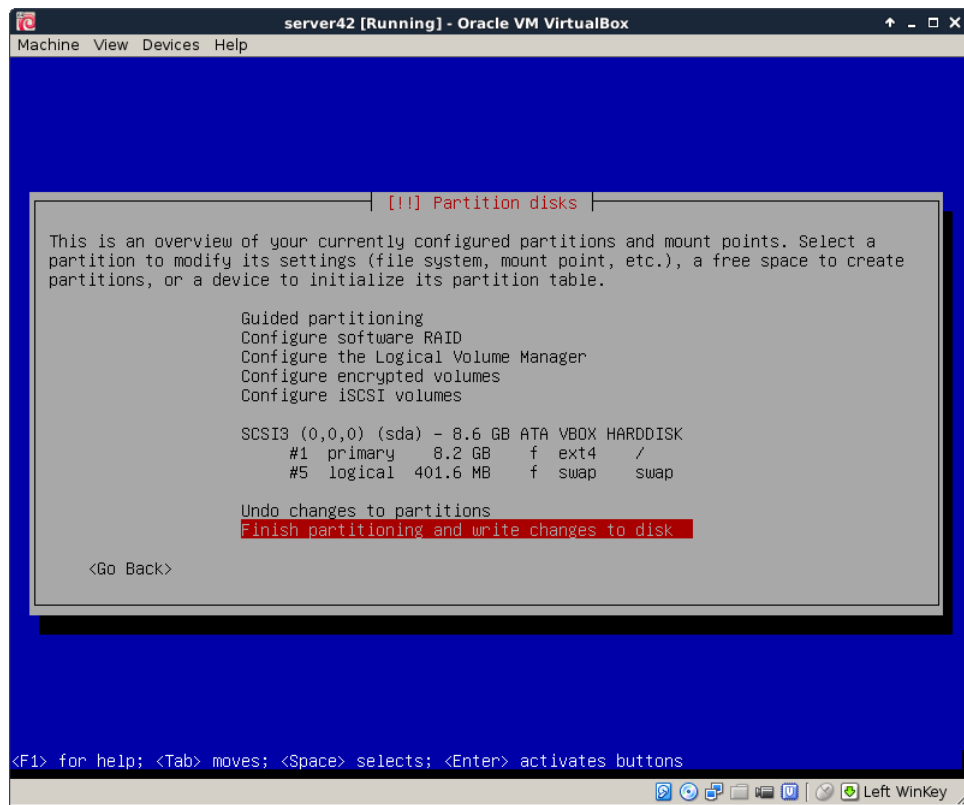
The **use entire disk** refers to the **virtual disk** that you created before in **Virtualbox**..



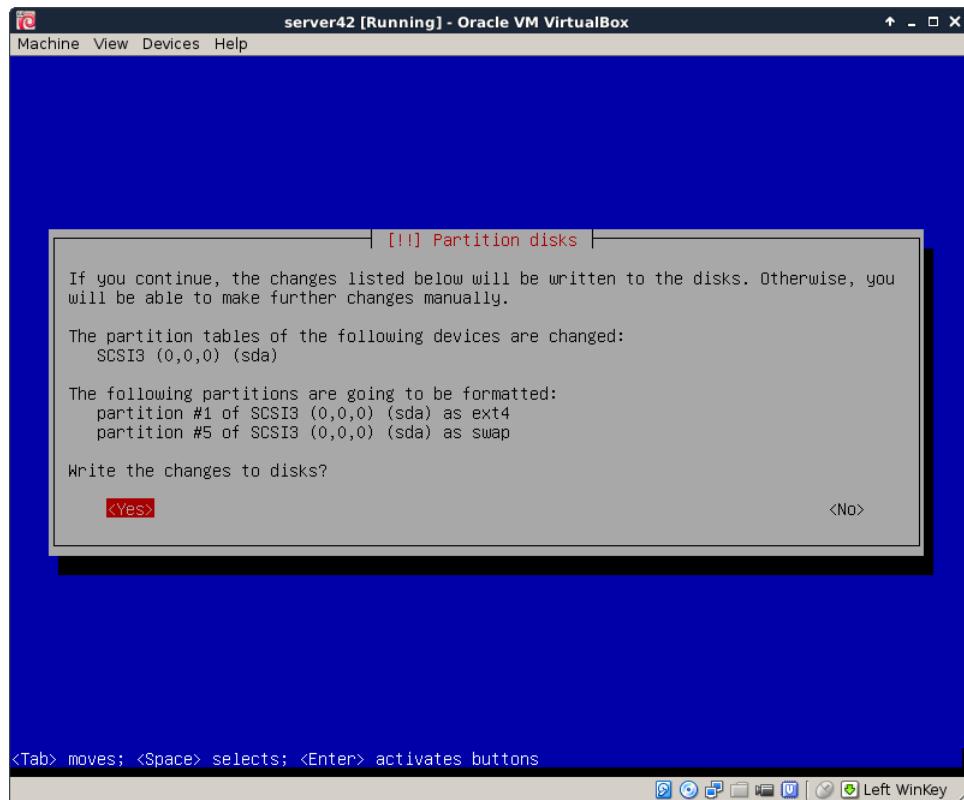
Again the default is probably what you want. Only change partitioning if you really know what you are doing.



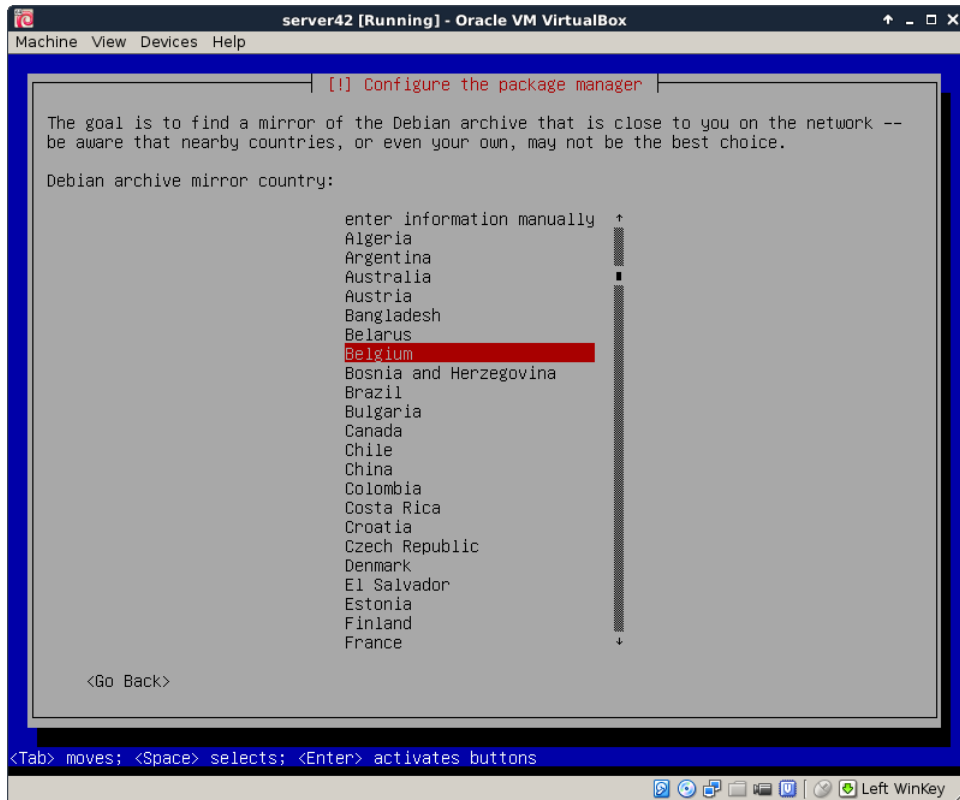
Accept the partition layout (again only change if you really know what you are doing).



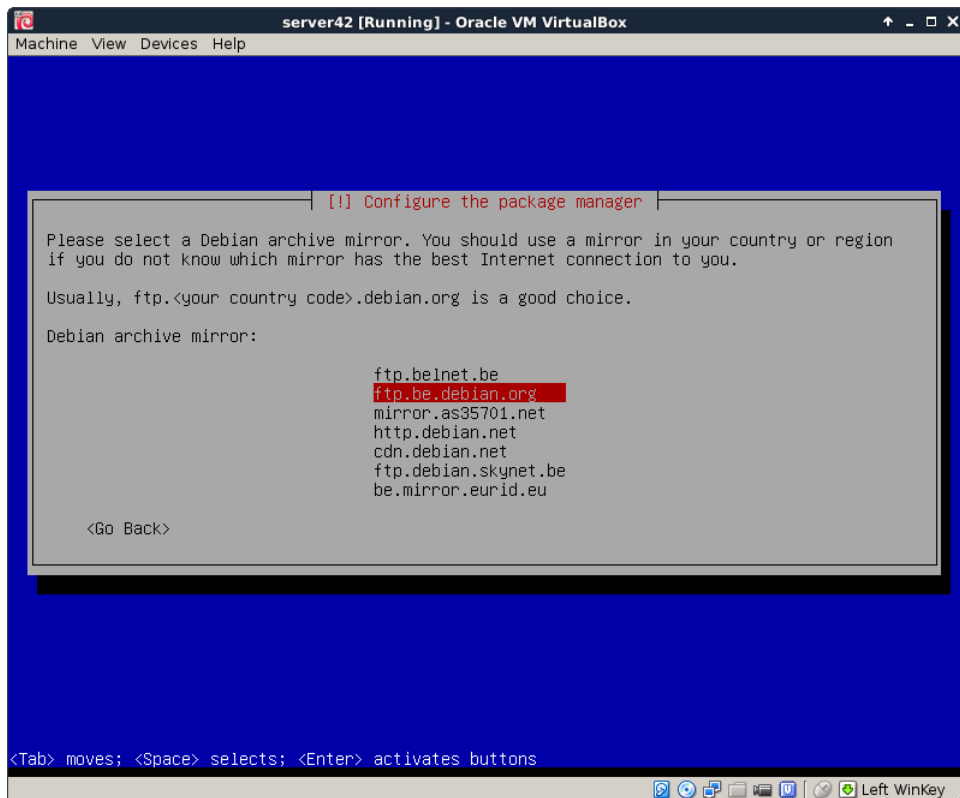
This is the point of no return, the magical moment where pressing **yes** will forever erase data on the (virtual) computer.



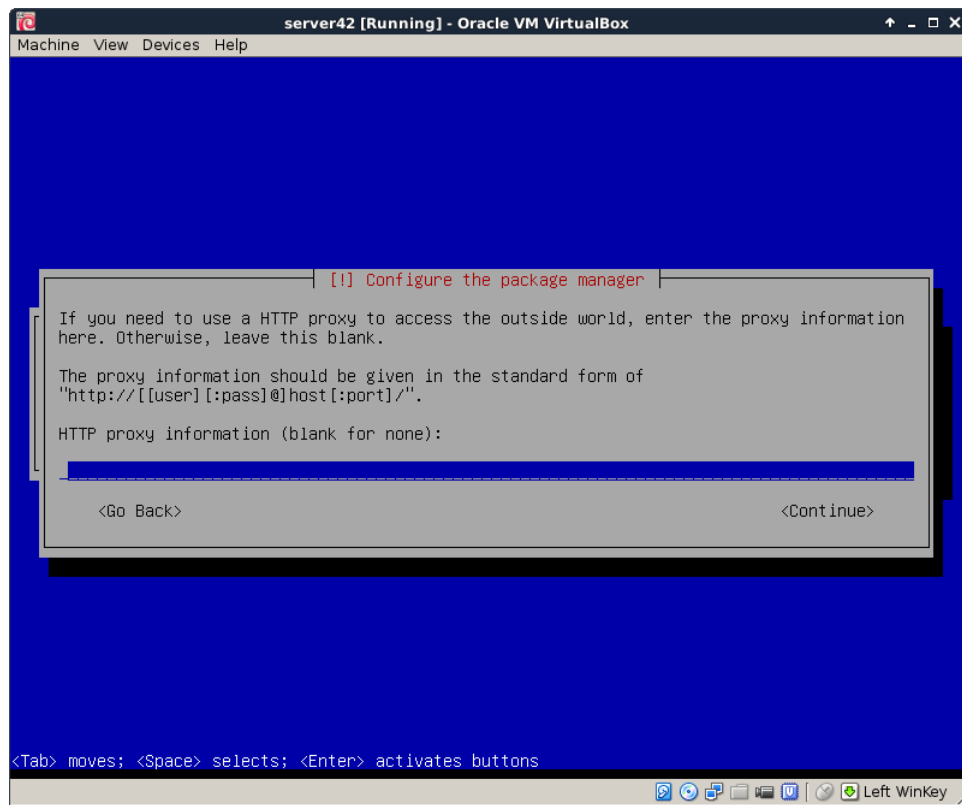
Software is downloaded from a mirror repository, preferably choose one that is close by (as in the same country).



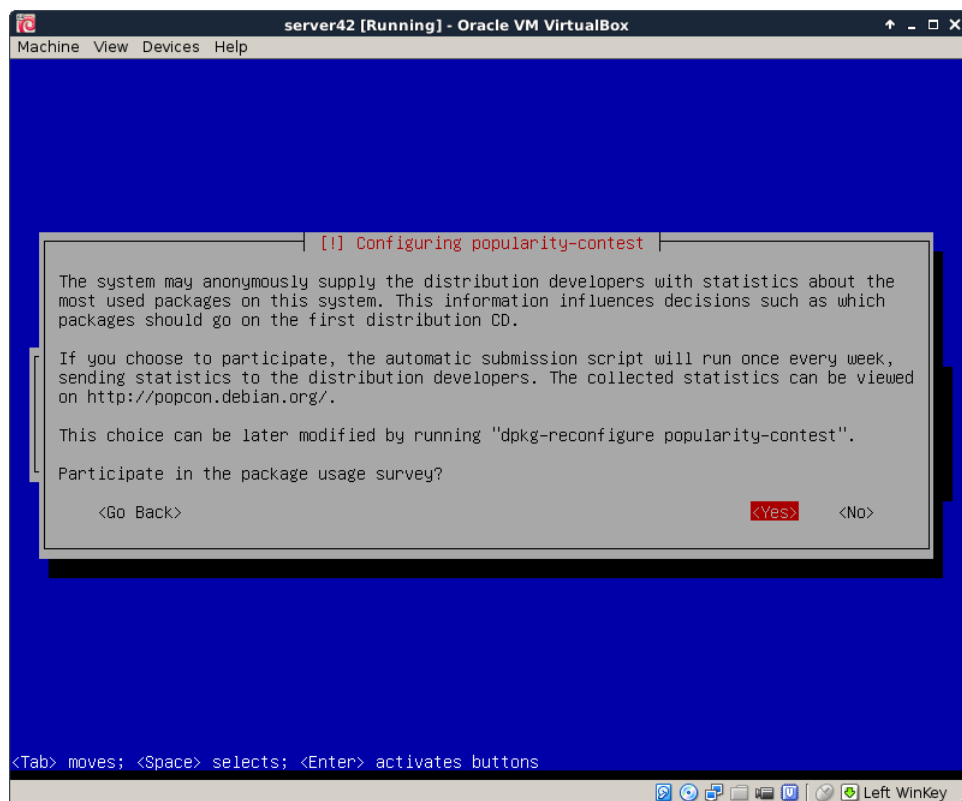
This setup was done in Belgium.



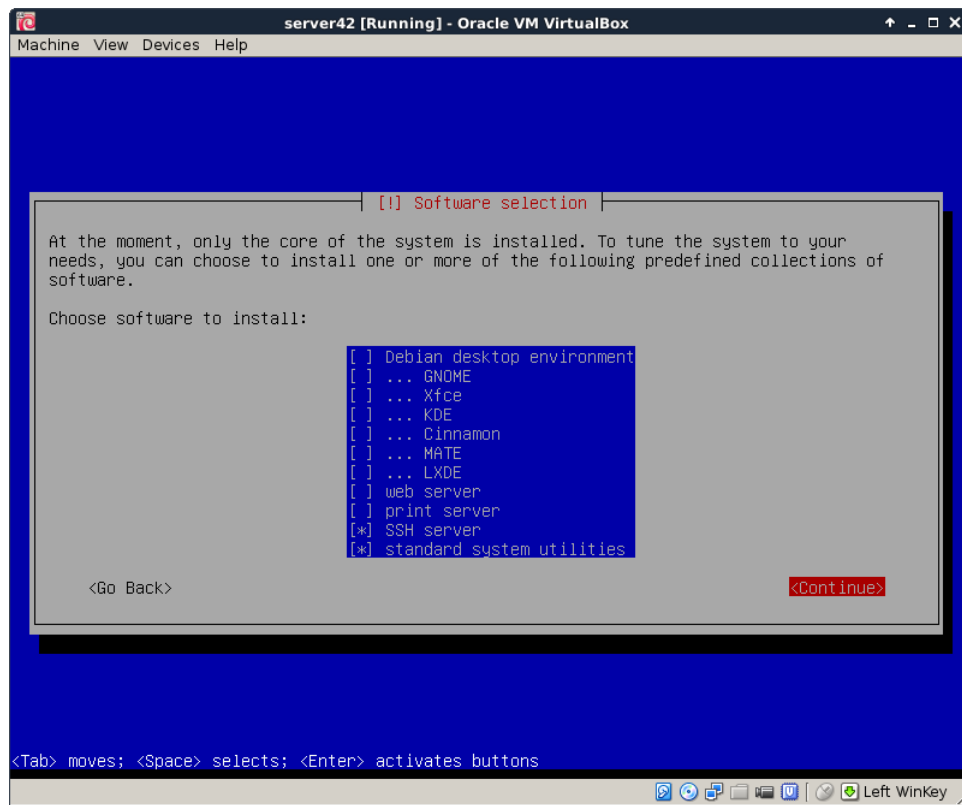
Leave the proxy field empty (unless you are sure that you are behind a proxy server).



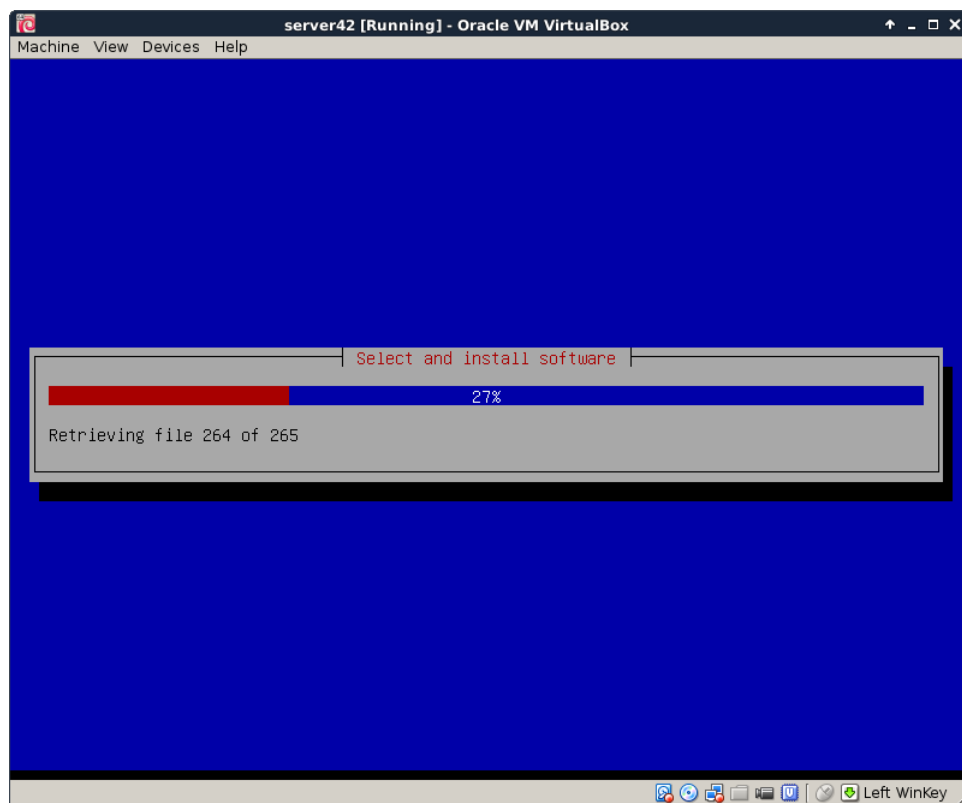
Choose whether you want to send anonymous statistics to the Debian project (it gathers data about installed packages). You can view the statistics here <http://popcon.debian.org/>.



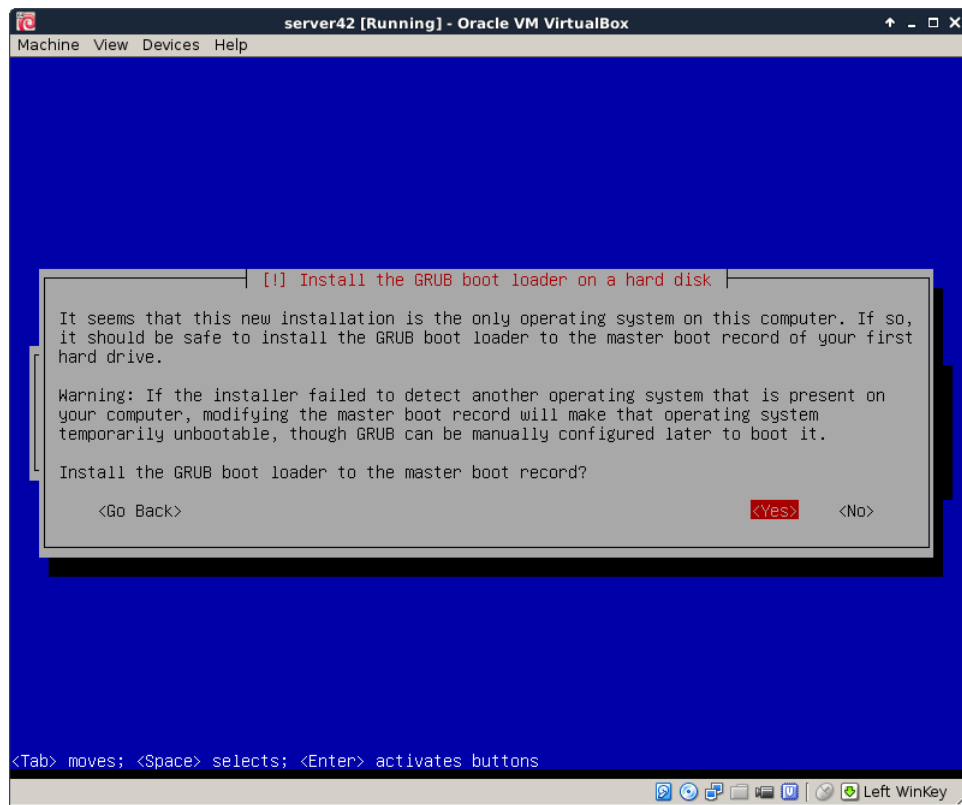
Choose what software to install, we do not need any graphical stuff for this training.



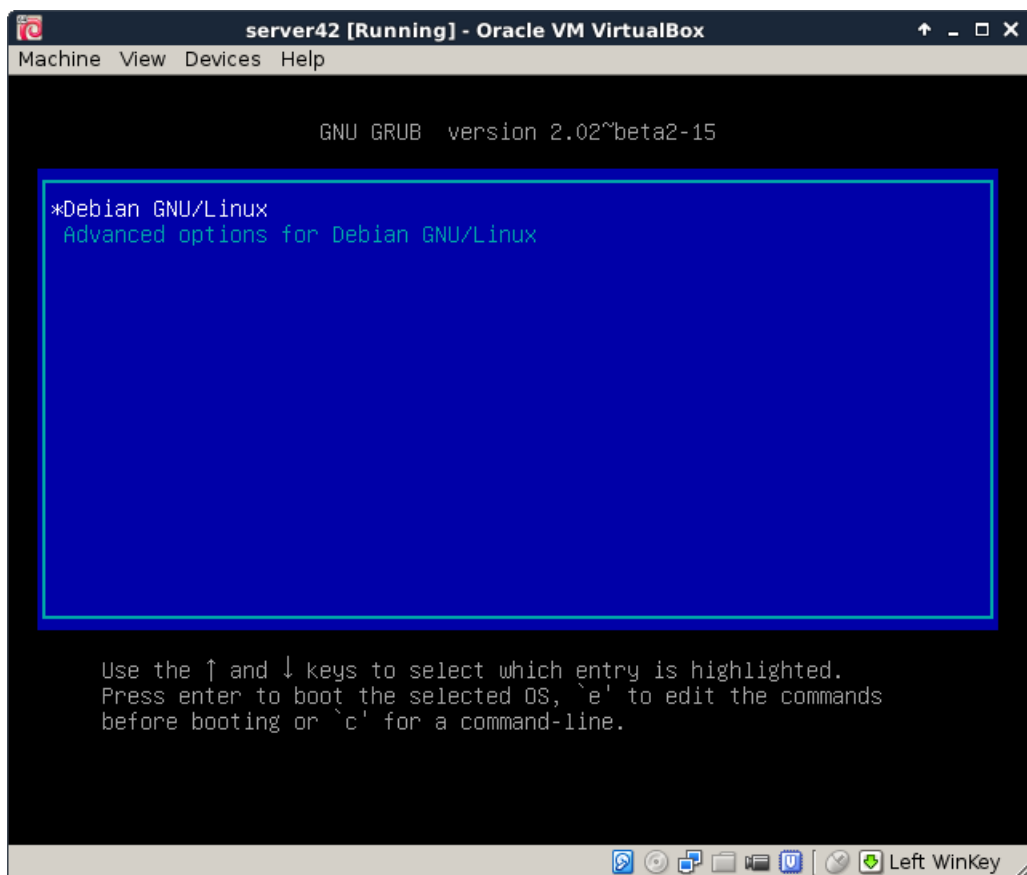
The latest versions are being downloaded.



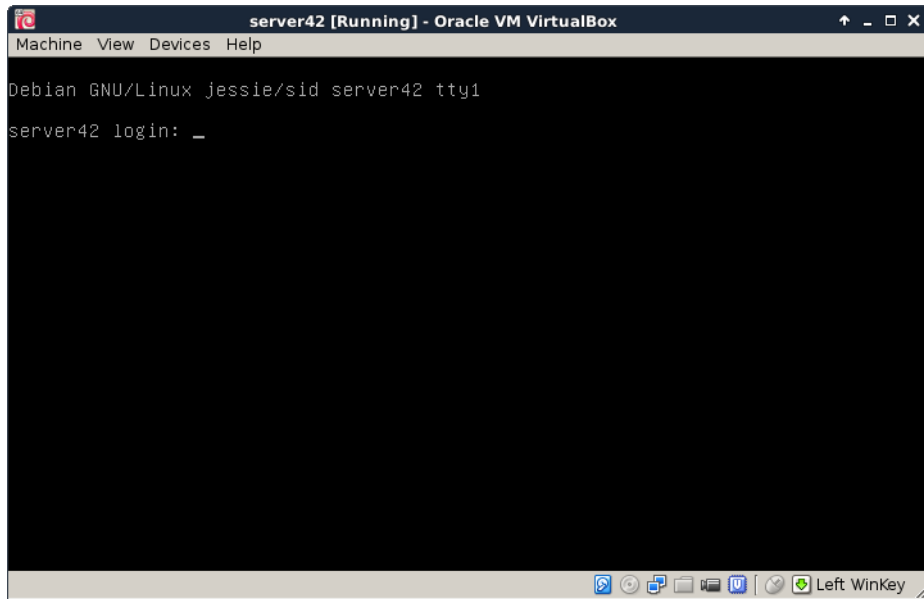
Say yes to install the bootloader on the virtual machine.



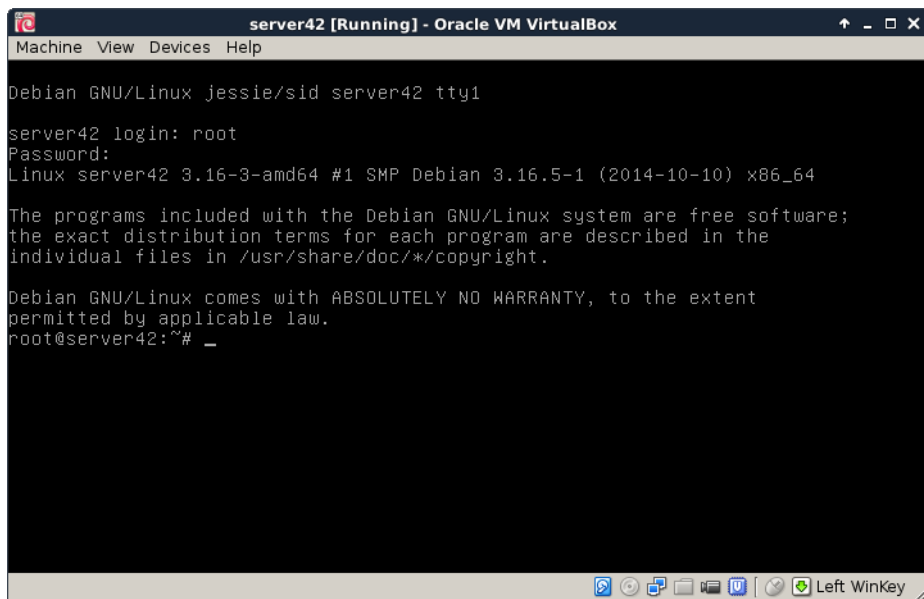
Booting for the first time shows the grub screen



A couple seconds later you should see a lot of text scrolling of the screen (**dmesg**). After which you are presented with this **getty** and are allowed your first logon.



You should now be able to log on to your virtual machine with the **root** account. Do you remember the password ? Was it **hunter2** ?



The screenshots in this book will look like this from now on. You can just type those commands in the terminal (after you logged on).

```
root@server42:~# who am i
root      tty1      2014-11-10 18:21
root@server42:~# hostname
server42
root@server42:~# date
Mon Nov 10 18:21:56 CET 2014
```

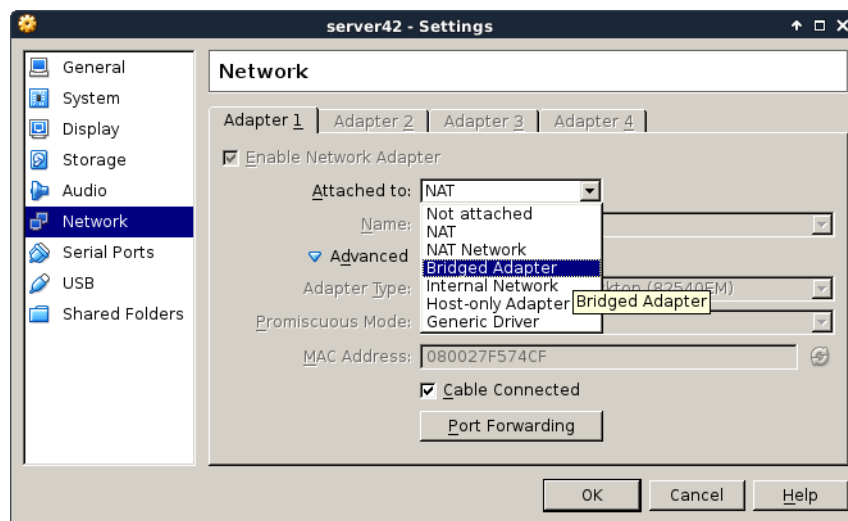
4.3. virtualbox networking

You can also log on from remote (or from your Windows/Mac/Linux host computer) using **ssh** or **putty**. Change the **network** settings in the virtual machine to **bridge**. This will enable your virtual machine to receive an ip address from your local dhcp server.

The default virtualbox networking is to attach virtual network cards to **nat**. This screenshot shows the ip address **10.0.2.15** when on **nat**:

```
root@server42:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:f5:74:cf
          inet addr:10.0.2.15  Bcast:10.0.2.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe5:74cf/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:11 errors:0 dropped:0 overruns:0 frame:0
          TX packets:19 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2352 (2.2 KiB)  TX bytes:1988 (1.9 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```



By shutting down the network interface and enabling it again, we force Debian to renew an ip address from the bridged network.

```
root@server42:~# # do not run ifdown while connected over ssh!
root@server42:~# ifdown eth0
Killed old client process
Internet Systems Consortium DHCP Client 4.3.1
Copyright 2004-2014 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/eth0/08:00:27:f5:74:cf
Sending on   LPF/eth0/08:00:27:f5:74:cf
```

```
Sending on   Socket/fallback
DHCPRELEASE on eth0 to 10.0.2.2 port 67
root@server42:~# # now enable bridge in virtualbox settings
root@server42:~# ifup eth0
Internet Systems Consortium DHCP Client 4.3.1
Copyright 2004-2014 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/eth0/08:00:27:f5:74:cf
Sending on   LPF/eth0/08:00:27:f5:74:cf
Sending on   Socket/fallback
DHCPDISCOVER on eth0 to 255.255.255.255 port 67 interval 8
DHCPDISCOVER on eth0 to 255.255.255.255 port 67 interval 8
DHCPREQUEST on eth0 to 255.255.255.255 port 67
DHCPOFFER from 192.168.1.42
DHCPACK from 192.168.1.42
bound to 192.168.1.111 -- renewal in 2938 seconds.
root@server42:~# ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 08:00:27:f5:74:cf
          inet addr:192.168.1.111  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fef5:74cf/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:15 errors:0 dropped:0 overruns:0 frame:0
          TX packets:31 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:3156 (3.0 KiB)  TX bytes:3722 (3.6 KiB)
root@server42:~#
```

Here is an example of **ssh** to this freshly installed computer. Note that **Debian 8** has disabled remote root access, so i need to use the normal user account.

```
paul@debian8:~$ ssh paul@192.168.1.111
paul@192.168.1.111's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
paul@server42:~$
paul@server42:~$ su -
Password:
root@server42:~#
```

TODO: putty screenshot here...

4.4. setting the hostname

The hostname of the server is asked during installation, so there is no need to configure this manually.

```
root@server42:~# hostname
server42
root@server42:~# cat /etc/hostname
server42
root@server42:~# dnsdomainname
paul.local
root@server42:~# grep server42 /etc/hosts
127.0.1.1      server42.paul.local      server42
root@server42:~#
```

4.5. adding a static ip address

This example shows how to add a static ip address to your server.

You can use **ifconfig** to set a static address that is active until the next **reboot** (or until the next **ifdown**).

a

```
root@server42:~# ifconfig eth0:0 10.104.33.39
```

Adding a couple of lines to the **/etc/network/interfaces** file to enable an extra ip address forever.

```
root@server42:~# vi /etc/network/interfaces
root@server42:~# tail -4 /etc/network/interfaces
auto eth0:0
iface eth0:0 inet static
address 10.104.33.39
netmask 255.255.0.0
root@server42:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:f5:74:cf
          inet addr:192.168.1.111  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fef5:74cf/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:528 errors:0 dropped:0 overruns:0 frame:0
          TX packets:333 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:45429 (44.3 KiB)  TX bytes:48763 (47.6 KiB)

eth0:0    Link encap:Ethernet  HWaddr 08:00:27:f5:74:cf
          inet addr:10.104.33.39  Bcast:10.255.255.255  Mask:255.0.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

root@server42:~#
```

4.6. Debian package management

To get all information about the newest packages form the online repository:

```
root@server42:~# aptitude update
Get: 1 http://ftp.be.debian.org jessie InRelease [191 kB]
Get: 2 http://security.debian.org jessie/updates InRelease [84.1 kB]
Get: 3 http://ftp.be.debian.org jessie-updates InRelease [117 kB]
Get: 4 http://ftp.be.debian.org jessie-backports InRelease [118 kB]
Get: 5 http://security.debian.org jessie/updates/main Sources [14 B]
Get: 6 http://ftp.be.debian.org jessie/main Sources/DiffIndex [7,876 B]
... (output truncated)
```

To download and apply all updates for all installed packages:

```
root@server42:~# aptitude upgrade
Resolving dependencies...
The following NEW packages will be installed:
  firmware-linux-free{a} irqbalance{a} libnuma1{a} linux-image-3.16.0-4-amd64{a}
The following packages will be upgraded:
  busybox file libc-bin libc6 libexpat1 libmagic1 libpaper-utils libpaper1 libsqlite3-0
  linux-image-amd64 locales multiarch-support
12 packages upgraded, 4 newly installed, 0 to remove and 0 not upgraded.
Need to get 44.9 MB of archives. After unpacking 161 MB will be used.
Do you want to continue? [Y/n/?]
... (output truncated)
```

To install new software (**vim** and **tmux** in this example):

```
root@server42:~# aptitude install vim tmux
The following NEW packages will be installed:
  tmux vim vim-runtime{a}
0 packages upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 6,243 kB of archives. After unpacking 29.0 MB will be used.
Do you want to continue? [Y/n/?]
Get: 1 http://ftp.be.debian.org/debian/ jessie/main tmux amd64 1.9-6 [245 kB]
Get: 2 http://ftp.be.debian.org/debian/ jessie/main vim-runtime all 2:7.4.488-1 [5,046 kB]
Get: 3 http://ftp.be.debian.org/debian/ jessie/main vim amd64 2:7.4.488-1 [952 kB]
```

Refer to the **package management** chapter in LinuxAdm.pdf for more information.

Chapter 5. installing CentOS 7

This module is a step by step demonstration of an actual installation of **CentOS 7**.

We start by downloading an image from the internet and install **CentOS 7** as a virtual machine in **Virtualbox**. We will also do some basic configuration of this new machine like setting an **ip address** and fixing a **hostname**.

This procedure should be very similar for other versions of **CentOS**, and also for distributions like **RHEL** (Red Hat Enterprise Linux) or **Fedora**. This procedure can also be helpful if you are using another virtualization solution.

5.1. download a CentOS 7 image

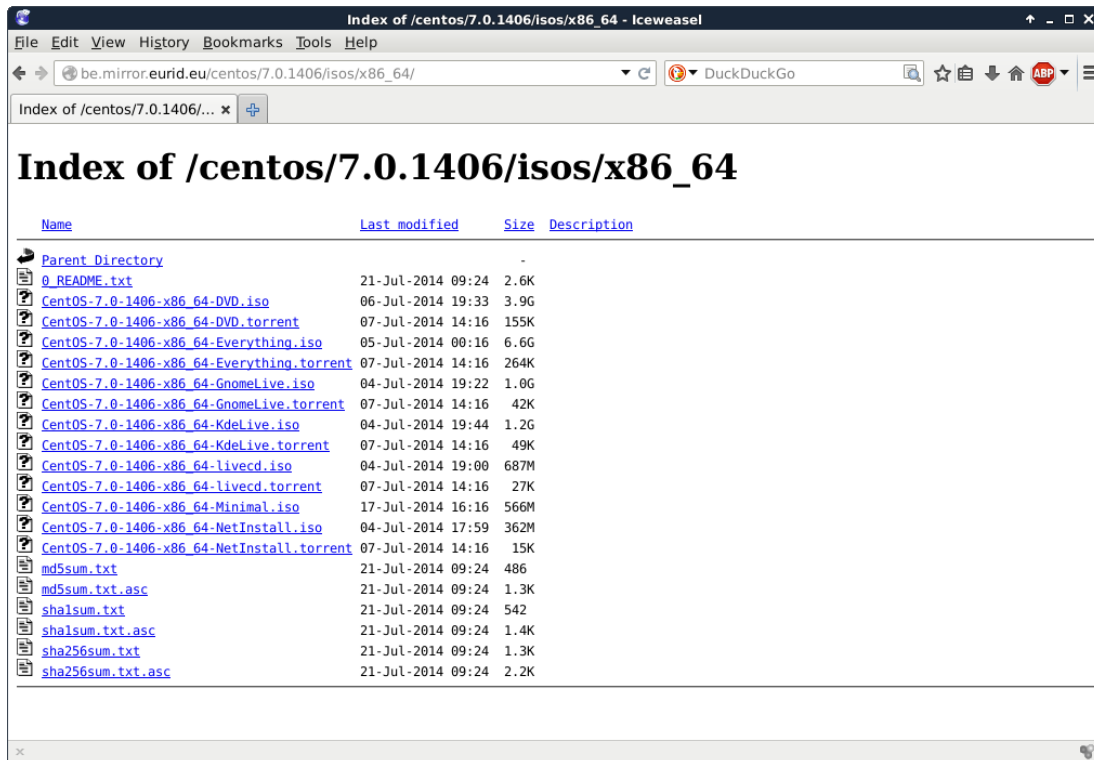
This demonstration uses a laptop computer with **Virtualbox** to install **CentOS 7** as a virtual machine. The first task is to download an **.iso** image of **CentOS 7**.

The **CentOS 7** website looks like this today (November 2014). They change the look regularly, so it may look different when you visit it.

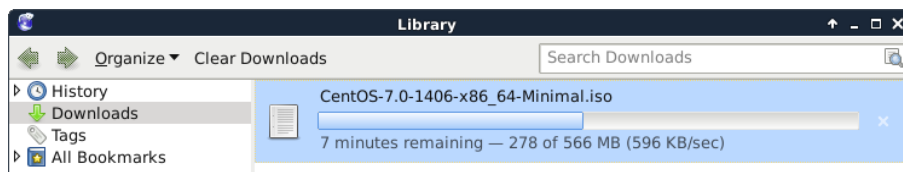


You can download a full DVD, which allows for an off line installation of a graphical **CentOS 7** desktop. You can select this because it should be easy and complete, and should get you started with a working **CentOS 7** virtual machine.

But I clicked instead on 'alternative downloads', selected **CentOS 7** and **x86_64** and ended up on a **mirror list**. Each mirror is a server that contains copies of **CentOS 7** media. I selected a Belgian mirror because I currently am in Belgium.



There is again the option for full DVD's and more. This demonstration will use the **minimal** .iso file, because it is much smaller in size. The download takes a couple of minutes.



Verify the size of the file after download to make sure it is complete. Probably a right click on the file and selecting 'properties' (if you use Windows or Mac OSX).

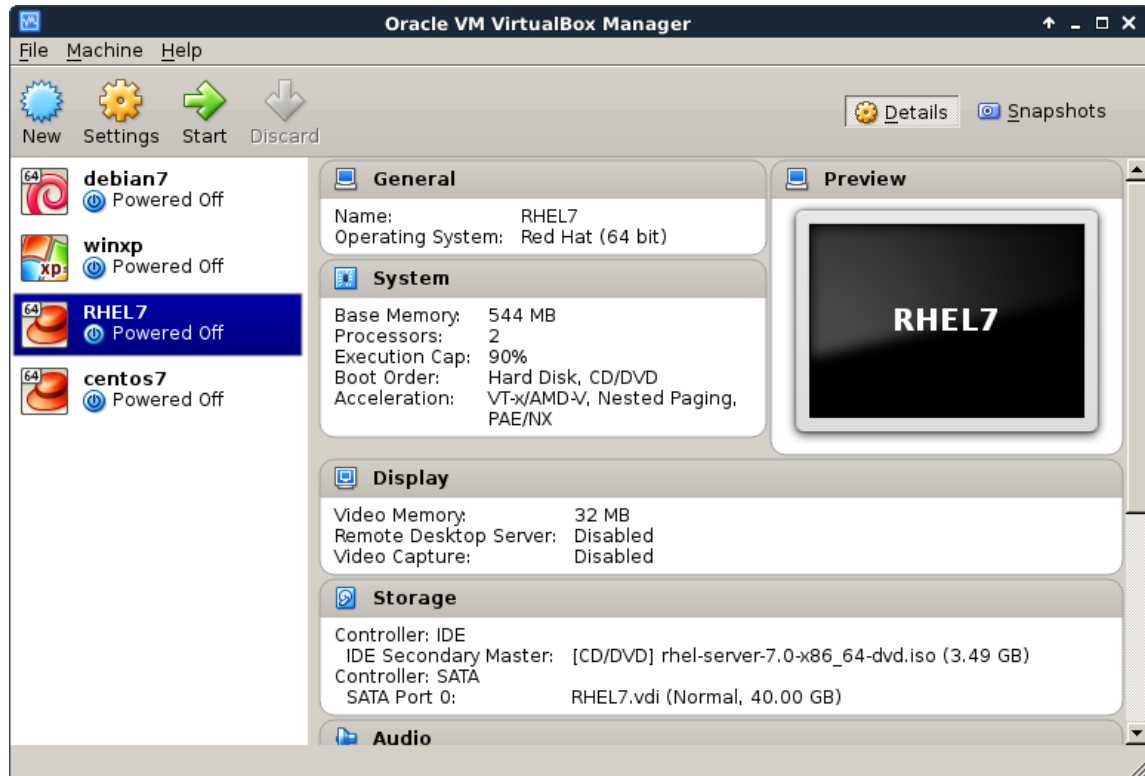
I use Linux on the laptop already:

```
paul@debian8:~$ ls -lh CentOS-7.0-1406-x86_64-Minimal.iso
-rw-r--r-- 1 paul paul 566M Nov  1 14:45 CentOS-7.0-1406-x86_64-Minimal.iso
```

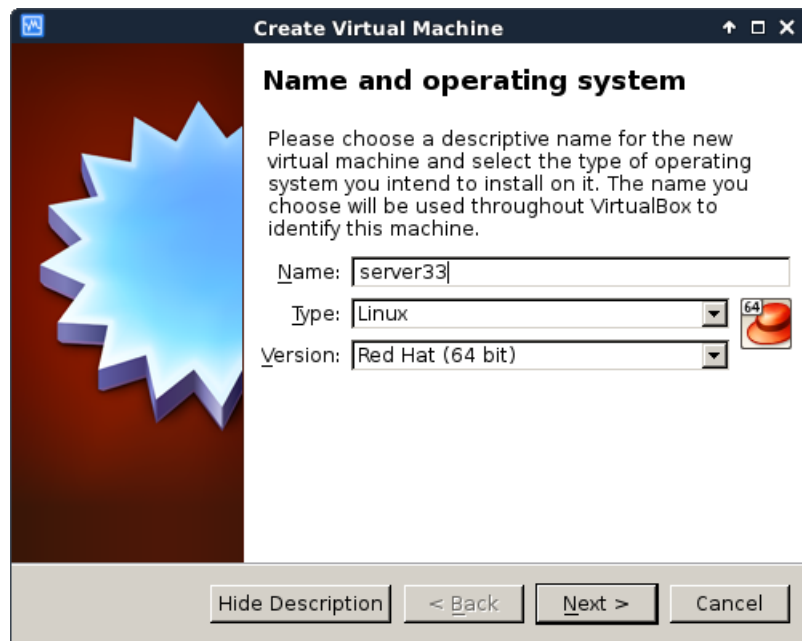
Do not worry if you do not understand the above command. Just try to make sure that the size of this file is the same as the size that is mentioned on the **CentOS 7** website.

5.2. Virtualbox

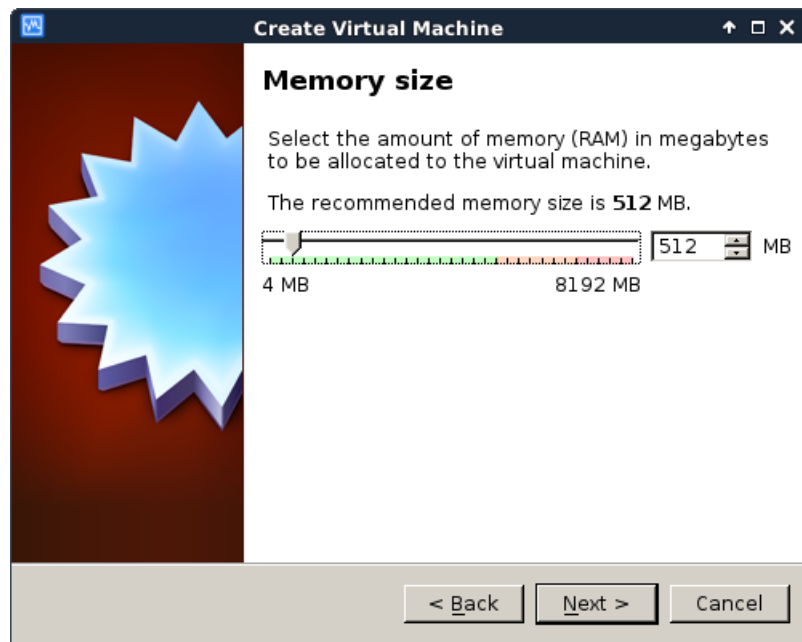
This screenshot shows up when I start Virtualbox. I already have four virtual machines, you might have none.



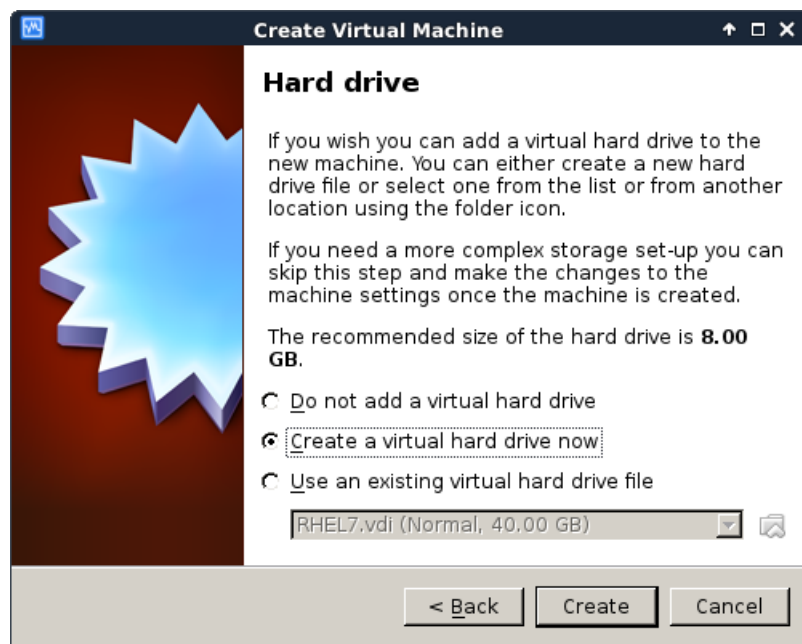
Below are the steps for creating a new virtual machine. Start by clicking **New** and give your machine a name (I chose **server33**). Click **Next**.



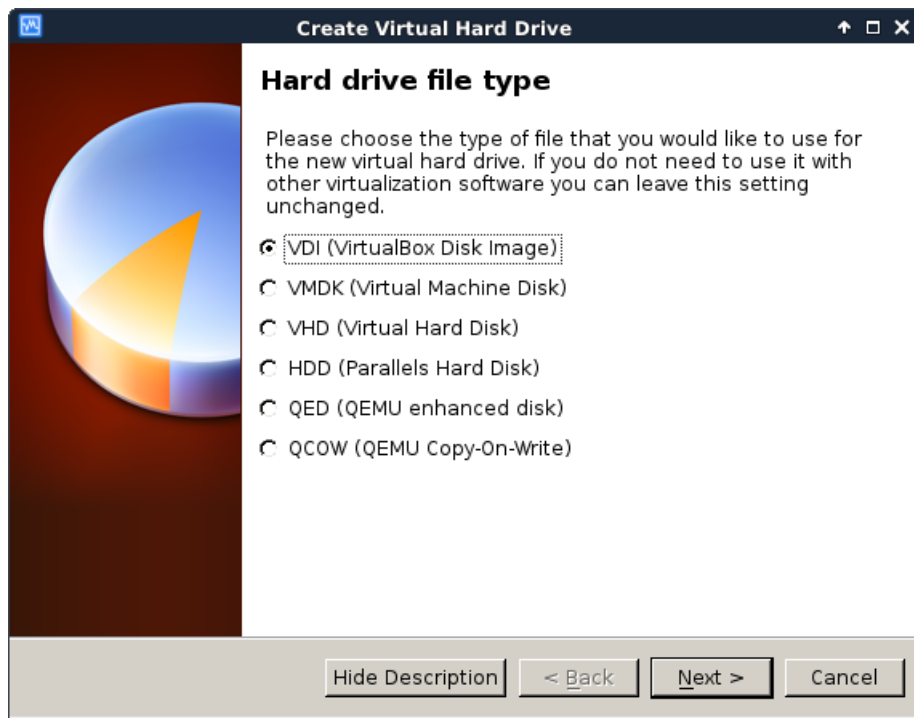
A Linux computer without graphical interface will run fine on **half a gigabyte** of RAM.



A Linux virtual machine will need a **virtual hard drive**.



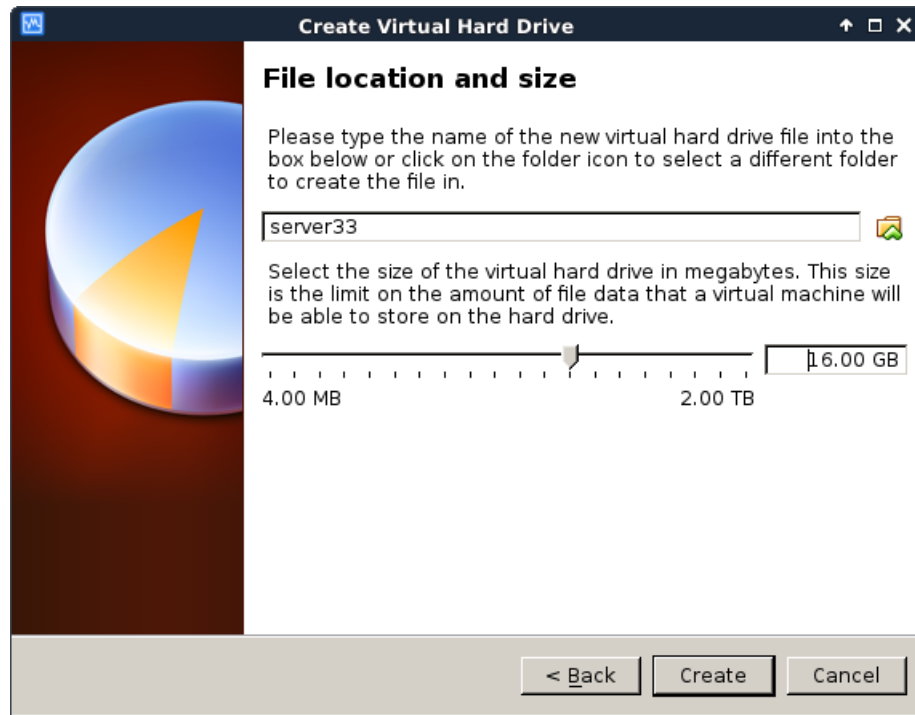
Any format will do for our purpose, so I left the default **vdi**.



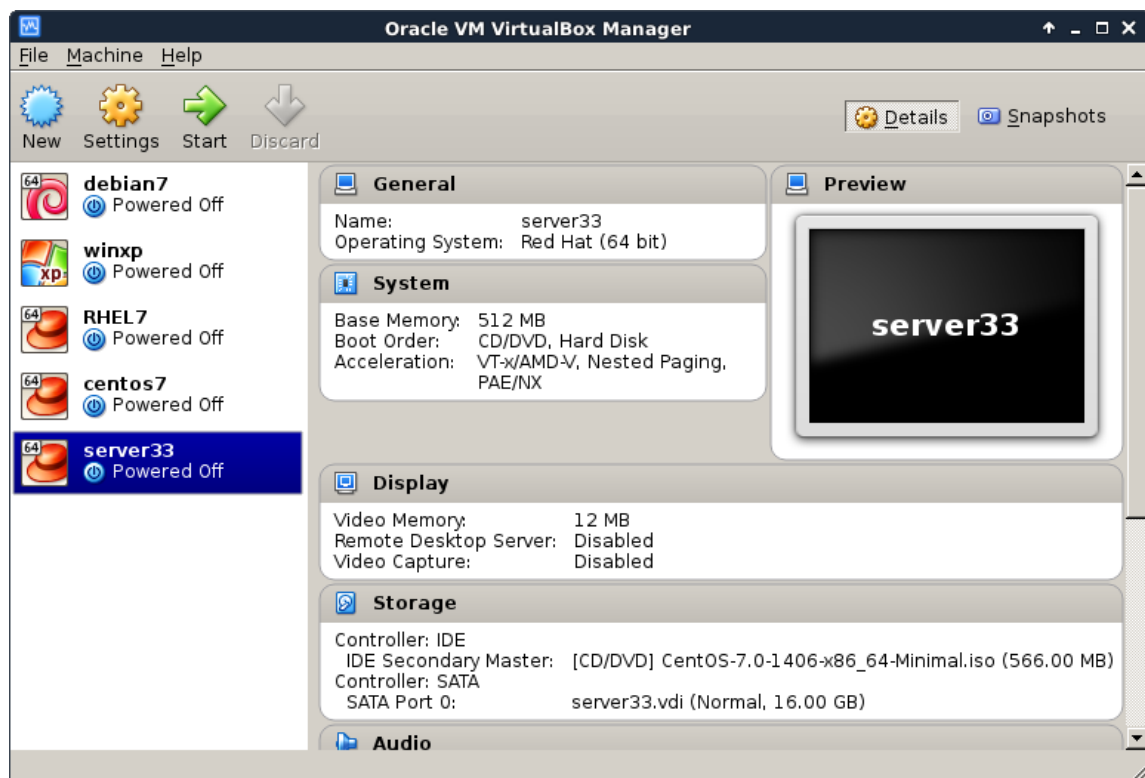
The default **dynamically allocated** type will save disk space (until we fill the virtual disk up to 100 percent). It makes the virtual machine a bit slower than **fixed size**, but the **fixed size** speed improvement is not worth it for our purpose.



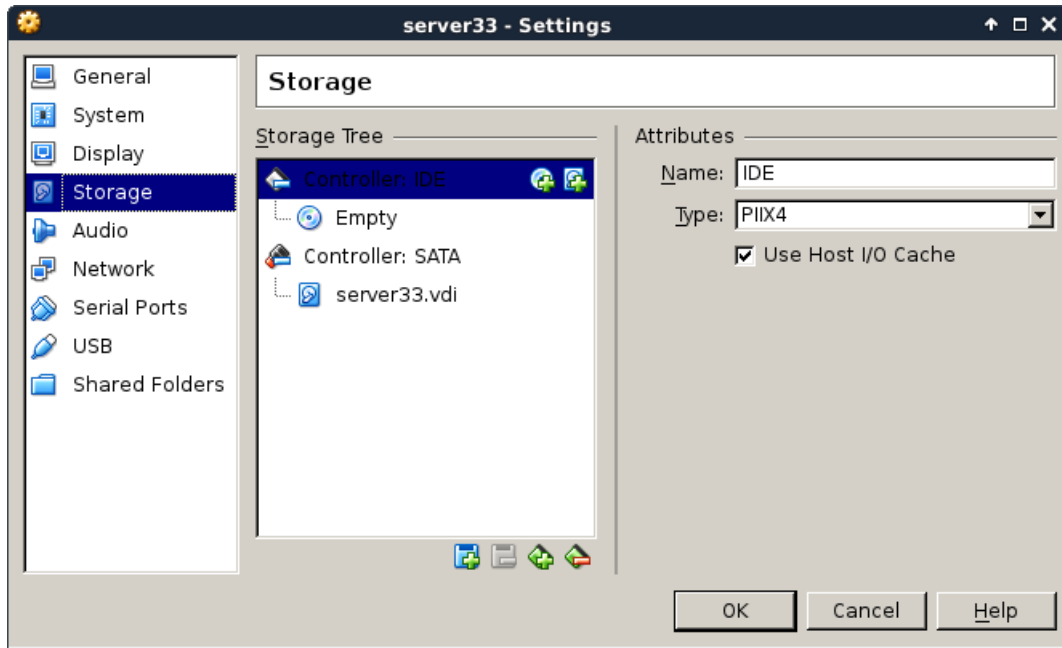
The name of the virtual disk file on the host computer will be **server33.vdi** in my case (I left it default and it uses the vm name). Also 16 GB should be enough to practice Linux. The file will stay much smaller than 16GB, unless you copy a lot of files to the virtual machine.



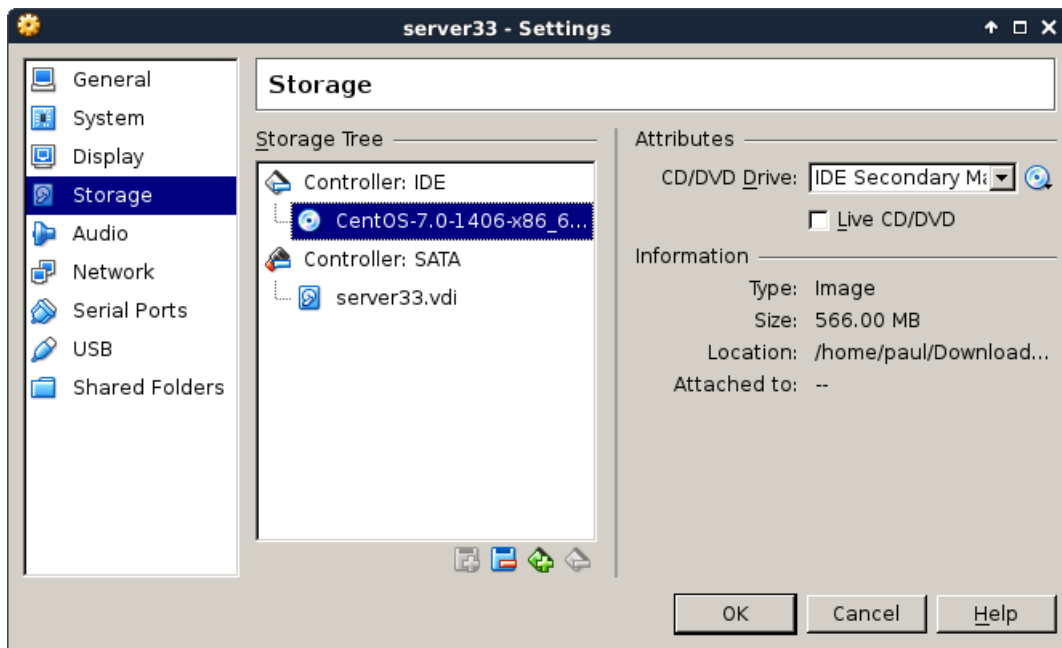
You should now be back to the start screen of **Virtualbox**. If all went well, then you should see the machine you just created in the list.



After finishing the setup, we go into the **Settings** of our virtual machine and attach the **.iso** file we downloaded before. Below is the default screenshot.



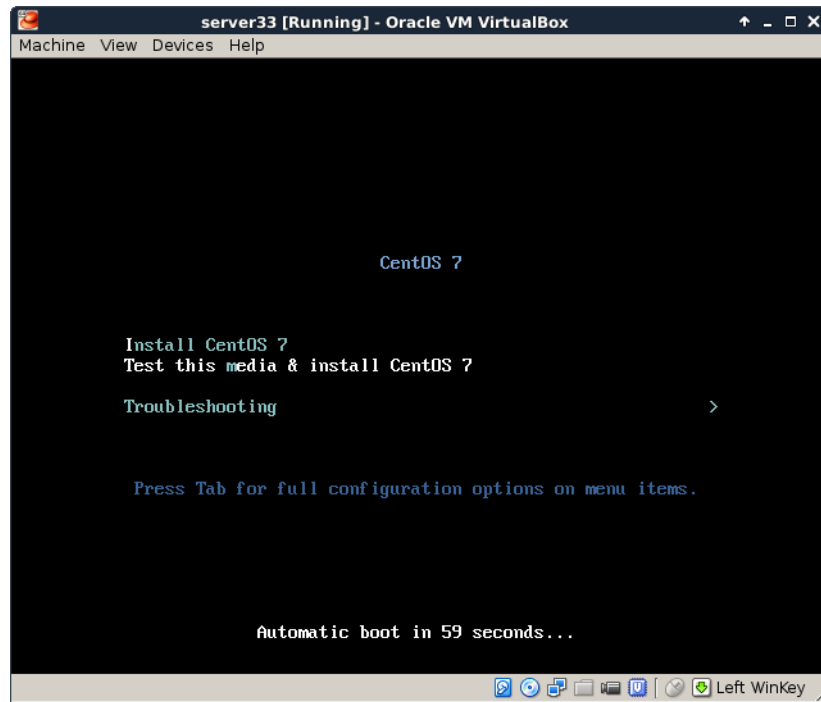
This is a screenshot with the **.iso** file properly attached.



5.3. CentOS 7 installing

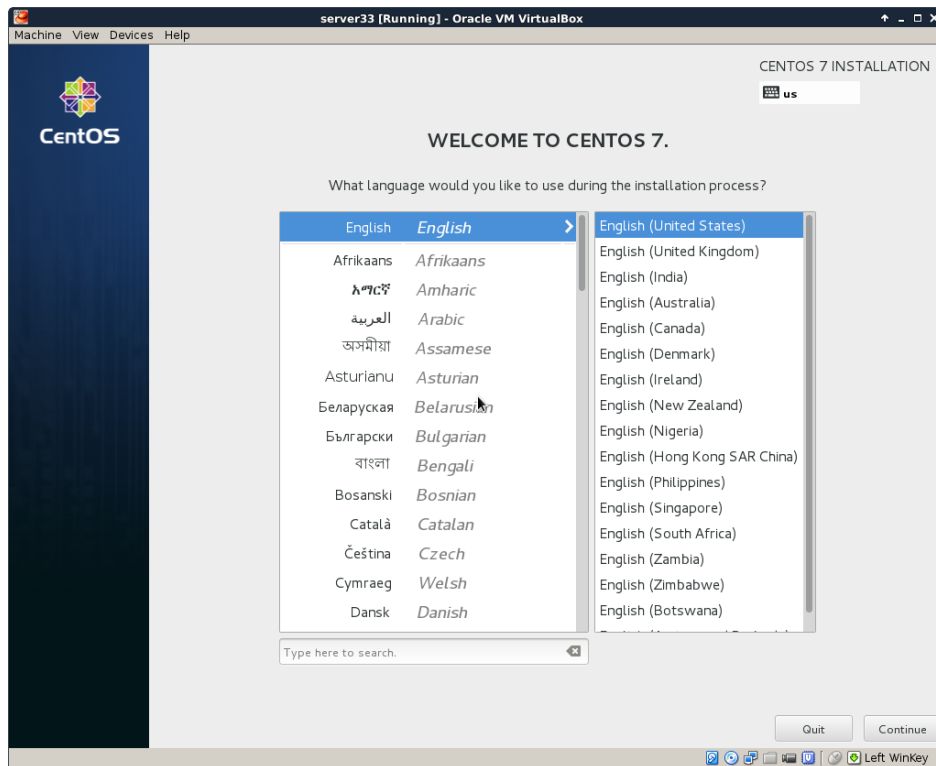
The screenshots below will show every step from starting the virtual machine for the first time (with the .iso file attached) until the first logon.

You should see this when booting, otherwise verify the attachment of the .iso file from the previous steps. Select **Test this media and install CentOS 7**.

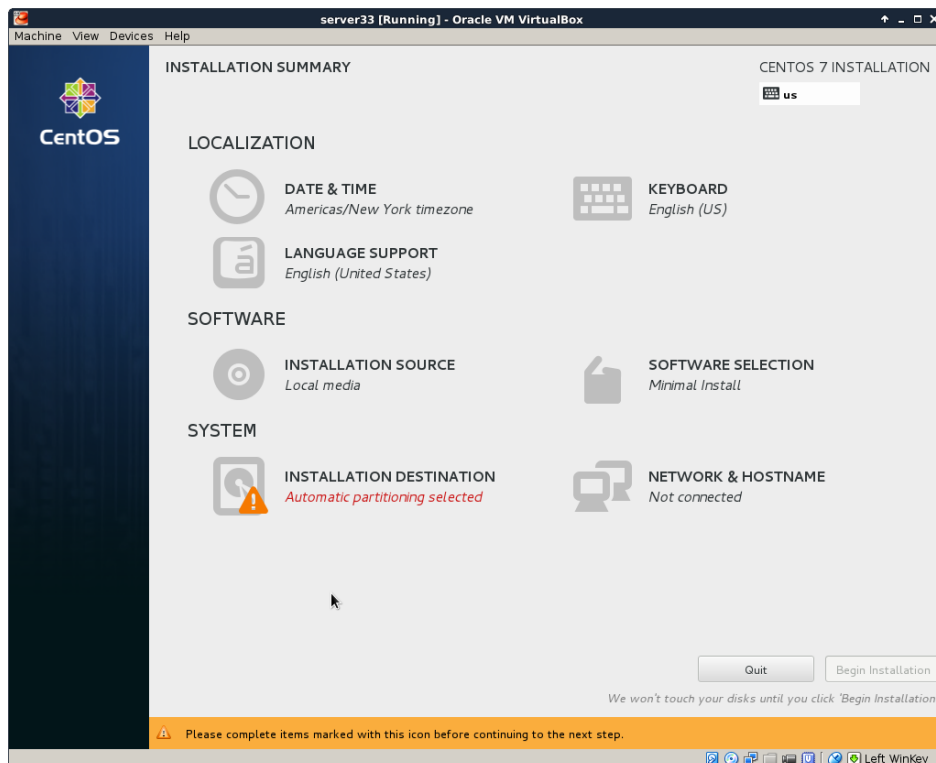


Carefully select the language in which you want your **CentOS**. I always install operating systems in English, even though my native language is not English.

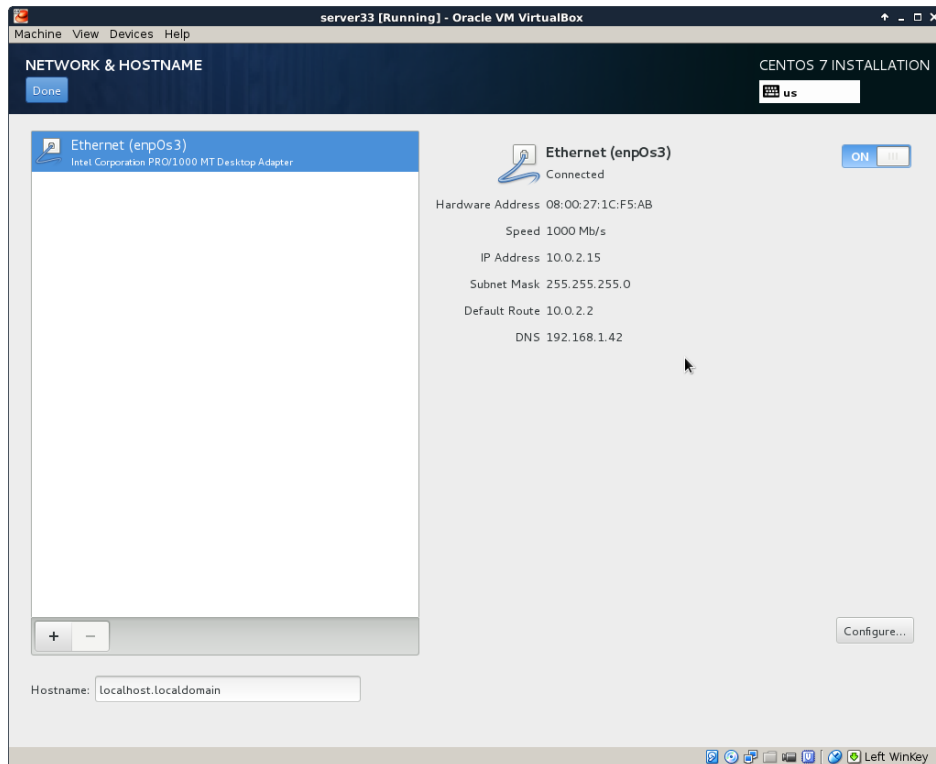
Also select the right keyboard, mine is a US qwerty, but yours may be different.



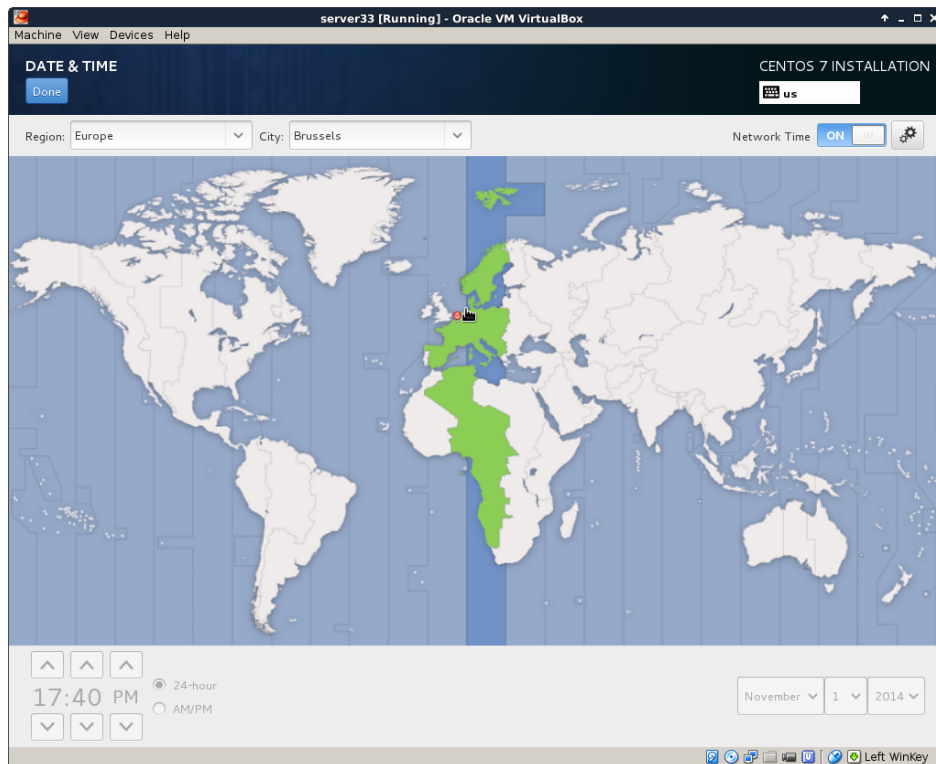
You should arrive at a summary page (with one or more warnings).



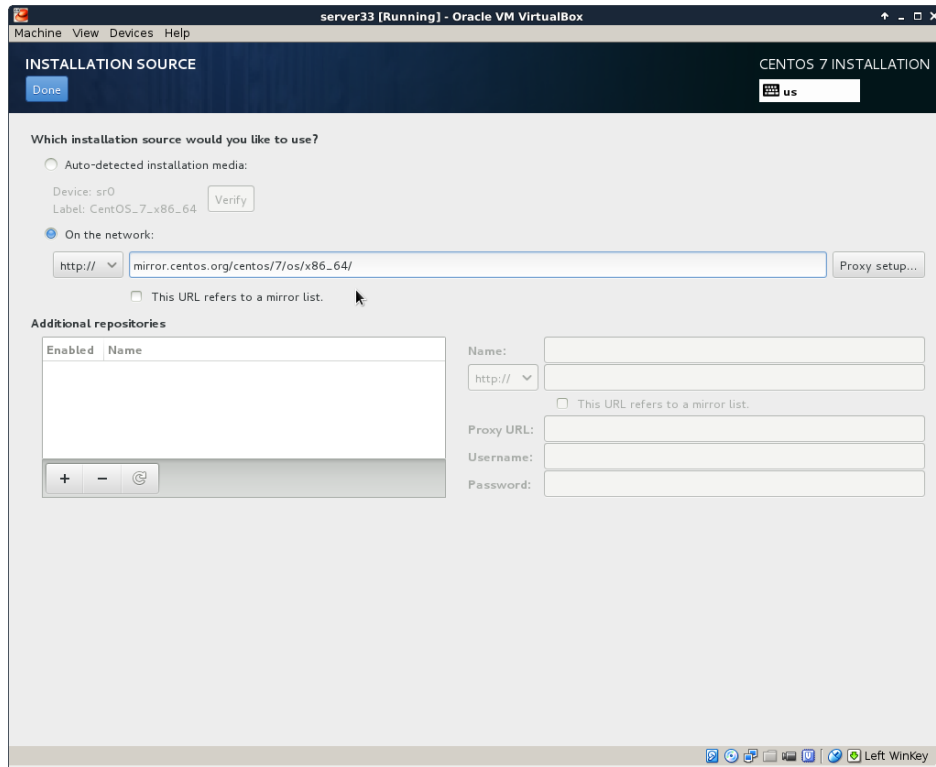
Start by configuring the network. During this demonstration I had a DHCP server running at 192.168.1.42, yours is probably different. Ask someone (a network administrator ?) for help if this step fails.



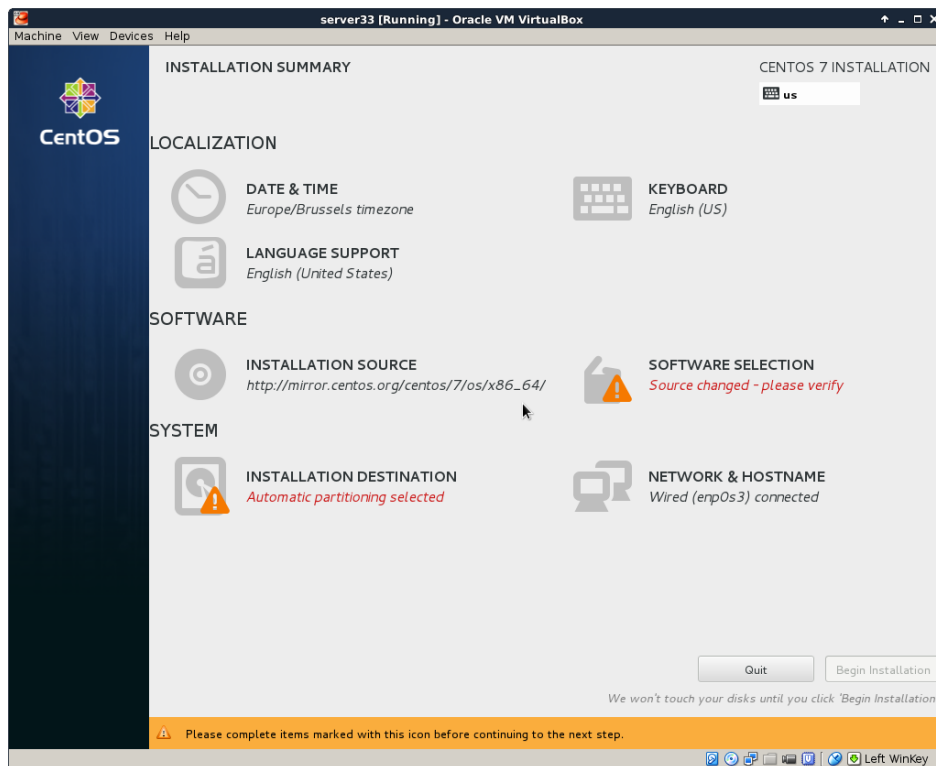
Select your time zone, and activate **ntp**.



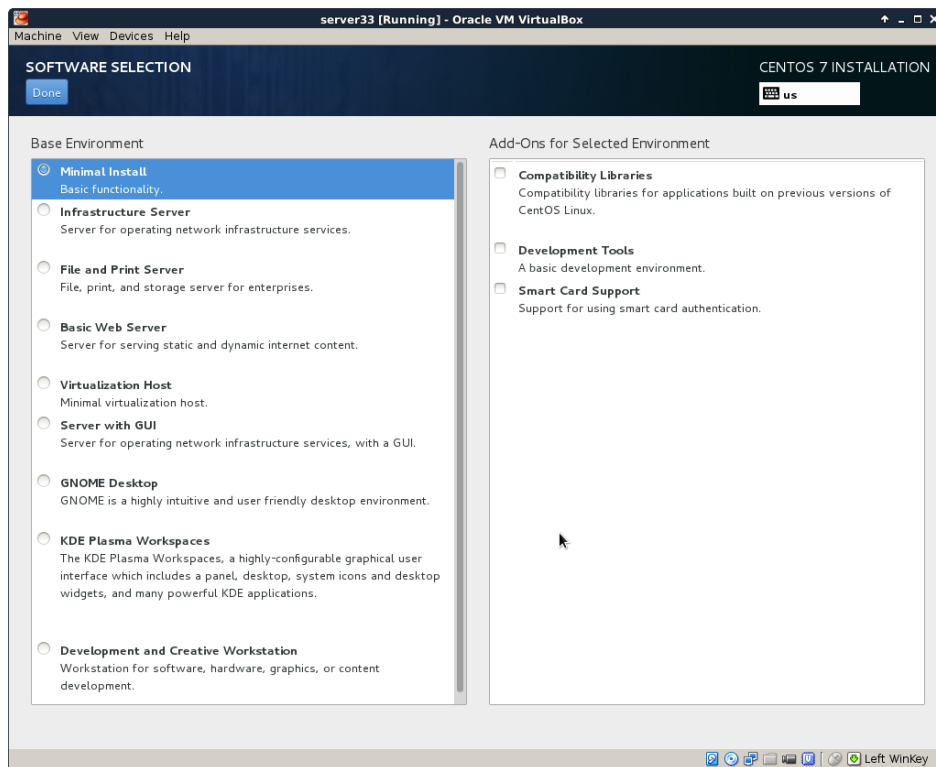
Choose a mirror that is close to you. If you can't find a local mirror, then you can copy the one from this screenshot (it is a general **CentOS** mirror).



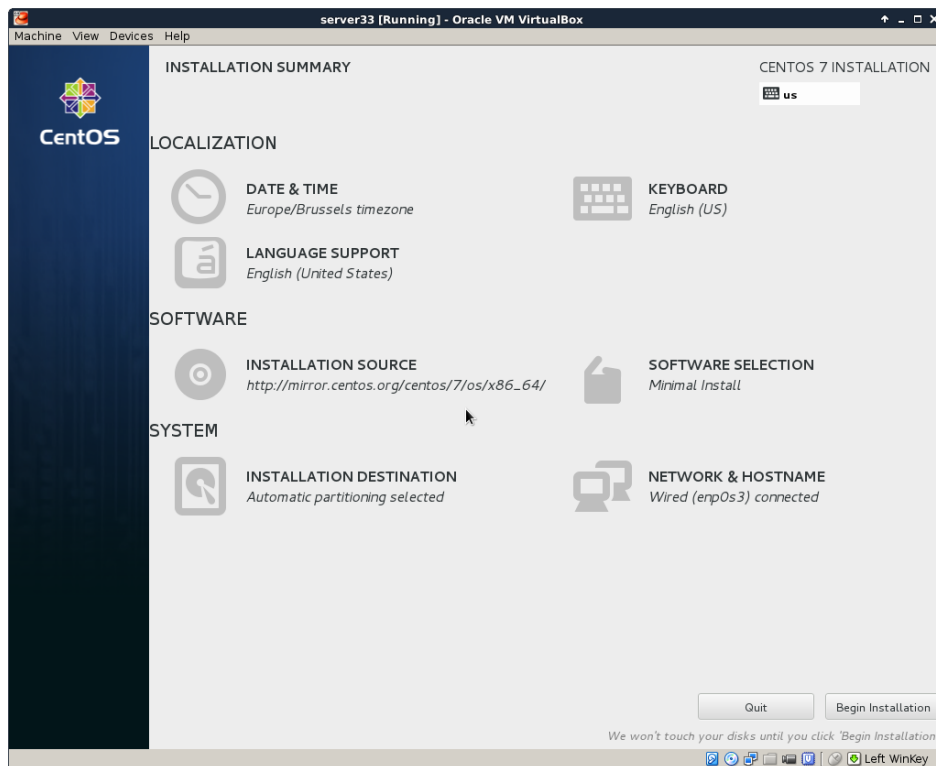
It can take a couple of seconds before the mirror is verified.



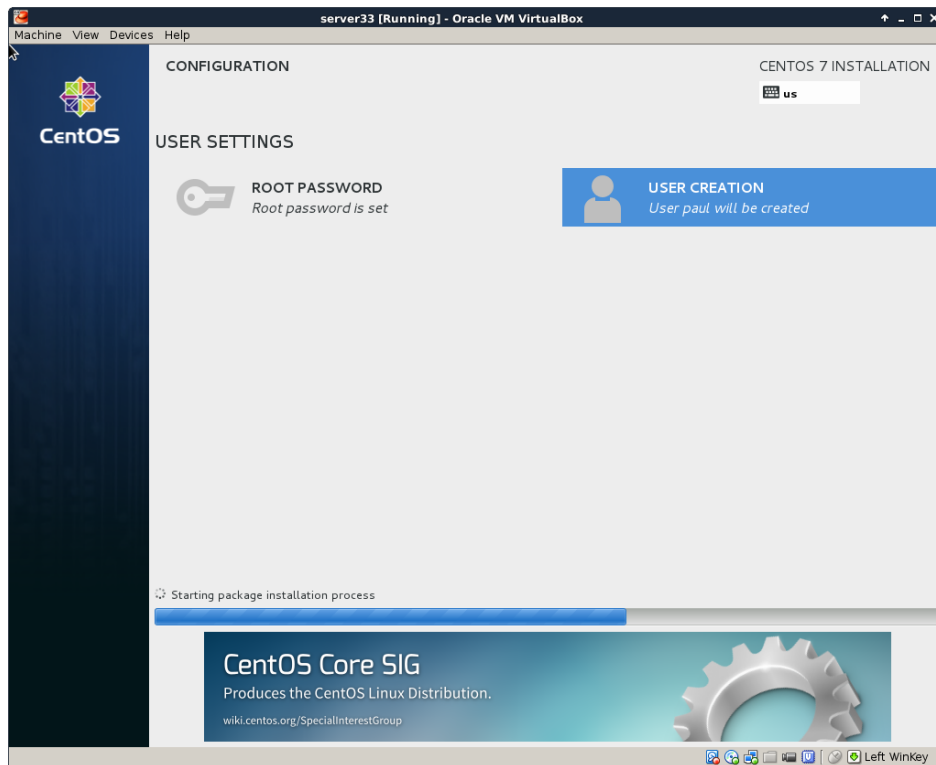
I did not select any software here (because I want to show it all in this training).



After configuring network, location, software and all, you should be back on this page. Make sure there are no warnings anymore (and that you made the correct choice everywhere).

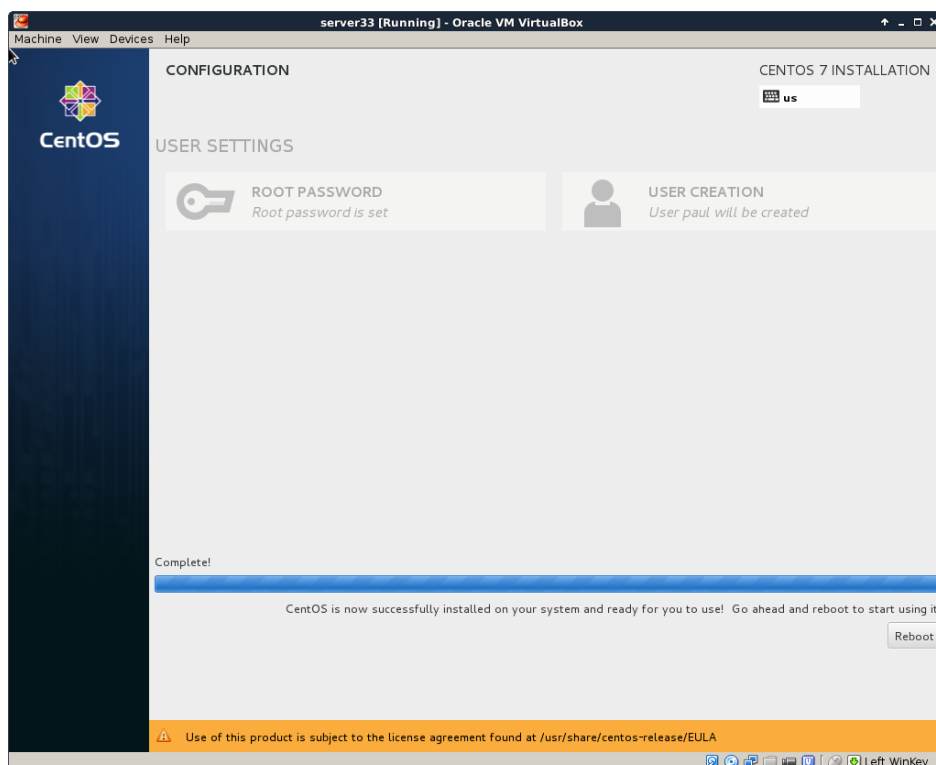


You can enter a **root password** and create a **user account** while the installation is downloading from the internet. This is the longest step, it can take several minutes (or up to an hour if you have a slow internet connection).

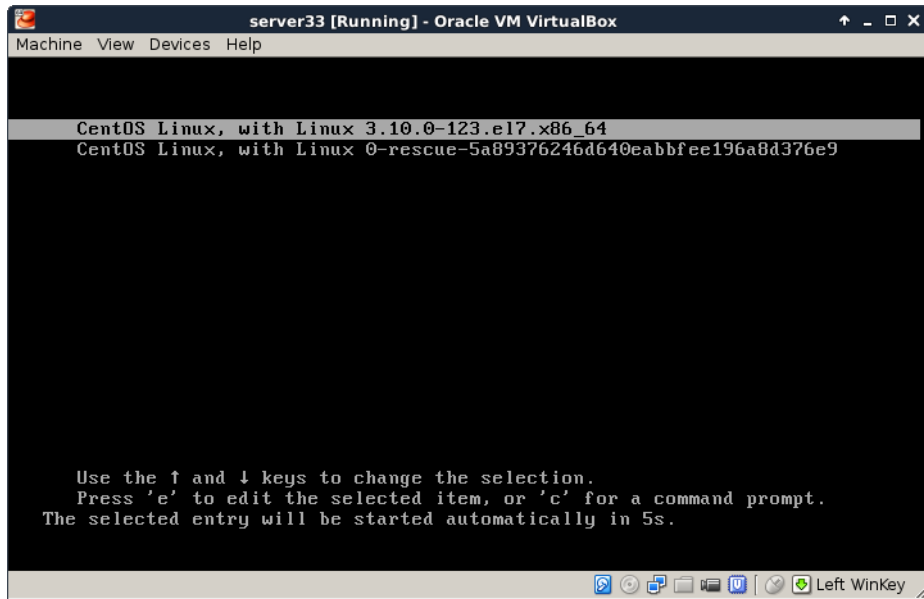


If you see this, then the installation was successful.

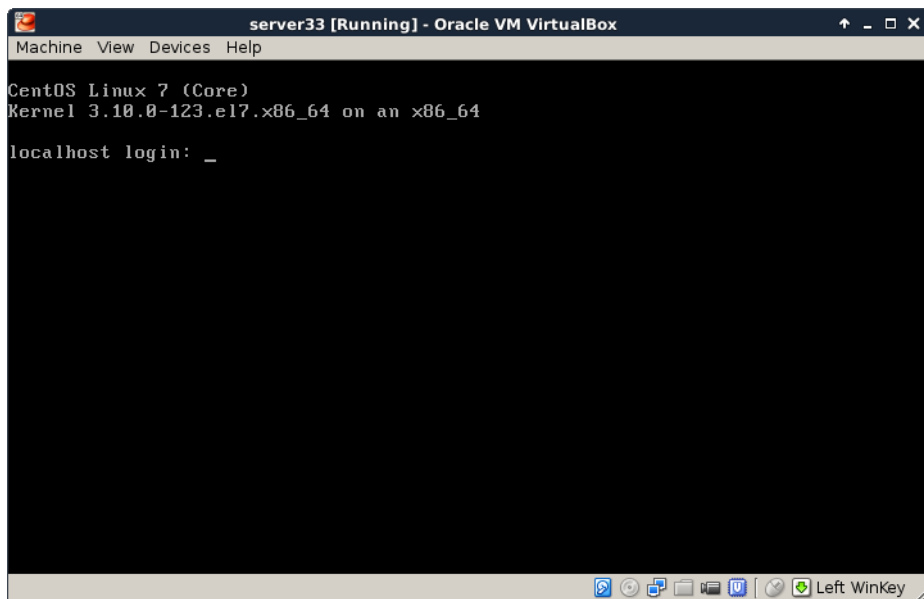
Time to reboot the computer and start **CentOS 7** for the first time.



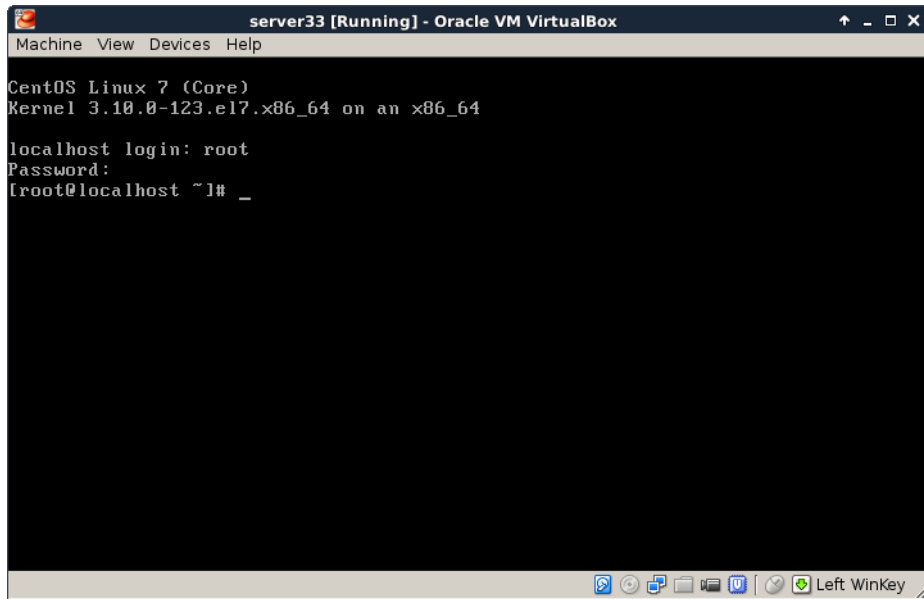
This screen will appear briefly when the virtual machines starts. You don't have to do anything.



After a couple of seconds, you should see a logon screen. This is called a **tty** or a **getty**. Here you can type **root** as username. The **login process** will then ask your password (nothing will appear on screen when you type your password).



And this is what it looks like after login. You are logged on to your own Linux machine, very good.



All subsequent screenshots will be text only, no images anymore.

For example this screenshot shows three commands being typed on my new CentOS 7 install.

```
[root@localhost ~]# who am i
root      pts/0      2014-11-01 22:14
[root@localhost ~]# hostname
localhost.localdomain
[root@localhost ~]# date
Sat Nov  1 22:14:37 CET 2014
```

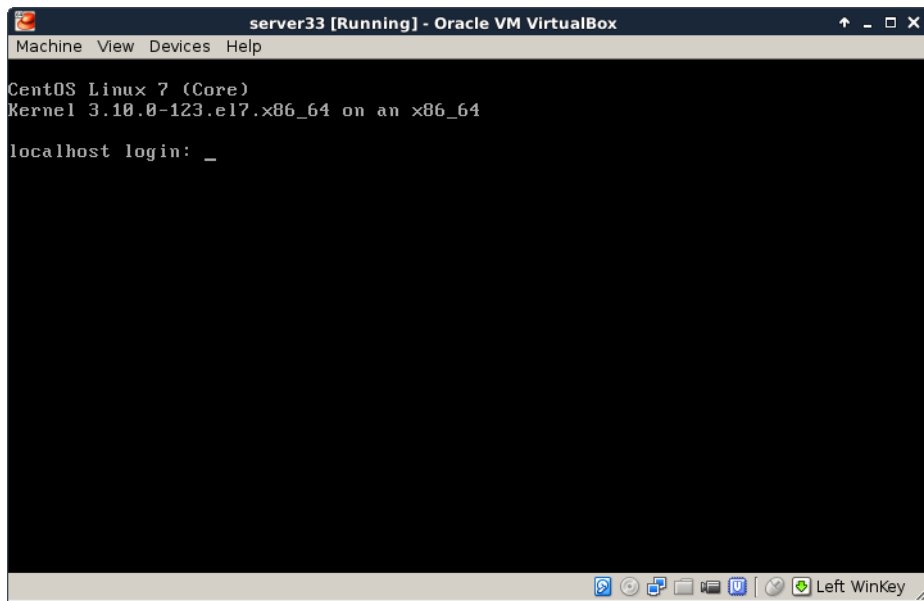
When using **ssh** the same commands will give this screenshot:

```
[root@localhost ~]# who am i
root      pts/0      2014-11-01 21:00 (192.168.1.35)
[root@localhost ~]# hostname
localhost.localdomain
[root@localhost ~]# date
Sat Nov  1 22:10:04 CET 2014
[root@localhost ~]#
```

If the last part is a bit too fast, take a look at the next topic **CentOS 7 first logon**.

5.4. CentOS 7 first logon

All you have to log on, after finishing the installation, is this screen in Virtualbox.



This is workable to learn Linux, and you will be able to practice a lot. But there are more ways to access your virtual machine, the next chapters discuss some of these and will also introduce some basic system configuration.

5.4.1. setting the hostname

Setting the hostname is as simple as changing the **/etc/hostname** file. As you can see here, it is set to **localhost.localdomain** by default.

```
[root@localhost ~]# cat /etc/hostname
localhost.localdomain
```

You could do **echo server33.netsec.local > /etc/hostname** followed by a **reboot**. But there is also the new **CentOS 7** way of setting a new hostname.

```
[root@localhost ~]# nmtui
```

The above command will give you a menu to choose from with a **set system hostname** option. Using this **nmtui** option will edit the **/etc/hostname** file for you.

```
[root@localhost ~]# cat /etc/hostname
server33.netsec.local
[root@localhost ~]# hostname
server33.netsec.local
[root@localhost ~]# dnsdomainname
netsec.local
```

For some reason the documentation on the **centos.org** and **docs.redhat.com** websites tell you to also execute this command:

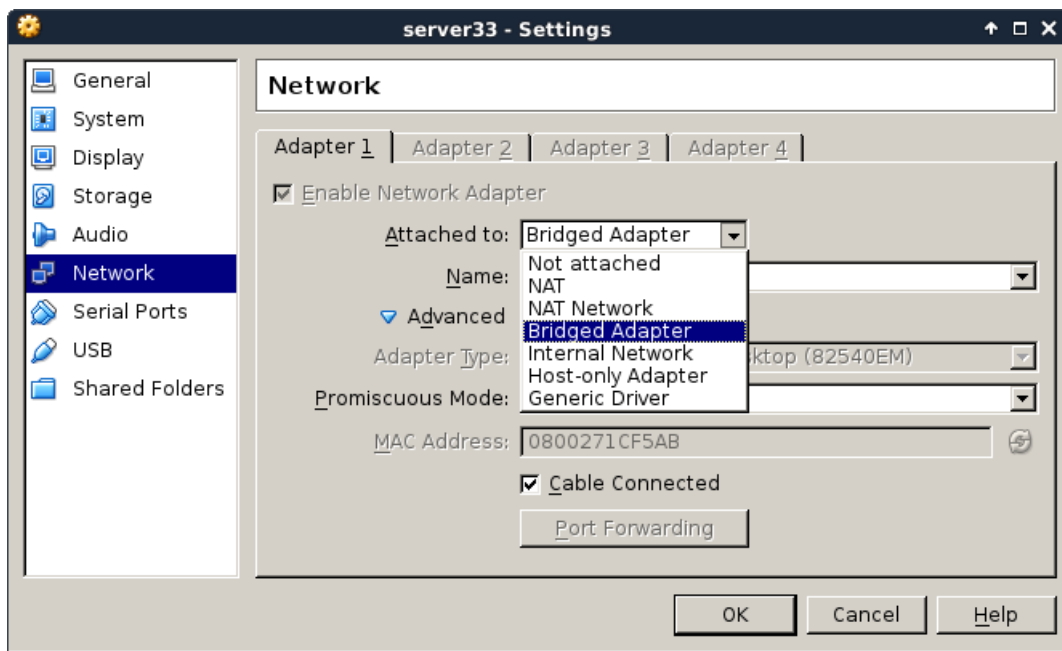
```
[root@localhost ~]# systemctl restart systemd-hostnamed
```

5.5. Virtualbox network interface

By default **Virtualbox** will connect your virtual machine over a **nat** interface. This will show up as a 10.0.2.15 (or similar).

```
[root@server33 ~]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast s\
tate UP qlen 1000
    link/ether 08:00:27:1c:f5:ab brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86399sec preferred_lft 86399sec
    inet6 fe80::a00:27ff:fe1c:f5ab/64 scope link
        valid_lft forever preferred_lft forever
```

You can change this to **bridge** (over your wi-fi or over the ethernet cable) and thus make it appear as if your virtual machine is directly on your local network (receiving an ip address from your real dhcp server).



You can make this change while the vm is running, provided that you execute this command:

```
[root@server33 ~]# systemctl restart network
[root@server33 ~]# ip a s dev enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast s\
tate UP qlen 1000
    link/ether 08:00:27:1c:f5:ab brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.110/24 brd 192.168.1.255 scope global dynamic enp0s3
        valid_lft 7199sec preferred_lft 7199sec
    inet6 fe80::a00:27ff:fe1c:f5ab/64 scope link
        valid_lft forever preferred_lft forever
[root@server33 ~]#
```

5.6. configuring the network

The new way of changing network configuration is through the **nmtui** tool. If you want to manually play with the files in **/etc/sysconfig/network-scripts** then you will first need to verify (and disable) **NetworkManager** on that interface.

Verify whether an interface is controlled by **NetworkManager** using the **nmcli** command (connected means managed by NM).

```
[root@server33 ~]# nmcli dev status
DEVICE  TYPE      STATE      CONNECTION
enp0s3  ethernet  connected  enp0s3
lo       loopback  unmanaged  --
```

Disable **NetworkManager** on an interface (enp0s3 in this case):

```
echo 'NM_CONTROLLED=no' >> /etc/sysconfig/network-scripts/ifcfg-enp0s3
```

You can restart the network without a reboot like this:

```
[root@server33 ~]# systemctl restart network
```

Also, forget **ifconfig** and instead use **ip a**.

```
[root@server33 ~]# ip a s dev enp0s3 | grep inet
    inet 192.168.1.110/24 brd 192.168.1.255 scope global dynamic enp0s3
    inet6 fe80::a00:27ff:fe1c:f5ab/64 scope link
[root@server33 ~]#
```

5.7. adding one static ip address

This example shows how to add one static ip address to your computer.

```
[root@server33 ~]# nmtui edit enp0s3
```

In this interface leave the IPv4 configuration to automatic, and add an ip address just below.

```
IPv4 CONFIGURATION <Automatic> <Hide>
Addresses 10.104.33.32/16 <Remove>
```

Execute this command after exiting **nmtui**.

```
[root@server33 ~]# systemctl restart network
```

And verify with **ip** (not with **ifconfig**):

```
[root@server33 ~]# ip a s dev enp0s3 | grep inet
    inet 192.168.1.110/24 brd 192.168.1.255 scope global dynamic enp0s3
    inet 10.104.33.32/16 brd 10.104.255.255 scope global enp0s3
    inet6 fe80::a00:27ff:fe1c:f5ab/64 scope link
[root@server33 ~]#
```

5.8. package management

Even with a network install, **CentOS 7** did not install the latest version of some packages. Luckily there is only one command to run (as root). This can take a while.

```
[root@server33 ~]# yum update
Loaded plugins: fastestmirror
Loading mirror speeds from cached hostfile
* base: centos.weepeetelecom.be
* extras: centos.weepeetelecom.be
* updates: centos.weepeetelecom.be
Resolving Dependencies
--> Running transaction check
---> Package NetworkManager.x86_64 1:0.9.9.1-13.git20140326.4dba720.el7 \
will be updated
... (output truncated)
```

You can also use **yum** to install one or more packages. Do not forget to run **yum update** from time to time.

```
[root@server33 ~]# yum update -y && yum install vim -y
Loaded plugins: fastestmirror
Loading mirror speeds from cached hostfile
* base: centos.weepeetelecom.be
... (output truncated)
```

Refer to the package management chapter for more information on installing and removing packages.

5.9. logon from Linux and MacOSX

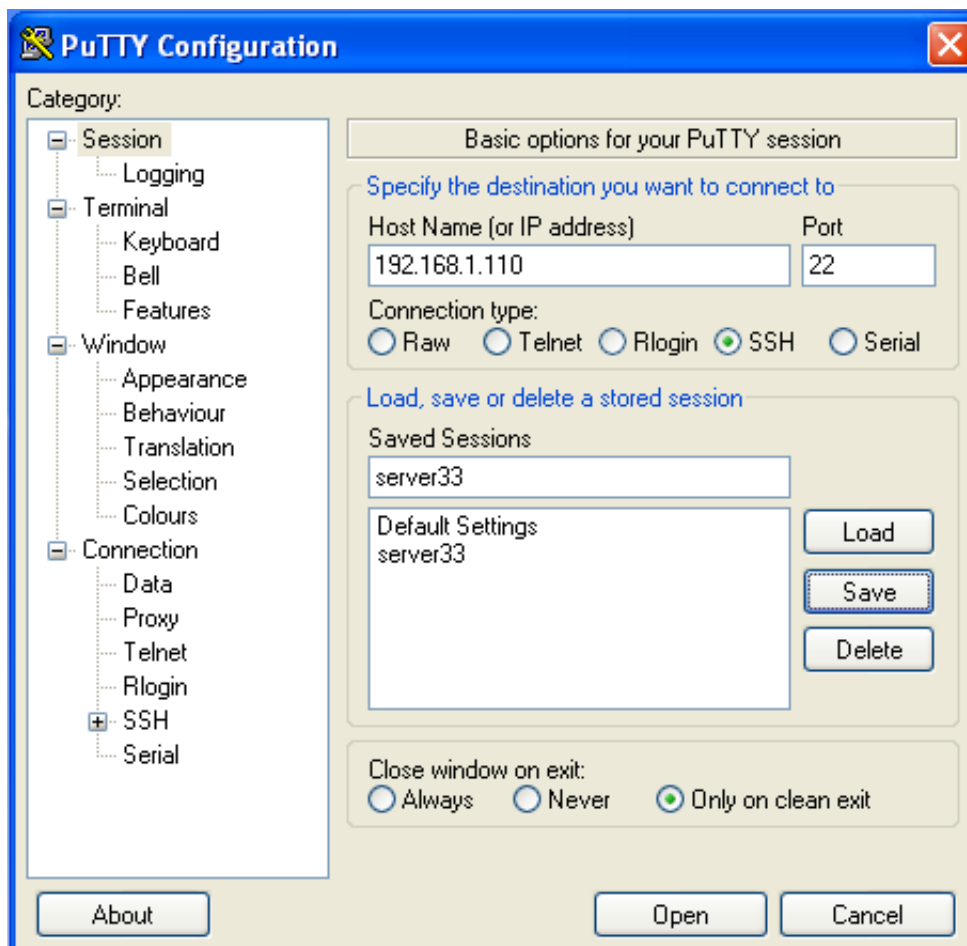
You can now open a terminal on Linux or MacOSX and use **ssh** to log on to your virtual machine.

```
paul@debian8:~$ ssh root@192.168.1.110
root@192.168.1.110's password:
Last login: Sun Nov  2 11:53:57 2014
[root@server33 ~]# hostname
server33.netsec.local
[root@server33 ~]#
```

5.10. logon from MS Windows

There is no **ssh** installed on MS Windows, but you can download **putty.exe** from <http://www.chiark.greenend.org.uk/~sgtatham/putty/download.html> (just Google it).

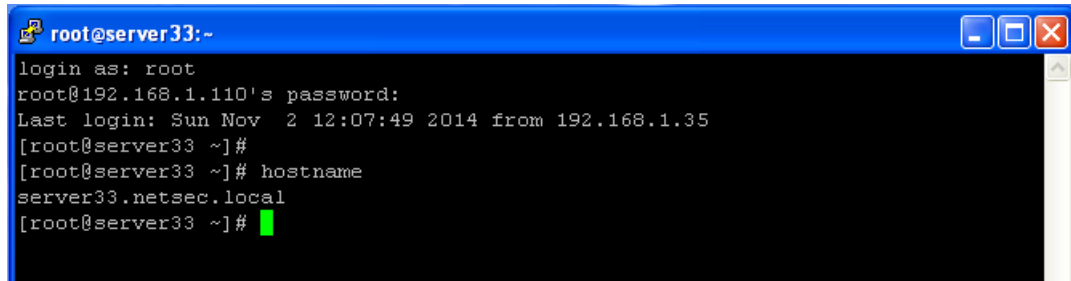
Use **putty.exe** as shown in this screenshot (I saved the ip address by giving it a name 'server33' and presing the 'save' button).



The first time you will get a message about keys, accept this (this is explained in the ssh chapter).



Enter your userid (or root) and the correct password (nothing will appear on the screen when typing a password).



Chapter 6. getting Linux at home

This chapter shows a Ubuntu install in Virtualbox. Consider it legacy and use CentOS7 or Debian8 instead (each have their own chapter now).

This book assumes you have access to a working Linux computer. Most companies have one or more Linux servers, if you have already logged on to it, then you 're all set (skip this chapter and go to the next).

Another option is to insert a Ubuntu Linux CD in a computer with (or without) Microsoft Windows and follow the installation. Ubuntu will resize (or create) partitions and setup a menu at boot time to choose Windows or Linux.

If you do not have access to a Linux computer at the moment, and if you are unable or unsure about installing Linux on your computer, then this chapter proposes a third option: installing Linux in a virtual machine.

Installation in a virtual machine (provided by **Virtualbox**) is easy and safe. Even when you make mistakes and crash everything on the virtual Linux machine, then nothing on the real computer is touched.

This chapter gives easy steps and screenshots to get a working Ubuntu server in a Virtualbox virtual machine. The steps are very similar to installing Fedora or CentOS or even Debian, and if you like you can also use VMWare instead of Virtualbox.

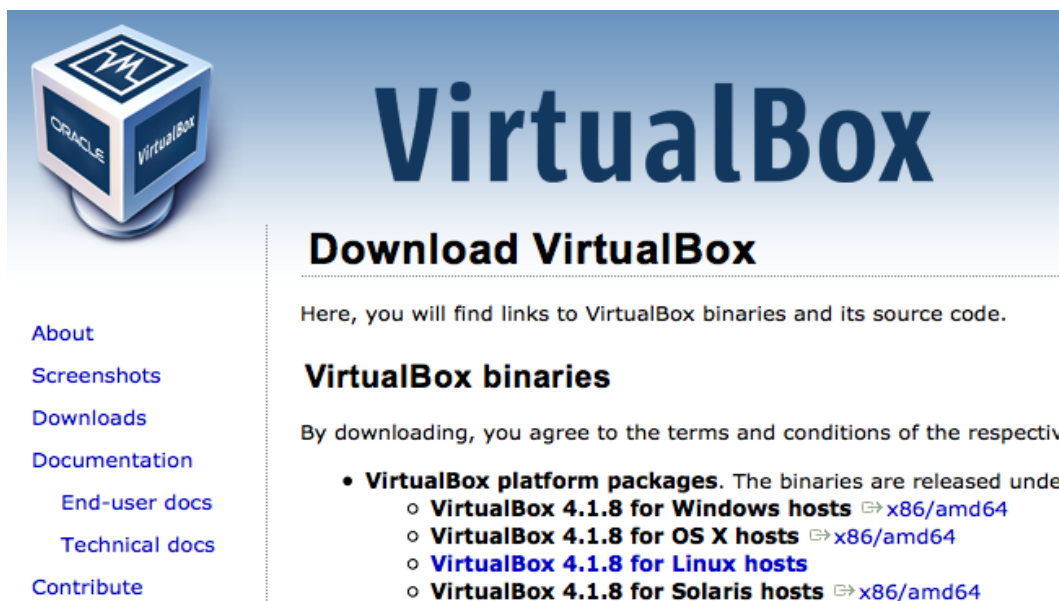
6.1. download a Linux CD image

Start by downloading a Linux CD image (an .ISO file) from the distribution of your choice from the Internet. Take care selecting the correct cpu architecture of your computer; choose **i386** if unsure. Choosing the wrong cpu type (like x86_64 when you have an old Pentium) will almost immediately fail to boot the CD.



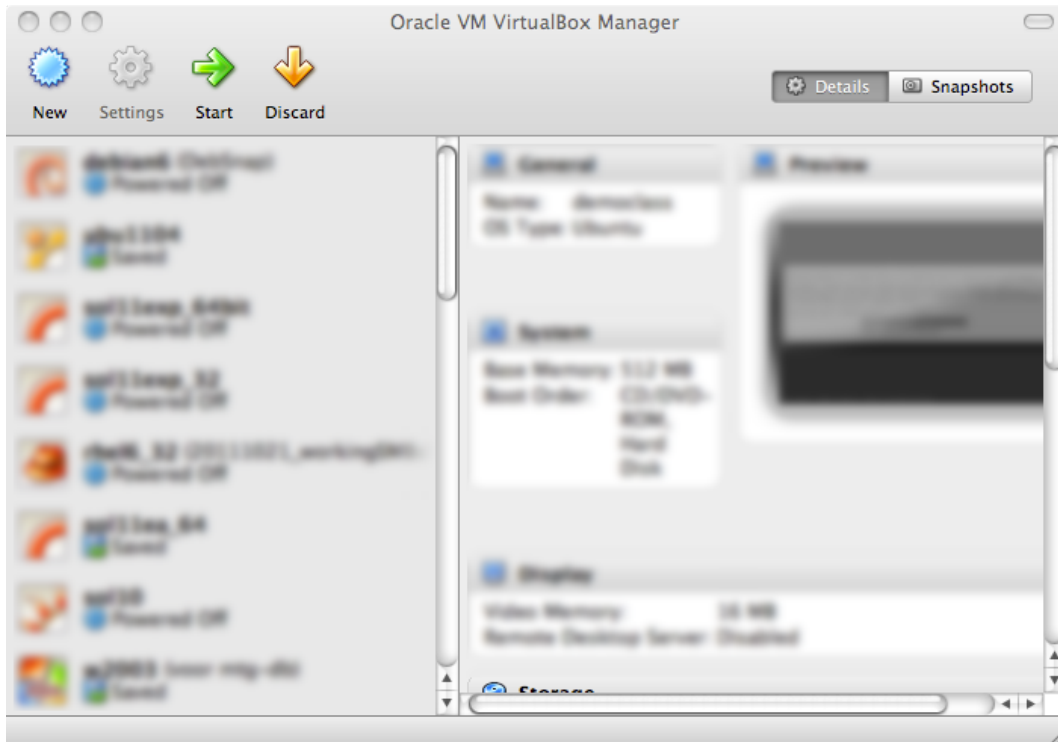
6.2. download Virtualbox

Step two (when the .ISO file has finished downloading) is to download Virtualbox. If you are currently running Microsoft Windows, then download and install Virtualbox for Windows!



6.3. create a virtual machine

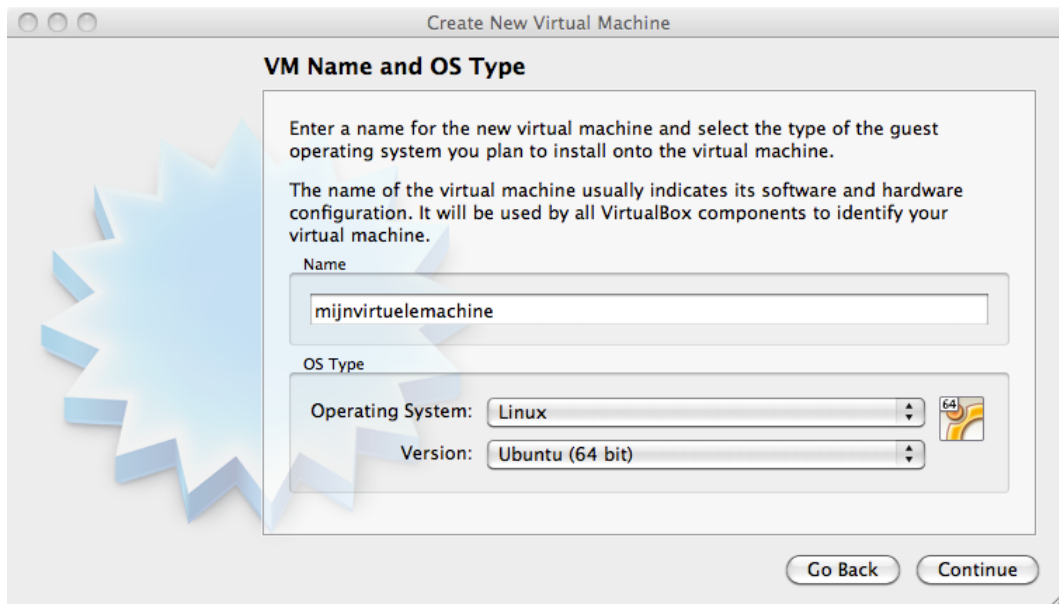
Now start Virtualbox. Contrary to the screenshot below, your left pane should be empty.



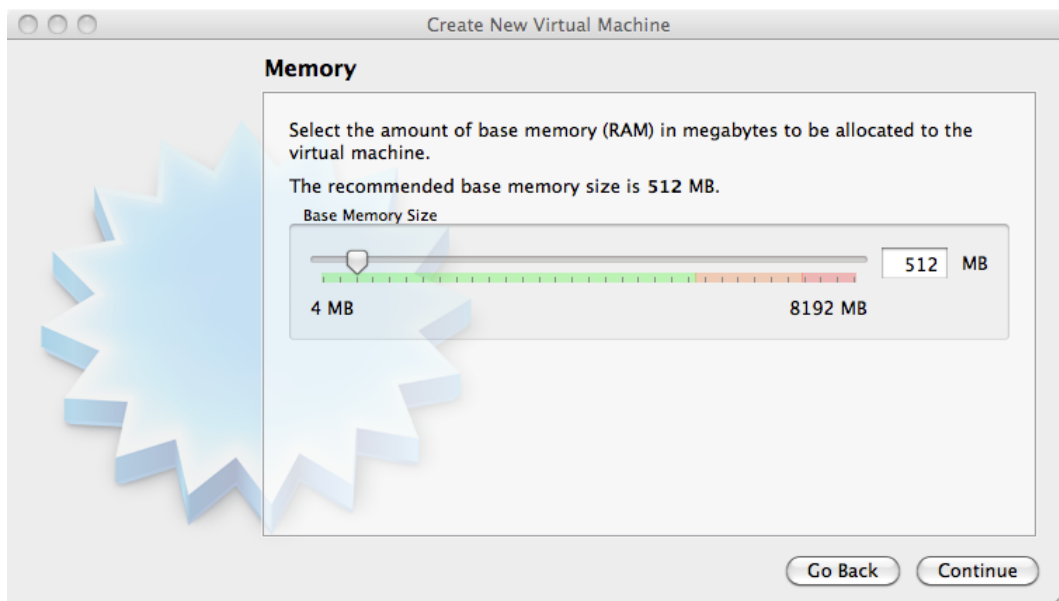
Click **New** to create a new virtual machine. We will walk together through the wizard. The screenshots below are taken on Mac OSX; they will be slightly different if you are running Microsoft Windows.



Name your virtual machine (and maybe select 32-bit or 64-bit).



Give the virtual machine some memory (512MB if you have 2GB or more, otherwise select 256MB).



Select to create a new disk (remember, this will be a virtual disk).



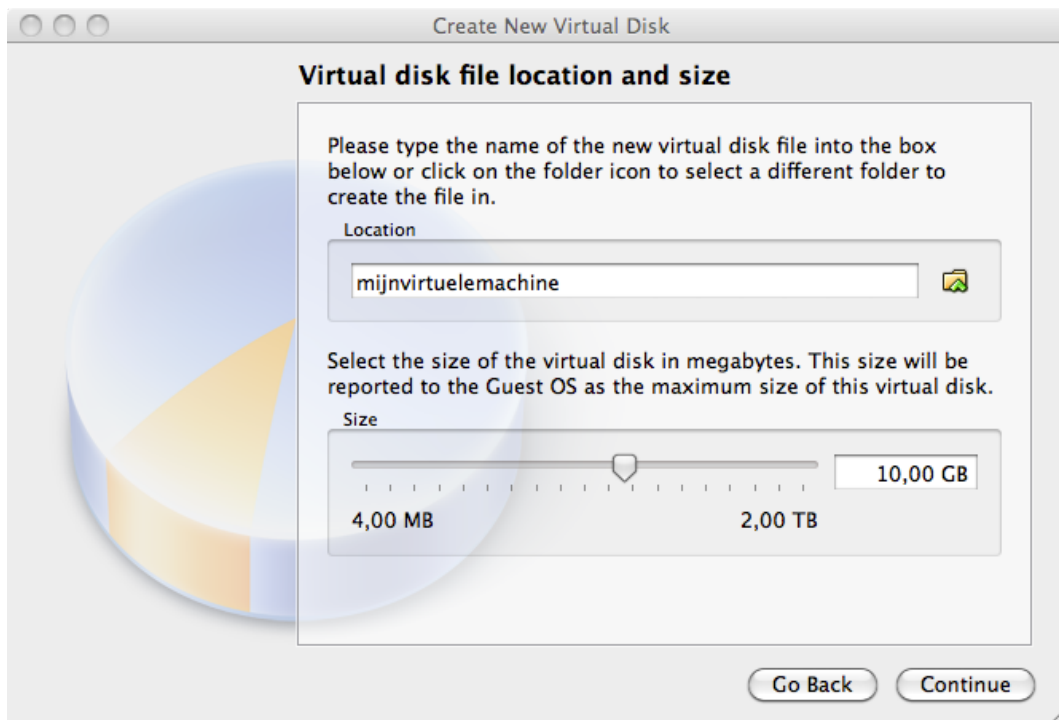
If you get the question below, choose vdi.



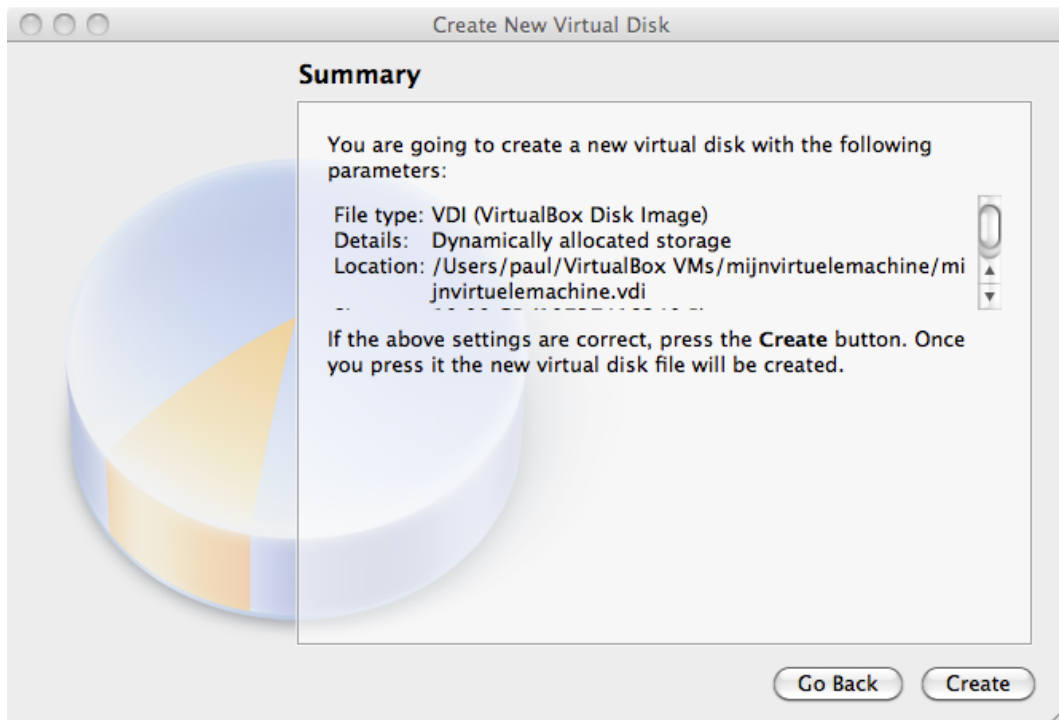
Choose **dynamically allocated** (fixed size is only useful in production or on really old, slow hardware).



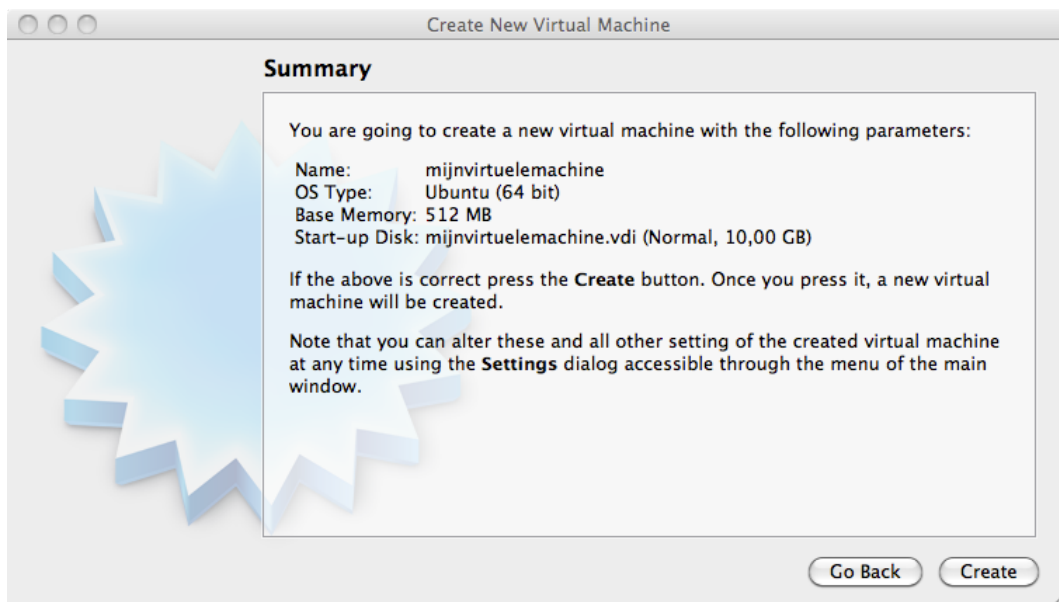
Choose between 10GB and 16GB as the disk size.



Click **create** to create the virtual disk.

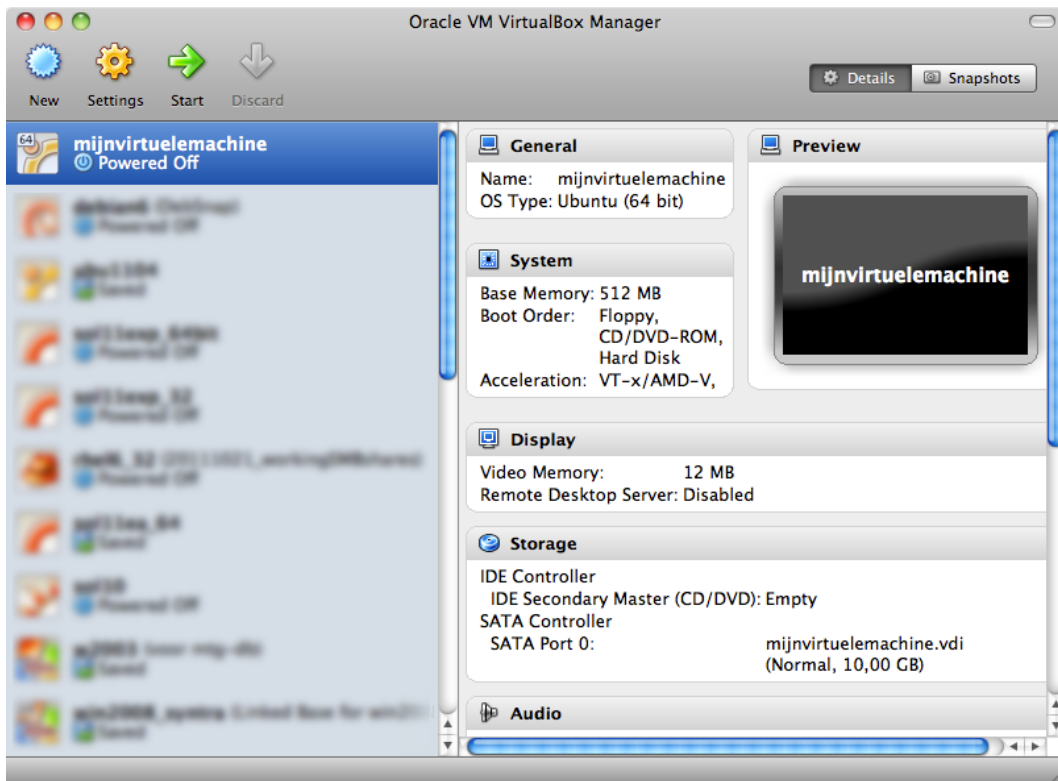


Click **create** to create the virtual machine.

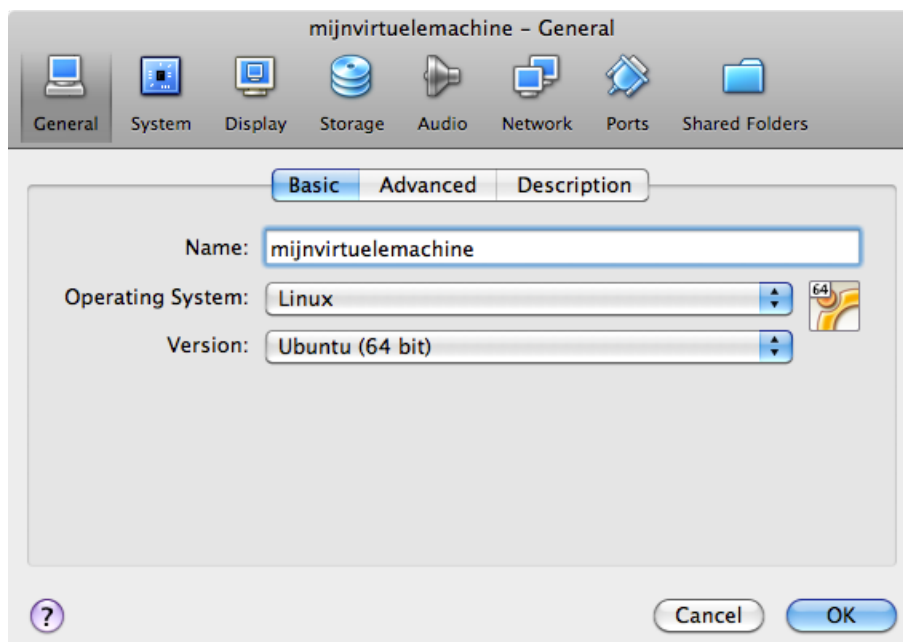


6.4. attach the CD image

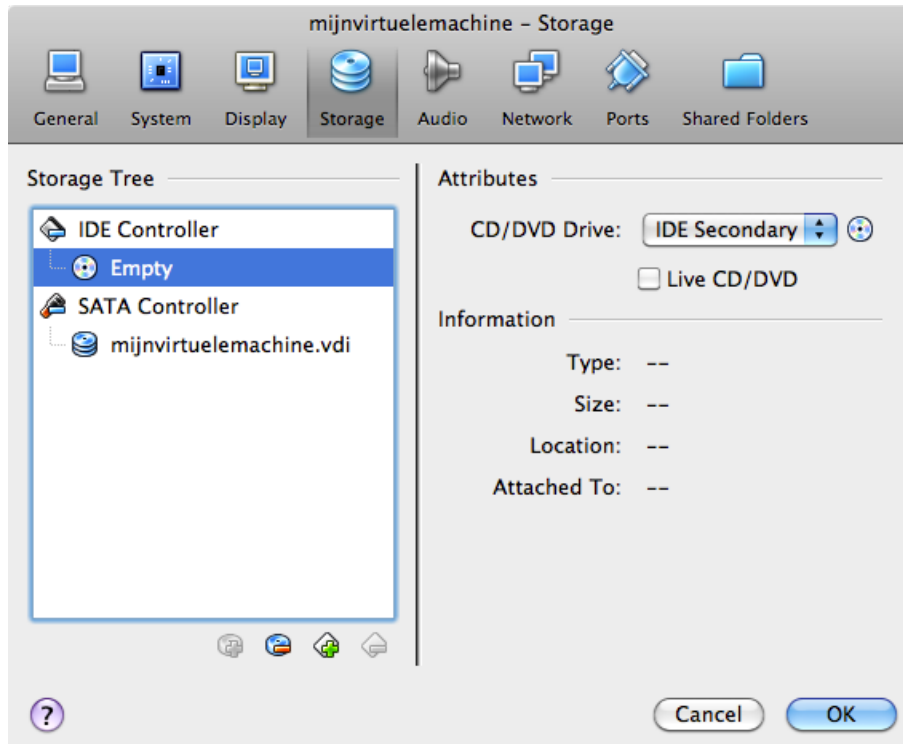
Before we start the virtual computer, let us take a look at some settings (click **Settings**).



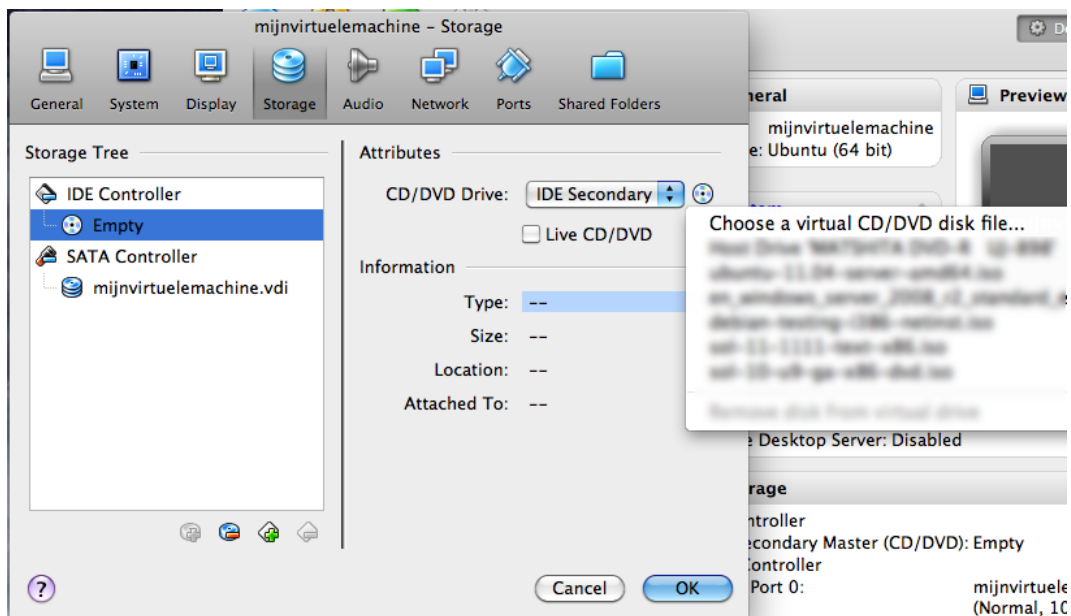
Do not worry if your screen looks different, just find the button named **storage**.



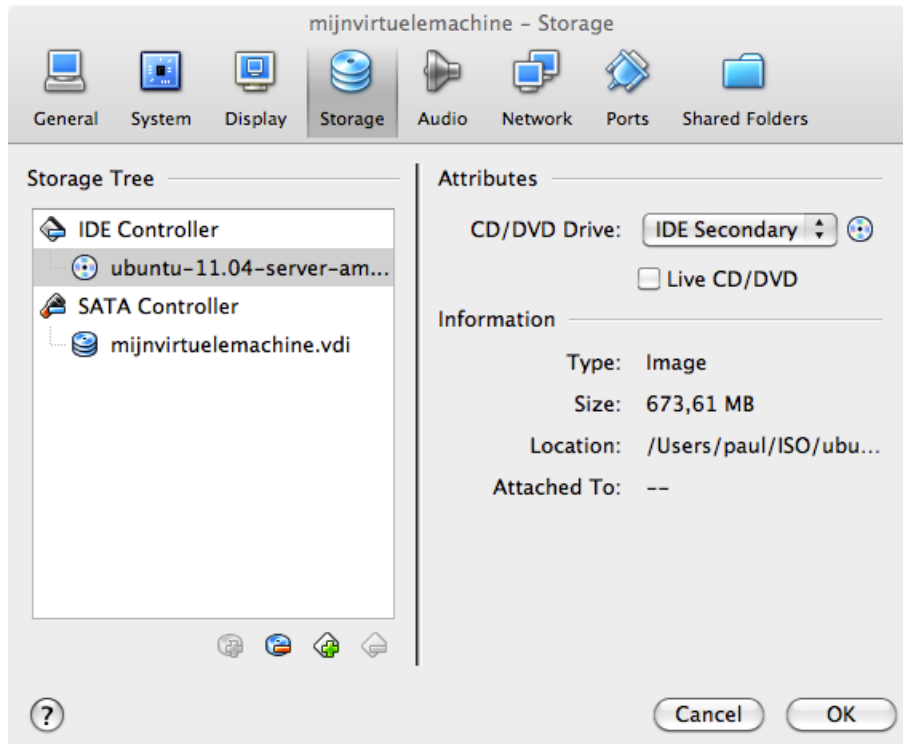
Remember the .ISO file you downloaded? Connect this .ISO file to this virtual machine by clicking on the CD icon next to **Empty**.



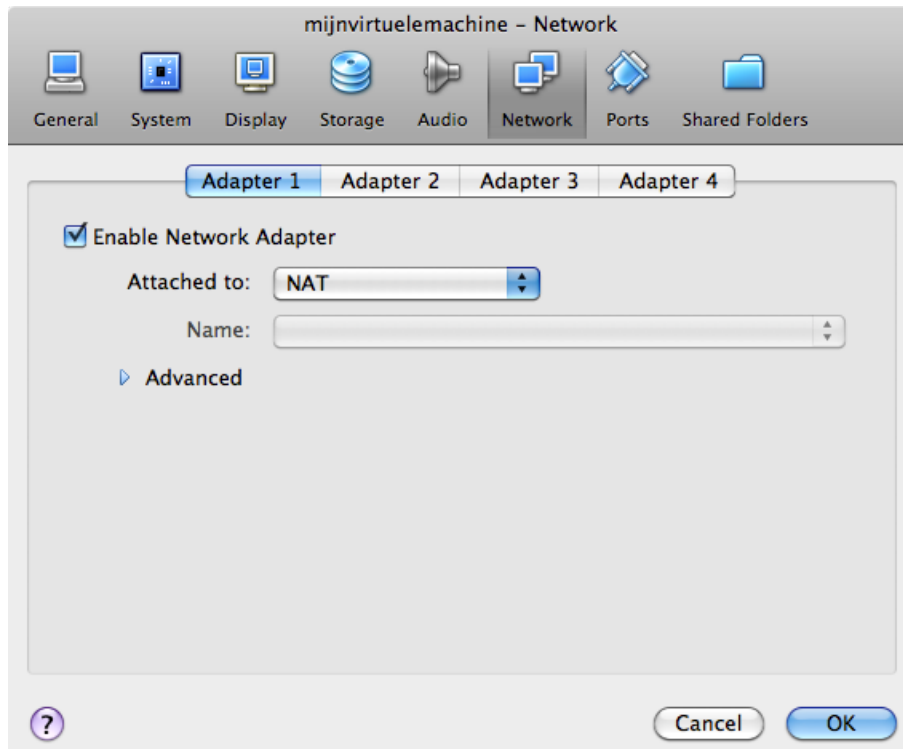
Now click on the other CD icon and attach your ISO file to this virtual CD drive.



Verify that your download is accepted. If Virtualbox complains at this point, then you probably did not finish the download of the CD (try downloading it again).



It could be useful to set the network adapter to bridge instead of NAT. Bridged usually will connect your virtual computer to the Internet.



6.5. install Linux

The virtual machine is now ready to start. When given a choice at boot, select **install** and follow the instructions on the screen. When the installation is finished, you can log on to the machine and start practising Linux!

Part III. first steps on the command line

Table of Contents

7. man pages	70
7.1. man \$command	71
7.2. man \$configfile	71
7.3. man \$daemon	71
7.4. man -k (apropos)	71
7.5. whatis	71
7.6. whereis	71
7.7. man sections	72
7.8. man \$section \$file	72
7.9. man man	72
7.10. mandb	72
8. working with directories	73
8.1. pwd	74
8.2. cd	74
8.3. absolute and relative paths	75
8.4. path completion	76
8.5. ls	76
8.6. mkdir	78
8.7. rmdir	78
8.8. practice: working with directories	80
8.9. solution: working with directories	81
9. working with files	83
9.1. all files are case sensitive	84
9.2. everything is a file	84
9.3. file	84
9.4. touch	85
9.5. rm	86
9.6. cp	87
9.7. mv	88
9.8. rename	89
9.9. practice: working with files	90
9.10. solution: working with files	91
10. working with file contents	93
10.1. head	94
10.2. tail	94
10.3. cat	95
10.4. tac	96
10.5. more and less	97
10.6. strings	97
10.7. practice: file contents	98
10.8. solution: file contents	99
11. the Linux file tree	100
11.1. filesystem hierarchy standard	101
11.2. man hier	101
11.3. the root directory /	101
11.4. binary directories	102
11.5. configuration directories	104
11.6. data directories	106
11.7. in memory directories	108
11.8. /usr Unix System Resources	113
11.9. /var variable data	115
11.10. practice: file system tree	117
11.11. solution: file system tree	119

Chapter 7. man pages

This chapter will explain the use of **man** pages (also called **manual pages**) on your Unix or Linux computer.

You will learn the **man** command together with related commands like **whereis**, **whatis** and **mandb**.

Most Unix files and commands have pretty good man pages to explain their use. Man pages also come in handy when you are using multiple flavours of Unix or several Linux distributions since options and parameters sometimes vary.

7.1. man \$command

Type **man** followed by a command (for which you want help) and start reading. Press **q** to quit the manpage. Some man pages contain examples (near the end).

```
paul@laika:~$ man whois
Reformatting whois(1), please wait...
```

7.2. man \$configfile

Most **configuration** files have their own manual.

```
paul@laika:~$ man syslog.conf
Reformatting syslog.conf(5), please wait...
```

7.3. man \$daemon

This is also true for most **daemons** (background programs) on your system..

```
paul@laika:~$ man syslogd
Reformatting syslogd(8), please wait...
```

7.4. man -k (apropos)

man -k (or **apropos**) shows a list of man pages containing a string.

```
paul@laika:~$ man -k syslog
lm-syslog-setup (8) - configure laptop mode to switch syslog.conf ...
logger (1) - a shell command interface to the syslog(3) ...
syslog-facility (8) - Setup and remove LOCALx facility for sysklogd
syslog.conf (5) - syslogd(8) configuration file
syslogd (8) - Linux system logging utilities.
syslogd-listfiles (8) - list system logfiles
```

7.5. whatis

To see just the description of a manual page, use **whatis** followed by a string.

```
paul@u810:~$ whatis route
route (8) - show / manipulate the IP routing table
```

7.6. whereis

The location of a manpage can be revealed with **whereis**.

```
paul@laika:~$ whereis -m whois
whois: /usr/share/man/man1/whois.1.gz
```

This file is directly readable by **man**.

```
paul@laika:~$ man /usr/share/man/man1/whois.1.gz
```

7.7. man sections

By now you will have noticed the numbers between the round brackets. **man man** will explain to you that these are section numbers. Executable programs and shell commands reside in section one.

```
1 Executable programs or shell commands
2 System calls (functions provided by the kernel)
3 Library calls (functions within program libraries)
4 Special files (usually found in /dev)
5 File formats and conventions eg /etc/passwd
6 Games
7 Miscellaneous (including macro packages and conventions), e.g. man(7)
8 System administration commands (usually only for root)
9 Kernel routines [Non standard]
```

7.8. man \$section \$file

Therefor, when referring to the man page of the passwd command, you will see it written as **passwd(1)**; when referring to the **passwd file**, you will see it written as **passwd(5)**. The screenshot explains how to open the man page in the correct section.

```
[paul@RHEL52 ~]$ man passwd      # opens the first manual found
[paul@RHEL52 ~]$ man 5 passwd    # opens a page from section 5
```

7.9. man man

If you want to know more about **man**, then Read The Fantastic Manual (RTFM).

Unfortunately, manual pages do not have the answer to everything...

```
paul@laika:~$ man woman
No manual entry for woman
```

7.10. mandb

Should you be convinced that a man page exists, but you can't access it, then try running **mandb** on Debian/Mint.

```
root@laika:~# mandb
0 man subdirectories contained newer manual pages.
0 manual pages were added.
0 stray cats were added.
0 old database entries were purged.
```

Or run **makewhatis** on CentOS/Redhat.

```
[root@centos65 ~]# apropos scsi
scsi: nothing appropriate
[root@centos65 ~]# makewhatis
[root@centos65 ~]# apropos scsi
hpsa          (4) - HP Smart Array SCSI driver
lsscsi        (8) - list SCSI devices (or hosts) and their attributes
sd            (4) - Driver for SCSI Disk Drives
st            (4) - SCSI tape device
```

Chapter 8. working with directories

This module is a brief overview of the most common commands to work with directories: **pwd**, **cd**, **ls**, **mkdir** and **rmdir**. These commands are available on any Linux (or Unix) system.

This module also discusses **absolute** and **relative paths** and **path completion** in the **bash** shell.

8.1. pwd

The **you are here** sign can be displayed with the **pwd** command (Print Working Directory). Go ahead, try it: Open a command line interface (also called a terminal, console or xterm) and type **pwd**. The tool displays your **current directory**.

```
paul@debian8:~$ pwd
/home/paul
```

8.2. cd

You can change your current directory with the **cd** command (Change Directory).

```
paul@debian8$ cd /etc
paul@debian8$ pwd
/etc
paul@debian8$ cd /bin
paul@debian8$ pwd
/bin
paul@debian8$ cd /home/paul/
paul@debian8$ pwd
/home/paul
```

8.2.1. cd ~

The **cd** is also a shortcut to get back into your home directory. Just typing **cd** without a target directory, will put you in your home directory. Typing **cd ~** has the same effect.

```
paul@debian8$ cd /etc
paul@debian8$ pwd
/etc
paul@debian8$ cd
paul@debian8$ pwd
/home/paul
paul@debian8$ cd ~
paul@debian8$ pwd
/home/paul
```

8.2.2. cd ..

To go to the **parent directory** (the one just above your current directory in the directory tree), type **cd ..**.

```
paul@debian8$ pwd
/usr/share/games
paul@debian8$ cd ..
paul@debian8$ pwd
/usr/share
```

*To stay in the current directory, type **cd .** ;-)* We will see useful use of the **.** character representing the current directory later.

8.2.3. `cd` -

Another useful shortcut with `cd` is to just type `cd` - to go to the previous directory.

```
paul@debian8$ pwd
/home/paul
paul@debian8$ cd /etc
paul@debian8$ pwd
/etc
paul@debian8$ cd -
/home/paul
paul@debian8$ cd -
/etc
```

8.3. absolute and relative paths

You should be aware of **absolute** and **relative** paths in the file tree. When you type a path starting with a **slash** (/), then the **root** of the file tree is assumed. If you don't start your path with a slash, then the current directory is the assumed starting point.

The screenshot below first shows the current directory **/home/paul**. From within this directory, you have to type `cd /home` instead of `cd home` to go to the **/home** directory.

```
paul@debian8$ pwd
/home/paul
paul@debian8$ cd home
bash: cd: home: No such file or directory
paul@debian8$ cd /home
paul@debian8$ pwd
/home
```

When inside **/home**, you have to type `cd paul` instead of `cd /paul` to enter the subdirectory **paul** of the current directory **/home**.

```
paul@debian8$ pwd
/home
paul@debian8$ cd /paul
bash: cd: /paul: No such file or directory
paul@debian8$ cd paul
paul@debian8$ pwd
/home/paul
```

In case your current directory is the **root directory** /, then both `cd /home` and `cd home` will get you in the **/home** directory.

```
paul@debian8$ pwd
/
paul@debian8$ cd home
paul@debian8$ pwd
/home
paul@debian8$ cd /
paul@debian8$ cd /home
paul@debian8$ pwd
/home
```

This was the last screenshot with `pwd` statements. From now on, the current directory will often be displayed in the prompt. Later in this book we will explain how the shell variable **\$PS1** can be configured to show this.

8.4. path completion

The **tab** key can help you in typing a path without errors. Typing **cd /et** followed by the **tab** key will expand the command line to **cd /etc/**. When typing **cd /Et** followed by the **tab** key, nothing will happen because you typed the wrong **path** (upper case E).

You will need fewer key strokes when using the **tab** key, and you will be sure your typed **path** is correct!

8.5. ls

You can list the contents of a directory with **ls**.

```
paul@debian8:~$ ls
allfiles.txt dmesg.txt  services  stuff  summer.txt
paul@debian8:~$
```

8.5.1. ls -a

A frequently used option with **ls** is **-a** to show all files. Showing all files means including the **hidden files**. When a file name on a Linux file system starts with a dot, it is considered a **hidden file** and it doesn't show up in regular file listings.

```
paul@debian8:~$ ls
allfiles.txt dmesg.txt  services  stuff  summer.txt
paul@debian8:~$ ls -a
.  allfiles.txt  .bash_profile  dmesg.txt  .lessht  stuff
.. .bash_history .bashrc        services   .ssh      summer.txt
paul@debian8:~$
```

8.5.2. ls -l

Many times you will be using options with **ls** to display the contents of the directory in different formats or to display different parts of the directory. Typing just **ls** gives you a list of files in the directory. Typing **ls -l** (that is a letter L, not the number 1) gives you a long listing.

```
paul@debian8:~$ ls -l
total 17296
-rw-r--r-- 1 paul paul 17584442 Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul   96650 Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul  19558 Sep 17 00:04 services
drwxr-xr-x 2 paul paul   4096 Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul     0 Sep 17 00:04 summer.txt
```

8.5.3. ls -lh

Another frequently used ls option is **-h**. It shows the numbers (file sizes) in a more human readable format. Also shown below is some variation in the way you can give the options to **ls**. We will explain the details of the output later in this book.

Note that we use the letter L as an option in this screenshot, not the number 1.

```
paul@debian8:~$ ls -l -h
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$ ls -lh
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$ ls -hl
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$ ls -h -l
total 17M
-rw-r--r-- 1 paul paul 17M Sep 17 00:03 allfiles.txt
-rw-r--r-- 1 paul paul 95K Sep 17 00:03 dmesg.txt
-rw-r--r-- 1 paul paul 20K Sep 17 00:04 services
drwxr-xr-x 2 paul paul 4.0K Sep 17 00:04 stuff
-rw-r--r-- 1 paul paul 0 Sep 17 00:04 summer.txt
paul@debian8:~$
```

8.6. mkdir

Walking around the Unix file tree is fun, but it is even more fun to create your own directories with **mkdir**. You have to give at least one parameter to **mkdir**, the name of the new directory to be created. Think before you type a leading `/`.

```
paul@debian8:~$ mkdir mydir
paul@debian8:~$ cd mydir
paul@debian8:~/mydir$ ls -al
total 8
drwxr-xr-x  2 paul paul 4096 Sep 17 00:07 .
drwxr-xr-x 48 paul paul 4096 Sep 17 00:07 ..
paul@debian8:~/mydir$ mkdir stuff
paul@debian8:~/mydir$ mkdir otherstuff
paul@debian8:~/mydir$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 otherstuff
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 stuff
paul@debian8:~/mydir$
```

8.6.1. mkdir -p

The following command will fail, because the **parent directory** of **threedirsdeep** does not exist.

```
paul@debian8:~$ mkdir mydir2/mysubdir2/threedirsdeep
mkdir: cannot create directory 'mydir2/mysubdir2/threedirsdeep': No such fi\
le or directory
```

When given the option **-p**, then **mkdir** will create **parent directories** as needed.

```
paul@debian8:~$ mkdir -p mydir2/mysubdir2/threedirsdeep
paul@debian8:~$ cd mydir2
paul@debian8:~/mydir2$ ls -l
total 4
drwxr-xr-x 3 paul paul 4096 Sep 17 00:11 mysubdir2
paul@debian8:~/mydir2$ cd mysubdir2
paul@debian8:~/mydir2/mysubdir2$ ls -l
total 4
drwxr-xr-x 2 paul paul 4096 Sep 17 00:11 threedirsdeep
paul@debian8:~/mydir2/mysubdir2$ cd threedirsdeep/
paul@debian8:~/mydir2/mysubdir2/threedirsdeep$ pwd
/home/paul/mydir2/mysubdir2/threedirsdeep
```

8.7. rmdir

When a directory is empty, you can use **rmdir** to remove the directory.

```
paul@debian8:~/mydir$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 otherstuff
drwxr-xr-x 2 paul paul 4096 Sep 17 00:08 stuff
paul@debian8:~/mydir$ rmdir otherstuff
paul@debian8:~/mydir$ cd ..
paul@debian8:~$ rmdir mydir
rmdir: failed to remove 'mydir': Directory not empty
paul@debian8:~$ rmdir mydir/stuff
paul@debian8:~$ rmdir mydir
paul@debian8:~$
```

8.7.1. **rmdir -p**

And similar to the **mkdir -p** option, you can also use **rmdir** to recursively remove directories.

```
paul@debian8:~$ mkdir -p test42/subdir  
paul@debian8:~$ rmdir -p test42/subdir  
paul@debian8:~$
```

8.8. practice: working with directories

1. Display your current directory.
2. Change to the /etc directory.
3. Now change to your home directory using only three key presses.
4. Change to the /boot/grub directory using only eleven key presses.
5. Go to the parent directory of the current directory.
6. Go to the root directory.
7. List the contents of the root directory.
8. List a long listing of the root directory.
9. Stay where you are, and list the contents of /etc.
10. Stay where you are, and list the contents of /bin and /sbin.
11. Stay where you are, and list the contents of ~.
12. List all the files (including hidden files) in your home directory.
13. List the files in /boot in a human readable format.
14. Create a directory testdir in your home directory.
15. Change to the /etc directory, stay here and create a directory newdir in your home directory.
16. Create in one command the directories ~/dir1/dir2/dir3 (dir3 is a subdirectory from dir2, and dir2 is a subdirectory from dir1).
17. Remove the directory testdir.
18. If time permits (or if you are waiting for other students to finish this practice), use and understand **pushd** and **popd**. Use the man page of **bash** to find information about these commands.

8.9. solution: working with directories

1. Display your current directory.

```
pwd
```

2. Change to the /etc directory.

```
cd /etc
```

3. Now change to your home directory using only three key presses.

```
cd (and the enter key)
```

4. Change to the /boot/grub directory using only eleven key presses.

```
cd /boot/grub (use the tab key)
```

5. Go to the parent directory of the current directory.

```
cd .. (with space between cd and ..)
```

6. Go to the root directory.

```
cd /
```

7. List the contents of the root directory.

```
ls
```

8. List a long listing of the root directory.

```
ls -l
```

9. Stay where you are, and list the contents of /etc.

```
ls /etc
```

10. Stay where you are, and list the contents of /bin and /sbin.

```
ls /bin /sbin
```

11. Stay where you are, and list the contents of ~.

```
ls ~
```

12. List all the files (including hidden files) in your home directory.

```
ls -al ~
```

13. List the files in /boot in a human readable format.

```
ls -lh /boot
```

14. Create a directory testdir in your home directory.

```
mkdir ~/testdir
```

15. Change to the /etc directory, stay here and create a directory newdir in your home directory.

```
cd /etc ; mkdir ~/newdir
```

16. Create in one command the directories ~/dir1/dir2/dir3 (dir3 is a subdirectory from dir2, and dir2 is a subdirectory from dir1).

```
mkdir -p ~/dir1/dir2/dir3
```

17. Remove the directory testdir.

```
rmdir testdir
```

18. If time permits (or if you are waiting for other students to finish this practice), use and understand **pushd** and **popd**. Use the man page of **bash** to find information about these commands.

```
man bash          # opens the manual
/pushd            # searches for pushd
n                 # next (do this two/three times)
```

The Bash shell has two built-in commands called **pushd** and **popd**. Both commands work with a common stack of previous directories. Pushd adds a directory to the stack and changes to a new current directory, popd removes a directory from the stack and sets the current directory.

```
paul@debian7:/etc$ cd /bin
paul@debian7:/bin$ pushd /lib
/lib /bin
paul@debian7:/lib$ pushd /proc
/proc /lib /bin
paul@debian7:/proc$ popd
/lib /bin
paul@debian7:/lib$ popd
/bin
```

Chapter 9. working with files

In this chapter we learn how to recognise, create, remove, copy and move files using commands like **file**, **touch**, **rm**, **cp**, **mv** and **rename**.

9.1. all files are case sensitive

Files on Linux (or any Unix) are **case sensitive**. This means that **FILE1** is different from **file1**, and **/etc/hosts** is different from **/etc/Hosts** (the latter one does not exist on a typical Linux computer).

This screenshot shows the difference between two files, one with upper case **W**, the other with lower case **w**.

```
paul@laika:~/Linux$ ls
winter.txt  Winter.txt
paul@laika:~/Linux$ cat winter.txt
It is cold.
paul@laika:~/Linux$ cat Winter.txt
It is very cold!
```

9.2. everything is a file

A **directory** is a special kind of **file**, but it is still a (case sensitive!) **file**. Each terminal window (for example **/dev/pts/4**), any hard disk or partition (for example **/dev/sdb1**) and any process are all represented somewhere in the **file system** as a **file**. It will become clear throughout this course that everything on Linux is a **file**.

9.3. file

The **file** utility determines the file type. Linux does not use extensions to determine the file type. The command line does not care whether a file ends in **.txt** or **.pdf**. As a system administrator, you should use the **file** command to determine the file type. Here are some examples on a typical Linux system.

```
paul@laika:~$ file pic33.png
pic33.png: PNG image data, 3840 x 1200, 8-bit/color RGBA, non-interlaced
paul@laika:~$ file /etc/passwd
/etc/passwd: ASCII text
paul@laika:~$ file HelloWorld.c
HelloWorld.c: ASCII C program text
```

The **file** command uses a magic file that contains patterns to recognise file types. The magic file is located in **/usr/share/file/magic**. Type **man 5 magic** for more information.

It is interesting to point out **file -s** for special files like those in **/dev** and **/proc**.

```
root@debian6~# file /dev/sda
/dev/sda: block special
root@debian6~# file -s /dev/sda
/dev/sda: x86 boot sector; partition 1: ID=0x83, active, starthead...
root@debian6~# file /proc/cpuinfo
/proc/cpuinfo: empty
root@debian6~# file -s /proc/cpuinfo
/proc/cpuinfo: ASCII C++ program text
```

9.4. touch

9.4.1. create an empty file

One easy way to create an empty file is with **touch**. (We will see many other ways for creating files later in this book.)

This screenshot starts with an empty directory, creates two files with **touch** and the lists those files.

```
paul@debian7:~$ ls -l
total 0
paul@debian7:~$ touch file42
paul@debian7:~$ touch file33
paul@debian7:~$ ls -l
total 0
-rw-r--r-- 1 paul paul 0 Oct 15 08:57 file33
-rw-r--r-- 1 paul paul 0 Oct 15 08:56 file42
paul@debian7:~$
```

9.4.2. touch -t

The **touch** command can set some properties while creating empty files. Can you determine what is set by looking at the next screenshot? If not, check the manual for **touch**.

```
paul@debian7:~$ touch -t 200505050000 SinkoDeMayo
paul@debian7:~$ touch -t 130207111630 BigBattle.txt
paul@debian7:~$ ls -l
total 0
-rw-r--r-- 1 paul paul 0 Jul 11 1302 BigBattle.txt
-rw-r--r-- 1 paul paul 0 Oct 15 08:57 file33
-rw-r--r-- 1 paul paul 0 Oct 15 08:56 file42
-rw-r--r-- 1 paul paul 0 May 5 2005 SinkoDeMayo
paul@debian7:~$
```

9.5. rm

9.5.1. remove forever

When you no longer need a file, use **rm** to remove it. Unlike some graphical user interfaces, the command line in general does not have a **waste bin** or **trash can** to recover files. When you use **rm** to remove a file, the file is gone. Therefore, be careful when removing files!

```
paul@debian7:~$ ls
BigBattle.txt file33 file42 SinkoDeMayo
paul@debian7:~$ rm BigBattle.txt
paul@debian7:~$ ls
file33 file42 SinkoDeMayo
paul@debian7:~$
```

9.5.2. rm -i

To prevent yourself from accidentally removing a file, you can type **rm -i**.

```
paul@debian7:~$ ls
file33 file42 SinkoDeMayo
paul@debian7:~$ rm -i file33
rm: remove regular empty file `file33'? yes
paul@debian7:~$ rm -i SinkoDeMayo
rm: remove regular empty file `SinkoDeMayo'? n
paul@debian7:~$ ls
file42 SinkoDeMayo
paul@debian7:~$
```

9.5.3. rm -rf

By default, **rm -r** will not remove non-empty directories. However **rm** accepts several options that will allow you to remove any directory. The **rm -rf** statement is famous because it will erase anything (providing that you have the permissions to do so). When you are logged on as root, be very careful with **rm -rf** (the **f** means **force** and the **r** means **recursive**) since being root implies that permissions don't apply to you. You can literally erase your entire file system by accident.

```
paul@debian7:~$ mkdir test
paul@debian7:~$ rm test
rm: cannot remove `test': Is a directory
paul@debian7:~$ rm -rf test
paul@debian7:~$ ls test
ls: cannot access test: No such file or directory
paul@debian7:~$
```

9.6. cp

9.6.1. copy one file

To copy a file, use **cp** with a source and a target argument.

```
paul@debian7:~$ ls
file42 SinkoDeMayo
paul@debian7:~$ cp file42 file42.copy
paul@debian7:~$ ls
file42 file42.copy SinkoDeMayo
```

9.6.2. copy to another directory

If the target is a directory, then the source files are copied to that target directory.

```
paul@debian7:~$ mkdir dir42
paul@debian7:~$ cp SinkoDeMayo dir42
paul@debian7:~$ ls dir42/
SinkoDeMayo
```

9.6.3. cp -r

To copy complete directories, use **cp -r** (the **-r** option forces **recursive** copying of all files in all subdirectories).

```
paul@debian7:~$ ls
dir42 file42 file42.copy SinkoDeMayo
paul@debian7:~$ cp -r dir42/ dir33
paul@debian7:~$ ls
dir33 dir42 file42 file42.copy SinkoDeMayo
paul@debian7:~$ ls dir33/
SinkoDeMayo
```

9.6.4. copy multiple files to directory

You can also use **cp** to copy multiple files into a directory. In this case, the last argument (a.k.a. the target) must be a directory.

```
paul@debian7:~$ cp file42 file42.copy SinkoDeMayo dir42/
paul@debian7:~$ ls dir42/
file42 file42.copy SinkoDeMayo
```

9.6.5. cp -i

To prevent **cp** from overwriting existing files, use the **-i** (for interactive) option.

```
paul@debian7:~$ cp SinkoDeMayo file42
paul@debian7:~$ cp SinkoDeMayo file42
paul@debian7:~$ cp -i SinkoDeMayo file42
cp: overwrite `file42'? n
paul@debian7:~$
```

9.7. mv

9.7.1. rename files with mv

Use **mv** to rename a file or to move the file to another directory.

```
paul@debian7:~$ ls
dir33 dir42 file42 file42.copy SinkoDeMayo
paul@debian7:~$ mv file42 file33
paul@debian7:~$ ls
dir33 dir42 file33 file42.copy SinkoDeMayo
paul@debian7:~$
```

When you need to rename only one file then **mv** is the preferred command to use.

9.7.2. rename directories with mv

The same **mv** command can be used to rename directories.

```
paul@debian7:~$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 dir33
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 dir42
-rw-r--r-- 1 paul paul  0 Oct 15 09:38 file33
-rw-r--r-- 1 paul paul  0 Oct 15 09:16 file42.copy
-rw-r--r-- 1 paul paul  0 May  5 2005 SinkoDeMayo
paul@debian7:~$ mv dir33 backup
paul@debian7:~$ ls -l
total 8
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 backup
drwxr-xr-x 2 paul paul 4096 Oct 15 09:36 dir42
-rw-r--r-- 1 paul paul  0 Oct 15 09:38 file33
-rw-r--r-- 1 paul paul  0 Oct 15 09:16 file42.copy
-rw-r--r-- 1 paul paul  0 May  5 2005 SinkoDeMayo
paul@debian7:~$
```

9.7.3. mv -i

The **mv** also has a **-i** switch similar to **cp** and **rm**.

this screenshot shows that **mv -i** will ask permission to overwrite an existing file.

```
paul@debian7:~$ mv -i file33 SinkoDeMayo
mv: overwrite `SinkoDeMayo'? no
paul@debian7:~$
```


9.8. rename

9.8.1. about rename

The **rename** command is one of the rare occasions where the Linux Fundamentals book has to make a distinction between Linux distributions. Almost every command in the **Fundamentals** part of this book works on almost every Linux computer. But **rename** is different.

Try to use **mv** whenever you need to rename only a couple of files.

9.8.2. rename on Debian/Ubuntu

The **rename** command on Debian uses regular expressions (regular expression or short regex are explained in a later chapter) to rename many files at once.

Below a **rename** example that switches all occurrences of **txt** to **png** for all file names ending in **.txt**.

```
paul@debian7:~/test42$ ls
abc.txt file33.txt file42.txt
paul@debian7:~/test42$ rename 's/\.txt /\.png/' *.txt
paul@debian7:~/test42$ ls
abc.png file33.png file42.png
```

This second example switches all (first) occurrences of **file** into **document** for all file names ending in **.png**.

```
paul@debian7:~/test42$ ls
abc.png file33.png file42.png
paul@debian7:~/test42$ rename 's/file/document/' *.png
paul@debian7:~/test42$ ls
abc.png document33.png document42.png
paul@debian7:~/test42$
```

9.8.3. rename on CentOS/RHEL/Fedora

On Red Hat Enterprise Linux, the syntax of **rename** is a bit different. The first example below renames all ***.conf** files replacing any occurrence of **.conf** with **.backup**.

```
[paul@centos7 ~]$ touch one.conf two.conf three.conf
[paul@centos7 ~]$ rename .conf .backup *.conf
[paul@centos7 ~]$ ls
one.backup three.backup two.backup
[paul@centos7 ~]$
```

The second example renames all **(*)** files replacing **one** with **ONE**.

```
[paul@centos7 ~]$ ls
one.backup three.backup two.backup
[paul@centos7 ~]$ rename one ONE *
[paul@centos7 ~]$ ls
ONE.backup three.backup two.backup
[paul@centos7 ~]$
```

9.9. practice: working with files

1. List the files in the /bin directory
2. Display the type of file of /bin/cat, /etc/passwd and /usr/bin/passwd.
- 3a. Download wolf.jpg and LinuxFun.pdf from <http://linux-training.be> (wget <http://linux-training.be/files/studentfiles/wolf.jpg> and wget <http://linux-training.be/files/books/LinuxFun.pdf>)

```
wget http://linux-training.be/files/studentfiles/wolf.jpg
wget http://linux-training.be/files/studentfiles/wolf.png
wget http://linux-training.be/files/books/LinuxFun.pdf
```
- 3b. Display the type of file of wolf.jpg and LinuxFun.pdf
- 3c. Rename wolf.jpg to wolf.pdf (use mv).
- 3d. Display the type of file of wolf.pdf and LinuxFun.pdf.
4. Create a directory ~/touched and enter it.
5. Create the files today.txt and yesterday.txt in touched.
6. Change the date on yesterday.txt to match yesterday's date.
7. Copy yesterday.txt to copy.yesterday.txt
8. Rename copy.yesterday.txt to kim
9. Create a directory called ~/testbackup and copy all files from ~/touched into it.
10. Use one command to remove the directory ~/testbackup and all files into it.
11. Create a directory ~/etcbackup and copy all *.conf files from /etc into it. Did you include all subdirectories of /etc ?
12. Use rename to rename all *.conf files to *.backup . (if you have more than one distro available, try it on all!)

9.10. solution: working with files

1. List the files in the /bin directory

```
ls /bin
```

2. Display the type of file of /bin/cat, /etc/passwd and /usr/bin/passwd.

```
file /bin/cat /etc/passwd /usr/bin/passwd
```

- 3a. Download wolf.jpg and LinuxFun.pdf from <http://linux-training.be> (wget <http://linux-training.be/files/studentfiles/wolf.jpg> and wget <http://linux-training.be/files/books/LinuxFun.pdf>)

```
wget http://linux-training.be/files/studentfiles/wolf.jpg
wget http://linux-training.be/files/studentfiles/wolf.png
wget http://linux-training.be/files/books/LinuxFun.pdf
```

- 3b. Display the type of file of wolf.jpg and LinuxFun.pdf

```
file wolf.jpg LinuxFun.pdf
```

- 3c. Rename wolf.jpg to wolf.pdf (use mv).

```
mv wolf.jpg wolf.pdf
```

- 3d. Display the type of file of wolf.pdf and LinuxFun.pdf.

```
file wolf.pdf LinuxFun.pdf
```

4. Create a directory ~/touched and enter it.

```
mkdir ~/touched ; cd ~/touched
```

5. Create the files today.txt and yesterday.txt in touched.

```
touch today.txt yesterday.txt
```

6. Change the date on yesterday.txt to match yesterday's date.

```
touch -t 200810251405 yesterday.txt (substitute 20081025 with yesterday)
```

7. Copy yesterday.txt to copy.yesterday.txt

```
cp yesterday.txt copy.yesterday.txt
```

8. Rename copy.yesterday.txt to kim

```
mv copy.yesterday.txt kim
```

9. Create a directory called ~/testbackup and copy all files from ~/touched into it.

```
mkdir ~/testbackup ; cp -r ~/touched ~/testbackup/
```

10. Use one command to remove the directory ~/testbackup and all files into it.

```
rm -rf ~/testbackup
```

11. Create a directory ~/etcbackup and copy all *.conf files from /etc into it. Did you include all subdirectories of /etc ?

```
cp -r /etc/*.conf ~/etcbackup
```

Only *.conf files that are directly in /etc/ are copied.

12. Use rename to rename all *.conf files to *.backup . (if you have more than one distro available, try it on all!)

On RHEL: touch 1.conf 2.conf ; rename conf backup *.conf

On Debian: touch 1.conf 2.conf ; rename 's/conf/backup/' *.conf

Chapter 10. working with file contents

In this chapter we will look at the contents of **text files** with **head**, **tail**, **cat**, **tac**, **more**, **less** and **strings**.

We will also get a glimpse of the possibilities of tools like **cat** on the command line.

10.1. head

You can use **head** to display the first ten lines of a file.

```
paul@debian7~$ head /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
root@debian7~#
```

The **head** command can also display the first **n** lines of a file.

```
paul@debian7~$ head -4 /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
paul@debian7~$
```

And **head** can also display the first **n** bytes.

```
paul@debian7~$ head -c14 /etc/passwd
root:x:0:0:roopaul@debian7~$
```

10.2. tail

Similar to **head**, the **tail** command will display the last ten lines of a file.

```
paul@debian7~$ tail /etc/services
vboxd      20012/udp
binkp      24554/tcp      # binkp fidonet protocol
asp        27374/tcp      # Address Search Protocol
asp        27374/udp
csync2     30865/tcp      # cluster synchronization tool
dirproxy   57000/tcp      # Detachable IRC Proxy
tfido      60177/tcp      # fidonet EMSI over telnet
fido       60179/tcp      # fidonet EMSI over TCP

# Local services
paul@debian7~$
```

You can give **tail** the number of lines you want to see.

```
paul@debian7~$ tail -3 /etc/services
fido       60179/tcp      # fidonet EMSI over TCP

# Local services
paul@debian7~$
```

The **tail** command has other useful options, some of which we will use during this course.

10.3. cat

The **cat** command is one of the most universal tools, yet all it does is copy **standard input** to **standard output**. In combination with the shell this can be very powerful and diverse. Some examples will give a glimpse into the possibilities. The first example is simple, you can use **cat** to display a file on the screen. If the file is longer than the screen, it will scroll to the end.

```
paul@debian8:~$ cat /etc/resolv.conf
domain linux-training.be
search linux-training.be
nameserver 192.168.1.42
```

10.3.1. concatenate

cat is short for **concatenate**. One of the basic uses of **cat** is to concatenate files into a bigger (or complete) file.

```
paul@debian8:~$ echo one >part1
paul@debian8:~$ echo two >part2
paul@debian8:~$ echo three >part3
paul@debian8:~$ cat part1
one
paul@debian8:~$ cat part2
two
paul@debian8:~$ cat part3
three
paul@debian8:~$ cat part1 part2 part3
one
two
three
paul@debian8:~$ cat part1 part2 part3 >all
paul@debian8:~$ cat all
one
two
three
paul@debian8:~$
```

10.3.2. create files

You can use **cat** to create flat text files. Type the **cat > winter.txt** command as shown in the screenshot below. Then type one or more lines, finishing each line with the enter key. After the last line, type and hold the Control (Ctrl) key and press d.

```
paul@debian8:~$ cat > winter.txt
It is very cold today!
paul@debian8:~$ cat winter.txt
It is very cold today!
paul@debian8:~$
```

The **Ctrl d** key combination will send an **EOF** (End of File) to the running process ending the **cat** command.

10.3.3. custom end marker

You can choose an end marker for **cat** with **<<** as is shown in this screenshot. This construction is called a **here directive** and will end the **cat** command.

```
paul@debian8:~$ cat > hot.txt <<stop
> It is hot today!
> Yes it is summer.
> stop
paul@debian8:~$ cat hot.txt
It is hot today!
Yes it is summer.
paul@debian8:~$
```

10.3.4. copy files

In the third example you will see that **cat** can be used to copy files. We will explain in detail what happens here in the bash shell chapter.

```
paul@debian8:~$ cat winter.txt
It is very cold today!
paul@debian8:~$ cat winter.txt > cold.txt
paul@debian8:~$ cat cold.txt
It is very cold today!
paul@debian8:~$
```

10.4. tac

Just one example will show you the purpose of **tac** (cat backwards).

```
paul@debian8:~$ cat count
one
two
three
four
paul@debian8:~$ tac count
four
three
two
one
```


10.5. more and less

The **more** command is useful for displaying files that take up more than one screen. More will allow you to see the contents of the file page by page. Use the space bar to see the next page, or **q** to quit. Some people prefer the **less** command to **more**.

10.6. strings

With the **strings** command you can display readable ascii strings found in (binary) files. This example locates the **ls** binary then displays readable strings in the binary file (output is truncated).

```
paul@laika:~$ which ls
/bin/ls
paul@laika:~$ strings /bin/ls
/lib/ld-linux.so.2
librt.so.1
__gmon_start__
_Jv_RegisterClasses
clock_gettime
libacl.so.1
...
```

10.7. practice: file contents

1. Display the first 12 lines of **/etc/services**.
2. Display the last line of **/etc/passwd**.
3. Use **cat** to create a file named **count.txt** that looks like this:

```
One  
Two  
Three  
Four  
Five
```

4. Use **cp** to make a backup of this file to **cnt.txt**.
5. Use **cat** to make a backup of this file to **catcnt.txt**.
6. Display **catcnt.txt**, but with all lines in reverse order (the last line first).
7. Use **more** to display **/etc/services**.
8. Display the readable character strings from the **/usr/bin/passwd** command.
9. Use **ls** to find the biggest file in **/etc**.
10. Open two terminal windows (or tabs) and make sure you are in the same directory in both. Type **echo this is the first line > tailing.txt** in the first terminal, then issue **tail -f tailing.txt** in the second terminal. Now go back to the first terminal and type **echo This is another line >> tailing.txt** (note the double >>), verify that the **tail -f** in the second terminal shows both lines. Stop the **tail -f** with **Ctrl-C**.
11. Use **cat** to create a file named **tailing.txt** that contains the contents of **tailing.txt** followed by the contents of **/etc/passwd**.
12. Use **cat** to create a file named **tailing.txt** that contains the contents of **tailing.txt** preceded by the contents of **/etc/passwd**.

10.8. solution: file contents

1. Display the first 12 lines of **/etc/services**.

```
head -12 /etc/services
```

2. Display the last line of **/etc/passwd**.

```
tail -1 /etc/passwd
```

3. Use **cat** to create a file named **count.txt** that looks like this:

```
cat > count.txt
One
Two
Three
Four
Five (followed by Ctrl-d)
```

4. Use **cp** to make a backup of this file to **cnt.txt**.

```
cp count.txt cnt.txt
```

5. Use **cat** to make a backup of this file to **catcnt.txt**.

```
cat count.txt > catcnt.txt
```

6. Display **catcnt.txt**, but with all lines in reverse order (the last line first).

```
tac catcnt.txt
```

7. Use **more** to display **/etc/services**.

```
more /etc/services
```

8. Display the readable character strings from the **/usr/bin/passwd** command.

```
strings /usr/bin/passwd
```

9. Use **ls** to find the biggest file in **/etc**.

```
ls -lrS /etc
```

10. Open two terminal windows (or tabs) and make sure you are in the same directory in both. Type **echo this is the first line > tailing.txt** in the first terminal, then issue **tail -f tailing.txt** in the second terminal. Now go back to the first terminal and type **echo This is another line >> tailing.txt** (note the double >>), verify that the **tail -f** in the second terminal shows both lines. Stop the **tail -f** with **Ctrl-C**.

11. Use **cat** to create a file named **tailing.txt** that contains the contents of **tailing.txt** followed by the contents of **/etc/passwd**.

```
cat /etc/passwd >> tailing.txt
```

12. Use **cat** to create a file named **tailing.txt** that contains the contents of **tailing.txt** preceded by the contents of **/etc/passwd**.

```
mv tailing.txt tmp.txt ; cat /etc/passwd tmp.txt > tailing.txt
```

Chapter 11. the Linux file tree

This chapter takes a look at the most common directories in the **Linux file tree**. It also shows that on Unix everything is a file.

11.1. filesystem hierarchy standard

Many Linux distributions partially follow the **Filesystem Hierarchy Standard**. The **FHS** may help make more Unix/Linux file system trees conform better in the future. The **FHS** is available online at <http://www.pathname.com/fhs/> where we read: "The filesystem hierarchy standard has been designed to be used by Unix distribution developers, package developers, and system implementers. However, it is primarily intended to be a reference and is not a tutorial on how to manage a Unix filesystem or directory hierarchy."

11.2. man hier

There are some differences in the filesystems between **Linux distributions**. For help about your machine, enter **man hier** to find information about the file system hierarchy. This manual will explain the directory structure on your computer.

11.3. the root directory /

All Linux systems have a directory structure that starts at the **root directory**. The root directory is represented by a **forward slash**, like this: /. Everything that exists on your Linux system can be found below this root directory. Let's take a brief look at the contents of the root directory.

```
[paul@RHELv4u3 ~]$ ls /  
bin  dev  home  media  mnt  proc  sbin  srv  tftpboot  usr  
boot  etc  lib  misc  opt  root  selinux  sys  tmp  var
```

11.4. binary directories

Binaries are files that contain compiled source code (or machine code). Binaries can be **executed** on the computer. Sometimes binaries are called **executables**.

11.4.1. /bin

The **/bin** directory contains **binaries** for use by all users. According to the FHS the **/bin** directory should contain **/bin/cat** and **/bin/date** (among others).

In the screenshot below you see common Unix/Linux commands like `cat`, `cp`, `cpio`, `date`, `dd`, `echo`, `grep`, and so on. Many of these will be covered in this book.

```
paul@laika:~$ ls /bin
archdetect      egrep            mt               setupcon
autopartition   false            mt-gnu           sh
bash            fgconsole       mv               sh.distrib
bunzip2         fgrep            nano             sleep
bzip2           fuser           nc               stralign
bzip2           fusermount      nc.traditional  stty
bzip2           get_mounthelp   netcat           su
bzip2           grep            netstat          sync
bzip2           gunzip           ntfs-3g          sysfs
bzip2           gzexe           ntfs-3g.probe   tailf
bzip2           gzip            parted_devices  tar
bzip2           hostname        parted_server   tempfile
bzip2           hw-detect       partman          touch
bzip2           ip              partman-commit  true
bzip2           kbd_mode        perform_recipe  ulockmgr
bzip2           kill            pidof            umount
...
```

11.4.2. other /bin directories

You can find a **/bin subdirectory** in many other directories. A user named **serena** could put her own programs in **/home/serena/bin**.

Some applications, often when installed directly from source will put themselves in **/opt**. A **samba server** installation can use **/opt/samba/bin** to store its binaries.

11.4.3. /sbin

/sbin contains binaries to configure the operating system. Many of the **system binaries** require **root** privilege to perform certain tasks.

Below a screenshot containing **system binaries** to change the ip address, partition a disk and create an ext4 file system.

```
paul@ubu1010:~$ ls -l /sbin/ifconfig /sbin/fdisk /sbin/mkfs.ext4
-rwxr-xr-x 1 root root 97172 2011-02-02 09:56 /sbin/fdisk
-rwxr-xr-x 1 root root 65708 2010-07-02 09:27 /sbin/ifconfig
-rwxr-xr-x 5 root root 55140 2010-08-18 18:01 /sbin/mkfs.ext4
```

11.4.4. /lib

Binaries found in **/bin** and **/sbin** often use **shared libraries** located in **/lib**. Below is a screenshot of the partial contents of **/lib**.

```
paul@laika:~$ ls /lib/libc*
/lib/libc-2.5.so      /lib/libcfont.so.0.0.0  /lib/libcom_err.so.2.1
/lib/libcap.so.1      /lib/libcidn-2.5.so     /lib/libconsole.so.0
/lib/libcap.so.1.10   /lib/libcidn.so.1       /lib/libconsole.so.0.0.0
/lib/libcfont.so.0    /lib/libcom_err.so.2    /lib/libcrypt-2.5.so
```

/lib/modules

Typically, the **Linux kernel** loads kernel modules from **/lib/modules/\$kernel-version/**. This directory is discussed in detail in the Linux kernel chapter.

/lib32 and /lib64

We currently are in a transition between **32-bit** and **64-bit** systems. Therefore, you may encounter directories named **/lib32** and **/lib64** which clarify the register size used during compilation time of the libraries. A 64-bit computer may have some 32-bit binaries and libraries for compatibility with legacy applications. This screenshot uses the **file** utility to demonstrate the difference.

```
paul@laika:~$ file /lib32/libc-2.5.so
/lib32/libc-2.5.so: ELF 32-bit LSB shared object, Intel 80386, \
version 1 (SYSV), for GNU/Linux 2.6.0, stripped
paul@laika:~$ file /lib64/libcap.so.1.10
/lib64/libcap.so.1.10: ELF 64-bit LSB shared object, AMD x86-64, \
version 1 (SYSV), stripped
```

The ELF (**Executable and Linkable Format**) is used in almost every Unix-like operating system since **System V**.

11.4.5. /opt

The purpose of **/opt** is to store **optional** software. In many cases this is software from outside the distribution repository. You may find an empty **/opt** directory on many systems.

A large package can install all its files in **/bin**, **/lib**, **/etc** subdirectories within **/opt/\$packagename/**. If for example the package is called **wp**, then it installs in **/opt/wp**, putting binaries in **/opt/wp/bin** and manpages in **/opt/wp/man**.

11.5. configuration directories

11.5.1. /boot

The **/boot** directory contains all files needed to boot the computer. These files don't change very often. On Linux systems you typically find the **/boot/grub** directory here. **/boot/grub** contains **/boot/grub/grub.cfg** (older systems may still have **/boot/grub/grub.conf**) which defines the boot menu that is displayed before the kernel starts.

11.5.2. /etc

All of the machine-specific **configuration files** should be located in **/etc**. Historically **/etc** stood for **etcetera**, today people often use the **Editable Text Configuration** backronym.

Many times the name of a configuration files is the same as the application, daemon, or protocol with **.conf** added as the extension.

```
paul@laika:~$ ls /etc/*.conf
/etc/adduser.conf      /etc/ld.so.conf      /etc/scrollkeeper.conf
/etc/brltty.conf       /etc/lftp.conf       /etc/sysctl.conf
/etc/ccertificates.conf /etc/libao.conf      /etc/syslog.conf
/etc/cvs-cron.conf     /etc/logrotate.conf  /etc/ucf.conf
/etc/ddclient.conf     /etc/ltrace.conf     /etc/uniconf.conf
/etc/debconf.conf      /etc/mke2fs.conf     /etc/updatedb.conf
/etc/deluser.conf      /etc/netscsid.conf   /etc/usplash.conf
/etc/fdmount.conf      /etc/nsswitch.conf   /etc/uswsusp.conf
/etc/hdparm.conf       /etc/pam.conf         /etc/vnc.conf
/etc/host.conf         /etc/pnm2ppa.conf    /etc/wodim.conf
/etc/inetd.conf        /etc/povray.conf     /etc/wvdial.conf
/etc/kernel-img.conf   /etc/resolv.conf
paul@laika:~$
```

There is much more to be found in **/etc**.

/etc/init.d/

A lot of Unix/Linux distributions have an **/etc/init.d** directory that contains scripts to start and stop **daemons**. This directory could disappear as Linux migrates to systems that replace the old **init** way of starting all **daemons**.

/etc/X11/

The graphical display (aka **X Window System** or just **X**) is driven by software from the X.org foundation. The configuration file for your graphical display is **/etc/X11/xorg.conf**.

/etc/skel/

The **skeleton** directory **/etc/skel** is copied to the home directory of a newly created user. It usually contains hidden files like a **.bashrc** script.

/etc/sysconfig/

This directory, which is not mentioned in the FHS, contains a lot of **Red Hat Enterprise Linux** configuration files. We will discuss some of them in greater detail. The screenshot below is the **/etc/sysconfig** directory from RHELv4u4 with everything installed.


```
paul@RHELv4u4:~$ ls /etc/sysconfig/
apmd          firstboot      irda           network        saslauthd
apm-scripts   grub           irqbalance     networking     selinux
authconfig    hidd           keyboard       ntpd           spamassassin
autofs        httpd          kudzu          openib.conf    squid
bluetooth     hwconf        lm_sensors     pand           syslog
clock         i18n          mouse          pcmcia         sys-config-sec
console       init           mouse.B        pgsql          sys-config-users
crond         installinfo   named          prelink        sys-logviewer
desktop       ipmi          netdump        rawdevices     tux
diskdump      iptables      netdump_id_dsa rhn            vncservers
dund          iptables-cfg  netdump_id_dsa.p samba          xinetd
paul@RHELv4u4:~$
```

The file **/etc/sysconfig/firstboot** tells the Red Hat Setup Agent not to run at boot time. If you want to run the Red Hat Setup Agent at the next reboot, then simply remove this file, and run **chkconfig --level 5 firstboot on**. The Red Hat Setup Agent allows you to install the latest updates, create a user account, join the Red Hat Network and more. It will then create the **/etc/sysconfig/firstboot** file again.

```
paul@RHELv4u4:~$ cat /etc/sysconfig/firstboot
RUN_FIRSTBOOT=NO
```

The **/etc/sysconfig/harddisks** file contains some parameters to tune the hard disks. The file explains itself.

You can see hardware detected by **kudzu** in **/etc/sysconfig/hwconf**. Kudzu is software from Red Hat for automatic discovery and configuration of hardware.

The keyboard type and keymap table are set in the **/etc/sysconfig/keyboard** file. For more console keyboard information, check the manual pages of **keymaps(5)**, **dumpkeys(1)**, **loadkeys(1)** and the directory **/lib/kbd/keymaps/**.

```
root@RHELv4u4:/etc/sysconfig# cat keyboard
KEYBOARDTYPE="pc"
KEYTABLE="us"
```

We will discuss networking files in this directory in the networking chapter.

11.6. data directories

11.6.1. /home

Users can store personal or project data under **/home**. It is common (but not mandatory by the fhs) practice to name the users home directory after the user name in the format **/home/\$USERNAME**. For example:

```
paul@ubu606:~$ ls /home
geert  annik  sandra  paul  tom
```

Besides giving every user (or every project or group) a location to store personal files, the home directory of a user also serves as a location to store the user profile. A typical Unix user profile contains many hidden files (files whose file name starts with a dot). The hidden files of the Unix user profiles contain settings specific for that user.

```
paul@ubu606:~$ ls -d /home/paul/. *
/home/paul/.                /home/paul/.bash_profile  /home/paul/.ssh
/home/paul/..               /home/paul/.bashrc        /home/paul/.viminfo
/home/paul/.bash_history    /home/paul/.lessht
```

11.6.2. /root

On many systems **/root** is the default location for personal data and profile of the **root user**. If it does not exist by default, then some administrators create it.

11.6.3. /srv

You may use **/srv** for data that is **served by your system**. The FHS allows locating cvs, rsync, ftp and www data in this location. The FHS also approves administrative naming in /srv, like /srv/project55/ftp and /srv/sales/www.

On Sun Solaris (or Oracle Solaris) **/export** is used for this purpose.

11.6.4. /media

The **/media** directory serves as a mount point for **removable media devices** such as CD-ROM's, digital cameras, and various usb-attached devices. Since **/media** is rather new in the Unix world, you could very well encounter systems running without this directory. Solaris 9 does not have it, Solaris 10 does. Most Linux distributions today mount all removable media in **/media**.

```
paul@debian5:~$ ls /media/
cdrom  cdrom0  usbdisk
```

11.6.5. /mnt

The **/mnt** directory should be empty and should only be used for temporary mount points (according to the FHS).

Unix and Linux administrators used to create many directories here, like `/mnt/something/`. You likely will encounter many systems with more than one directory created and/or mounted inside `/mnt` to be used for various local and remote filesystems.

11.6.6. `/tmp`

Applications and users should use `/tmp` to store temporary data when needed. Data stored in `/tmp` may use either disk space or RAM. Both of which are managed by the operating system. Never use `/tmp` to store data that is important or which you wish to archive.

11.7. in memory directories

11.7.1. /dev

Device files in **/dev** appear to be ordinary files, but are not actually located on the hard disk. The **/dev** directory is populated with files as the kernel is recognising hardware.

common physical devices

Common hardware such as hard disk devices are represented by device files in **/dev**. Below a screenshot of SATA device files on a laptop and then IDE attached drives on a desktop. (The detailed meaning of these devices will be discussed later.)

```
#
# SATA or SCSI or USB
#
paul@laika:~$ ls /dev/sd*
/dev/sda /dev/sda1 /dev/sda2 /dev/sda3 /dev/sdb /dev/sdb1 /dev/sdb2

#
# IDE or ATAPI
#
paul@barry:~$ ls /dev/hd*
/dev/hda /dev/hda1 /dev/hda2 /dev/hdb /dev/hdb1 /dev/hdb2 /dev/hdc
```

Besides representing physical hardware, some device files are special. These special devices can be very useful.

/dev/tty and /dev/pts

For example, **/dev/tty1** represents a terminal or console attached to the system. (Don't break your head on the exact terminology of 'terminal' or 'console', what we mean here is a command line interface.) When typing commands in a terminal that is part of a graphical interface like Gnome or KDE, then your terminal will be represented as **/dev/pts/1** (1 can be another number).

/dev/null

On Linux you will find other special devices such as **/dev/null** which can be considered a black hole; it has unlimited storage, but nothing can be retrieved from it. Technically speaking, anything written to **/dev/null** will be discarded. **/dev/null** can be useful to discard unwanted output from commands. */dev/null is not a good location to store your backups ;-).*

11.7.2. /proc conversation with the kernel

/proc is another special directory, appearing to be ordinary files, but not taking up disk space. It is actually a view of the kernel, or better, what the kernel manages, and is a means to interact with it directly. **/proc** is a proc filesystem.

```
paul@RHELV4u4:~$ mount -t proc
```

```
none on /proc type proc (rw)
```

When listing the /proc directory you will see many numbers (on any Unix) and some interesting files (on Linux)

```
mul@laika:~$ ls /proc
1          2339  4724  5418  6587  7201      cmdline  mounts
10175     2523  4729  5421  6596  7204      cpuinfo   mtrr
10211     2783  4741  5658  6599  7206      crypto    net
10239     2975  4873  5661  6638  7214      devices   pagetypeinfo
141       29775  4874  5665  6652  7216      diskstats partitions
15045     29792  4878  5927  6719  7218      dma        sched_debug
1519      2997  4879  6      6736  7223      driver     scsi
1548      3      4881  6032  6737  7224      execdomains self
1551      30228  4882  6033  6755  7227      fb          slabinfo
1554      3069  5      6145  6762  7260      filesystems stat
1557      31422  5073  6298  6774  7267      fs          swaps
1606      3149  5147  6414  6816  7275      ide         sys
180       31507  5203  6418  6991  7282      interrupts sysrq-trigger
181       3189  5206  6419  6993  7298      iomem       sysvipc
182       3193  5228  6420  6996  7319      ioports     timer_list
18898     3246  5272  6421  7157  7330      irq         timer_stats
19799     3248  5291  6422  7163  7345      kallsyms    tty
19803     3253  5294  6423  7164  7513      kcore       uptime
19804     3372  5356  6424  7171  7525      key-users   version
1987      4      5370  6425  7175  7529      kmsg        version_signature
1989      42      5379  6426  7188  9964      loadavg     vmcore
2         45      5380  6430  7189  acpi       locks       vmnet
20845     4542  5412  6450  7191  asound     meminfo     vmstat
221       46      5414  6551  7192  buddyinfo misc         zoneinfo
2338     4704  5416  6568  7199  bus        modules
```

Let's investigate the file properties inside **/proc**. Looking at the date and time will display the current date and time showing the files are constantly updated (a view on the kernel).

```
paul@RHELv4u4:~$ date
Mon Jan 29 18:06:32 EST 2007
paul@RHELv4u4:~$ ls -al /proc/cpuinfo
-r--r--r-- 1 root root 0 Jan 29 18:06 /proc/cpuinfo
paul@RHELv4u4:~$
paul@RHELv4u4:~$ ...time passes...
paul@RHELv4u4:~$
paul@RHELv4u4:~$ date
Mon Jan 29 18:10:00 EST 2007
paul@RHELv4u4:~$ ls -al /proc/cpuinfo
-r--r--r-- 1 root root 0 Jan 29 18:10 /proc/cpuinfo
```

Most files in /proc are 0 bytes, yet they contain data--sometimes a lot of data. You can see this by executing cat on files like **/proc/cpuinfo**, which contains information about the CPU.

```
paul@RHELv4u4:~$ file /proc/cpuinfo
/proc/cpuinfo: empty
paul@RHELv4u4:~$ cat /proc/cpuinfo
processor       : 0
vendor_id      : AuthenticAMD
cpu family     : 15
model          : 43
```

```
model name      : AMD Athlon(tm) 64 X2 Dual Core Processor 4600+
stepping        : 1
cpu MHz         : 2398.628
cache size      : 512 KB
fdiv_bug        : no
hlt_bug         : no
f00f_bug        : no
coma_bug        : no
fpu             : yes
fpu_exception   : yes
cpuid level     : 1
wp              : yes
flags           : fpu vme de pse tsc msr pae mce cx8 apic mtrr pge...
bogomips        : 4803.54
```

Just for fun, here is /proc/cpuinfo on a Sun Sunblade 1000...

```
paul@pasha:~$ cat /proc/cpuinfo
cpu : TI UltraSparc III (Cheetah)
fpu : UltraSparc III integrated FPU
promlib : Version 3 Revision 2
prom : 4.2.2
type : sun4u
ncpus probed : 2
ncpus active : 2
Cpu0Bogo : 498.68
Cpu0ClkTck : 000000002cb41780
Cpu1Bogo : 498.68
Cpu1ClkTck : 000000002cb41780
MMU Type : Cheetah
State:
CPU0: online
CPU1: online
```

Most of the files in /proc are read only, some require root privileges, some files are writable, and many files in **/proc/sys** are writable. Let's discuss some of the files in /proc.

/proc/interrupts

On the x86 architecture, **/proc/interrupts** displays the interrupts.

```
paul@RHELv4u4:~$ cat /proc/interrupts
          CPU0
 0:   13876877    IO-APIC-edge  timer
 1:         15    IO-APIC-edge  i8042
 8:          1    IO-APIC-edge  rtc
 9:          0    IO-APIC-level  acpi
12:         67    IO-APIC-edge  i8042
14:        128    IO-APIC-edge  ide0
15:       124320    IO-APIC-edge  ide1
169:       111993    IO-APIC-level  ioc0
177:        2428    IO-APIC-level  eth0
NMI:          0
LOC:       13878037
ERR:          0
MIS:          0
```

On a machine with two CPU's, the file looks like this.

```
paul@laika:~$ cat /proc/interrupts
          CPU0           CPU1
 0:   860013             0    IO-APIC-edge  timer
 1:    4533             0    IO-APIC-edge  i8042
 7:         0             0    IO-APIC-edge  parport0
 8:   6588227           0    IO-APIC-edge  rtc
10:    2314             0    IO-APIC-fasteoi  acpi
12:    133             0    IO-APIC-edge  i8042
14:         0             0    IO-APIC-edge  libata
15:    72269           0    IO-APIC-edge  libata
18:         1             0    IO-APIC-fasteoi  yenta
19:   115036           0    IO-APIC-fasteoi  eth0
20:   126871           0    IO-APIC-fasteoi  libata, ohci1394
21:    30204           0    IO-APIC-fasteoi  ehci_hcd:usb1, uhci_hcd:usb2
22:    1334             0    IO-APIC-fasteoi  saa7133[0], saa7133[0]
24:   234739           0    IO-APIC-fasteoi  nvidia
NMI:         72         42
LOC:   8600000   859994
ERR:         0
```

/proc/kcore

The physical memory is represented in **/proc/kcore**. Do not try to cat this file, instead use a debugger. The size of **/proc/kcore** is the same as your physical memory, plus four bytes.

```
paul@laika:~$ ls -lh /proc/kcore
-r----- 1 root root 2.0G 2007-01-30 08:57 /proc/kcore
paul@laika:~$
```

11.7.3. /sys Linux 2.6 hot plugging

The `/sys` directory was created for the Linux 2.6 kernel. Since 2.6, Linux uses **sysfs** to support **usb** and **IEEE 1394 (FireWire)** hot plug devices. See the manual pages of `udev(8)` (the successor of **devfs**) and `hotplug(8)` for more info (or visit <http://linux-hotplug.sourceforge.net/>).

Basically the `/sys` directory contains kernel information about hardware.

11.8. /usr Unix System Resources

Although **/usr** is pronounced like user, remember that it stands for **Unix System Resources**. The **/usr** hierarchy should contain **shareable, read only** data. Some people choose to mount **/usr** as read only. This can be done from its own partition or from a read only NFS share (NFS is discussed later).

11.8.1. /usr/bin

The **/usr/bin** directory contains a lot of commands.

```
paul@deb508:~$ ls /usr/bin | wc -l
1395
```

(On Solaris the **/bin** directory is a symbolic link to **/usr/bin**.)

11.8.2. /usr/include

The **/usr/include** directory contains general use include files for C.

```
paul@ubu1010:~$ ls /usr/include/
aolib.h      expat_config.h  math.h        search.h
af_vfs.h     expat_external.h mcheck.h      semaphore.h
aio.h        expat.h         memory.h      setjmp.h
AL           fcntl.h         menu.h        sgTTY.h
aliases.h    features.h      mntent.h      shadow.h
...
```

11.8.3. /usr/lib

The **/usr/lib** directory contains libraries that are not directly executed by users or scripts.

```
paul@deb508:~$ ls /usr/lib | head -7
4Suite
ao
apt
arj
aspell
avahi
bonobo
```

11.8.4. /usr/local

The **/usr/local** directory can be used by an administrator to install software locally.

```
paul@deb508:~$ ls /usr/local/
bin  etc  games  include  lib  man  sbin  share  src
paul@deb508:~$ du -sh /usr/local/
128K /usr/local/
```

11.8.5. /usr/share

The **/usr/share** directory contains architecture independent data. As you can see, this is a fairly large directory.

```
paul@deb508:~$ ls /usr/share/ | wc -l
```

```
263
paul@deb508:~$ du -sh /usr/share/
1.3G /usr/share/
```

This directory typically contains **/usr/share/man** for manual pages.

```
paul@deb508:~$ ls /usr/share/man
cs  fr   hu  it.UTF-8  man2  man6  pl.ISO8859-2  sv
de  fr.ISO8859-1  id  ja   man3  man7  pl.UTF-8      tr
es  fr.UTF-8      it  ko   man4  man8  pt_BR        zh_CN
fi  gl   it.ISO8859-1  man1  man5  pl    ru         zh_TW
```

And it contains **/usr/share/games** for all static game data (so no high-scores or play logs).

```
paul@ubu1010:~$ ls /usr/share/games/
openttd  wesnoth
```

11.8.6. /usr/src

The **/usr/src** directory is the recommended location for kernel source files.

```
paul@deb508:~$ ls -l /usr/src/
total 12
drwxr-xr-x  4 root root 4096 2011-02-01 14:43 linux-headers-2.6.26-2-686
drwxr-xr-x 18 root root 4096 2011-02-01 14:43 linux-headers-2.6.26-2-common
drwxr-xr-x  3 root root 4096 2009-10-28 16:01 linux-kbuild-2.6.26
```

11.9. /var variable data

Files that are unpredictable in size, such as log, cache and spool files, should be located in **/var**.

11.9.1. /var/log

The **/var/log** directory serves as a central point to contain all log files.

```
[paul@RHEL4b ~]$ ls /var/log
acpid          cron.2      maillog.2   quagga      secure.4
amanda         cron.3      maillog.3   radius      spooler
anaconda.log   cron.4      maillog.4   rpmpkgs     spooler.1
anaconda.syslog cups        mailman     rpmpkgs.1   spooler.2
anaconda.xlog dmesg       messages    rpmpkgs.2   spooler.3
audit          exim        messages.1  rpmpkgs.3   spooler.4
boot.log       gdm         messages.2  rpmpkgs.4   squid
boot.log.1     httpd       messages.3  sa           uucp
boot.log.2     iiim        messages.4  samba        vbox
boot.log.3     iptraf      mysqld.log  scrollkeeper.log vmware-tools-guestd
boot.log.4     lastlog     news        secure       wtmp
canna          mail        pgsql       secure.1     wtmp.1
cron           maillog     ppp         secure.2     Xorg.0.log
cron.1         maillog.1   prelink.log secure.3     Xorg.0.log.old
```

11.9.2. /var/log/messages

A typical first file to check when troubleshooting on Red Hat (and derivatives) is the **/var/log/messages** file. By default this file will contain information on what just happened to the system. The file is called **/var/log/syslog** on Debian and Ubuntu.

```
[root@RHEL4b ~]# tail /var/log/messages
Jul 30 05:13:56 anacron: anacron startup succeeded
Jul 30 05:13:56 atd: atd startup succeeded
Jul 30 05:13:57 messagebus: messagebus startup succeeded
Jul 30 05:13:57 cups-config-daemon: cups-config-daemon startup succeeded
Jul 30 05:13:58 haldaemon: haldaemon startup succeeded
Jul 30 05:14:00 fstab-sync[3560]: removed all generated mount points
Jul 30 05:14:01 fstab-sync[3628]: added mount point /media/cdrom for...
Jul 30 05:14:01 fstab-sync[3646]: added mount point /media/floppy for...
Jul 30 05:16:46 sshd(pam_unix)[3662]: session opened for user paul by...
Jul 30 06:06:37 su(pam_unix)[3904]: session opened for user root by paul
```

11.9.3. /var/cache

The **/var/cache** directory can contain **cache data** for several applications.

```
paul@ubu1010:~$ ls /var/cache/
apt          dictionaries-common  gdm      man          software-center
binfmts      flashplugin-installer hald     pm-utils
cups         fontconfig           jockey   pppconfig
debconf      fonts                ldconfig samba
```

11.9.4. /var/spool

The **/var/spool** directory typically contains spool directories for **mail** and **cron**, but also serves as a parent directory for other spool files (for example print spool files).

11.9.5. /var/lib

The **/var/lib** directory contains application state information.

Red Hat Enterprise Linux for example keeps files pertaining to **rpm** in **/var/lib/rpm/**.

11.9.6. /var/...

/var also contains Process ID files in **/var/run** (soon to be replaced with **/run**) and temporary files that survive a reboot in **/var/tmp** and information about file locks in **/var/lock**. There will be more examples of **/var** usage further in this book.

11.10. practice: file system tree

1. Does the file **/bin/cat** exist ? What about **/bin/dd** and **/bin/echo**. What is the type of these files ?

2. What is the size of the Linux kernel file(s) (vmlinu*) in **/boot** ?

3. Create a directory **~/test**. Then issue the following commands:

```
cd ~/test
```

```
dd if=/dev/zero of=zeros.txt count=1 bs=100
```

```
od zeroes.txt
```

dd will copy one times (count=1) a block of size 100 bytes (bs=100) from the file **/dev/zero** to **~/test/zeros.txt**. Can you describe the functionality of **/dev/zero** ?

4. Now issue the following command:

```
dd if=/dev/random of=random.txt count=1 bs=100 ; od random.txt
```

dd will copy one times (count=1) a block of size 100 bytes (bs=100) from the file **/dev/random** to **~/test/random.txt**. Can you describe the functionality of **/dev/random** ?

5. Issue the following two commands, and look at the first character of each output line.

```
ls -l /dev/sd* /dev/hd*
```

```
ls -l /dev/tty* /dev/input/mou*
```

The first **ls** will show block(b) devices, the second **ls** shows character(c) devices. Can you tell the difference between block and character devices ?

6. Use **cat** to display **/etc/hosts** and **/etc/resolv.conf**. What is your idea about the purpose of these files ?

7. Are there any files in **/etc/skel/** ? Check also for hidden files.

8. Display **/proc/cpuinfo**. On what architecture is your Linux running ?

9. Display **/proc/interrupts**. What is the size of this file ? Where is this file stored ?

10. Can you enter the **/root** directory ? Are there (hidden) files ?

11. Are **ifconfig**, **fdisk**, **parted**, **shutdown** and **grub-install** present in **/sbin** ? Why are these binaries in **/sbin** and not in **/bin** ?

12. Is **/var/log** a file or a directory ? What about **/var/spool** ?

13. Open two command prompts (Ctrl-Shift-T in gnome-terminal) or terminals (Ctrl-Alt-F1, Ctrl-Alt-F2, ...) and issue the **who am i** in both. Then try to echo a word from one terminal to the other.

14. Read the man page of **random** and explain the difference between **/dev/random** and **/dev/urandom**.

11.11. solution: file system tree

1. Does the file **/bin/cat** exist ? What about **/bin/dd** and **/bin/echo**. What is the type of these files ?

```
ls /bin/cat ; file /bin/cat
```

```
ls /bin/dd ; file /bin/dd
```

```
ls /bin/echo ; file /bin/echo
```

2. What is the size of the Linux kernel file(s) (vmlinu*) in **/boot** ?

```
ls -lh /boot/vm*
```

3. Create a directory **~/test**. Then issue the following commands:

```
cd ~/test
```

```
dd if=/dev/zero of=zeros.txt count=1 bs=100
```

```
od zeroes.txt
```

dd will copy one times (count=1) a block of size 100 bytes (bs=100) from the file **/dev/zero** to **~/test/zeros.txt**. Can you describe the functionality of **/dev/zero** ?

/dev/zero is a Linux special device. It can be considered a source of zeroes. You cannot send something to **/dev/zero**, but you can read zeroes from it.

4. Now issue the following command:

```
dd if=/dev/random of=random.txt count=1 bs=100 ; od random.txt
```

dd will copy one times (count=1) a block of size 100 bytes (bs=100) from the file **/dev/random** to **~/test/random.txt**. Can you describe the functionality of **/dev/random** ?

/dev/random acts as a **random number generator** on your Linux machine.

5. Issue the following two commands, and look at the first character of each output line.

```
ls -l /dev/sd* /dev/hd*
```

```
ls -l /dev/tty* /dev/input/mou*
```

The first **ls** will show block(b) devices, the second **ls** shows character(c) devices. Can you tell the difference between block and character devices ?

Block devices are always written to (or read from) in blocks. For hard disks, blocks of 512 bytes are common. Character devices act as a stream of characters (or bytes). Mouse and keyboard are typical character devices.

6. Use **cat** to display **/etc/hosts** and **/etc/resolv.conf**. What is your idea about the purpose of these files ?

```
/etc/hosts contains hostnames with their ip address
```

```
/etc/resolv.conf should contain the ip address of a DNS name server.
```

7. Are there any files in **/etc/skel/** ? Check also for hidden files.

Issue "ls -al /etc/skel/". Yes, there should be hidden files there.

8. Display **/proc/cpuinfo**. On what architecture is your Linux running ?

The file should contain at least one line with Intel or other cpu.

9. Display **/proc/interrupts**. What is the size of this file ? Where is this file stored ?

The size is zero, yet the file contains data. It is not stored anywhere because /proc is a virtual file system that allows you to talk with the kernel. (If you answered "stored in RAM-memory, that is also correct...).

10. Can you enter the **/root** directory ? Are there (hidden) files ?

Try "cd /root". The **/root** directory is not accessible for normal users on most modern Linux sy

11. Are ifconfig, fdisk, parted, shutdown and grub-install present in **/sbin** ? Why are these binaries in **/sbin** and not in /bin ?

Because those files are only meant for system administrators.

12. Is **/var/log** a file or a directory ? What about **/var/spool** ?

Both are directories.

13. Open two command prompts (Ctrl-Shift-T in gnome-terminal) or terminals (Ctrl-Alt-F1, Ctrl-Alt-F2, ...) and issue the **who am i** in both. Then try to echo a word from one terminal to the other.

tty-terminal: echo Hello > /dev/tty1

pts-terminal: echo Hello > /dev/pts/1

14. Read the man page of **random** and explain the difference between **/dev/random** and **/dev/urandom**.

man 4 random

Part IV. shell expansion

Table of Contents

12. commands and arguments	124
12.1. arguments	125
12.2. white space removal	125
12.3. single quotes	126
12.4. double quotes	126
12.5. echo and quotes	126
12.6. commands	127
12.7. aliases	128
12.8. displaying shell expansion	129
12.9. practice: commands and arguments	130
12.10. solution: commands and arguments	132
13. control operators	134
13.1. ; semicolon	135
13.2. & ampersand	135
13.3. \$? dollar question mark	135
13.4. && double ampersand	136
13.5. double vertical bar	136
13.6. combining && and 	136
13.7. # pound sign	137
13.8. \ escaping special characters	137
13.9. practice: control operators	138
13.10. solution: control operators	139
14. shell variables	140
14.1. \$ dollar sign	141
14.2. case sensitive	141
14.3. creating variables	141
14.4. quotes	142
14.5. set	142
14.6. unset	142
14.7. \$PS1	143
14.8. \$PATH	144
14.9. env	145
14.10. export	145
14.11. delineate variables	146
14.12. unbound variables	146
14.13. practice: shell variables	147
14.14. solution: shell variables	148
15. shell embedding and options	149
15.1. shell embedding	150
15.2. shell options	151
15.3. practice: shell embedding	152
15.4. solution: shell embedding	153
16. shell history	154
16.1. repeating the last command	155
16.2. repeating other commands	155
16.3. history	155
16.4. !n	155
16.5. Ctrl-r	156
16.6. \$HISTSIZE	156
16.7. \$HISTFILE	156
16.8. \$HISTFILESIZE	156
16.9. prevent recording a command	157
16.10. (optional)regular expressions	157
16.11. (optional) Korn shell history	157
16.12. practice: shell history	158

16.13. solution: shell history	159
17. file globbing	160
17.1. * asterisk	161
17.2. ? question mark	161
17.3. [] square brackets	162
17.4. a-z and 0-9 ranges	163
17.5. \$LANG and square brackets	163
17.6. preventing file globbing	164
17.7. practice: shell globbing	165
17.8. solution: shell globbing	166

Chapter 12. commands and arguments

This chapter introduces you to **shell expansion** by taking a close look at **commands** and **arguments**. Knowing **shell expansion** is important because many **commands** on your Linux system are processed and most likely changed by the **shell** before they are executed.

The command line interface or **shell** used on most Linux systems is called **bash**, which stands for **Bourne again shell**. The **bash** shell incorporates features from **sh** (the original Bourne shell), **cs**h (the C shell), and **ksh** (the Korn shell).

This chapter frequently uses the **echo** command to demonstrate shell features. The **echo** command is very simple: it echoes the input that it receives.

```
paul@laika:~$ echo Burtonville
Burtonville
paul@laika:~$ echo Smurfs are blue
Smurfs are blue
```

12.1. arguments

One of the primary features of a shell is to perform a **command line scan**. When you enter a command at the shell's command prompt and press the enter key, then the shell will start scanning that line, cutting it up in **arguments**. While scanning the line, the shell may make many changes to the **arguments** you typed.

This process is called **shell expansion**. When the shell has finished scanning and modifying that line, then it will be executed.

12.2. white space removal

Parts that are separated by one or more consecutive **white spaces** (or tabs) are considered separate **arguments**, any white space is removed. The first **argument** is the command to be executed, the other **arguments** are given to the command. The shell effectively cuts your command into one or more arguments.

This explains why the following four different command lines are the same after **shell expansion**.

```
[paul@RHELv4u3 ~]$ echo Hello World
Hello World
[paul@RHELv4u3 ~]$ echo Hello  World
Hello World
[paul@RHELv4u3 ~]$ echo  Hello  World
Hello World
[paul@RHELv4u3 ~]$ echo      Hello      World
Hello World
```

The **echo** command will display each argument it receives from the shell. The **echo** command will also add a new white space between the arguments it received.

12.3. single quotes

You can prevent the removal of white spaces by quoting the spaces. The contents of the quoted string are considered as one argument. In the screenshot below the **echo** receives only one **argument**.

```
[paul@RHEL4b ~]$ echo 'A line with      single      quotes'
A line with      single      quotes
[paul@RHEL4b ~]$
```

12.4. double quotes

You can also prevent the removal of white spaces by double quoting the spaces. Same as above, **echo** only receives one **argument**.

```
[paul@RHEL4b ~]$ echo "A line with      double      quotes"
A line with      double      quotes
[paul@RHEL4b ~]$
```

Later in this book, when discussing **variables** we will see important differences between single and double quotes.

12.5. echo and quotes

Quoted lines can include special escaped characters recognised by the **echo** command (when using **echo -e**). The screenshot below shows how to use **\n** for a newline and **\t** for a tab (usually eight white spaces).

```
[paul@RHEL4b ~]$ echo -e "A line with \na newline"
A line with
a newline
[paul@RHEL4b ~]$ echo -e 'A line with \na newline'
A line with
a newline
[paul@RHEL4b ~]$ echo -e "A line with \ta tab"
A line with      a tab
[paul@RHEL4b ~]$ echo -e 'A line with \ta tab'
A line with      a tab
[paul@RHEL4b ~]$
```

The echo command can generate more than white spaces, tabs and newlines. Look in the man page for a list of options.

12.6. commands

12.6.1. external or builtin commands ?

Not all commands are external to the shell, some are **builtin**. **External commands** are programs that have their own binary and reside somewhere in the file system. Many external commands are located in **/bin** or **/sbin**. **Builtin commands** are an integral part of the shell program itself.

12.6.2. type

To find out whether a command given to the shell will be executed as an **external command** or as a **builtin command**, use the **type** command.

```
paul@laika:~$ type cd
cd is a shell builtin
paul@laika:~$ type cat
cat is /bin/cat
```

As you can see, the **cd** command is **builtin** and the **cat** command is **external**.

You can also use this command to show you whether the command is **aliased** or not.

```
paul@laika:~$ type ls
ls is aliased to `ls --color=auto'
```

12.6.3. running external commands

Some commands have both builtin and external versions. When one of these commands is executed, the builtin version takes priority. To run the external version, you must enter the full path to the command.

```
paul@laika:~$ type -a echo
echo is a shell builtin
echo is /bin/echo
paul@laika:~$ /bin/echo Running the external echo command...
Running the external echo command...
```

12.6.4. which

The **which** command will search for binaries in the **\$PATH** environment variable (variables will be explained later). In the screenshot below, it is determined that **cd** is **builtin**, and **ls**, **cp**, **rm**, **mv**, **mkdir**, **pwd**, and **which** are external commands.

```
[root@RHEL4b ~]# which cp ls cd mkdir pwd
/bin/cp
/bin/ls
/usr/bin/which: no cd in (/usr/kerberos/sbin:/usr/kerberos/bin:...)
/bin/mkdir
/bin/pwd
```

12.7. aliases

12.7.1. create an alias

The shell allows you to create **aliases**. Aliases are often used to create an easier to remember name for an existing command or to easily supply parameters.

```
[paul@RHELv4u3 ~]$ cat count.txt
one
two
three
[paul@RHELv4u3 ~]$ alias dog=tac
[paul@RHELv4u3 ~]$ dog count.txt
three
two
one
```

12.7.2. abbreviate commands

An **alias** can also be useful to abbreviate an existing command.

```
paul@laika:~$ alias ll='ls -lh --color=auto'
paul@laika:~$ alias c='clear'
paul@laika:~$
```

12.7.3. default options

Aliases can be used to supply commands with default options. The example below shows how to set the **-i** option default when typing **rm**.

```
[paul@RHELv4u3 ~]$ rm -i winter.txt
rm: remove regular file `winter.txt'? no
[paul@RHELv4u3 ~]$ rm winter.txt
[paul@RHELv4u3 ~]$ ls winter.txt
ls: winter.txt: No such file or directory
[paul@RHELv4u3 ~]$ touch winter.txt
[paul@RHELv4u3 ~]$ alias rm='rm -i'
[paul@RHELv4u3 ~]$ rm winter.txt
rm: remove regular empty file `winter.txt'? no
[paul@RHELv4u3 ~]$
```

Some distributions enable default aliases to protect users from accidentally erasing files ('rm -i', 'mv -i', 'cp -i')

12.7.4. viewing aliases

You can provide one or more aliases as arguments to the **alias** command to get their definitions. Providing no arguments gives a complete list of current aliases.

```
paul@laika:~$ alias c ll
alias c='clear'
alias ll='ls -lh --color=auto'
```


12.7.5. unalias

You can undo an alias with the **unalias** command.

```
[paul@RHEL4b ~]$ which rm
/bin/rm
[paul@RHEL4b ~]$ alias rm='rm -i'
[paul@RHEL4b ~]$ which rm
alias rm='rm -i'
        /bin/rm
[paul@RHEL4b ~]$ unalias rm
[paul@RHEL4b ~]$ which rm
/bin/rm
[paul@RHEL4b ~]$
```

12.8. displaying shell expansion

You can display shell expansion with **set -x**, and stop displaying it with **set +x**. You might want to use this further on in this course, or when in doubt about exactly what the shell is doing with your command.

```
[paul@RHELv4u3 ~]$ set -x
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ echo $USER
+ echo paul
paul
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ echo \ $USER
+ echo '$USER'
$USER
++ echo -ne '\033]0;paul@RHELv4u3:~\007'
[paul@RHELv4u3 ~]$ set +x
+ set +x
[paul@RHELv4u3 ~]$ echo $USER
paul
```

12.9. practice: commands and arguments

1. How many **arguments** are in this line (not counting the command itself).

```
touch '/etc/cron/cron.allow' 'file 42.txt' "file 33.txt"
```

2. Is **tac** a shell builtin command ?

3. Is there an existing alias for **rm** ?

4. Read the man page of **rm**, make sure you understand the **-i** option of rm. Create and remove a file to test the **-i** option.

5. Execute: **alias rm='rm -i'** . Test your alias with a test file. Does this work as expected ?

6. List all current aliases.

7a. Create an alias called 'city' that echoes your hometown.

7b. Use your alias to test that it works.

8. Execute **set -x** to display shell expansion for every command.

9. Test the functionality of **set -x** by executing your **city** and **rm** aliases.

10 Execute **set +x** to stop displaying shell expansion.

11. Remove your city alias.

12. What is the location of the **cat** and the **passwd** commands ?

13. Explain the difference between the following commands:

```
echo
```

```
/bin/echo
```

14. Explain the difference between the following commands:

```
echo Hello
```

```
echo -n Hello
```

15. Display **A B C** with two spaces between B and C.

(optional)16. Complete the following command (do not use spaces) to display exactly the following output:

```
4+4      =8
10+14    =24
```

17. Use **echo** to display the following exactly:

```
??\
```

Find two solutions with single quotes, two with double quotes and one without quotes (and say thank you to René and Darioush from Google for this extra).

18. Use one **echo** command to display three words on three lines.

12.10. solution: commands and arguments

1. How many **arguments** are in this line (not counting the command itself).

```
touch '/etc/cron/cron.allow' 'file 42.txt' "file 33.txt"
```

answer: three

2. Is **tac** a shell builtin command ?

```
type tac
```

3. Is there an existing alias for **rm** ?

```
alias rm
```

4. Read the man page of **rm**, make sure you understand the **-i** option of rm. Create and remove a file to test the **-i** option.

```
man rm
```

```
touch testfile
```

```
rm -i testfile
```

5. Execute: **alias rm='rm -i'** . Test your alias with a test file. Does this work as expected ?

```
touch testfile
```

```
rm testfile (should ask for confirmation)
```

6. List all current aliases.

```
alias
```

7a. Create an alias called 'city' that echoes your hometown.

```
alias city='echo Antwerp'
```

7b. Use your alias to test that it works.

```
city (it should display Antwerp)
```

8. Execute **set -x** to display shell expansion for every command.

```
set -x
```

9. Test the functionality of **set -x** by executing your **city** and **rm** aliases.

```
shell should display the resolved aliases and then execute the command:
paul@deb503:~$ set -x
paul@deb503:~$ city
+ echo antwerp
antwerp
```

10 Execute **set +x** to stop displaying shell expansion.

```
set +x
```

11. Remove your city alias.

```
unalias city
```

12. What is the location of the **cat** and the **passwd** commands ?

```
which cat (probably /bin/cat)
```

```
which passwd (probably /usr/bin/passwd)
```

13. Explain the difference between the following commands:

```
echo
```

```
/bin/echo
```

The **echo** command will be interpreted by the shell as the **built-in echo** command. The **/bin/echo** command will make the shell execute the **echo binary** located in the **/bin** directory.

14. Explain the difference between the following commands:

```
echo Hello
```

```
echo -n Hello
```

The **-n** option of the **echo** command will prevent echo from echoing a trailing newline. **echo Hello** will echo six characters in total, **echo -n hello** only echoes five characters.

(The **-n** option might not work in the Korn shell.)

15. Display **A B C** with two spaces between B and C.

```
echo "A B  C"
```

16. Complete the following command (do not use spaces) to display exactly the following output:

```
4+4      =8
10+14    =24
```

The solution is to use tabs with **\t**.

```
echo -e "4+4\t=8" ; echo -e "10+14\t=24"
```

17. Use **echo** to display the following exactly:

```
??\
echo '??\
echo -e '??\
echo "??\
echo -e "??\
echo ??\
```

Find two solutions with single quotes, two with double quotes and one without quotes (and say thank you to René and Darioush from Google for this extra).

18. Use one **echo** command to display three words on three lines.

```
echo -e "one \ntwo \nthree"
```

Chapter 13. control operators

In this chapter we put more than one command on the command line using **control operators**. We also briefly discuss related parameters (\$?) and similar special characters(&).

13.1. ; semicolon

You can put two or more commands on the same line separated by a semicolon `;`. The shell will scan the line until it reaches the semicolon. All the arguments before this semicolon will be considered a separate command from all the arguments after the semicolon. Both series will be executed sequentially with the shell waiting for each command to finish before starting the next one.

```
[paul@RHELv4u3 ~]$ echo Hello
Hello
[paul@RHELv4u3 ~]$ echo World
World
[paul@RHELv4u3 ~]$ echo Hello ; echo World
Hello
World
[paul@RHELv4u3 ~]$
```

13.2. & ampersand

When a line ends with an ampersand `&`, the shell will not wait for the command to finish. You will get your shell prompt back, and the command is executed in background. You will get a message when this command has finished executing in background.

```
[paul@RHELv4u3 ~]$ sleep 20 &
[1] 7925
[paul@RHELv4u3 ~]$
...wait 20 seconds...
[paul@RHELv4u3 ~]$
[1]+  Done                  sleep 20
```

The technical explanation of what happens in this case is explained in the chapter about **processes**.

13.3. \$? dollar question mark

The exit code of the previous command is stored in the shell variable `$?`. Actually `$?` is a shell parameter and not a variable, since you cannot assign a value to `$?`.

```
paul@debian5:~/test$ touch file1
paul@debian5:~/test$ echo $?
0
paul@debian5:~/test$ rm file1
paul@debian5:~/test$ echo $?
0
paul@debian5:~/test$ rm file1
rm: cannot remove `file1': No such file or directory
paul@debian5:~/test$ echo $?
1
paul@debian5:~/test$
```

13.4. && double ampersand

The shell will interpret **&&** as a **logical AND**. When using **&&** the second command is executed only if the first one succeeds (returns a zero exit status).

```
paul@barry:~$ echo first && echo second
first
second
paul@barry:~$ zecho first && echo second
-bash: zecho: command not found
```

Another example of the same **logical AND** principle. This example starts with a working **cd** followed by **ls**, then a non-working **cd** which is **not** followed by **ls**.

```
[paul@RHELv4u3 ~]$ cd gen && ls
file1 file3 File55 fileab FileAB fileabc
file2 File4 FileA Fileab fileab2
[paul@RHELv4u3 gen]$ cd gen && ls
-bash: cd: gen: No such file or directory
```

13.5. || double vertical bar

The **||** represents a **logical OR**. The second command is executed only when the first command fails (returns a non-zero exit status).

```
paul@barry:~$ echo first || echo second ; echo third
first
third
paul@barry:~$ zecho first || echo second ; echo third
-bash: zecho: command not found
second
third
paul@barry:~$
```

Another example of the same **logical OR** principle.

```
[paul@RHELv4u3 ~]$ cd gen || ls
[paul@RHELv4u3 gen]$ cd gen || ls
-bash: cd: gen: No such file or directory
file1 file3 File55 fileab FileAB fileabc
file2 File4 FileA Fileab fileab2
```

13.6. combining && and ||

You can use this logical AND and logical OR to write an **if-then-else** structure on the command line. This example uses **echo** to display whether the **rm** command was successful.

```
paul@laika:~/test$ rm file1 && echo It worked! || echo It failed!
It worked!
paul@laika:~/test$ rm file1 && echo It worked! || echo It failed!
rm: cannot remove `file1': No such file or directory
It failed!
paul@laika:~/test$
```


13.7. # pound sign

Everything written after a **pound sign** (#) is ignored by the shell. This is useful to write a **shell comment**, but has no influence on the command execution or shell expansion.

```
paul@debian4:~$ mkdir test      # we create a directory
paul@debian4:~$ cd test        ##### we enter the directory
paul@debian4:~/test$ ls        # is it empty ?
paul@debian4:~/test$
```

13.8. \ escaping special characters

The backslash \ character enables the use of control characters, but without the shell interpreting it, this is called **escaping** characters.

```
[paul@RHELV4u3 ~]$ echo hello \; world
hello ; world
[paul@RHELV4u3 ~]$ echo hello\ \ \ world
hello  world
[paul@RHELV4u3 ~]$ echo escaping \\ \#\ &\ \"\ \'
escaping \ # & " '
[paul@RHELV4u3 ~]$ echo escaping \\?\*\\"\'
escaping \?*\"'
```

13.8.1. end of line backslash

Lines ending in a backslash are continued on the next line. The shell does not interpret the newline character and will wait on shell expansion and execution of the command line until a newline without backslash is encountered.

```
[paul@RHEL4b ~]$ echo This command line \
> is split in three \
> parts
This command line is split in three parts
[paul@RHEL4b ~]$
```

13.9. practice: control operators

0. Each question can be answered by one command line!
1. When you type **passwd**, which file is executed ?
2. What kind of file is that ?
3. Execute the **pwd** command twice. (remember 0.)
4. Execute **ls** after **cd /etc**, but only if **cd /etc** did not error.
5. Execute **cd /etc** after **cd etc**, but only if **cd etc** fails.
6. Echo **it worked** when **touch test42** works, and echo **it failed** when the **touch** failed. All on one command line as a normal user (not root). Test this line in your home directory and in **/bin/**.
7. Execute **sleep 6**, what is this command doing ?
8. Execute **sleep 200** in background (do not wait for it to finish).
9. Write a command line that executes **rm file55**. Your command line should print 'success' if file55 is removed, and print 'failed' if there was a problem.
- (optional)10. Use echo to display "Hello World with strange" characters \ * [} ~ \ \." (including all quotes)

13.10. solution: control operators

0. Each question can be answered by one command line!

1. When you type **passwd**, which file is executed ?

```
which passwd
```

2. What kind of file is that ?

```
file /usr/bin/passwd
```

3. Execute the **pwd** command twice. (remember 0.)

```
pwd ; pwd
```

4. Execute **ls** after **cd /etc**, but only if **cd /etc** did not error.

```
cd /etc && ls
```

5. Execute **cd /etc** after **cd etc**, but only if **cd etc** fails.

```
cd etc || cd /etc
```

6. Echo **it worked** when **touch test42** works, and echo **it failed** when the **touch** failed. All on one command line as a normal user (not root). Test this line in your home directory and in **/bin/**.

```
paul@deb503:~$ cd ; touch test42 && echo it worked || echo it failed
it worked
paul@deb503:~$ cd /bin; touch test42 && echo it worked || echo it failed
touch: cannot touch `test42': Permission denied
it failed
```

7. Execute **sleep 6**, what is this command doing ?

```
pausing for six seconds
```

8. Execute **sleep 200** in background (do not wait for it to finish).

```
sleep 200 &
```

9. Write a command line that executes **rm file55**. Your command line should print 'success' if file55 is removed, and print 'failed' if there was a problem.

```
rm file55 && echo success || echo failed
```

(optional)10. Use echo to display "Hello World with strange' characters \ * [] ~ \\" (including all quotes)

```
echo \"Hello World with strange\\' characters \\ \\* \\[ \\} \\~ \\\\ \\. \\\"
```

```
or
```

```
echo \\\"\"Hello World with strange' characters \ * [ ] ~ \\ . \"\"
```

Chapter 14. shell variables

In this chapter we learn to manage environment **variables** in the shell. These **variables** are often needed by applications.

14.1. \$ dollar sign

Another important character interpreted by the shell is the dollar sign **\$**. The shell will look for an **environment variable** named like the string following the **dollar sign** and replace it with the value of the variable (or with nothing if the variable does not exist).

These are some examples using \$HOSTNAME, \$USER, \$UID, \$SHELL, and \$HOME.

```
[paul@RHELv4u3 ~]$ echo This is the $SHELL shell
This is the /bin/bash shell
[paul@RHELv4u3 ~]$ echo This is $SHELL on computer $HOSTNAME
This is /bin/bash on computer RHELv4u3.localdomain
[paul@RHELv4u3 ~]$ echo The userid of $USER is $UID
The userid of paul is 500
[paul@RHELv4u3 ~]$ echo My homedir is $HOME
My homedir is /home/paul
```

14.2. case sensitive

This example shows that shell variables are case sensitive!

```
[paul@RHELv4u3 ~]$ echo Hello $USER
Hello paul
[paul@RHELv4u3 ~]$ echo Hello $user
Hello
```

14.3. creating variables

This example creates the variable **\$MyVar** and sets its value. It then uses **echo** to verify the value.

```
[paul@RHELv4u3 gen]$ MyVar=555
[paul@RHELv4u3 gen]$ echo $MyVar
555
[paul@RHELv4u3 gen]$
```

14.4. quotes

Notice that double quotes still allow the parsing of variables, whereas single quotes prevent this.

```
[paul@RHELv4u3 ~]$ MyVar=555
[paul@RHELv4u3 ~]$ echo $MyVar
555
[paul@RHELv4u3 ~]$ echo "$MyVar"
555
[paul@RHELv4u3 ~]$ echo '$MyVar'
$MyVar
```

The bash shell will replace variables with their value in double quoted lines, but not in single quoted lines.

```
paul@laika:~$ city=Burtonville
paul@laika:~$ echo "We are in $city today."
We are in Burtonville today.
paul@laika:~$ echo 'We are in $city today.'
We are in $city today.
```

14.5. set

You can use the **set** command to display a list of environment variables. On Ubuntu and Debian systems, the **set** command will also list shell functions after the shell variables. Use **set | more** to see the variables then.

14.6. unset

Use the **unset** command to remove a variable from your shell environment.

```
[paul@RHEL4b ~]$ MyVar=8472
[paul@RHEL4b ~]$ echo $MyVar
8472
[paul@RHEL4b ~]$ unset MyVar
[paul@RHEL4b ~]$ echo $MyVar

[paul@RHEL4b ~]$
```

14.7. \$PS1

The **\$PS1** variable determines your shell prompt. You can use backslash escaped special characters like **\u** for the username or **\w** for the working directory. The **bash** manual has a complete reference.

In this example we change the value of **\$PS1** a couple of times.

```
paul@deb503:~$ PS1=prompt
prompt
promptPS1='prompt '
prompt
prompt PS1='> '
>
> PS1='\u@\h$ '
paul@deb503$
paul@deb503$ PS1='\u@\h:\w$'
paul@deb503:~$
```

To avoid unrecoverable mistakes, you can set normal user prompts to green and the root prompt to red. Add the following to your **.bashrc** for a green user prompt:

```
# color prompt by paul
RED='\[\033[01;31m\'
WHITE='\[\033[01;00m\'
GREEN='\[\033[01;32m\'
BLUE='\[\033[01;34m\'
export PS1="${debian_chroot:+($debian_chroot)}$GREEN\u$WHITE@$BLUE\h$WHITE\w\$ "
```

14.8. \$PATH

The **\$PATH** variable determines where the shell is looking for commands to execute (unless the command is builtin or aliased). This variable contains a list of directories, separated by colons.

```
[[paul@RHEL4b ~]$ echo $PATH
/usr/kerberos/bin:/usr/local/bin:/bin:/usr/bin:
```

The shell will not look in the current directory for commands to execute! (Looking for executables in the current directory provided an easy way to hack PC-DOS computers). If you want the shell to look in the current directory, then add a **.** at the end of your **\$PATH**.

```
[paul@RHEL4b ~]$ PATH=$PATH:.
[paul@RHEL4b ~]$ echo $PATH
/usr/kerberos/bin:/usr/local/bin:/bin:/usr/bin:.
[paul@RHEL4b ~]$
```

Your path might be different when using **su** instead of **su -** because the latter will take on the environment of the target user. The root user typically has **/sbin** directories added to the **\$PATH** variable.

```
[paul@RHEL3 ~]$ su
Password:
[root@RHEL3 paul]# echo $PATH
/usr/local/bin:/bin:/usr/bin:/usr/X11R6/bin
[root@RHEL3 paul]# exit
[paul@RHEL3 ~]$ su -
Password:
[root@RHEL3 ~]# echo $PATH
/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin:
[root@RHEL3 ~]#
```


14.9. env

The **env** command without options will display a list of **exported variables**. The difference with **set** with options is that **set** lists all variables, including those not exported to child shells.

But **env** can also be used to start a clean shell (a shell without any inherited environment). The **env -i** command clears the environment for the subshell.

Notice in this screenshot that **bash** will set the **\$SHELL** variable on startup.

```
[paul@RHEL4b ~]$ bash -c 'echo $SHELL $HOME $USER'
/bin/bash /home/paul paul
[paul@RHEL4b ~]$ env -i bash -c 'echo $SHELL $HOME $USER'
/bin/bash
[paul@RHEL4b ~]$
```

You can use the **env** command to set the **\$LANG**, or any other, variable for just one instance of **bash** with one command. The example below uses this to show the influence of the **\$LANG** variable on file globbing (see the chapter on file globbing).

```
[paul@RHEL4b test]$ env LANG=C bash -c 'ls File[a-z]'
Filea Fileb
[paul@RHEL4b test]$ env LANG=en_US.UTF-8 bash -c 'ls File[a-z]'
Filea FileA Fileb FileB
[paul@RHEL4b test]$
```

14.10. export

You can export shell variables to other shells with the **export** command. This will export the variable to child shells.

```
[paul@RHEL4b ~]$ var3=three
[paul@RHEL4b ~]$ var4=four
[paul@RHEL4b ~]$ export var4
[paul@RHEL4b ~]$ echo $var3 $var4
three four
[paul@RHEL4b ~]$ bash
[paul@RHEL4b ~]$ echo $var3 $var4
four
```

But it will not export to the parent shell (previous screenshot continued).

```
[paul@RHEL4b ~]$ export var5=five
[paul@RHEL4b ~]$ echo $var3 $var4 $var5
four five
[paul@RHEL4b ~]$ exit
exit
[paul@RHEL4b ~]$ echo $var3 $var4 $var5
three four
[paul@RHEL4b ~]$
```

14.11. delineate variables

Until now, we have seen that bash interprets a variable starting from a dollar sign, continuing until the first occurrence of a non-alphanumeric character that is not an underscore. In some situations, this can be a problem. This issue can be resolved with curly braces like in this example.

```
[paul@RHEL4b ~]$ prefix=Super
[paul@RHEL4b ~]$ echo Hello $prefixman and $prefixgirl
Hello  and
[paul@RHEL4b ~]$ echo Hello ${prefix}man and ${prefix}girl
Hello Superman and Supergirl
[paul@RHEL4b ~]$
```

14.12. unbound variables

The example below tries to display the value of the **\$MyVar** variable, but it fails because the variable does not exist. By default the shell will display nothing when a variable is unbound (does not exist).

```
[paul@RHELV4u3 gen]$ echo $MyVar
[paul@RHELV4u3 gen]$
```

There is, however, the **nounset** shell option that you can use to generate an error when a variable does not exist.

```
paul@laika:~$ set -u
paul@laika:~$ echo $Myvar
bash: Myvar: unbound variable
paul@laika:~$ set +u
paul@laika:~$ echo $Myvar

paul@laika:~$
```

In the bash shell **set -u** is identical to **set -o nounset** and likewise **set +u** is identical to **set +o nounset**.

14.13. practice: shell variables

1. Use `echo` to display Hello followed by your username. (use a bash variable!)
2. Create a variable **answer** with a value of **42**.
3. Copy the value of `$LANG` to `$MyLANG`.
4. List all current shell variables.
5. List all exported shell variables.
6. Do the **env** and **set** commands display your variable ?
6. Destroy your **answer** variable.
7. Create two variables, and **export** one of them.
8. Display the exported variable in an interactive child shell.
9. Create a variable, give it the value 'Dumb', create another variable with value 'do'. Use **echo** and the two variables to echo Dumbledore.
10. Find the list of backslash escaped characters in the manual of bash. Add the time to your **PS1** prompt.

14.14. solution: shell variables

1. Use `echo` to display Hello followed by your username. (use a bash variable!)

```
echo Hello $USER
```

2. Create a variable **answer** with a value of **42**.

```
answer=42
```

3. Copy the value of `$LANG` to `$MyLANG`.

```
MyLANG=$LANG
```

4. List all current shell variables.

```
set
```

```
set | more on Ubuntu/Debian
```

5. List all exported shell variables.

```
env  
export  
declare -x
```

6. Do the **env** and **set** commands display your variable ?

```
env | more  
set | more
```

6. Destroy your **answer** variable.

```
unset answer
```

7. Create two variables, and **export** one of them.

```
var1=1; export var2=2
```

8. Display the exported variable in an interactive child shell.

```
bash  
echo $var2
```

9. Create a variable, give it the value 'Dumb', create another variable with value 'do'. Use **echo** and the two variables to echo Dumbledore.

```
varx=Dumb; vary=do
```

```
echo ${varx}le${vary}re  
solution by Yves from Dexia : echo $varx'le'$vary're'  
solution by Erwin from Telenet : echo "$varx"le"$vary"re
```

10. Find the list of backslash escaped characters in the manual of bash. Add the time to your **PS1** prompt.

```
PS1='\t \u@\h \w$ '
```

Chapter 15. shell embedding and options

This chapter takes a brief look at **child shells**, **embedded shells** and **shell options**.

15.1. shell embedding

Shells can be **embedded** on the command line, or in other words, the command line can spawn new processes containing a fork of the current shell. You can use variables to prove that new shells are created. In the screenshot below, the variable \$var1 only exists in the (temporary) sub shell.

```
[paul@RHELv4u3 gen]$ echo $var1
[paul@RHELv4u3 gen]$ echo $(var1=5;echo $var1)
5
[paul@RHELv4u3 gen]$ echo $var1
[paul@RHELv4u3 gen]$
```

You can embed a shell in an **embedded shell**, this is called **nested embedding** of shells.

This screenshot shows an embedded shell inside an embedded shell.

```
paul@deb503:~$ A=shell
paul@deb503:~$ echo ${B=sub;echo ${C=sub;echo ${C=sub;echo ${C=sub;echo $A}}}}
shell subshell subsubshell
```

15.1.1. backticks

Single embedding can be useful to avoid changing your current directory. The screenshot below uses **backticks** instead of dollar-bracket to embed.

```
[paul@RHELv4u3 ~]$ echo `cd /etc; ls -d * | grep pass`
passwd passwd- passwd.OLD
[paul@RHELv4u3 ~]$
```

You can only use the **\$()** notation to nest embedded shells, **backticks** cannot do this.

15.1.2. backticks or single quotes

Placing the embedding between **backticks** uses one character less than the dollar and parenthesis combo. Be careful however, backticks are often confused with single quotes. The technical difference between ' and ` is significant!

```
[paul@RHELv4u3 gen]$ echo `var1=5;echo $var1`
5
[paul@RHELv4u3 gen]$ echo 'var1=5;echo $var1'
var1=5;echo $var1
[paul@RHELv4u3 gen]$
```

15.2. shell options

Both **set** and **unset** are builtin shell commands. They can be used to set options of the bash shell itself. The next example will clarify this. By default, the shell will treat unset variables as a variable having no value. By setting the **-u** option, the shell will treat any reference to unset variables as an error. See the man page of bash for more information.

```
[paul@RHEL4b ~]$ echo $var123
[paul@RHEL4b ~]$ set -u
[paul@RHEL4b ~]$ echo $var123
-bash: var123: unbound variable
[paul@RHEL4b ~]$ set +u
[paul@RHEL4b ~]$ echo $var123
[paul@RHEL4b ~]$
```

To list all the set options for your shell, use **echo \$-**. The **noclobber** (or **-C**) option will be explained later in this book (in the I/O redirection chapter).

```
[paul@RHEL4b ~]$ echo $-
himBH
[paul@RHEL4b ~]$ set -C ; set -u
[paul@RHEL4b ~]$ echo $-
himuBCH
[paul@RHEL4b ~]$ set +C ; set +u
[paul@RHEL4b ~]$ echo $-
himBH
[paul@RHEL4b ~]$
```

When typing **set** without options, you get a list of all variables without function when the shell is on **posix** mode. You can set bash in posix mode typing **set -o posix**.

15.3. practice: shell embedding

1. Find the list of shell options in the man page of **bash**. What is the difference between **set -u** and **set -o nounset**?
2. Activate **nounset** in your shell. Test that it shows an error message when using non-existing variables.
3. Deactivate nounset.
4. Execute **cd /var** and **ls** in an embedded shell.

The **echo** command is only needed to show the result of the **ls** command. Omitting will result in the shell trying to execute the first file as a command.

5. Create the variable **embvar** in an embedded shell and echo it. Does the variable exist in your current shell now ?
6. Explain what "set -x" does. Can this be useful ?

(optional)7. Given the following screenshot, add exactly four characters to that command line so that the total output is FirstMiddleLast.

```
[paul@RHEL4b ~]$ echo First; echo Middle; echo Last
```

8. Display a **long listing** (**ls -l**) of the **passwd** command using the **which** command inside an embedded shell.

15.4. solution: shell embedding

1. Find the list of shell options in the man page of **bash**. What is the difference between **set -u** and **set -o nounset**?

read the manual of bash (man bash), search for nounset -- both mean the same thing.

2. Activate **nounset** in your shell. Test that it shows an error message when using non-existing variables.

```
set -u
OR
set -o nounset
```

Both these lines have the same effect.

3. Deactivate nounset.

```
set +u
OR
set +o nounset
```

4. Execute **cd /var** and **ls** in an embedded shell.

```
echo $(cd /var ; ls)
```

The **echo** command is only needed to show the result of the **ls** command. Omitting will result in the shell trying to execute the first file as a command.

5. Create the variable embvar in an embedded shell and echo it. Does the variable exist in your current shell now ?

```
echo $(embvar=emb;echo $embvar) ; echo $embvar #the last echo fails
```

```
$embvar does not exist in your current shell
```

6. Explain what "set -x" does. Can this be useful ?

```
It displays shell expansion for troubleshooting your command.
```

(optional)7. Given the following screenshot, add exactly four characters to that command line so that the total output is FirstMiddleLast.

```
[paul@RHEL4b ~]$ echo First; echo Middle; echo Last
```

```
echo -n First; echo -n Middle; echo Last
```

8. Display a **long listing** (ls -l) of the **passwd** command using the **which** command inside an embedded shell.

```
ls -l $(which passwd)
```

Chapter 16. shell history

The shell makes it easy for us to repeat commands, this chapter explains how.

16.1. repeating the last command

To repeat the last command in bash, type **!!**. This is pronounced as **bang bang**.

```
paul@debian5:~/test42$ echo this will be repeated > file42.txt
paul@debian5:~/test42$ !!
echo this will be repeated > file42.txt
paul@debian5:~/test42$
```

16.2. repeating other commands

You can repeat other commands using one **bang** followed by one or more characters. The shell will repeat the last command that started with those characters.

```
paul@debian5:~/test42$ touch file42
paul@debian5:~/test42$ cat file42
paul@debian5:~/test42$ !to
touch file42
paul@debian5:~/test42$
```

16.3. history

To see older commands, use **history** to display the shell command history (or use **history n** to see the last n commands).

```
paul@debian5:~/test$ history 10
38  mkdir test
39  cd test
40  touch file1
41  echo hello > file2
42  echo It is very cold today > winter.txt
43  ls
44  ls -l
45  cp winter.txt summer.txt
46  ls -l
47  history 10
```

16.4. !n

When typing **!** followed by the number preceding the command you want repeated, then the shell will echo the command and execute it.

```
paul@debian5:~/test$ !43
ls
file1 file2 summer.txt winter.txt
```

16.5. Ctrl-r

Another option is to use **ctrl-r** to search in the history. In the screenshot below i only typed **ctrl-r** followed by four characters **apti** and it finds the last command containing these four consecutive characters.

```
paul@debian5:~$  
(reverse-i-search)`apti': sudo aptitude install screen
```

16.6. \$HISTSIZE

The `$HISTSIZE` variable determines the number of commands that will be remembered in your current environment. Most distributions default this variable to 500 or 1000.

```
paul@debian5:~$ echo $HISTSIZE  
500
```

You can change it to any value you like.

```
paul@debian5:~$ HISTSIZE=15000  
paul@debian5:~$ echo $HISTSIZE  
15000
```

16.7. \$HISTFILE

The `$HISTFILE` variable points to the file that contains your history. The **bash** shell defaults this value to `~/.bash_history`.

```
paul@debian5:~$ echo $HISTFILE  
/home/paul/.bash_history
```

A session history is saved to this file when you **exit** the session!

*Closing a gnome-terminal with the mouse, or typing **reboot** as root will NOT save your terminal's history.*

16.8. \$HISTFILESIZE

The number of commands kept in your history file can be set using `$HISTFILESIZE`.

```
paul@debian5:~$ echo $HISTFILESIZE  
15000
```

16.9. prevent recording a command

You can prevent a command from being recorded in **history** using a space prefix.

```
paul@debian8:~/github$ echo abc
abc
paul@debian8:~/github$ echo def
def
paul@debian8:~/github$ echo ghi
ghi
paul@debian8:~/github$ history 3
9501  echo abc
9502  echo ghi
9503  history 3
```

16.10. (optional)regular expressions

It is possible to use **regular expressions** when using the **bang** to repeat commands. The screenshot below switches 1 into 2.

```
paul@debian5:~/test$ cat file1
paul@debian5:~/test$ !c:s/1/2
cat file2
hello
paul@debian5:~/test$
```

16.11. (optional) Korn shell history

Repeating a command in the **Korn shell** is very similar. The Korn shell also has the **history** command, but uses the letter **r** to recall lines from history.

This screenshot shows the history command. Note the different meaning of the parameter.

```
$ history 17
17  clear
18  echo hoi
19  history 12
20  echo world
21  history 17
```

Repeating with **r** can be combined with the line numbers given by the history command, or with the first few letters of the command.

```
$ r e
echo world
world
$ cd /etc
$ r
cd /etc
$
```

16.12. practice: shell history

1. Issue the command **echo The answer to the meaning of life, the universe and everything is 42.**
2. Repeat the previous command using only two characters (there are two solutions!)
3. Display the last 5 commands you typed.
4. Issue the long **echo** from question 1 again, using the line numbers you received from the command in question 3.
5. How many commands can be kept in memory for your current shell session ?
6. Where are these commands stored when exiting the shell ?
7. How many commands can be written to the **history file** when exiting your current shell session ?
8. Make sure your current bash shell remembers the next 5000 commands you type.
9. Open more than one console (by press Ctrl-shift-t in gnome-terminal, or by opening an extra putty.exe in MS Windows) with the same user account. When is command history written to the history file ?

16.13. solution: shell history

1. Issue the command **echo The answer to the meaning of life, the universe and everything is 42.**

```
echo The answer to the meaning of life, the universe and everything is 42
```

2. Repeat the previous command using only two characters (there are two solutions!)

```
!!  
OR  
!e
```

3. Display the last 5 commands you typed.

```
paul@ubu1010:~$ history 5  
52  ls -l  
53  ls  
54  df -h | grep sda  
55  echo The answer to the meaning of life, the universe and everything is 42  
56  history 5
```

You will receive different line numbers.

4. Issue the long **echo** from question 1 again, using the line numbers you received from the command in question 3.

```
paul@ubu1010:~$ !55  
echo The answer to the meaning of life, the universe and everything is 42  
The answer to the meaning of life, the universe and everything is 42
```

5. How many commands can be kept in memory for your current shell session ?

```
echo $HISTSIZE
```

6. Where are these commands stored when exiting the shell ?

```
echo $HISTFILE
```

7. How many commands can be written to the **history file** when exiting your current shell session ?

```
echo $HISTFILESIZE
```

8. Make sure your current bash shell remembers the next 5000 commands you type.

```
HISTSIZE=5000
```

9. Open more than one console (by press Ctrl-shift-t in gnome-terminal, or by opening an extra putty.exe in MS Windows) with the same user account. When is command history written to the history file ?

```
when you type exit
```

Chapter 17. file globbing

The shell is also responsible for **file globbing** (or dynamic filename generation). This chapter will explain **file globbing**.

17.1. * asterisk

The asterisk ***** is interpreted by the shell as a sign to generate filenames, matching the asterisk to any combination of characters (even none). When no path is given, the shell will use filenames in the current directory. See the man page of **glob(7)** for more information. (This is part of LPI topic 1.103.3.)

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls File*
File4 File55 FileA Fileab FileAB
[paul@RHELv4u3 gen]$ ls file*
file1 file2 file3 fileab fileabc
[paul@RHELv4u3 gen]$ ls *ile55
File55
[paul@RHELv4u3 gen]$ ls F*ile55
File55
[paul@RHELv4u3 gen]$ ls F*55
File55
[paul@RHELv4u3 gen]$
```

17.2. ? question mark

Similar to the asterisk, the question mark **?** is interpreted by the shell as a sign to generate filenames, matching the question mark with exactly one character.

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls File?
File4 FileA
[paul@RHELv4u3 gen]$ ls Fil?4
File4
[paul@RHELv4u3 gen]$ ls Fil??
File4 FileA
[paul@RHELv4u3 gen]$ ls File??
File55 Fileab FileAB
[paul@RHELv4u3 gen]$
```

17.3. [] square brackets

The square bracket `[` is interpreted by the shell as a sign to generate filenames, matching any of the characters between `[` and the first subsequent `]`. The order in this list between the brackets is not important. Each pair of brackets is replaced by exactly one character.

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls File[5A]
FileA
[paul@RHELv4u3 gen]$ ls File[A5]
FileA
[paul@RHELv4u3 gen]$ ls File[A5][5b]
File55
[paul@RHELv4u3 gen]$ ls File[a5][5b]
File55 Fileab
[paul@RHELv4u3 gen]$ ls File[a5][5b][abcdefghijklm]
ls: File[a5][5b][abcdefghijklm]: No such file or directory
[paul@RHELv4u3 gen]$ ls file[a5][5b][abcdefghijklm]
fileabc
[paul@RHELv4u3 gen]$
```

You can also exclude characters from a list between square brackets with the exclamation mark `!`. And you are allowed to make combinations of these **wild cards**.

```
[paul@RHELv4u3 gen]$ ls
file1 file2 file3 File4 File55 FileA fileab Fileab FileAB fileabc
[paul@RHELv4u3 gen]$ ls file[a5][!Z]
fileab
[paul@RHELv4u3 gen]$ ls file[!5]*
file1 file2 file3 fileab fileabc
[paul@RHELv4u3 gen]$ ls file[!5]?
fileab
[paul@RHELv4u3 gen]$
```

17.4. a-z and 0-9 ranges

The bash shell will also understand ranges of characters between brackets.

```
[paul@RHELv4u3 gen]$ ls
file1 file3 File55 fileab FileAB fileabc
file2 File4 FileA Fileab fileab2
[paul@RHELv4u3 gen]$ ls file[a-z]*
fileab fileab2 fileabc
[paul@RHELv4u3 gen]$ ls file[0-9]
file1 file2 file3
[paul@RHELv4u3 gen]$ ls file[a-z][a-z][0-9]*
fileab2
[paul@RHELv4u3 gen]$
```

17.5. \$LANG and square brackets

But, don't forget the influence of the **LANG** variable. Some languages include lower case letters in an upper case range (and vice versa).

```
paul@RHELv4u4:~/test$ ls [A-Z]ile?
file1 file2 file3 File4
paul@RHELv4u4:~/test$ ls [a-z]ile?
file1 file2 file3 File4
paul@RHELv4u4:~/test$ echo $LANG
en_US.UTF-8
paul@RHELv4u4:~/test$ LANG=C
paul@RHELv4u4:~/test$ echo $LANG
C
paul@RHELv4u4:~/test$ ls [a-z]ile?
file1 file2 file3
paul@RHELv4u4:~/test$ ls [A-Z]ile?
File4
paul@RHELv4u4:~/test$
```

If **\$LC_ALL** is set, then this will also need to be reset to prevent file globbing.

17.6. preventing file globbing

The screenshot below should be no surprise. The **echo *** will echo a ***** when in an empty directory. And it will echo the names of all files when the directory is not empty.

```
paul@ubu1010:~$ mkdir test42
paul@ubu1010:~$ cd test42
paul@ubu1010:~/test42$ echo *
*
paul@ubu1010:~/test42$ touch file42 file33
paul@ubu1010:~/test42$ echo *
file33 file42
```

Globbering can be prevented using quotes or by escaping the special characters, as shown in this screenshot.

```
paul@ubu1010:~/test42$ echo *
file33 file42
paul@ubu1010:~/test42$ echo \*
*
paul@ubu1010:~/test42$ echo '*'
*
paul@ubu1010:~/test42$ echo ""
*
```

17.7. practice: shell globbing

1. Create a test directory and enter it.

2. Create the following files :

```
file1
file10
file11
file2
File2
File3
file33
fileAB
filea
fileA
fileAAA
file(
file 2
```

(the last one has 6 characters including a space)

3. List (with `ls`) all files starting with file

4. List (with `ls`) all files starting with File

5. List (with `ls`) all files starting with file and ending in a number.

6. List (with `ls`) all files starting with file and ending with a letter

7. List (with `ls`) all files starting with File and having a digit as fifth character.

8. List (with `ls`) all files starting with File and having a digit as fifth character and nothing else.

9. List (with `ls`) all files starting with a letter and ending in a number.

10. List (with `ls`) all files that have exactly five characters.

11. List (with `ls`) all files that start with f or F and end with 3 or A.

12. List (with `ls`) all files that start with f have i or R as second character and end in a number.

13. List all files that do not start with the letter F.

14. Copy the value of `$LANG` to `$MyLANG`.

15. Show the influence of `$LANG` in listing A-Z or a-z ranges.

16. You receive information that one of your servers was cracked, the cracker probably replaced the `ls` command. You know that the **echo** command is safe to use. Can **echo** replace **ls** ? How can you list the files in the current directory with **echo** ?

17. Is there another command besides `cd` to change directories ?

17.8. solution: shell globbing

1. Create a test directory and enter it.

```
mkdir testdir; cd testdir
```

2. Create the following files :

```
file1
file10
file11
file2
File2
File3
file33
fileAB
filea
fileA
fileAAA
file(
file 2
```

(the last one has 6 characters including a space)

```
touch file1 file10 file11 file2 File2 File3
touch file33 fileAB filea fileA fileAAA
touch "file("
touch "file 2"
```

3. List (with ls) all files starting with file

```
ls file*
```

4. List (with ls) all files starting with File

```
ls File*
```

5. List (with ls) all files starting with file and ending in a number.

```
ls file*[0-9]
```

6. List (with ls) all files starting with file and ending with a letter

```
ls file*[a-z]
```

7. List (with ls) all files starting with File and having a digit as fifth character.

```
ls File[0-9]*
```

8. List (with ls) all files starting with File and having a digit as fifth character and nothing else.

```
ls File[0-9]
```

9. List (with ls) all files starting with a letter and ending in a number.

```
ls [a-z]*[0-9]
```

10. List (with ls) all files that have exactly five characters.

```
ls ?????
```

11. List (with `ls`) all files that start with `f` or `F` and end with `3` or `A`.

```
ls [fF]*[3A]
```

12. List (with `ls`) all files that start with `f` have `i` or `R` as second character and end in a number.

```
ls f[iR]*[0-9]
```

13. List all files that do not start with the letter `F`.

```
ls [!F]*
```

14. Copy the value of `$LANG` to `$MyLANG`.

```
MyLANG=$LANG
```

15. Show the influence of `$LANG` in listing `A-Z` or `a-z` ranges.

```
see example in book
```

16. You receive information that one of your servers was cracked, the cracker probably replaced the `ls` command. You know that the **echo** command is safe to use. Can **echo** replace **ls** ? How can you list the files in the current directory with **echo** ?

```
echo *
```

17. Is there another command besides `cd` to change directories ?

```
pushd popd
```

Part V. pipes and commands

Table of Contents

18. I/O redirection	170
18.1. stdin, stdout, and stderr	171
18.2. output redirection	172
18.3. error redirection	174
18.4. output redirection and pipes	175
18.5. joining stdout and stderr	175
18.6. input redirection	176
18.7. confusing redirection	177
18.8. quick file clear	177
18.9. practice: input/output redirection	178
18.10. solution: input/output redirection	179
19. filters	180
19.1. cat	181
19.2. tee	181
19.3. grep	181
19.4. cut	183
19.5. tr	183
19.6. wc	184
19.7. sort	185
19.8. uniq	186
19.9. comm	187
19.10. od	188
19.11. sed	189
19.12. pipe examples	190
19.13. practice: filters	191
19.14. solution: filters	192
20. basic Unix tools	194
20.1. find	195
20.2. locate	196
20.3. date	196
20.4. cal	197
20.5. sleep	197
20.6. time	198
20.7. gzip - gunzip	199
20.8. zcat - zmore	199
20.9. bzip2 - bunzip2	200
20.10. bzip2 - bzcat	200
20.11. practice: basic Unix tools	201
20.12. solution: basic Unix tools	202
21. regular expressions	204
21.1. regex versions	205
21.2. grep	206
21.3. rename	211
21.4. sed	214
21.5. bash history	218

Chapter 18. I/O redirection

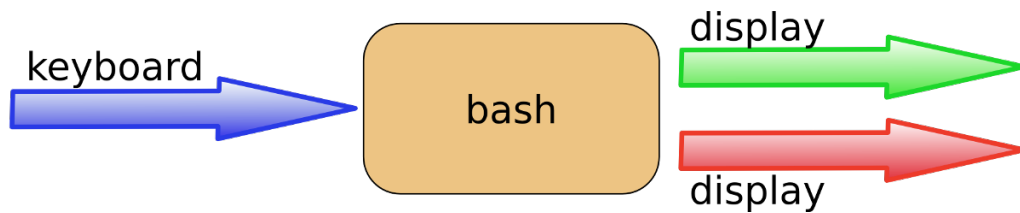
One of the powers of the Unix command line is the use of **input/output redirection** and **pipes**.

This chapter explains **redirection** of input, output and error streams.

18.1. stdin, stdout, and stderr

The bash shell has three basic streams; it takes input from **stdin** (stream **0**), it sends output to **stdout** (stream **1**) and it sends error messages to **stderr** (stream **2**) .

The drawing below has a graphical interpretation of these three streams.

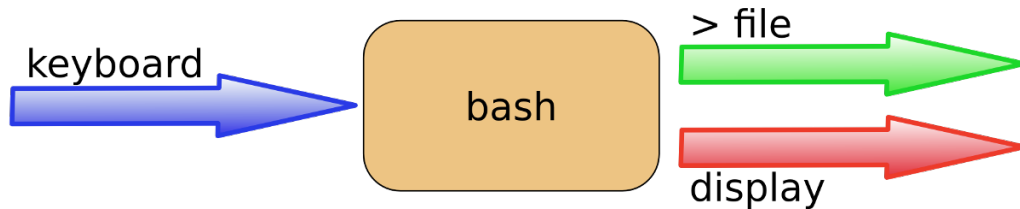


The next sections will explain how to redirect these streams.

18.2. output redirection

18.2.1. > stdout

stdout can be redirected with a **greater than** sign. While scanning the line, the shell will see the > sign and will clear the file.



The > notation is in fact the abbreviation of **1>** (**stdout** being referred to as stream **1**).

```
[paul@RHELv4u3 ~]$ echo It is cold today!
It is cold today!
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$
```

Note that the bash shell effectively **removes** the redirection from the command line before argument 0 is executed. This means that in the case of this command:

```
echo hello > greetings.txt
```

the shell only counts two arguments (echo = argument 0, hello = argument 1). The redirection is removed before the argument counting takes place.

18.2.2. output file is erased

While scanning the line, the shell will see the > sign and **will clear the file!** Since this happens before resolving **argument 0**, this means that even when the command fails, the file will have been cleared!

```
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$ zcho It is cold today! > winter.txt
-bash: zcho: command not found
[paul@RHELv4u3 ~]$ cat winter.txt
[paul@RHELv4u3 ~]$
```

18.2.3. noclobber

Erasing a file while using > can be prevented by setting the **noclobber** option.

```
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$ set -o noclobber
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
-bash: winter.txt: cannot overwrite existing file
[paul@RHELv4u3 ~]$ set +o noclobber
[paul@RHELv4u3 ~]$
```

18.2.4. overruling noclobber

The **noclobber** can be overruled with >|.

```
[paul@RHELv4u3 ~]$ set -o noclobber
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
-bash: winter.txt: cannot overwrite existing file
[paul@RHELv4u3 ~]$ echo It is very cold today! >| winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is very cold today!
[paul@RHELv4u3 ~]$
```

18.2.5. >> append

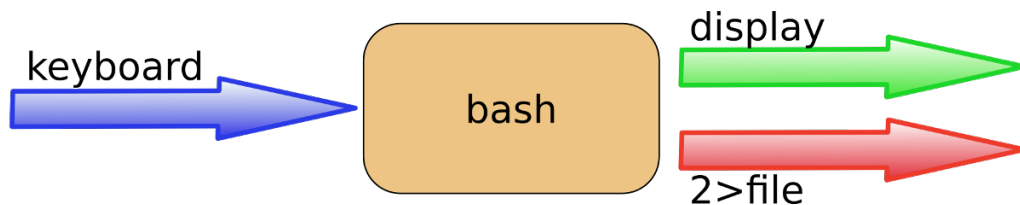
Use >> to **append** output to a file.

```
[paul@RHELv4u3 ~]$ echo It is cold today! > winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
[paul@RHELv4u3 ~]$ echo Where is the summer ? >> winter.txt
[paul@RHELv4u3 ~]$ cat winter.txt
It is cold today!
Where is the summer ?
[paul@RHELv4u3 ~]$
```

18.3. error redirection

18.3.1. 2> stderr

Redirecting **stderr** is done with **2>**. This can be very useful to prevent error messages from cluttering your screen.



The screenshot below shows redirection of **stdout** to a file, and **stderr** to **/dev/null**. Writing **1>** is the same as **>**.

```
[paul@RHELv4u3 ~]$ find / > allfiles.txt 2> /dev/null
[paul@RHELv4u3 ~]$
```

18.3.2. 2>&1

To redirect both **stdout** and **stderr** to the same file, use **2>&1**.

```
[paul@RHELv4u3 ~]$ find / > allfiles_and_errors.txt 2>&1
[paul@RHELv4u3 ~]$
```

Note that the order of redirections is significant. For example, the command

```
ls > dirlist 2>&1
```

directs both standard output (file descriptor 1) and standard error (file descriptor 2) to the file `dirlist`, while the command

```
ls 2>&1 > dirlist
```

directs only the standard output to file `dirlist`, because the standard error made a copy of the standard output before the standard output was redirected to `dirlist`.

18.4. output redirection and pipes

By default you cannot `grep` inside **stderr** when using pipes on the command line, because only **stdout** is passed.

```
paul@debian7:~$ rm file42 file33 file1201 | grep file42
rm: cannot remove 'file42': No such file or directory
rm: cannot remove 'file33': No such file or directory
rm: cannot remove 'file1201': No such file or directory
```

With `2>&1` you can force **stderr** to go to **stdout**. This enables the next command in the pipe to act on both streams.

```
paul@debian7:~$ rm file42 file33 file1201 2>&1 | grep file42
rm: cannot remove 'file42': No such file or directory
```

You cannot use both `1>&2` and `2>&1` to switch **stdout** and **stderr**.

```
paul@debian7:~$ rm file42 file33 file1201 2>&1 1>&2 | grep file42
rm: cannot remove 'file42': No such file or directory
paul@debian7:~$ echo file42 2>&1 1>&2 | sed 's/file42/FILE42/'
FILE42
```

You need a third stream to switch **stdout** and **stderr** after a pipe symbol.

```
paul@debian7:~$ echo file42 3>&1 1>&2 2>&3 | sed 's/file42/FILE42/'
file42
paul@debian7:~$ rm file42 3>&1 1>&2 2>&3 | sed 's/file42/FILE42/'
rm: cannot remove 'FILE42': No such file or directory
```

18.5. joining stdout and stderr

The `&>` construction will put both **stdout** and **stderr** in one stream (to a file).

```
paul@debian7:~$ rm file42 &> out_and_err
paul@debian7:~$ cat out_and_err
rm: cannot remove 'file42': No such file or directory
paul@debian7:~$ echo file42 &> out_and_err
paul@debian7:~$ cat out_and_err
file42
paul@debian7:~$
```

18.6. input redirection

18.6.1. < stdin

Redirecting **stdin** is done with < (short for 0<).

```
[paul@RHEL4b ~]$ cat < text.txt
one
two
[paul@RHEL4b ~]$ tr 'onetw' 'ONEZZ' < text.txt
ONE
ZZO
[paul@RHEL4b ~]$
```

18.6.2. << here document

The **here document** (sometimes called here-is-document) is a way to append input until a certain sequence (usually EOF) is encountered. The **EOF** marker can be typed literally or can be called with Ctrl-D.

```
[paul@RHEL4b ~]$ cat <<EOF > text.txt
> one
> two
> EOF
[paul@RHEL4b ~]$ cat text.txt
one
two
[paul@RHEL4b ~]$ cat <<bro1 > text.txt
> bro1
> bro1
[paul@RHEL4b ~]$ cat text.txt
bro1
[paul@RHEL4b ~]$
```

18.6.3. <<< here string

The **here string** can be used to directly pass strings to a command. The result is the same as using **echo string | command** (but you have one less process running).

```
paul@ubu1110~$ base64 <<< linux-training.be
bGludXgt dHJhaw5pbmcuYmUK
paul@ubu1110~$ base64 -d <<< bGludXgt dHJhaw5pbmcuYmUK
linux-training.be
```

See rfc 3548 for more information about **base64**.

18.7. confusing redirection

The shell will scan the whole line before applying redirection. The following command line is very readable and is correct.

```
cat winter.txt > snow.txt 2> errors.txt
```

But this one is also correct, but less readable.

```
2> errors.txt cat winter.txt > snow.txt
```

Even this will be understood perfectly by the shell.

```
< winter.txt > snow.txt 2> errors.txt cat
```

18.8. quick file clear

So what is the quickest way to clear a file ?

```
>foo
```

And what is the quickest way to clear a file when the **noclobber** option is set ?

```
>|bar
```

18.9. practice: input/output redirection

1. Activate the **noclobber** shell option.
2. Verify that **noclobber** is active by repeating an **ls** on **/etc/** with redirected output to a file.
3. When listing all shell options, which character represents the **noclobber** option ?
4. Deactivate the **noclobber** option.
5. Make sure you have two shells open on the same computer. Create an empty **tailing.txt** file. Then type **tail -f tailing.txt**. Use the second shell to **append** a line of text to that file. Verify that the first shell displays this line.
6. Create a file that contains the names of five people. Use **cat** and output redirection to create the file and use a **here document** to end the input.

18.10. solution: input/output redirection

1. Activate the **noclobber** shell option.

```
set -o noclobber  
set -C
```

2. Verify that **noclobber** is active by repeating an **ls** on **/etc/** with redirected output to a file.

```
ls /etc > etc.txt  
ls /etc > etc.txt (should not work)
```

4. When listing all shell options, which character represents the **noclobber** option ?

```
echo $- (noclobber is visible as C)
```

5. Deactivate the **noclobber** option.

```
set +o noclobber
```

6. Make sure you have two shells open on the same computer. Create an empty **tailing.txt** file. Then type **tail -f tailing.txt**. Use the second shell to **append** a line of text to that file. Verify that the first shell displays this line.

```
paul@deb503:~$ > tailing.txt  
paul@deb503:~$ tail -f tailing.txt  
hello  
world  
  
in the other shell:  
paul@deb503:~$ echo hello >> tailing.txt  
paul@deb503:~$ echo world >> tailing.txt
```

7. Create a file that contains the names of five people. Use **cat** and output redirection to create the file and use a **here document** to end the input.

```
paul@deb503:~$ cat > tennis.txt << ace  
> Justine Henin  
> Venus Williams  
> Serena Williams  
> Martina Hingis  
> Kim Clijsters  
> ace  
paul@deb503:~$ cat tennis.txt  
Justine Henin  
Venus Williams  
Serena Williams  
Martina Hingis  
Kim Clijsters  
paul@deb503:~$
```

Chapter 19. filters

Commands that are created to be used with a **pipe** are often called **filters**. These **filters** are very small programs that do one specific thing very efficiently. They can be used as **building blocks**.

This chapter will introduce you to the most common **filters**. The combination of simple commands and filters in a long **pipe** allows you to design elegant solutions.

19.1. cat

When between two **pipes**, the **cat** command does nothing (except putting **stdin** on **stdout**).

```
[paul@RHEL4b pipes]$ tac count.txt | cat | cat | cat | cat | cat
five
four
three
two
one
[paul@RHEL4b pipes]$
```

19.2. tee

Writing long **pipes** in Unix is fun, but sometimes you may want intermediate results. This is where **tee** comes in handy. The **tee** filter puts **stdin** on **stdout** and also into a file. So **tee** is almost the same as **cat**, except that it has two identical outputs.

```
[paul@RHEL4b pipes]$ tac count.txt | tee temp.txt | tac
one
two
three
four
five
[paul@RHEL4b pipes]$ cat temp.txt
five
four
three
two
one
[paul@RHEL4b pipes]$
```

19.3. grep

The **grep** filter is famous among Unix users. The most common use of **grep** is to filter lines of text containing (or not containing) a certain string.

```
[paul@RHEL4b pipes]$ cat tennis.txt
Amelie Mauresmo, Fra
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
Venus Williams, USA
[paul@RHEL4b pipes]$ cat tennis.txt | grep Williams
Serena Williams, usa
Venus Williams, USA
```

You can write this without the **cat**.

```
[paul@RHEL4b pipes]$ grep Williams tennis.txt
Serena Williams, usa
Venus Williams, USA
```

One of the most useful options of **grep** is **grep -i** which filters in a case insensitive way.

```
[paul@RHEL4b pipes]$ grep Bel tennis.txt
Justine Henin, Bel
[paul@RHEL4b pipes]$ grep -i Bel tennis.txt
```

```
Kim Clijsters, BEL
Justine Henin, Bel
[paul@RHEL4b pipes]$
```

Another very useful option is **grep -v** which outputs lines not matching the string.

```
[paul@RHEL4b pipes]$ grep -v Fra tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
Venus Williams, USA
[paul@RHEL4b pipes]$
```

And of course, both options can be combined to filter all lines not containing a case insensitive string.

```
[paul@RHEL4b pipes]$ grep -vi usa tennis.txt
Amelie Mauresmo, Fra
Kim Clijsters, BEL
Justine Henin, Bel
[paul@RHEL4b pipes]$
```

With **grep -A1** one line **after** the result is also displayed.

```
paul@debian5:~/pipes$ grep -A1 Henin tennis.txt
Justine Henin, Bel
Serena Williams, usa
```

With **grep -B1** one line **before** the result is also displayed.

```
paul@debian5:~/pipes$ grep -B1 Henin tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
```

With **grep -C1** (context) one line **before** and one **after** are also displayed. All three options (A,B, and C) can display any number of lines (using e.g. A2, B4 or C20).

```
paul@debian5:~/pipes$ grep -C1 Henin tennis.txt
Kim Clijsters, BEL
Justine Henin, Bel
Serena Williams, usa
```

19.4. cut

The **cut** filter can select columns from files, depending on a delimiter or a count of bytes. The screenshot below uses **cut** to filter for the username and userid in the **/etc/passwd** file. It uses the colon as a delimiter, and selects fields 1 and 3.

```
[[paul@RHEL4b pipes]$ cut -d: -f1,3 /etc/passwd | tail -4
Figo:510
Pfaff:511
Harry:516
Hermione:517
[paul@RHEL4b pipes]$
```

When using a space as the delimiter for **cut**, you have to quote the space.

```
[paul@RHEL4b pipes]$ cut -d" " -f1 tennis.txt
Amelie
Kim
Justine
Serena
Venus
[paul@RHEL4b pipes]$
```

This example uses **cut** to display the second to the seventh character of **/etc/passwd**.

```
[paul@RHEL4b pipes]$ cut -c2-7 /etc/passwd | tail -4
igo:x:
faff:x
arry:x
ermion
[paul@RHEL4b pipes]$
```

19.5. tr

You can translate characters with **tr**. The screenshot shows the translation of all occurrences of e to E.

```
[paul@RHEL4b pipes]$ cat tennis.txt | tr 'e' 'E'
AmElie MaurEsmo, Fra
Kim ClIjstErS, BEL
JustinE HEnin, BEL
SErEna Williams, usa
VENus Williams, USA
```

Here we set all letters to uppercase by defining two ranges.

```
[paul@RHEL4b pipes]$ cat tennis.txt | tr 'a-z' 'A-Z'
AMELIE MAURES MO, FRA
KIM CLIJSTERS, BEL
JUSTINE HENIN, BEL
SERENA WILLIAMS, USA
VENUS WILLIAMS, USA
[paul@RHEL4b pipes]$
```

Here we translate all newlines to spaces.

```
[paul@RHEL4b pipes]$ cat count.txt
one
two
```

```
three
four
five
[paul@RHEL4b pipes]$ cat count.txt | tr '\n' ' '
one two three four five [paul@RHEL4b pipes]$
```

The **tr -s** filter can also be used to squeeze multiple occurrences of a character to one.

```
[paul@RHEL4b pipes]$ cat spaces.txt
one  two      three
    four  five six
[paul@RHEL4b pipes]$ cat spaces.txt | tr -s ' '
one two three
    four five six
[paul@RHEL4b pipes]$
```

You can also use **tr** to 'encrypt' texts with **rot13**.

```
[paul@RHEL4b pipes]$ cat count.txt | tr 'a-z' 'nopqrstuvwxyzabcdefghijklm'
bar
gjb
guerr
sbhe
svir
[paul@RHEL4b pipes]$ cat count.txt | tr 'a-z' 'n-za-m'
bar
gjb
guerr
sbhe
svir
[paul@RHEL4b pipes]$
```

This last example uses **tr -d** to delete characters.

```
paul@debian5:~/pipes$ cat tennis.txt | tr -d e
Amlı Maursmo, Fra
Kim Clijstrs, BEL
Justin Hnin, Bl
Srna Williams, usa
Vnus Williams, USA
```

19.6. wc

Counting words, lines and characters is easy with **wc**.

```
[paul@RHEL4b pipes]$ wc tennis.txt
 5 15 100 tennis.txt
[paul@RHEL4b pipes]$ wc -l tennis.txt
5 tennis.txt
[paul@RHEL4b pipes]$ wc -w tennis.txt
15 tennis.txt
[paul@RHEL4b pipes]$ wc -c tennis.txt
100 tennis.txt
[paul@RHEL4b pipes]$
```


19.7. sort

The **sort** filter will default to an alphabetical sort.

```
paul@debian5:~/pipes$ cat music.txt
Queen
Brel
Led Zeppelin
Abba
paul@debian5:~/pipes$ sort music.txt
Abba
Brel
Led Zeppelin
Queen
```

But the **sort** filter has many options to tweak its usage. This example shows sorting different columns (column 1 or column 2).

```
[paul@RHEL4b pipes]$ sort -k1 country.txt
Belgium, Brussels, 10
France, Paris, 60
Germany, Berlin, 100
Iran, Teheran, 70
Italy, Rome, 50
[paul@RHEL4b pipes]$ sort -k2 country.txt
Germany, Berlin, 100
Belgium, Brussels, 10
France, Paris, 60
Italy, Rome, 50
Iran, Teheran, 70
```

The screenshot below shows the difference between an alphabetical sort and a numerical sort (both on the third column).

```
[paul@RHEL4b pipes]$ sort -k3 country.txt
Belgium, Brussels, 10
Germany, Berlin, 100
Italy, Rome, 50
France, Paris, 60
Iran, Teheran, 70
[paul@RHEL4b pipes]$ sort -n -k3 country.txt
Belgium, Brussels, 10
Italy, Rome, 50
France, Paris, 60
Iran, Teheran, 70
Germany, Berlin, 100
```

19.8. uniq

With **uniq** you can remove duplicates from a **sorted list**.

```
paul@debian5:~/pipes$ cat music.txt
Queen
Brel
Queen
Abba
paul@debian5:~/pipes$ sort music.txt
Abba
Brel
Queen
Queen
paul@debian5:~/pipes$ sort music.txt | uniq
Abba
Brel
Queen
```

uniq can also count occurrences with the **-c** option.

```
paul@debian5:~/pipes$ sort music.txt | uniq -c
  1 Abba
  1 Brel
  2 Queen
```

19.9. comm

Comparing streams (or files) can be done with the **comm**. By default **comm** will output three columns. In this example, Abba, Cure and Queen are in both lists, Bowie and Sweet are only in the first file, Turner is only in the second.

```
paul@debian5:~/pipes$ cat > list1.txt
Abba
Bowie
Cure
Queen
Sweet
paul@debian5:~/pipes$ cat > list2.txt
Abba
Cure
Queen
Turner
paul@debian5:~/pipes$ comm list1.txt list2.txt
      Abba
Bowie
      Cure
      Queen
Sweet
      Turner
```

The output of **comm** can be easier to read when outputting only a single column. The digits point out which output columns should not be displayed.

```
paul@debian5:~/pipes$ comm -12 list1.txt list2.txt
Abba
Cure
Queen
paul@debian5:~/pipes$ comm -13 list1.txt list2.txt
Turner
paul@debian5:~/pipes$ comm -23 list1.txt list2.txt
Bowie
Sweet
```

19.10. od

European humans like to work with ascii characters, but computers store files in bytes. The example below creates a simple file, and then uses **od** to show the contents of the file in hexadecimal bytes

```
paul@laika:~/test$ cat > text.txt
abcdefg
1234567
paul@laika:~/test$ od -t x1 text.txt
00000000 61 62 63 64 65 66 67 0a 31 32 33 34 35 36 37 0a
00000020
```

The same file can also be displayed in octal bytes.

```
paul@laika:~/test$ od -b text.txt
00000000 141 142 143 144 145 146 147 012 061 062 063 064 065 066 067 012
00000020
```

And here is the file in ascii (or backslashed) characters.

```
paul@laika:~/test$ od -c text.txt
00000000  a  b  c  d  e  f  g  \n  1  2  3  4  5  6  7  \n
00000020
```

19.11. sed

The stream **editor** **sed** can perform editing functions in the stream, using **regular expressions**.

```
paul@debian5:~/pipes$ echo level5 | sed 's/5/42/'
level42
paul@debian5:~/pipes$ echo level5 | sed 's/level/jump/'
jump5
```

Add **g** for global replacements (all occurrences of the string per line).

```
paul@debian5:~/pipes$ echo level5 level7 | sed 's/level/jump/'
jump5 level7
paul@debian5:~/pipes$ echo level5 level7 | sed 's/level/jump/g'
jump5 jump7
```

With **d** you can remove lines from a stream containing a character.

```
paul@debian5:~/test42$ cat tennis.txt
Venus Williams, USA
Martina Hingis, SUI
Justine Henin, BE
Serena williams, USA
Kim Clijsters, BE
Yanina Wickmayer, BE
paul@debian5:~/test42$ cat tennis.txt | sed '/BE/d'
Venus Williams, USA
Martina Hingis, SUI
Serena williams, USA
```

19.12. pipe examples

19.12.1. who | wc

How many users are logged on to this system ?

```
[paul@RHEL4b pipes]$ who
root      tty1          Jul 25 10:50
paul      pts/0          Jul 25 09:29 (laika)
Harry     pts/1          Jul 25 12:26 (barry)
paul      pts/2          Jul 25 12:26 (pasha)
[paul@RHEL4b pipes]$ who | wc -l
4
```

19.12.2. who | cut | sort

Display a sorted list of logged on users.

```
[paul@RHEL4b pipes]$ who | cut -d' ' -f1 | sort
Harry
paul
paul
root
```

Display a sorted list of logged on users, but every user only once .

```
[paul@RHEL4b pipes]$ who | cut -d' ' -f1 | sort | uniq
Harry
paul
root
```

19.12.3. grep | cut

Display a list of all bash **user accounts** on this computer. Users accounts are explained in detail later.

```
paul@debian5:~$ grep bash /etc/passwd
root:x:0:0:root:/root:/bin/bash
paul:x:1000:1000:paul,,,:/home/paul:/bin/bash
serena:x:1001:1001:./home/serena:/bin/bash
paul@debian5:~$ grep bash /etc/passwd | cut -d: -f1
root
paul
serena
```

19.13. practice: filters

1. Put a sorted list of all bash users in `bashusers.txt`.
2. Put a sorted list of all logged on users in `onlineusers.txt`.
3. Make a list of all filenames in `/etc` that contain the string **conf** in their filename.
4. Make a sorted list of all files in `/etc` that contain the case insensitive string **conf** in their filename.
5. Look at the output of `/sbin/ifconfig`. Write a line that displays only ip address and the subnet mask.
6. Write a line that removes all non-letters from a stream.
7. Write a line that receives a text file, and outputs all words on a separate line.
8. Write a spell checker on the command line. (There may be a dictionary in `/usr/share/dict/`.)

19.14. solution: filters

1. Put a sorted list of all bash users in bashusers.txt.

```
grep bash /etc/passwd | cut -d: -f1 | sort > bashusers.txt
```

2. Put a sorted list of all logged on users in onlineusers.txt.

```
who | cut -d' ' -f1 | sort > onlineusers.txt
```

3. Make a list of all filenames in **/etc** that contain the string **conf** in their filename.

```
ls /etc | grep conf
```

4. Make a sorted list of all files in **/etc** that contain the case insensitive string **conf** in their filename.

```
ls /etc | grep -i conf | sort
```

5. Look at the output of **/sbin/ifconfig**. Write a line that displays only ip address and the subnet mask.

```
/sbin/ifconfig | head -2 | grep 'inet ' | tr -s ' ' | cut -d' ' -f3,5
```

6. Write a line that removes all non-letters from a stream.

```
paul@deb503:~$ cat text
This is, yes really! , a text with ?&* too many str$ange# characters ;-)
paul@deb503:~$ cat text | tr -d ',!$?.*&^%#@;()-'
This is yes really a text with too many strange characters
```

7. Write a line that receives a text file, and outputs all words on a separate line.

```
paul@deb503:~$ cat text2
it is very cold today without the sun

paul@deb503:~$ cat text2 | tr ' ' '\n'
it
is
very
cold
today
without
the
sun
```

8. Write a spell checker on the command line. (There may be a dictionary in **/usr/share/dict/**.)

```
paul@rhel ~$ echo "The zun is shining today" > text

paul@rhel ~$ cat > DICT
is
shining
sun
the
```


today

```
paul@rhel ~$ cat text | tr 'A-Z ' 'a-z\n' | sort | uniq | comm -23 - DICT  
zun
```

You could also add the solution from question number 6 to remove non-letters, and **tr -s ' '** to remove redundant spaces.

Chapter 20. basic Unix tools

This chapter introduces commands to **find** or **locate** files and to **compress** files, together with other common tools that were not discussed before. While the tools discussed here are technically not considered **filters**, they can be used in **pipes**.

20.1. find

The **find** command can be very useful at the start of a pipe to search for files. Here are some examples. You might want to add **2>/dev/null** to the command lines to avoid cluttering your screen with error messages.

Find all files in **/etc** and put the list in **etcfiles.txt**

```
find /etc > etcfiles.txt
```

Find all files of the entire system and put the list in **allfiles.txt**

```
find / > allfiles.txt
```

Find files that end in **.conf** in the current directory (and all subdirs).

```
find . -name "*.conf"
```

Find files of type file (not directory, pipe or etc.) that end in **.conf**.

```
find . -type f -name "*.conf"
```

Find files of type directory that end in **.bak**.

```
find /data -type d -name "*.bak"
```

Find files that are newer than **file42.txt**

```
find . -newer file42.txt
```

Find can also execute another command on every file found. This example will look for ***.odf** files and copy them to **/backup/**.

```
find /data -name "*.odf" -exec cp {} /backup/ \;
```

Find can also execute, after your confirmation, another command on every file found. This example will remove ***.odf** files if you approve of it for every file found.

```
find /data -name "*.odf" -ok rm {} \;
```

20.2. locate

The **locate** tool is very different from **find** in that it uses an index to locate files. This is a lot faster than traversing all the directories, but it also means that it is always outdated. If the index does not exist yet, then you have to create it (as root on Red Hat Enterprise Linux) with the **updatedb** command.

```
[paul@RHEL4b ~]$ locate Samba
warning: locate: could not open database: /var/lib/slocate/slocate.db:...
warning: You need to run the 'updatedb' command (as root) to create th...
Please have a look at /etc/updatedb.conf to enable the daily cron job.
[paul@RHEL4b ~]$ updatedb
fatal error: updatedb: You are not authorized to create a default sloc...
[paul@RHEL4b ~]$ su -
Password:
[root@RHEL4b ~]# updatedb
[root@RHEL4b ~]#
```

Most Linux distributions will schedule the **updatedb** to run once every day.

20.3. date

The **date** command can display the date, time, time zone and more.

```
paul@rhel55 ~$ date
Sat Apr 17 12:44:30 CEST 2010
```

A date string can be customised to display the format of your choice. Check the man page for more options.

```
paul@rhel55 ~$ date +%A %d-%m-%Y'
Saturday 17-04-2010
```

Time on any Unix is calculated in number of seconds since 1969 (the first second being the first second of the first of January 1970). Use **date +%s** to display Unix time in seconds.

```
paul@rhel55 ~$ date +%s
1271501080
```

When will this seconds counter reach two thousand million ?

```
paul@rhel55 ~$ date -d '1970-01-01 + 2000000000 seconds'
Wed May 18 04:33:20 CEST 2033
```

20.4. cal

The **cal** command displays the current month, with the current day highlighted.

```
paul@rhel55 ~$ cal
    April 2010
Su Mo Tu We Th Fr Sa
                1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30
```

You can select any month in the past or the future.

```
paul@rhel55 ~$ cal 2 1970
    February 1970
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
```

20.5. sleep

The **sleep** command is sometimes used in scripts to wait a number of seconds. This example shows a five second **sleep**.

```
paul@rhel55 ~$ sleep 5
paul@rhel55 ~$
```

20.6. time

The **time** command can display how long it takes to execute a command. The **date** command takes only a little time.

```
paul@rhel55 ~$ time date
Sat Apr 17 13:08:27 CEST 2010

real    0m0.014s
user    0m0.008s
sys     0m0.006s
```

The **sleep 5** command takes five **real** seconds to execute, but consumes little **cpu time**.

```
paul@rhel55 ~$ time sleep 5

real    0m5.018s
user    0m0.005s
sys     0m0.011s
```

This **bzip2** command compresses a file and uses a lot of **cpu time**.

```
paul@rhel55 ~$ time bzip2 text.txt

real    0m2.368s
user    0m0.847s
sys     0m0.539s
```

20.7. gzip - gunzip

Users never have enough disk space, so compression comes in handy. The **gzip** command can make files take up less space.

```
paul@rhel55 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
paul@rhel55 ~$ gzip text.txt
paul@rhel55 ~$ ls -lh text.txt.gz
-rw-rw-r-- 1 paul paul 760K Apr 17 13:11 text.txt.gz
```

You can get the original back with **gunzip**.

```
paul@rhel55 ~$ gunzip text.txt.gz
paul@rhel55 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
```

20.8. zcat - zmore

Text files that are compressed with **gzip** can be viewed with **zcat** and **zmore**.

```
paul@rhel55 ~$ head -4 text.txt
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
paul@rhel55 ~$ gzip text.txt
paul@rhel55 ~$ zcat text.txt.gz | head -4
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
```

20.9. bzip2 - bunzip2

Files can also be compressed with **bzip2** which takes a little more time than **gzip**, but compresses better.

```
paul@rhel55 ~$ bzip2 text.txt
paul@rhel55 ~$ ls -lh text.txt.bz2
-rw-rw-r-- 1 paul paul 569K Apr 17 13:11 text.txt.bz2
```

Files can be uncompressed again with **bunzip2**.

```
paul@rhel55 ~$ bunzip2 text.txt.bz2
paul@rhel55 ~$ ls -lh text.txt
-rw-rw-r-- 1 paul paul 6.4M Apr 17 13:11 text.txt
```

20.10. bzip2 - bzip2

And in the same way **bzcat** and **bzmore** can display files compressed with **bzip2**.

```
paul@rhel55 ~$ bzip2 text.txt
paul@rhel55 ~$ bzcat text.txt.bz2 | head -4
/
/opt
/opt/VBoxGuestAdditions-3.1.6
/opt/VBoxGuestAdditions-3.1.6/routines.sh
```


20.11. practice: basic Unix tools

1. Explain the difference between these two commands. This question is very important. If you don't know the answer, then look back at the **shell** chapter.

```
find /data -name "*.txt"
```

```
find /data -name *.txt
```

2. Explain the difference between these two statements. Will they both work when there are 200 **.odf** files in **/data** ? How about when there are 2 million **.odf** files ?

```
find /data -name "*.odf" > data_odf.txt
```

```
find /data/*.odf > data_odf.txt
```

3. Write a find command that finds all files created after January 30th 2010.

4. Write a find command that finds all *.odf files created in September 2009.

5. Count the number of *.conf files in /etc and all its subdirs.

6. Here are two commands that do the same thing: copy *.odf files to /backup/. What would be a reason to replace the first command with the second ? Again, this is an important question.

```
cp -r /data/*.odf /backup/
```

```
find /data -name "*.odf" -exec cp {} /backup/ \;
```

7. Create a file called **loctest.txt**. Can you find this file with **locate** ? Why not ? How do you make locate find this file ?

8. Use find and -exec to rename all .htm files to .html.

9. Issue the **date** command. Now display the date in YYYY/MM/DD format.

10. Issue the **cal** command. Display a calendar of 1582 and 1752. Notice anything special ?

20.12. solution: basic Unix tools

1. Explain the difference between these two commands. This question is very important. If you don't know the answer, then look back at the **shell** chapter.

```
find /data -name "*.txt"
```

```
find /data -name *.txt
```

When ***.txt** is quoted then the shell will not touch it. The **find** tool will look in the **/data** for all files ending in **.txt**.

When ***.txt** is not quoted then the shell might expand this (when one or more files that ends in **.txt** exist in the current directory). The **find** might show a different result, or can result in a syntax error.

2. Explain the difference between these two statements. Will they both work when there are 200 **.odf** files in **/data** ? How about when there are 2 million **.odf** files ?

```
find /data -name "*.odf" > data_odf.txt
```

```
find /data/*.odf > data_odf.txt
```

The first **find** will output all **.odf** filenames in **/data** and all subdirectories. The shell will redirect this to a file.

The second find will output all files named **.odf** in **/data** and will also output all files that exist in directories named ***.odf** (in **/data**).

With two million files the command line would be expanded beyond the maximum that the shell can accept. The last part of the command line would be lost.

3. Write a find command that finds all files created after January 30th 2010.

```
touch -t 201001302359 marker_date  
find . -type f -newer marker_date
```

```
There is another solution :  
find . -type f -newerat "20100130 23:59:59"
```

4. Write a find command that finds all ***.odf** files created in September 2009.

```
touch -t 200908312359 marker_start  
touch -t 200910010000 marker_end  
find . -type f -name "*.odf" -newer marker_start ! -newer marker_end
```

The exclamation mark **! -newer** can be read as **not newer**.

5. Count the number of ***.conf** files in **/etc** and all its subdirs.

```
find /etc -type f -name '*.conf' | wc -l
```

6. Here are two commands that do the same thing: copy ***.odf** files to **/backup/**. What would be a reason to replace the first command with the second ? Again, this is an important question.

```
cp -r /data/*.odf /backup/
```

```
find /data -name "*.odf" -exec cp {} /backup/ \;
```

The first might fail when there are too many files to fit on one command line.

7. Create a file called **loctest.txt**. Can you find this file with **locate** ? Why not ? How do you make locate find this file ?

You cannot locate this with **locate** because it is not yet in the index.

```
updatedb
```

8. Use find and -exec to rename all .htm files to .html.

```
paul@rhel55 ~$ find . -name '*.htm'
./one.htm
./two.htm
paul@rhel55 ~$ find . -name '*.htm' -exec mv {} {}l \;
paul@rhel55 ~$ find . -name '*.htm*'
./one.html
./two.html
```

9. Issue the **date** command. Now display the date in YYYY/MM/DD format.

```
date +%Y/%m/%d
```

10. Issue the **cal** command. Display a calendar of 1582 and 1752. Notice anything special ?

```
cal 1582
```

The calendars are different depending on the country. Check <http://linux-training.be/files/studentfiles/dates.txt>

Chapter 21. regular expressions

Regular expressions are a very powerful tool in Linux. They can be used with a variety of programs like bash, vi, rename, grep, sed, and more.

This chapter introduces you to the basics of **regular expressions**.

21.1. regex versions

There are three different versions of regular expression syntax:

```
BRE: Basic Regular Expressions  
ERE: Extended Regular Expressions  
PRCE: Perl Regular Expressions
```

Depending on the tool being used, one or more of these syntaxes can be used.

For example the **grep** tool has the **-E** option to force a string to be read as ERE while **-G** forces BRE and **-P** forces PRCE.

Note that **grep** also has **-F** to force the string to be read literally.

The **sed** tool also has options to choose a regex syntax.

Read the manual of the tools you use!

21.2. grep

21.2.1. print lines matching a pattern

grep is a popular Linux tool to search for lines that match a certain pattern. Below are some examples of the simplest **regular expressions**.

This is the contents of the test file. This file contains three lines (or three **newline** characters).

```
paul@rhel65:~$ cat names
Tania
Laura
Valentina
```

When **grepping** for a single character, only the lines containing that character are returned.

```
paul@rhel65:~$ grep u names
Laura
paul@rhel65:~$ grep e names
Valentina
paul@rhel65:~$ grep i names
Tania
Valentina
```

The pattern matching in this example should be very straightforward; if the given character occurs on a line, then **grep** will return that line.

21.2.2. concatenating characters

Two concatenated characters will have to be concatenated in the same way to have a match.

This example demonstrates that **ia** will match **Tania** but not **Valentina** and **in** will match **Valentina** but not **Tania**.

```
paul@rhel65:~$ grep a names
Tania
Laura
Valentina
paul@rhel65:~$ grep ia names
Tania
paul@rhel65:~$ grep in names
Valentina
paul@rhel65:~$
```

21.2.3. one or the other

PRCE and ERE both use the pipe symbol to signify OR. In this example we **grep** for lines containing the letter i or the letter a.

```
paul@debian7:~$ cat list
Tania
Laura
paul@debian7:~$ grep -E 'i|a' list
Tania
Laura
```

Note that we use the **-E** switch of grep to force interpretation of our string as an ERE.

We need to **escape** the pipe symbol in a BRE to get the same logical OR.

```
paul@debian7:~$ grep -G 'i|a' list
paul@debian7:~$ grep -G 'i\|a' list
Tania
Laura
```

21.2.4. one or more

The ***** signifies zero, one or more occurrences of the previous and the **+** signifies one or more of the previous.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o*' list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o+' list2
lol
lool
loool
paul@debian7:~$
```

21.2.5. match the end of a string

For the following examples, we will use this file.

```
paul@debian7:~$ cat names
Tania
Laura
Valentina
Fleur
Floor
```

The two examples below show how to use the **dollar character** to match the end of a string.

```
paul@debian7:~$ grep a$ names
Tania
Laura
Valentina
paul@debian7:~$ grep r$ names
Fleur
Floor
```

21.2.6. match the start of a string

The **caret character** (^) will match a string at the start (or the beginning) of a line.

Given the same file as above, here are two examples.

```
paul@debian7:~$ grep ^Val names
Valentina
paul@debian7:~$ grep ^F names
Fleur
Floor
```

Both the dollar sign and the little hat are called **anchors** in a regex.

21.2.7. separating words

Regular expressions use a `\b` sequence to reference a word separator. Take for example this file:

```
paul@debian7:~$ cat text
The governer is governing.
The winter is over.
Can you get over there?
```

Simply grepping for **over** will give too many results.

```
paul@debian7:~$ grep over text
The governer is governing.
The winter is over.
Can you get over there?
```

Surrounding the searched word with spaces is not a good solution (because other characters can be word separators). This screenshot below show how to use `\b` to find only the searched word:

```
paul@debian7:~$ grep '\bover\b' text
The winter is over.
Can you get over there?
paul@debian7:~$
```

Note that **grep** also has a `-w` option to grep for words.

```
paul@debian7:~$ cat text
The governer is governing.
The winter is over.
Can you get over there?
paul@debian7:~$ grep -w over text
The winter is over.
Can you get over there?
paul@debian7:~$
```

21.2.8. grep features

Sometimes it is easier to combine a simple regex with **grep** options, than it is to write a more complex regex. These options were discussed before:

```
grep -i
grep -v
grep -w
grep -A5
grep -B5
grep -C5
```

21.2.9. preventing shell expansion of a regex

The dollar sign is a special character, both for the regex and also for the shell (remember variables and embedded shells). Therefore it is advised to always quote the regex, this prevents shell expansion.

```
paul@debian7:~$ grep 'r$' names
Fleur
Floor
```

21.3. rename

21.3.1. the rename command

On Debian Linux the **/usr/bin/rename** command is a link to **/usr/bin/prename** installed by the **perl** package.

```
paul@pi ~ $ dpkg -S $(readlink -f $(which rename))
perl: /usr/bin/prename
```

Red Hat derived systems do not install the same **rename** command, so this section does not describe **rename** on Red Hat (unless you copy the perl script manually).

There is often confusion on the internet about the rename command because solutions that work fine in Debian (and Ubuntu, xubuntu, Mint, ...) cannot be used in Red Hat (and CentOS, Fedora, ...).

21.3.2. perl

The **rename** command is actually a perl script that uses **perl regular expressions**. The complete manual for these can be found by typing **perldoc perlrequick** (after installing **perldoc**).

```
root@pi:~# aptitude install perl-doc
The following NEW packages will be installed:
  perl-doc
0 packages upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 8,170 kB of archives. After unpacking 13.2 MB will be used.
Get: 1 http://mirrordirector.raspbian.org/raspbian/ wheezy/main perl-do...
Fetched 8,170 kB in 19s (412 kB/s)
Selecting previously unselected package perl-doc.
(Reading database ... 67121 files and directories currently installed.)
Unpacking perl-doc (from .../perl-doc_5.14.2-21+rpi2_all.deb) ...
Adding 'diversion of /usr/bin/perldoc to /usr/bin/perldoc.stub by perl-doc'
Processing triggers for man-db ...
Setting up perl-doc (5.14.2-21+rpi2) ...

root@pi:~# perldoc perlrequick
```

21.3.3. well known syntax

The most common use of the **rename** is to search for filenames matching a certain **string** and replacing this string with an **other string**.

This is often presented as **s/string/other string/** as seen in this example:

```
paul@pi ~ $ ls
abc      allfiles.TXT  bllfiles.TXT  Scratch      tennis2.TXT
abc.conf backup        cllfiles.TXT  temp.TXT     tennis.TXT
paul@pi ~ $ rename 's/TXT/text/' *
paul@pi ~ $ ls
abc      allfiles.text  bllfiles.text  Scratch      tennis2.text
abc.conf backup        cllfiles.text  temp.text    tennis.text
```

And here is another example that uses **rename** with the well know syntax to change the extensions of the same files once more:

```
paul@pi ~ $ ls
abc      allfiles.text  bllfiles.text  Scratch      tennis2.text
abc.conf backup        cllfiles.text  temp.text    tennis.text
paul@pi ~ $ rename 's/text/txt/' *.text
paul@pi ~ $ ls
abc      allfiles.txt  bllfiles.txt  Scratch      tennis2.txt
abc.conf backup        cllfiles.txt  temp.txt     tennis.txt
paul@pi ~ $
```

These two examples appear to work because the strings we used only exist at the end of the filename. Remember that file extensions have no meaning in the bash shell.

The next example shows what can go wrong with this syntax.

```
paul@pi ~ $ touch atxt.txt
paul@pi ~ $ rename 's/txt/problem/' atxt.txt
paul@pi ~ $ ls
abc      allfiles.txt  backup        cllfiles.txt  temp.txt     tennis.txt
abc.conf aproblem.txt  bllfiles.txt  Scratch      tennis2.txt
paul@pi ~ $
```

Only the first occurrence of the searched string is replaced.

21.3.4. a global replace

The syntax used in the previous example can be described as **s/regex/replacement/**. This is simple and straightforward, you enter a **regex** between the first two slashes and a **replacement string** between the last two.

This example expands this syntax only a little, by adding a **modifier**.

```
paul@pi ~ $ rename -n 's/TXT/txt/g' aTXT.TXT
aTXT.TXT renamed as atxt.txt
paul@pi ~ $
```

The syntax we use now can be described as **s/regex/replacement/g** where **s** signifies **switch** and **g** stands for **global**.

Note that this example used the **-n** switch to show what is being done (instead of actually renaming the file).

21.3.5. case insensitive replace

Another **modifier** that can be useful is **i**. this example shows how to replace a case insensitive string with another string.

```
paul@debian7:~/files$ ls
file1.text file2.TEXT file3.txt
paul@debian7:~/files$ rename 's/.text/.txt/i' *
paul@debian7:~/files$ ls
file1.txt file2.txt file3.txt
paul@debian7:~/files$
```

21.3.6. renaming extensions

Command line Linux has no knowledge of MS-DOS like extensions, but many end users and graphical application do use them.

Here is an example on how to use **rename** to only rename the file extension. It uses the dollar sign to mark the ending of the filename.

```
paul@pi ~ $ ls *.txt
allfiles.txt bllfiles.txt cllfiles.txt really.txt.txt temp.txt tennis.txt
paul@pi ~ $ rename 's/.txt$/.TXT/' *.txt
paul@pi ~ $ ls *.TXT
allfiles.TXT bllfiles.TXT cllfiles.TXT really.txt.TXT
temp.TXT    tennis.TXT
paul@pi ~ $
```

Note that the **dollar sign** in the regex means **at the end**. Without the dollar sign this command would fail on the `really.txt.txt` file.

21.4. sed

21.4.1. stream editor

The **stream editor** or short **sed** uses **regex** for stream editing.

In this example **sed** is used to replace a string.

```
echo Sunday | sed 's/Sun/Mon/'  
Monday
```

The slashes can be replaced by a couple of other characters, which can be handy in some cases to improve readability.

```
echo Sunday | sed 's:Sun:Mon:'  
Monday  
echo Sunday | sed 's_Sun_Mon_'  
Monday  
echo Sunday | sed 's|Sun|Mon|'  
Monday
```

21.4.2. interactive editor

While **sed** is meant to be used in a stream, it can also be used interactively on a file.

```
paul@debian7:~/files$ echo Sunday > today  
paul@debian7:~/files$ cat today  
Sunday  
paul@debian7:~/files$ sed -i 's/Sun/Mon/' today  
paul@debian7:~/files$ cat today  
Monday
```

21.4.3. simple back referencing

The **ampersand** character can be used to reference the searched (and found) string.

In this example the **ampersand** is used to double the occurrence of the found string.

```
echo Sunday | sed 's/Sun/&&/'  
SunSunday  
echo Sunday | sed 's/day/&&/'  
Sundayday
```

21.4.4. back referencing

Parentheses (often called round brackets) are used to group sections of the regex so they can later be referenced.

Consider this simple example:

```
paul@debian7:~$ echo Sunday | sed 's_(Sun_)_1ny_'  
Sunnyday  
paul@debian7:~$ echo Sunday | sed 's_(Sun_)_1ny _1_'  
Sunny Sunday
```

21.4.5. a dot for any character

In a **regex** a simple dot can signify any character.

```
paul@debian7:~$ echo 2014-04-01 | sed 's/....-.../YYYY-MM-DD/'  
YYYY-MM-DD  
paul@debian7:~$ echo abcd-ef-gh | sed 's/....-.../YYYY-MM-DD/'  
YYYY-MM-DD
```

21.4.6. multiple back referencing

When more than one pair of **parentheses** is used, each of them can be referenced separately by consecutive numbers.

```
paul@debian7:~$ echo 2014-04-01 | sed 's/\(....\)-(..)-\(...\)/\1+\2+\3/'  
2014+04+01  
paul@debian7:~$ echo 2014-04-01 | sed 's/\(....\)-(..)-\(...\)/\3:\2:\1/'  
01:04:2014
```

This feature is called **grouping**.

21.4.7. white space

The `\s` can refer to white space such as a space or a tab.

This example looks for white spaces (`\s`) globally and replaces them with 1 space.

```
paul@debian7:~$ echo -e 'today\tis\twarm'
today  is      warm
paul@debian7:~$ echo -e 'today\tis\twarm' | sed 's_\s_ _g'
today is warm
```

21.4.8. optional occurrence

A question mark signifies that the previous is **optional**.

The example below searches for three consecutive letter o, but the third o is optional.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'ooo?' list2
lool
loool
paul@debian7:~$ cat list2 | sed 's/ooo\?/A/'
ll
lol
lAl
lAl
```


21.4.9. exactly n times

You can demand an exact number of times the oprevious has to occur.

This example wants exactly three o's.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o{3}' list2
loool
paul@debian7:~$ cat list2 | sed 's/o\{3\}/A/'
ll
lol
lool
lAl
paul@debian7:~$
```

21.4.10. between n and m times

And here we demand exactly from minimum 2 to maximum 3 times.

```
paul@debian7:~$ cat list2
ll
lol
lool
loool
paul@debian7:~$ grep -E 'o{2,3}' list2
lool
loool
paul@debian7:~$ grep 'o\{2,3\}' list2
lool
loool
paul@debian7:~$ cat list2 | sed 's/o\{2,3\}/A/'
ll
lol
lAl
lAl
paul@debian7:~$
```

21.5. bash history

The **bash shell** can also interpret some regular expressions.

This example shows how to manipulate the exclamation mark history feature of the bash shell.

```
paul@debian7:~$ mkdir hist
paul@debian7:~$ cd hist/
paul@debian7:~/hist$ touch file1 file2 file3
paul@debian7:~/hist$ ls -l file1
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file1
paul@debian7:~/hist$ !l
ls -l file1
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file1
paul@debian7:~/hist$ !l:s/1/3
ls -l file3
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file3
paul@debian7:~/hist$
```

This also works with the history numbers in bash.

```
paul@debian7:~/hist$ history 6
2089  mkdir hist
2090  cd hist/
2091  touch file1 file2 file3
2092  ls -l file1
2093  ls -l file3
2094  history 6
paul@debian7:~/hist$ !2092
ls -l file1
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file1
paul@debian7:~/hist$ !2092:s/1/2
ls -l file2
-rw-r--r-- 1 paul paul 0 Apr 15 22:07 file2
paul@debian7:~/hist$
```

Part VI. vi

Table of Contents

22. Introduction to vi	221
22.1. command mode and insert mode	222
22.2. start typing (a A i I o O)	222
22.3. replace and delete a character (r x X)	223
22.4. undo and repeat (u .)	223
22.5. cut, copy and paste a line (dd yy p P)	223
22.6. cut, copy and paste lines (3dd 2yy)	224
22.7. start and end of a line (0 or ^ and \$)	224
22.8. join two lines (J) and more	224
22.9. words (w b)	225
22.10. save (or not) and exit (:w :q :q!)	225
22.11. Searching (/ ?)	225
22.12. replace all (:1,\$ s/foo/bar/g)	226
22.13. reading files (:r :r !cmd)	226
22.14. text buffers	226
22.15. multiple files	226
22.16. abbreviations	227
22.17. key mappings	228
22.18. setting options	228
22.19. practice: vi(m)	229
22.20. solution: vi(m)	230

Chapter 22. Introduction to vi

The **vi** editor is installed on almost every Unix. Linux will very often install **vim** (**vi improved**) which is similar. Every system administrator should know **vi(m)**, because it is an easy tool to solve problems.

The **vi** editor is not intuitive, but once you get to know it, **vi** becomes a very powerful application. Most Linux distributions will include the **vimtutor** which is a 45 minute lesson in **vi(m)**.

22.1. command mode and insert mode

The vi editor starts in **command mode**. In command mode, you can type commands. Some commands will bring you into **insert mode**. In insert mode, you can type text. The **escape key** will return you to command mode.

Table 22.1. getting to command mode

key	action
Esc	set vi(m) in command mode.

22.2. start typing (a A i I o O)

The difference between a A i I o and O is the location where you can start typing. a will append after the current character and A will append at the end of the line. i will insert before the current character and I will insert at the beginning of the line. o will put you in a new line after the current line and O will put you in a new line before the current line.

Table 22.2. switch to insert mode

command	action
a	start typing after the current character
A	start typing at the end of the current line
i	start typing before the current character
I	start typing at the start of the current line
o	start typing on a new line after the current line
O	start typing on a new line before the current line

22.3. replace and delete a character (r x X)

When in command mode (it doesn't hurt to hit the escape key more than once) you can use the x key to delete the current character. The big X key (or shift x) will delete the character left of the cursor. Also when in command mode, you can use the r key to replace one single character. The r key will bring you in insert mode for just one key press, and will return you immediately to command mode.

Table 22.3. replace and delete

command	action
x	delete the character below the cursor
X	delete the character before the cursor
r	replace the character below the cursor
p	paste after the cursor (here the last deleted character)
xp	switch two characters

22.4. undo and repeat (u .)

When in command mode, you can undo your mistakes with u. You can do your mistakes twice with . (in other words, the . will repeat your last command).

Table 22.4. undo and repeat

command	action
u	undo the last action
.	repeat the last action

22.5. cut, copy and paste a line (dd yy p P)

When in command mode, dd will cut the current line. yy will copy the current line. You can paste the last copied or cut line after (p) or before (P) the current line.

Table 22.5. cut, copy and paste a line

command	action
dd	cut the current line
yy	(yank yank) copy the current line
p	paste after the current line
P	paste before the current line

22.6. cut, copy and paste lines (3dd 2yy)

When in command mode, before typing `dd` or `yy`, you can type a number to repeat the command a number of times. Thus, `5dd` will cut 5 lines and `4yy` will copy (yank) 4 lines. That last one will be noted by vi in the bottom left corner as "4 line yanked".

Table 22.6. cut, copy and paste lines

command	action
<code>3dd</code>	cut three lines
<code>4yy</code>	copy four lines

22.7. start and end of a line (0 or ^ and \$)

When in command mode, the `0` and the caret `^` will bring you to the start of the current line, whereas the `$` will put the cursor at the end of the current line. You can add `0` and `$` to the `d` command, `d0` will delete every character between the current character and the start of the line. Likewise `d$` will delete everything from the current character till the end of the line. Similarly `y0` and `y$` will yank till start and end of the current line.

Table 22.7. start and end of line

command	action
<code>0</code>	jump to start of current line
<code>^</code>	jump to start of current line
<code>\$</code>	jump to end of current line
<code>d0</code>	delete until start of line
<code>d\$</code>	delete until end of line

22.8. join two lines (J) and more

When in command mode, pressing `J` will append the next line to the current line. With `yy` you duplicate a line and with `ddp` you switch two lines.

Table 22.8. join two lines

command	action
<code>J</code>	join two lines
<code>yy</code>	duplicate a line
<code>ddp</code>	switch two lines

22.9. words (w b)

When in command mode, **w** will jump to the next word and **b** will move to the previous word. **w** and **b** can also be combined with **d** and **y** to copy and cut words (**dw db yw yb**).

Table 22.9. words

command	action
w	forward one word
b	back one word
3w	forward three words
dw	delete one word
yw	yank (copy) one word
5yb	yank five words back
7dw	delete seven words

22.10. save (or not) and exit (:w :q :q!)

Pressing the colon **:** will allow you to give instructions to **vi** (technically speaking, typing the colon will open the **ex** editor). **:w** will write (save) the file, **:q** will quit an unchanged file without saving, and **:q!** will quit **vi** discarding any changes. **:wq** will save and quit and is the same as typing **ZZ** in command mode.

Table 22.10. save and exit vi

command	action
:w	save (write)
:w fname	save as fname
:q	quit
:wq	save and quit
ZZ	save and quit
:q!	quit (discarding your changes)
:w!	save (and write to non-writable file!)

The last one is a bit special. With **:w!** **vi** will try to **chmod** the file to get write permission (this works when you are the owner) and will **chmod** it back when the write succeeds. This should always work when you are root (and the file system is writable).

22.11. Searching (/ ?)

When in command mode typing **/** will allow you to search in **vi** for strings (can be a regular expression). Typing **/foo** will do a forward search for the string **foo** and typing **?bar** will do a backward search for **bar**.

Table 22.11. searching

command	action
/string	forward search for string

command	action
?string	backward search for string
n	go to next occurrence of search string
/^string	forward search string at beginning of line
/string\$	forward search string at end of line
/br[aeio]l	search for bral brel bril and brol
^\<he\>	search for the word he (and not for here or the)

22.12. replace all (:1,\$ s/foo/bar/g)

To replace all occurrences of the string foo with bar, first switch to ex mode with : . Then tell vi which lines to use, for example 1,\$ will do the replace all from the first to the last line. You can write 1,5 to only process the first five lines. The s/foo/bar/g will replace all occurrences of foo with bar.

Table 22.12. replace

command	action
:4,8 s/foo/bar/g	replace foo with bar on lines 4 to 8
:1,\$ s/foo/bar/g	replace foo with bar on all lines

22.13. reading files (:r :r !cmd)

When in command mode, :r foo will read the file named foo, :r !foo will execute the command foo. The result will be put at the current location. Thus :r !ls will put a listing of the current directory in your text file.

Table 22.13. read files and input

command	action
:r fname	(read) file fname and paste contents
:r !cmd	execute cmd and paste its output

22.14. text buffers

There are 36 buffers in vi to store text. You can use them with the " character.

Table 22.14. text buffers

command	action
"add	delete current line and put text in buffer a
"g7yy	copy seven lines into buffer g
"ap	paste from buffer a

22.15. multiple files

You can edit multiple files with vi. Here are some tips.

Table 22.15. multiple files

command	action
vi file1 file2 file3	start editing three files
:args	lists files and marks active file
:n	start editing the next file
:e	toggle with last edited file
:rew	rewind file pointer to first file

22.16. abbreviations

With **:ab** you can put abbreviations in vi. Use **:una** to undo the abbreviation.

Table 22.16. abbreviations

command	action
:ab str long string	abbreviate str to be 'long string'
:una str	un-abbreviate str

22.17. key mappings

Similarly to their abbreviations, you can use mappings with **:map** for command mode and **:map!** for insert mode.

This example shows how to set the F6 function key to toggle between **set number** and **set nonumber**. The <bar> separates the two commands, **set number!** toggles the state and **set number?** reports the current state.

```
:map <F6> :set number!<bar>set number?<CR>
```

22.18. setting options

Some options that you can set in vim.

```
:set number ( also try :se nu )
:set nonumber
:syntax on
:syntax off
:set all (list all options)
:set tabstop=8
:set tx (CR/LF style endings)
:set notx
```

You can set these options (and much more) in **~/.vimrc** for vim or in **~/.exrc** for standard vi.

```
paul@barry:~$ cat ~/.vimrc
set number
set tabstop=8
set textwidth=78
map <F6> :set number!<bar>set number?<CR>
paul@barry:~$
```

22.19. practice: vi(m)

1. Start the vimtutor and do some or all of the exercises. You might need to run **aptitude install vim** on xubuntu.
2. What 3 key sequence in command mode will duplicate the current line.
3. What 3 key sequence in command mode will switch two lines' place (line five becomes line six and line six becomes line five).
4. What 2 key sequence in command mode will switch a character's place with the next one.
5. vi can understand macro's. A macro can be recorded with q followed by the name of the macro. So qa will record the macro named a. Pressing q again will end the recording. You can recall the macro with @ followed by the name of the macro. Try this example: i 1 'Escape Key' qa yyp 'Ctrl a' q 5@a (Ctrl a will increase the number with one).
6. Copy /etc/passwd to your ~/passwd. Open the last one in vi and press Ctrl v. Use the arrow keys to select a Visual Block, you can copy this with y or delete it with d. Try pasting it.
7. What does dwwP do when you are at the beginning of a word in a sentence ?

22.20. solution: vi(m)

1. Start the vimtutor and do some or all of the exercises. You might need to run **aptitude install vim** on xubuntu.

```
vimtutor
```

2. What 3 key sequence in command mode will duplicate the current line.

```
yyp
```

3. What 3 key sequence in command mode will switch two lines' place (line five becomes line six and line six becomes line five).

```
ddp
```

4. What 2 key sequence in command mode will switch a character's place with the next one.

```
xp
```

5. vi can understand macro's. A macro can be recorded with q followed by the name of the macro. So qa will record the macro named a. Pressing q again will end the recording. You can recall the macro with @ followed by the name of the macro. Try this example: i 1 'Escape Key' qa yyp 'Ctrl a' q 5@a (Ctrl a will increase the number with one).

6. Copy /etc/passwd to your ~/passwd. Open the last one in vi and press Ctrl v. Use the arrow keys to select a Visual Block, you can copy this with y or delete it with d. Try pasting it.

```
cp /etc/passwd ~  
vi passwd  
(press Ctrl-V)
```

7. What does **dwwP** do when you are at the beginning of a word in a sentence ?

dwwP can switch the current word with the next word.

Part VII. scripting

Table of Contents

23. scripting introduction	233
23.1. prerequisites	234
23.2. hello world	234
23.3. she-bang	234
23.4. comment	235
23.5. variables	235
23.6. sourcing a script	235
23.7. troubleshooting a script	236
23.8. prevent setuid root spoofing	236
23.9. practice: introduction to scripting	237
23.10. solution: introduction to scripting	238
24. scripting loops	239
24.1. test []	240
24.2. if then else	241
24.3. if then elif	241
24.4. for loop	241
24.5. while loop	242
24.6. until loop	242
24.7. practice: scripting tests and loops	243
24.8. solution: scripting tests and loops	244
25. scripting parameters	246
25.1. script parameters	247
25.2. shift through parameters	248
25.3. runtime input	248
25.4. sourcing a config file	249
25.5. get script options with getopts	250
25.6. get shell options with shopt	251
25.7. practice: parameters and options	252
25.8. solution: parameters and options	253
26. more scripting	254
26.1. eval	255
26.2. (())	255
26.3. let	256
26.4. case	257
26.5. shell functions	258
26.6. practice : more scripting	259
26.7. solution : more scripting	260

Chapter 23. scripting introduction

Shells like **bash** and **Korn** have support for programming constructs that can be saved as **scripts**. These **scripts** in turn then become more **shell** commands. Many Linux commands are **scripts**. **User profile scripts** are run when a user logs on and **init scripts** are run when a **daemon** is stopped or started.

This means that system administrators also need basic knowledge of **scripting** to understand how their servers and their applications are started, updated, upgraded, patched, maintained, configured and removed, and also to understand how a user environment is built.

The goal of this chapter is to give you enough information to be able to read and understand scripts. Not to become a writer of complex scripts.

23.1. prerequisites

You should have read and understood **part III shell expansion** and **part IV pipes and commands** before starting this chapter.

23.2. hello world

Just like in every programming course, we start with a simple **hello_world** script. The following script will output **Hello World**.

```
echo Hello World
```

After creating this simple script in **vi** or with **echo**, you'll have to **chmod +x hello_world** to make it executable. And unless you add the scripts directory to your path, you'll have to type the path to the script for the shell to be able to find it.

```
[paul@RHEL4a ~]$ echo echo Hello World > hello_world
[paul@RHEL4a ~]$ chmod +x hello_world
[paul@RHEL4a ~]$ ./hello_world
Hello World
[paul@RHEL4a ~]$
```

23.3. she-bang

Let's expand our example a little further by putting **#!/bin/bash** on the first line of the script. The **#!** is called a **she-bang** (sometimes called **sha-bang**), where the **she-bang** is the first two characters of the script.

```
#!/bin/bash
echo Hello World
```

You can never be sure which shell a user is running. A script that works flawlessly in **bash** might not work in **ksh**, **csh**, or **dash**. To instruct a shell to run your script in a certain shell, you can start your script with a **she-bang** followed by the shell it is supposed to run in. This script will run in a bash shell.

```
#!/bin/bash
echo -n hello
echo A bash subshell `echo -n hello`
```

This script will run in a Korn shell (unless **/bin/ksh** is a hard link to **/bin/bash**). The **/etc/shells** file contains a list of shells on your system.

```
#!/bin/ksh
echo -n hello
echo a Korn subshell `echo -n hello`
```

23.4. comment

Let's expand our example a little further by adding comment lines.

```
#!/bin/bash
#
# Hello World Script
#
echo Hello World
```

23.5. variables

Here is a simple example of a variable inside a script.

```
#!/bin/bash
#
# simple variable in script
#
var1=4
echo var1 = $var1
```

Scripts can contain variables, but since scripts are run in their own shell, the variables do not survive the end of the script.

```
[paul@RHEL4a ~]$ echo $var1

[paul@RHEL4a ~]$ ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1

[paul@RHEL4a ~]$
```

23.6. sourcing a script

Luckily, you can force a script to run in the same shell; this is called **sourcing** a script.

```
[paul@RHEL4a ~]$ source ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1
4
[paul@RHEL4a ~]$
```

The above is identical to the below.

```
[paul@RHEL4a ~]$ . ./vars
var1 = 4
[paul@RHEL4a ~]$ echo $var1
4
[paul@RHEL4a ~]$
```

23.7. troubleshooting a script

Another way to run a script in a separate shell is by typing **bash** with the name of the script as a parameter.

```
paul@debian6~/test$ bash runme
42
```

Expanding this to **bash -x** allows you to see the commands that the shell is executing (after shell expansion).

```
paul@debian6~/test$ bash -x runme
+ var4=42
+ echo 42
42
paul@debian6~/test$ cat runme
# the runme script
var4=42
echo $var4
paul@debian6~/test$
```

Notice the absence of the commented (#) line, and the replacement of the variable before execution of **echo**.

23.8. prevent setuid root spoofing

Some user may try to perform **setuid** based script **root spoofing**. This is a rare but possible attack. To improve script security and to avoid interpreter spoofing, you need to add **--** after the **#!/bin/bash**, which disables further option processing so the shell will not accept any options.

```
#!/bin/bash -
or
#!/bin/bash --
```

Any arguments after the **--** are treated as filenames and arguments. An argument of **-** is equivalent to **--**.

23.9. practice: introduction to scripting

0. Give each script a different name, keep them for later!
1. Write a script that outputs the name of a city.
2. Make sure the script runs in the bash shell.
3. Make sure the script runs in the Korn shell.
4. Create a script that defines two variables, and outputs their value.
5. The previous script does not influence your current shell (the variables do not exist outside of the script). Now run the script so that it influences your current shell.
6. Is there a shorter way to **source** the script ?
7. Comment your scripts so that you know what they are doing.

23.10. solution: introduction to scripting

0. Give each script a different name, keep them for later!

1. Write a script that outputs the name of a city.

```
$ echo 'echo Antwerp' > first.bash
$ chmod +x first.bash
$ ./first.bash
Antwerp
```

2. Make sure the script runs in the bash shell.

```
$ cat first.bash
#!/bin/bash
echo Antwerp
```

3. Make sure the script runs in the Korn shell.

```
$ cat first.bash
#!/bin/ksh
echo Antwerp
```

Note that while `first.bash` will technically work as a Korn shell script, the name ending in `.bash` is confusing.

4. Create a script that defines two variables, and outputs their value.

```
$ cat second.bash
#!/bin/bash

var33=300
var42=400

echo $var33 $var42
```

5. The previous script does not influence your current shell (the variables do not exist outside of the script). Now run the script so that it influences your current shell.

```
source second.bash
```

6. Is there a shorter way to **source** the script ?

```
. ./second.bash
```

7. Comment your scripts so that you know what they are doing.

```
$ cat second.bash
#!/bin/bash
# script to test variables and sourcing

# define two variables
var33=300
var42=400

# output the value of these variables
echo $var33 $var42
```

Chapter 24. scripting loops

24.1. test []

The **test** command can test whether something is true or false. Let's start by testing whether 10 is greater than 55.

```
[paul@RHEL4b ~]$ test 10 -gt 55 ; echo $?
1
[paul@RHEL4b ~]$
```

The test command returns 1 if the test fails. And as you see in the next screenshot, test returns 0 when a test succeeds.

```
[paul@RHEL4b ~]$ test 56 -gt 55 ; echo $?
0
[paul@RHEL4b ~]$
```

If you prefer true and false, then write the test like this.

```
[paul@RHEL4b ~]$ test 56 -gt 55 && echo true || echo false
true
[paul@RHEL4b ~]$ test 6 -gt 55 && echo true || echo false
false
```

The test command can also be written as square brackets, the screenshot below is identical to the one above.

```
[paul@RHEL4b ~]$ [ 56 -gt 55 ] && echo true || echo false
true
[paul@RHEL4b ~]$ [ 6 -gt 55 ] && echo true || echo false
false
```

Below are some example tests. Take a look at **man test** to see more options for tests.

[-d foo]	Does the directory foo exist ?
[-e bar]	Does the file bar exist ?
['/etc' = \$PWD]	Is the string /etc equal to the variable \$PWD ?
[\$1 != 'secret']	Is the first parameter different from secret ?
[55 -lt \$bar]	Is 55 less than the value of \$bar ?
[\$foo -ge 1000]	Is the value of \$foo greater or equal to 1000 ?
["abc" < \$bar]	Does abc sort before the value of \$bar ?
[-f foo]	Is foo a regular file ?
[-r bar]	Is bar a readable file ?
[foo -nt bar]	Is file foo newer than file bar ?
[-o nounset]	Is the shell option nounset set ?

Tests can be combined with logical AND and OR.

```
paul@RHEL4b:~$ [ 66 -gt 55 -a 66 -lt 500 ] && echo true || echo false
true
paul@RHEL4b:~$ [ 66 -gt 55 -a 660 -lt 500 ] && echo true || echo false
false
paul@RHEL4b:~$ [ 66 -gt 55 -o 660 -lt 500 ] && echo true || echo false
true
```


24.2. if then else

The **if then else** construction is about choice. If a certain condition is met, then execute something, else execute something else. The example below tests whether a file exists, and if the file exists then a proper message is echoed.

```
#!/bin/bash

if [ -f isit.txt ]
then echo isit.txt exists!
else echo isit.txt not found!
fi
```

If we name the above script 'choice', then it executes like this.

```
[paul@RHEL4a scripts]$ ./choice
isit.txt not found!
[paul@RHEL4a scripts]$ touch isit.txt
[paul@RHEL4a scripts]$ ./choice
isit.txt exists!
[paul@RHEL4a scripts]$
```

24.3. if then elif

You can nest a new **if** inside an **else** with **elif**. This is a simple example.

```
#!/bin/bash
count=42
if [ $count -eq 42 ]
then
    echo "42 is correct."
elif [ $count -gt 42 ]
then
    echo "Too much."
else
    echo "Not enough."
fi
```

24.4. for loop

The example below shows the syntax of a classical **for loop** in bash.

```
for i in 1 2 4
do
    echo $i
done
```

An example of a **for loop** combined with an embedded shell.

```
#!/bin/ksh
for counter in `seq 1 20`
do
    echo counting from 1 to 20, now at $counter
    sleep 1
done
```

The same example as above can be written without the embedded shell using the bash **{from..to}** shorthand.

```
#!/bin/bash
for counter in {1..20}
do
    echo counting from 1 to 20, now at $counter
    sleep 1
done
```

This **for loop** uses file globbing (from the shell expansion). Putting the instruction on the command line has identical functionality.

```
kahlan@solexp11$ ls
count.ksh  go.ksh
kahlan@solexp11$ for file in *.ksh ; do cp $file $file.backup ; done
kahlan@solexp11$ ls
count.ksh  count.ksh.backup  go.ksh  go.ksh.backup
```

24.5. while loop

Below a simple example of a **while loop**.

```
i=100;
while [ $i -ge 0 ] ;
do
    echo Counting down, from 100 to 0, now at $i;
    let i--;
done
```

Endless loops can be made with **while true** or **while :**, where the **colon** is the equivalent of **no operation** in the **Korn** and **bash** shells.

```
#!/bin/ksh
# endless loop
while :
do
    echo hello
    sleep 1
done
```

24.6. until loop

Below a simple example of an **until loop**.

```
let i=100;
until [ $i -le 0 ] ;
do
    echo Counting down, from 100 to 1, now at $i;
    let i--;
done
```

24.7. practice: scripting tests and loops

1. Write a script that uses a **for** loop to count from 3 to 7.
2. Write a script that uses a **for** loop to count from 1 to 17000.
3. Write a script that uses a **while** loop to count from 3 to 7.
4. Write a script that uses an **until** loop to count down from 8 to 4.
5. Write a script that counts the number of files ending in **.txt** in the current directory.
6. Wrap an **if** statement around the script so it is also correct when there are zero files ending in **.txt**.

24.8. solution: scripting tests and loops

1. Write a script that uses a **for** loop to count from 3 to 7.

```
#!/bin/bash

for i in 3 4 5 6 7
do
    echo Counting from 3 to 7, now at $i
done
```

2. Write a script that uses a **for** loop to count from 1 to 17000.

```
#!/bin/bash

for i in `seq 1 17000`
do
    echo Counting from 1 to 17000, now at $i
done
```

3. Write a script that uses a **while** loop to count from 3 to 7.

```
#!/bin/bash

i=3
while [ $i -le 7 ]
do
    echo Counting from 3 to 7, now at $i
    let i=i+1
done
```

4. Write a script that uses an **until** loop to count down from 8 to 4.

```
#!/bin/bash

i=8
until [ $i -lt 4 ]
do
    echo Counting down from 8 to 4, now at $i
    let i=i-1
done
```

5. Write a script that counts the number of files ending in **.txt** in the current directory.

```
#!/bin/bash

let i=0
for file in *.txt
do
    let i++
done
echo "There are $i files ending in .txt"
```

6. Wrap an **if** statement around the script so it is also correct when there are zero files ending in **.txt**.

```
#!/bin/bash

ls *.txt > /dev/null 2>&1
if [ $? -ne 0 ]
```

```
then echo "There are 0 files ending in .txt"
else
  let i=0
  for file in *.txt
  do
    let i++
  done
  echo "There are $i files ending in .txt"
fi
```

Chapter 25. scripting parameters

25.1. script parameters

A **bash** shell script can have parameters. The numbering you see in the script below continues if you have more parameters. You also have special parameters containing the number of parameters, a string of all of them, and also the process id, and the last return code. The man page of **bash** has a full list.

```
#!/bin/bash
echo The first argument is $1
echo The second argument is $2
echo The third argument is $3

echo \$ $$ PID of the script
echo \# $# count arguments
echo \? $? last return code
echo \* $* all the arguments
```

Below is the output of the script above in action.

```
[paul@RHEL4a scripts]$ ./pars one two three
The first argument is one
The second argument is two
The third argument is three
$ 5610 PID of the script
# 3 count arguments
? 0 last return code
* one two three all the arguments
```

Once more the same script, but with only two parameters.

```
[paul@RHEL4a scripts]$ ./pars 1 2
The first argument is 1
The second argument is 2
The third argument is
$ 5612 PID of the script
# 2 count arguments
? 0 last return code
* 1 2 all the arguments
[paul@RHEL4a scripts]$
```

Here is another example, where we use **\$0**. The **\$0** parameter contains the name of the script.

```
paul@debian6-$ cat myname
echo this script is called $0
paul@debian6-$ ./myname
this script is called ./myname
paul@debian6-$ mv myname test42
paul@debian6-$ ./test42
this script is called ./test42
```

25.2. shift through parameters

The **shift** statement can parse all **parameters** one by one. This is a sample script.

```
kahlan@solexp11$ cat shift.ksh
#!/bin/ksh

if [ "$#" == "0" ]
then
    echo You have to give at least one parameter.
    exit 1
fi

while (( $# ))
do
    echo You gave me $1
    shift
done
```

Below is some sample output of the script above.

```
kahlan@solexp11$ ./shift.ksh one
You gave me one
kahlan@solexp11$ ./shift.ksh one two three 1201 "33 42"
You gave me one
You gave me two
You gave me three
You gave me 1201
You gave me 33 42
kahlan@solexp11$ ./shift.ksh
You have to give at least one parameter.
```

25.3. runtime input

You can ask the user for input with the **read** command in a script.

```
#!/bin/bash
echo -n Enter a number:
read number
```


25.4. sourcing a config file

The **source** (as seen in the shell chapters) can be used to source a configuration file.

Below a sample configuration file for an application.

```
[paul@RHEL4a scripts]$ cat myApp.conf
# The config file of myApp

# Enter the path here
myAppPath=/var/myApp

# Enter the number of quines here
quines=5
```

And here an application that uses this file.

```
[paul@RHEL4a scripts]$ cat myApp.bash
#!/bin/bash
#
# Welcome to the myApp application
#

. ./myApp.conf

echo There are $quines quines
```

The running application can use the values inside the sourced configuration file.

```
[paul@RHEL4a scripts]$ ./myApp.bash
There are 5 quines
[paul@RHEL4a scripts]$
```

25.5. get script options with getopt

The **getopts** function allows you to parse options given to a command. The following script allows for any combination of the options a, f and z.

```
kahlan@solexp11$ cat options.ksh
#!/bin/ksh

while getopts ":afz" option;
do
  case $option in
    a)
      echo received -a
      ;;
    f)
      echo received -f
      ;;
    z)
      echo received -z
      ;;
    *)
      echo "invalid option -$OPTARG"
      ;;
  esac
done
```

This is sample output from the script above. First we use correct options, then we enter twice an invalid option.

```
kahlan@solexp11$ ./options.ksh
kahlan@solexp11$ ./options.ksh -af
received -a
received -f
kahlan@solexp11$ ./options.ksh -zfg
received -z
received -f
invalid option -g
kahlan@solexp11$ ./options.ksh -a -b -z
received -a
invalid option -b
received -z
```

You can also check for options that need an argument, as this example shows.

```
kahlan@solexp11$ cat argoptions.ksh
#!/bin/ksh

while getopts ":af:z" option;
do
  case $option in
    a)
      echo received -a
      ;;
    f)
      echo received -f with $OPTARG
      ;;
    z)
      echo received -z
      ;;
    :)
      echo "option -$OPTARG needs an argument"
      ;;
    *)
      echo "invalid option -$OPTARG"
      ;;
  esac
done
```

This is sample output from the script above.

```
kahlan@solexp11$ ./argoptions.ksh -a -f hello -z
received -a
received -f with hello
received -z
kahlan@solexp11$ ./argoptions.ksh -zaf 42
received -z
received -a
received -f with 42
kahlan@solexp11$ ./argoptions.ksh -zf
received -z
option -f needs an argument
```

25.6. get shell options with shopt

You can toggle the values of variables controlling optional shell behaviour with the **shopt** built-in shell command. The example below first verifies whether the `cdspell` option is set; it is not. The next `shopt` command sets the value, and the third `shopt` command verifies that the option really is set. You can now use minor spelling mistakes in the `cd` command. The man page of `bash` has a complete list of options.

```
paul@laika:~$ shopt -q cdspell ; echo $?
1
paul@laika:~$ shopt -s cdspell
paul@laika:~$ shopt -q cdspell ; echo $?
0
paul@laika:~$ cd /Etc
/etc
```

25.7. practice: parameters and options

1. Write a script that receives four parameters, and outputs them in reverse order.
2. Write a script that receives two parameters (two filenames) and outputs whether those files exist.
3. Write a script that asks for a filename. Verify existence of the file, then verify that you own the file, and whether it is writable. If not, then make it writable.
4. Make a configuration file for the previous script. Put a logging switch in the config file, logging means writing detailed output of everything the script does to a log file in /tmp.

25.8. solution: parameters and options

1. Write a script that receives four parameters, and outputs them in reverse order.

```
echo $4 $3 $2 $1
```

2. Write a script that receives two parameters (two filenames) and outputs whether those files exist.

```
#!/bin/bash

if [ -f $1 ]
then echo $1 exists!
else echo $1 not found!
fi

if [ -f $2 ]
then echo $2 exists!
else echo $2 not found!
fi
```

3. Write a script that asks for a filename. Verify existence of the file, then verify that you own the file, and whether it is writable. If not, then make it writable.

4. Make a configuration file for the previous script. Put a logging switch in the config file, logging means writing detailed output of everything the script does to a log file in /tmp.

Chapter 26. more scripting

26.1. eval

eval reads arguments as input to the shell (the resulting commands are executed). This allows using the value of a variable as a variable.

```
paul@deb503:~/test42$ answer=42
paul@deb503:~/test42$ word=answer
paul@deb503:~/test42$ eval x=\$$word ; echo $x
42
```

Both in **bash** and **Korn** the arguments can be quoted.

```
kahlan@solexp11$ answer=42
kahlan@solexp11$ word=answer
kahlan@solexp11$ eval "y=\$$word" ; echo $y
42
```

Sometimes the **eval** is needed to have correct parsing of arguments. Consider this example where the **date** command receives one parameter **1 week ago**.

```
paul@debian6~$ date --date="1 week ago"
Thu Mar  8 21:36:25 CET 2012
```

When we set this command in a variable, then executing that variable fails unless we use **eval**.

```
paul@debian6~$ lastweek='date --date="1 week ago"'
paul@debian6~$ $lastweek
date: extra operand `ago'
Try `date --help' for more information.
paul@debian6~$ eval $lastweek
Thu Mar  8 21:36:39 CET 2012
```

26.2. (())

The **(())** allows for evaluation of numerical expressions.

```
paul@deb503:~/test42$ (( 42 > 33 )) && echo true || echo false
true
paul@deb503:~/test42$ (( 42 > 1201 )) && echo true || echo false
false
paul@deb503:~/test42$ var42=42
paul@deb503:~/test42$ (( 42 == var42 )) && echo true || echo false
true
paul@deb503:~/test42$ (( 42 == $var42 )) && echo true || echo false
true
paul@deb503:~/test42$ var42=33
paul@deb503:~/test42$ (( 42 == var42 )) && echo true || echo false
false
```

26.3. let

The **let** built-in shell function instructs the shell to perform an evaluation of arithmetic expressions. It will return 0 unless the last arithmetic expression evaluates to 0.

```
[paul@RHEL4b ~]$ let x="3 + 4" ; echo $x
7
[paul@RHEL4b ~]$ let x="10 + 100/10" ; echo $x
20
[paul@RHEL4b ~]$ let x="10-2+100/10" ; echo $x
18
[paul@RHEL4b ~]$ let x="10*2+100/10" ; echo $x
30
```

The **shell** can also convert between different bases.

```
[paul@RHEL4b ~]$ let x="0xFF" ; echo $x
255
[paul@RHEL4b ~]$ let x="0xC0" ; echo $x
192
[paul@RHEL4b ~]$ let x="0xA8" ; echo $x
168
[paul@RHEL4b ~]$ let x="8#70" ; echo $x
56
[paul@RHEL4b ~]$ let x="8#77" ; echo $x
63
[paul@RHEL4b ~]$ let x="16#c0" ; echo $x
192
```

There is a difference between assigning a variable directly, or using **let** to evaluate the arithmetic expressions (even if it is just assigning a value).

```
kahlan@solexp11$ dec=15 ; oct=017 ; hex=0x0f
kahlan@solexp11$ echo $dec $oct $hex
15 017 0x0f
kahlan@solexp11$ let dec=15 ; let oct=017 ; let hex=0x0f
kahlan@solexp11$ echo $dec $oct $hex
15 15 15
```


26.4. case

You can sometimes simplify nested if statements with a **case** construct.

```
[paul@RHEL4b ~]$ ./help
What animal did you see ? lion
You better start running fast!
[paul@RHEL4b ~]$ ./help
What animal did you see ? dog
Don't worry, give it a cookie.
[paul@RHEL4b ~]$ cat help
#!/bin/bash
#
# Wild Animals Helpdesk Advice
#
echo -n "What animal did you see ? "
read animal
case $animal in
    "lion" | "tiger")
        echo "You better start running fast!"
        ;;
    "cat")
        echo "Let that mouse go..."
        ;;
    "dog")
        echo "Don't worry, give it a cookie."
        ;;
    "chicken" | "goose" | "duck" )
        echo "Eggs for breakfast!"
        ;;
    "liger")
        echo "Approach and say 'Ah you big fluffy kitty...'"
        ;;
    "babelfish")
        echo "Did it fall out your ear ?"
        ;;
    *)
        echo "You discovered an unknown animal, name it!"
        ;;
esac
[paul@RHEL4b ~]$
```

26.5. shell functions

Shell **functions** can be used to group commands in a logical way.

```
kahlan@solexp11$ cat funcs.ksh
#!/bin/ksh

function greetings {
echo Hello World!
echo and hello to $USER to!
}

echo We will now call a function
greetings
echo The end
```

This is sample output from this script with a **function**.

```
kahlan@solexp11$ ./funcs.ksh
We will now call a function
Hello World!
and hello to kahlan to!
The end
```

A shell function can also receive parameters.

```
kahlan@solexp11$ cat addfunc.ksh
#!/bin/ksh

function plus {
let result="$1 + $2"
echo $1 + $2 = $result
}

plus 3 10
plus 20 13
plus 20 22
```

This script produces the following output.

```
kahlan@solexp11$ ./addfunc.ksh
3 + 10 = 13
20 + 13 = 33
20 + 22 = 42
```

26.6. practice : more scripting

1. Write a script that asks for two numbers, and outputs the sum and product (as shown here).

```
Enter a number: 5
Enter another number: 2

Sum:      5 + 2 = 7
Product:   5 x 2 = 10
```

2. Improve the previous script to test that the numbers are between 1 and 100, exit with an error if necessary.
3. Improve the previous script to congratulate the user if the sum equals the product.
4. Write a script with a case insensitive case statement, using the shopt nocasematch option. The nocasematch option is reset to the value it had before the scripts started.
5. If time permits (or if you are waiting for other students to finish this practice), take a look at Linux system scripts in /etc/init.d and /etc/rc.d and try to understand them. Where does execution of a script start in /etc/init.d/samba ? There are also some hidden scripts in ~, we will discuss them later.

26.7. solution : more scripting

1. Write a script that asks for two numbers, and outputs the sum and product (as shown here).

```
Enter a number: 5
Enter another number: 2

Sum:      5 + 2 = 7
Product:  5 x 2 = 10
```

```
#!/bin/bash

echo -n "Enter a number : "
read n1

echo -n "Enter another number : "
read n2

let sum="$n1+$n2"
let pro="$n1*$n2"

echo -e "Sum\t: $n1 + $n2 = $sum"
echo -e "Product\t: $n1 * $n2 = $pro"
```

2. Improve the previous script to test that the numbers are between 1 and 100, exit with an error if necessary.

```
echo -n "Enter a number between 1 and 100 : "
read n1

if [ $n1 -lt 1 -o $n1 -gt 100 ]
then
    echo Wrong number...
    exit 1
fi
```

3. Improve the previous script to congratulate the user if the sum equals the product.

```
if [ $sum -eq $pro ]
then echo Congratulations $sum == $pro
fi
```

4. Write a script with a case insensitive case statement, using the shopt nocasematch option. The nocasematch option is reset to the value it had before the scripts started.

```
#!/bin/bash
#
# Wild Animals Case Insensitive Helpdesk Advice
#

if shopt -q nocasematch; then
    nocase=yes;
else
    nocase=no;
    shopt -s nocasematch;
fi

echo -n "What animal did you see ? "
read animal

case $animal in
```

```
"lion" | "tiger")
    echo "You better start running fast!"
;;
"cat")
    echo "Let that mouse go..."
;;
"dog")
    echo "Don't worry, give it a cookie."
;;
"chicken" | "goose" | "duck" )
    echo "Eggs for breakfast!"
;;
"liger")
    echo "Approach and say 'Ah you big fluffy kitty.'"
;;
"babelfish")
    echo "Did it fall out your ear ?"
;;
*)
    echo "You discovered an unknown animal, name it!"
;;
esac

if [ nocase = yes ] ; then
    shopt -s nocasematch;
else
    shopt -u nocasematch;
fi
```

5. If time permits (or if you are waiting for other students to finish this practice), take a look at Linux system scripts in `/etc/init.d` and `/etc/rc.d` and try to understand them. Where does execution of a script start in `/etc/init.d/samba` ? There are also some hidden scripts in `~`, we will discuss them later.

Part VIII. local user management

Table of Contents

27. introduction to users	265
27.1. whoami	266
27.2. who	266
27.3. who am i	266
27.4. w	266
27.5. id	266
27.6. su to another user	267
27.7. su to root	267
27.8. su as root	267
27.9. su - \$username	267
27.10. su -	267
27.11. run a program as another user	268
27.12. visudo	268
27.13. sudo su -	269
27.14. sudo logging	269
27.15. practice: introduction to users	270
27.16. solution: introduction to users	271
28. user management	273
28.1. user management	274
28.2. /etc/passwd	274
28.3. root	274
28.4. useradd	275
28.5. /etc/default/useradd	275
28.6. userdel	275
28.7. usermod	275
28.8. creating home directories	276
28.9. /etc/skel/	276
28.10. deleting home directories	276
28.11. login shell	277
28.12. chsh	277
28.13. practice: user management	278
28.14. solution: user management	279
29. user passwords	281
29.1. passwd	282
29.2. shadow file	282
29.3. encryption with passwd	283
29.4. encryption with openssl	283
29.5. encryption with crypt	284
29.6. /etc/login.defs	285
29.7. chage	285
29.8. disabling a password	286
29.9. editing local files	286
29.10. practice: user passwords	287
29.11. solution: user passwords	288
30. user profiles	290
30.1. system profile	291
30.2. ~/.bash_profile	291
30.3. ~/.bash_login	292
30.4. ~/.profile	292
30.5. ~/.bashrc	292
30.6. ~/.bash_logout	293
30.7. Debian overview	294
30.8. RHEL5 overview	294
30.9. practice: user profiles	295
30.10. solution: user profiles	296

31. groups	297
31.1. groupadd	298
31.2. group file	298
31.3. groups	298
31.4. usermod	299
31.5. groupmod	299
31.6. groupdel	299
31.7. gpasswd	300
31.8. newgrp	301
31.9. vigr	301
31.10. practice: groups	302
31.11. solution: groups	303

Chapter 27. introduction to users

This little chapter will teach you how to identify your user account on a Unix computer using commands like **who am i**, **id**, and more.

In a second part you will learn how to become another user with the **su** command.

And you will learn how to run a program as another user with **sudo**.

27.1. whoami

The **whoami** command tells you your username.

```
[paul@centos7 ~]$ whoami
paul
[paul@centos7 ~]$
```

27.2. who

The **who** command will give you information about who is logged on the system.

```
[paul@centos7 ~]$ who
root      pts/0      2014-10-10 23:07 (10.104.33.101)
paul      pts/1      2014-10-10 23:30 (10.104.33.101)
laura     pts/2      2014-10-10 23:34 (10.104.33.96)
tania     pts/3      2014-10-10 23:39 (10.104.33.91)
[paul@centos7 ~]$
```

27.3. who am i

With **who am i** the **who** command will display only the line pointing to your current session.

```
[paul@centos7 ~]$ who am i
paul      pts/1      2014-10-10 23:30 (10.104.33.101)
[paul@centos7 ~]$
```

27.4. w

The **w** command shows you who is logged on and what they are doing.

```
[paul@centos7 ~]$ w
23:34:07 up 31 min,  2 users,  load average: 0.00, 0.01, 0.02
USER      TTY      LOGIN@  IDLE   JCPU   PCPU WHAT
root      pts/0    23:07   15.00s  0.01s  0.01s top
paul      pts/1    23:30    7.00s  0.00s  0.00s w
[paul@centos7 ~]$
```

27.5. id

The **id** command will give you your user id, primary group id, and a list of the groups that you belong to.

```
paul@debian7:~$ id
uid=1000(paul) gid=1000(paul) groups=1000(paul)
```

On RHEL/CentOS you will also get **SELinux** context information with this command.

```
[root@centos7 ~]# id
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r\
:unconfined_t:s0-s0:c0.c1023
```

27.6. su to another user

The **su** command allows a user to run a shell as another user.

```
laura@debian7:~$ su tania
Password:
tania@debian7:/home/laura$
```

27.7. su to root

Yes you can also **su** to become **root**, when you know the **root** password.

```
laura@debian7:~$ su root
Password:
root@debian7:/home/laura#
```

27.8. su as root

You need to know the password of the user you want to substitute to, unless your are logged in as **root**. The **root** user can become any existing user without knowing that user's password.

```
root@debian7:~# id
uid=0(root) gid=0(root) groups=0(root)
root@debian7:~# su - valentina
valentina@debian7:~$
```

27.9. su - \$username

By default, the **su** command maintains the same shell environment. To become another user and also get the target user's environment, issue the **su -** command followed by the target username.

```
root@debian7:~# su laura
laura@debian7:/root$ exit
exit
root@debian7:~# su - laura
laura@debian7:~$ pwd
/home/laura
```

27.10. su -

When no username is provided to **su** or **su -**, the command will assume **root** is the target.

```
tania@debian7:~$ su -
Password:
root@debian7:~#
```

27.11. run a program as another user

The `sudo` program allows a user to start a program with the credentials of another user. Before this works, the system administrator has to set up the `/etc/sudoers` file. This can be useful to delegate administrative tasks to another user (without giving the root password).

The screenshot below shows the usage of `sudo`. User **paul** received the right to run `useradd` with the credentials of **root**. This allows **paul** to create new users on the system without becoming **root** and without knowing the **root password**.

First the command fails for **paul**.

```
paul@debian7:~$ /usr/sbin/useradd -m valentina
useradd: Permission denied.
useradd: cannot lock /etc/passwd; try again later.
```

But with `sudo` it works.

```
paul@debian7:~$ sudo /usr/sbin/useradd -m valentina
[sudo] password for paul:
paul@debian7:~$
```

27.12. visudo

Check the man page of **visudo** before playing with the `/etc/sudoers` file. Editing the `sudoers` is out of scope for this fundamentals book.

```
paul@rhel65:~$ apropos visudo
visudo          (8) - edit the sudoers file
paul@rhel65:~$
```

27.13. sudo su -

On some Linux systems like Ubuntu and Xubuntu, the **root** user does not have a password set. This means that it is not possible to login as **root** (extra security). To perform tasks as **root**, the first user is given all **sudo rights** via the **/etc/sudoers**. In fact all users that are members of the admin group can use sudo to run all commands as root.

```
root@laika:~# grep admin /etc/sudoers
# Members of the admin group may gain root privileges
%admin ALL=(ALL) ALL
```

The end result of this is that the user can type **sudo su -** and become root without having to enter the root password. The sudo command does require you to enter your own password. Thus the password prompt in the screenshot below is for sudo, not for su.

```
paul@laika:~$ sudo su -
Password:
root@laika:~#
```

27.14. sudo logging

Using **sudo** without authorization will result in a severe warning:

```
paul@rhel65:~$ sudo su -

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for paul:
paul is not in the sudoers file. This incident will be reported.
paul@rhel65:~$
```

The root user can see this in the **/var/log/secure** on Red Hat and in **/var/log/auth.log** on Debian).

```
root@rhel65:~# tail /var/log/secure | grep sudo | tr -s ' '
Apr 13 16:03:42 rhel65 sudo: paul : user NOT in sudoers ; TTY=pts/0 ; PWD=\
/home/paul ; USER=root ; COMMAND=/bin/su -
root@rhel65:~#
```

27.15. practice: introduction to users

1. Run a command that displays only your currently logged on user name.
2. Display a list of all logged on users.
3. Display a list of all logged on users including the command they are running at this very moment.
4. Display your user name and your unique user identification (userid).
5. Use **su** to switch to another user account (unless you are root, you will need the password of the other account). And get back to the previous account.
6. Now use **su -** to switch to another user and notice the difference.

Note that **su -** gets you into the home directory of **Tania**.

7. Try to create a new user account (when using your normal user account). this should fail. (Details on adding user accounts are explained in the next chapter.)
8. Now try the same, but with **sudo** before your command.

27.16. solution: introduction to users

1. Run a command that displays only your currently logged on user name.

```
laura@debian7:~$ whoami
laura
laura@debian7:~$ echo $USER
laura
```

2. Display a list of all logged on users.

```
laura@debian7:~$ who
laura pts/0 2014-10-13 07:22 (10.104.33.101)
laura@debian7:~$
```

3. Display a list of all logged on users including the command they are running at this very moment.

```
laura@debian7:~$ w
07:47:02 up 16 min, 2 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU WHAT
root      pts/0    10.104.33.101  07:30    6.00s  0.04s  0.00s w
root      pts/1    10.104.33.101  07:46    6.00s  0.01s  0.00s sleep 42
laura@debian7:~$
```

4. Display your user name and your unique user identification (userid).

```
laura@debian7:~$ id
uid=1005(laura) gid=1007(laura) groups=1007(laura)
laura@debian7:~$
```

5. Use **su** to switch to another user account (unless you are root, you will need the password of the other account). And get back to the previous account.

```
laura@debian7:~$ su tania
Password:
tania@debian7:/home/laura$ id
uid=1006(tania) gid=1008(tania) groups=1008(tania)
tania@debian7:/home/laura$ exit
laura@debian7:~$
```

6. Now use **su -** to switch to another user and notice the difference.

```
laura@debian7:~$ su - tania
Password:
tania@debian7:~$ pwd
/home/tania
tania@debian7:~$ logout
laura@debian7:~$
```

Note that **su -** gets you into the home directory of **Tania**.

7. Try to create a new user account (when using your normal user account). this should fail. (Details on adding user accounts are explained in the next chapter.)

```
laura@debian7:~$ useradd valentina
-su: useradd: command not found
laura@debian7:~$ /usr/sbin/useradd valentina
useradd: Permission denied.
useradd: cannot lock /etc/passwd; try again later.
```

It is possible that **useradd** is located in **/sbin/useradd** on your computer.

8. Now try the same, but with **sudo** before your command.

```
laura@debian7:~$ sudo /usr/sbin/useradd valentina
[sudo] password for laura:
laura is not in the sudoers file. This incident will be reported.
laura@debian7:~$
```

Notice that **laura** has no permission to use the **sudo** on this system.

Chapter 28. user management

This chapter will teach you how to use **useradd**, **usermod** and **userdel** to create, modify and remove user accounts.

You will need **root** access on a Linux computer to complete this chapter.

28.1. user management

User management on Linux can be done in three complementary ways. You can use the **graphical** tools provided by your distribution. These tools have a look and feel that depends on the distribution. If you are a novice Linux user on your home system, then use the graphical tool that is provided by your distribution. This will make sure that you do not run into problems.

Another option is to use **command line tools** like `useradd`, `usermod`, `gpasswd`, `passwd` and others. Server administrators are likely to use these tools, since they are familiar and very similar across many different distributions. This chapter will focus on these command line tools.

A third and rather extremist way is to **edit the local configuration files** directly using `vi` (or `vipw/vigr`). Do not attempt this as a novice on production systems!

28.2. /etc/passwd

The local user database on Linux (and on most Unixes) is **/etc/passwd**.

```
[root@RHEL5 ~]# tail /etc/passwd
inge:x:518:524:art dealer:/home/inge:/bin/ksh
ann:x:519:525:flute player:/home/ann:/bin/bash
frederik:x:520:526:rubius poet:/home/frederik:/bin/bash
steven:x:521:527:roman emperor:/home/steven:/bin/bash
pascale:x:522:528:artist:/home/pascale:/bin/ksh
geert:x:524:530:kernel developer:/home/geert:/bin/bash
wim:x:525:531:master damuti:/home/wim:/bin/bash
sandra:x:526:532:radish stresser:/home/sandra:/bin/bash
annelies:x:527:533:sword fighter:/home/annelies:/bin/bash
laura:x:528:534:art dealer:/home/laura:/bin/ksh
```

As you can see, this file contains seven columns separated by a colon. The columns contain the username, an x, the user id, the primary group id, a description, the name of the home directory, and the login shell.

More information can be found by typing **man 5 passwd**.

```
[root@RHEL5 ~]# man 5 passwd
```

28.3. root

The **root** user also called the **superuser** is the most powerful account on your Linux system. This user can do almost anything, including the creation of other users. The root user always has `userid 0` (regardless of the name of the account).

```
[root@RHEL5 ~]# head -1 /etc/passwd
root:x:0:0:root:/root:/bin/bash
```

28.4. useradd

You can add users with the **useradd** command. The example below shows how to add a user named yanina (last parameter) and at the same time forcing the creation of the home directory (-m), setting the name of the home directory (-d), and setting a description (-c).

```
[root@RHEL5 ~]# useradd -m -d /home/yanina -c "yanina wickmayer" yanina
[root@RHEL5 ~]# tail -1 /etc/passwd
yanina:x:529:529:yanina wickmayer:/home/yanina:/bin/bash
```

The user named yanina received userid 529 and **primary group** id 529.

28.5. /etc/default/useradd

Both Red Hat Enterprise Linux and Debian/Ubuntu have a file called **/etc/default/useradd** that contains some default user options. Besides using cat to display this file, you can also use **useradd -D**.

```
[root@RHEL4 ~]# useradd -D
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/bash
SKEL=/etc/skel
```

28.6. userdel

You can delete the user yanina with **userdel**. The -r option of userdel will also remove the home directory.

```
[root@RHEL5 ~]# userdel -r yanina
```

28.7. usermod

You can modify the properties of a user with the **usermod** command. This example uses **usermod** to change the description of the user harry.

```
[root@RHEL4 ~]# tail -1 /etc/passwd
harry:x:516:520:harry potter:/home/harry:/bin/bash
[root@RHEL4 ~]# usermod -c 'wizard' harry
[root@RHEL4 ~]# tail -1 /etc/passwd
harry:x:516:520:wizard:/home/harry:/bin/bash
```

28.8. creating home directories

The easiest way to create a home directory is to supply the **-m** option with **useradd** (it is likely set as a default option on Linux).

A less easy way is to create a home directory manually with **mkdir** which also requires setting the owner and the permissions on the directory with **chmod** and **chown** (both commands are discussed in detail in another chapter).

```
[root@RHEL5 ~]# mkdir /home/laura
[root@RHEL5 ~]# chown laura:laura /home/laura
[root@RHEL5 ~]# chmod 700 /home/laura
[root@RHEL5 ~]# ls -ld /home/laura/
drwx----- 2 laura laura 4096 Jun 24 15:17 /home/laura/
```

28.9. /etc/skel/

When using **useradd** the **-m** option, the **/etc/skel/** directory is copied to the newly created home directory. The **/etc/skel/** directory contains some (usually hidden) files that contain profile settings and default values for applications. In this way **/etc/skel/** serves as a default home directory and as a default user profile.

```
[root@RHEL5 ~]# ls -la /etc/skel/
total 48
drwxr-xr-x  2 root root  4096 Apr  1 00:11 .
drwxr-xr-x 97 root root 12288 Jun 24 15:36 ..
-rw-r--r--  1 root root    24 Jul 12  2006 .bash_logout
-rw-r--r--  1 root root   176 Jul 12  2006 .bash_profile
-rw-r--r--  1 root root   124 Jul 12  2006 .bashrc
```

28.10. deleting home directories

The **-r** option of **userdel** will make sure that the home directory is deleted together with the user account.

```
[root@RHEL5 ~]# ls -ld /home/wim/
drwx----- 2 wim wim 4096 Jun 24 15:19 /home/wim/
[root@RHEL5 ~]# userdel -r wim
[root@RHEL5 ~]# ls -ld /home/wim/
ls: /home/wim/: No such file or directory
```

28.11. login shell

The **/etc/passwd** file specifies the **login shell** for the user. In the screenshot below you can see that user **annelies** will log in with the **/bin/bash** shell, and user **laura** with the **/bin/ksh** shell.

```
[root@RHEL5 ~]# tail -2 /etc/passwd
annelies:x:527:533:sword fighter:/home/annelies:/bin/bash
laura:x:528:534:art dealer:/home/laura:/bin/ksh
```

You can use the **usermod** command to change the shell for a user.

```
[root@RHEL5 ~]# usermod -s /bin/bash laura
[root@RHEL5 ~]# tail -1 /etc/passwd
laura:x:528:534:art dealer:/home/laura:/bin/bash
```

28.12. chsh

Users can change their login shell with the **chsh** command. First, user **harry** obtains a list of available shells (he could also have done a **cat /etc/shells**) and then changes his login shell to the **Korn shell** (**/bin/ksh**). At the next login, **harry** will default into **ksh** instead of **bash**.

```
[laura@centos7 ~]$ chsh -l
/bin/sh
/bin/bash
/sbin/nologin
/usr/bin/sh
/usr/bin/bash
/usr/sbin/nologin
/bin/ksh
/bin/tcsh
/bin/csh
[laura@centos7 ~]$
```

Note that the **-l** option does not exist on **Debian** and that the above screenshot assumes that **ksh** and **csh** shells are installed.

The screenshot below shows how **laura** can change her default shell (active on next login).

```
[laura@centos7 ~]$ chsh -s /bin/ksh
Changing shell for laura.
Password:
Shell changed.
```

28.13. practice: user management

1. Create a user account named **serena**, including a home directory and a description (or comment) that reads **Serena Williams**. Do all this in one single command.
2. Create a user named **venus**, including home directory, bash shell, a description that reads **Venus Williams** all in one single command.
3. Verify that both users have correct entries in **/etc/passwd**, **/etc/shadow** and **/etc/group**.
4. Verify that their home directory was created.
5. Create a user named **einstime** with **/bin/date** as his default login shell.
7. What happens when you log on with the **einstime** user ? Can you think of a useful real world example for changing a user's login shell to an application ?
8. Create a file named **welcome.txt** and make sure every new user will see this file in their home directory.
9. Verify this setup by creating (and deleting) a test user account.
10. Change the default login shell for the **serena** user to **/bin/bash**. Verify before and after you make this change.

28.14. solution: user management

1. Create a user account named **serena**, including a home directory and a description (or comment) that reads **Serena Williams**. Do all this in one single command.

```
root@debian7:~# useradd -m -c 'Serena Williams' serena
```

2. Create a user named **venus**, including home directory, bash shell, a description that reads **Venus Williams** all in one single command.

```
root@debian7:~# useradd -m -c "Venus Williams" -s /bin/bash venus
```

3. Verify that both users have correct entries in **/etc/passwd**, **/etc/shadow** and **/etc/group**.

```
root@debian7:~# tail -2 /etc/passwd
serena:x:1008:1010:Serena Williams:/home/serena:/bin/sh
venus:x:1009:1011:Venus Williams:/home/venus:/bin/bash
root@debian7:~# tail -2 /etc/shadow
serena:!:16358:0:99999:7:::
venus:!:16358:0:99999:7:::
root@debian7:~# tail -2 /etc/group
serena:x:1010:
venus:x:1011:
```

4. Verify that their home directory was created.

```
root@debian7:~# ls -lrt /home | tail -2
drwxr-xr-x 2 serena  serena  4096 Oct 15 10:50 serena
drwxr-xr-x 2 venus   venus   4096 Oct 15 10:59 venus
root@debian7:~#
```

5. Create a user named **einstime** with **/bin/date** as his default login shell.

```
root@debian7:~# useradd -s /bin/date einstime
```

Or even better:

```
root@debian7:~# useradd -s $(which date) einstime
```

7. What happens when you log on with the **einstime** user ? Can you think of a useful real world example for changing a user's login shell to an application ?

```
root@debian7:~# su - einstime
Wed Oct 15 11:05:56 UTC 2014 # You get the output of the date command
root@debian7:~#
```

It can be useful when users need to access only one application on the server. Just logging in opens the application for them, and closing the application automatically logs them out.

8. Create a file named **welcome.txt** and make sure every new user will see this file in their home directory.

```
root@debian7:~# echo Hello > /etc/skel/welcome.txt
```

9. Verify this setup by creating (and deleting) a test user account.

```
root@debian7:~# useradd -m test
root@debian7:~# ls -l /home/test
total 4
-rw-r--r-- 1 test test 6 Oct 15 11:16 welcome.txt
root@debian7:~# userdel -r test
root@debian7:~#
```

10. Change the default login shell for the **serena** user to **/bin/bash**. Verify before and after you make this change.

```
root@debian7:~# grep serena /etc/passwd
serena:x:1008:1010:Serena Williams:/home/serena:/bin/sh
root@debian7:~# usermod -s /bin/bash serena
root@debian7:~# grep serena /etc/passwd
serena:x:1008:1010:Serena Williams:/home/serena:/bin/bash
root@debian7:~#
```

Chapter 29. user passwords

This chapter will tell you more about passwords for local users.

Three methods for setting passwords are explained; using the **passwd** command, using **openssl passwd**, and using the **crypt** function in a C program.

The chapter will also discuss password settings and disabling, suspending or locking accounts.

29.1. passwd

Passwords of users can be set with the **passwd** command. Users will have to provide their old password before twice entering the new one.

```
[tania@centos7 ~]$ passwd
Changing password for user tania.
Changing password for tania.
(current) UNIX password:
New password:
BAD PASSWORD: The password is shorter than 8 characters
New password:
BAD PASSWORD: The password is a palindrome
New password:
BAD PASSWORD: The password is too similar to the old one
passwd: Have exhausted maximum number of retries for service
```

As you can see, the passwd tool will do some basic verification to prevent users from using too simple passwords. The **root** user does not have to follow these rules (there will be a warning though). The **root** user also does not have to provide the old password before entering the new password twice.

```
root@debian7:~# passwd tania
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

29.2. shadow file

User passwords are encrypted and kept in **/etc/shadow**. The **/etc/shadow** file is read only and can only be read by root. We will see in the file permissions section how it is possible for users to change their password. For now, you will have to know that users can change their password with the **/usr/bin/passwd** command.

```
[root@centos7 ~]# tail -4 /etc/shadow
paul:$6$ikp2Xta5BT.Tml.p$2TZjNnOYNNQKpwLJqoGJbVsZG5/Fti8ovBRd.VzRbiDSl7TEq\
IaSMH.TeBKnTS/SjlMruW8qffc0JNORW.BTW1:16338:0:99999:7:::
tania:$6$8Z/zovxj$9qvoqT8i9KIrmN.k4EQwAF5ryz5yzNwEvYjAa9L5XVXQu.z4DlpmREH\
eQpQzvRnqFdKkvj17H5ST.c79HDZW0:16356:0:99999:7:::
laura:$6$glDuTY5e$/NYYWLxfHgZFWeoujaXSMcR.Mz.lG0xtcxFocFVJNb98nbTPhWFXfKWG\
SyYh1WCv6763Wq54.w24Yr3uAZB0m/:16356:0:99999:7:::
valentina:$6$jrzA6PVI$1uQgqR6En9mZB6mKJ3LXRB4CnFko6LRhbh.v4iqUk9MVreui1lv7\
GxH0UDSKA0N55ZRNhGHa6T2ouFnVno/0o1:16356:0:99999:7:::
[root@centos7 ~]#
```

The **/etc/shadow** file contains nine colon separated columns. The nine fields contain (from left to right) the user name, the encrypted password (note that only inge and laura have an encrypted password), the day the password was last changed (day 1 is January 1, 1970), number of days the password must be left unchanged, password expiry day, warning number of days before password expiry, number of days after expiry before disabling the account, and the day the account was disabled (again, since 1970). The last field has no meaning yet.

All the passwords in the screenshot above are hashes of **hunter2**.

29.3. encryption with passwd

Passwords are stored in an encrypted format. This encryption is done by the **crypt** function. The easiest (and recommended) way to add a user with a password to the system is to add the user with the **useradd -m user** command, and then set the user's password with **passwd**.

```
[root@RHEL4 ~]# useradd -m xavier
[root@RHEL4 ~]# passwd xavier
Changing password for user xavier.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
[root@RHEL4 ~]#
```

29.4. encryption with openssl

Another way to create users with a password is to use the **-p** option of **useradd**, but that option requires an encrypted password. You can generate this encrypted password with the **openssl passwd** command.

The **openssl passwd** command will generate several distinct hashes for the same password, for this it uses a **salt**.

```
paul@rhel65:~$ openssl passwd hunter2
86jcUNlnGDFpY
paul@rhel65:~$ openssl passwd hunter2
Yj7mD090Anvq6
paul@rhel65:~$ openssl passwd hunter2
YqDcJeGoDbzKA
paul@rhel65:~$
```

This **salt** can be chosen and is visible as the first two characters of the hash.

```
paul@rhel65:~$ openssl passwd -salt 42 hunter2
42ZrbtP1Ze8G.
paul@rhel65:~$ openssl passwd -salt 42 hunter2
42ZrbtP1Ze8G.
paul@rhel65:~$ openssl passwd -salt 42 hunter2
42ZrbtP1Ze8G.
paul@rhel65:~$
```

This example shows how to create a user with password.

```
root@rhel65:~# useradd -m -p $(openssl passwd hunter2) mohamed
```

Note that this command puts the password in your command history!

29.5. encryption with crypt

A third option is to create your own C program using the crypt function, and compile this into a command.

```
paul@rhel65:~$ cat MyCrypt.c
#include <stdio.h>
#define __USE_XOPEN
#include <unistd.h>

int main(int argc, char** argv)
{
    if(argc==3)
    {
        printf("%s\n", crypt(argv[1],argv[2]));
    }
    else
    {
        printf("Usage: MyCrypt $password $salt\n" );
    }
    return 0;
}
```

This little program can be compiled with **gcc** like this.

```
paul@rhel65:~$ gcc MyCrypt.c -o MyCrypt -lcrypt
```

To use it, we need to give two parameters to MyCrypt. The first is the unencrypted password, the second is the salt. The salt is used to perturb the encryption algorithm in one of 4096 different ways. This variation prevents two users with the same password from having the same entry in **/etc/shadow**.

```
paul@rhel65:~$ ./MyCrypt hunter2 42
42ZrbtP1Ze8G.
paul@rhel65:~$ ./MyCrypt hunter2 33
33d6taYSiEUXI
```

Did you notice that the first two characters of the password are the **salt**?

The standard output of the crypt function is using the DES algorithm which is old and can be cracked in minutes. A better method is to use **md5** passwords which can be recognized by a salt starting with \$1\$.

```
paul@rhel65:~$ ./MyCrypt hunter2 '$1$42'
$1$42$7l6Y3xT5282XmZrtD0F9f0
paul@rhel65:~$ ./MyCrypt hunter2 '$6$42'
$6$42$0qFFAVnI3gTSYG0yI9TZX9cpyQzwIop7HwpG1LLEsNBiMr4w60vLX1KDa./UpwXfrFk1i...
```

The **md5** salt can be up to eight characters long. The salt is displayed in **/etc/shadow** between the second and third \$, so never use the password as the salt!

```
paul@rhel65:~$ ./MyCrypt hunter2 '$1$hunter2'
$1$hunter2$YVxrxDmidq7Xf8Gdt6qM2.
```

29.6. /etc/login.defs

The **/etc/login.defs** file contains some default settings for user passwords like password aging and length settings. (You will also find the numerical limits of user ids and group ids and whether or not a home directory should be created by default).

```
root@rhel65:~# grep ^PASS /etc/login.defs
PASS_MAX_DAYS    99999
PASS_MIN_DAYS    0
PASS_MIN_LEN     5
PASS_WARN_AGE    7
```

Debian also has this file.

```
root@debian7:~# grep PASS /etc/login.defs
# PASS_MAX_DAYS    Maximum number of days a password may be used.
# PASS_MIN_DAYS    Minimum number of days allowed between password changes.
# PASS_WARN_AGE    Number of days warning given before a password expires.
PASS_MAX_DAYS    99999
PASS_MIN_DAYS    0
PASS_WARN_AGE    7
#PASS_CHANGE_TRIES
#PASS_ALWAYS_WARN
#PASS_MIN_LEN
#PASS_MAX_LEN
# NO_PASSWORD_CONSOLE
root@debian7:~#
```

29.7. chage

The **chage** command can be used to set an expiration date for a user account (-E), set a minimum (-m) and maximum (-M) password age, a password expiration date, and set the number of warning days before the password expiration date. Much of this functionality is also available from the **passwd** command. The **-l** option of **chage** will list these settings for a user.

```
root@rhel65:~# chage -l paul
Last password change                : Mar 27, 2014
Password expires                    : never
Password inactive                   : never
Account expires                     : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
root@rhel65:~#
```

29.8. disabling a password

Passwords in **/etc/shadow** cannot begin with an exclamation mark. When the second field in **/etc/passwd** starts with an exclamation mark, then the password can not be used.

Using this feature is often called **locking**, **disabling**, or **suspending** a user account. Besides **vi** (or **vim**) you can also accomplish this with **usermod**.

The first command in the next screenshot will show the hashed password of **laura** in **/etc/shadow**. The next command disables the password of **laura**, making it impossible for Laura to authenticate using this password.

```
root@debian7:~# grep laura /etc/shadow | cut -c1-70
laura:$6$JYj4JZqp$stwwACp30tE1R2aZuE87j.nbW.puDkNUYV7mCHfCVMa3CoDUJV
root@debian7:~# usermod -L laura
```

As you can see below, the password hash is simply preceded with an exclamation mark.

```
root@debian7:~# grep laura /etc/shadow | cut -c1-70
laura: !$6$JYj4JZqp$stwwACp30tE1R2aZuE87j.nbW.puDkNUYV7mCHfCVMa3CoDUJ
root@debian7:~#
```

The root user (and users with **sudo** rights on **su**) still will be able to **su** into the **laura** account (because the password is not needed here). Also note that **laura** will still be able to login if she has set up passwordless ssh!

```
root@debian7:~# su - laura
laura@debian7:~$
```

You can unlock the account again with **usermod -U**.

```
root@debian7:~# usermod -U laura
root@debian7:~# grep laura /etc/shadow | cut -c1-70
laura:$6$JYj4JZqp$stwwACp30tE1R2aZuE87j.nbW.puDkNUYV7mCHfCVMa3CoDUJV
```

Watch out for tiny differences in the command line options of **passwd**, **usermod**, and **useradd** on different Linux distributions. Verify the local files when using features like "disabling, suspending, or locking" on user accounts and their passwords.

29.9. editing local files

If you still want to manually edit the **/etc/passwd** or **/etc/shadow**, after knowing these commands for password management, then use **vim** instead of **vi(m)** directly. The **vim** tool will do proper locking of the file.

```
[root@RHEL5 ~]# vim /etc/passwd
vim: the password file is busy (/etc/ptmp present)
```

29.10. practice: user passwords

1. Set the password for **serena** to **hunter2**.
2. Also set a password for **venus** and then lock the **venus** user account with **usermod**. Verify the locking in **/etc/shadow** before and after you lock it.
3. Use **passwd -d** to disable the **serena** password. Verify the **serena** line in **/etc/shadow** before and after disabling.
4. What is the difference between locking a user account and disabling a user account's password like we just did with **usermod -L** and **passwd -d**?
5. Try changing the password of serena to serena as serena.
6. Make sure **serena** has to change her password in 10 days.
7. Make sure every new user needs to change their password every 10 days.
8. Take a backup as root of **/etc/shadow**. Use **vi** to copy an encrypted **hunter2** hash from **venus** to **serena**. Can **serena** now log on with **hunter2** as a password ?
9. Why use **vipw** instead of **vi** ? What could be the problem when using **vi** or **vim** ?
10. Use **chsh** to list all shells (only works on RHEL/CentOS/Fedora), and compare to **cat /etc/shells**.
11. Which **useradd** option allows you to name a home directory ?
12. How can you see whether the password of user **serena** is locked or unlocked ? Give a solution with **grep** and a solution with **passwd**.

29.11. solution: user passwords

1. Set the password for **serena** to **hunter2**.

```
root@debian7:~# passwd serena
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

2. Also set a password for **venus** and then lock the **venus** user account with **usermod**. Verify the locking in **/etc/shadow** before and after you lock it.

```
root@debian7:~# passwd venus
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@debian7:~# grep venus /etc/shadow | cut -c1-70
venus:$6$gswzXICW$uSnKFV1kFKZmTPaMVS4AvNA/K0270xN0v5LHdV9ed0gTyXrjUeM/
root@debian7:~# usermod -L venus
root@debian7:~# grep venus /etc/shadow | cut -c1-70
venus:!!$6$gswzXICW$uSnKFV1kFKZmTPaMVS4AvNA/K0270xN0v5LHdV9ed0gTyXrjUeM
```

Note that **usermod -L** precedes the password hash with an exclamation mark (!).

3. Use **passwd -d** to disable the **serena** password. Verify the **serena** line in **/etc/shadow** before and after disabling.

```
root@debian7:~# grep serena /etc/shadow | cut -c1-70
serena:$6$Es/omrPE$F2Ypu8kpLrfKdW0v/UIwA5jrYyBD2nwZ/dt.i/IypRgiPZSdB/B
root@debian7:~# passwd -d serena
passwd: password expiry information changed.
root@debian7:~# grep serena /etc/shadow
serena::16358:0:99999:7:::
root@debian7:~#
```

4. What is the difference between locking a user account and disabling a user account's password like we just did with **usermod -L** and **passwd -d**?

Locking will prevent the user from logging on to the system with his password by putting a **!** in front of the password in **/etc/shadow**.

Disabling with **passwd** will erase the password from **/etc/shadow**.

5. Try changing the password of serena to serena as serena.

```
log on as serena, then execute: passwd serena... it should fail!
```

6. Make sure **serena** has to change her password in 10 days.

```
chage -M 10 serena
```

7. Make sure every new user needs to change their password every 10 days.

```
vi /etc/login.defs (and change PASS_MAX_DAYS to 10)
```


8. Take a backup as root of **/etc/shadow**. Use **vi** to copy an encrypted **hunter2** hash from **venus** to **serena**. Can **serena** now log on with **hunter2** as a password ?

Yes.

9. Why use **vipw** instead of **vi** ? What could be the problem when using **vi** or **vim** ?

vipw will give a warning when someone else is already using that file (with **vipw**).

10. Use **chsh** to list all shells (only works on RHEL/CentOS/Fedora), and compare to **cat /etc/shells**.

```
chsh -l  
cat /etc/shells
```

11. Which **useradd** option allows you to name a home directory ?

-d

12. How can you see whether the password of user **serena** is locked or unlocked ? Give a solution with **grep** and a solution with **passwd**.

```
grep serena /etc/shadow
```

```
passwd -S serena
```

Chapter 30. user profiles

Logged on users have a number of preset (and customized) aliases, variables, and functions, but where do they come from ? The **shell** uses a number of startup files that are executed (or rather **sourced**) whenever the shell is invoked. What follows is an overview of startup scripts.

30.1. system profile

Both the **bash** and the **ksh** shell will verify the existence of **/etc/profile** and **source** it if it exists.

When reading this script, you will notice (both on Debian and on Red Hat Enterprise Linux) that it builds the **PATH** environment variable (among others). The script might also change the **PS1** variable, set the **HOSTNAME** and execute even more scripts like **/etc/inputrc**

This screenshot uses **grep** to show **PATH** manipulation in **/etc/profile** on Debian.

```
root@debian7:~# grep PATH /etc/profile
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
PATH="/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games"
export PATH
root@debian7:~#
```

This screenshot uses **grep** to show **PATH** manipulation in **/etc/profile** on RHEL7/CentOS7.

```
[root@centos7 ~]# grep PATH /etc/profile
case ":${PATH}:" in
    PATH=$PATH:$1
    PATH=$1:$PATH
export PATH USER LOGNAME MAIL HOSTNAME HISTSIZE HISTCONTROL
[root@centos7 ~]#
```

The **root user** can use this script to set aliases, functions, and variables for every user on the system.

30.2. ~/.bash_profile

When this file exists in the home directory, then **bash** will source it. On Debian Linux 5/6/7 this file does not exist by default.

RHEL7/CentOS7 uses a small **~/.bash_profile** where it checks for the existence of **~/.bashrc** and then sources it. It also adds **\$HOME/bin** to the **\$PATH** variable.

```
[root@rhel7 ~]# cat /home/paul/.bash_profile
# .bash_profile

# Get the aliases and functions
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
fi

# User specific environment and startup programs

PATH=$PATH:$HOME/.local/bin:$HOME/bin

export PATH
[root@rhel7 ~]#
```

30.3. ~/.bash_login

When **.bash_profile** does not exist, then **bash** will check for **~/.bash_login** and source it.

Neither Debian nor Red Hat have this file by default.

30.4. ~/.profile

When neither **~/.bash_profile** and **~/.bash_login** exist, then **bash** will verify the existence of **~/.profile** and execute it. This file does not exist by default on Red Hat.

On Debian this script can execute **~/.bashrc** and will add **\$HOME/bin** to the **\$PATH** variable.

```
root@debian7:~# tail -11 /home/paul/.profile
if [ -n "$BASH_VERSION" ]; then
    # include .bashrc if it exists
    if [ -f "$HOME/.bashrc" ]; then
        . "$HOME/.bashrc"
    fi
fi

# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/bin" ] ; then
    PATH="$HOME/bin:$PATH"
fi
```

RHEL/CentOS does not have this file by default.

30.5. ~/.bashrc

The **~/.bashrc** script is often sourced by other scripts. Let us take a look at what it does by default.

Red Hat uses a very simple **~/.bashrc**, checking for **/etc/bashrc** and sourcing it. It also leaves room for custom aliases and functions.

```
[root@rhel7 ~]# cat /home/paul/.bashrc
# .bashrc

# Source global definitions
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi

# Uncomment the following line if you don't like systemctl's auto-paging feature:
# export SYSTEMD_PAGER=

# User specific aliases and functions
```

On Debian this script is quite a bit longer and configures **\$PS1**, some history variables and a number of active and inactive aliases.

```
root@debian7:~# wc -l /home/paul/.bashrc
110 /home/paul/.bashrc
```

30.6. ~/.bash_logout

When exiting **bash**, it can execute **~/.bash_logout**.

Debian use this opportunity to clear the console screen.

```
serena@deb503:~$ cat ~/.bash_logout
# ~/.bash_logout: executed by bash(1) when login shell exits.

# when leaving the console clear the screen to increase privacy

if [ "$SHLVL" = 1 ]; then
    [ -x /usr/bin/clear_console ] && /usr/bin/clear_console -q
fi
```

Red Hat Enterprise Linux 5 will simple call the **/usr/bin/clear** command in this script.

```
[serena@rhel53 ~]$ cat ~/.bash_logout
# ~/.bash_logout

/usr/bin/clear
```

Red Hat Enterprise Linux 6 and 7 create this file, but leave it empty (except for a comment).

```
paul@rhel65:~$ cat ~/.bash_logout
# ~/.bash_logout
```

30.7. Debian overview

Below is a table overview of when Debian is running any of these bash startup scripts.

Table 30.1. Debian User Environment

script	su	su -	ssh	gdm
~/bashrc	no	yes	yes	yes
~/.profile	no	yes	yes	yes
/etc/profile	no	yes	yes	yes
/etc/bash.bashrc	yes	no	no	yes

30.8. RHEL5 overview

Below is a table overview of when Red Hat Enterprise Linux 5 is running any of these bash startup scripts.

Table 30.2. Red Hat User Environment

script	su	su -	ssh	gdm
~/bashrc	yes	yes	yes	yes
~/.bash_profile	no	yes	yes	yes
/etc/profile	no	yes	yes	yes
/etc/bashrc	yes	yes	yes	yes

30.9. practice: user profiles

1. Make a list of all the profile files on your system.
2. Read the contents of each of these, often they **source** extra scripts.
3. Put a unique variable, alias and function in each of those files.
4. Try several different ways to obtain a shell (su, su -, ssh, tmux, gnome-terminal, Ctrl-alt-F1, ...) and verify which of your custom variables, aliases and function are present in your environment.
5. Do you also know the order in which they are executed?
6. When an application depends on a setting in \$HOME/.profile, does it matter whether \$HOME/.bash_profile exists or not ?

30.10. solution: user profiles

1. Make a list of all the profile files on your system.

```
ls -a ~ ; ls -l /etc/pro* /etc/bash*
```

2. Read the contents of each of these, often they **source** extra scripts.
3. Put a unique variable, alias and function in each of those files.
4. Try several different ways to obtain a shell (su, su -, ssh, tmux, gnome-terminal, Ctrl-alt-F1, ...) and verify which of your custom variables, aliases and function are present in your environment.
5. Do you also know the order in which they are executed?

```
same name aliases, functions and variables will overwrite each other
```

6. When an application depends on a setting in \$HOME/.profile, does it matter whether \$HOME/.bash_profile exists or not ?

```
Yes it does matter. (man bash /INVOCATION)
```

Chapter 31. groups

Users can be listed in **groups**. Groups allow you to set permissions on the group level instead of having to set permissions for every individual user.

Every Unix or Linux distribution will have a graphical tool to manage groups. Novice users are advised to use this graphical tool. More experienced users can use command line tools to manage users, but be careful: Some distributions do not allow the mixed use of GUI and CLI tools to manage groups (YaST in Novell Suse). Senior administrators can edit the relevant files directly with **vi** or **vigr**.

31.1. groupadd

Groups can be created with the **groupadd** command. The example below shows the creation of five (empty) groups.

```
root@laika:~# groupadd tennis
root@laika:~# groupadd football
root@laika:~# groupadd snooker
root@laika:~# groupadd formula1
root@laika:~# groupadd salsa
```

31.2. group file

Users can be a member of several groups. Group membership is defined by the **/etc/group** file.

```
root@laika:~# tail -5 /etc/group
tennis:x:1006:
football:x:1007:
snooker:x:1008:
formula1:x:1009:
salsa:x:1010:
root@laika:~#
```

The first field is the group's name. The second field is the group's (encrypted) password (can be empty). The third field is the group identification or **GID**. The fourth field is the list of members, these groups have no members.

31.3. groups

A user can type the **groups** command to see a list of groups where the user belongs to.

```
[harry@RHEL4b ~]$ groups
harry sports
[harry@RHEL4b ~]$
```

31.4. usermod

Group membership can be modified with the `useradd` or **usermod** command.

```
root@laika:~# usermod -a -G tennis inge
root@laika:~# usermod -a -G tennis katrien
root@laika:~# usermod -a -G salsa katrien
root@laika:~# usermod -a -G snooker sandra
root@laika:~# usermod -a -G formula1 annelies
root@laika:~# tail -5 /etc/group
tennis:x:1006:inge,katrien
football:x:1007:
snooker:x:1008:sandra
formula1:x:1009:annelies
salsa:x:1010:katrien
root@laika:~#
```

Be careful when using **usermod** to add users to groups. By default, the **usermod** command will **remove** the user from every group of which he is a member if the group is not listed in the command! Using the **-a** (append) switch prevents this behaviour.

31.5. groupmod

You can change the group name with the **groupmod** command.

```
root@laika:~# groupmod -n darts snooker
root@laika:~# tail -5 /etc/group
tennis:x:1006:inge,katrien
football:x:1007:
formula1:x:1009:annelies
salsa:x:1010:katrien
darts:x:1008:sandra
```

31.6. groupdel

You can permanently remove a group with the **groupdel** command.

```
root@laika:~# groupdel tennis
root@laika:~#
```

31.7. gpasswd

You can delegate control of group membership to another user with the **gpasswd** command. In the example below we delegate permissions to add and remove group members to serena for the sports group. Then we **su** to serena and add harry to the sports group.

```
[root@RHEL4b ~]# gpasswd -A serena sports
[root@RHEL4b ~]# su - serena
[serena@RHEL4b ~]$ id harry
uid=516(harry) gid=520(harry) groups=520(harry)
[serena@RHEL4b ~]$ gpasswd -a harry sports
Adding user harry to group sports
[serena@RHEL4b ~]$ id harry
uid=516(harry) gid=520(harry) groups=520(harry),522(sports)
[serena@RHEL4b ~]$ tail -1 /etc/group
sports:x:522:serena,venus,harry
[serena@RHEL4b ~]$
```

Group administrators do not have to be a member of the group. They can remove themselves from a group, but this does not influence their ability to add or remove members.

```
[serena@RHEL4b ~]$ gpasswd -d serena sports
Removing user serena from group sports
[serena@RHEL4b ~]$ exit
```

Information about group administrators is kept in the **/etc/gshadow** file.

```
[root@RHEL4b ~]# tail -1 /etc/gshadow
sports:!:serena:venus,harry
[root@RHEL4b ~]#
```

To remove all group administrators from a group, use the **gpasswd** command to set an empty administrators list.

```
[root@RHEL4b ~]# gpasswd -A "" sports
```

31.8. newgrp

You can start a **child shell** with a new temporary **primary group** using the **newgrp** command.

```
root@rhel65:~# mkdir prigroup
root@rhel65:~# cd prigroup/
root@rhel65:~/prigroup# touch standard.txt
root@rhel65:~/prigroup# ls -l
total 0
-rw-r--r--. 1 root root 0 Apr 13 17:49 standard.txt
root@rhel65:~/prigroup# echo $SHLVL
1
root@rhel65:~/prigroup# newgrp tennis
root@rhel65:~/prigroup# echo $SHLVL
2
root@rhel65:~/prigroup# touch newgrp.txt
root@rhel65:~/prigroup# ls -l
total 0
-rw-r--r--. 1 root tennis 0 Apr 13 17:49 newgrp.txt
-rw-r--r--. 1 root root 0 Apr 13 17:49 standard.txt
root@rhel65:~/prigroup# exit
exit
root@rhel65:~/prigroup#
```

31.9. vigr

Similar to **vipw**, the **vigr** command can be used to manually edit the **/etc/group** file, since it will do proper locking of the file. Only experienced senior administrators should use **vi** or **vigr** to manage groups.

31.10. practice: groups

1. Create the groups tennis, football and sports.
2. In one command, make venus a member of tennis and sports.
3. Rename the football group to foot.
4. Use vi to add serena to the tennis group.
5. Use the id command to verify that serena is a member of tennis.
6. Make someone responsible for managing group membership of foot and sports. Test that it works.

31.11. solution: groups

1. Create the groups tennis, football and sports.

```
groupadd tennis ; groupadd football ; groupadd sports
```

2. In one command, make venus a member of tennis and sports.

```
usermod -a -G tennis,sports venus
```

3. Rename the football group to foot.

```
groupmod -n foot football
```

4. Use vi to add serena to the tennis group.

```
vi /etc/group
```

5. Use the id command to verify that serena is a member of tennis.

```
id (and after logoff logon serena should be member)
```

6. Make someone responsible for managing group membership of foot and sports. Test that it works.

```
gpasswd -A (to make manager)
```

```
gpasswd -a (to add member)
```

Part IX. file security

Table of Contents

32. standard file permissions	306
32.1. file ownership	307
32.2. list of special files	309
32.3. permissions	310
32.4. practice: standard file permissions	315
32.5. solution: standard file permissions	316
33. advanced file permissions	318
33.1. sticky bit on directory	319
33.2. setgid bit on directory	319
33.3. setgid and setuid on regular files	320
33.4. setuid on sudo	320
33.5. practice: sticky, setuid and setgid bits	321
33.6. solution: sticky, setuid and setgid bits	322
34. access control lists	324
34.1. acl in /etc/fstab	325
34.2. getfacl	325
34.3. setfacl	325
34.4. remove an acl entry	326
34.5. remove the complete acl	326
34.6. the acl mask	326
34.7. eiciel	327
35. file links	328
35.1. inodes	329
35.2. about directories	330
35.3. hard links	331
35.4. symbolic links	332
35.5. removing links	332
35.6. practice : links	333
35.7. solution : links	334

Chapter 32. standard file permissions

This chapter contains details about basic file security through **file ownership** and **file permissions**.

32.1. file ownership

32.1.1. user owner and group owner

The **users** and **groups** of a system can be locally managed in `/etc/passwd` and `/etc/group`, or they can be in a NIS, LDAP, or Samba domain. These users and groups can **own** files. Actually, every file has a **user owner** and a **group owner**, as can be seen in the following screenshot.

```
paul@rhel65:~/owners$ ls -lh
total 636K
-rw-r--r--. 1 paul snooker 1.1K Apr  8 18:47 data.odt
-rw-r--r--. 1 paul paul    626K Apr  8 18:46 file1
-rw-r--r--. 1 root tennis  185 Apr  8 18:46 file2
-rw-rw-r--. 1 root root      0 Apr  8 18:47 stuff.txt
paul@rhel65:~/owners$
```

User paul owns three files; file1 has paul as **user owner** and has the group paul as **group owner**, data.odt is **group owned** by the group snooker, file2 by the group tennis.

The last file is called stuff.txt and is owned by the root user and the root group.

32.1.2. listing user accounts

You can use the following command to list all local user accounts.

```
paul@debian7-$ cut -d: -f1 /etc/passwd | column
root      ntp      sam      bert     naomi
daemon    mysql    tom      rino     matthias2
bin       paul     wouter   antonio  bram
sys       maarten  robrecht simon    fabrice
sync      kevin    bilal    sven     chimene
games     yuri     dimitri  wouter2  messagebus
man       william  ahmed    tarik    roger
lp        yves     dylan    jan      frank
mail      kris     robin    ian      toon
news      hamid    matthias ivan     rinus
uucp      vladimir ben      azeddine eddy
proxy     abiy     mike     eric     bram2
www-data  david    kevin2   kamel    keith
backup    chahid   kenzo    ischa    jesse
list      stef     aaron    bart     frederick
irc       joeri    lorenzo  omer     hans
gnats     glenn    jens     kurt     dries
nobody    yannick  ruben    steve    steve2
libuuid   christof jelle    constantin tomas
Debian-exim george   stefaan  sam2     johan
statd     joost    marc     bjorn    tom2
sshd      arno     thomas   ronald
```

32.1.3. chgrp

You can change the group owner of a file using the **chgrp** command.

```
root@rhel65:/home/paul/owners# ls -l file2
-rw-r--r-- 1 root tennis 185 Apr  8 18:46 file2
root@rhel65:/home/paul/owners# chgrp snooker file2
root@rhel65:/home/paul/owners# ls -l file2
-rw-r--r-- 1 root snooker 185 Apr  8 18:46 file2
root@rhel65:/home/paul/owners#
```

32.1.4. chown

The user owner of a file can be changed with **chown** command.

```
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 root paul 0 2008-08-06 14:11 FileForPaul
root@laika:/home/paul# chown paul FileForPaul
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 paul paul 0 2008-08-06 14:11 FileForPaul
```

You can also use **chown** to change both the user owner and the group owner.

```
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 paul paul 0 2008-08-06 14:11 FileForPaul
root@laika:/home/paul# chown root:project42 FileForPaul
root@laika:/home/paul# ls -l FileForPaul
-rw-r--r-- 1 root project42 0 2008-08-06 14:11 FileForPaul
```

32.2. list of special files

When you use **ls -l**, for each file you can see ten characters before the user and group owner. The first character tells us the type of file. Regular files get a **-**, directories get a **d**, symbolic links are shown with an **l**, pipes get a **p**, character devices a **c**, block devices a **b**, and sockets an **s**.

Table 32.1. Unix special files

first character	file type
-	normal file
d	directory
l	symbolic link
p	named pipe
b	block device
c	character device
s	socket

Below a screenshot of a character device (the console) and a block device (the hard disk).

```
paul@debian6lt~$ ls -ld /dev/console /dev/sda
crw----- 1 root root  5, 1 Mar 15 12:45 /dev/console
brw-rw---- 1 root disk  8, 0 Mar 15 12:45 /dev/sda
```

And here you can see a directory, a regular file and a symbolic link.

```
paul@debian6lt~$ ls -ld /etc /etc/hosts /etc/motd
drwxr-xr-x 128 root root 12288 Mar 15 18:34 /etc
-rw-r--r--  1 root root   372 Dec 10 17:36 /etc/hosts
lrwxrwxrwx  1 root root    13 Dec  5 10:36 /etc/motd -> /var/run/motd
```

32.3. permissions

32.3.1. rwx

The nine characters following the file type denote the permissions in three triplets. A permission can be **r** for read access, **w** for write access, and **x** for execute. You need the **r** permission to list (`ls`) the contents of a directory. You need the **x** permission to enter (`cd`) a directory. You need the **w** permission to create files in or remove files from a directory.

Table 32.2. standard Unix file permissions

permission	on a file	on a directory
r (read)	read file contents (<code>cat</code>)	read directory contents (<code>ls</code>)
w (write)	change file contents (<code>vi</code>)	create files in (<code>touch</code>)
x (execute)	execute the file	enter the directory (<code>cd</code>)

32.3.2. three sets of rwx

We already know that the output of `ls -l` starts with ten characters for each file. This screenshot shows a regular file (because the first character is a `-`).

```
paul@RHELv4u4:~/test$ ls -l proc42.bash
-rwxr-xr-- 1 paul proj 984 Feb  6 12:01 proc42.bash
```

Below is a table describing the function of all ten characters.

Table 32.3. Unix file permissions position

position	characters	function
1	-	this is a regular file
2-4	rwx	permissions for the user owner
5-7	r-x	permissions for the group owner
8-10	r--	permissions for others

When you are the **user owner** of a file, then the **user owner permissions** apply to you. The rest of the permissions have no influence on your access to the file.

When you belong to the **group** that is the **group owner** of a file, then the **group owner permissions** apply to you. The rest of the permissions have no influence on your access to the file.

When you are not the **user owner** of a file and you do not belong to the **group owner**, then the **others permissions** apply to you. The rest of the permissions have no influence on your access to the file.

32.3.3. permission examples

Some example combinations on files and directories are seen in this screenshot. The name of the file explains the permissions.

```
paul@laika:~/perms$ ls -lh
total 12K
drwxr-xr-x 2 paul paul 4.0K 2007-02-07 22:26 AllEnter_UserCreateDelete
-rwxrwxrwx 1 paul paul  0 2007-02-07 22:21 EveryoneFullControl.txt
-r--r----- 1 paul paul  0 2007-02-07 22:21 OnlyOwnersRead.txt
-rwxrwx--- 1 paul paul  0 2007-02-07 22:21 OwnersAll_RestNothing.txt
dr-xr-x--- 2 paul paul 4.0K 2007-02-07 22:25 UserAndGroupEnter
dr-x----- 2 paul paul 4.0K 2007-02-07 22:25 OnlyUserEnter
paul@laika:~/perms$
```

To summarise, the first **rwX** triplet represents the permissions for the **user owner**. The second triplet corresponds to the **group owner**; it specifies permissions for all members of that group. The third triplet defines permissions for all **other** users that are not the user owner and are not a member of the group owner.

32.3.4. setting permissions (chmod)

Permissions can be changed with **chmod**. The first example gives the user owner execute permissions.

```
paul@laika:~/perms$ ls -l permissions.txt
-rw-r--r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod u+x permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxr--r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

This example removes the group owners read permission.

```
paul@laika:~/perms$ chmod g-r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx---r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

This example removes the others read permission.

```
paul@laika:~/perms$ chmod o-r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx----- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

This example gives all of them the write permission.

```
paul@laika:~/perms$ chmod a+w permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx-w--w- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

You don't even have to type the a.

```
paul@laika:~/perms$ chmod +x permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwx-wx-wx 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

You can also set explicit permissions.

```
paul@laika:~/perms$ chmod u=rw permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw--wx-wx 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Feel free to make any kind of combination.

```
paul@laika:~/perms$ chmod u=rw,g=rw,o=r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw-rw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

Even fishy combinations are accepted by chmod.

```
paul@laika:~/perms$ chmod u=rwx,ug+rw,o=r permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxrw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```


32.3.5. setting octal permissions

Most Unix administrators will use the **old school** octal system to talk about and set permissions. Look at the triplet bitwise, equating r to 4, w to 2, and x to 1.

Table 32.4. Octal permissions

binary	octal	permission
000	0	---
001	1	--x
010	2	-w-
011	3	-wx
100	4	r--
101	5	r-x
110	6	rw-
111	7	rwX

This makes **777** equal to **rw-rw-rwx** and by the same logic, **654** mean **rw-r-xr--**. The **chmod** command will accept these numbers.

```
paul@laika:~/perms$ chmod 777 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxrwxrwx 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod 664 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rw-rw-r-- 1 paul paul 0 2007-02-07 22:34 permissions.txt
paul@laika:~/perms$ chmod 750 permissions.txt
paul@laika:~/perms$ ls -l permissions.txt
-rwxr-x--- 1 paul paul 0 2007-02-07 22:34 permissions.txt
```

32.3.6. umask

When creating a file or directory, a set of default permissions are applied. These default permissions are determined by the **umask**. The **umask** specifies permissions that you do not want set on by default. You can display the **umask** with the **umask** command.

```
[Harry@RHEL4b ~]$ umask
0002
[Harry@RHEL4b ~]$ touch test
[Harry@RHEL4b ~]$ ls -l test
-rw-rw-r-- 1 Harry Harry 0 Jul 24 06:03 test
[Harry@RHEL4b ~]$
```

As you can also see, the file is also not executable by default. This is a general security feature among Unixes; newly created files are never executable by default. You have to explicitly do a **chmod +x** to make a file executable. This also means that the 1 bit in the **umask** has no meaning--a **umask** of 0022 is the same as 0033.

32.3.7. mkdir -m

When creating directories with **mkdir** you can use the **-m** option to set the **mode**. This screenshot explains.

```
paul@debian5~$ mkdir -m 700 MyDir
paul@debian5~$ mkdir -m 777 Public
paul@debian5~$ ls -dl MyDir/ Public/
drwx----- 2 paul paul 4096 2011-10-16 19:16 MyDir/
drwxrwxrwx 2 paul paul 4096 2011-10-16 19:16 Public/
```

32.3.8. cp -p

To preserve permissions and time stamps from source files, use **cp -p**.

```
paul@laika:~/perms$ cp file* cp
paul@laika:~/perms$ cp -p file* cpp
paul@laika:~/perms$ ll *
-rwx----- 1 paul paul    0 2008-08-25 13:26 file33
-rwxr-x--- 1 paul paul    0 2008-08-25 13:26 file42

cp:
total 0
-rwx----- 1 paul paul 0 2008-08-25 13:34 file33
-rwxr-x--- 1 paul paul 0 2008-08-25 13:34 file42

cpp:
total 0
-rwx----- 1 paul paul 0 2008-08-25 13:26 file33
-rwxr-x--- 1 paul paul 0 2008-08-25 13:26 file42
```

32.4. practice: standard file permissions

1. As normal user, create a directory ~/permissions. Create a file owned by yourself in there.
2. Copy a file owned by root from /etc/ to your permissions dir, who owns this file now ?
3. As root, create a file in the users ~/permissions directory.
4. As normal user, look at who owns this file created by root.
5. Change the ownership of all files in ~/permissions to yourself.
6. Make sure you have all rights to these files, and others can only read.
7. With chmod, is 770 the same as rwxrwx--- ?
8. With chmod, is 664 the same as r-xr-xr-- ?
9. With chmod, is 400 the same as r----- ?
10. With chmod, is 734 the same as rwxr-xr-- ?
- 11a. Display the umask in octal and in symbolic form.
- 11b. Set the umask to 077, but use the symbolic format to set it. Verify that this works.
12. Create a file as root, give only read to others. Can a normal user read this file ? Test writing to this file with vi.
- 13a. Create a file as normal user, give only read to others. Can another normal user read this file ? Test writing to this file with vi.
- 13b. Can root read this file ? Can root write to this file with vi ?
14. Create a directory that belongs to a group, where every member of that group can read and write to files, and create files. Make sure that people can only delete their own files.

32.5. solution: standard file permissions

1. As normal user, create a directory ~/permissions. Create a file owned by yourself in there.

```
mkdir ~/permissions ; touch ~/permissions/myfile.txt
```

2. Copy a file owned by root from /etc/ to your permissions dir, who owns this file now ?

```
cp /etc/hosts ~/permissions/
```

The copy is owned by you.

3. As root, create a file in the users ~/permissions directory.

```
(become root)# touch /home/username/permissions/rootfile
```

4. As normal user, look at who owns this file created by root.

```
ls -l ~/permissions
```

The file created by root is owned by root.

5. Change the ownership of all files in ~/permissions to yourself.

```
chown user ~/permissions/*
```

You cannot become owner of the file that belongs to root.

6. Make sure you have all rights to these files, and others can only read.

```
chmod 644 (on files)
```

```
chmod 755 (on directories)
```

7. With chmod, is 770 the same as rwxrwx--- ?

yes

8. With chmod, is 664 the same as r-xr-xr-- ?

No

9. With chmod, is 400 the same as r----- ?

yes

10. With chmod, is 734 the same as rwxr-xr-- ?

no

- 11a. Display the umask in octal and in symbolic form.

```
umask ; umask -S
```

- 11b. Set the umask to 077, but use the symbolic format to set it. Verify that this works.

```
umask -S u=rwx,go=
```

12. Create a file as root, give only read to others. Can a normal user read this file ? Test writing to this file with vi.

```
(become root)
```

```
# echo hello > /home/username/root.txt
```

```
# chmod 744 /home/username/root.txt
```

```
(become user)
```

```
vi ~/root.txt
```

13a. Create a file as normal user, give only read to others. Can another normal user read this file ? Test writing to this file with vi.

```
echo hello > file ; chmod 744 file
```

Yes, others can read this file

13b. Can root read this file ? Can root write to this file with vi ?

Yes, root can read and write to this file. Permissions do not apply to root.

14. Create a directory that belongs to a group, where every member of that group can read and write to files, and create files. Make sure that people can only delete their own files.

```
mkdir /home/project42 ; groupadd project42
```

```
chgrp project42 /home/project42 ; chmod 775 /home/project42
```

You can not yet do the last part of this exercise...

Chapter 33. advanced file permissions

33.1. sticky bit on directory

You can set the **sticky bit** on a directory to prevent users from removing files that they do not own as a user owner. The sticky bit is displayed at the same location as the x permission for others. The sticky bit is represented by a **t** (meaning x is also there) or a **T** (when there is no x for others).

```
root@RHELv4u4:~# mkdir /project55
root@RHELv4u4:~# ls -ld /project55
drwxr-xr-x  2 root root 4096 Feb  7 17:38 /project55
root@RHELv4u4:~# chmod +t /project55/
root@RHELv4u4:~# ls -ld /project55
drwxr-xr-t  2 root root 4096 Feb  7 17:38 /project55
root@RHELv4u4:~#
```

The **sticky bit** can also be set with octal permissions, it is binary 1 in the first of four triplets.

```
root@RHELv4u4:~# chmod 1775 /project55/
root@RHELv4u4:~# ls -ld /project55
drwxrwxr-t  2 root root 4096 Feb  7 17:38 /project55
root@RHELv4u4:~#
```

You will typically find the **sticky bit** on the **/tmp** directory.

```
root@barry:~# ls -ld /tmp
drwxrwxrwt 6 root root 4096 2009-06-04 19:02 /tmp
```

33.2. setgid bit on directory

setgid can be used on directories to make sure that all files inside the directory are owned by the group owner of the directory. The **setgid** bit is displayed at the same location as the x permission for group owner. The **setgid** bit is represented by an **s** (meaning x is also there) or a **S** (when there is no x for the group owner). As this example shows, even though **root** does not belong to the group **proj55**, the files created by root in **/project55** will belong to **proj55** since the **setgid** is set.

```
root@RHELv4u4:~# groupadd proj55
root@RHELv4u4:~# chown root:proj55 /project55/
root@RHELv4u4:~# chmod 2775 /project55/
root@RHELv4u4:~# touch /project55/fromroot.txt
root@RHELv4u4:~# ls -ld /project55/
drwxrwsr-x  2 root proj55 4096 Feb  7 17:45 /project55/
root@RHELv4u4:~# ls -l /project55/
total 4
-rw-r--r--  1 root proj55 0 Feb  7 17:45 fromroot.txt
root@RHELv4u4:~#
```

You can use the **find** command to find all **setgid** directories.

```
paul@laika:~$ find / -type d -perm -2000 2> /dev/null
/var/log/mysql
/var/log/news
/var/local
...
```

33.3. setgid and setuid on regular files

These two permissions cause an executable file to be executed with the permissions of the **file owner** instead of the **executing owner**. This means that if any user executes a program that belongs to the **root user**, and the **setuid** bit is set on that program, then the program runs as **root**. This can be dangerous, but sometimes this is good for security.

Take the example of passwords; they are stored in **/etc/shadow** which is only readable by **root**. (The **root** user never needs permissions anyway.)

```
root@RHELV4u4:~# ls -l /etc/shadow
-r----- 1 root root 1260 Jan 21 07:49 /etc/shadow
```

Changing your password requires an update of this file, so how can normal non-root users do this? Let's take a look at the permissions on the **/usr/bin/passwd**.

```
root@RHELV4u4:~# ls -l /usr/bin/passwd
-r-s--x--x 1 root root 21200 Jun 17 2005 /usr/bin/passwd
```

When running the **passwd** program, you are executing it with **root** credentials.

You can use the **find** command to find all **setuid** programs.

```
paul@laika:~$ find /usr/bin -type f -perm -04000
/usr/bin/arping
/usr/bin/kgrantpty
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/fping6
/usr/bin/passwd
/usr/bin/gpasswd
...
```

In most cases, setting the **setuid** bit on executables is sufficient. Setting the **setgid** bit will result in these programs to run with the credentials of their group owner.

33.4. setuid on sudo

The **sudo** binary has the **setuid** bit set, so any user can run it with the effective userid of root.

```
paul@rhel65:~$ ls -l $(which sudo)
---s--x--x. 1 root root 123832 Oct 7 2013 /usr/bin/sudo
paul@rhel65:~$
```


33.5. practice: sticky, setuid and setgid bits

- 1a. Set up a directory, owned by the group sports.
- 1b. Members of the sports group should be able to create files in this directory.
- 1c. All files created in this directory should be group-owned by the sports group.
- 1d. Users should be able to delete only their own user-owned files.
- 1e. Test that this works!
2. Verify the permissions on **/usr/bin/passwd**. Remove the **setuid**, then try changing your password as a normal user. Reset the permissions back and try again.
3. If time permits (or if you are waiting for other students to finish this practice), read about file attributes in the man page of `chattr` and `lsattr`. Try setting the `i` attribute on a file and test that it works.

33.6. solution: sticky, setuid and setgid bits

1a. Set up a directory, owned by the group sports.

```
groupadd sports
```

```
mkdir /home/sports
```

```
chown root:sports /home/sports
```

1b. Members of the sports group should be able to create files in this directory.

```
chmod 770 /home/sports
```

1c. All files created in this directory should be group-owned by the sports group.

```
chmod 2770 /home/sports
```

1d. Users should be able to delete only their own user-owned files.

```
chmod +t /home/sports
```

1e. Test that this works!

Log in with different users (group members and others and root), create files and watch the permissions. Try changing and deleting files...

2. Verify the permissions on **/usr/bin/passwd**. Remove the **setuid**, then try changing your password as a normal user. Reset the permissions back and try again.

```
root@deb503:~# ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
root@deb503:~# chmod 755 /usr/bin/passwd
root@deb503:~# ls -l /usr/bin/passwd
-rwxr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
```

A normal user cannot change password now.

```
root@deb503:~# chmod 4755 /usr/bin/passwd
root@deb503:~# ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 31704 2009-11-14 15:41 /usr/bin/passwd
```

3. If time permits (or if you are waiting for other students to finish this practice), read about file attributes in the man page of **chattr** and **lsattr**. Try setting the **i** attribute on a file and test that it works.

```
paul@laika:~$ sudo su -
[sudo] password for paul:
root@laika:~# mkdir attr
root@laika:~# cd attr/
root@laika:~/attr# touch file42
root@laika:~/attr# lsattr
----- ./file42
root@laika:~/attr# chattr +i file42
```

```
root@laika:~/attr# lsattr
----i----- ./file42
root@laika:~/attr# rm -rf file42
rm: cannot remove `file42': Operation not permitted
root@laika:~/attr# chattr -i file42
root@laika:~/attr# rm -rf file42
root@laika:~/attr#
```

Chapter 34. access control lists

Standard Unix permissions might not be enough for some organisations. This chapter introduces **access control lists** or **acl's** to further protect files and directories.

34.1. acl in /etc/fstab

File systems that support **access control lists**, or **acls**, have to be mounted with the **acl** option listed in **/etc/fstab**. In the example below, you can see that the root file system has **acl** support, whereas **/home/data** does not.

```
root@laika:~# tail -4 /etc/fstab
/dev/sda1      /              ext3          acl,relatime  0 1
/dev/sdb2      /home/data     auto          noacl,defaults 0 0
pasha:/home/r  /home/pasha   nfs           defaults      0 0
wolf:/srv/data /home/wolf     nfs           defaults      0 0
```

34.2. getfacl

Reading **acls** can be done with **/usr/bin/getfacl**. This screenshot shows how to read the **acl** of **file33** with **getfacl**.

```
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
group::r--
mask::rwx
other::r--
```

34.3. setfacl

Writing or changing **acls** can be done with **/usr/bin/setfacl**. These screenshots show how to change the **acl** of **file33** with **setfacl**.

First we add **user sandra** with octal permission **7** to the **acl**.

```
paul@laika:~/test$ setfacl -m u:sandra:7 file33
```

Then we add the **group tennis** with octal permission **6** to the **acl** of the same file.

```
paul@laika:~/test$ setfacl -m g:tennis:6 file33
```

The result is visible with **getfacl**.

```
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
user:sandra:rwx
group::r--
group:tennis:rw-
mask::rwx
other::r--
```

34.4. remove an acl entry

The **-x** option of the **setfacl** command will remove an **acl** entry from the targeted file.

```
paul@laika:~/test$ setfacl -m u:sandra:7 file33
paul@laika:~/test$ getfacl file33 | grep sandra
user:sandra:rwx
paul@laika:~/test$ setfacl -x sandra file33
paul@laika:~/test$ getfacl file33 | grep sandra
```

Note that omitting the **u** or **g** when defining the **acl** for an account will default it to a user account.

34.5. remove the complete acl

The **-b** option of the **setfacl** command will remove the **acl** from the targeted file.

```
paul@laika:~/test$ setfacl -b file33
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
group::r--
other::r--
```

34.6. the acl mask

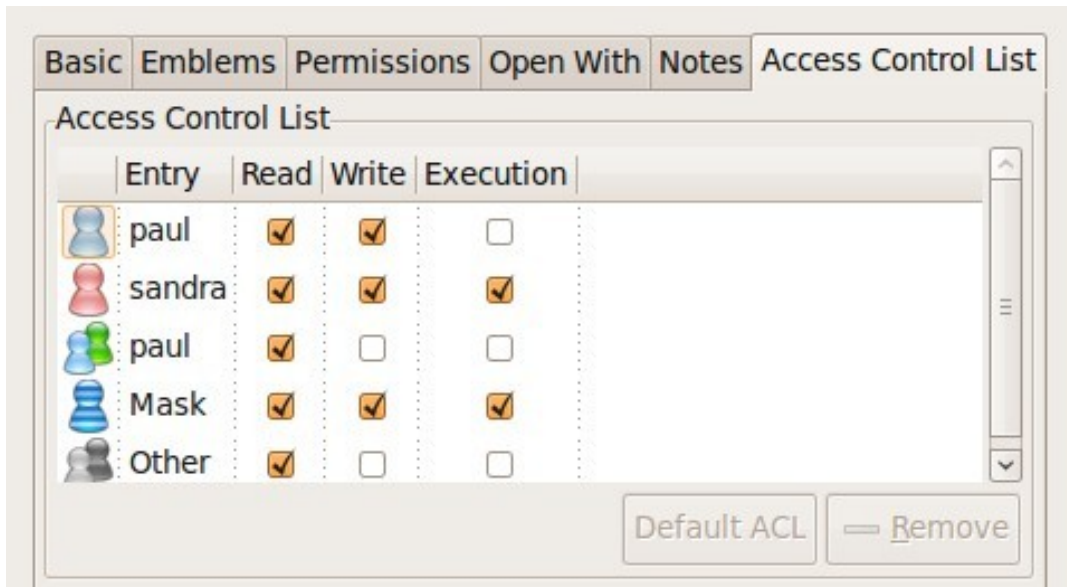
The **acl mask** defines the maximum effective permissions for any entry in the **acl**. This **mask** is calculated every time you execute the **setfacl** or **chmod** commands.

You can prevent the calculation by using the **--no-mask** switch.

```
paul@laika:~/test$ setfacl --no-mask -m u:sandra:7 file33
paul@laika:~/test$ getfacl file33
# file: file33
# owner: paul
# group: paul
user::rw-
user:sandra:rwx   #effective:rw-
group::r--
mask::rw-
other::r--
```

34.7. eiciel

Desktop users might want to use **eiciel** to manage **acls** with a graphical tool.



You will need to install **eiciel** and **nautilus-actions** to have an extra tab in **nautilus** to manage **acls**.

```
paul@laika:~$ sudo aptitude install eiciel nautilus-actions
```

Chapter 35. file links

An average computer using Linux has a file system with many **hard links** and **symbolic links**.

To understand links in a file system, you first have to understand what an **inode** is.

35.1. inodes

35.1.1. inode contents

An **inode** is a data structure that contains metadata about a file. When the file system stores a new file on the hard disk, it stores not only the contents (data) of the file, but also extra properties like the name of the file, the creation date, its permissions, the owner of the file, and more. All this information (except the name of the file and the contents of the file) is stored in the **inode** of the file.

The **ls -l** command will display some of the inode contents, as seen in this screenshot.

```
root@rhel53 ~# ls -ld /home/project42/
drwxr-xr-x 4 root pro42 4.0K Mar 27 14:29 /home/project42/
```

35.1.2. inode table

The **inode table** contains all of the **inodes** and is created when you create the file system (with **mkfs**). You can use the **df -i** command to see how many **inodes** are used and free on mounted file systems.

```
root@rhel53 ~# df -i
Filesystem          Inodes    IUsed   IFree IUse% Mounted on
/dev/mapper/VolGroup00-LogVol00
                    4947968  115326 4832642    3% /
/dev/hda1            26104      45   26059    1% /boot
tmpfs                64417        1   64416    1% /dev/shm
/dev/sda1            262144     2207  259937    1% /home/project42
/dev/sdb1             74400     5519   68881    8% /home/project33
/dev/sdb5              0          0         0    - /home/sales
/dev/sdb6            100744      11  100733    1% /home/research
```

In the **df -i** screenshot above you can see the **inode** usage for several mounted **file systems**. You don't see numbers for **/dev/sdb5** because it is a **fat** file system.

35.1.3. inode number

Each **inode** has a unique number (the inode number). You can see the **inode** numbers with the **ls -li** command.

```
paul@RHELv4u4:~/test$ touch file1
paul@RHELv4u4:~/test$ touch file2
paul@RHELv4u4:~/test$ touch file3
paul@RHELv4u4:~/test$ ls -li
total 12
817266 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file1
817267 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb  5 15:38 file3
paul@RHELv4u4:~/test$
```

These three files were created one after the other and got three different **inodes** (the first column). All the information you see with this **ls** command resides in the **inode**, except for the filename (which is contained in the directory).

35.1.4. inode and file contents

Let's put some data in one of the files.

```
paul@RHELv4u4:~/test$ ls -li
total 16
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 1 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
paul@RHELv4u4:~/test$ cat file2
It is winter now and it is very cold.
We do not like the cold, we prefer hot summer nights.
paul@RHELv4u4:~/test$
```

The data that is displayed by the **cat** command is not in the **inode**, but somewhere else on the disk. The **inode** contains a pointer to that data.

35.2. about directories

35.2.1. a directory is a table

A **directory** is a special kind of file that contains a table which maps filenames to inodes. Listing our current directory with **ls -ali** will display the contents of the directory file.

```
paul@RHELv4u4:~/test$ ls -ali
total 32
817262 drwxrwxr-x 2 paul paul 4096 Feb 5 15:42 .
800768 drwx----- 16 paul paul 4096 Feb 5 15:42 ..
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 1 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
paul@RHELv4u4:~/test$
```

35.2.2. . and ..

You can see five names, and the mapping to their five inodes. The dot **.** is a mapping to itself, and the dotdot **..** is a mapping to the parent directory. The three other names are mappings to different inodes.

35.3. hard links

35.3.1. creating hard links

When we create a **hard link** to a file with **ln**, an extra entry is added in the directory. A new file name is mapped to an existing inode.

```
paul@RHELv4u4:~/test$ ln file2 hardlink_to_file2
paul@RHELv4u4:~/test$ ls -li
total 24
817266 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file1
817270 -rw-rw-r-- 2 paul paul 92 Feb 5 15:42 file2
817268 -rw-rw-r-- 1 paul paul 0 Feb 5 15:38 file3
817270 -rw-rw-r-- 2 paul paul 92 Feb 5 15:42 hardlink_to_file2
paul@RHELv4u4:~/test$
```

Both files have the same inode, so they will always have the same permissions and the same owner. Both files will have the same content. Actually, both files are equal now, meaning you can safely remove the original file, the hardlinked file will remain. The inode contains a counter, counting the number of hard links to itself. When the counter drops to zero, then the inode is emptied.

35.3.2. finding hard links

You can use the **find** command to look for files with a certain inode. The screenshot below shows how to search for all filenames that point to **inode** 817270. Remember that an **inode** number is unique to its partition.

```
paul@RHELv4u4:~/test$ find / -inum 817270 2> /dev/null
/home/paul/test/file2
/home/paul/test/hardlink_to_file2
```

35.4. symbolic links

Symbolic links (sometimes called **soft links**) do not link to inodes, but create a name to name mapping. Symbolic links are created with **ln -s**. As you can see below, the **symbolic link** gets an inode of its own.

```
paul@RHELv4u4:~/test$ ln -s file2 symlink_to_file2
paul@RHELv4u4:~/test$ ls -li
total 32
817273 -rw-rw-r-- 1 paul paul 13 Feb  5 17:06 file1
817270 -rw-rw-r-- 2 paul paul 106 Feb  5 17:04 file2
817268 -rw-rw-r-- 1 paul paul  0 Feb  5 15:38 file3
817270 -rw-rw-r-- 2 paul paul 106 Feb  5 17:04 hardlink_to_file2
817267 lrwxrwxrwx 1 paul paul  5 Feb  5 16:55 symlink_to_file2 -> file2
paul@RHELv4u4:~/test$
```

Permissions on a symbolic link have no meaning, since the permissions of the target apply. Hard links are limited to their own partition (because they point to an inode), symbolic links can link anywhere (other file systems, even networked).

35.5. removing links

Links can be removed with **rm**.

```
paul@laika:~$ touch data.txt
paul@laika:~$ ln -s data.txt sl_data.txt
paul@laika:~$ ln data.txt hl_data.txt
paul@laika:~$ rm sl_data.txt
paul@laika:~$ rm hl_data.txt
```

35.6. practice : links

1. Create two files named winter.txt and summer.txt, put some text in them.
2. Create a hard link to winter.txt named hlwinter.txt.
3. Display the inode numbers of these three files, the hard links should have the same inode.
4. Use the find command to list the two hardlinked files
5. Everything about a file is in the inode, except two things : name them!
6. Create a symbolic link to summer.txt called slsummer.txt.
7. Find all files with inode number 2. What does this information tell you ?
8. Look at the directories /etc/init.d/ /etc/rc2.d/ /etc/rc3.d/ ... do you see the links ?
9. Look in /lib with ls -l...
10. Use **find** to look in your home directory for regular files that do not(!) have one hard link.

35.7. solution : links

1. Create two files named winter.txt and summer.txt, put some text in them.

```
echo cold > winter.txt ; echo hot > summer.txt
```

2. Create a hard link to winter.txt named hlwinter.txt.

```
ln winter.txt hlwinter.txt
```

3. Display the inode numbers of these three files, the hard links should have the same inode.

```
ls -li winter.txt summer.txt hlwinter.txt
```

4. Use the find command to list the two hardlinked files

```
find . -inum xyz #replace xyz with the inode number
```

5. Everything about a file is in the inode, except two things : name them!

The name of the file is in a directory, and the contents is somewhere on the disk.

6. Create a symbolic link to summer.txt called slsummer.txt.

```
ln -s summer.txt slsummer.txt
```

7. Find all files with inode number 2. What does this information tell you ?

It tells you there is more than one inode table (one for every formatted partition + virtual file systems)

8. Look at the directories /etc/init.d/ /etc/rc.d/ /etc/rc3.d/ ... do you see the links ?

```
ls -l /etc/init.d
```

```
ls -l /etc/rc2.d
```

```
ls -l /etc/rc3.d
```

9. Look in /lib with ls -l...

```
ls -l /lib
```

10. Use **find** to look in your home directory for regular files that do not(!) have one hard link.

```
find ~ ! -links 1 -type f
```

Part X. process management

Table of Contents

36. introduction to processes	337
36.1. terminology	338
36.2. basic process management	339
36.3. signalling processes	343
36.4. practice : basic process management	346
36.5. solution : basic process management	347
37. process priorities	349
37.1. priority and nice values	350
37.2. practice : process priorities	353
37.3. solution : process priorities	354
38. background jobs	356
38.1. background processes	357
38.2. practice : background processes	359
38.3. solution : background processes	360

Chapter 36. introduction to processes

36.1. terminology

36.1.1. process

A **process** is compiled source code that is currently running on the system.

36.1.2. PID

All processes have a **process id** or **PID**.

36.1.3. PPID

Every process has a parent process (with a **PPID**). The **child** process is often started by the **parent** process.

36.1.4. init

The **init** process always has process ID 1. The **init** process is started by the **kernel** itself so technically it does not have a parent process. **init** serves as a **foster parent** for **orphaned** processes.

36.1.5. kill

When a process stops running, the process dies, when you want a process to die, you **kill** it.

36.1.6. daemon

Processes that start at system startup and keep running forever are called **daemon** processes or **daemons**. These **daemons** never die.

36.1.7. zombie

When a process is killed, but it still shows up on the system, then the process is referred to as **zombie**. You cannot kill zombies, because they are already dead.

36.2. basic process management

36.2.1. \$\$ and \$PPID

Some shell environment variables contain information about processes. The **\$\$** variable will hold your current **process ID**, and **\$PPID** contains the **parent PID**. Actually **\$\$** is a shell parameter and not a variable, you cannot assign a value to it.

Below we use **echo** to display the values of **\$\$** and **\$PPID**.

```
[paul@RHEL4b ~]$ echo $$ $PPID
4224 4223
```

36.2.2. pidof

You can find all process id's by name using the **pidof** command.

```
root@rhel53 ~# pidof mingetty
2819 2798 2797 2796 2795 2794
```

36.2.3. parent and child

Processes have a **parent-child** relationship. Every process has a parent process.

When starting a new **bash** you can use **echo** to verify that the **pid** from before is the **ppid** of the new shell. The **child** process from above is now the **parent** process.

```
[paul@RHEL4b ~]$ bash
[paul@RHEL4b ~]$ echo $$ $PPID
4812 4224
```

Typing **exit** will end the current process and brings us back to our original values for **\$\$** and **\$PPID**.

```
[paul@RHEL4b ~]$ echo $$ $PPID
4812 4224
[paul@RHEL4b ~]$ exit
exit
[paul@RHEL4b ~]$ echo $$ $PPID
4224 4223
[paul@RHEL4b ~]$
```

36.2.4. fork and exec

A process starts another process in two phases. First the process creates a **fork** of itself, an identical copy. Then the forked process executes an **exec** to replace the forked process with the target child process.

```
[paul@RHEL4b ~]$ echo $$
4224
[paul@RHEL4b ~]$ bash
[paul@RHEL4b ~]$ echo $$ $PPID
5310 4224
[paul@RHEL4b ~]$
```

36.2.5. exec

With the **exec** command, you can execute a process without forking a new process. In the following screenshot a **Korn shell** (ksh) is started and is being replaced with a **bash shell** using the **exec** command. The **pid** of the **bash shell** is the same as the **pid** of the **Korn shell**. Exiting the child **bash shell** will get me back to the parent **bash**, not to the **Korn shell** (which does not exist anymore).

```
[paul@RHEL4b ~]$ echo $$
4224
[paul@RHEL4b ~]$ ksh
$ echo $$ $PPID
5343 4224
$ exec bash
[paul@RHEL4b ~]$ echo $$ $PPID
5343 4224
[paul@RHEL4b ~]$ exit
exit
[paul@RHEL4b ~]$ echo $$
4224
```

PID of bash

PID of ksh and bash

PID of bash and bash

36.2.6. ps

One of the most common tools on Linux to look at processes is **ps**. The following screenshot shows the parent child relationship between three bash processes.

```
[paul@RHEL4b ~]$ echo $$ $PPID
4224 4223
[paul@RHEL4b ~]$ bash
[paul@RHEL4b ~]$ echo $$ $PPID
4866 4224
[paul@RHEL4b ~]$ bash
[paul@RHEL4b ~]$ echo $$ $PPID
4884 4866
[paul@RHEL4b ~]$ ps fx
  PID TTY          STAT       TIME COMMAND
 4223 ?            S          0:01 sshd: paul@pts/0
 4224 pts/0        Ss         0:00  \_ -bash
 4866 pts/0        S          0:00      \_ bash
 4884 pts/0        S          0:00          \_ bash
 4902 pts/0        R+         0:00              \_ ps fx
[paul@RHEL4b ~]$ exit
exit
[paul@RHEL4b ~]$ ps fx
  PID TTY          STAT       TIME COMMAND
 4223 ?            S          0:01 sshd: paul@pts/0
 4224 pts/0        Ss         0:00  \_ -bash
 4866 pts/0        S          0:00      \_ bash
 4903 pts/0        R+         0:00          \_ ps fx
[paul@RHEL4b ~]$ exit
exit
[paul@RHEL4b ~]$ ps fx
  PID TTY          STAT       TIME COMMAND
 4223 ?            S          0:01 sshd: paul@pts/0
 4224 pts/0        Ss         0:00  \_ -bash
 4904 pts/0        R+         0:00      \_ ps fx
[paul@RHEL4b ~]$
```

On Linux, **ps fax** is often used. On Solaris **ps -ef** (which also works on Linux) is common. Here is a partial output from **ps fax**.

```
[paul@RHEL4a ~]$ ps fax
PID TTY          STAT       TIME COMMAND
1 ?            S          0:00 init [5]

...

3713 ?          Ss         0:00 /usr/sbin/sshd
5042 ?          Ss         0:00  \_ sshd: paul [priv]
5044 ?          S          0:00      \_ sshd: paul@pts/1
5045 pts/1        Ss         0:00          \_ -bash
5077 pts/1        R+         0:00              \_ ps fax
```

36.2.7. pgrep

Similar to the **ps -C**, you can also use **pgrep** to search for a process by its command name.

```
[paul@RHEL5 ~]$ sleep 1000 &
[1] 32558
[paul@RHEL5 ~]$ pgrep sleep
32558
[paul@RHEL5 ~]$ ps -C sleep
  PID TTY          TIME CMD
 32558 pts/3        00:00:00 sleep
```

You can also list the command name of the process with **pgrep**.

```
paul@laika:~$ pgrep -l sleep
9661 sleep
```

36.2.8. top

Another popular tool on Linux is **top**. The **top** tool can order processes according to **cpu usage** or other properties. You can also **kill** processes from within **top**. Press **h** inside **top** for help.

In case of trouble, **top** is often the first tool to fire up, since it also provides you memory and swap space information.

36.3. signalling processes

36.3.1. kill

The **kill** command will kill (or stop) a process. The screenshot shows how to use a standard **kill** to stop the process with **pid** 1942.

```
paul@ubuntu910:~$ kill 1942
paul@ubuntu910:~$
```

By using the **kill** we are sending a **signal** to the process.

36.3.2. list signals

Running processes can receive signals from each other or from the users. You can have a list of signals by typing **kill -l**, that is a letter **l**, not the number 1.

```
[paul@RHEL4a ~]$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL
5) SIGTRAP     6) SIGABRT    7) SIGBUS      8) SIGFPE
9) SIGKILL    10) SIGUSR1   11) SIGSEGV    12) SIGUSR2
13) SIGPIPE   14) SIGALRM   15) SIGTERM    17) SIGCHLD
18) SIGCONT   19) SIGSTOP   20) SIGTSTP    21) SIGTTIN
22) SIGTTOU   23) SIGURG    24) SIGXCPU    25) SIGXFSZ
26) SIGVTALRM 27) SIGPROF   28) SIGWINCH   29) SIGIO
30) SIGPWR     31) SIGSYS    34) SIGRTMIN   35) SIGRTMIN+1
36) SIGRTMIN+2 37) SIGRTMIN+3 38) SIGRTMIN+4 39) SIGRTMIN+5
40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8 43) SIGRTMIN+9
44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13
52) SIGRTMAX-12 53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9
56) SIGRTMAX-8 57) SIGRTMAX-7 58) SIGRTMAX-6 59) SIGRTMAX-5
60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2 63) SIGRTMAX-1
64) SIGRTMAX
[paul@RHEL4a ~]$
```

36.3.3. kill -1 (SIGHUP)

It is common on Linux to use the first signal **SIGHUP** (or HUP or 1) to tell a process that it should re-read its configuration file. Thus, the **kill -1 1** command forces the **init** process (**init** always runs with **pid** 1) to re-read its configuration file.

```
root@deb503:~# kill -1 1
root@deb503:~#
```

It is up to the developer of the process to decide whether the process can do this running, or whether it needs to stop and start. It is up to the user to read the documentation of the program.

36.3.4. kill -15 (SIGTERM)

The **SIGTERM** signal is also called a **standard kill**. Whenever **kill** is executed without specifying the signal, a **kill -15** is assumed.

Both commands in the screenshot below are identical.

```
paul@ubuntu910:~$ kill 1942
paul@ubuntu910:~$ kill -15 1942
```

36.3.5. kill -9 (SIGKILL)

The **SIGKILL** is different from most other signals in that it is not being sent to the process, but to the **Linux kernel**. A **kill -9** is also called a **sure kill**. The **kernel** will shoot down the process. As a developer you have no means to intercept a **kill -9** signal.

```
root@rhel53 ~# kill -9 3342
```

36.3.6. SIGSTOP and SIGCONT

A running process can be **suspended** when it receives a **SIGSTOP** signal. This is the same as **kill -19** on Linux, but might have a different number in other Unix systems.

A **suspended** process does not use any **cpu cycles**, but it stays in memory and can be re-animated with a **SIGCONT** signal (**kill -18** on Linux).

Both signals will be used in the section about **background** processes.

36.3.7. pkill

You can use the **pkill** command to kill a process by its command name.

```
[paul@RHEL5 ~]$ sleep 1000 &
[1] 30203
[paul@RHEL5 ~]$ pkill sleep
[1]+  Terminated                  sleep 1000
[paul@RHEL5 ~]$
```

36.3.8. killall

The **killall** command will send a **signal 15** to all processes with a certain name.

```
paul@rhel65:~$ sleep 8472 &
[1] 18780
paul@rhel65:~$ sleep 1201 &
[2] 18781
paul@rhel65:~$ jobs
[1]-  Running                  sleep 8472 &
[2]+  Running                  sleep 1201 &
paul@rhel65:~$ killall sleep
[1]-  Terminated              sleep 8472
[2]+  Terminated              sleep 1201
paul@rhel65:~$ jobs
paul@rhel65:~$
```

36.3.9. killall5

Its SysV counterpart **killall5** can be used when shutting down the system. This screenshot shows how Red Hat Enterprise Linux 5.3 uses **killall5** when halting the system.

```
root@rhel53 ~# grep killall /etc/init.d/halt
action $"Sending all processes the TERM signal..." /sbin/killall5 -15
action $"Sending all processes the KILL signal..." /sbin/killall5 -9
```

36.3.10. top

Inside **top** the **k** key allows you to select a **signal** and **pid** to kill. Below is a partial screenshot of the line just below the summary in **top** after pressing **k**.

```
PID to kill: 1932
Kill PID 1932 with signal [15]: 9
```

36.4. practice : basic process management

1. Use **ps** to search for the **init** process by name.
2. What is the **process id** of the **init** process ?
3. Use the **who am i** command to determine your terminal name.
4. Using your terminal name from above, use **ps** to find all processes associated with your terminal.
5. What is the **process id** of your shell ?
6. What is the **parent process id** of your shell ?
7. Start two instances of the **sleep 3342** in background.
8. Locate the **process id** of all **sleep** commands.
9. Display only those two **sleep** processes in **top**. Then quit top.
10. Use a **standard kill** to kill one of the **sleep** processes.
11. Use one command to kill all **sleep** processes.

36.5. solution : basic process management

1. Use **ps** to search for the **init** process by name.

```
root@rhel53 ~# ps -C init
  PID TTY          TIME CMD
    1 ?            00:00:04 init
```

2. What is the **process id** of the **init** process ?

1

3. Use the **who am i** command to determine your terminal name.

```
root@rhel53 ~# who am i
paul      pts/0      2010-04-12 17:44 (192.168.1.38)
```

4. Using your terminal name from above, use **ps** to find all processes associated with your terminal.

```
oot@rhel53 ~# ps fax | grep pts/0
 2941 ?          S      0:00      \_ sshd: paul@pts/0
 2942 pts/0      Ss     0:00          \_ -bash
 2972 pts/0      S      0:00              \_ su -
 2973 pts/0      S      0:00                  \_ -bash
 3808 pts/0      R+     0:00                      \_ ps fax
 3809 pts/0      R+     0:00                          \_ grep pts/0
```

or also

```
root@rhel53 ~# ps -ef | grep pts/0
paul      2941  2939  0 17:44 ?          00:00:00 sshd: paul@pts/0
paul      2942  2941  0 17:44 pts/0      00:00:00 -bash
root      2972  2942  0 17:45 pts/0      00:00:00 su -
root      2973  2972  0 17:45 pts/0      00:00:00 -bash
root      3816  2973  0 21:25 pts/0      00:00:00 ps -ef
root      3817  2973  0 21:25 pts/0      00:00:00 grep pts/0
```

5. What is the **process id** of your shell ?

2973 in the screenshot above, probably different for you

echo \$\$ should display same number as the one you found

6. What is the **parent process id** of your shell ?

2972 in the screenshot above, probably different for you

in this example the PPID is from the **su -** command, but when inside gnome then for example gnome-terminal can be the parent process

7. Start two instances of the **sleep 3342** in background.

```
sleep 3342 &  
sleep 3342 &
```

8. Locate the **process id** of all **sleep** commands.

```
pidof sleep
```

9. Display only those two **sleep** processes in **top**. Then quit top.

```
top -p pidx,pidy (replace pidx pidy with the actual numbers)
```

10. Use a **standard kill** to kill one of the **sleep** processes.

```
kill pidx
```

11. Use one command to kill all **sleep** processes.

```
pkill sleep
```

Chapter 37. process priorities

37.1. priority and nice values

37.1.1. introduction

All processes have a **priority** and a **nice** value. Higher priority processes will get more **cpu time** than lower priority processes. You can influence this with the **nice** and **renice** commands.

37.1.2. pipes (mkfifo)

Processes can communicate with each other via **pipes**. These **pipes** can be created with the **mkfifo** command.

The screenshots shows the creation of four distinct pipes (in a new directory).

```
paul@ubuntu910:~$ mkdir procs
paul@ubuntu910:~$ cd procs/
paul@ubuntu910:~/procs$ mkfifo pipe33a pipe33b pipe42a pipe42b
paul@ubuntu910:~/procs$ ls -l
total 0
prw-r--r-- 1 paul paul 0 2010-04-12 13:21 pipe33a
prw-r--r-- 1 paul paul 0 2010-04-12 13:21 pipe33b
prw-r--r-- 1 paul paul 0 2010-04-12 13:21 pipe42a
prw-r--r-- 1 paul paul 0 2010-04-12 13:21 pipe42b
paul@ubuntu910:~/procs$
```

37.1.3. some fun with cat

To demonstrate the use of the **top** and **renice** commands we will make the **cat** command use the previously created **pipes** to generate a full load on the **cpu**.

The **cat** is copied with a distinct name to the current directory. (This enables us to easily recognize the processes within **top**. You could do the same exercise without copying the **cat** command, but using different users. Or you could just look at the **pid** of each process.)

```
paul@ubuntu910:~/procs$ cp /bin/cat proj33
paul@ubuntu910:~/procs$ cp /bin/cat proj42
paul@ubuntu910:~/procs$ echo -n x | ./proj33 - pipe33a > pipe33b &
[1] 1670
paul@ubuntu910:~/procs$ ./proj33 <pipe33b >pipe33a &
[2] 1671
paul@ubuntu910:~/procs$ echo -n z | ./proj42 - pipe42a > pipe42b &
[3] 1673
paul@ubuntu910:~/procs$ ./proj42 <pipe42b >pipe42a &
[4] 1674
```

The commands you see above will create two **proj33** processes that use **cat** to bounce the x character between **pipe33a** and **pipe33b**. And ditto for the z character and **proj42**.

37.1.4. top

Just running **top** without options or arguments will display all processes and an overview of innformation. The top of the **top** screen might look something like this.

```
top - 13:59:29 up 48 min,  4 users,  load average: 1.06, 0.25, 0.14
Tasks: 139 total,   3 running, 136 sleeping,   0 stopped,   0 zombie
Cpu(s):  0.3%us, 99.7%sy,  0.0%ni,  0.0%id,  0.0%wa,  0.0%hi,  0.0%si,  0.0%st
Mem:    509352k total,  460040k used,    49312k free,   66752k buffers
Swap:   746980k total,    0k used,   746980k free,   247324k cached
```

Notice the **cpu idle time (0.0%id)** is zero. This is because our **cat** processes are consuming the whole **cpu**. Results can vary on systems with four or more **cpu cores**.

37.1.5. top -p

The **top -p 1670,1671,1673,1674** screenshot below shows four processes, all of then using approximately 25 percent of the **cpu**.

```
paul@ubuntu910:~$ top -p 1670,1671,1673,1674
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1674	paul	20	0	2972	616	524	S	26.6	0.1	0:11.92	proj42
1670	paul	20	0	2972	616	524	R	25.0	0.1	0:23.16	proj33
1671	paul	20	0	2972	616	524	S	24.6	0.1	0:23.07	proj33
1673	paul	20	0	2972	620	524	R	23.0	0.1	0:11.48	proj42

All four processes have an equal **priority (PR)**, and are battling for **cpu time**. On some systems the **Linux kernel** might attribute slightly varying **priority values**, but the result will still be four processes fighting for **cpu time**.

37.1.6. renice

Since the processes are already running, we need to use the **renice** command to change their **nice value (NI)**.

The screenshot shows how to use **renice** on both the **proj33** processes.

```
paul@ubuntu910:~$ renice +8 1670
1670: old priority 0, new priority 8
paul@ubuntu910:~$ renice +8 1671
1671: old priority 0, new priority 8
```

Normal users can attribute a **nice value** from zero to 20 to processes they own. Only the **root** user can use negative nice values. Be very careful with negative nice values, since they can make it impossible to use the keyboard or ssh to a system.

37.1.7. impact of nice values

The impact of a nice value on running processes can vary. The screenshot below shows the result of our **renice +8** command. Look at the **%CPU** values.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1674	paul	20	0	2972	616	524	S	46.6	0.1	0:22.37	proj42
1673	paul	20	0	2972	620	524	R	42.6	0.1	0:21.65	proj42
1671	paul	28	8	2972	616	524	S	5.7	0.1	0:29.65	proj33
1670	paul	28	8	2972	616	524	R	4.7	0.1	0:29.82	proj33

Important to remember is to always make less important processes nice to more important processes. Using **negative nice values** can have a severe impact on a system's usability.

37.1.8. nice

The **nice** works identical to the **renice** but it is used when starting a command.

The screenshot shows how to start a script with a **nice** value of five.

```
paul@ubuntu910:~$ nice -5 ./backup.sh
```


37.2. practice : process priorities

1. Create a new directory and create six **pipes** in that directory.
2. Bounce a character between two **pipes**.
3. Use **top** and **ps** to display information (pid, ppid, priority, nice value, ...) about these two cat processes.
4. Bounce another character between two other pipes, but this time start the commands **nice**. Verify that all **cat** processes are battling for the cpu. (Feel free to fire up two more cats with the remaining pipes).
5. Use **ps** to verify that the two new **cat** processes have a **nice** value. Use the -o and -C options of **ps** for this.
6. Use **renice** to increase the nice value from 10 to 15. Notice the difference with the usual commands.

37.3. solution : process priorities

1. Create a new directory and create six **pipes** in that directory.

```
[paul@rhel53 ~]$ mkdir pipes ; cd pipes
[paul@rhel53 pipes]$ mkfifo p1 p2 p3 p4 p5 p6
[paul@rhel53 pipes]$ ls -l
total 0
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p1
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p2
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p3
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p4
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p5
prw-rw-r-- 1 paul paul 0 Apr 12 22:15 p6
```

2. Bounce a character between two **pipes**.

```
[paul@rhel53 pipes]$ echo -n x | cat - p1 > p2 &
[1] 4013
[paul@rhel53 pipes]$ cat <p2 >p1 &
[2] 4016
```

3. Use **top** and **ps** to display information (pid, ppid, priority, nice value, ...) about these two cat processes.

```
top (probably the top two lines)

[paul@rhel53 pipes]$ ps -C cat
  PID TTY          TIME CMD
 4013 pts/0        00:03:38 cat
 4016 pts/0        00:01:07 cat

[paul@rhel53 pipes]$ ps fax | grep cat
 4013 pts/0      R      4:00 |          \_ cat - p1
 4016 pts/0      S      1:13 |          \_ cat
 4044 pts/0      S+     0:00 |          \_ grep cat
```

4. Bounce another character between two other pipes, but this time start the commands **nice**. Verify that all **cat** processes are battling for the cpu. (Feel free to fire up two more cats with the remaining pipes).

```
echo -n y | nice cat - p3 > p4 &
nice cat <p4 >p3 &
```

5. Use **ps** to verify that the two new **cat** processes have a **nice** value. Use the **-o** and **-C** options of **ps** for this.

```
[paul@rhel53 pipes]$ ps -C cat -o pid,ppid,pri,ni,comm
  PID PPID PRI  NI COMMAND
 4013 3947 14   0 cat
 4016 3947 21   0 cat
 4025 3947 13  10 cat
 4026 3947 13  10 cat
```

6. Use **renice** to increase the nice value from 10 to 15. Notice the difference with the usual commands.

```
[paul@rhel53 pipes]$ renice +15 4025
4025: old priority 10, new priority 15
[paul@rhel53 pipes]$ renice +15 4026
```

4026: old priority 10, new priority 15

```
[paul@rhel53 pipes]$ ps -C cat -o pid,ppid,pri,ni,comm
  PID  PPID  PRI   NI COMMAND
  4013   3947   14    0  cat
  4016   3947   21    0  cat
  4025   3947    9   15  cat
  4026   3947    8   15  cat
```

Chapter 38. background jobs

38.1. background processes

38.1.1. jobs

Stuff that runs in background of your current shell can be displayed with the **jobs** command. By default you will not have any **jobs** running in background.

```
root@rhel53 ~# jobs
root@rhel53 ~#
```

This **jobs** command will be used several times in this section.

38.1.2. control-Z

Some processes can be **suspended** with the **Ctrl-Z** key combination. This sends a **SIGSTOP** signal to the **Linux kernel**, effectively freezing the operation of the process.

When doing this in **vi(m)**, then **vi(m)** goes to the background. The background **vi(m)** can be seen with the **jobs** command.

```
[paul@RHEL4a ~]$ vi procdemo.txt
[5]+  Stopped                  vim procdemo.txt
[paul@RHEL4a ~]$ jobs
[5]+  Stopped                  vim procdemo.txt
```

38.1.3. & ampersand

Processes that are started in background using the **&** character at the end of the command line are also visible with the **jobs** command.

```
[paul@RHEL4a ~]$ find / > allfiles.txt 2> /dev/null &
[6] 5230
[paul@RHEL4a ~]$ jobs
[5]+  Stopped                  vim procdemo.txt
[6]-  Running                  find / >allfiles.txt 2>/dev/null &
[paul@RHEL4a ~]$
```

38.1.4. jobs -p

An interesting option is **jobs -p** to see the **process id** of background processes.

```
[paul@RHEL4b ~]$ sleep 500 &
[1] 4902
[paul@RHEL4b ~]$ sleep 400 &
[2] 4903
[paul@RHEL4b ~]$ jobs -p
4902
4903
[paul@RHEL4b ~]$ ps `jobs -p`
```

```
PID TTY      STAT   TIME COMMAND
4902 pts/0    S       0:00 sleep 500
4903 pts/0    S       0:00 sleep 400
[paul@RHEL4b ~]$
```

38.1.5. fg

Running the **fg** command will bring a background job to the foreground. The number of the background job to bring forward is the parameter of **fg**.

```
[paul@RHEL5 ~]$ jobs
[1]  Running                  sleep 1000 &
[2]-  Running                  sleep 1000 &
[3]+  Running                  sleep 2000 &
[paul@RHEL5 ~]$ fg 3
sleep 2000
```

38.1.6. bg

Jobs that are **suspended** in background can be started in background with **bg**. The **bg** will send a **SIGCONT** signal.

Below an example of the sleep command (suspended with **Ctrl-Z**) being reactivated in background with **bg**.

```
[paul@RHEL5 ~]$ jobs
[paul@RHEL5 ~]$ sleep 5000 &
[1] 6702
[paul@RHEL5 ~]$ sleep 3000

[2]+  Stopped                  sleep 3000
[paul@RHEL5 ~]$ jobs
[1]-  Running                  sleep 5000 &
[2]+  Stopped                  sleep 3000
[paul@RHEL5 ~]$ bg 2
[2]+ sleep 3000 &
[paul@RHEL5 ~]$ jobs
[1]-  Running                  sleep 5000 &
[2]+  Running                  sleep 3000 &
[paul@RHEL5 ~]$
```

38.2. practice : background processes

1. Use the **jobs** command to verify whether you have any processes running in background.
2. Use **vi** to create a little text file. Suspend **vi** in background.
3. Verify with **jobs** that **vi** is suspended in background.
4. Start **find / > allfiles.txt 2>/dev/null** in foreground. Suspend it in background before it finishes.
5. Start two long **sleep** processes in background.
6. Display all **jobs** in background.
7. Use the **kill** command to suspend the last **sleep** process.
8. Continue the **find** process in background (make sure it runs again).
9. Put one of the **sleep** commands back in foreground.
10. (if time permits, a general review question...) Explain in detail where the numbers come from in the next screenshot. When are the variables replaced by their value ? By which shell ?

```
[paul@RHEL4b ~]$ echo $$ $PPID
4224 4223
[paul@RHEL4b ~]$ bash -c "echo $$ $PPID"
4224 4223
[paul@RHEL4b ~]$ bash -c 'echo $$ $PPID'
5059 4224
[paul@RHEL4b ~]$ bash -c `echo $$ $PPID`
4223: 4224: command not found
```

38.3. solution : background processes

1. Use the **jobs** command to verify whether you have any processes running in background.

```
jobs (maybe the catfun is still running?)
```

2. Use **vi** to create a little text file. Suspend **vi** in background.

```
vi text.txt
(inside vi press ctrl-z)
```

3. Verify with **jobs** that **vi** is suspended in background.

```
[paul@rhel53 ~]$ jobs
[1]+  Stopped                  vim text.txt
```

4. Start **find / > allfiles.txt 2>/dev/null** in foreground. Suspend it in background before it finishes.

```
[paul@rhel53 ~]$ find / > allfiles.txt 2>/dev/null
    (press ctrl-z)
[2]+  Stopped                  find / > allfiles.txt 2> /dev/null
```

5. Start two long **sleep** processes in background.

```
sleep 4000 & ; sleep 5000 &
```

6. Display all **jobs** in background.

```
[paul@rhel53 ~]$ jobs
[1]-  Stopped                  vim text.txt
[2]+  Stopped                  find / > allfiles.txt 2> /dev/null
[3]   Running                  sleep 4000 &
[4]   Running                  sleep 5000 &
```

7. Use the **kill** command to suspend the last **sleep** process.

```
[paul@rhel53 ~]$ kill -SIGSTOP 4519
[paul@rhel53 ~]$ jobs
[1]   Stopped                  vim text.txt
[2]-  Stopped                  find / > allfiles.txt 2> /dev/null
[3]   Running                  sleep 4000 &
[4]+  Stopped                  sleep 5000
```

8. Continue the **find** process in background (make sure it runs again).

```
bg 2 (verify the job-id in your jobs list)
```

9. Put one of the **sleep** commands back in foreground.

```
fg 3 (again verify your job-id)
```

10. (if time permits, a general review question...) Explain in detail where the numbers come from in the next screenshot. When are the variables replaced by their value ? By which shell ?

```
[paul@RHEL4b ~]$ echo $$ $PPID
4224 4223
[paul@RHEL4b ~]$ bash -c "echo $$ $PPID"
```



```
4224 4223
[paul@RHEL4b ~]$ bash -c 'echo $$ $PPID'
5059 4224
[paul@RHEL4b ~]$ bash -c `echo $$ $PPID`
4223: 4224: command not found
```

The current bash shell will replace the \$\$ and \$PPID while scanning the line, and before executing the echo command.

```
[paul@RHEL4b ~]$ echo $$ $PPID
4224 4223
```

The variables are now double quoted, but the current bash shell will replace \$\$ and \$PPID while scanning the line, and before executing the bash -c command.

```
[paul@RHEL4b ~]$ bash -c "echo $$ $PPID"
4224 4223
```

The variables are now single quoted. The current bash shell will **not** replace the \$\$ and the \$PPID. The bash -c command will be executed before the variables replaced with their value. This latter bash is the one replacing the \$\$ and \$PPID with their value.

```
[paul@RHEL4b ~]$ bash -c 'echo $$ $PPID'
5059 4224
```

With backticks the shell will still replace both variable before the embedded echo is executed. The result of this echo is the two process id's. These are given as commands to bash -c. But two numbers are not commands!

```
[paul@RHEL4b ~]$ bash -c `echo $$ $PPID`
4223: 4224: command not found
```

Part XI. disk management

Table of Contents

39. disk devices	365
39.1. terminology	366
39.2. device naming	368
39.3. discovering disk devices	369
39.4. erasing a hard disk	374
39.5. advanced hard disk settings	375
39.6. practice: hard disk devices	376
39.7. solution: hard disk devices	377
40. disk partitions	379
40.1. about partitions	380
40.2. discovering partitions	381
40.3. partitioning new disks	383
40.4. about the partition table	385
40.5. GUID partition table	386
40.6. labeling with parted	386
40.7. practice: partitions	388
40.8. solution: partitions	389
41. file systems	390
41.1. about file systems	391
41.2. common file systems	392
41.3. putting a file system on a partition	395
41.4. tuning a file system	396
41.5. checking a file system	397
41.6. practice: file systems	398
41.7. solution: file systems	399
42. mounting	400
42.1. mounting local file systems	401
42.2. displaying mounted file systems	402
42.3. from start to finish	404
42.4. permanent mounts	405
42.5. securing mounts	406
42.6. mounting remote file systems	407
42.7. practice: mounting file systems	408
42.8. solution: mounting file systems	409
43. troubleshooting tools	411
43.1. lsof	412
43.2. fuser	413
43.3. chroot	414
43.4. iostat	415
43.5. iotop	416
43.6. vmstat	417
43.7. practice: troubleshooting tools	418
43.8. solution: troubleshooting tools	419
44. introduction to uuid's	420
44.1. about unique objects	421
44.2. tune2fs	421
44.3. uuid	421
44.4. uuid in /etc/fstab	422
44.5. uuid as a boot device	423
44.6. practice: uuid and filesystems	424
44.7. solution: uuid and filesystems	425
45. introduction to raid	426
45.1. hardware or software	426
45.2. raid levels	427
45.3. building a software raid5 array	429

45.4. practice: raid	432
45.5. solution: raid	433
46. logical volume management	434
46.1. introduction to lvm	435
46.2. lvm terminology	436
46.3. example: using lvm	437
46.4. example: extend a logical volume	439
46.5. example: resize a physical Volume	441
46.6. example: mirror a logical volume	443
46.7. example: snapshot a logical volume	444
46.8. verifying existing physical volumes	445
46.9. verifying existing volume groups	447
46.10. verifying existing logical volumes	448
46.11. manage physical volumes	449
46.12. manage volume groups	451
46.13. manage logical volumes	453
46.14. practice : lvm	455
46.15. solution : lvm	456
47. iSCSI devices	460
47.1. iSCSI terminology	461
47.2. iSCSI Target in RHEL/CentOS	461
47.3. iSCSI Initiator in RHEL/CentOS	463
47.4. iSCSI target on Debian	465
47.5. iSCSI target setup with dd files	466
47.6. iSCSI initiator on ubuntu	468
47.7. using iSCSI devices	470
47.8. practice: iSCSI devices	471
47.9. solution: iSCSI devices	472
48. introduction to multipathing	473
48.1. install multipath	474
48.2. configure multipath	474
48.3. network	475
48.4. start multipathd and iscsi	475
48.5. multipath list	477
48.6. using the device	478
48.7. practice: multipathing	479
48.8. solution: multipathing	480

Chapter 39. disk devices

This chapter teaches you how to locate and recognise **hard disk devices**. This prepares you for the next chapter, where we put **partitions** on these devices.

39.1. terminology

39.1.1. platter, head, track, cylinder, sector

Data is commonly stored on magnetic or optical **disk platters**. The platters are rotated (at high speeds). Data is read by **heads**, which are very close to the surface of the platter, without touching it! The heads are mounted on an arm (sometimes called a comb or a fork).

Data is written in concentric circles called **tracks**. Track zero is (usually) on the outside. The time it takes to position the head over a certain track is called the **seek time**. Often the platters are stacked on top of each other, hence the set of tracks accessible at a certain position of the comb forms a **cylinder**. Tracks are divided into 512 byte **sectors**, with more unused space (**gap**) between the sectors on the outside of the platter.

When you break down the advertised **access time** of a hard drive, you will notice that most of that time is taken by movement of the heads (about 65%) and **rotational latency** (about 30%).

39.1.2. ide or scsi

Actually, the title should be **ata** or **scsi**, since ide is an ata compatible device. Most desktops use **ata devices**, most servers use **scsi**.

39.1.3. ata

An **ata controller** allows two devices per bus, one **master** and one **slave**. Unless your controller and devices support **cable select**, you have to set this manually with jumpers.

With the introduction of **sata** (serial ata), the original ata was renamed to **parallel ata**. Optical drives often use **atapi**, which is an ATA interface using the SCSI communication protocol.

39.1.4. scsi

A **scsi controller** allows more than two devices. When using **SCSI (small computer system interface)**, each device gets a unique **scsi id**. The **scsi controller** also needs a **scsi id**, do not use this id for a scsi-attached device.

Older 8-bit SCSI is now called **narrow**, whereas 16-bit is **wide**. When the bus speeds was doubled to 10Mhz, this was known as **fast SCSI**. Doubling to 20Mhz made it **ultra SCSI**. Take a look at <http://en.wikipedia.org/wiki/SCSI> for more SCSI standards.

39.1.5. block device

Random access hard disk devices have an abstraction layer called **block device** to enable formatting in fixed-size (usually 512 bytes) blocks. Blocks can be accessed independent of access to other blocks.

```
[root@centos65 ~]# lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda                                 8:0      0   40G  0 disk
--sda1                             8:1      0   500M  0 part /boot
--sda2                             8:2      0  39.5G  0 part
  --VolGroup-lv_root (dm-0) 253:0      0  38.6G  0 lvm /
  --VolGroup-lv_swap (dm-1) 253:1      0   928M  0 lvm [SWAP]
sdb                                8:16     0    72G  0 disk
sdc                                8:32     0   144G  0 disk
```

A block device has the letter b to denote the file type in the output of **ls -l**.

```
[root@centos65 ~]# ls -l /dev/sd*
brw-rw----. 1 root disk 8,  0 Apr 19 10:12 /dev/sda
brw-rw----. 1 root disk 8,  1 Apr 19 10:12 /dev/sda1
brw-rw----. 1 root disk 8,  2 Apr 19 10:12 /dev/sda2
brw-rw----. 1 root disk 8, 16 Apr 19 10:12 /dev/sdb
brw-rw----. 1 root disk 8, 32 Apr 19 10:12 /dev/sdc
```

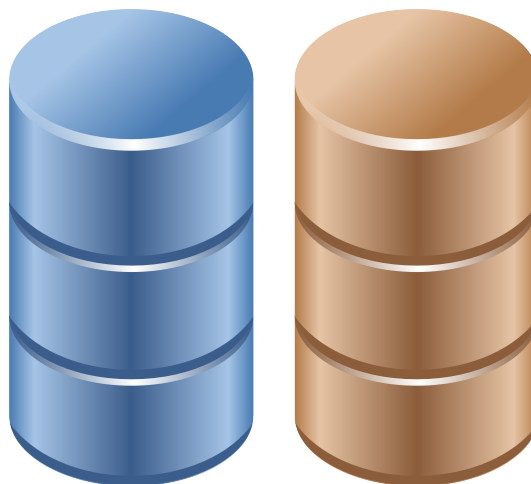
Note that a **character device** is a constant stream of characters, being denoted by a c in **ls -l**. Note also that the **ISO 9660** standard for cdrom uses a **2048 byte** block size.

Old hard disks (and floppy disks) use **cylinder-head-sector** addressing to access a sector on the disk. Most current disks use **LBA (Logical Block Addressing)**.

39.1.6. solid state drive

A **solid state drive** or **ssd** is a block device without moving parts. It is comparable to **flash memory**. An **ssd** is more expensive than a hard disk, but it typically has a much faster access time.

In this book we will use the following pictograms for **spindle disks** (in brown) and **solid state disks** (in blue).



39.2. device naming

39.2.1. ata (ide) device naming

All **ata** drives on your system will start with **/dev/hd** followed by a unit letter. The master hdd on the first **ata controller** is **/dev/hda**, the slave is **/dev/hdb**. For the second controller, the names of the devices are **/dev/hdc** and **/dev/hdd**.

Table 39.1. ide device naming

controller	connection	device name
ide0	master	/dev/hda
	slave	/dev/hdb
ide1	master	/dev/hdc
	slave	/dev/hdd

It is possible to have only **/dev/hda** and **/dev/hdd**. The first one is a single ata hard disk, the second one is the cdrom (by default configured as slave).

39.2.2. scsi device naming

scsi drives follow a similar scheme, but all start with **/dev/sd**. When you run out of letters (after **/dev/sdz**), you can continue with **/dev/sdaa** and **/dev/sdab** and so on. (We will see later on that **lvm** volumes are commonly seen as **/dev/md0**, **/dev/md1** etc.)

Below a **sample** of how scsi devices on a Linux can be named. Adding a scsi disk or raid controller with a lower scsi address will change the naming scheme (shifting the higher scsi addresses one letter further in the alphabet).

Table 39.2. scsi device naming

device	scsi id	device name
disk 0	0	/dev/sda
disk 1	1	/dev/sdb
raid controller 0	5	/dev/sdc
raid controller 1	6	/dev/sdd

A modern Linux system will use **/dev/sd*** for scsi and sata devices, and also for sd-cards, usb-sticks, (legacy) ATA/IDE devices and solid state drives.

39.3. discovering disk devices

39.3.1. fdisk

You can start by using **/sbin/fdisk** to find out what kind of disks are seen by the kernel. Below the result on old Debian desktop, with two **ata-ide disks** present.

```
root@barry:~# fdisk -l | grep Disk
Disk /dev/hda: 60.0 GB, 60022480896 bytes
Disk /dev/hdb: 81.9 GB, 81964302336 bytes
```

And here an example of **sata and scsi disks** on a server with CentOS. Remember that **sata** disks are also presented to you with the **scsi** `/dev/sd*` notation.

```
[root@centos65 ~]# fdisk -l | grep 'Disk /dev/sd'
Disk /dev/sda: 42.9 GB, 42949672960 bytes
Disk /dev/sdb: 77.3 GB, 77309411328 bytes
Disk /dev/sdc: 154.6 GB, 154618822656 bytes
Disk /dev/sdd: 154.6 GB, 154618822656 bytes
```

Here is an overview of disks on a RHEL4u3 server with two real 72GB **scsi disks**. This server is attached to a **NAS** with four **NAS disks** of half a terabyte. On the NAS disks, four LVM (`/dev/mdx`) software RAID devices are configured.

```
[root@tsvtl1 ~]# fdisk -l | grep Disk
Disk /dev/sda: 73.4 GB, 73407488000 bytes
Disk /dev/sdb: 73.4 GB, 73407488000 bytes
Disk /dev/sdc: 499.0 GB, 499036192768 bytes
Disk /dev/sdd: 499.0 GB, 499036192768 bytes
Disk /dev/sde: 499.0 GB, 499036192768 bytes
Disk /dev/sdf: 499.0 GB, 499036192768 bytes
Disk /dev/md0: 271 MB, 271319040 bytes
Disk /dev/md2: 21.4 GB, 21476081664 bytes
Disk /dev/md3: 21.4 GB, 21467889664 bytes
Disk /dev/md1: 21.4 GB, 21476081664 bytes
```

You can also use **fdisk** to obtain information about one specific hard disk device.

```
[root@centos65 ~]# fdisk -l /dev/sdc
Disk /dev/sdc: 154.6 GB, 154618822656 bytes
255 heads, 63 sectors/track, 18798 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x00000000
```

Later we will use **fdisk** to do dangerous stuff like creating and deleting partitions.

39.3.2. dmesg

Kernel boot messages can be seen after boot with **dmesg**. Since hard disk devices are detected by the kernel during boot, you can also use dmesg to find information about disk devices.

```
[root@centos65 ~]# dmesg | grep 'sd[a-z]' | head
sd 0:0:0:0: [sda] 83886080 512-byte logical blocks: (42.9 GB/40.0 GiB)
sd 0:0:0:0: [sda] Write Protect is off
sd 0:0:0:0: [sda] Mode Sense: 00 3a 00 00
sd 0:0:0:0: [sda] Write cache: enabled, read cache: enabled, doesn't support \
DPO or FUA
sda: sda1 sda2
sd 0:0:0:0: [sda] Attached SCSI disk
sd 3:0:0:0: [sdb] 150994944 512-byte logical blocks: (77.3 GB/72.0 GiB)
sd 3:0:0:0: [sdb] Write Protect is off
sd 3:0:0:0: [sdb] Mode Sense: 00 3a 00 00
sd 3:0:0:0: [sdb] Write cache: enabled, read cache: enabled, doesn't support \
DPO or FUA
```

Here is another example of **dmesg** on a computer with a 200GB ata disk.

```
paul@barry:~$ dmesg | grep -i "ata disk"
[ 2.624149] hda: ST360021A, ATA DISK drive
[ 2.904150] hdb: Maxtor 6Y080L0, ATA DISK drive
[ 3.472148] hdd: WDC WD2000BB-98DWA0, ATA DISK drive
```

Third and last example of **dmesg** running on RHEL5.3.

```
root@rhel53 ~# dmesg | grep -i "scsi disk"
sd 0:0:2:0: Attached scsi disk sda
sd 0:0:3:0: Attached scsi disk sdb
sd 0:0:6:0: Attached scsi disk sdc
```

39.3.3. /sbin/lshw

The **lshw** tool will **list hardware**. With the right options **lshw** can show a lot of information about disks (and partitions).

Below a truncated screenshot on Debian 6:

```
root@debian6~# lshw -class volume | grep -A1 -B2 scsi
    description: Linux raid autodetect partition
    physical id: 1
    bus info: scsi@1:0.0.0,1
    logical name: /dev/sdb1
--
    description: Linux raid autodetect partition
    physical id: 1
    bus info: scsi@2:0.0.0,1
    logical name: /dev/sdc1
--
    description: Linux raid autodetect partition
    physical id: 1
    bus info: scsi@3:0.0.0,1
    logical name: /dev/sdd1
--
    description: Linux raid autodetect partition
    physical id: 1
    bus info: scsi@4:0.0.0,1
    logical name: /dev/sde1
--
    vendor: Linux
    physical id: 1
    bus info: scsi@0:0.0.0,1
    logical name: /dev/sda1
--
    vendor: Linux
    physical id: 2
    bus info: scsi@0:0.0.0,2
    logical name: /dev/sda2
--
    description: Extended partition
    physical id: 3
    bus info: scsi@0:0.0.0,3
    logical name: /dev/sda3
```

Redhat and CentOS do not have this tool (unless you add a repository).

39.3.4. /sbin/lsscsi

The **lsscsi** command provides a nice readable output of all scsi (and scsi emulated devices). This first screenshot shows **lsscsi** on a SPARC system.

```
root@shaka:~# lsscsi
[0:0:0:0]    disk    Adaptec  RAID5          V1.0  /dev/sda
[1:0:0:0]    disk    SEAGATE  ST336605FSUN36G  0438  /dev/sdb
root@shaka:~#
```

Below a screenshot of **lsscsi** on a QNAP NAS (which has four 750GB disks and boots from a usb stick).

```
lroot@debian6~# lsscsi
[0:0:0:0]    disk    SanDisk  Cruzer Edge      1.19  /dev/sda
[1:0:0:0]    disk    ATA      ST3750330AS      SD04  /dev/sdb
[2:0:0:0]    disk    ATA      ST3750330AS      SD04  /dev/sdc
[3:0:0:0]    disk    ATA      ST3750330AS      SD04  /dev/sdd
[4:0:0:0]    disk    ATA      ST3750330AS      SD04  /dev/sde
```

This screenshot shows the classic output of **lsscsi**.

```
root@debian6~# lsscsi -c
Attached devices:
Host: scsi0 Channel: 00 Target: 00 Lun: 00
  Vendor: SanDisk  Model: Cruzer Edge      Rev: 1.19
  Type:   Direct-Access              ANSI SCSI revision: 02
Host: scsi1 Channel: 00 Target: 00 Lun: 00
  Vendor: ATA      Model: ST3750330AS    Rev: SD04
  Type:   Direct-Access              ANSI SCSI revision: 05
Host: scsi2 Channel: 00 Target: 00 Lun: 00
  Vendor: ATA      Model: ST3750330AS    Rev: SD04
  Type:   Direct-Access              ANSI SCSI revision: 05
Host: scsi3 Channel: 00 Target: 00 Lun: 00
  Vendor: ATA      Model: ST3750330AS    Rev: SD04
  Type:   Direct-Access              ANSI SCSI revision: 05
Host: scsi4 Channel: 00 Target: 00 Lun: 00
  Vendor: ATA      Model: ST3750330AS    Rev: SD04
  Type:   Direct-Access              ANSI SCSI revision: 05
```

39.3.5. /proc/scsi/scsi

Another way to locate **scsi** (or **sd**) devices is via **/proc/scsi/scsi**.

This screenshot is from a **sparc** computer with adaptec RAID5.

```
root@shaka:~# cat /proc/scsi/scsi
Attached devices:
Host: scsi0 Channel: 00 Id: 00 Lun: 00
  Vendor: Adaptec Model: RAID5 Rev: V1.0
  Type: Direct-Access ANSI SCSI revision: 02
Host: scsi1 Channel: 00 Id: 00 Lun: 00
  Vendor: SEAGATE Model: ST336605FSUN36G Rev: 0438
  Type: Direct-Access ANSI SCSI revision: 03
root@shaka:~#
```

Here we run **cat /proc/scsi/scsi** on the QNAP from above (with Debian Linux).

```
root@debian6~# cat /proc/scsi/scsi
Attached devices:
Host: scsi0 Channel: 00 Id: 00 Lun: 00
  Vendor: SanDisk Model: Cruzer Edge Rev: 1.19
  Type: Direct-Access ANSI SCSI revision: 02
Host: scsi1 Channel: 00 Id: 00 Lun: 00
  Vendor: ATA Model: ST3750330AS Rev: SD04
  Type: Direct-Access ANSI SCSI revision: 05
Host: scsi2 Channel: 00 Id: 00 Lun: 00
  Vendor: ATA Model: ST3750330AS Rev: SD04
  Type: Direct-Access ANSI SCSI revision: 05
Host: scsi3 Channel: 00 Id: 00 Lun: 00
  Vendor: ATA Model: ST3750330AS Rev: SD04
  Type: Direct-Access ANSI SCSI revision: 05
Host: scsi4 Channel: 00 Id: 00 Lun: 00
  Vendor: ATA Model: ST3750330AS Rev: SD04
  Type: Direct-Access ANSI SCSI revision: 05
```

Note that some recent versions of Debian have this disabled in the kernel. You can enable it (after a kernel compile) using this entry:

```
# CONFIG_SCSI_PROC_FS is not set
```

Redhat and CentOS have this by default (if there are scsi devices present).

```
[root@centos65 ~]# cat /proc/scsi/scsi
Attached devices:
Host: scsi0 Channel: 00 Id: 00 Lun: 00
  Vendor: ATA Model: VBOX HARDDISK Rev: 1.0
  Type: Direct-Access ANSI SCSI revision: 05
Host: scsi3 Channel: 00 Id: 00 Lun: 00
  Vendor: ATA Model: VBOX HARDDISK Rev: 1.0
  Type: Direct-Access ANSI SCSI revision: 05
Host: scsi4 Channel: 00 Id: 00 Lun: 00
  Vendor: ATA Model: VBOX HARDDISK Rev: 1.0
  Type: Direct-Access ANSI SCSI revision: 05
```

39.4. erasing a hard disk

Before selling your old hard disk on the internet, it may be a good idea to erase it. By simply repartitioning, or by using the Microsoft Windows format utility, or even after an **mkfs** command, some people will still be able to read most of the data on the disk.

```
root@debian6~# aptitude search foremost autopsy sleuthkit | tr -s ' '
p autopsy - graphical interface to SleuthKit
p foremost - Forensics application to recover data
p sleuthkit - collection of tools for forensics analysis
```

Although technically the **/sbin/badblocks** tool is meant to look for bad blocks, you can use it to completely erase all data from a disk. Since this is really writing to every sector of the disk, it can take a long time!

```
root@RHELv4u2:~# badblocks -ws /dev/sdb
Testing with pattern 0xaa: done
Reading and comparing: done
Testing with pattern 0x55: done
Reading and comparing: done
Testing with pattern 0xff: done
Reading and comparing: done
Testing with pattern 0x00: done
Reading and comparing: done
```

The previous screenshot overwrites every sector of the disk **four times**. Erasing **once** with a tool like **dd** is enough to destroy all data.

Warning, this screenshot shows how to permanently destroy all data on a block device.

```
[root@rhel65 ~]# dd if=/dev/zero of=/dev/sdb
```

39.5. advanced hard disk settings

Tweaking of hard disk settings (dma, gap, ...) are not covered in this course. Several tools exists, **hdparm** and **sdparm** are two of them.

hdparm can be used to display or set information and parameters about an ATA (or SATA) hard disk device. The **-i** and **-I** options will give you even more information about the physical properties of the device.

```
root@laika:~# hdparm /dev/sdb

/dev/sdb:
IO_support    =  0 (default 16-bit)
readonly      =  0 (off)
readahead     = 256 (on)
geometry      = 12161/255/63, sectors = 195371568, start = 0
```

Below **hdparm** info about a 200GB IDE disk.

```
root@barry:~# hdparm /dev/hdd

/dev/hdd:
multcount     =  0 (off)
IO_support    =  0 (default)
unmaskirq     =  0 (off)
using_dma     =  1 (on)
keepsettings  =  0 (off)
readonly      =  0 (off)
readahead     = 256 (on)
geometry      = 24321/255/63, sectors = 390721968, start = 0
```

Here a screenshot of **sdparm** on Ubuntu 10.10.

```
root@ubu1010:~# aptitude install sdparm
...
root@ubu1010:~# sdparm /dev/sda | head -1
/dev/sda: ATA      FUJITSU MJA2160B  0081
root@ubu1010:~# man sdparm
```

Use **hdparm** and **sdparm** with care.

39.6. practice: hard disk devices

About this lab: To practice working with hard disks, you will need some hard disks. When there are no physical hard disk available, you can use virtual disks in **vmware** or **VirtualBox**. The teacher will help you in attaching a couple of ATA and/or SCSI disks to a virtual machine. The results of this lab can be used in the next three labs (partitions, file systems, mounting).

It is advised to attach three 1GB disks and three 2GB disks to the virtual machine. This will allow for some freedom in the practices of this chapter as well as the next chapters (raid, lvm, iSCSI).

1. Use **dmesg** to make a list of hard disk devices detected at boot-up.
2. Use **fdisk** to find the total size of all hard disk devices on your system.
3. Stop a virtual machine, add three virtual 1 gigabyte **scsi** hard disk devices and one virtual 400 megabyte **ide** hard disk device. If possible, also add another virtual 400 megabyte **ide** disk.
4. Use **dmesg** to verify that all the new disks are properly detected at boot-up.
5. Verify that you can see the disk devices in **/dev**.
6. Use **fdisk** (with **grep** and **/dev/null**) to display the total size of the new disks.
7. Use **badblocks** to completely erase one of the smaller hard disks.
8. Look at **/proc/scsi/scsi**.
9. If possible, install **lsscsi**, **lshw** and use them to list the disks.

39.7. solution: hard disk devices

1. Use **dmesg** to make a list of hard disk devices detected at boot-up.

Some possible answers...

```
dmesg | grep -i disk
```

```
Looking for ATA disks: dmesg | grep hd[abcd]
```

```
Looking for ATA disks: dmesg | grep -i "ata disk"
```

```
Looking for SCSI disks: dmesg | grep sd[a-f]
```

```
Looking for SCSI disks: dmesg | grep -i "scsi disk"
```

2. Use **fdisk** to find the total size of all hard disk devices on your system.

```
fdisk -l
```

3. Stop a virtual machine, add three virtual 1 gigabyte **scsi** hard disk devices and one virtual 400 megabyte **ide** hard disk device. If possible, also add another virtual 400 megabyte **ide** disk.

This exercise happens in the settings of vmware or VirtualBox.

4. Use **dmesg** to verify that all the new disks are properly detected at boot-up.

See 1.

5. Verify that you can see the disk devices in **/dev**.

```
SCSI+SATA: ls -l /dev/sd*
```

```
ATA: ls -l /dev/hd*
```

6. Use **fdisk** (with **grep** and **/dev/null**) to display the total size of the new disks.

```
root@rhel53 ~# fdisk -l 2>/dev/null | grep [MGT]B
Disk /dev/hda: 21.4 GB, 21474836480 bytes
Disk /dev/hdb: 1073 MB, 1073741824 bytes
Disk /dev/sda: 2147 MB, 2147483648 bytes
Disk /dev/sdb: 2147 MB, 2147483648 bytes
Disk /dev/sdc: 2147 MB, 2147483648 bytes
```

7. Use **badblocks** to completely erase one of the smaller hard disks.

```
#Verify the device (/dev/sdc??) you want to erase before typing this.
#
root@rhel53 ~# badblocks -ws /dev/sdc
Testing with pattern 0xaa: done
Reading and comparing: done
Testing with pattern 0x55: done
Reading and comparing: done
Testing with pattern 0xff: done
Reading and comparing: done
Testing with pattern 0x00: done
Reading and comparing: done
```

8. Look at **/proc/scsi/scsi**.

```
root@rhel53 ~# cat /proc/scsi/scsi
```

Attached devices:

```
Host: scsi0 Channel: 00 Id: 02 Lun: 00
  Vendor: VBOX      Model: HARDDISK      Rev: 1.0
  Type:   Direct-Access                    ANSI SCSI revision: 05
Host: scsi0 Channel: 00 Id: 03 Lun: 00
  Vendor: VBOX      Model: HARDDISK      Rev: 1.0
  Type:   Direct-Access                    ANSI SCSI revision: 05
Host: scsi0 Channel: 00 Id: 06 Lun: 00
  Vendor: VBOX      Model: HARDDISK      Rev: 1.0
  Type:   Direct-Access                    ANSI SCSI revision: 05
```

9. If possible, install **lsscsi**, **lshw** and use them to list the disks.

```
Debian,Ubuntu: aptitude install lsscsi lshw
```

```
Fedora: yum install lsscsi lshw
```

```
root@rhel53 ~# lsscsi
[0:0:2:0]    disk      VBOX      HARDDISK      1.0    /dev/sda
[0:0:3:0]    disk      VBOX      HARDDISK      1.0    /dev/sdb
[0:0:6:0]    disk      VBOX      HARDDISK      1.0    /dev/sdc
```

Chapter 40. disk partitions

This chapter continues on the **hard disk devices** from the previous one. Here we will put **partitions** on those devices.

This chapter prepares you for the next chapter, where we put **file systems** on our partitions.

40.1. about partitions

40.1.1. primary, extended and logical

Linux requires you to create one or more **partitions**. The next paragraphs will explain how to create and use partitions.

A partition's **geometry** and size is usually defined by a starting and ending cylinder (sometimes by sector). Partitions can be of type **primary** (maximum four), **extended** (maximum one) or **logical** (contained within the extended partition). Each partition has a **type field** that contains a code. This determines the computer's operating system or the partitions file system.

Table 40.1. primary, extended and logical partitions

Partition Type	naming
Primary (max 4)	1-4
Extended (max 1)	1-4
Logical	5-

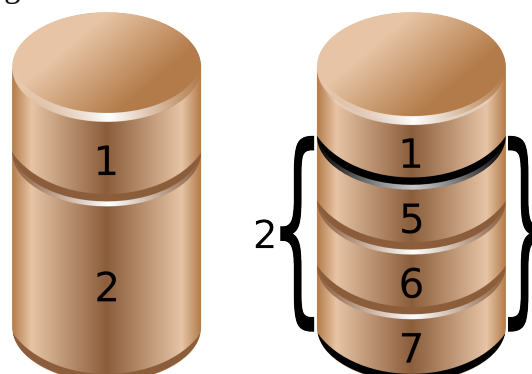
40.1.2. partition naming

We saw before that hard disk devices are named `/dev/hdx` or `/dev/sdx` with `x` depending on the hardware configuration. Next is the partition number, starting the count at 1. Hence the four (possible) primary partitions are numbered 1 to 4. Logical partition counting always starts at 5. Thus `/dev/hda2` is the second partition on the first ATA hard disk device, and `/dev/hdb5` is the first logical partition on the second ATA hard disk device. Same for SCSI, `/dev/sdb3` is the third partition on the second SCSI disk.

Table 40.2. Partition naming

partition	device
<code>/dev/hda1</code>	first primary partition on <code>/dev/hda</code>
<code>/dev/hda2</code>	second primary or extended partition on <code>/dev/hda</code>
<code>/dev/sda5</code>	first logical drive on <code>/dev/sda</code>
<code>/dev/sdb6</code>	second logical on <code>/dev/sdb</code>

The picture below shows two (spindle) disks with partitions. Note that an extended partition is a container holding logical drives.



40.2. discovering partitions

40.2.1. fdisk -l

In the **fdisk -l** example below you can see that two partitions exist on **/dev/sdb**. The first partition spans 31 cylinders and contains a Linux swap partition. The second partition is much bigger.

```
root@laika:~# fdisk -l /dev/sdb

Disk /dev/sdb: 100.0 GB, 100030242816 bytes
255 heads, 63 sectors/track, 12161 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

   Device Boot      Start         End      Blocks    Id  System
/dev/sdb1             1           31       248976    82  Linux swap / Solaris
/dev/sdb2            32        12161      97434225    83  Linux
root@laika:~#
```

40.2.2. /proc/partitions

The **/proc/partitions** file contains a table with major and minor number of partitioned devices, their number of blocks and the device name in **/dev**. Verify with **/proc/devices** to link the major number to the proper device.

```
paul@RHELv4u4:~$ cat /proc/partitions
major minor  #blocks  name

   3     0     524288  hda
   3    64     734003  hdb
   8     0    8388608  sda
   8     1    104391   sda1
   8     2    8281507  sda2
   8    16    1048576  sdb
   8    32    1048576  sdc
   8    48    1048576  sdd
 253     0    7176192  dm-0
 253     1    1048576  dm-1
```

The **major** number corresponds to the device type (or driver) and can be found in **/proc/devices**. In this case 3 corresponds to **ide** and 8 to **sd**. The **major** number determines the **device driver** to be used with this device.

The **minor** number is a unique identification of an instance of this device type. The **devices.txt** file in the kernel tree contains a full list of major and minor numbers.

40.2.3. parted and others

You may be interested in alternatives to **fdisk** like **parted**, **cfdisk**, **sfdisk** and **gparted**. This course mainly uses **fdisk** to partition hard disks.

parted is recommended by some Linux distributions for handling storage with **gpt** instead of **mbr**.

Below a screenshot of **parted** on CentOS.

```
[root@centos65 ~]# rpm -q parted
parted-2.1-21.el6.x86_64
[root@centos65 ~]# parted /dev/sda
GNU Parted 2.1
Using /dev/sda
Welcome to GNU Parted! Type 'help' to view a list of commands.
(parted) print
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sda: 42.9GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
```

Number	Start	End	Size	Type	File system	Flags
1	1049kB	525MB	524MB	primary	ext4	boot
2	525MB	42.9GB	42.4GB	primary		lvm

```
(parted)
```

40.3. partitioning new disks

In the example below, we bought a new disk for our system. After the new hardware is properly attached, you can use **fdisk** and **parted** to create the necessary partition(s). This example uses **fdisk**, but there is nothing wrong with using **parted**.

40.3.1. recognising the disk

First, we check with **fdisk -l** whether Linux can see the new disk. Yes it does, the new disk is seen as **/dev/sdb**, but it does not have any partitions yet.

```
root@RHELV4u2:~# fdisk -l

Disk /dev/sda: 12.8 GB, 12884901888 bytes
255 heads, 63 sectors/track, 1566 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

Device Boot      Start         End      Blocks   Id  System
/dev/sda1  *          1          13       104391   83   Linux
/dev/sda2          14        1566    12474472+  8e   Linux LVM

Disk /dev/sdb: 1073 MB, 1073741824 bytes
255 heads, 63 sectors/track, 130 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

Disk /dev/sdb doesn't contain a valid partition table
```

40.3.2. opening the disk with fdisk

Then we create a partition with **fdisk** on **/dev/sdb**. First we start the **fdisk** tool with **/dev/sdb** as argument. Be very very careful not to partition the wrong disk!!

```
root@RHELV4u2:~# fdisk /dev/sdb
Device contains neither a valid DOS partition table, nor Sun, SGI...
Building a new DOS disklabel. Changes will remain in memory only,
until you decide to write them. After that, of course, the previous
content won't be recoverable.

Warning: invalid flag 0x0000 of partition table 4 will be corrected...
```

40.3.3. empty partition table

Inside the **fdisk** tool, we can issue the **p** command to see the current disks partition table.

```
Command (m for help): p

Disk /dev/sdb: 1073 MB, 1073741824 bytes
255 heads, 63 sectors/track, 130 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

Device Boot      Start         End      Blocks   Id  System
```

40.3.4. create a new partition

No partitions exist yet, so we issue **n** to create a new partition. We choose **p** for primary, **1** for the partition number, **1** for the start cylinder and **14** for the end cylinder.

```
Command (m for help): n
Command action
e   extended
p   primary partition (1-4)
p
Partition number (1-4): 1
First cylinder (1-130, default 1):
Using default value 1
Last cylinder or +size or +sizeM or +sizeK (1-130, default 130): 14
```

We can now issue **p** again to verify our changes, but they are not yet written to disk. This means we can still cancel this operation! But it looks good, so we use **w** to write the changes to disk, and then quit the fdisk tool.

```
Command (m for help): p

Disk /dev/sdb: 1073 MB, 1073741824 bytes
255 heads, 63 sectors/track, 130 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

Device Boot      Start         End      Blocks   Id  System
/dev/sdb1          1          14        112423+   83  Linux

Command (m for help): w
The partition table has been altered!

Calling ioctl() to re-read partition table.
Syncing disks.
root@RHELv4u2:~#
```

40.3.5. display the new partition

Let's verify again with **fdisk -l** to make sure reality fits our dreams. Indeed, the screenshot below now shows a partition on **/dev/sdb**.

```
root@RHELv4u2:~# fdisk -l

Disk /dev/sda: 12.8 GB, 12884901888 bytes
255 heads, 63 sectors/track, 1566 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

Device Boot      Start         End      Blocks   Id  System
/dev/sda1 *          1          13        104391   83  Linux
/dev/sda2          14        1566     12474472+   8e  Linux LVM

Disk /dev/sdb: 1073 MB, 1073741824 bytes
255 heads, 63 sectors/track, 130 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes

Device Boot      Start         End      Blocks   Id  System
/dev/sdb1          1          14        112423+   83  Linux
root@RHELv4u2:~#
```


40.4. about the partition table

40.4.1. master boot record

The **partition table** information (primary and extended partitions) is written in the **master boot record** or **mbr**. You can use **dd** to copy the mbr to a file.

This example copies the master boot record from the first SCSI hard disk.

```
dd if=/dev/sda of=/SCSIdisk.mbr bs=512 count=1
```

The same tool can also be used to wipe out all information about partitions on a disk. This example writes zeroes over the master boot record.

```
dd if=/dev/zero of=/dev/sda bs=512 count=1
```

Or to wipe out the whole partition or disk.

```
dd if=/dev/zero of=/dev/sda
```

40.4.2. partprobe

Don't forget that after restoring a **master boot record** with **dd**, that you need to force the kernel to reread the partition table with **partprobe**. After running **partprobe**, the partitions can be used again.

```
[root@RHEL5 ~]# partprobe  
[root@RHEL5 ~]#
```

40.4.3. logical drives

The **partition table** does not contain information about **logical drives**. So the **dd** backup of the **mbr** only works for primary and extended partitions. To backup the partition table including the logical drives, you can use **sfdisk**.

This example shows how to backup all partition and logical drive information to a file.

```
sfdisk -d /dev/sda > parttable.sda.sfdisk
```

The following example copies the **mbr** and all **logical drive** info from /dev/sda to /dev/sdb.

```
sfdisk -d /dev/sda | sfdisk /dev/sdb
```

40.5. GUID partition table

gpt was developed because of the limitations of the 1980s **mbr** partitioning scheme (for example only four partitions can be defined, and they have a maximum size two terabytes).

Since 2010 **gpt** is a part of the **uefi** specification, but it is also used on **bios** systems.

Newer versions of **fdisk** work fine with **gpt**, but most production servers today (mid 2015) still have an older **fdisk**.. You can use **parted** instead.

40.6. labeling with parted

parted is an interactive tool, just like **fdisk**. Type **help** in **parted** for a list of commands and options.

This screenshot shows how to start **parted** to manage partitions on **/dev/sdb**.

```
[root@rhel71 ~]# parted /dev/sdb
GNU Parted 3.1
Using /dev/sdb
Welcome to GNU Parted! Type 'help' to view a list of commands.
(parted)
```

Each command also has built-in help. For example **help mklabel** will list all supported labels. Note that we only discussed **mbr**(msdos) and **gpt** in this book.

```
(parted) help mklabel
mklabel,mktable LABEL-TYPE          create a new disklabel (partition table)

    LABEL-TYPE is one of: aix, amiga, bsd, dvh, gpt, mac, msdos, pc98, sun, loop
(parted)
```

We create an **mbr** label.

```
(parted) mklabel msdos>
Warning: The existing disk label on /dev/sdb will be destroyed and all data on
this disk will be lost. Do you want to continue?
Yes/No? yes
(parted) mklabel gpt
Warning: The existing disk label on /dev/sdb will be destroyed and all data on
this disk will be lost. Do you want to continue?
Yes/No? Y
(parted)
```

40.6.1. partitioning with parted

Once labeled it is easy to create partitions with **parted**. This screenshot starts with an unpartitioned (but **gpt** labeled) disk.

```
(parted) print
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sdb: 8590MB
Sector size (logical/physical): 512B/512B
Partition Table: gpt
Disk Flags:

Number  Start  End  Size  File system  Name  Flags
(parted)
```

This example shows how to create two primary partitions of equal size.

```
(parted) mkpart primary 0 50%
Warning: The resulting partition is not properly aligned for best performance.
Ignore/Cancel? I
(parted) mkpart primary 50% 100%
(parted)
```

Verify with **print** and exit with **quit**. Since **parted** works directly on the disk, there is no need to **w(rite)** like in **fdisk**.

```
(parted) print
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sdb: 8590MB
Sector size (logical/physical): 512B/512B
Partition Table: gpt
Disk Flags:

Number  Start  End  Size  File system  Name  Flags
1       17.4kB 4295MB 4295MB                primary
2       4295MB 8589MB 4294MB                primary
(parted) quit
Information: You may need to update /etc/fstab.

[root@rhel71 ~]#
```

40.7. practice: partitions

1. Use **fdisk -l** to display existing partitions and sizes.
2. Use **df -h** to display existing partitions and sizes.
3. Compare the output of **fdisk** and **df**.
4. Create a 200MB primary partition on a small disk.
5. Create a 400MB primary partition and two 300MB logical drives on a big disk.
6. Use **df -h** and **fdisk -l** to verify your work.
7. Compare the output again of **fdisk** and **df**. Do both commands display the new partitions ?
8. Create a backup with **dd** of the **mbr** that contains your 200MB primary partition.
9. Take a backup of the **partition table** containing your 400MB primary and 300MB logical drives. Make sure the logical drives are in the backup.
10. (optional) Remove all your partitions with **fdisk**. Then restore your backups.

40.8. solution: partitions

1. Use **fdisk -l** to display existing partitions and sizes.

```
as root: # fdisk -l
```

2. Use **df -h** to display existing partitions and sizes.

```
df -h
```

3. Compare the output of **fdisk** and **df**.

Some partitions will be listed in both outputs (maybe /dev/sda1 or /dev/hda1).

4. Create a 200MB primary partition on a small disk.

Choose one of the disks you added (this example uses /dev/sdc).

```
root@rhel53 ~# fdisk /dev/sdc
...
Command (m for help): n
Command action
   e   extended
   p   primary partition (1-4)
p
Partition number (1-4): 1
First cylinder (1-261, default 1): 1
Last cylinder or +size or +sizeM or +sizeK (1-261, default 261): +200m
Command (m for help): w
The partition table has been altered!
Calling ioctl() to re-read partition table.
Syncing disks.
```

5. Create a 400MB primary partition and two 300MB logical drives on a big disk.

Choose one of the disks you added (this example uses /dev/sdb)

```
fdisk /dev/sdb
```

```
inside fdisk : n p 1 +400m enter --- n e 2 enter enter --- n l +300m (twice)
```

6. Use **df -h** and **fdisk -l** to verify your work.

```
fdisk -l ; df -h
```

7. Compare the output again of **fdisk** and **df**. Do both commands display the new partitions ?

The newly created partitions are visible with **fdisk**.

But they are not displayed by **df**.

8. Create a backup with **dd** of the **mbr** that contains your 200MB primary partition.

```
dd if=/dev/sdc of=bootsector.sdc.dd count=1 bs=512
```

9. Take a backup of the **partition table** containing your 400MB primary and 300MB logical drives. Make sure the logical drives are in the backup.

```
sfdisk -d /dev/sdb > parttable.sdb.sfdisk
```

Chapter 41. file systems

When you are finished partitioning the hard disk, you can put a **file system** on each partition.

This chapter builds on the **partitions** from the previous chapter, and prepares you for the next one where we will **mount** the filesystems.

41.1. about file systems

A file system is a way of organizing files on your partition. Besides file-based storage, file systems usually include **directories** and **access control**, and contain meta information about files like access times, modification times and file ownership.

The properties (length, character set, ...) of filenames are determined by the file system you choose. Directories are usually implemented as files, you will have to learn how this is implemented! Access control in file systems is tracked by user ownership (and group ownership and membership) in combination with one or more access control lists.

41.1.1. man fs

The manual page about filesystems is accessed by typing **man fs**.

```
[root@rhel65 ~]# man fs
```

41.1.2. /proc/filesystems

The Linux kernel will inform you about currently loaded file system drivers in **/proc/filesystems**.

```
root@rhel53 ~# cat /proc/filesystems | grep -v nodev
ext2
iso9660
ext3
```

41.1.3. /etc/filesystems

The **/etc/filesystems** file contains a list of autodetected filesystems (in case the **mount** command is used without the **-t** option).

Help for this file is provided by **man mount**.

```
[root@rhel65 ~]# man mount
```

41.2. common file systems

41.2.1. ext2 and ext3

Once the most common Linux file systems is the **ext2** (the second extended) file system. A disadvantage is that file system checks on ext2 can take a long time.

ext2 was being replaced by **ext3** on most Linux machines. They are essentially the same, except for the **journaling** which is only present in ext3.

Journaling means that changes are first written to a journal on the disk. The journal is flushed regularly, writing the changes in the file system. Journaling keeps the file system in a consistent state, so you don't need a file system check after an unclean shutdown or power failure.

41.2.2. creating ext2 and ext3

You can create these file systems with the **/sbin/mkfs** or **/sbin/mke2fs** commands. Use **mke2fs -j** to create an **ext3** file system.

You can convert an ext2 to ext3 with **tune2fs -j**. You can mount an ext3 file system as ext2, but then you lose the journaling. Do not forget to run **mkinitrd** if you are booting from this device.

41.2.3. ext4

The newest incarnation of the ext file system is named **ext4** and is available in the Linux kernel since 2008. **ext4** supports larger files (up to 16 terabyte) and larger file systems than **ext3** (and many more features).

Development started by making **ext3** fully capable for 64-bit. When it turned out the changes were significant, the developers decided to name it **ext4**.

41.2.4. xfs

Redhat Enterprise Linux 7 will have **XFS** as the default file system. This is a highly scalable high-performance file system.

xfs was created for **Irix** and for a couple of years it was also used in **FreeBSD**. It is supported by the Linux kernel, but rarely used in distributions outside of the Redhat/CentOS realm.

41.2.5. vfat

The **vfat** file system exists in a couple of forms : **fat12** for floppy disks, **fat16** on **ms-dos**, and **fat32** for larger disks. The Linux **vfat** implementation supports all of these, but **vfat** lacks a lot of features like security and links. **fat** disks can be read by every operating system, and are used a lot for digital cameras, **usb** sticks and to exchange data between different OS'ses on a home user's computer.

41.2.6. iso 9660

iso 9660 is the standard format for cdroms. Chances are you will encounter this file system also on your hard disk in the form of images of cdroms (often with the .iso extension). The **iso 9660** standard limits filenames to the 8.3 format. The Unix world didn't like this, and thus added the **rock ridge** extensions, which allows for filenames up to 255 characters and Unix-style file-modes, ownership and symbolic links. Another extensions to **iso 9660** is **joliet**, which adds 64 unicode characters to the filename. The **el torito** standard extends **iso 9660** to be able to boot from CD-ROM's.

41.2.7. udf

Most optical media today (including cd's and dvd's) use **udf**, the Universal Disk Format.

41.2.8. swap

All things considered, swap is not a file system. But to use a partition as a **swap partition** it must be formatted and mounted as swap space.

41.2.9. gfs

Linux clusters often use a dedicated cluster filesystem like GFS, GFS2, ClusterFS, ...

41.2.10. and more...

You may encounter **reiserfs** on older Linux systems. Maybe you will see Sun's **zfs** or the open source **btrfs**. This last one requires a chapter on itself.

41.2.11. /proc/filesystems

The **/proc/filesystems** file displays a list of supported file systems. When you mount a file system without explicitly defining one, then mount will first try to probe **/etc/filesystems** and then probe **/proc/filesystems** for all the filesystems without the **nodev** label. If **/etc/filesystems** ends with a line containing only an asterisk (*) then both files are probed.

```
paul@RHELv4u4:~$ cat /proc/filesystems
nodev    sysfs
nodev    rootfs
nodev    bdev
nodev    proc
nodev    sockfs
nodev    binfmt_misc
nodev    usbfs
nodev    usbdevfs
nodev    futexfs
nodev    tmpfs
nodev    pipefs
nodev    eventpollfs
nodev    devpts
nodev    ext2
nodev    ramfs
nodev    hugetlbfs
nodev    iso9660
nodev    relayfs
nodev    mqueue
nodev    selinuxfs
nodev    ext3
nodev    rpc_pipefs
nodev    vmware-hgfs
nodev    autofs
paul@RHELv4u4:~$
```

41.3. putting a file system on a partition

We now have a fresh partition. The system binaries to make file systems can be found with `ls`.

```
[root@RHEL4b ~]# ls -ls /sbin/mk*
-rwxr-xr-x 3 root root 34832 Apr 24 2006 /sbin/mke2fs
-rwxr-xr-x 3 root root 34832 Apr 24 2006 /sbin/mkfs.ext2
-rwxr-xr-x 3 root root 34832 Apr 24 2006 /sbin/mkfs.ext3
-rwxr-xr-x 3 root root 28484 Oct 13 2004 /sbin/mkdosfs
-rwxr-xr-x 3 root root 28484 Oct 13 2004 /sbin/mkfs.msdos
-rwxr-xr-x 3 root root 28484 Oct 13 2004 /sbin/mkfs.vfat
-rwxr-xr-x 1 root root 20313 Apr 10 2006 /sbin/mkinitrd
-rwxr-x-- 1 root root 15444 Oct 5 2004 /sbin/mkzonedb
-rwxr-xr-x 1 root root 15300 May 24 2006 /sbin/mkfs.cramfs
-rwxr-xr-x 1 root root 13036 May 24 2006 /sbin/mkswap
-rwxr-xr-x 1 root root 6912 May 24 2006 /sbin/mkfs
-rwxr-xr-x 1 root root 5905 Aug 3 2004 /sbin/mkbootdisk
[root@RHEL4b ~]#
```

It is time for you to read the manual pages of **mkfs** and **mke2fs**. In the example below, you see the creation of an **ext2 file system** on `/dev/sdb1`. In real life, you might want to use options like `-m0` and `-j`.

```
root@RHELV4u2:~# mke2fs /dev/sdb1
mke2fs 1.35 (28-Feb-2004)
Filesystem label=
OS type: Linux
Block size=1024 (log=0)
Fragment size=1024 (log=0)
28112 inodes, 112420 blocks
5621 blocks (5.00%) reserved for the super user
First data block=1
Maximum filesystem blocks=67371008
14 block groups
8192 blocks per group, 8192 fragments per group
2008 inodes per group
Superblock backups stored on blocks:
8193, 24577, 40961, 57345, 73729

Writing inode tables: done
Writing superblocks and filesystem accounting information: done

This filesystem will be automatically checked every 37 mounts or
180 days, whichever comes first. Use tune2fs -c or -i to override.
```

41.4. tuning a file system

You can use **tune2fs** to list and set file system settings. The first screenshot lists the reserved space for root (which is set at five percent).

```
[root@rhel4 ~]# tune2fs -l /dev/sda1 | grep -i "block count"
Block count:          104388
Reserved block count:  5219
[root@rhel4 ~]#
```

This example changes this value to ten percent. You can use **tune2fs** while the file system is active, even if it is the root file system (as in this example).

```
[root@rhel4 ~]# tune2fs -m10 /dev/sda1
tune2fs 1.35 (28-Feb-2004)
Setting reserved blocks percentage to 10 (10430 blocks)
[root@rhel4 ~]# tune2fs -l /dev/sda1 | grep -i "block count"
Block count:          104388
Reserved block count:  10430
[root@rhel4 ~]#
```

41.5. checking a file system

The **fsck** command is a front end tool used to check a file system for errors.

```
[root@RHEL4b ~]# ls /sbin/*fsck*
/sbin/dosfsck  /sbin/fsck          /sbin/fsck.ext2  /sbin/fsck.msdos
/sbin/e2fsck   /sbin/fsck.cramfs    /sbin/fsck.ext3  /sbin/fsck.vfat
[root@RHEL4b ~]#
```

The last column in **/etc/fstab** is used to determine whether a file system should be checked at boot-up.

```
[paul@RHEL4b ~]$ grep ext /etc/fstab
/dev/VolGroup00/LogVol00 / ext3 defaults 1 1
LABEL=/boot /boot ext3 defaults 1 2
[paul@RHEL4b ~]$
```

Manually checking a mounted file system results in a warning from fsck.

```
[root@RHEL4b ~]# fsck /boot
fsck 1.35 (28-Feb-2004)
e2fsck 1.35 (28-Feb-2004)
/dev/sda1 is mounted.

WARNING!!! Running e2fsck on a mounted filesystem may cause
SEVERE filesystem damage.

Do you really want to continue (y/n)? no

check aborted.
```

But after unmounting fsck and **e2fsck** can be used to check an ext2 file system.

```
[root@RHEL4b ~]# fsck /boot
fsck 1.35 (28-Feb-2004)
e2fsck 1.35 (28-Feb-2004)
/boot: clean, 44/26104 files, 17598/104388 blocks
[root@RHEL4b ~]# fsck -p /boot
fsck 1.35 (28-Feb-2004)
/boot: clean, 44/26104 files, 17598/104388 blocks
[root@RHEL4b ~]# e2fsck -p /dev/sda1
/boot: clean, 44/26104 files, 17598/104388 blocks
```

41.6. practice: file systems

1. List the filesystems that are known by your system.
2. Create an **ext2** filesystem on the 200MB partition.
3. Create an **ext3** filesystem on one of the 300MB logical drives.
4. Create an **ext4** on the 400MB partition.
5. Set the reserved space for root on the ext3 filesystem to 0 percent.
6. Verify your work with **fdisk** and **df**.
7. Perform a file system check on all the new file systems.

41.7. solution: file systems

1. List the filesystems that are known by your system.

```
man fs
```

```
cat /proc/filesystems
```

```
cat /etc/filesystems (not on all Linux distributions)
```

2. Create an **ext2** filesystem on the 200MB partition.

```
mke2fs /dev/sdc1 (replace sdc1 with the correct partition)
```

3. Create an **ext3** filesystem on one of the 300MB logical drives.

```
mke2fs -j /dev/sdb5 (replace sdb5 with the correct partition)
```

4. Create an **ext4** on the 400MB partition.

```
mkfs.ext4 /dev/sdb1 (replace sdb1 with the correct partition)
```

5. Set the reserved space for root on the ext3 filesystem to 0 percent.

```
tune2fs -m 0 /dev/sdb5
```

6. Verify your work with **fdisk** and **df**.

```
mkfs (mke2fs) makes no difference in the output of these commands
```

```
The big change is in the next topic: mounting
```

7. Perform a file system check on all the new file systems.

```
fsck /dev/sdb1
```

```
fsck /dev/sdc1
```

```
fsck /dev/sdb5
```

Chapter 42. mounting

Once you've put a file system on a partition, you can **mount** it. Mounting a file system makes it available for use, usually as a directory. We say **mounting a file system** instead of mounting a partition because we will see later that we can also mount file systems that do not exist on partitions.

On all **Unix** systems, every file and every directory is part of one big file tree. To access a file, you need to know the full path starting from the root directory. When adding a **file system** to your computer, you need to make it available somewhere in the file tree. The directory where you make a file system available is called a **mount point**.

42.1. mounting local file systems

42.1.1. mkdir

This example shows how to create a new **mount point** with **mkdir**.

```
root@RHELV4u2:~# mkdir /home/project42
```

42.1.2. mount

When the **mount point** is created, and a **file system** is present on the partition, then **mount** can **mount** the **file system** on the **mount point directory**.

```
root@RHELV4u2:~# mount -t ext2 /dev/sdb1 /home/project42/
```

Once mounted, the new file system is accessible to users.

42.1.3. /etc/filesystems

Actually the explicit **-t ext2** option to set the file system is not always necessary. The **mount** command is able to automatically detect a lot of file systems.

When mounting a file system without specifying explicitly the file system, then **mount** will first probe **/etc/filesystems**. Mount will skip lines with the **nodev** directive.

```
paul@RHELV4u4:~$ cat /etc/filesystems
ext3
ext2
nodev proc
nodev devpts
iso9660
vfat
hfs
```

42.1.4. /proc/filesystems

When **/etc/filesystems** does not exist, or ends with a single ***** on the last line, then **mount** will read **/proc/filesystems**.

```
[root@RHEL52 ~]# cat /proc/filesystems | grep -v ^nodev
ext2
iso9660
ext3
```

42.1.5. umount

You can **unmount** a mounted file system using the **umount** command.

```
root@pasha:~# umount /home/reet
```

42.2. displaying mounted file systems

To display all mounted file systems, issue the **mount** command. Or look at the files **/proc/mounts** and **/etc/mtab**.

42.2.1. mount

The simplest and most common way to view all mounts is by issuing the **mount** command without any arguments.

```
root@RHELv4u2:~# mount | grep /dev/sdb
/dev/sdb1 on /home/project42 type ext2 (rw)
```

42.2.2. /proc/mounts

The kernel provides the info in **/proc/mounts** in file form, but **/proc/mounts** does not exist as a file on any hard disk. Looking at **/proc/mounts** is looking at information that comes directly from the kernel.

```
root@RHELv4u2:~# cat /proc/mounts | grep /dev/sdb
/dev/sdb1 /home/project42 ext2 rw 0 0
```

42.2.3. /etc/mtab

The **/etc/mtab** file is not updated by the kernel, but is maintained by the **mount** command. Do not edit **/etc/mtab** manually.

```
root@RHELv4u2:~# cat /etc/mtab | grep /dev/sdb
/dev/sdb1 /home/project42 ext2 rw 0 0
```

42.2.4. df

A more user friendly way to look at mounted file systems is **df**. The **df (diskfree)** command has the added benefit of showing you the free space on each mounted disk. Like a lot of Linux commands, **df** supports the **-h** switch to make the output more **human readable**.

```
root@RHELv4u2:~# df
Filesystem            1K-blocks      Used Available Use% Mounted on
/dev/mapper/VolGroup00-LogVol00
11707972 6366996 4746240 58% /
/dev/sda1              101086      9300    86567  10% /boot
none                 127988        0    127988   0% /dev/shm
/dev/sdb1             108865     1550    101694   2% /home/project42
root@RHELv4u2:~# df -h
Filesystem            Size  Used Avail Use% Mounted on
/dev/mapper/VolGroup00-LogVol00
12G  6.1G  4.6G  58% /
/dev/sda1              99M   9.1M   85M  10% /boot
none                 125M     0  125M   0% /dev/shm
/dev/sdb1             107M   1.6M  100M   2% /home/project42
```

42.2.5. df -h

In the **df -h** example below you can see the size, free space, used gigabytes and percentage and mount point of a partition.

```
root@laika:~# df -h | egrep -e "(sdb2|File)"
Filesystem            Size Used Avail Use% Mounted on
/dev/sdb2             92G   83G   8.6G  91% /media/sdb2
```

42.2.6. du

The **du** command can summarize **disk usage** for files and directories. By using **du** on a mount point you effectively get the disk space used on a file system.

While **du** can go display each subdirectory recursively, the **-s** option will give you a total summary for the parent directory. This option is often used together with **-h**. This means **du -sh** on a mount point gives the total amount used by the file system in that partition.

```
root@debian6~# du -sh /boot /srv/wolf
6.2M /boot
1.1T /srv/wolf
```

42.3. from start to finish

Below is a screenshot that show a summary roadmap starting with detection of the hardware (/dev/sdb) up until mounting on /mnt.

```
[root@centos65 ~]# dmesg | grep '\[sdb\]'
sd 3:0:0:0: [sdb] 150994944 512-byte logical blocks: (77.3 GB/72.0 GiB)
sd 3:0:0:0: [sdb] Write Protect is off
sd 3:0:0:0: [sdb] Mode Sense: 00 3a 00 00
sd 3:0:0:0: [sdb] Write cache: enabled, read cache: enabled, doesn't support \
DPO or FUA
sd 3:0:0:0: [sdb] Attached SCSI disk

[root@centos65 ~]# parted /dev/sdb

(parted) mklabel msdos
(parted) mkpart primary ext4 1 77000
(parted) print
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sdb: 77.3GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos

Number  Start   End     Size    Type    File system  Flags
  1      1049kB  77.0GB  77.0GB  primary

(parted) quit
[root@centos65 ~]# mkfs.ext4 /dev/sdb1
mke2fs 1.41.12 (17-May-2010)
Filesystem label=
OS type: Linux
Block size=4096 (log=2)
Fragment size=4096 (log=2)
Stride=0 blocks, Stripe width=0 blocks
4702208 inodes, 18798592 blocks
939929 blocks (5.00%) reserved for the super user
First data block=0
Maximum filesystem blocks=4294967296
574 block groups
32768 blocks per group, 32768 fragments per group
8192 inodes per group
( output truncated )
...
[root@centos65 ~]# mount /dev/sdb1 /mnt
[root@centos65 ~]# mount | grep mnt
/dev/sdb1 on /mnt type ext4 (rw)
[root@centos65 ~]# df -h | grep mnt
/dev/sdb1          71G  180M   67G   1% /mnt
[root@centos65 ~]# du -sh /mnt
20K    /mnt
[root@centos65 ~]# umount /mnt
```

42.4. permanent mounts

Until now, we performed all mounts manually. This works nice, until the next reboot. Luckily there is a way to tell your computer to automatically mount certain file systems during boot.

42.4.1. /etc/fstab

The file system table located in **/etc/fstab** contains a list of file systems, with an option to automatically mount each of them at boot time.

Below is a sample **/etc/fstab** file.

```
root@RHELv4u2:~# cat /etc/fstab
/dev/VolGroup00/LogVol00 /                ext3    defaults    1 1
LABEL=/boot             /boot    ext3    defaults    1 2
none                    /dev/pts devpts   gid=5,mode=620 0 0
none                    /dev/shm tmpfs    defaults    0 0
none                    /proc    proc     defaults    0 0
none                    /sys     sysfs    defaults    0 0
/dev/VolGroup00/LogVol01 swap        swap     defaults    0 0
```

By adding the following line, we can automate the mounting of a file system.

```
/dev/sdb1                /home/project42    ext2    defaults    0 0
```

42.4.2. mount /mountpoint

Adding an entry to **/etc/fstab** has the added advantage that you can simplify the **mount** command. The command in the screenshot below forces **mount** to look for the partition info in **/etc/fstab**.

```
root@rhel65:~# mount /home/project42
```

42.5. securing mounts

File systems can be secured with several **mount options**. Here are some examples.

42.5.1. ro

The **ro** option will mount a file system as read only, preventing anyone from writing.

```
root@rhel53 ~# mount -t ext2 -o ro /dev/hdb1 /home/project42
root@rhel53 ~# touch /home/project42/testwrite
touch: cannot touch `/home/project42/testwrite': Read-only file system
```

42.5.2. noexec

The **noexec** option will prevent the execution of binaries and scripts on the mounted file system.

```
root@rhel53 ~# mount -t ext2 -o noexec /dev/hdb1 /home/project42
root@rhel53 ~# cp /bin/cat /home/project42
root@rhel53 ~# /home/project42/cat /etc/hosts
-bash: /home/project42/cat: Permission denied
root@rhel53 ~# echo echo hello > /home/project42/helloscript
root@rhel53 ~# chmod +x /home/project42/helloscript
root@rhel53 ~# /home/project42/helloscript
-bash: /home/project42/helloscript: Permission denied
```

42.5.3. nosuid

The **nosuid** option will ignore **setuid** bit set binaries on the mounted file system.

Note that you can still set the **setuid** bit on files.

```
root@rhel53 ~# mount -o nosuid /dev/hdb1 /home/project42
root@rhel53 ~# cp /bin/sleep /home/project42/
root@rhel53 ~# chmod 4555 /home/project42/sleep
root@rhel53 ~# ls -l /home/project42/sleep
-r-sr-xr-x 1 root root 19564 Jun 24 17:57 /home/project42/sleep
```

But users cannot exploit the **setuid** feature.

```
root@rhel53 ~# su - paul
[paul@rhel53 ~]$ /home/project42/sleep 500 &
[1] 2876
[paul@rhel53 ~]$ ps -f 2876
UID      PID  PPID  C  STIME TTY          STAT      TIME CMD
paul      2876  2853   0  17:58 pts/0    S          0:00 /home/project42/sleep 500
[paul@rhel53 ~]$
```

42.5.4. noacl

To prevent cluttering permissions with **acl**'s, use the **noacl** option.

```
root@rhel53 ~# mount -o noacl /dev/hdb1 /home/project42
```

More **mount options** can be found in the manual page of **mount**.

42.6. mounting remote file systems

42.6.1. smb/cifs

The Samba team (samba.org) has a Unix/Linux service that is compatible with the SMB/CIFS protocol. This protocol is mainly used by networked Microsoft Windows computers.

Connecting to a Samba server (or to a Microsoft computer) is also done with the **mount** command.

This example shows how to connect to the **10.0.0.42** server, to a share named **data2**.

```
[root@centos65 ~]# mount -t cifs -o user=paul //10.0.0.42/data2 /home/data2
Password:
[root@centos65 ~]# mount | grep cifs
//10.0.0.42/data2 on /home/data2 type cifs (rw)
```

The above requires **yum install cifs-client**.

42.6.2. nfs

Unix servers often use **nfs** (aka the network file system) to share directories over the network. Setting up an nfs server is discussed later. Connecting as a client to an nfs server is done with **mount**, and is very similar to connecting to local storage.

This command shows how to connect to the nfs server named **server42**, which is sharing the directory **/srv/data**. The **mount point** at the end of the command (**/home/data**) must already exist.

```
[root@centos65 ~]# mount -t nfs server42:/srv/data /home/data
[root@centos65 ~]#
```

If this **server42** has ip-address **10.0.0.42** then you can also write:

```
[root@centos65 ~]# mount -t nfs 10.0.0.42:/srv/data /home/data
[root@centos65 ~]# mount | grep data
10.0.0.42:/srv/data on /home/data type nfs (rw,vers=4,addr=10.0.0.42,clienta\
ddr=10.0.0.33)
```

42.6.3. nfs specific mount options

```
bg If mount fails, retry in background.
fg (default) If mount fails, retry in foreground.
soft Stop trying to mount after X attempts.
hard (default) Continue trying to mount.
```

The **soft+bg** options combined guarantee the fastest client boot if there are NFS problems.

```
retrans=X Try X times to connect (over udp).
tcp Force tcp (default and supported)
udp Force udp (unsupported)
```

42.7. practice: mounting file systems

1. Mount the small 200MB partition on /home/project22.
2. Mount the big 400MB primary partition on /mnt, the copy some files to it (everything in / etc). Then umount, and mount the file system as read only on /srv/nfs/salesnumbers. Where are the files you copied ?
3. Verify your work with **fdisk**, **df** and **mount**. Also look in **/etc/mtab** and **/proc/mounts**.
4. Make both mounts permanent, test that it works.
5. What happens when you mount a file system on a directory that contains some files ?
6. What happens when you mount two file systems on the same mount point ?
7. (optional) Describe the difference between these commands: find, locate, updatedb, makewhatis, whereis, apropos, which and type.
8. (optional) Perform a file system check on the partition mounted at /srv/nfs/salesnumbers.

42.8. solution: mounting file systems

1. Mount the small 200MB partition on /home/project22.

```
mkdir /home/project22
mount /dev/sdc1 /home/project22
```

2. Mount the big 400MB primary partition on /mnt, the copy some files to it (everything in / etc). Then umount, and mount the file system as read only on /srv/nfs/salesnumbers. Where are the files you copied ?

```
mount /dev/sdb1 /mnt
cp -r /etc /mnt
ls -l /mnt
```

```
umount /mnt
ls -l /mnt
```

```
mkdir -p /srv/nfs/salesnumbers
mount /dev/sdb1 /srv/nfs/salesnumbers
```

You see the files in /srv/nfs/salenumbers now...

But physically they are on ext3 on partition /dev/sdb1

3. Verify your work with **fdisk**, **df** and **mount**. Also look in **/etc/mtab** and **/proc/mounts**.

```
fdisk -l
df -h
mount
```

All three the above commands should show your mounted partitions.

```
grep project22 /etc/mtab
grep project22 /proc/mounts
```

4. Make both mounts permanent, test that it works.

add the following lines to /etc/fstab

```
/dev/sdc1 /home/project22 auto defaults 0 0
/dev/sdb1 /srv/nfs/salesnumbers auto defaults 0 0
```

5. What happens when you mount a file system on a directory that contains some files ?

The files are hidden until **umount**.

6. What happens when you mount two file systems on the same mount point ?

Only the last mounted fs is visible.

7. (optional) Describe the difference between these commands: find, locate, updatedb, makewhatis, whereis, apropos, which and type.

```
man find
man locate
...
```

8. (optional) Perform a file system check on the partition mounted at /srv/nfs/salesnumbers.

```
# umount /srv/nfs/salesnumbers (optional but recommended)
# fsck /dev/sdb1
```

Chapter 43. troubleshooting tools

This chapter introduces some tools that go beyond **df -h** and **du -sh**. Tools that will enable you to troubleshoot a variety of issues with **file systems** and storage.

43.1. lsof

List open files with **lsof**.

When invoked without options, **lsof** will list all open files. You can see the command (`init` in this case), its PID (1) and the user (`root`) has opened the root directory and `/sbin/init`. The FD (file descriptor) column shows that `/` is both the root directory (`rtd`) and current working directory (`cwd`) for the `/sbin/init` command. The FD column displays **rtd** for root directory, **cwd** for current directory and **txt** for text (both including data and code).

```
root@debian7:~# lsof | head -4
COMMAND PID  TID  USER  FD      TYPE    DEVICE  SIZE/OFF      NODE NAME
init      1          root   cwd     DIR     254,0    4096         2 /
init      1          root   rtd     DIR     254,0    4096         2 /
init      1          root   txt     REG     254,0   36992    130856 /sbin/init
```

Other options in the FD column besides `w` for writing, are `r` for reading and `u` for both reading and writing. You can look at open files for a process id by typing **lsof -p PID**. For `init` this would look like this:

```
lsof -p 1
```

The screenshot below shows basic use of **lsof** to prove that `vi` keeps a `.swp` file open (even when stopped in background) on our freshly mounted file system.

```
[root@RHEL65 ~]# df -h | grep sdb
/dev/sdb1                541M   17M  497M   4% /srv/project33
[root@RHEL65 ~]# vi /srv/project33/busyfile.txt
[1]+  Stopped                  vi /srv/project33/busyfile.txt
[root@RHEL65 ~]# lsof /srv/*
COMMAND  PID USER  FD  TYPE DEVICE SIZE/OFF NODE NAME
vi        3243 root   3u  REG  8,17   4096   12 /srv/project33/.busyfile.txt.swp
```

Here we see that **rsyslog** has a couple of log files open for writing (the FD column).

```
root@debian7:~# lsof /var/log/*
COMMAND  PID USER  FD  TYPE DEVICE SIZE/OFF      NODE NAME
rsyslogd 2013 root   1w  REG  254,0  454297 1308187 /var/log/syslog
rsyslogd 2013 root   2w  REG  254,0  419328 1308189 /var/log/kern.log
rsyslogd 2013 root   5w  REG  254,0  116725 1308200 /var/log/debug
rsyslogd 2013 root   6w  REG  254,0  309847 1308201 /var/log/messages
rsyslogd 2013 root   7w  REG  254,0   17591 1308188 /var/log/daemon.log
rsyslogd 2013 root   8w  REG  254,0  101768 1308186 /var/log/auth.log
```

You can specify a specific user with **lsof -u**. This example shows the current working directory for a couple of command line programs.

```
[paul@RHEL65 ~]$ lsof -u paul | grep home
bash      3302 paul   cwd    DIR   253,0    4096   788024 /home/paul
lsof      3329 paul   cwd    DIR   253,0    4096   788024 /home/paul
grep      3330 paul   cwd    DIR   253,0    4096   788024 /home/paul
lsof      3331 paul   cwd    DIR   253,0    4096   788024 /home/paul
```

The `-u` switch of **lsof** also supports the `^` character meaning 'not'. To see all open files, but not those open by root:

```
lsof -u^root
```

43.2. fuser

The **fuser** command will display the 'user' of a file system.

In this example we still have a **vi** process in background and we use **fuser** to find the process id of the process using this file system.

```
[root@RHEL65 ~]# jobs
[1]+  Stopped                  vi /srv/project33/busyfile.txt
[root@RHEL65 ~]# fuser -m /srv/project33/
/srv/project33/:      3243
```

Adding the **-u** switch will also display the user name.

```
[root@RHEL65 ~]# fuser -m -u /srv/project33/
/srv/project33/:      3243(root)
```

You can quickly kill all processes that are using a specific file (or directory) with the **-k** switch.

```
[root@RHEL65 ~]# fuser -m -k -u /srv/project33/
/srv/project33/:      3243(root)
[1]+  Killed                  vi /srv/project33/busyfile.txt
[root@RHEL65 ~]# fuser -m -u /srv/project33/
[root@RHEL65 ~]#
```

This example shows all processes that are using the current directory (bash and vi in this case).

```
root@debian7:~/test42# vi file42

[1]+  Stopped                  vi file42
root@debian7:~/test42# fuser -v .
          USER      PID ACCESS COMMAND
/root/test42:    root      2909 ..c.. bash
                root      3113 ..c.. vi
```

This example shows that the **vi** command actually accesses **/usr/bin/vim.basic** as an **executable** file.

```
root@debian7:~/test42# fuser -v $(which vi)
          USER      PID ACCESS COMMAND
/usr/bin/vim.basic:  root      3113 ...e. vi
```

The last example shows how to find the process that is accessing a specific file.

```
[root@RHEL65 ~]# vi /srv/project33/busyfile.txt

[1]+  Stopped                  vi /srv/project33/busyfile.txt
[root@RHEL65 ~]# fuser -v -m /srv/project33/busyfile.txt
          USER      PID ACCESS COMMAND
/srv/project33/busyfile.txt:    root      13938 F.... vi

[root@RHEL65 ~]# ps -fp 13938
UID      PID  PPID  C  STIME TTY          TIME CMD
root     13938  3110   0  15:47 pts/0      00:00:00 vi /srv/project33/busyfile.txt
```

43.3. chroot

The **chroot** command creates a shell with an alternate root directory. It effectively hides anything outside of this directory.

In the example below we assume that our system refuses to start (maybe because there is a problem with **/etc/fstab** or the mounting of the root file system).

We start a live system (booted from cd/dvd/usb) to troubleshoot our server. The live system will not use our main hard disk as root device

```
root@livecd:~# df -h | grep root
rootfs          186M   11M  175M    6% /
/dev/loop0      807M  807M    0 100% /lib/live/mount/rootfs/filesystem.squashfs
root@livecd:~# mount | grep root
/dev/loop0 on /lib/live/mount/rootfs/filesystem.squashfs type squashfs (ro)
```

We create some test file on the current rootfs.

```
root@livecd:~# touch /file42
root@livecd:~# mkdir /dir42
root@livecd:~# ls /
bin  dir42  home      lib64  opt    run      srv  usr
boot etc    initrd.img media  proc   sbin     sys  var
dev  file42  lib       mnt    root   selinux  tmp  vmlinuz
```

First we mount the root file system from the disk (which is on **lvm** so we use **/dev/mapper** instead of **/dev/sda5**).

```
root@livecd:~# mount /dev/mapper/packer--debian--7-root /mnt
```

We are now ready to **chroot** into the rootfs on disk.

```
root@livecd:~# cd /mnt
root@livecd:/mnt# chroot /mnt
root@livecd:/# ls /
bin  dev  initrd.img  lost+found  opt    run      srv  usr      vmlinuz
boot etc    lib         media       proc   sbin     sys  vagrant
data home  lib64       mnt         root   selinux  tmp  var
```

Our test files (file42 and dir42) are not visible because they are out of the **chrooted** environment.

Note that the **hostname** of the chrooted environment is identical to the existing hostname.

To exit the **chrooted** file system:

```
root@livecd:/# exit
exit
root@livecd:~# ls /
bin  dir42  home      lib64  opt    run      srv  usr
boot etc    initrd.img media  proc   sbin     sys  var
dev  file42  lib       mnt    root   selinux  tmp  vmlinuz
```

43.4. iostat

iostat reports IO statistics every given period of time. It also includes a small cpu usage summary. This example shows **iostat** running every ten seconds with **/dev/sdc** and **/dev/sde** showing a lot of write activity.

```
[root@RHEL65 ~]# iostat 10 3
Linux 2.6.32-431.el6.x86_64 (RHEL65) 06/16/2014 _x86_64_ (1 CPU)

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           5.81    0.00   3.15    0.18    0.00   90.85

Device:            tps    Blk_read/s    Blk_wrtn/s    Blk_read    Blk_wrtn
sda                 42.08        1204.10        1634.88    1743708    2367530
sdb                 1.20         7.69         45.78     11134     66292
sdc                 0.92         5.30         45.82     7672    66348
sdd                 0.91         5.29         45.78     7656    66292
sde                 1.04         6.28         91.49     9100   132496
sdf                 0.70         3.40         91.46     4918   132440
sdg                 0.69         3.40         91.46     4918   132440
dm-0                191.68       1045.78       1362.30   1514434   1972808
dm-1                 49.26       150.54       243.55   218000    352696

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           56.11    0.00   16.83    0.10    0.00   26.95

Device:            tps    Blk_read/s    Blk_wrtn/s    Blk_read    Blk_wrtn
sda                257.01       10185.97         76.95    101656         768
sdb                 0.00         0.00         0.00         0           0
sdc                 3.81         1.60      2953.11         16    29472
sdd                 0.00         0.00         0.00         0           0
sde                 4.91         1.60     4813.63         16   48040
sdf                 0.00         0.00         0.00         0           0
sdg                 0.00         0.00         0.00         0           0
dm-0                283.77       10185.97         76.95    101656         768
dm-1                 0.00         0.00         0.00         0           0

avg-cpu:  %user   %nice %system %iowait  %steal   %idle
           67.65    0.00   31.11    0.11    0.00    1.13

Device:            tps    Blk_read/s    Blk_wrtn/s    Blk_read    Blk_wrtn
sda                466.86       26961.09        178.28    238336     1576
sdb                 0.00         0.00         0.00         0           0
sdc                 31.45         0.90     24997.29         8   220976
sdd                 0.00         0.00         0.00         0           0
sde                 0.34         0.00         5.43         0         48
sdf                 0.00         0.00         0.00         0           0
sdg                 0.00         0.00         0.00         0           0
dm-0                503.62       26938.46        178.28    238136     1576
dm-1                 2.83         22.62         0.00        200           0

[root@RHEL65 ~]#
```

Other options are to specify the disks you want to monitor (every 5 seconds here):

```
iostat sdd sde sdf 5
```

Or to show statistics per partition:

```
iostat -p sde -p sdf 5
```

43.5. iotop

iotop works like the **top** command but orders processes by input/output instead of by CPU.

By default **iotop** will show all processes. This example uses **iotop -o** to only display processes with actual I/O.

```
[root@RHEL65 ~]# iotop -o
Total DISK READ: 8.63 M/s | Total DISK WRITE: 0.00 B/s
  TID  PRIO  USER   DISK READ  DISK WRITE  SWAPIN      IO>   COMMAND
15000  be/4  root    2.43 M/s    0.00 B/s    0.00 %    14.60 % tar cjf /srv/di...
25000  be/4  root    6.20 M/s    0.00 B/s    0.00 %     6.15 % tar czf /srv/di...
24988  be/4  root     0.00 B/s    7.21 M/s    0.00 %     0.00 % gzip
25003  be/4  root     0.00 B/s   1591.19 K/s  0.00 %     0.00 % gzip
25004  be/4  root     0.00 B/s   193.51 K/s  0.00 %     0.00 % bzip2
```

Use the **-b** switch to create a log of **iotop** output (instead of the default interactive view).

```
[root@RHEL65 ~]# iotop -bod 10
Total DISK READ: 12.82 M/s | Total DISK WRITE: 5.69 M/s
  TID  PRIO  USER   DISK READ  DISK WRITE  SWAPIN      IO   COMMAND
25153  be/4  root    2.05 M/s    0.00 B/s    0.00 %    7.81 % tar cjf /srv/di...
25152  be/4  root   10.77 M/s    0.00 B/s    0.00 %    2.94 % tar czf /srv/di...
25144  be/4  root   408.54 B/s    0.00 B/s    0.00 %    0.05 % python /usr/sbi...
12516  be/3  root     0.00 B/s   1491.33 K/s  0.00 %    0.04 % [jbd2/sdc1-8]
12522  be/3  root     0.00 B/s    45.48 K/s    0.00 %    0.01 % [jbd2/sde1-8]
25158  be/4  root     0.00 B/s    0.00 B/s    0.00 %    0.00 % [flush-8:64]
25155  be/4  root     0.00 B/s   493.12 K/s  0.00 %    0.00 % bzip2
25156  be/4  root     0.00 B/s    2.81 M/s    0.00 %    0.00 % gzip
25159  be/4  root     0.00 B/s   528.63 K/s  0.00 %    0.00 % [flush-8:32]
```

This is an example of **iotop** to track disk I/O every ten seconds for one user named **vagrant** (and only one process of this user, but this can be omitted). The **-a** switch accumulates I/O over time.

```
[root@RHEL65 ~]# iotop -q -a -u vagrant -b -p 5216 -d 10 -n 10
Total DISK READ: 0.00 B/s | Total DISK WRITE: 0.00 B/s
  TID  PRIO  USER   DISK READ  DISK WRITE  SWAPIN      IO   COMMAND
 5216  be/4  vagrant  0.00 B      0.00 B      0.00 %    0.00 % gzip
Total DISK READ: 818.22 B/s | Total DISK WRITE: 20.78 M/s
 5216  be/4  vagrant  0.00 B     213.89 M    0.00 %    0.00 % gzip
Total DISK READ: 2045.95 B/s | Total DISK WRITE: 23.16 M/s
 5216  be/4  vagrant  0.00 B     430.70 M    0.00 %    0.00 % gzip
Total DISK READ: 1227.50 B/s | Total DISK WRITE: 22.37 M/s
 5216  be/4  vagrant  0.00 B     642.02 M    0.00 %    0.00 % gzip
Total DISK READ: 818.35 B/s | Total DISK WRITE: 16.44 M/s
 5216  be/4  vagrant  0.00 B     834.09 M    0.00 %    0.00 % gzip
Total DISK READ: 6.95 M/s | Total DISK WRITE: 8.74 M/s
 5216  be/4  vagrant  0.00 B     920.69 M    0.00 %    0.00 % gzip
Total DISK READ: 21.71 M/s | Total DISK WRITE: 11.99 M/s
```


43.6. vmstat

While **vmstat** is mainly a memory monitoring tool, it is worth mentioning here for its reporting on summary I/O data for block devices and swap space.

This example shows some disk activity (underneath the **-----io----** column), without swapping.

```
[root@RHEL65 ~]# vmstat 5 10
procs -----memory----- ---swap-- -----io---- --system-- -----cpu-----
 r  b  swpd   free   buff  cache   si   so    bi    bo    in   cs  us  sy  id  wa  st
 0  0   5420   9092  14020 340876    7   12   235   252   77  100   2   1  98   0   0
 2  0   5420   6104  13840 338176    0    0  7401  7812  747 1887  38  12  50   0   0
 2  0   5420  10136  13696 336012    0    0 11334    14 1725 4036  76  24   0   0   0
 0  0   5420  14160  13404 341552    0    0 10161  9914 1174 1924  67  15  18   0   0
 0  0   5420  14300  13420 341564    0    0     0    16   28   18   0   0 100   0   0
 0  0   5420  14300  13420 341564    0    0     0     0   22   16   0   0 100   0   0
...
[root@RHEL65 ~]#
```

You can benefit from **vmstat**'s ability to display memory in kilobytes, megabytes or even kibibytes and mebibytes using **-S** (followed by **k** **K** **m** or **M**).

```
[root@RHEL65 ~]# vmstat -SM 5 10
procs -----memory----- ---swap-- -----io---- --system-- -----cpu-----
 r  b  swpd   free   buff  cache   si   so    bi    bo    in   cs  us  sy  id  wa  st
 0  0     5     14     11   334    0    0   259   255   79  107   2   1  97   0   0
 0  0     5     14     11   334    0    0     0     2   21   18   0   0 100   0   0
 0  0     5     15     11   334    0    0     6     0   35   31   0   0 100   0   0
 2  0     5     6     11   336    0    0 17100  7814 1378 2945  48  21  31   0   0
 2  0     5     6     11   336    0    0 13193    14 1662 3343  78  22   0   0   0
 2  0     5     13     11   330    0    0 11656  9781 1419 2642  82  18   0   0   0
 2  0     5     9     11   334    0    0 10705  2716 1504 2657  81  19   0   0   0
 1  0     5     14     11   336    0    0  6467  3788  765 1384  43   9  48   0   0
 0  0     5     14     11   336    0    0     0    13   28   24   0   0 100   0   0
 0  0     5     14     11   336    0    0     0     0   20   15   0   0 100   0   0
[root@RHEL65 ~]#
```

vmstat is also discussed in other chapters.

43.7. practice: troubleshooting tools

0. It is imperative that you practice these tools **before** trouble arises. It will help you get familiar with the tools and allow you to create a base line of normal behaviour for your systems.
1. Read the theory on **fuser** and explore its man page. Use this command to find files that you open yourself.
2. Read the theory on **lsof** and explore its man page. Use this command to find files that you open yourself.
3. Boot a live image on an existing computer (virtual or real) and **chroot** into to it.
4. Start one or more disk intensive jobs and monitor them with **iostat** and **iotop** (compare to **vmstat**).

43.8. solution: troubleshooting tools

0. It is imperative that you practice these tools **before** trouble arises. It will help you get familiar with the tools and allow you to create a base line of normal behaviour for your systems.

1. Read the theory on **fuser** and explore its man page. Use this command to find files that you open yourself.

2. Read the theory on **lsof** and explore its man page. Use this command to find files that you open yourself.

3. Boot a live image on an existing computer (virtual or real) and **chroot** into to it.

4. Start one or more disk intensive jobs and monitor them with **iostat** and **iotop** (compare to **vmstat**).

Chapter 44. introduction to uuid's

A **uuid** or **universally unique identifier** is used to uniquely identify objects. This 128bit standard allows anyone to create a unique **uuid**.

This chapter takes a brief look at **uuid's**.

44.1. about unique objects

Older versions of Linux have a **vol_id** utility to display the **uuid** of a file system.

```
root@debian5:~# vol_id --uuid /dev/sda1
193c3c9b-2c40-9290-8b71-4264ee4d4c82
```

Red Hat Enterprise Linux 5 puts **vol_id** in **/lib/udev/vol_id**, which is not in the **\$PATH**. The syntax is also a bit different from Debian/Ubuntu/Mint.

```
root@rhel53 ~# /lib/udev/vol_id -u /dev/hda1
48a6a316-9ca9-4214-b5c6-e7b33a77e860
```

This utility is not available in standard installations of RHEL6 or Debian6.

44.2. tune2fs

Use **tune2fs** to find the **uuid** of a file system.

```
[root@RHEL5 ~]# tune2fs -l /dev/sda1 | grep UUID
Filesystem UUID:      11cfc8bc-07c0-4c3f-9f64-78422ef1dd5c
[root@RHEL5 ~]# /lib/udev/vol_id -u /dev/sda1
11cfc8bc-07c0-4c3f-9f64-78422ef1dd5c
```

44.3. uuid

There is more information in the manual of **uuid**, a tool that can generate uuid's.

```
[root@rhel65 ~]# yum install uuid
(output truncated)
[root@rhel65 ~]# man uuid
```

44.4. uuid in /etc/fstab

You can use the **uuid** to make sure that a volume is universally uniquely identified in **/etc/fstab**. The device name can change depending on the disk devices that are present at boot time, but a **uuid** never changes.

First we use **tune2fs** to find the **uuid**.

```
[root@RHEL5 ~]# tune2fs -l /dev/sdc1 | grep UUID
Filesystem UUID: 7626d73a-2bb6-4937-90ca-e451025d64e8
```

Then we check that it is properly added to **/etc/fstab**, the **uuid** replaces the variable devicename **/dev/sdc1**.

```
[root@RHEL5 ~]# grep UUID /etc/fstab
UUID=7626d73a-2bb6-4937-90ca-e451025d64e8 /home/pro42 ext3 defaults 0 0
```

Now we can mount the volume using the mount point defined in **/etc/fstab**.

```
[root@RHEL5 ~]# mount /home/pro42
[root@RHEL5 ~]# df -h | grep 42
/dev/sdc1 397M 11M 366M 3% /home/pro42
```

The real test now, is to remove **/dev/sdb** from the system, reboot the machine and see what happens. After the reboot, the disk previously known as **/dev/sdc** is now **/dev/sdb**.

```
[root@RHEL5 ~]# tune2fs -l /dev/sdb1 | grep UUID
Filesystem UUID: 7626d73a-2bb6-4937-90ca-e451025d64e8
```

And thanks to the **uuid** in **/etc/fstab**, the mountpoint is mounted on the same disk as before.

```
[root@RHEL5 ~]# df -h | grep sdb
/dev/sdb1 397M 11M 366M 3% /home/pro42
```

44.5. uuid as a boot device

Recent Linux distributions (Debian, Ubuntu, ...) use **grub** with a **uuid** to identify the root file system.

This example shows how a **root=/dev/sda1** is replaced with a **uuid**.

```
title          Ubuntu 9.10, kernel 2.6.31-19-generic
uuid           f001ba5d-9077-422a-9634-8d23d57e782a
kernel         /boot/vmlinuz-2.6.31-19-generic \
root=UUID=f001ba5d-9077-422a-9634-8d23d57e782a ro quiet splash
initrd         /boot/initrd.img-2.6.31-19-generic
```

The screenshot above contains only four lines. The line starting with **root=** is the continuation of the **kernel** line.

RHEL and CentOS boot from LVM after a default install.

44.6. practice: uuid and filesystems

1. Find the **uuid** of one of your **ext3** partitions with **tune2fs** (and **vol_id** if you are on RHEL5).
2. Use this **uuid** in **/etc/fstab** and test that it works with a simple **mount**.
3. (optional) Test it also by removing a disk (so the device name is changed). You can edit settings in vmware/Virtualbox to remove a hard disk.
4. Display the **root=** directive in **/boot/grub/menu.lst**. (We see later in the course how to maintain this file.)
5. (optional on ubuntu) Replace the **/dev/xxx** in **/boot/grub/menu.lst** with a **uuid** (use an extra stanza for this). Test that it works.

44.7. solution: uuid and filesystems

1. Find the **uuid** of one of your **ext3** partitions with **tune2fs** (and **vol_id** if you are on RHEL5).

```
root@rhel55:~# /lib/udev/vol_id -u /dev/hda1
60926898-2c78-49b4-a71d-c1d6310c87cc

root@ubu1004:~# tune2fs -l /dev/sda2 | grep UUID
Filesystem UUID:          3007b743-1dce-2d62-9a59-cf25f85191b7
```

2. Use this **uuid** in **/etc/fstab** and test that it works with a simple **mount**.

```
tail -1 /etc/fstab
UUID=60926898-2c78-49b4-a71d-c1d6310c87cc /home/pro42 ext3 defaults 0 0
```

3. (optional) Test it also by removing a disk (so the device name is changed). You can edit settings in vmware/Virtualbox to remove a hard disk.

4. Display the **root=** directive in **/boot/grub/menu.lst**. (We see later in the course how to maintain this file.)

```
paul@deb503:~$ grep ^[^#] /boot/grub/menu.lst | grep root=
kernel      /boot/vmlinuz-2.6.26-2-686 root=/dev/hda1 ro selinux=1 quiet
kernel      /boot/vmlinuz-2.6.26-2-686 root=/dev/hda1 ro selinux=1 single
```

5. (optional on ubuntu) Replace the **/dev/xxx** in **/boot/grub/menu.lst** with a **uuid** (use an extra stanza for this). Test that it works.

Chapter 45. introduction to raid

45.1. hardware or software

Redundant Array of Independent (originally Inexpensive) Disks or **RAID** can be set up using hardware or software. Hardware RAID is more expensive, but offers better performance. Software RAID is cheaper and easier to manage, but it uses your CPU and your memory.

Where ten years ago nobody was arguing about the best choice being hardware RAID, this has changed since technologies like mdadm, lvm and even zfs focus more on managability. The workload on the cpu for software RAID used to be high, but cpu's have gotten a lot faster.

45.2. raid levels

45.2.1. raid 0

raid 0 uses two or more disks, and is often called **striping** (or stripe set, or striped volume). Data is divided in **chunks**, those chunks are evenly spread across every disk in the array. The main advantage of **raid 0** is that you can create **larger drives**. **raid 0** is the only **raid** without redundancy.

45.2.2. jbod

jbod uses two or more disks, and is often called **concatenating** (spanning, spanned set, or spanned volume). Data is written to the first disk, until it is full. Then data is written to the second disk... The main advantage of **jbod** (Just a Bunch of Disks) is that you can create **larger drives**. JBOD offers no redundancy.

45.2.3. raid 1

raid 1 uses exactly two disks, and is often called **mirroring** (or mirror set, or mirrored volume). All data written to the array is written on each disk. The main advantage of **raid 1** is **redundancy**. The main disadvantage is that you lose at least half of your available disk space (in other words, you at least double the cost).

45.2.4. raid 2, 3 and 4 ?

raid 2 uses bit level striping, **raid 3** byte level, and **raid 4** is the same as **raid 5**, but with a dedicated parity disk. This is actually slower than **raid 5**, because every write would have to write parity to this one (bottleneck) disk. It is unlikely that you will ever see these **raid** levels in production.

45.2.5. raid 5

raid 5 uses **three** or more disks, each divided into chunks. Every time chunks are written to the array, one of the disks will receive a **parity** chunk. Unlike **raid 4**, the parity chunk will alternate between all disks. The main advantage of this is that **raid 5** will allow for full data recovery in case of **one** hard disk failure.

45.2.6. raid 6

raid 6 is very similar to **raid 5**, but uses two parity chunks. **raid 6** protects against two hard disk failures. Oracle Solaris **zfs** calls this **raidz2** (and also had **raidz3** with triple parity).

45.2.7. raid 0+1

raid 0+1 is a mirror(1) of stripes(0). This means you first create two **raid 0 stripe** sets, and then you set them up as a mirror set. For example, when you have six 100GB disks, then the stripe sets are each 300GB. Combined in a mirror, this makes 300GB total. **raid 0+1** will survive one disk failure. It will only survive the second disk failure if this disk is in the same stripe set as the previous failed disk.

45.2.8. raid 1+0

raid 1+0 is a stripe(0) of mirrors(1). For example, when you have six 100GB disks, then you first create three mirrors of 100GB each. You then stripe them together into a 300GB drive. In this example, as long as not all disks in the same mirror fail, it can survive up to three hard disk failures.

45.2.9. raid 50

raid 5+0 is a stripe(0) of **raid 5** arrays. Suppose you have nine disks of 100GB, then you can create three **raid 5** arrays of 200GB each. You can then combine them into one large stripe set.

45.2.10. many others

There are many other nested **raid** combinations, like **raid 30**, **51**, **60**, **100**, **150**, ...

45.3. building a software raid5 array

45.3.1. do we have three disks?

First, you have to attach some disks to your computer. In this scenario, three brand new disks of eight gigabyte each are added. Check with **fdisk -l** that they are connected.

```
[root@rhel6c ~]# fdisk -l 2> /dev/null | grep MB
Disk /dev/sdb: 8589 MB, 8589934592 bytes
Disk /dev/sdc: 8589 MB, 8589934592 bytes
Disk /dev/sdd: 8589 MB, 8589934592 bytes
```

45.3.2. fd partition type

The next step is to create a partition of type **fd** on every disk. The **fd** type is to set the partition as **Linux RAID autodetect**. See this (truncated) screenshot:

```
[root@rhel6c ~]# fdisk /dev/sdd
...
Command (m for help): n
Command action
  e   extended
  p   primary partition (1-4)
p
Partition number (1-4): 1
First cylinder (1-1044, default 1):
Using default value 1
Last cylinder, +cylinders or +size{K,M,G} (1-1044, default 1044):
Using default value 1044

Command (m for help): t
Selected partition 1
Hex code (type L to list codes): fd
Changed system type of partition 1 to fd (Linux raid autodetect)

Command (m for help): w
The partition table has been altered!

Calling ioctl() to re-read partition table.
Syncing disks.
```

45.3.3. verify all three partitions

Now all three disks are ready for **raid 5**, so we have to tell the system what to do with these disks.

```
[root@rhel6c ~]# fdisk -l 2> /dev/null | grep raid
/dev/sdb1      1      1044      8385898+  fd  Linux raid autodetect
/dev/sdc1      1      1044      8385898+  fd  Linux raid autodetect
/dev/sdd1      1      1044      8385898+  fd  Linux raid autodetect
```

45.3.4. create the raid5

The next step used to be *create the **raid table** in **/etc/raidtab***. Nowadays, you can just issue the command **mdadm** with the correct parameters.

The command below is split on two lines to fit this print, but you should type it on one line, without the backslash (\).

```
[root@rhel6c ~]# mdadm --create /dev/md0 --chunk=64 --level=5 --raid-\
devices=3 /dev/sdb1 /dev/sdc1 /dev/sdd1
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
```

Below a partial screenshot how **fdisk -l** sees the **raid 5**.

```
[root@rhel6c ~]# fdisk -l /dev/md0

Disk /dev/md0: 17.2 GB, 17172135936 bytes
2 heads, 4 sectors/track, 4192416 cylinders
Units = cylinders of 8 * 512 = 4096 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 65536 bytes / 131072 bytes
Disk identifier: 0x00000000

Disk /dev/md0 doesn't contain a valid partition table
```

We could use this software **raid 5** array in the next topic: **lvm**.

45.3.5. /proc/mdstat

The status of the raid devices can be seen in **/proc/mdstat**. This example shows a **raid 5** in the process of rebuilding.

```
[root@rhel6c ~]# cat /proc/mdstat
Personalities : [raid6] [raid5] [raid4]
md0 : active raid5 sdd1[3] sdc1[1] sdb1[0]
      16769664 blocks super 1.2 level 5, 64k chunk, algorithm 2 [3/2] [UU_]
      [=====>.....]  recovery = 62.8% (5266176/8384832) finish=0\
      .3min speed=139200K/sec
```

This example shows an active software **raid 5**.

```
[root@rhel6c ~]# cat /proc/mdstat
Personalities : [raid6] [raid5] [raid4]
md0 : active raid5 sdd1[3] sdc1[1] sdb1[0]
      16769664 blocks super 1.2 level 5, 64k chunk, algorithm 2 [3/3] [UUU]
```

45.3.6. mdadm --detail

Use **mdadm --detail** to get information on a raid device.

```
[root@rhel6c ~]# mdadm --detail /dev/md0
/dev/md0:
    Version : 1.2
  Creation Time : Sun Jul 17 13:48:41 2011
    Raid Level : raid5
    Array Size : 16769664 (15.99 GiB 17.17 GB)
  Used Dev Size : 8384832 (8.00 GiB 8.59 GB)
    Raid Devices : 3
    Total Devices : 3
 Persistence : Superblock is persistent

    Update Time : Sun Jul 17 13:49:43 2011
      State : clean
 Active Devices : 3
Working Devices : 3
 Failed Devices : 0
  Spare Devices : 0


    Layout : left-symmetric
   Chunk Size : 64K

       Name : rhel6c:0 (local to host rhel6c)
      UUID : c10fd9c3:08f9a25f:be913027:999c8e1f
     Events : 18

   Number   Major   Minor   RaidDevice State
     0         8       17         0     active sync  /dev/sdb1
     1         8       33         1     active sync  /dev/sdc1
     3         8       49         2     active sync  /dev/sdd1
```

45.3.7. removing a software raid

The software raid is visible in **/proc/mdstat** when active. To remove the raid completely so you can use the disks for other purposes, you stop (de-activate) it with **mdadm**.

```
[root@rhel6c ~]# mdadm --stop /dev/md0
mdadm: stopped /dev/md0
```

The disks can now be repartitioned.

45.3.8. further reading

Take a look at the man page of **mdadm** for more information. Below an example command to add a new partition while removing a faulty one.

```
mdadm /dev/md0 --add /dev/sdd1 --fail /dev/sdb1 --remove /dev/sdb1
```

45.4. practice: raid

1. Add three virtual disks of 1GB each to a virtual machine.
2. Create a software **raid 5** on the three disks. (It is not necessary to put a filesystem on it)
3. Verify with **fdisk** and in **/proc** that the **raid 5** exists.
4. Stop and remove the **raid 5**.
5. Create a **raid 1** to mirror two disks.

45.5. solution: raid

1. Add three virtual disks of 1GB each to a virtual machine.
2. Create a software **raid 5** on the three disks. (It is not necessary to put a filesystem on it)
3. Verify with **fdisk** and in **/proc** that the **raid 5** exists.
4. Stop and remove the **raid 5**.
5. Create a **raid 1** to mirror two disks.

```
[root@rhel6c ~]# mdadm --create /dev/md0 --level=1 --raid-devices=2 \  
/dev/sdb1 /dev/sdc1  
mdadm: Defaulting to version 1.2 metadata  
mdadm: array /dev/md0 started.  
[root@rhel6c ~]# cat /proc/mdstat  
Personalities : [raid6] [raid5] [raid4] [raid1]  
md0 : active raid1 sdc1[1] sdb1[0]  
      8384862 blocks super 1.2 [2/2] [UU]  
      [====>.....]    resync = 20.8% (1745152/8384862) \  
finish=0.5min speed=218144K/sec
```

Chapter 46. logical volume management

Most **lvm** implementations support **physical storage grouping**, **logical volume resizing** and **data migration**.

Physical storage grouping is a fancy name for grouping multiple block devices (hard disks, but also iSCSI etc) into a logical mass storage device. To enlarge this physical group, block devices (including partitions) can be added at a later time.

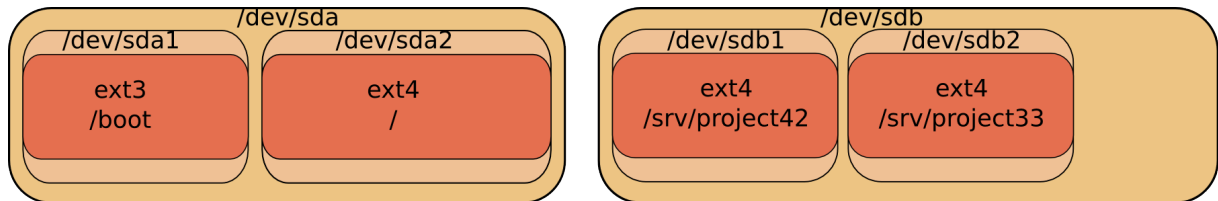
The size of **lvm volumes** on this **physical group** is independent of the individual size of the components. The total size of the group is the limit.

One of the nice features of **lvm** is the logical volume resizing. You can increase the size of an **lvm volume**, sometimes even without any downtime. Additionally, you can migrate data away from a failing hard disk device, create mirrors and create snapshots.

46.1. introduction to lvm

46.1.1. problems with standard partitions

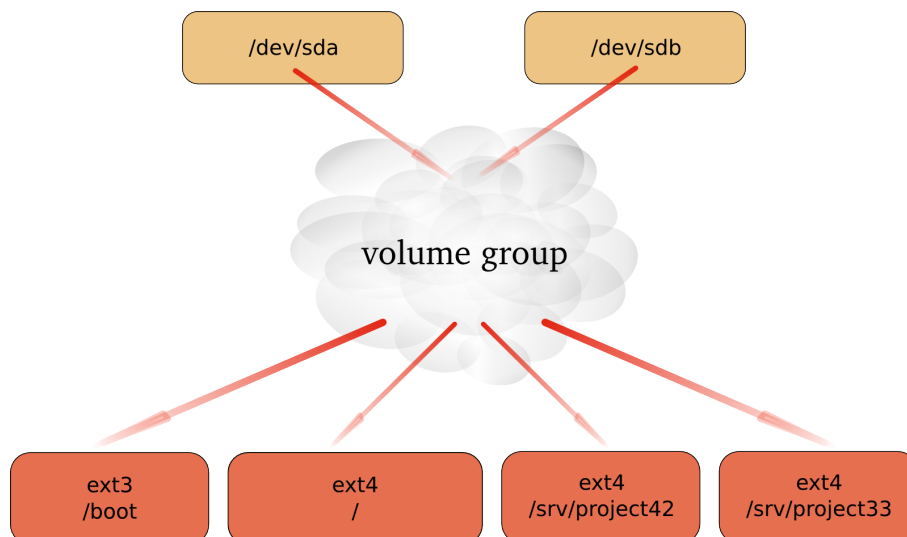
There are some problems when working with hard disks and standard partitions. Consider a system with a small and a large hard disk device, partitioned like this. The first disk (`/dev/sda`) is partitioned in two, the second disk (`/dev/sdb`) has two partitions and some empty space.



In the example above, consider the options when you want to enlarge the space available for `/srv/project42`. What can you do? The solution will always force you to unmount the file system, take a backup of the data, remove and recreate partitions, and then restore the data and remount the file system.

46.1.2. solution with lvm

Using **lvm** will create a virtual layer between the mounted file systems and the hardware devices. This virtual layer will allow for an administrator to enlarge a mounted file system in use. When **lvm** is properly used, then there is no need to unmount the file system to enlarge it.



46.2. lvm terminology

46.2.1. physical volume (pv)

A **physical volume** is any block device (a disk, a partition, a RAID device or even an iSCSI device). All these devices can become a member of a **volume group**.

The commands used to manage a **physical volume** start with pv.

```
[root@centos65 ~]# pv
pvchange  pvck      pvcreate  pvdisplay pvmove    pvremove
pvresize  pvs         pvscan
```

46.2.2. volume group (vg)

A **volume group** is an abstraction layer between **block devices** and **logical volumes**.

The commands used to manage a **volume group** start with vg.

```
[root@centos65 ~]# vg
vgcfgbackup  vgconvert      vgextend      vgmknodes     vgs
vgcfgrestore vgcreate       vgimport      vgreduce      vgscan
vgchange     vgdisplay      vgimportclone vgremove      vgsplit
vgck         vgexport       vgmerge       vgrename
```

46.2.3. logical volume (lv)

A **logical volume** is created in a **volume group**. Logical volumes that contain a file system can be mounted. The use of **logical volumes** is similar to the use of **partitions** and is accomplished with the same standard commands (mkfs, mount, fsck, df, ...).

The commands used to manage a **logical volume** start with lv.

```
[root@centos65 ~]# lv
lvchange     lvextend      lvmdiskscan  lvmsar        lvresize
lvconvert    lvm           lvmdump      lvreduce      lvs
lvcreate     lvmchange     lvmetad      lvremove      lvscan
lvdisplay    lvmconf       lvmsadc      lvrename
```

46.3. example: using lvm

This example shows how you can use a device (in this case `/dev/sdc`, but it could have been `/dev/sdb` or any other disk or partition) with lvm, how to create a volume group (vg) and how to create and use a logical volume (vg/lvol0).

First thing to do, is create physical volumes that can join the volume group with **pvcreate**. This command makes a disk or partition available for use in Volume Groups. The screenshot shows how to present the SCSI Disk device to LVM.

```
root@RHEL4:~# pvcreate /dev/sdc
Physical volume "/dev/sdc" successfully created
```

*Note: lvm **will** work fine when using the complete device, but another operating system on the same computer (or on the same SAN) will not recognize lvm and will mark the block device as being empty! You can avoid this by creating a partition that spans the whole device, then run **pvcreate** on the partition instead of the disk.*

Then **vgcreate** creates a volume group using one device. Note that more devices could be added to the volume group.

```
root@RHEL4:~# vgcreate vg /dev/sdc
Volume group "vg" successfully created
```

The last step **lvcreate** creates a logical volume.

```
root@RHEL4:~# lvcreate --size 500m vg
Logical volume "lvol0" created
```

The logical volume `/dev/vg/lvol0` can now be formatted with `ext3`, and mounted for normal use.

```
root@RHELv4u2:~# mke2fs -m0 -j /dev/vg/lvol0
mke2fs 1.35 (28-Feb-2004)
Filesystem label=
OS type: Linux
Block size=1024 (log=0)
Fragment size=1024 (log=0)
128016 inodes, 512000 blocks
0 blocks (0.00%) reserved for the super user
First data block=1
Maximum filesystem blocks=67633152
63 block groups
8192 blocks per group, 8192 fragments per group
2032 inodes per group
Superblock backups stored on blocks:
8193, 24577, 40961, 57345, 73729, 204801, 221185, 401409

Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done

This filesystem will be automatically checked every 37 mounts or
180 days, whichever comes first. Use tune2fs -c or -i to override.
root@RHELv4u2:~# mkdir /home/project10
root@RHELv4u2:~# mount /dev/vg/lvol0 /home/project10/
root@RHELv4u2:~# df -h | grep proj
/dev/mapper/vg-lvol0 485M 11M 474M 3% /home/project10
```

A logical volume is very similar to a partition, it can be formatted with a file system, and can be mounted so users can access it.

46.4. example: extend a logical volume

A logical volume can be extended without unmounting the file system. Whether or not a volume can be extended depends on the file system it uses. Volumes that are mounted as `vfat` or `ext2` cannot be extended, so in the example here we use the `ext3` file system.

The `fdisk` command shows us newly added scsi-disks that will serve our lvm volume. This volume will then be extended. First, take a look at these disks.

```
[root@RHEL5 ~]# fdisk -l | grep sd[bc]
Disk /dev/sdb doesn't contain a valid partition table
Disk /dev/sdc doesn't contain a valid partition table
Disk /dev/sdb: 1181 MB, 1181115904 bytes
Disk /dev/sdc: 429 MB, 429496320 bytes
```

You already know how to partition a disk, below the first disk is partitioned (in one big primary partition), the second disk is left untouched.

```
[root@RHEL5 ~]# fdisk -l | grep sd[bc]
Disk /dev/sdc doesn't contain a valid partition table
Disk /dev/sdb: 1181 MB, 1181115904 bytes
/dev/sdb1          1          143      1148616    83    Linux
Disk /dev/sdc: 429 MB, 429496320 bytes
```

You also know how to prepare disks for lvm with **pvcreate**, and how to create a volume group with **vgcreate**. This example adds both the partitioned disk and the untouched disk to the volume group named **vg2**.

```
[root@RHEL5 ~]# pvcreate /dev/sdb1
Physical volume "/dev/sdb1" successfully created
[root@RHEL5 ~]# pvcreate /dev/sdc
Physical volume "/dev/sdc" successfully created
[root@RHEL5 ~]# vgcreate vg2 /dev/sdb1 /dev/sdc
Volume group "vg2" successfully created
```

You can use **pvdisk** to verify that both the disk and the partition belong to the volume group.

```
[root@RHEL5 ~]# pvdisk | grep -B1 vg2
PV Name          /dev/sdb1
VG Name          vg2
--
PV Name          /dev/sdc
VG Name          vg2
```

And you are familiar both with the **lvcreate** command to create a small logical volume and the **mke2fs** command to put `ext3` on it.

```
[root@RHEL5 ~]# lvcreate --size 200m vg2
Logical volume "lv0" created
[root@RHEL5 ~]# mke2fs -m20 -j /dev/vg2/lv0
...
```

As you see, we end up with a mounted logical volume that according to **df** is almost 200 megabyte in size.

```
[root@RHEL5 ~]# mkdir /home/resizetest
[root@RHEL5 ~]# mount /dev/vg2/lvol0 /home/resizetest/
[root@RHEL5 ~]# df -h | grep resizetest
194M 5.6M 149M 4% /home/resizetest
```

Extending the volume is easy with **lvextend**.

```
[root@RHEL5 ~]# lvextend -L +100 /dev/vg2/lvol0
Extending logical volume lvol0 to 300.00 MB
Logical volume lvol0 successfully resized
```

But as you can see, there is a small problem: it appears that **df** is not able to display the extended volume in its full size. This is because the filesystem is only set for the size of the volume before the extension was added.

```
[root@RHEL5 ~]# df -h | grep resizetest
194M 5.6M 149M 4% /home/resizetest
```

With **lvdisplay** however we can see that the volume is indeed extended.

```
[root@RHEL5 ~]# lvdisplay /dev/vg2/lvol0 | grep Size
LV Size              300.00 MB
```

To finish the extension, you need **resize2fs** to span the filesystem over the full size of the logical volume.

```
[root@RHEL5 ~]# resize2fs /dev/vg2/lvol0
resize2fs 1.39 (29-May-2006)
Filesystem at /dev/vg2/lvol0 is mounted on /home/resizetest; on-line re\
sizing required
Performing an on-line resize of /dev/vg2/lvol0 to 307200 (1k) blocks.
The filesystem on /dev/vg2/lvol0 is now 307200 blocks long.
```

Congratulations, you just successfully expanded a logical volume.

```
[root@RHEL5 ~]# df -h | grep resizetest
291M 6.1M 225M 3% /home/resizetest
[root@RHEL5 ~]#
```


46.5. example: resize a physical Volume

This is a humble demonstration of how to resize a physical Volume with lvm (after you resize it with fdisk). The demonstration starts with a 100MB partition named /dev/sde1. We used fdisk to create it, and to verify the size.

```
[root@RHEL5 ~]# fdisk -l 2>/dev/null | grep sde1
/dev/sde1          1          100      102384    83    Linux
[root@RHEL5 ~]#
```

Now we can use pvcreate to create the Physical Volume, followed by pvs to verify the creation.

```
[root@RHEL5 ~]# pvcreate /dev/sde1
Physical volume "/dev/sde1" successfully created
[root@RHEL5 ~]# pvs | grep sde1
/dev/sde1          lvm2  --      99.98M  99.98M
[root@RHEL5 ~]#
```

The next step is to use fdisk to enlarge the partition (actually deleting it and then recreating /dev/sde1 with more cylinders).

```
[root@RHEL5 ~]# fdisk /dev/sde

Command (m for help): p

Disk /dev/sde: 858 MB, 858993152 bytes
64 heads, 32 sectors/track, 819 cylinders
Units = cylinders of 2048 * 512 = 1048576 bytes

   Device Boot      Start         End      Blocks    Id  System
/dev/sde1          1           100      102384    83   Linux

Command (m for help): d
Selected partition 1

Command (m for help): n
Command action
   e   extended
   p   primary partition (1-4)
p
Partition number (1-4):
Value out of range.
Partition number (1-4): 1
First cylinder (1-819, default 1):
Using default value 1
Last cylinder or +size or +sizeM or +sizeK (1-819, default 819): 200

Command (m for help): w
The partition table has been altered!

Calling ioctl() to re-read partition table.
Syncing disks.
[root@RHEL5 ~]#
```

When we now use `fdisk` and `pvs` to verify the size of the partition and the Physical Volume, then there is a size difference. LVM is still using the old size.

```
[root@RHEL5 ~]# fdisk -l 2>/dev/null | grep sde1
/dev/sde1      1      200      204784    83  Linux
[root@RHEL5 ~]# pvs | grep sde1
/dev/sde1      lvm2 --    99.98M  99.98M
[root@RHEL5 ~]#
```

Executing `pvresize` on the Physical Volume will make `lvm` aware of the size change of the partition. The correct size can be displayed with `pvs`.

```
[root@RHEL5 ~]# pvresize /dev/sde1
Physical volume "/dev/sde1" changed
1 physical volume(s) resized / 0 physical volume(s) not resized
[root@RHEL5 ~]# pvs | grep sde1
/dev/sde1      lvm2 --   199.98M 199.98M
[root@RHEL5 ~]#
```

46.6. example: mirror a logical volume

We start by creating three physical volumes for lvm. Then we verify the creation and the size with pvs. Three physical disks because lvm uses two disks for the mirror and a third disk for the mirror log!

```
[root@RHEL5 ~]# pvcreate /dev/sdb /dev/sdc /dev/sdd
Physical volume "/dev/sdb" successfully created
Physical volume "/dev/sdc" successfully created
Physical volume "/dev/sdd" successfully created
[root@RHEL5 ~]# pvs
PV          VG          Fmt  Attr  PSize   PFree
/dev/sdb    VG          lvm2  --    409.60M 409.60M
/dev/sdc    VG          lvm2  --    409.60M 409.60M
/dev/sdd    VG          lvm2  --    409.60M 409.60M
```

Then we create the Volume Group and verify again with pvs. Notice how the three physical volumes now belong to vg33, and how the size is rounded down (in steps of the extent size, here 4MB).

```
[root@RHEL5 ~]# vgcreate vg33 /dev/sdb /dev/sdc /dev/sdd
Volume group "vg33" successfully created
[root@RHEL5 ~]# pvs
PV          VG          Fmt  Attr  PSize   PFree
/dev/sda2   VolGroup00  lvm2  a-    15.88G  0
/dev/sdb    vg33        lvm2  a-    408.00M 408.00M
/dev/sdc    vg33        lvm2  a-    408.00M 408.00M
/dev/sdd    vg33        lvm2  a-    408.00M 408.00M
[root@RHEL5 ~]#
```

The last step is to create the Logical Volume with **lvcreate**. Notice the **-m 1** switch to create one mirror. Notice also the change in free space in all three Physical Volumes!

```
[root@RHEL5 ~]# lvcreate --size 300m -n lvmir -m 1 vg33
Logical volume "lvmir" created
[root@RHEL5 ~]# pvs
PV          VG          Fmt  Attr  PSize   PFree
/dev/sda2   VolGroup00  lvm2  a-    15.88G  0
/dev/sdb    vg33        lvm2  a-    408.00M 108.00M
/dev/sdc    vg33        lvm2  a-    408.00M 108.00M
/dev/sdd    vg33        lvm2  a-    408.00M 404.00M
```

You can see the copy status of the mirror with lvs. It currently shows a 100 percent copy.

```
[root@RHEL5 ~]# lvs vg33/lvmir
LV   VG   Attr   LSize   Origin Snap%   Move Log           Copy%
lvmir vg33 mwi-ao 300.00M                                     lvmir_mlog 100.00
```

46.7. example: snapshot a logical volume

A snapshot is a virtual copy of all the data at a point in time on a volume. A snapshot Logical Volume will retain a copy of all changed files of the snapshotted Logical Volume.

The example below creates a snapshot of the bigLV Logical Volume.

```
[root@RHEL5 ~]# lvcreate -L100M -s -n snapLV vg42/bigLV
Logical volume "snapLV" created
[root@RHEL5 ~]#
```

You can see with lvs that the snapshot snapLV is indeed a snapshot of bigLV. Moments after taking the snapshot, there are few changes to bigLV (0.02 percent).

```
[root@RHEL5 ~]# lvs
  LV      VG      Attr      LSize   Origin Snap%  Move Log Copy%
  bigLV   vg42      owi-a- 200.00M
  snapLV  vg42      swi-a- 100.00M bigLV    0.02
[root@RHEL5 ~]#
```

But after using bigLV for a while, more changes are done. This means the snapshot volume has to keep more original data (10.22 percent).

```
[root@RHEL5 ~]# lvs | grep vg42
  bigLV   vg42      owi-ao 200.00M
  snapLV  vg42      swi-a- 100.00M bigLV    10.22
[root@RHEL5 ~]#
```

You can now use regular backup tools (dump, tar, cpio, ...) to take a backup of the snapshot Logical Volume. This backup will contain all data as it existed on bigLV at the time the snapshot was taken. When the backup is done, you can remove the snapshot.

```
[root@RHEL5 ~]# lvremove vg42/snapLV
Do you really want to remove active logical volume "snapLV"? [y/n]: y
Logical volume "snapLV" successfully removed
[root@RHEL5 ~]#
```

46.8. verifying existing physical volumes

46.8.1. lvmdiskscan

To get a list of block devices that can be used with LVM, use **lvmdiskscan**. The example below uses **grep** to limit the result to SCSI devices.

```
[root@RHEL5 ~]# lvmdiskscan | grep sd
/dev/sda1      [      101.94 MB]
/dev/sda2      [      15.90 GB] LVM physical volume
/dev/sdb       [      409.60 MB]
/dev/sdc       [      409.60 MB]
/dev/sdd       [      409.60 MB] LVM physical volume
/dev/sde1      [       95.98 MB]
/dev/sde5      [      191.98 MB]
/dev/sdf       [      819.20 MB] LVM physical volume
/dev/sdg1      [      818.98 MB]
```

46.8.2. pvs

The easiest way to verify whether devices are known to lvm is with the **pvs** command. The screenshot below shows that only `/dev/sda2` is currently known for use with LVM. It shows that `/dev/sda2` is part of `VolGroup00` and is almost 16GB in size. It also shows `/dev/sdc` and `/dev/sdd` as part of `vg33`. The device `/dev/sdb` is known to lvm, but not linked to any Volume Group.

```
[root@RHEL5 ~]# pvs
PV          VG          Fmt Attr PSize  PFree
/dev/sda2   VolGroup00  lvm2 a-   15.88G    0
/dev/sdb    lvm2  --   409.60M  409.60M
/dev/sdc    vg33   a-   408.00M  408.00M
/dev/sdd    vg33   a-   408.00M  408.00M
```

46.8.3. pvscan

The **pvscan** command will scan all disks for existing Physical Volumes. The information is similar to **pvs**, plus you get a line with total sizes.

```
[root@RHEL5 ~]# pvscan
PV /dev/sdc    VG vg33          lvm2 [408.00 MB / 408.00 MB free]
PV /dev/sdd    VG vg33          lvm2 [408.00 MB / 408.00 MB free]
PV /dev/sda2   VG VolGroup00    lvm2 [15.88 GB / 0    free]
PV /dev/sdb    lvm2             lvm2 [409.60 MB]
Total: 4 [17.07 GB] / in use: 3 [16.67 GB] / in no VG: 1 [409.60 MB]
```

46.8.4. pvdisplay

Use **pvdisplay** to get more information about physical volumes. You can also use **pvdisplay** without an argument to display information about all physical (lvm) volumes.

```
[root@RHEL5 ~]# pvdisplay /dev/sda2
--- Physical volume ---
PV Name                /dev/sda2
VG Name                VolGroup00
PV Size                15.90 GB / not usable 20.79 MB
Allocatable            yes (but full)
PE Size (KByte)        32768
Total PE               508
Free PE                0
Allocated PE           508
PV UUID                TobYfp-Ggg0-Rf8r-xtLd-5XgN-RSPc-8vkTHD

[root@RHEL5 ~]#
```

46.9. verifying existing volume groups

46.9.1. vgs

Similar to **pvs** is the use of **vgs** to display a quick overview of all volume groups. There is only one volume group in the screenshot below, it is named VolGroup00 and is almost 16GB in size.

```
[root@RHEL5 ~]# vgs
VG          #PV #LV #SN Attr   VSize  VFree
VolGroup00   1   2   0 wz--n- 15.88G    0
[root@RHEL5 ~]#
```

46.9.2. vgscan

The **vgscan** command will scan all disks for existing Volume Groups. It will also update the **/etc/lvm/.cache** file. This file contains a list of all current lvm devices.

```
[root@RHEL5 ~]# vgscan
Reading all physical volumes. This may take a while...
Found volume group "VolGroup00" using metadata type lvm2
[root@RHEL5 ~]#
```

LVM will run the **vgscan** automatically at boot-up, so if you add hot swap devices, then you will need to run **vgscan** to update **/etc/lvm/.cache** with the new devices.

46.9.3. vgdisplay

The **vgdisplay** command will give you more detailed information about a volume group (or about all volume groups if you omit the argument).

```
[root@RHEL5 ~]# vgdisplay VolGroup00
--- Volume group ---
VG Name                VolGroup00
System ID
Format                 lvm2
Metadata Areas         1
Metadata Sequence No   3
VG Access               read/write
VG Status               resizable
MAX LV                 0
Cur LV                 2
Open LV                 2
Max PV                 0
Cur PV                 1
Act PV                 1
VG Size                 15.88 GB
PE Size                 32.00 MB
Total PE                508
Alloc PE / Size         508 / 15.88 GB
Free PE / Size           0 / 0
VG UUID                 qsXvJb-71qV-9l7U-ishX-FobM-qptE-VXmKIg
[root@RHEL5 ~]#
```

46.10. verifying existing logical volumes

46.10.1. lvs

Use **lvs** for a quick look at all existing logical volumes. Below you can see two logical volumes named LogVol00 and LogVol01.

```
[root@RHEL5 ~]# lvs
LV      VG      Attr   LSize   Origin Snap%   Move Log Copy%
LogVol00 VolGroup00 -wi-ao 14.88G
LogVol01 VolGroup00 -wi-ao  1.00G
[root@RHEL5 ~]#
```

46.10.2. lvscan

The **lvscan** command will scan all disks for existing Logical Volumes.

```
[root@RHEL5 ~]# lvscan
ACTIVE          '/dev/VolGroup00/LogVol00' [14.88 GB] inherit
ACTIVE          '/dev/VolGroup00/LogVol01' [1.00 GB] inherit
[root@RHEL5 ~]#
```

46.10.3. lvdisplay

More detailed information about logical volumes is available through the **lvdisplay(1)** command.

```
[root@RHEL5 ~]# lvdisplay VolGroup00/LogVol01
--- Logical volume ---
LV Name                /dev/VolGroup00/LogVol01
VG Name                VolGroup00
LV UUID                RnTGK6-xWsi-t530-ksJx-7cax-co5c-A1KlDp
LV Write Access        read/write
LV Status              available
# open                 1
LV Size                1.00 GB
Current LE             32
Segments               1
Allocation             inherit
Read ahead sectors     0
Block device           253:1
[root@RHEL5 ~]#
```


46.11. manage physical volumes

46.11.1. pvcreate

Use the **pvcreate** command to add devices to lvm. This example shows how to add a disk (or hardware RAID device) to lvm.

```
[root@RHEL5 ~]# pvcreate /dev/sdb
Physical volume "/dev/sdb" successfully created
[root@RHEL5 ~]#
```

This example shows how to add a partition to lvm.

```
[root@RHEL5 ~]# pvcreate /dev/sdc1
Physical volume "/dev/sdc1" successfully created
[root@RHEL5 ~]#
```

You can also add multiple disks or partitions as target to pvcreate. This example adds three disks to lvm.

```
[root@RHEL5 ~]# pvcreate /dev/sde /dev/sdf /dev/sdg
Physical volume "/dev/sde" successfully created
Physical volume "/dev/sdf" successfully created
Physical volume "/dev/sdg" successfully created
[root@RHEL5 ~]#
```

46.11.2. pvremove

Use the **pvremove** command to remove physical volumes from lvm. The devices may not be in use.

```
[root@RHEL5 ~]# pvremove /dev/sde /dev/sdf /dev/sdg
Labels on physical volume "/dev/sde" successfully wiped
Labels on physical volume "/dev/sdf" successfully wiped
Labels on physical volume "/dev/sdg" successfully wiped
[root@RHEL5 ~]#
```

46.11.3. pvresize

When you used fdisk to resize a partition on a disk, then you must use **pvresize** to make lvm recognize the new size of the physical volume that represents this partition.

```
[root@RHEL5 ~]# pvresize /dev/sde1
Physical volume "/dev/sde1" changed
1 physical volume(s) resized / 0 physical volume(s) not resized
```

46.11.4. pvchange

With **pvchange** you can prevent the allocation of a Physical Volume in a new Volume Group or Logical Volume. This can be useful if you plan to remove a Physical Volume.

```
[root@RHEL5 ~]# pvchange -xn /dev/sdd
Physical volume "/dev/sdd" changed
1 physical volume changed / 0 physical volumes not changed
[root@RHEL5 ~]#
```

To revert your previous decision, this example shows you how to re-enable the Physical Volume to allow allocation.

```
[root@RHEL5 ~]# pvchange -xy /dev/sdd
Physical volume "/dev/sdd" changed
1 physical volume changed / 0 physical volumes not changed
[root@RHEL5 ~]#
```

46.11.5. pvmove

With **pvmove** you can move Logical Volumes from within a Volume Group to another Physical Volume. This must be done before removing a Physical Volume.

```
[root@RHEL5 ~]# pvs | grep vg1
/dev/sdf   vg1      lvm2 a-   816.00M    0
/dev/sdg   vg1      lvm2 a-   816.00M 816.00M
[root@RHEL5 ~]# pvmove /dev/sdf
/dev/sdf: Moved: 70.1%
/dev/sdf: Moved: 100.0%
[root@RHEL5 ~]# pvs | grep vg1
/dev/sdf   vg1      lvm2 a-   816.00M 816.00M
/dev/sdg   vg1      lvm2 a-   816.00M    0
```

46.12. manage volume groups

46.12.1. vgcreate

Use the **vgcreate** command to create a volume group. You can immediately name all the physical volumes that span the volume group.

```
[root@RHEL5 ~]# vgcreate vg42 /dev/sde /dev/sdf
Volume group "vg42" successfully created
[root@RHEL5 ~]#
```

46.12.2. vgextend

Use the **vgextend** command to extend an existing volume group with a physical volume.

```
[root@RHEL5 ~]# vgextend vg42 /dev/sdg
Volume group "vg42" successfully extended
[root@RHEL5 ~]#
```

46.12.3. vgremove

Use the **vgremove** command to remove volume groups from lvm. The volume groups may not be in use.

```
[root@RHEL5 ~]# vgremove vg42
Volume group "vg42" successfully removed
[root@RHEL5 ~]#
```

46.12.4. vgreduce

Use the **vgreduce** command to remove a Physical Volume from the Volume Group.

The following example adds Physical Volume /dev/sdg to the vg1 Volume Group using **vgextend**. And then removes it again using **vgreduce**.

```
[root@RHEL5 ~]# pvs | grep sdg
/dev/sdg          lvm2 -- 819.20M 819.20M
[root@RHEL5 ~]# vgextend vg1 /dev/sdg
Volume group "vg1" successfully extended
[root@RHEL5 ~]# pvs | grep sdg
/dev/sdg  vg1      lvm2 a- 816.00M 816.00M
[root@RHEL5 ~]# vgreduce vg1 /dev/sdg
Removed "/dev/sdg" from volume group "vg1"
[root@RHEL5 ~]# pvs | grep sdg
/dev/sdg          lvm2 -- 819.20M 819.20M
```

46.12.5. vgchange

Use the **vgchange** command to change parameters of a Volume Group.

This example shows how to prevent Physical Volumes from being added or removed to the Volume Group `vg1`.

```
[root@RHEL5 ~]# vgchange -xn vg1
Volume group "vg1" successfully changed
[root@RHEL5 ~]# vgextend vg1 /dev/sdg
Volume group vg1 is not resizable.
```

You can also use `vgchange` to change most other properties of a Volume Group. This example changes the maximum number of Logical Volumes and maximum number of Physical Volumes that `vg1` can serve.

```
[root@RHEL5 ~]# vgdisplay vg1 | grep -i max
MAX LV          0
Max PV          0
[root@RHEL5 ~]# vgchange -l16 vg1
Volume group "vg1" successfully changed
[root@RHEL5 ~]# vgchange -p8 vg1
Volume group "vg1" successfully changed
[root@RHEL5 ~]# vgdisplay vg1 | grep -i max
MAX LV          16
Max PV          8
```

46.12.6. vgmerge

Merging two Volume Groups into one is done with **vgmerge**. The following example merges `vg2` into `vg1`, keeping all the properties of `vg1`.

```
[root@RHEL5 ~]# vgmerge vg1 vg2
Volume group "vg2" successfully merged into "vg1"
[root@RHEL5 ~]#
```

46.13. manage logical volumes

46.13.1. lvcreate

Use the **lvcreate** command to create Logical Volumes in a Volume Group. This example creates an 8GB Logical Volume in Volume Group vg42.

```
[root@RHEL5 ~]# lvcreate -L5G vg42
Logical volume "lvol0" created
[root@RHEL5 ~]#
```

As you can see, lvm automatically names the Logical Volume **lvol0**. The next example creates a 200MB Logical Volume named MyLV in Volume Group vg42.

```
[root@RHEL5 ~]# lvcreate -L200M -nMyLV vg42
Logical volume "MyLV" created
[root@RHEL5 ~]#
```

The next example does the same thing, but with different syntax.

```
[root@RHEL5 ~]# lvcreate --size 200M -n MyLV vg42
Logical volume "MyLV" created
[root@RHEL5 ~]#
```

This example creates a Logical Volume that occupies 10 percent of the Volume Group.

```
[root@RHEL5 ~]# lvcreate -l 10%VG -n MyLV2 vg42
Logical volume "MyLV2" created
[root@RHEL5 ~]#
```

This example creates a Logical Volume that occupies 30 percent of the remaining free space in the Volume Group.

```
[root@RHEL5 ~]# lvcreate -l 30%FREE -n MyLV3 vg42
Logical volume "MyLV3" created
[root@RHEL5 ~]#
```

46.13.2. lvremove

Use the **lvremove** command to remove Logical Volumes from a Volume Group. Removing a Logical Volume requires the name of the Volume Group.

```
[root@RHEL5 ~]# lvremove vg42/MyLV
Do you really want to remove active logical volume "MyLV"? [y/n]: y
Logical volume "MyLV" successfully removed
[root@RHEL5 ~]#
```

Removing multiple Logical Volumes will request confirmation for each individual volume.

```
[root@RHEL5 ~]# lvremove vg42/MyLV vg42/MyLV2 vg42/MyLV3
Do you really want to remove active logical volume "MyLV"? [y/n]: y
Logical volume "MyLV" successfully removed
Do you really want to remove active logical volume "MyLV2"? [y/n]: y
Logical volume "MyLV2" successfully removed
Do you really want to remove active logical volume "MyLV3"? [y/n]: y
Logical volume "MyLV3" successfully removed
[root@RHEL5 ~]#
```

46.13.3. lvextend

Extending the volume is easy with **lvextend**. This example extends a 200MB Logical Volume with 100 MB.

```
[root@RHEL5 ~]# lvdisplay /dev/vg2/lvol0 | grep Size
LV Size                200.00 MB
[root@RHEL5 ~]# lvextend -L +100 /dev/vg2/lvol0
Extending logical volume lvol0 to 300.00 MB
Logical volume lvol0 successfully resized
[root@RHEL5 ~]# lvdisplay /dev/vg2/lvol0 | grep Size
LV Size                300.00 MB
```

The next example creates a 100MB Logical Volume, and then extends it to 500MB.

```
[root@RHEL5 ~]# lvcreate --size 100M -n extLV vg42
Logical volume "extLV" created
[root@RHEL5 ~]# lvextend -L 500M vg42/extLV
Extending logical volume extLV to 500.00 MB
Logical volume extLV successfully resized
[root@RHEL5 ~]#
```

This example doubles the size of a Logical Volume.

```
[root@RHEL5 ~]# lvextend -l+100%LV vg42/extLV
Extending logical volume extLV to 1000.00 MB
Logical volume extLV successfully resized
[root@RHEL5 ~]#
```

46.13.4. lvrename

Renaming a Logical Volume is done with **lvrename**. This example renames extLV to bigLV in the vg42 Volume Group.

```
[root@RHEL5 ~]# lvrename vg42/extLV vg42/bigLV
Renamed "extLV" to "bigLV" in volume group "vg42"
[root@RHEL5 ~]#
```

46.14. practice : lvm

1. Create a volume group that contains a complete disk and a partition on another disk.
2. Create two logical volumes (a small one and a bigger one) in this volume group. Format them with ext3, mount them and copy some files to them.
3. Verify usage with fdisk, mount, pvs, vgs, lvs, pvdisplay, vgdisplay, lvdisplay and df. Does fdisk give you any information about lvm?
4. Enlarge the small logical volume by 50 percent, and verify your work!
5. Take a look at other commands that start with vg* , pv* or lv*.
6. Create a mirror and a striped Logical Volume.
7. Convert a linear logical volume to a mirror.
8. Convert a mirror logical volume to a linear.
9. Create a snapshot of a Logical Volume, take a backup of the snapshot. Then delete some files on the Logical Volume, then restore your backup.
10. Move your volume group to another disk (keep the Logical Volumes mounted).
11. If time permits, split a Volume Group with vgsplit, then merge it again with vgmerge.

46.15. solution : lvm

1. Create a volume group that contains a complete disk and a partition on another disk.

step 1: select disks:

```
root@rhel65:~# fdisk -l | grep Disk
Disk /dev/sda: 8589 MB, 8589934592 bytes
Disk identifier: 0x00055ca0
Disk /dev/sdb: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdc: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
...
```

I choose /dev/sdb and /dev/sdc for now.

step 2: partition /dev/sdc

```
root@rhel65:~# fdisk /dev/sdc
Device contains neither a valid DOS partition table, nor Sun, SGI or OSF disk\
label
Building a new DOS disklabel with disk identifier 0x94c0e5d5.
Changes will remain in memory only, until you decide to write them.
After that, of course, the previous content won't be recoverable.

Warning: invalid flag 0x0000 of partition table 4 will be corrected by w(rite)

WARNING: DOS-compatible mode is deprecated. It's strongly recommended to
switch off the mode (command 'c') and change display units to
sectors (command 'u').

Command (m for help): n
Command action
   e   extended
   p   primary partition (1-4)
p
Partition number (1-4): 1
First cylinder (1-130, default 1):
Using default value 1
Last cylinder, +cylinders or +size{K,M,G} (1-130, default 130):
Using default value 130

Command (m for help): w
The partition table has been altered!

Calling ioctl() to re-read partition table.
Syncing disks.
```

step 3: pvcreate and vgcreate

```
root@rhel65:~# pvcreate /dev/sdb /dev/sdc1
Physical volume "/dev/sdb" successfully created
Physical volume "/dev/sdc1" successfully created
root@rhel65:~# vgcreate VG42 /dev/sdb /dev/sdc1
Volume group "VG42" successfully created
```


2. Create two logical volumes (a small one and a bigger one) in this volume group. Format them with ext3, mount them and copy some files to them.

```
root@rhel65:~# lvcreate --size 200m --name LVsmall VG42
Logical volume "LVsmall" created
root@rhel65:~# lvcreate --size 600m --name LVbig VG42
Logical volume "LVbig" created
root@rhel65:~# ls -l /dev/mapper/VG42-LVsmall
lrwxrwxrwx. 1 root root 7 Apr 20 20:41 /dev/mapper/VG42-LVsmall -> ../dm-2
root@rhel65:~# ls -l /dev/VG42/LVsmall
lrwxrwxrwx. 1 root root 7 Apr 20 20:41 /dev/VG42/LVsmall -> ../dm-2
root@rhel65:~# ls -l /dev/dm-2
brw-rw----. 1 root disk 253, 2 Apr 20 20:41 /dev/dm-2
```

```
root@rhel65:~# mkfs.ext3 /dev/mapper/VG42-LVsmall
mke2fs 1.41.12 (17-May-2010)
Filesystem label=
OS type: Linux
Block size=1024 (log=0)
Fragment size=1024 (log=0)
Stride=0 blocks, Stripe width=0 blocks
51200 inodes, 204800 blocks
10240 blocks (5.00%) reserved for the super user
First data block=1
Maximum filesystem blocks=67371008
25 block groups
8192 blocks per group, 8192 fragments per group
2048 inodes per group
Superblock backups stored on blocks:
 8193, 24577, 40961, 57345, 73729

Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

This filesystem will be automatically checked every 39 mounts or
180 days, whichever comes first. Use tune2fs -c or -i to override.
```

```
root@rhel65:~# mkfs.ext3 /dev/VG42/LVbig
mke2fs 1.41.12 (17-May-2010)
Filesystem label=
OS type: Linux
Block size=4096 (log=2)
Fragment size=4096 (log=2)
Stride=0 blocks, Stripe width=0 blocks
38400 inodes, 153600 blocks
7680 blocks (5.00%) reserved for the super user
First data block=0
Maximum filesystem blocks=159383552
5 block groups
32768 blocks per group, 32768 fragments per group
7680 inodes per group
Superblock backups stored on blocks:
 32768, 98304

Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

This filesystem will be automatically checked every 25 mounts or
180 days, whichever comes first. Use tune2fs -c or -i to override.
```

The mounting and copying of files.

```
root@rhel65:~# mkdir /srv/LVsmall
root@rhel65:~# mkdir /srv/LVbig
root@rhel65:~# mount /dev/mapper/VG42-LVsmall /srv/LVsmall
root@rhel65:~# mount /dev/VG42/LVbig /srv/LVbig
root@rhel65:~# cp -r /etc /srv/LVsmall/
root@rhel65:~# cp -r /var/log /srv/LVbig/
```

3. Verify usage with `fdisk`, `mount`, `pvs`, `vgs`, `lvs`, `pvdisplay`, `vgdisplay`, `lvdisplay` and `df`. Does `fdisk` give you any information about `lvm`?

Run all those commands (only two are shown below), then answer 'no'.

```
root@rhel65:~# df -h
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/VolGroup-lv_root 6.7G    1.4G    5.0G   21% /
tmpfs                      246M         0   246M    0% /dev/shm
/dev/sda1                  485M     77M    383M   17% /boot
/dev/mapper/VG42-LVsmall    194M     30M    154M   17% /srv/LVsmall
/dev/mapper/VG42-LVbig      591M     20M    541M    4% /srv/LVbig
root@rhel65:~# mount | grep VG42
/dev/mapper/VG42-LVsmall on /srv/LVsmall type ext3 (rw)
/dev/mapper/VG42-LVbig on /srv/LVbig type ext3 (rw)
```

4. Enlarge the small logical volume by 50 percent, and verify your work!

```
root@rhel65:~# lvextend VG42/LVsmall -l+50%LV
  Extending logical volume LVsmall to 300.00 MiB
  Logical volume LVsmall successfully resized
root@rhel65:~# resize2fs /dev/mapper/VG42-LVsmall
resize2fs 1.41.12 (17-May-2010)
Filesystem at /dev/mapper/VG42-LVsmall is mounted on /srv/LVsmall; on-line res\
izing required
old desc_blocks = 1, new_desc_blocks = 2
Performing an on-line resize of /dev/mapper/VG42-LVsmall to 307200 (1k) blocks.
The filesystem on /dev/mapper/VG42-LVsmall is now 307200 blocks long.

root@rhel65:~# df -h | grep small
/dev/mapper/VG42-LVsmall    291M     31M    246M   12% /srv/LVsmall
root@rhel65:~#
```

5. Take a look at other commands that start with `vg*` , `pv*` or `lv*`.
6. Create a mirror and a striped Logical Volume.
7. Convert a linear logical volume to a mirror.
8. Convert a mirror logical volume to a linear.
9. Create a snapshot of a Logical Volume, take a backup of the snapshot. Then delete some files on the Logical Volume, then restore your backup.
10. Move your volume group to another disk (keep the Logical Volumes mounted).
11. If time permits, split a Volume Group with `vgsplit`, then merge it again with `vgmerge`.

Chapter 47. iSCSI devices

This chapter teaches you how to setup an **iSCSI target server** and an **iSCSI initiator client**.

47.1. iSCSI terminology

iSCSI is a protocol that enables SCSI over IP. This means that you can have local SCSI devices (like `/dev/sdb`) without having the storage hardware in the local computer.

The computer holding the physical storage hardware is called the **iSCSI Target**. Each individual addressable iSCSI device on the target server will get a **LUN number**.

The iSCSI client computer that is connecting to the Target server is called an **Initiator**. An initiator will send SCSI commands over IP instead of directly to the hardware. The Initiator will connect to the Target.

47.2. iSCSI Target in RHEL/CentOS

This section will describe iSCSI Target setup on RHEL6, RHEL7 and CentOS.

Start with installing the **iSCSI Target** package.

```
yum install scsi-target-utils
```

We configure three local disks in `/etc/tgt/targets.conf` to become three LUN's.

```
<target iqn.2008-09.com.example:server.target2>
  direct-store /dev/sdb
  direct-store /dev/sdc
  direct-store /dev/sdd
  incominguser paul hunter2
</target>
```

Restart the service.

```
[root@centos65 ~]# service tgtd start
Starting SCSI target daemon: [ OK ]
```

The standard local port for iSCSI Target is 3260, in case of doubt you can verify this with **netstat**.

```
[root@server1 tgt]# netstat -ntpl | grep tgt
tcp    0      0 0.0.0.0:3260        0.0.0.0:*          LISTEN      1670/tgtd
tcp    0      0 :::3260            :::*                LISTEN      1670/tgtd
```

The **tgt-admin -s** command should now give you a nice overview of the three LUN's (and also LUN 0 for the controller).

```
[root@server1 tgt]# tgt-admin -s
Target 1: iqn.2014-04.be.linux-training:server1.target1
System information:
  Driver: iscsi
  State: ready
I_T nexus information:
LUN information:
  LUN: 0
    Type: controller
    SCSI ID: IET      00010000
    SCSI SN: beaf10
    Size: 0 MB, Block size: 1
    Online: Yes
    Removable media: No
    Prevent removal: No
    Readonly: No
    Backing store type: null
    Backing store path: None
    Backing store flags:
  LUN: 1
    Type: disk
    SCSI ID: IET      00010001
    SCSI SN: VB9f23197b-af6cfb60
    Size: 1074 MB, Block size: 512
    Online: Yes
    Removable media: No
    Prevent removal: No
    Readonly: No
    Backing store type: rdwr
    Backing store path: /dev/sdb
    Backing store flags:
  LUN: 2
    Type: disk
    SCSI ID: IET      00010002
    SCSI SN: VB8f554351-a1410828
    Size: 1074 MB, Block size: 512
    Online: Yes
    Removable media: No
    Prevent removal: No
    Readonly: No
    Backing store type: rdwr
    Backing store path: /dev/sdc
    Backing store flags:
  LUN: 3
    Type: disk
    SCSI ID: IET      00010003
    SCSI SN: VB1035d2f0-7ae90b49
    Size: 1074 MB, Block size: 512
    Online: Yes
    Removable media: No
    Prevent removal: No
    Readonly: No
    Backing store type: rdwr
    Backing store path: /dev/sdd
    Backing store flags:
Account information:
ACL information:
  ALL
```

47.3. iSCSI Initiator in RHEL/CentOS

This section will describe iSCSI Initiator setup on RHEL6, RHEL7 and CentOS.

Start with installing the **iSCSI Initiator** package.

```
[root@server2 ~]# yum install iscsi-initiator-utils
```

Then ask the **iSCSI target server** to send you the target names.

```
[root@server2 ~]# iscsiadm -m discovery -t sendtargets -p 192.168.1.95:3260
Starting iscsid: [ OK ]
192.168.1.95:3260,1 iqn.2014-04.be.linux-training:centos65.target1
```

We received **iqn.2014-04.be.linux-training:centos65.target1**.

We use this iqn to configure the username and the password (paul and hunter2) that we set on the target server.

```
[root@server2 iscsi]# iscsiadm -m node --targetname iqn.2014-04.be.linux-tra\
ining:centos65.target1 --portal "192.168.1.95:3260" --op=update --name node.\
session.auth.username --value=paul
[root@server2 iscsi]# iscsiadm -m node --targetname iqn.2014-04.be.linux-tra\
ining:centos65.target1 --portal "192.168.1.95:3260" --op=update --name node.\
session.auth.password --value=hunter2
[root@server2 iscsi]# iscsiadm -m node --targetname iqn.2014-04.be.linux-tra\
ining:centos65.target1 --portal "192.168.1.95:3260" --op=update --name node.\
session.auth.authmethod --value=CHAP
```

RHEL and CentOS will store these in **/var/lib/iscsi/nodes/**.

```
[root@server2 iscsi]# grep auth /var/lib/iscsi/nodes/iqn.2014-04.be.linux-tr\
aining:centos65.target1/192.168.1.95:3260/1/default
node.session.auth.authmethod = CHAP
node.session.auth.username = paul
node.session.auth.password = hunter2
node.conn[0].timeo.auth_timeout = 45
[root@server2 iscsi]#
```

A restart of the **iscsi** service will add three new devices to our system.

```
[root@server2 iscsi]# fdisk -l | grep Disk
Disk /dev/sda: 42.9 GB, 42949672960 bytes
Disk identifier: 0x0004f229
Disk /dev/sdb: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdc: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdd: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sde: 2147 MB, 2147483648 bytes
Disk identifier: 0x00000000
Disk /dev/sdf: 2147 MB, 2147483648 bytes
Disk identifier: 0x00000000
Disk /dev/sdg: 2147 MB, 2147483648 bytes
Disk identifier: 0x00000000
Disk /dev/mapper/VolGroup-lv_root: 41.4 GB, 41448112128 bytes
Disk identifier: 0x00000000
Disk /dev/mapper/VolGroup-lv_swap: 973 MB, 973078528 bytes
Disk identifier: 0x00000000
[root@server2 iscsi]# service iscsi restart
Stopping iscsi: [ OK ]
Starting iscsi: [ OK ]
[root@server2 iscsi]# fdisk -l | grep Disk
Disk /dev/sda: 42.9 GB, 42949672960 bytes
Disk identifier: 0x0004f229
Disk /dev/sdb: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdc: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdd: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sde: 2147 MB, 2147483648 bytes
Disk identifier: 0x00000000
Disk /dev/sdf: 2147 MB, 2147483648 bytes
Disk identifier: 0x00000000
Disk /dev/sdg: 2147 MB, 2147483648 bytes
Disk identifier: 0x00000000
Disk /dev/mapper/VolGroup-lv_root: 41.4 GB, 41448112128 bytes
Disk identifier: 0x00000000
Disk /dev/mapper/VolGroup-lv_swap: 973 MB, 973078528 bytes
Disk identifier: 0x00000000
Disk /dev/sdh: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdi: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdj: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
```

You can verify iscsi status with:

```
service iscsi status
```


47.4. iSCSI target on Debian

Installing the software for the target server requires **iscsitarget** on Ubuntu and Debian, and an extra **iscsitarget-dkms** for the kernel modules only on Debian.

```
root@debby6:~# aptitude install iscsitarget
The following NEW packages will be installed:
  iscsitarget
0 packages upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 69.4 kB of archives. After unpacking 262 kB will be used.
Get:1 http://ftp.belnet.be/debian/ squeeze/main iscsitarget i386 1.4.20.2-1\
[69.4 kB]
Fetched 69.4 kB in 0s (415 kB/s)
Selecting previously deselected package iscsitarget.
(Reading database ... 36441 files and directories currently installed.)
Unpacking iscsitarget (from .../iscsitarget_1.4.20.2-1_i386.deb) ...
Processing triggers for man-db ...
Setting up iscsitarget (1.4.20.2-1) ...
iscsitarget not enabled in "/etc/default/iscsitarget", not starting...(warning).
```

On Debian 6 you will also need **aptitude install iscsitarget-dkms** for the kernel modules, on Debian 5 this is **aptitude install iscsitarget-modules-`uname -a`**. Ubuntu includes the kernel modules in the main package.

The iSCSI target server is disabled by default, so we enable it.

```
root@debby6:~# cat /etc/default/iscsitarget
ISCSITARGET_ENABLE=false
root@debby6:~# vi /etc/default/iscsitarget
root@debby6:~# cat /etc/default/iscsitarget
ISCSITARGET_ENABLE=true
```

47.5. iSCSI target setup with dd files

You can use LVM volumes (/dev/md0/lvol0), physical partitions (/dev/sda), raid devices (/dev/md0) or just plain files for storage. In this demo, we use files created with **dd**.

This screenshot shows how to create three small files (100MB, 200MB and 300MB).

```
root@debby6:~# mkdir /iscsi
root@debby6:~# dd if=/dev/zero of=/iscsi/lun1.img bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB) copied, 0.315825 s, 332 MB/s
root@debby6:~# dd if=/dev/zero of=/iscsi/lun2.img bs=1M count=200
200+0 records in
200+0 records out
209715200 bytes (210 MB) copied, 1.08342 s, 194 MB/s
root@debby6:~# dd if=/dev/zero of=/iscsi/lun3.img bs=1M count=300
300+0 records in
300+0 records out
314572800 bytes (315 MB) copied, 1.36209 s, 231 MB/s
```

We need to declare these three files as iSCSI targets in **/etc/iet/ietd.conf** (used to be **/etc/ietd.conf**).

```
root@debby6:/etc/iet# cp ietd.conf ietd.conf.original
root@debby6:/etc/iet# > ietd.conf
root@debby6:/etc/iet# vi ietd.conf
root@debby6:/etc/iet# cat ietd.conf
Target iqn.2010-02.be.linux-training:storage.lun1
IncomingUser isuser hunter2
OutgoingUser
Lun 0 Path=/iscsi/lun1.img,Type=fileio
Alias LUN1

Target iqn.2010-02.be.linux-training:storage.lun2
IncomingUser isuser hunter2
OutgoingUser
Lun 0 Path=/iscsi/lun2.img,Type=fileio
Alias LUN2

Target iqn.2010-02.be.linux-training:storage.lun3
IncomingUser isuser hunter2
OutgoingUser
Lun 0 Path=/iscsi/lun3.img,Type=fileio
Alias LUN3
```

We also need to add our devices to the **/etc/initiators.allow** file.

```
root@debby6:/etc/iet# cp initiators.allow initiators.allow.original
root@debby6:/etc/iet# > initiators.allow
root@debby6:/etc/iet# vi initiators.allow
root@debby6:/etc/iet# cat initiators.allow
iqn.2010-02.be.linux-training:storage.lun1
iqn.2010-02.be.linux-training:storage.lun2
iqn.2010-02.be.linux-training:storage.lun3
```

Time to start the server now:

```
root@debby6:/etc/iet# /etc/init.d/iscsitarget start
Starting iSCSI enterprise target service:.
root@debby6:/etc/iet#
```

Verify activation of the storage devices in **/proc/net/iet**:

```
root@debby6:/etc/iet# cat /proc/net/iet/volume
tid:3 name:iqn.2010-02.be.linux-training:storage.lun3
  lun:0 state:0 iotype:fileio iomode:wt blocks:614400 blocksize:\
512 path:/iscsi/lun3.img
tid:2 name:iqn.2010-02.be.linux-training:storage.lun2
  lun:0 state:0 iotype:fileio iomode:wt blocks:409600 blocksize:\
512 path:/iscsi/lun2.img
tid:1 name:iqn.2010-02.be.linux-training:storage.lun1
  lun:0 state:0 iotype:fileio iomode:wt blocks:204800 blocksize:\
512 path:/iscsi/lun1.img
root@debby6:/etc/iet# cat /proc/net/iet/session
tid:3 name:iqn.2010-02.be.linux-training:storage.lun3
tid:2 name:iqn.2010-02.be.linux-training:storage.lun2
tid:1 name:iqn.2010-02.be.linux-training:storage.lun1
```

47.6. iSCSI initiator on ubuntu

First we install the iSCSI client software (on another computer than the target).

```
root@ubu1104:~# aptitude install open-iscsi
Reading package lists... Done
Building dependency tree
Reading state information... Done
Reading extended state information
Initializing package states... Done
The following NEW packages will be installed:
  open-iscsi open-iscsi-utils{a}
```

Then we set the iSCSI client to start automatically.

```
root@ubu1104:/etc/iscsi# cp iscsid.conf iscsid.conf.original
root@ubu1104:/etc/iscsi# vi iscsid.conf
root@ubu1104:/etc/iscsi# grep ^node.startup iscsid.conf
node.startup = automatic
```

Or you could start it manually.

```
root@ubu1104:/etc/iscsi/nodes# /etc/init.d/open-iscsi start
* Starting iSCSI initiator service iscsid          [ OK ]
* Setting up iSCSI targets                        [ OK ]
root@ubu1104:/etc/iscsi/nodes#
```

Now we can connect to the Target server and use **iscsiadm** to discover the devices it offers:

```
root@ubu1104:/etc/iscsi# iscsiadm -m discovery -t st -p 192.168.1.31
192.168.1.31:3260,1 iqn.2010-02.be.linux-training:storage.lun2
192.168.1.31:3260,1 iqn.2010-02.be.linux-training:storage.lun1
192.168.1.31:3260,1 iqn.2010-02.be.linux-training:storage.lun3
```

We can use the same **iscsiadm** to edit the files in **/etc/iscsi/nodes/**.

```
root@ubu1104:/etc/iscsi# iscsiadm -m node --targetname "iqn.2010-02.be.linu\
x-training:storage.lun1" --portal "192.168.1.31:3260" --op=update --name no\
de.session.auth.authmethod --value=CHAP
root@ubu1104:/etc/iscsi# iscsiadm -m node --targetname "iqn.2010-02.be.linu\
x-training:storage.lun1" --portal "192.168.1.31:3260" --op=update --name no\
de.session.auth.username --value=isuser
root@ubu1104:/etc/iscsi# iscsiadm -m node --targetname "iqn.2010-02.be.linu\
x-training:storage.lun1" --portal "192.168.1.31:3260" --op=update --name no\
de.session.auth.password --value=hunter2
```

Repeat the above for the other two devices.

Restart the initiator service to log in to the target.

```
root@ubu1104:/etc/iscsi/nodes# /etc/init.d/open-iscsi restart
* Disconnecting iSCSI targets [ OK ]
* Stopping iSCSI initiator service [ OK ]
* Starting iSCSI initiator service iscsid [ OK ]
* Setting up iSCSI targets
```

Use **fdisk -l** to enjoy three new iSCSI devices.

```
root@ubu1104:/etc/iscsi/nodes# fdisk -l 2> /dev/null | grep Disk
Disk /dev/sda: 17.2 GB, 17179869184 bytes
Disk identifier: 0x0001983f
Disk /dev/sdb: 209 MB, 209715200 bytes
Disk identifier: 0x00000000
Disk /dev/sdd: 314 MB, 314572800 bytes
Disk identifier: 0x00000000
Disk /dev/sdc: 104 MB, 104857600 bytes
Disk identifier: 0x00000000
```

The Target (the server) now shows active sessions.

```
root@debby6:/etc/iet# cat /proc/net/iet/session
tid:3 name:iqn.2010-02.be.linux-training:storage.lun3
  sid:5348024611832320 initiator:iqn.1993-08.org.debian:01:8983ed2d770
  cid:0 ip:192.168.1.35 state:active hd:none dd:none
tid:2 name:iqn.2010-02.be.linux-training:storage.lun2
  sid:4785074624856576 initiator:iqn.1993-08.org.debian:01:8983ed2d770
  cid:0 ip:192.168.1.35 state:active hd:none dd:none
tid:1 name:iqn.2010-02.be.linux-training:storage.lun1
  sid:5066549618344448 initiator:iqn.1993-08.org.debian:01:8983ed2d770
  cid:0 ip:192.168.1.35 state:active hd:none dd:none
root@debby6:/etc/iet#
```

47.7. using iSCSI devices

There is no difference between using SCSI or iSCSI devices once they are connected : partition, make filesystem, mount.

```
root@ubu1104:/etc/iscsi/nodes# history | tail -13
 94 fdisk /dev/sdc
 95 fdisk /dev/sdd
 96 fdisk /dev/sdb
 97 mke2fs /dev/sdb1
 98 mke2fs -j /dev/sdc1
 99 mkfs.ext4 /dev/sdd1
100 mkdir /mnt/is1
101 mkdir /mnt/is2
102 mkdir /mnt/is3
103 mount /dev/sdb1 /mnt/is1
104 mount /dev/sdc1 /mnt/is2
105 mount /dev/sdd1 /mnt/is3
106 history | tail -13
root@ubu1104:/etc/iscsi/nodes# mount | grep is
/dev/sdb1 on /mnt/is1 type ext2 (rw)
/dev/sdc1 on /mnt/is2 type ext3 (rw)
/dev/sdd1 on /mnt/is3 type ext4 (rw)
```

47.8. practice: iSCSI devices

1. Set up a target (using an LVM and a SCSI device) and an initiator that connects to both.

47.9. solution: iSCSI devices

1. Set up a target (using an LVM and a SCSI device) and an initiator that connects to both.

This solution was done on **Debian/ubuntu/Mint**. For RHEL/CentOS check the theory.

Decide (with a partner) on a computer to be the Target and another computer to be the Initiator.

On the Target computer:

First install `iscsitarget` using the standard tools for installing software in your distribution. Then use your knowledge from the previous chapter to setup a logical volume (`/dev/vg/lvol0`) and use the RAID chapter to setup `/dev/md0`. Then perform the following step:

```
vi /etc/default/iscsitarget (set enable to true)
```

Add your devices to `/etc/iet/ietf.conf`

```
root@debby6:/etc/iet# cat ietd.conf
Target iqn.2010-02.be.linux-training:storage.lun1
  IncomingUser isuser hunter2
  OutgoingUser
  Lun 0 Path=/dev/vg/lvol0,Type=fileio
  Alias LUN1
Target iqn.2010-02.be.linux-training:storage.lun2
  IncomingUser isuser hunter2
  OutgoingUser
  Lun 0 Path=/dev/md0,Type=fileio
  Alias LUN2
```

Add both devices to `/etc/iet/initiators.allow`

```
root@debby6:/etc/iet# cat initiators.allow
iqn.2010-02.be.linux-training:storage.lun1
iqn.2010-02.be.linux-training:storage.lun2
```

Now start the `iscsitarget` daemon and move over to the Initiator.

On the Initiator computer:

Install `open-iscsi` and start the daemon.

Then use `iscsiadm -m discovery -t st -p 'target-ip'` to see the `iscsi` devices on the Target.

Edit the files `/etc/iscsi/nodes/` as shown in the book. Then restart the `iSCSI` daemon and run `fdisk -l` to see the `iSCSI` devices.

Chapter 48. introduction to multipathing

48.1. install multipath

RHEL and CentOS need the **device-mapper-multipath** package.

```
yum install device-mapper-multipath
```

This will create a sample `multipath.conf` in `/usr/share/doc/device-mapper-multipath-0.4.9/multipath.conf`.

There is no `/etc/multipath.conf` until you initialize it with **mpathconf**.

```
[root@server2 ~]# mpathconf --enable --with_multipathd y
Starting multipathd daemon: [ OK ]
[root@server2 ~]# wc -l /etc/multipath.conf
99 /etc/multipath.conf
```

48.2. configure multipath

You can now choose to either edit `/etc/multipath.conf` or use **mpathconf** to change this file for you.

```
[root@server2 ~]# grep user_friendly_names /etc/multipath.conf
user_friendly_names yes
# user_friendly_names yes
[root@server2 ~]# mpathconf --enable --user_friendly_names n
[root@server2 ~]# grep user_friendly_names /etc/multipath.conf
user_friendly_names no
# user_friendly_names yes
[root@server2 ~]# mpathconf --enable --user_friendly_names y
[root@server2 ~]# grep user_friendly_names /etc/multipath.conf
user_friendly_names yes
# user_friendly_names yes
```

48.3. network

This example uses three networks, make sure the iSCSI Target is connected to all three networks.

```
[root@server1 tgt]# ifconfig | grep -B1 192.168
eth1      Link encap:Ethernet  HWaddr 08:00:27:4E:AB:8E
          inet addr:192.168.1.98  Bcast:192.168.1.255  Mask:255.255.255.0
--
eth2      Link encap:Ethernet  HWaddr 08:00:27:3F:A9:D1
          inet addr:192.168.2.98  Bcast:192.168.2.255  Mask:255.255.255.0
--
eth3      Link encap:Ethernet  HWaddr 08:00:27:94:52:26
          inet addr:192.168.3.98  Bcast:192.168.3.255  Mask:255.255.255.0
```

The same must be true for the multipath Initiator:

```
[root@server2 ~]# ifconfig | grep -B1 192.168
eth1      Link encap:Ethernet  HWaddr 08:00:27:A1:43:41
          inet addr:192.168.1.99  Bcast:192.168.1.255  Mask:255.255.255.0
--
eth2      Link encap:Ethernet  HWaddr 08:00:27:12:A8:70
          inet addr:192.168.2.99  Bcast:192.168.2.255  Mask:255.255.255.0
--
eth3      Link encap:Ethernet  HWaddr 08:00:27:6E:99:9B
          inet addr:192.168.3.99  Bcast:192.168.3.255  Mask:255.255.255.0
```

48.4. start multipathd and iscsi

Time to start (or restart) both the multipathd and iscsi services:

```
[root@server2 ~]# service multipathd restart
Stopping multipathd daemon:          [ OK ]
Starting multipathd daemon:         [ OK ]
[root@server2 ~]# service iscsi restart
Stopping iscsi:                      [ OK ]
Starting iscsi:                      [ OK ]
```

This shows **fdisk** output when leaving the default `friendly_names` option to yes. The bottom three are the multipath devices to use.

```
[root@server2 ~]# fdisk -l | grep Disk
Disk /dev/sda: 42.9 GB, 42949672960 bytes
Disk identifier: 0x0004f229
Disk /dev/sdb: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdc: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdd: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sde: 2147 MB, 2147483648 bytes
Disk identifier: 0x00000000
Disk /dev/sdf: 2147 MB, 2147483648 bytes
Disk identifier: 0x00000000
Disk /dev/sdg: 2147 MB, 2147483648 bytes
Disk identifier: 0x00000000
Disk /dev/mapper/VolGroup-lv_root: 41.4 GB, 41448112128 bytes
Disk identifier: 0x00000000
Disk /dev/mapper/VolGroup-lv_swap: 973 MB, 973078528 bytes
Disk identifier: 0x00000000
Disk /dev/sdh: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdi: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdj: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdl: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdn: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdk: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdm: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdp: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/sdo: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/mapper/mpathh: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/mapper/mpathi: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
Disk /dev/mapper/mpathj: 1073 MB, 1073741824 bytes
Disk identifier: 0x00000000
[root@server2 ~]#
```

48.5. multipath list

You can list the multipath connections and devices with **multipath -ll**.

```
[root@server2 ~]# multipath -ll
mpathj (1IET      00010001) dm-4 Reddy,VBOX HARDDISK
size=1.0G features='0' hwhandler='0' wp=rw
|-+- policy='round-robin 0' prio=1 status=active
|  `- 13:0:0:1 sdh 8:112 active ready running
|-+- policy='round-robin 0' prio=1 status=enabled
|  `- 12:0:0:1 sdi 8:128 active ready running
`-+- policy='round-robin 0' prio=1 status=enabled
   `- 14:0:0:1 sdm 8:192 active ready running
mpathi (1IET      00010003) dm-3 Reddy,VBOX HARDDISK
size=1.0G features='0' hwhandler='0' wp=rw
|-+- policy='round-robin 0' prio=1 status=active
|  `- 13:0:0:3 sdk 8:160 active ready running
|-+- policy='round-robin 0' prio=1 status=enabled
|  `- 12:0:0:3 sdn 8:208 active ready running
`-+- policy='round-robin 0' prio=1 status=enabled
   `- 14:0:0:3 sdp 8:240 active ready running
mpathh (1IET      00010002) dm-2 Reddy,VBOX HARDDISK
size=1.0G features='0' hwhandler='0' wp=rw
|-+- policy='round-robin 0' prio=1 status=active
|  `- 12:0:0:2 sdl 8:176 active ready running
|-+- policy='round-robin 0' prio=1 status=enabled
|  `- 13:0:0:2 sdj 8:144 active ready running
`-+- policy='round-robin 0' prio=1 status=enabled
   `- 14:0:0:2 sdo 8:224 active ready running
[root@server2 ~]#
```

The IET (iSCSI Enterprise Target) ID should match the ones you see on the Target server.

```
[root@server1 ~]# tgt-admin -s | grep -e LUN -e IET -e dev
LUN information:
  LUN: 0
    SCSI ID: IET      00010000
  LUN: 1
    SCSI ID: IET      00010001
    Backing store path: /dev/sdb
  LUN: 2
    SCSI ID: IET      00010002
    Backing store path: /dev/sdc
  LUN: 3
    SCSI ID: IET      00010003
    Backing store path: /dev/sdd
```

48.6. using the device

The rest is standard mkfs, mkdir, mount:

```
[root@server2 ~]# mkfs.ext4 /dev/mapper/mpathi
mke2fs 1.41.12 (17-May-2010)
Filesystem label=
OS type: Linux
Block size=4096 (log=2)
Fragment size=4096 (log=2)
Stride=0 blocks, Stripe width=0 blocks
65536 inodes, 262144 blocks
13107 blocks (5.00%) reserved for the super user
First data block=0
Maximum filesystem blocks=268435456
8 block groups
32768 blocks per group, 32768 fragments per group
8192 inodes per group
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376

Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done

This filesystem will be automatically checked every 38 mounts or
180 days, whichever comes first.  Use tune2fs -c or -i to override.
[root@server2 ~]# mkdir /srv/multipath
[root@server2 ~]# mount /dev/mapper/mpathi /srv/multipath/
[root@server2 ~]# df -h /srv/multipath/
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/mpathi 1008M        34M   924M   4% /srv/multipath
```

48.7. practice: multipathing

1. Find a partner and decide who will be iSCSI Target and who will be iSCSI Initiator and Multipather. Set up Multipath as we did in the theory.
2. Uncomment the big 'defaults' section in `/etc/multipath.conf` and disable friendly names. Verify that multipath can work. You may need to check the manual for `/lib/dev/scsi_id` and for **`multipath.conf`**.

48.8. solution: multipathing

1. Find a partner and decide who will be iSCSI Target and who will be iSCSI Initiator and Multipather. Set up Multipath as we did in the theory.

Look in the theory...

2. Uncomment the big 'defaults' section in `/etc/multipath.conf` and disable friendly names. Verify that multipath can work. You may need to check the manual for `/lib/dev/scsi_id` and for **`multipath.conf`**.

`vi multipath.conf`

```
remove # for the big defaults section
add # for the very small one with friendly_names active
add the --replace-whitespace option to scsi_id.

defaults {
    udev_dir                /dev
    polling_interval        10
    path_selector            "round-robin 0"
    path_grouping_policy    multibus
    getuid_callout          "/lib/udev/scsi_id --whitelisted --replace\
- whitespace --device=/dev/%n"
    prio                    const
    path_checker            readsector0
    rr_min_io               100
    max_fds                 8192
    rr_weight               priorities
    failback                immediate
    no_path_retry           fail
    user_friendly_names     no
}
```

The names now (after service restart) look like:

```
root@server2 etc]# multipath -ll
1IET_00010001 dm-8 Reddy,VBOX HARDDISK
size=1.0G features='0' hwhandler='0' wp=rw
`-+- policy='round-robin 0' prio=1 status=active
   |- 17:0:0:1 sdh 8:112 active ready running
   |- 16:0:0:1 sdi 8:128 active ready running
   `-- 15:0:0:1 sdn 8:208 active ready running
1IET_00010003 dm-10 Reddy,VBOX HARDDISK
size=1.0G features='0' hwhandler='0' wp=rw
`-+- policy='round-robin 0' prio=1 status=active
   |- 17:0:0:3 sdl 8:176 active ready running
   |- 16:0:0:3 sdm 8:192 active ready running
   `-- 15:0:0:3 sdp 8:240 active ready running
1IET_00010002 dm-9 Reddy,VBOX HARDDISK
size=1.0G features='0' hwhandler='0' wp=rw
`-+- policy='round-robin 0' prio=1 status=active
   |- 17:0:0:2 sdj 8:144 active ready running
   |- 16:0:0:2 sdk 8:160 active ready running
   `-- 15:0:0:2 sdo 8:224 active ready running
```


Did you blacklist your own devices ?

```
vi multipath.conf
--> search for blacklist:
add
    devnode "^sd[a-g]"
```

Part XII. boot management

Table of Contents

49. bootloader	484
49.1. boot terminology	485
49.2. grub	488
49.3. grub2	493
49.4. lilo	494
49.5. practice: bootloader	495
49.6. solution: bootloader	496
50. init and runlevels	497
50.1. system init(ialization)	498
50.2. daemon or demon ?	503
50.3. starting and stopping daemons	503
50.4. chkconfig	504
50.5. update-rc.d	506
50.6. bum	507
50.7. runlevels	508
50.8. systemd	510
50.9. practice: init	516
50.10. solution : init	517

Chapter 49. bootloader

This chapter briefly discusses the boot sequence of an (Intel 32-bit or 64-bit) Linux computer.

Systems booting with **lilo** are rare nowadays, so this section is brief.

The most common bootloader on Linux systems today is **grub**, yet this is not a Linux project. Distributions like **FreeBSD** and **Solaris** also use **grub**.

Likewise, **grub** is not limited to Intel architecture. It can also load kernels on PowerPC.

Note that **grub**, while still the default in Debian, is slowly being replaced in most distributions with **grub2**.

49.1. boot terminology

The exact order of things that happen when starting a computer system, depends on the hardware architecture (**Intel x86** is different from **Sun Sparc** etc), on the boot loader (**grub** is different from **lilo**) and on the operating system (**Linux**, **Solaris**, **BSD** etc). Most of this chapter is focused on booting **Linux** on **Intel x86** with **grub**.

49.1.1. post

A computer starts booting the moment you turn on the power (no kidding). This first process is called **post** or **power on self test**. If all goes well then this leads to the **bios**. If all goes not so well, then you might hear nothing, or hear beeping, or see an error message on the screen, or maybe see smoke coming out of the computer (burning hardware smells bad!).

49.1.2. bios

All **Intel x86** computers will have a **basic input/output system** or **bios** to detect, identify and initialize hardware. The **bios** then goes looking for a **boot device**. This can be a floppy, hard disk, cdrom, network card or usb drive.

During the **bios** you can see a message on the screen telling you which key (often **Del** or **F2**) to press to enter the **bios** setup.

PhoenixBIOS Setup Utility					
Main	Advanced	Security	Power	Boot	Exit
System Time: [18:06:55] System Date: [06/01/2009] Legacy Diskette A: [1.44/1.25 MB 3½"] Legacy Diskette B: [Disabled] ▶ Primary Master [None] ▶ Primary Slave [None] ▶ Secondary Master [VMware Virtual ID] ▶ Secondary Slave [None] ▶ Keyboard Features System Memory: 640 KB Extended Memory: 261120 KB Boot-time Diagnostic Screen: [Disabled]					Item Specific Help <Tab>, <Shift-Tab>, or <Enter> selects field.
F1	Help	↑↓	Select Item	-/+	Change Values
Esc	Exit	↔	Select Menu	Enter	Select ▶ Sub-Menu
					F9 Setup Defaults
					F10 Save and Exit

49.1.3. openboot

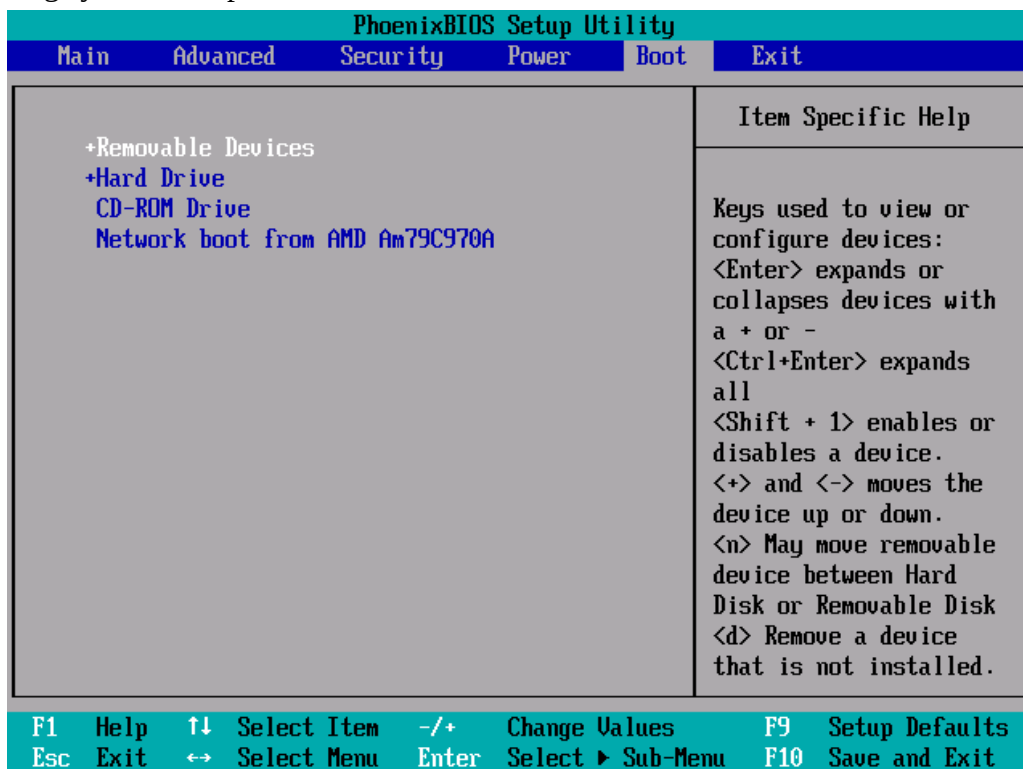
Sun **sparc** systems start with **openboot** to test the hardware and to boot the operating system. **Bill Callkins** explains **openboot** in his Solaris System Administration books. The details of **openboot** are not the focus of this course.

49.1.4. boot password

The **bios** allows you to set a password. Do not forget this password, or you will have to open up the hardware to reset it. You can sometimes set a password to boot the system, and another password to protect the **bios** from being modified.

49.1.5. boot device

The **bios** will look for a **boot device** in the order configured in the bios setup. Usually an operating system on a production server boots of a hard disk.



49.1.6. master boot record

The **master boot record** or **mbr** is the first sector of a hard disk. The partitioning of a disk in **primary** partitions, and the active partition are defined in the **mbr**.

The **mbr** is 512 bytes long and can be copied with **dd**.

```
dd if=/dev/sda of=bootsect.mbr count=1 bs=512
```

49.1.7. bootloader

The **mbr** is executed by the **bios** and contains either (a small) **bootloader** or code to load a **bootloader**.

Looking at the **mbr** with **od** can reveal information about the **bootloader**.

```
paul@laika:~$ sudo dd if=/dev/sda count=1 bs=16 skip=24 2>/dev/null | od -c
00000000 376  G  R  U  B      \0  G  e  o  m  \0  H  a  r  d
00000020
```

There are a variety of bootloaders available, most common on **Intel** architecture is **grub**, which is replacing **lilo** in many places. When installing **Linux** on **sparc** architecture, you can choose **sil**, **Itanium** systems can use **elilo**, **IBM S/390** and **zSeries** use **z/IPL**, **Alpha** uses **milo** and **PowerPC** architectures use **yaboot** (yet another boot loader).

Bootable cd's and dvd's often use **syslinux**.

49.1.8. kernel

The goal of all this is to load an operating system, or rather the **kernel** of an operating system. A typical bootloader like **grub** will copy a kernel from hard disk to memory, and will then hand control of the computer to the kernel (execute the kernel).

Once the Linux kernel is loaded, the bootloader turns control over to it. From that moment on, the kernel is in control of the system. After discussing bootloaders, we continue with the **init system** that starts all the daemons.

49.2. grub

49.2.1. /boot/grub/grub.cfg

Debian switched to **grub2**, which will be discussed in the next section. The main boot menu configuration file for **grub2** is **grub.cfg**.

```
root@debian7:~# ls -l /boot/grub/grub.cfg
-r--r--r-- 1 root root 2453 May 13 17:22 /boot/grub/grub.cfg
root@debian7:~#
```

49.2.2. /boot/grub/grub.conf

Distributions like Red Hat Enterprise Linux 6 use **grub.conf** and provide a symbolic link from **/boot/grub/menu.lst** and from **/etc/grub.conf** to this file.

```
[root@centos65 ~]# ls -l /boot/grub/menu.lst
lrwxrwxrwx. 1 root root 11 Mar  7 11:53 /boot/grub/menu.lst -> ./grub.conf
[root@centos65 ~]# ls -l /boot/grub/grub.conf
-rw----- 1 root root 1189 May  5 11:47 /boot/grub/grub.conf
[root@centos65 ~]#
```

The file currently (RHEL 6.5) looks like this:

```
[root@centos65 ~]# more /boot/grub/grub.conf
# grub.conf generated by anaconda
#
# Note that you do not have to rerun grub after making changes to this file
# NOTICE: You have a /boot partition. This means that
#           all kernel and initrd paths are relative to /boot/, eg.
#           root (hd0,0)
#           kernel /vmlinuz-version ro root=/dev/mapper/VolGroup-lv_root
#           initrd /initrd-[generic-]version.img
#boot=/dev/sda
default=0
timeout=5
splashimage=(hd0,0)/grub/splash.xpm.gz
hiddenmenu
title CentOS (2.6.32-431.11.2.el6.x86_64)
    root (hd0,0)
    kernel /vmlinuz-2.6.32-431.11.2.el6.x86_64 ro root=/dev/mapper/VolGr\
oup-lv_root rd_NO_LUKS LANG=en_US.UTF-8 rd_NO_MD rd_LVM_LV=VolGroup/lv_swap \
SYSFONT=latacyrheb-sun16 crashkernel=auto rd_LVM_LV=VolGroup/lv_root KEYBO\
ARDTYPE=pc KEYTABLE=us rd_NO_DM rhgb quiet
    initrd /initramfs-2.6.32-431.11.2.el6.x86_64.img
title CentOS (2.6.32-431.el6.x86_64)
    root (hd0,0)
    kernel /vmlinuz-2.6.32-431.el6.x86_64 ro root=/dev/mapper/VolGroup-l\
v_root rd_NO_LUKS LANG=en_US.UTF-8 rd_NO_MD rd_LVM_LV=VolGroup/lv_swap SYSFO\
NT=latacyrheb-sun16 crashkernel=auto rd_LVM_LV=VolGroup/lv_root KEYBOARDTY\
PE=pc KEYTABLE=us rd_NO_DM rhgb quiet
    initrd /initramfs-2.6.32-431.el6.x86_64.img
[root@centos65 ~]#
```


49.2.3. menu commands

The **menu commands** must be at the top of **grub**'s configuration file.

default

The **default** command sets a default **entry** to start. The first **entry** has number 0.

```
default=0
```

Each entry or **stanza** starts with a **title** directive.

fallback

In case the **default** does not boot, use the **fallback** entry instead.

```
fallback=1
```

timeout

The **timeout** will wait a number of seconds before booting the **default** entry.

```
timeout=5
```

hiddenmenu

The **hiddenmenu** will hide the **grub** menu unless the user presses **Esc** before the **timeout** expires.

```
hiddenmenu
```

title

With **title** we can start a new **entry** or **stanza**.

```
title CentOS (2.6.32-431.11.2.el6.x86_64)
```

password

You can add a **password** to prevent interactive selection of a boot environment while **grub** is running.

```
password --md5 $1$Ec.id/$T2C2ahI/EG3WRRsmmu/HN/
```

Use the **grub** interactive shell to create the password hash.

```
grub> md5crypt
```

```
Password: *****
```

```
Encrypted: $1$Ec.id/$T2C2ahI/EG3WRRsmmu/HN/
```

49.2.4. stanza commands

Every **operating system** or **kernel** that you want to boot with **grub** will have a **stanza** aka an **entry** of a couple of lines. Listed here are some of the common **stanza** commands.

boot

Technically the **boot** command is only mandatory when running the **grub command line**. This command does not have any parameters and can only be set as the last command of a stanza.

```
boot
```

kernel

The **kernel** command points to the location of the kernel. To boot Linux this means booting a **gzip** compressed **zImage** or **bzip2** compressed **bzImage**.

This screenshot shows a **kernel** command used to load a Debian kernel.

```
kernel /boot/vmlinuz-2.6.17-2-686 root=/dev/hda1 ro
```

And this is how RHEL 5 uses the **kernel** command.

```
kernel /vmlinuz-2.6.18-128.el5 ro root=/dev/VolGroup00/LogVol00 rhgb quiet
```

All parameters in the kernel line can be read by the kernel itself or by any other program (which are started later) by reading **/proc/cmdline**

initrd

Many **Linux** installations will need an **initial ramdisk** at boot time. This can be set in **grub** with the **initrd** command.

Here a screenshot of Debian 4.0

```
initrd /boot/initrd.img-2.6.17-2-686
```

And the same for Red Hat Enterprise Linux 5

```
initrd /initrd-2.6.18-128.el5.img
```

root

The **root** command accepts the root device as a parameter.

The **root** command will point to the hard disk and partition to use, with **hd0** as the first hard disk device and **hd1** as the second hard disk device. The same numbering is used for partitions, so **hd0,0** is the first partition on the first disk and **hd0,1** is the second partition on that disk.

```
root (hd0,0)
```

savedefault

The **savedefault** command can be used together with **default saved** as a menu command. This combination will set the currently booted stanza as the next default stanza to boot.

```
default saved
timeout 10

title Linux
root (hd0,0)
kernel /boot/vmlinuz
savedefault

title DOS
root (hd0,1)
makeactive
chainloader +1
savedefault
```

49.2.5. chainloading

With **grub** booting, there are two choices: loading an operating system or **chainloading** another bootloader. The **chainloading** feature of grub loads the bootsector of a partition (that contains an operating system).

Some older operating systems require a **primary partition** that is set as **active**. Only one partition can be set **active** so **grub** can do this on the fly just before **chainloading**.

This screenshot shows how to set the first primary partition **active** with **grub**.

```
root (hd0,0)
makeactive
```

Chainloading refers to grub loading another operating system's bootloader. The **chainloader** switch receives one option: the number of sectors to read and boot. For **DOS** and **OS/2** one sector is enough. Note that **DOS** requires the boot/root partition to be active!

Here is a complete example to **chainload** an old operating system.

```
title MS-DOS 6.22
root (hd0,1)
makeactive
chainloader +1
```

49.2.6. simple stanza examples

This is a screenshot of a **Debian 4** stanza.

```
title    Debian GNU/Linux, kernel 2.6.17-2-686
root     (hd0,0)
kernel   /boot/vmlinuz-2.6.17-2-686 root=/dev/hda1 ro
initrd    /boot/initrd.img-2.6.17-2-686
```

Here a screenshot of a **Red Hat Enterprise Linux 5** stanza.

```
title Red Hat Enterprise Linux Server (2.6.18-128.el5)
root (hd0,0)
kernel /vmlinuz-2.6.18-98.el5 ro root=/dev/VolGroup00/LogVol00 rhgb quiet
initrd /initrd-2.6.18-98.el5.img
```

49.2.7. editing grub at boot time

At boot time, when the **grub** menu is displayed, you can type **e** to edit the current stanza. This enables you to add parameters to the kernel.

One such parameter, useful when you lost the root password, is **single**. This will boot the kernel in single user mode (although some distributions will still require you to type the root password).

```
kernel /boot/vmlinuz-2.6.17-2-686 root=/dev/hda1 ro single
```

Another option to reset a root password is to use an **init=/bin/bash** parameter.

```
kernel /boot/vmlinuz-2.6.17-2-686 root=/dev/hda1 ro init=/bin/bash
```

Note that some distributions will disable this option at kernel compile time.

49.2.8. installing grub

Run the **grub-install** command to install **grub**. The command requires a destination for overwriting the **boot sector** or **mbr**.

```
# grub-install /dev/hda
```

You will rarely have to do this manually, since grub is installed when installing the operating system and does not need any re-install when changing configuration (as is the case for **lilo**).

49.3. grub2

49.3.1. grub 2.0 ?

The main configuration file is now **/boot/grub/grub.cfg**. And while this file may look familiar, one should never edit this file directly (because it is generated!).

```
root@debian7:~# ls -l /boot/grub/grub.cfg
-r--r--r-- 1 root root 2453 May 13 17:22 /boot/grub/grub.cfg
root@debian7:~# head -3 /boot/grub/grub.cfg
#
# DO NOT EDIT THIS FILE
#
```

49.3.2. /etc/grub.d/40_custom

The **/etc/grub.d/40_custom** file can be changed to include custom entries. These entries are automatically added to grub.

```
root@debian7:~# ls -l /etc/grub.d/40_custom
-rwxr-xr-x 1 root root 214 Jul  3 2013 /etc/grub.d/40_custom
root@debian7:~# cat /etc/grub.d/40_custom
#!/bin/sh
exec tail -n +3 $0
# This file provides an easy way to add custom menu entries.  Simply type the
# menu entries you want to add after this comment.  Be careful not to change
# the 'exec tail' line above.
```

49.3.3. /etc/default/grub

The new configuration file for changing grub is now **/etc/default/grub**.

```
root@debian7:~# head /etc/default/grub
# If you change this file, run 'update-grub' afterwards to update
# /boot/grub/grub.cfg.
# For full documentation of the options in this file, see:
#   info -f grub -n 'Simple configuration'

GRUB_DEFAULT=0
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR=`lsb_release -i -s 2> /dev/null || echo Debian`
GRUB_CMDLINE_LINUX_DEFAULT="quiet"
GRUB_CMDLINE_LINUX="debian-installer=en_US"
```

49.3.4. update-grub

Whenever the **/etc/default/grub** file is changed, you will need to run **update-grub** to apply the changes.

```
root@debian7:~# vi /etc/default/grub
root@debian7:~# update-grub
Generating grub.cfg ...
Found linux image: /boot/vmlinuz-3.2.0-4-amd64
Found initrd image: /boot/initrd.img-3.2.0-4-amd64
done
```

49.4. lilo

49.4.1. Linux loader

lilo used to be the most used Linux bootloader, but is steadily being replaced with **grub** and recently **grub2**.

49.4.2. lilo.conf

Here is an example of a **lilo.conf** file. The **delay** switch receives a number in tenths of a second. So the delay below is three seconds, not thirty!

```
boot = /dev/hda
delay = 30

image = /boot/vmlinuz
  root = /dev/hda1
  label = Red Hat 5.2

image = /boot/vmlinuz
  root = /dev/hda2
  label = S.U.S.E. 8.0

other = /dev/hda4
  table = /dev/hda
  label = MS-DOS 6.22
```

The configuration file shows three example stanzas. The first one boots Red Hat from the first partition on the first disk (hda1). The second stanza boots Suse 8.0 from the next partition. The last one loads MS-DOS.

49.5. practice: bootloader

0. Find out whether your system is using lilo, grub or grub2. Only do the practices that are appropriate for your system.
1. Make a copy of the kernel, initrd and System.map files in /boot. Put the copies also in /boot but replace 2.x or 3.x with 4.0 (just imagine that Linux 4.0 is out.).
2. Add a stanza in grub for the 4.0 files. Make sure the title is different.
3. Set the boot menu timeout to 30 seconds.
4. Reboot and test the new stanza.

49.6. solution: bootloader

0. Find out whether your system is using lilo, grub or grub2. Only do the practices that are appropriate for your system.

1. Make a copy of the kernel, initrd and System.map files in /boot. Put the copies also in /boot but replace 2.x or 3.x with 4.0 (just imagine that Linux 4.0 is out.).

```
[root@centos65 boot]# uname -r
2.6.32-431.11.2.el6.x86_64
[root@centos65 boot]# cp System.map-2.6.32-431.11.2.el6.x86_64 System.map-4.0
[root@centos65 boot]# cp vmlinuz-2.6.32-431.11.2.el6.x86_64 vmlinuz-4.0
[root@centos65 boot]# cp initramfs-2.6.32-431.11.2.el6.x86_64.img initramfs-4.0\
.img
```

Do not forget that the initrd (or initramfs) file ends in **.img**.

2. Add a stanza in grub for the 4.0 files. Make sure the title is different.

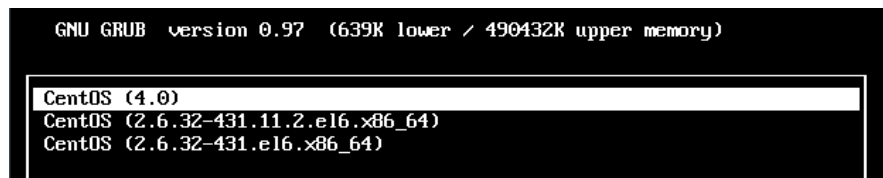
```
[root@centos65 grub]# cut -c1-70 menu.lst | tail -12
title CentOS (4.0)
    root (hd0,0)
    kernel /vmlinuz-4.0 ro root=/dev/mapper/VolGroup-lv_root rd_NO_LUKS L
    initrd /initramfs-4.0.img
title CentOS (2.6.32-431.11.2.el6.x86_64)
    root (hd0,0)
    kernel /vmlinuz-2.6.32-431.11.2.el6.x86_64 ro root=/dev/mapper/VolGro
    initrd /initramfs-2.6.32-431.11.2.el6.x86_64.img
title CentOS (2.6.32-431.el6.x86_64)
    root (hd0,0)
    kernel /vmlinuz-2.6.32-431.el6.x86_64 ro root=/dev/mapper/VolGroup-lv
    initrd /initramfs-2.6.32-431.el6.x86_64.img
[root@centos65 grub]#
```

3. Set the boot menu timeout to 30 seconds.

```
[root@centos65 grub]# vi menu.lst
[root@centos65 grub]# grep timeout /boot/grub/grub.conf
timeout=30
```

4. Reboot and test the new stanza.

```
[root@centos65 grub]# reboot
```



Select your stanza and if it boots then you did it correct.

Chapter 50. init and runlevels

Many Unix and Linux distributions use **init** scripts to start daemons in the same way that **Unix System V** did. This chapter will explain in detail how that works.

Init starts daemons by using scripts, where each script starts one daemon, and where each script waits for the previous script to finish. This serial process of starting daemons is slow, and although slow booting is not a problem on servers where uptime is measured in years, the recent uptake of Linux on the desktop results in user complaints.

To improve Linux startup speed, **Canonical** has developed **upstart**, which was first used in Ubuntu. Solaris also used **init** up to Solaris 9, for Solaris 10 **Sun** developed **Service Management Facility**. Both systems start daemons in parallel and can replace the SysV init scripts. There is also an ongoing effort to create **initng** (init next generation).

In 2014 the **systemd** initiative has taken a lead when after Fedora, RHEL7 and CentOS7 also Debian has chosen this to be the preferred replacement for init. The end of this module contains an introduction to **systemd**.

50.1. system init(ialization)

50.1.1. process id 1

The kernel receives system control from the bootloader. After a while the kernel starts the **init daemon**. The **init** daemon (**/sbin/init**) is the first daemon that is started and receives **process id 1** (PID 1). **Init** never dies.

50.1.2. configuration in /etc/inittab

When **/sbin/init** is started, it will first read its configuration file **/etc/inittab**. In that file, it will look for the value of **initdefault** (3 in the screenshot below).

```
[paul@rhel4 ~]$ grep ^id /etc/inittab
id:3:initdefault:
```

50.1.3. initdefault

The value found in **initdefault** indicates the default **runlevel**. Some Linux distributions have a brief description of runlevels in **/etc/inittab**, like here on Red Hat Enterprise Linux 4.

```
# Default runlevel. The runlevels used by RHS are:
# 0 - halt (Do NOT set initdefault to this)
# 1 - Single user mode
# 2 - Multiuser, without NFS (The same as 3, if you don't have network)
# 3 - Full multiuser mode
# 4 - unused
# 5 - X11
# 6 - reboot (Do NOT set initdefault to this)
```

Runlevel 0 means the system is shutting down. **Runlevel 1** is used for troubleshooting, only the root user can log on, and only at the console. **Runlevel 3** is typical for servers, whereas **runlevel 5** is typical for desktops (graphical logon). Besides runlevels 0, 1 and 6, the use may vary depending on the distribution. Debian and derived Linux systems have full network and GUI logon on runlevels 2 to 5. So always verify the proper meaning of runlevels on your system.

50.1.4. sysinit script

/etc/rc.d/rc.sysinit

The next line in **/etc/inittab** in Red Hat and derivatives is the following.

```
si::sysinit:/etc/rc.d/rc.sysinit
```

This means that independent of the selected runlevel, **init** will run the **/etc/rc.d/rc.sysinit** script. This script initializes hardware, sets some basic environment, populates **/etc/mtab** while mounting file systems, starts swap and more.

```
[paul@rhel ~]$ egrep -e"^# Ini" -e"^# Sta" -e"^# Che" /etc/rc.d/rc.sysinit
# Check SELinux status
# Initialize hardware
# Start the graphical boot, if necessary; /usr may not be mounted yet...
# Initialize ACPI bits
# Check filesystems
# Start the graphical boot, if necessary and not done yet.
# Check to see if SELinux requires a relabel
# Initialize pseudo-random number generator
# Start up swapping.
# Initialize the serial ports.
```

That **egrep** command could also have been written with **grep** like this :

```
grep "^# \(Ini\|Sta\|Che\)".
```

/etc/init.d/rcS

Debian has the following line after **initdefault**.

```
si::sysinit:/etc/init.d/rcS
```

The **/etc/init.d/rcS** script will always run on Debian (independent of the selected runlevel). The script is actually running all scripts in the **/etc/rcS.d/** directory in alphabetical order.

```
root@barry:~# cat /etc/init.d/rcS
#!/bin/sh
#
# rcS
#
# Call all S??* scripts in /etc/rcS.d/ in numerical/alphabetical order
#
exec /etc/init.d/rc S
```

50.1.5. rc scripts

Init will continue to read **/etc/inittab** and meets this section on Debian Linux.

```
l0:0:wait:/etc/init.d/rc 0
l1:1:wait:/etc/init.d/rc 1
l2:2:wait:/etc/init.d/rc 2
l3:3:wait:/etc/init.d/rc 3
l4:4:wait:/etc/init.d/rc 4
l5:5:wait:/etc/init.d/rc 5
l6:6:wait:/etc/init.d/rc 6
```

On Red Hat Enterprise Linux it is identical except **init.d** is **rc.d**.

```
l0:0:wait:/etc/rc.d/rc 0
l1:1:wait:/etc/rc.d/rc 1
l2:2:wait:/etc/rc.d/rc 2
l3:3:wait:/etc/rc.d/rc 3
l4:4:wait:/etc/rc.d/rc 4
l5:5:wait:/etc/rc.d/rc 5
l6:6:wait:/etc/rc.d/rc 6
```

In both cases, this means that **init** will start the rc script with the runlevel as the only parameter. Actually **/etc/inittab** has fields separated by colons. The second field determines the runlevel in which this line should be executed. So in both cases, only one line of the seven will be executed, depending on the runlevel set by **initdefault**.

50.1.6. rc directories

When you take a look any of the **/etc/rcX.d/** directories, then you will see a lot of (links to) scripts whose name start with either uppercase K or uppercase S.

```
[root@RHEL52 rc3.d]# ls -l | tail -4
lrwxrwxrwx 1 root root 19 Oct 11 2008 S98haldaemon -> ../init.d/haldaemon
lrwxrwxrwx 1 root root 19 Oct 11 2008 S99firstboot -> ../init.d/firstboot
lrwxrwxrwx 1 root root 11 Jan 21 04:16 S99local -> ../rc.local
lrwxrwxrwx 1 root root 16 Jan 21 04:17 S99smartd -> ../init.d/smartd
```

The **/etc/rcX.d/** directories only contain links to scripts in **/etc/init.d/**. Links allow for the script to have a different name. When entering a runlevel, all scripts that start with uppercase K or uppercase S will be started in alphabetical order. Those that start with K will be started first, with **stop** as the only parameter. The remaining scripts with S will be started with **start** as the only parameter.

All this is done by the **/etc/rc.d/rc** script on Red Hat and by the **/etc/init.d/rc** script on Debian.

50.1.7. mingetty

mingetty in /etc/inittab

Almost at the end of **/etc/inittab** there is a section to start and **respawn** several **mingetty** daemons.

```
[root@RHEL4b ~]# grep getty /etc/inittab
# Run gettys in standard runlevels
1:2345:respawn:/sbin/mingetty tty1
2:2345:respawn:/sbin/mingetty tty2
3:2345:respawn:/sbin/mingetty tty3
4:2345:respawn:/sbin/mingetty tty4
5:2345:respawn:/sbin/mingetty tty5
6:2345:respawn:/sbin/mingetty tty6
```

mingetty and /bin/login

This **/sbin/mingetty** will display a message on a virtual console and allow you to type a userid. Then it executes the **/bin/login** command with that userid. The **/bin/login** program will verify whether that user exists in **/etc/passwd** and prompt for (and verify) a password. If the password is correct, **/bin/login** passes control to the shell listed in **/etc/passwd**.

respawning mingetty

The mingetty daemons are started by **init** and watched until they die (user exits the shell and is logged out). When this happens, the **init** daemon will **respawn** a new mingetty. So even if you **kill** a mingetty daemon, it will be restarted automatically.

This example shows that init respawns mingetty daemons. Look at the PID's of the last two mingetty processes.

```
[root@RHEL52 ~]# ps -C mingetty
  PID TTY          TIME CMD
 2407 tty1      00:00:00 mingetty
 2408 tty2      00:00:00 mingetty
 2409 tty3      00:00:00 mingetty
 2410 tty4      00:00:00 mingetty
 2411 tty5      00:00:00 mingetty
 2412 tty6      00:00:00 mingetty
```

When we **kill** the last two mingettys, then **init** will notice this and start them again (with a different PID).

```
[root@RHEL52 ~]# kill 2411 2412
[root@RHEL52 ~]# ps -C mingetty
  PID TTY          TIME CMD
 2407 tty1      00:00:00 mingetty
 2408 tty2      00:00:00 mingetty
 2409 tty3      00:00:00 mingetty
 2410 tty4      00:00:00 mingetty
 2821 tty5      00:00:00 mingetty
 2824 tty6      00:00:00 mingetty
```

disabling a mingetty

You can disable a mingetty for a certain tty by removing the runlevel from the second field in its line in `/etc/inittab`. Don't forget to tell init about the change of its configuration file with **kill -1 1**.

The example below shows how to disable mingetty on tty3 to tty6 in runlevels 4 and 5.

```
[root@RHEL52 ~]# grep getty /etc/inittab
# Run gettys in standard runlevels
1:2345:respawn:/sbin/mingetty tty1
2:2345:respawn:/sbin/mingetty tty2
3:23:respawn:/sbin/mingetty tty3
4:23:respawn:/sbin/mingetty tty4
5:23:respawn:/sbin/mingetty tty5
6:23:respawn:/sbin/mingetty tty6
```

50.2. daemon or demon ?

A **daemon** is a process that runs in background, without a link to a GUI or terminal. Daemons are usually started at system boot, and stay alive until the system shuts down. In more recent technical writings, daemons are often referred to as **services**.

Unix **daemons** are not to be confused with demons. **Evi Nemeth**, co-author of the UNIX System Administration Handbook has the following to say about daemons:

Many people equate the word "daemon" with the word "demon", implying some kind of satanic connection between UNIX and the underworld. This is an egregious misunderstanding. "Daemon" is actually a much older form of "demon"; daemons have no particular bias towards good or evil, but rather serve to help define a person's character or personality. The ancient Greeks' concept of a "personal daemon" was similar to the modern concept of a "guardian angel"

50.3. starting and stopping daemons

The K and S scripts are links to the real scripts in **/etc/init.d/**. These can also be used when the system is running to start and stop daemons (or services). Most of them accept the following parameters: start, stop, restart, status.

For example in this screenshot we restart the samba daemon.

```
root@laika:~# /etc/init.d/samba restart
* Stopping Samba daemons...          [ OK ]
* Starting Samba daemons...          [ OK ]
```

You can achieve the same result on RHEL/Fedora with the **service** command.

```
[root@RHEL4b ~]# service smb restart
Shutting down SMB services:           [ OK ]
Shutting down NMB services:          [ OK ]
Starting SMB services:               [ OK ]
Starting NMB services:               [ OK ]
```

You might also want to take a look at **chkconfig**, **update-rc.d**.

50.4. chkconfig

The purpose of **chkconfig** is to relieve system administrators of manually managing all the links and scripts in **/etc/init.d** and **/etc/rcX.d/**.

50.4.1. chkconfig --list

Here we use **chkconfig** to list the status of a service in the different runlevels. You can see that the **crond** daemon (or service) is only activated in runlevels 2 to 5.

```
[root@RHEL52 ~]# chkconfig --list crond
crond          0:off 1:off 2:on 3:on 4:on 5:on 6:off
```

When you compare the screenshot above with the one below, you can see that **off** equals to a K link to the script, whereas **on** equals to an S link.

```
[root@RHEL52 etc]# find ./rc?.d/ -name \*crond -exec ls -l {} \;|cut -b40-
./rc0.d/K60crond -> ../init.d/crond
./rc1.d/K60crond -> ../init.d/crond
./rc2.d/S90crond -> ../init.d/crond
./rc3.d/S90crond -> ../init.d/crond
./rc4.d/S90crond -> ../init.d/crond
./rc5.d/S90crond -> ../init.d/crond
./rc6.d/K60crond -> ../init.d/crond
```

50.4.2. runlevel configuration

Here you see how to use **chkconfig** to disable (or enable) a service in a certain runlevel.

This screenshot shows how to disable **crond** in runlevel 3.

```
[root@RHEL52 ~]# chkconfig --level 3 crond off
[root@RHEL52 ~]# chkconfig --list crond
crond          0:off 1:off 2:on 3:off 4:on 5:on 6:off
```

This screenshot shows how to enable **crond** in runlevels 3 and 4.

```
[root@RHEL52 ~]# chkconfig --level 34 crond on
[root@RHEL52 ~]# chkconfig --list crond
crond          0:off 1:off 2:on 3:on 4:on 5:on 6:off
```


50.4.3. chkconfig configuration

Every script in `/etc/init.d/` can have (comment) lines to tell `chkconfig` what to do with the service. The line with `# chkconfig:` contains the runlevels in which the service should be started (2345), followed by the priority for start (90) and stop (60).

```
[root@RHEL52 ~]# head -9 /etc/init.d/crond | tail -5
# chkconfig: 2345 90 60
# description: cron is a standard UNIX program that runs user-specified
#               programs at periodic scheduled times. vixie cron adds a
#               number of features to the basic UNIX cron, including better
#               security and more powerful configuration options.
```

50.4.4. enable and disable services

Services can be enabled or disabled in all runlevels with one command. Runlevels 0, 1 and 6 are always stopping services (or calling the scripts with **stop**) even when their name starts with uppercase S.

```
[root@RHEL52 ~]# chkconfig crond off
[root@RHEL52 ~]# chkconfig --list crond
crond          0:off  1:off  2:off  3:off  4:off  5:off  6:off
[root@RHEL52 ~]# chkconfig crond on
[root@RHEL52 ~]# chkconfig --list crond
crond          0:off  1:off  2:on   3:on   4:on   5:on   6:off
```

50.5. update-rc.d

50.5.1. about update-rc.d

The Debian equivalent of **chkconfig** is called **update-rc.d**. This tool is designed for use in scripts, if you prefer a graphical tool then look at **bum**.

When there are existing links in **/etc/rcX.d/** then **update-rc.d** does not do anything. This is to avoid that post installation scripts using **update-rc.d** are overwriting changes made by a system administrator.

```
root@barry:~# update-rc.d cron remove
update-rc.d: /etc/init.d/cron exists during rc.d purge (use -f to force)
```

As you can see in the next screenshot, nothing changed for the cron daemon.

```
root@barry:~# find /etc/rc?.d/ -name '*cron' -exec ls -l {} \;|cut -b44-
/etc/rc0.d/K11cron -> ../init.d/cron
/etc/rc1.d/K11cron -> ../init.d/cron
/etc/rc2.d/S89cron -> ../init.d/cron
/etc/rc3.d/S89cron -> ../init.d/cron
/etc/rc4.d/S89cron -> ../init.d/cron
/etc/rc5.d/S89cron -> ../init.d/cron
/etc/rc6.d/K11cron -> ../init.d/cron
```

50.5.2. removing a service

Here we remove **cron** from all runlevels. Remember that the proper way to disable a service is to put K scripts oin all runlevels!

```
root@barry:~# update-rc.d -f cron remove
Removing any system startup links for /etc/init.d/cron ...
/etc/rc0.d/K11cron
/etc/rc1.d/K11cron
/etc/rc2.d/S89cron
/etc/rc3.d/S89cron
/etc/rc4.d/S89cron
/etc/rc5.d/S89cron
/etc/rc6.d/K11cron
root@barry:~# find /etc/rc?.d/ -name '*cron' -exec ls -l {} \;|cut -b44-
root@barry:~#
```

50.5.3. enable a service

This screenshot shows how to use **update-rc.d** to enable a service in runlevels 2, 3, 4 and 5 and disable the service in runlevels 0, 1 and 6.

```
root@barry:~# update-rc.d cron defaults
Adding system startup for /etc/init.d/cron ...
/etc/rc0.d/K20cron -> ../init.d/cron
/etc/rc1.d/K20cron -> ../init.d/cron
/etc/rc6.d/K20cron -> ../init.d/cron
/etc/rc2.d/S20cron -> ../init.d/cron
/etc/rc3.d/S20cron -> ../init.d/cron
/etc/rc4.d/S20cron -> ../init.d/cron
/etc/rc5.d/S20cron -> ../init.d/cron
```

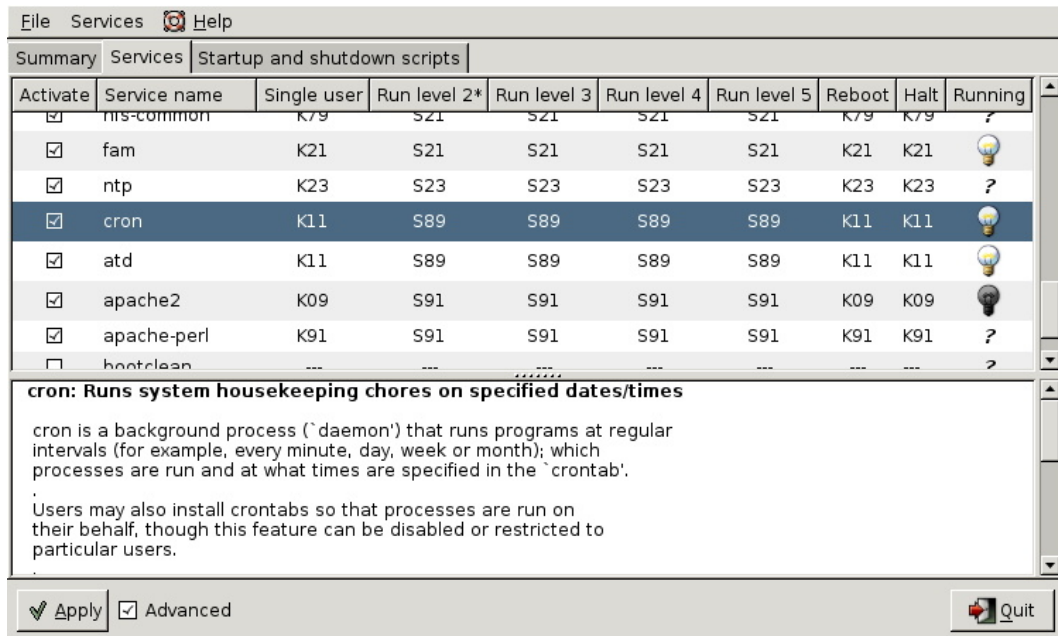
50.5.4. customize a service

And here is an example on how to set your custom configuration for the cron daemon.

```
root@barry:~# update-rc.d -n cron start 11 2 3 4 5 . stop 89 0 1 6 .
Adding system startup for /etc/init.d/cron ...
/etc/rc0.d/K89cron -> ../init.d/cron
/etc/rc1.d/K89cron -> ../init.d/cron
/etc/rc6.d/K89cron -> ../init.d/cron
/etc/rc2.d/S11cron -> ../init.d/cron
/etc/rc3.d/S11cron -> ../init.d/cron
/etc/rc4.d/S11cron -> ../init.d/cron
/etc/rc5.d/S11cron -> ../init.d/cron
```

50.6. bum

This screenshot shows **bum** in advanced mode.



50.7. runlevels

50.7.1. display the runlevel

You can see your current runlevel with the **runlevel** or **who -r** commands.

The **runlevel** command is typical Linux and will output the previous and the current runlevel. If there was no previous runlevel, then it will mark it with the letter N.

```
[root@RHEL4b ~]# runlevel
N 3
```

The history of **who -r** dates back to Seventies Unix, it still works on Linux.

```
[root@RHEL4b ~]# who -r
run-level 3   Jul 28 09:15                last=S
```

50.7.2. changing the runlevel

You can switch to another runlevel with the **telinit** command. On Linux **/sbin/telinit** is usually a (hard) link to **/sbin/init**.

This screenshot shows how to switch from runlevel 2 to runlevel 3 without reboot.

```
root@barry:~# runlevel
N 2
root@barry:~# init 3
root@barry:~# runlevel
2 3
```

50.7.3. /sbin/shutdown

The **shutdown** command is used to properly shut down a system.

Common switches used with **shutdown** are **-a**, **-t**, **-h** and **-r**.

The **-a** switch forces **/sbin/shutdown** to use **/etc/shutdown.allow**. The **-t** switch is used to define the number of seconds between the sending of the **TERM** signal and the **KILL** signal. The **-h** switch halts the system instead of changing to runlevel 1. The **-r** switch tells **/sbin/shutdown** to reboot after shutting down.

This screenshot shows how to use **shutdown** with five seconds between TERM and KILL signals.

```
root@barry:~# shutdown -t5 -h now
```

The **now** is the time argument. This can be **+m** for the number of minutes to wait before shutting down (with **now** as an alias for **+0**). The command will also accept **hh:mm** instead of **+m**.

50.7.4. halt, reboot and poweroff

The binary **/sbin/reboot** is the same as **/sbin/halt** and **/sbin/poweroff**. Depending on the name we use to call the command, it can behave differently.

When in runlevel 0 or 6 **halt**, **reboot** and **poweroff** will tell the kernel to **halt**, **reboot** or **poweroff** the system.

When not in runlevel 0 or 6, typing **reboot** as root actually calls the **shutdown** command with the **-r** switch and typing **poweroff** will switch off the power when halting the system.

50.7.5. /var/log/wtmp

halt, **reboot** and **poweroff** all write to **/var/log/wtmp**. To look at **/var/log/wtmp**, we need to use the **last**.

```
[root@RHEL52 ~]# last | grep reboot
reboot    system boot    2.6.18-128.el5    Fri May 29 11:44    (192+05:01)
reboot    system boot    2.6.18-128.el5    Wed May 27 12:10    (06:49)
reboot    system boot    2.6.18-128.el5    Mon May 25 19:34    (1+15:59)
reboot    system boot    2.6.18-128.el5    Mon Feb  9 13:20    (106+21:13)
```

50.7.6. Ctrl-Alt-Del

When **rc** is finished starting all those scripts, **init** will continue to read **/etc/inittab**. The next line is about what to do when the user hits **Ctrl-Alt-Delete** on the keyboard.

Here is what Debian 4.0 does.

```
root@barry:~# grep -i ctrl /etc/inittab
# What to do when CTRL-ALT-DEL is pressed.
ca:12345:ctrlaltdel:/sbin/shutdown -t1 -a -r now
```

Which is very similar to the default Red Hat Enterprise Linux 5.2 action.

```
[root@RHEL52 ~]# grep -i ctrl /etc/inittab
# Trap CTRL-ALT-DELETE
ca::ctrlaltdel:/sbin/shutdown -t3 -r now
```

One noticeable difference is that Debian forces shutdown to use **/etc/shutdown.allow**, where Red Hat allows everyone to invoke **shutdown** pressing **Ctrl-Alt-Delete**.

50.7.7. UPS and loss of power

```
[root@RHEL52 ~]# grep ^p /etc/inittab
pf::powerfail:/sbin/shutdown -f -h +2 "Power Failure; System Shutting Down"
pr:12345:powerokwait:/sbin/shutdown -c "Power Restored; Shutdown Cancelled"
```

It will read commands on what to execute in case of **powerfailure**, **powerok** and **Ctrl-Alt-Delete**. The **init** process never stops keeping an eye on power failures and that triple key combo.

```
root@barry:~# grep ^p /etc/inittab
pf::powerwait:/etc/init.d/powerfail start
pn::powerfailnow:/etc/init.d/powerfail now
po::powerokwait:/etc/init.d/powerfail stop
```

50.8. systemd

It is likely that **systemd** will replace all the standard init/runlevel/rc functionality. Both Red Hat and Debian have decided in 2014 that **systemd** will be replacing **init** in future releases (RHEL7/CentOS7 and Debian 8).

The screenshot below shows **systemd** running as **pid 1** on RHEL7.

```
[root@rhel7 ~]# ps fax | grep systemd | cut -c1-76
  1 ?      Ss      0:01 /usr/lib/systemd/systemd --switched-root --system
505 ?      Ss      0:00 /usr/lib/systemd/systemd-journald
545 ?      Ss      0:00 /usr/lib/systemd/systemd-udev
670 ?      Ss      0:00 /usr/lib/systemd/systemd-logind
677 ?      Ssl     0:00 /bin/dbus-daemon --system --address=systemd: --no
2662 pts/1  S+      0:00      \_ grep --color=auto systemd
[root@rhel7 ~]#
```

Debian 8 (not yet released in September 2014) uses parts of **systemd**, but still has **init** as **pid 1**.

```
root@debian8:~# ps fax | grep systemd
2042 ?      S        0:00 /sbin/cgmanager --daemon -m name=systemd
10127 pts/4  S+       0:00      |      \_ grep systemd
2777 ?      S        0:00 /lib/systemd/systemd-logind
root@debian8:~#
```

50.8.1. systemd targets

The first command to learn is **systemctl list-units --type=target** (or the shorter version **systemctl -t target**). It will show you the different targets on the system.

```
[root@rhel7 ~]# systemctl list-units --type=target
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
basic.target                        loaded active active Basic System
cryptsetup.target                  loaded active active Encrypted Volumes
getty.target                        loaded active active Login Prompts
graphical.target                   loaded active active Graphical Interface
local-fs-pre.target                loaded active active Local File Systems (Pre)
local-fs.target                    loaded active active Local File Systems
multi-user.target                  loaded active active Multi-User System
network.target                     loaded active active Network
nfs.target                         loaded active active Network File System Server
paths.target                       loaded active active Paths
remote-fs.target                   loaded active active Remote File Systems
slices.target                      loaded active active Slices
sockets.target                     loaded active active Sockets
swap.target                        loaded active active Swap
sysinit.target                     loaded active active System Initialization
timers.target                      loaded active active Timers
```

LOAD = Reflects whether the unit definition was properly loaded.
 ACTIVE = The high-level unit activation state, i.e. generalization of SUB.
 SUB = The low-level unit activation state, values depend on unit type.

16 loaded units listed. Pass --all to see loaded but inactive units, too.
 To show all installed unit files use 'systemctl list-unit-files'.
 [root@rhel7 ~]#

Targets are the replacement of runlevels and define specific points to reach when booting the system. For example the **graphical.target** is reached when you get a graphical interface, and the **nfs.target** requires a running nfs server.

To switch to a target (for example **multi-user.target**), we now use **systemctl isolate** (instead of the equivalent **init 3** to change the runlevel).

```
[root@rhel7 ~]# ps fax | wc -l
169
[root@rhel7 ~]# systemctl isolate multi-user.target
[root@rhel7 ~]# ps fax | wc -l
129
[root@rhel7 ~]#
```

To change the default target, we again use this **systemctl** command (instead of editing the **/etc/inittab** file).

```
[root@rhel7 ~]# systemctl enable multi-user.target --force
rm '/etc/systemd/system/default.target'
ln -s '/usr/lib/systemd/system/multi-user.target' '/etc/systemd/system/default\
.target'
[root@rhel7 ~]#
```

This command removed the file **/etc/systemd/system/default.target** and replaced it with a symbolic link to the **multi-user-.target** target.

50.8.2. systemd dependencies

Dependencies are no longer defined by alphabetical order of running scripts, but by configuration in `/etc/systemd/system/`. For example here are the required services for the **multi-user.target** on Red Hat Enterprise 7.

```
[root@rhel7 ~]# ls /etc/systemd/system/multi-user.target.wants/
abrt-ccpp.service      hypervkvpd.service    postfix.service
abrt-d.service         hypervssd.service     remote-fs.target
abrt-oops.service      irqbalance.service    rhsmcertd.service
abrt-vmcore.service    ksm.service           rngd.service
abrt-xorg.service      ksmtuned.service      rpcbind.service
atd.service            libstoragemgmt.service rsyslog.service
auditd.service         libvirt.service       smartd.service
avahi-daemon.service   mdmonitor.service     sshd.service
chronyd.service        ModemManager.service  sysstat.service
crond.service          NetworkManager.service tuned.service
cups.path              nfs.target             vmttoolsd.service
[root@rhel7 ~]#
```

Debian8 is not fully migrated yet.

```
root@debian8:~# ls /etc/systemd/system/multi-user.target.wants/
anacron.service        binfmt-support.service pppd-dns.service  ssh.service
atd.service            fancontrol.service     remote-fs.target  rsyslog.service
avahi-daemon.service   lm-sensors.service
```

Typical **rc scripts** are replaced with services. Issue the **systemctl list-units -t service --all** (or **systemctl -at service**) to get a list of all services on your system.

```
[root@rhel7 ~]# systemctl -at service | head -5 | column -t | cut -c1-78
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
abrt-ccpp.service                  loaded active exited Install      ABRT      coredump
abrt-oops.service                  loaded active running ABRT        kernel    log
abrt-vmcore.service                loaded inactive dead    Harvest     vmcores   for
abrt-xorg.service                  loaded active running ABRT        Xorg      log
[root@rhel7 ~]#
```

And here an example on how to see the status of the **sshd** service.

```
[root@rhel7 ~]# systemctl status sshd.service
sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled)
   Active: active (running) since Wed 2014-09-10 13:42:21 CEST; 55min ago
 Main PID: 1400 (sshd)
   CGroup: /system.slice/sshd.service
           --1400 /usr/sbin/sshd -D

Sep 10 13:42:21 rhel7 systemd[1]: Started OpenSSH server daemon.
Sep 10 13:42:21 rhel7 sshd[1400]: Server listening on 0.0.0.0 port 22.
Sep 10 13:42:21 rhel7 sshd[1400]: Server listening on :: port 22.
[root@rhel7 ~]#
```


50.8.3. systemd services

The **chkconfig** and **service** commands are considered 'legacy'. They are replaced with **systemctl**.

This screenshot shows the new way to start and stop a service.

```
[root@rhel7 ~]# systemctl start crond.service
[root@rhel7 ~]# systemctl show crond.service | grep State
LoadState=loaded
ActiveState=active
SubState=running
UnitFileState=enabled
[root@rhel7 ~]# systemctl stop crond.service
[root@rhel7 ~]# systemctl show crond.service | grep State
LoadState=loaded
ActiveState=inactive
SubState=dead
UnitFileState=enabled
[root@rhel7 ~]#
```

And here is the new way to stop and disable a service.

```
[root@rhel7 ~]# systemctl stop crond.service
[root@rhel7 ~]# systemctl disable crond.service
rm '/etc/systemd/system/multi-user.target.wants/crond.service'
[root@rhel7 ~]# systemctl show crond.service | grep State
LoadState=loaded
ActiveState=inactive
SubState=dead
UnitFileState=disabled
[root@rhel7 ~]#
```

This screenshot shows how to enable and start the service again.

```
[root@rhel7 ~]# systemctl enable crond.service
ln -s '/usr/lib/systemd/system/crond.service' '/etc/systemd/system/multi-user.\
target.wants/crond.service'
[root@rhel7 ~]# systemctl start crond.service
[root@rhel7 ~]# systemctl show crond.service | grep State
LoadState=loaded
ActiveState=active
SubState=running
UnitFileState=enabled
[root@rhel7 ~]#
```

50.8.4. systemd signalling

You can also use **systemd** to **kill** problematic services.

```
[root@rhel7 ~]# systemctl show crond.service | grep State
LoadState=loaded
ActiveState=active
SubState=running
UnitFileState=enabled
[root@rhel7 ~]# systemctl kill -s SIGKILL crond.service
[root@rhel7 ~]# systemctl show crond.service | grep State
LoadState=loaded
ActiveState=failed
SubState=failed
UnitFileState=enabled
[root@rhel7 ~]#
```

50.8.5. systemd shutdown

The **poweroff**, **halt** and **reboot** commands are considered legacy now and are handled by **systemctl**. The table below shows the legacy commands on the left and their new **systemd** equivalent on the right.

Table 50.1. systemd power management

legacy command	systemd command
poweroff	systemctl poweroff
reboot	systemctl reboot
halt	systemctl halt
pm-suspend	systemctl suspend
pm-hibernate	systemctl hibernate

50.8.6. remote systemd

The **systemctl** utility has a built-in remote control providing there is an **ssh daemon** running on the remote system.

This screenshot shows how to use **systemctl** to verify a service on an other RHEL server.

```
[root@rhel7 ~]# systemctl -H root@192.168.1.65 status sshd
root@192.168.1.65's password:
sshd.service - OpenSSH server daemon
  Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled)
  Active: active (running) since Thu 2014-09-11 13:04:10 CEST; 16min ago
  Process: 1328 ExecStartPre=/usr/sbin/sshd-keygen (code=exited, status=0/SUCCESS)
  Main PID: 1363 (sshd)
  CGroup: /system.slice/sshd.service
[root@rhel7 ~]#
```

50.8.7. there is more systemd

There are other tools...

systemd-analyze	systemd-loginctl
systemd-ask-password	systemd-machine-id-setup
systemd-cat	systemd-notify
systemd-cgls	systemd-nspawn
systemd-cgtop	systemd-run
systemd-coredumpctl	systemd-stdio-bridge
systemd-delta	systemd-sysv-convert
systemd-detect-virt	systemd-tmpfiles
systemd-inhibit	systemd-tty-ask-password-agent

For example **systemd-analyze blame** will give you an overview of the time it took for each service to boot.

```
[root@rhel7 ~]# systemd-analyze blame | head
    1.977s firewall.service
    1.096s tuned.service
    993ms postfix.service
    939ms iprinit.service
    925ms vboxadd-x11.service
    880ms firstboot-graphical.service
    839ms accounts-daemon.service
    829ms network.service
    822ms iprupdate.service
    795ms boot.mount
[root@rhel7 ~]#
```

50.9. practice: init

1. Change **/etc/inittab** so that only two mingetty's are respawned. Kill the other **mingetty**'s and verify that they don't come back.
2. Use the Red Hat Enterprise Linux virtual machine. Go to runlevel 5, display the current and previous runlevel, then go back to runlevel 3.
3. Is the sysinit script on your computers setting or changing the PATH environment variable ?
4. List all init.d scripts that are started in runlevel 2.
5. Write a script that acts like a daemon script in **/etc/init.d/**. It should have a case statement to act on start/stop/restart and status. Test the script!
6. Use **chkconfig** to setup your script to start in runlevels 3,4 and 5, and to stop in any other runlevel.

50.10. solution : init

1. Change **/etc/inittab** so that only two mingetty's are respawned. Kill the other **mingetty's** and verify that they don't come back.

Killing the mingetty's will result in init respawning them. You can edit **/etc/inittab** so it looks like the screenshot below. Don't forget to also run **kill -1 1**.

```
[root@RHEL5 ~]# grep tty /etc/inittab
# Run gettys in standard runlevels
1:2345:respawn:/sbin/mingetty tty1
2:2345:respawn:/sbin/mingetty tty2
3:2:respawn:/sbin/mingetty tty3
4:2:respawn:/sbin/mingetty tty4
5:2:respawn:/sbin/mingetty tty5
6:2:respawn:/sbin/mingetty tty6
[root@RHEL5 ~]#
```

2. Use the Red Hat Enterprise Linux virtual machine. Go to runlevel 5, display the current and previous runlevel, then go back to runlevel 3.

```
init 5 (watch the console for the change taking place)
runlevel
init 3 (again you can follow this on the console)
```

3. Is the sysinit script on your computers setting or changing the PATH environment variable ?

On Red Hat, grep for PATH in **/etc/rc.sysinit**, on Debian/Ubuntu check **/etc/rc.local** and **/etc/ini.t/rc.local**. The answer is probably no, but on RHEL5 the **rc.sysinit** script does set the HOSTNAME variable.

```
[root@RHEL5 etc]# grep HOSTNAME rc.sysinit
```

4. List all init.d scripts that are started in runlevel 2.

```
root@RHEL5 ~# chkconfig --list | grep '2:on'
```

5. Write a script that acts like a daemon script in **/etc/init.d/**. It should have a case statement to act on start/stop/restart and status. Test the script!

The script could look something like this.

```
#!/bin/bash
#
# chkconfig: 345 99 01
# description: pold demo script
#
# /etc/init.d/pold
```

```
#
case "$1" in
  start)
    echo -n "Starting pold..."
    sleep 1;
    touch /var/lock/subsys/pold
    echo "done."
    echo pold started >> /var/log/messages
    ;;
  stop)
    echo -n "Stopping pold..."
    sleep 1;
    rm -rf /var/lock/subsys/pold
    echo "done."
    echo pold stopped >> /var/log/messages
    ;;
  *)
    echo "Usage: /etc/init.d/pold {start|stop}"
    exit 1
    ;;
esac
exit 0
```

The **touch /var/lock/subsys/pold** is mandatory and must be the same filename as the script name, if you want the stop sequence (the K01pold link) to be run.

6. Use **chkconfig** to setup your script to start in runlevels 3,4 and 5, and to stop in any other runlevel.

```
chkconfig --add pold
```

The command above will only work when the **# chkconfig:** and **# description:** lines in the pold script are there.

Part XIII. system management

Table of Contents

51. scheduling	521
51.1. one time jobs with at	522
51.2. cron	524
51.3. practice : scheduling	526
51.4. solution : scheduling	527
52. logging	528
52.1. login logging	529
52.2. syslogd	532
52.3. logger	535
52.4. watching logs	535
52.5. rotating logs	536
52.6. practice : logging	537
52.7. solution : logging	538
53. memory management	540
53.1. displaying memory and cache	541
53.2. managing swap space	542
53.3. monitoring memory with vmstat	544
53.4. practice : memory	545
53.5. solution : memory	546
54. resource monitoring	547
54.1. four basic resources	548
54.2. top	548
54.3. free	548
54.4. watch	549
54.5. vmstat	549
54.6. iostat	550
54.7. mpstat	551
54.8. sadc and sar	551
54.9. ntop	552
54.10. iftop	552
54.11. iptraf	552
54.12. nmon	553
54.13. htop	553
55. package management	554
55.1. package terminology	555
55.2. deb package management	557
55.3. apt-get	559
55.4. aptitude	562
55.5. apt	563
55.6. rpm	564
55.7. yum	566
55.8. alien	573
55.9. downloading software outside the repository	574
55.10. compiling software	574
55.11. practice: package management	575
55.12. solution: package management	576

Chapter 51. scheduling

Linux administrators use the **at** to schedule one time jobs. Recurring jobs are better scheduled with **cron**. The next two sections will discuss both tools.

51.1. one time jobs with at

51.1.1. at

Simple scheduling can be done with the **at** command. This screenshot shows the scheduling of the date command at 22:01 and the sleep command at 22:03.

```
root@laika:~# at 22:01
at> date
at> <EOT>
job 1 at Wed Aug  1 22:01:00 2007
root@laika:~# at 22:03
at> sleep 10
at> <EOT>
job 2 at Wed Aug  1 22:03:00 2007
root@laika:~#
```

In real life you will hopefully be scheduling more useful commands ;-)

51.1.2. atq

It is easy to check when jobs are scheduled with the **atq** or **at -l** commands.

```
root@laika:~# atq
1      Wed Aug  1 22:01:00 2007 a root
2      Wed Aug  1 22:03:00 2007 a root
root@laika:~# at -l
1      Wed Aug  1 22:01:00 2007 a root
2      Wed Aug  1 22:03:00 2007 a root
root@laika:~#
```

The **at** command understands English words like tomorrow and teatime to schedule commands the next day and at four in the afternoon.

```
root@laika:~# at 10:05 tomorrow
at> sleep 100
at> <EOT>
job 5 at Thu Aug  2 10:05:00 2007
root@laika:~# at teatime tomorrow
at> tea
at> <EOT>
job 6 at Thu Aug  2 16:00:00 2007
root@laika:~# atq
6      Thu Aug  2 16:00:00 2007 a root
5      Thu Aug  2 10:05:00 2007 a root
root@laika:~#
```

51.1.3. atrm

Jobs in the at queue can be removed with **atrm**.

```
root@laika:~# atq
6          Thu Aug  2 16:00:00 2007 a root
5          Thu Aug  2 10:05:00 2007 a root
root@laika:~# atrm 5
root@laika:~# atq
6          Thu Aug  2 16:00:00 2007 a root
root@laika:~#
```

51.1.4. at.allow and at.deny

You can also use the **/etc/at.allow** and **/etc/at.deny** files to manage who can schedule jobs with at.

The **/etc/at.allow** file can contain a list of users that are allowed to schedule **at** jobs. When **/etc/at.allow** does not exist, then everyone can use **at** unless their username is listed in **/etc/at.deny**.

If none of these files exist, then everyone can use **at**.

51.2. cron

51.2.1. crontab file

The **crontab(1)** command can be used to maintain the **crontab(5)** file. Each user can have their own crontab file to schedule jobs at a specific time. This time can be specified with five fields in this order: minute, hour, day of the month, month and day of the week. If a field contains an asterisk (*), then this means all values of that field.

The following example means : run script42 eight minutes after two, every day of the month, every month and every day of the week.

```
8 14 * * * script42
```

Run script8472 every month on the first of the month at 25 past midnight.

```
25 0 1 * * script8472
```

Run this script33 every two minutes on Sunday (both 0 and 7 refer to Sunday).

```
*/2 * * * 0
```

Instead of these five fields, you can also type one of these: @reboot, @yearly or @annually, @monthly, @weekly, @daily or @midnight, and @hourly.

51.2.2. crontab command

Users should not edit the crontab file directly, instead they should type **crontab -e** which will use the editor defined in the EDITOR or VISUAL environment variable. Users can display their cron table with **crontab -l**.

51.2.3. cron.allow and cron.deny

The **cron daemon crond** is reading the cron tables, taking into account the **/etc/cron.allow** and **/etc/cron.deny** files.

These files work in the same way as **at.allow** and **at.deny**. When the **cron.allow** file exists, then your username has to be in it, otherwise you cannot use **cron**. When the **cron.allow** file does not exist, then your username cannot be in the **cron.deny** file if you want to use **cron**.

51.2.4. /etc/crontab

The **/etc/crontab** file contains entries for when to run hourly/daily/weekly/monthly tasks. It will look similar to this output.

```
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

20 3 * * *      root    run-parts --report /etc/cron.daily
40 3 * * 7      root    run-parts --report /etc/cron.weekly
55 3 1 * *      root    run-parts --report /etc/cron.monthly
```

51.2.5. /etc/cron.*

The directories shown in the next screenshot contain the tasks that are run at the times scheduled in **/etc/crontab**. The **/etc/cron.d** directory is for special cases, to schedule jobs that require finer control than hourly/daily/weekly/monthly.

```
paul@laika:~$ ls -ld /etc/cron.*
drwxr-xr-x 2 root root 4096 2008-04-11 09:14 /etc/cron.d
drwxr-xr-x 2 root root 4096 2008-04-19 15:04 /etc/cron.daily
drwxr-xr-x 2 root root 4096 2008-04-11 09:14 /etc/cron.hourly
drwxr-xr-x 2 root root 4096 2008-04-11 09:14 /etc/cron.monthly
drwxr-xr-x 2 root root 4096 2008-04-11 09:14 /etc/cron.weekly
```

51.2.6. /etc/cron.*

Note that Red Hat uses **anacron** to schedule daily, weekly and monthly cron jobs.

```
root@rhel65:/etc# cat anacrontab
# /etc/anacrontab: configuration file for anacron

# See anacron(8) and anacrontab(5) for details.

SHELL=/bin/sh
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
# the maximal random delay added to the base delay of the jobs
RANDOM_DELAY=45
# the jobs will be started during the following hours only
START_HOURS_RANGE=3-22

#period in days   delay in minutes   job-identifier   command
1                5                cron.daily      nice run-parts /etc/cron.daily
7                25               cron.weekly      nice run-parts /etc/cron.weekly
@monthly 45       cron.monthly      nice run-parts /etc/cron.monthly
root@rhel65:/etc#
```

51.3. practice : scheduling

1. Schedule two jobs with **at**, display the **at queue** and remove a job.
2. As normal user, use **crontab -e** to schedule a script to run every four minutes.
3. As root, display the **crontab** file of your normal user.
4. As the normal user again, remove your **crontab** file.
5. Take a look at the **cron** files and directories in **/etc** and understand them. What is the **run-parts** command doing ?

51.4. solution : scheduling

1. Schedule two jobs with **at**, display the **at queue** and remove a job.

```
root@rhel55 ~# at 9pm today
at> echo go to bed >> /root/todo.txt
at> <EOT>
job 1 at 2010-11-14 21:00
root@rhel55 ~# at 17h31 today
at> echo go to lunch >> /root/todo.txt
at> <EOT>
job 2 at 2010-11-14 17:31
root@rhel55 ~# atq
2 2010-11-14 17:31 a root
1 2010-11-14 21:00 a root
root@rhel55 ~# atrm 1
root@rhel55 ~# atq
2 2010-11-14 17:31 a root
root@rhel55 ~# date
Sun Nov 14 17:31:01 CET 2010
root@rhel55 ~# cat /root/todo.txt
go to lunch
```

2. As normal user, use **crontab -e** to schedule a script to run every four minutes.

```
paul@rhel55 ~$ crontab -e
no crontab for paul - using an empty one
crontab: installing new crontab
```

3. As root, display the **crontab** file of your normal user.

```
root@rhel55 ~# crontab -l -u paul
*/4 * * * * echo `date` >> /home/paul/crontest.txt
```

4. As the normal user again, remove your **crontab** file.

```
paul@rhel55 ~$ crontab -r
paul@rhel55 ~$ crontab -l
no crontab for paul
```

5. Take a look at the **cron** files and directories in **/etc** and understand them. What is the **run-parts** command doing ?

```
run-parts runs a script in a directory
```

Chapter 52. logging

This chapter has three distinct subjects.

First we look at login logging ; how can we find out who is logging in to the system, when and from where. And who is not logging in, who fails at **su** or **ssh**.

Second we discuss how to configure the syslog daemon, and how to test it with **logger**.

The last part is mostly about **rotating logs** and mentions the **tail -f** and **watch** commands for **watching logs**.

52.1. login logging

To keep track of who is logging into the system, Linux can maintain the `/var/log/wtmp`, `/var/log/btmp`, `/var/run/utmp` and `/var/log/lastlog` files.

52.1.1. `/var/run/utmp` (who)

Use the **who** command to see the `/var/run/utmp` file. This command is showing you all the **currently** logged in users. Notice that the `utmp` file is in `/var/run` and not in `/var/log`.

```
[root@rhel4 ~]# who
paul      pts/1      Feb 14 18:21 (192.168.1.45)
sandra    pts/2      Feb 14 18:11 (192.168.1.42)
inge      pts/3      Feb 14 12:01 (192.168.1.33)
els       pts/4      Feb 14 14:33 (192.168.1.19)
```

52.1.2. `/var/log/wtmp` (last)

The `/var/log/wtmp` file is updated by the **login program**. Use **last** to see the `/var/run/wtmp` file.

```
[root@rhel4a ~]# last | head
paul      pts/1      192.168.1.45      Wed Feb 14 18:39   still logged in
reboot    system boot 2.6.9-42.0.8.ELs Wed Feb 14 18:21   (01:15)
nicolas   pts/5      pc-dss.telematic Wed Feb 14 12:32 - 13:06 (00:33)
stefaan   pts/3      pc-sde.telematic Wed Feb 14 12:28 - 12:40 (00:12)
nicolas   pts/3      pc-nae.telematic Wed Feb 14 11:36 - 12:21 (00:45)
nicolas   pts/3      pc-nae.telematic Wed Feb 14 11:34 - 11:36 (00:01)
dirk      pts/5      pc-dss.telematic Wed Feb 14 10:03 - 12:31 (02:28)
nicolas   pts/3      pc-nae.telematic Wed Feb 14 09:45 - 11:34 (01:48)
dimitri   pts/5      rhel4             Wed Feb 14 07:57 - 08:38 (00:40)
stefaan   pts/4      pc-sde.telematic Wed Feb 14 07:16 - down   (05:50)
[root@rhel4a ~]#
```

The `last` command can also be used to get a list of last reboots.

```
[paul@rekkie ~]$ last reboot
reboot    system boot 2.6.16-rekkie    Mon Jul 30 05:13   (370+08:42)

wtmp begins Tue May 30 23:11:45 2006
[paul@rekkie ~]
```

52.1.3. /var/log/lastlog (lastlog)

Use **lastlog** to see the /var/log/lastlog file.

```
[root@rhel4a ~]# lastlog | tail
tim          pts/5    10.170.1.122      Tue Feb 13 09:36:54 +0100 2007
rm           pts/6    rhel4             Tue Feb 13 10:06:56 +0100 2007
henk                                     **Never logged in**
stefaan      pts/3    pc-sde.telematic  Wed Feb 14 12:28:38 +0100 2007
dirk         pts/5    pc-dss.telematic  Wed Feb 14 10:03:11 +0100 2007
arsene                                     **Never logged in**
nicolas      pts/5    pc-dss.telematic  Wed Feb 14 12:32:18 +0100 2007
dimitri      pts/5    rhel4             Wed Feb 14 07:57:19 +0100 2007
bashuserm    pts/7    rhel4             Tue Feb 13 10:35:40 +0100 2007
kornuserm    pts/5    rhel4             Tue Feb 13 10:06:17 +0100 2007
[root@rhel4a ~]#
```

52.1.4. /var/log/btmp (lastb)

There is also the **lastb** command to display the /var/log/btmp file. This file is updated by the login program when entering the wrong password, so it contains failed login attempts. Many computers will not have this file, resulting in no logging of failed login attempts.

```
[root@RHEL4b ~]# lastb
lastb: /var/log/btmp: No such file or directory
Perhaps this file was removed by the operator to prevent logging lastb\
info.
[root@RHEL4b ~]#
```

The reason given for this is that users sometimes type their password by mistake instead of their login, so this world readable file poses a security risk. You can enable bad login logging by simply creating the file. Doing a `chmod o-r /var/log/btmp` improves security.

```
[root@RHEL4b ~]# touch /var/log/btmp
[root@RHEL4b ~]# ll /var/log/btmp
-rw-r--r-- 1 root root 0 Jul 30 06:12 /var/log/btmp
[root@RHEL4b ~]# chmod o-r /var/log/btmp
[root@RHEL4b ~]# lastb

btmp begins Mon Jul 30 06:12:19 2007
[root@RHEL4b ~]#
```

Failed logins via ssh, rlogin or su are not registered in /var/log/btmp. Failed logins via tty are.

```
[root@RHEL4b ~]# lastb
HalvarFl tty3          Mon Jul 30 07:10 - 07:10 (00:00)
Maria tty1             Mon Jul 30 07:09 - 07:09 (00:00)
Roberto tty1           Mon Jul 30 07:09 - 07:09 (00:00)

btmp begins Mon Jul 30 07:09:32 2007
[root@RHEL4b ~]#
```

52.1.5. su and ssh logins

Depending on the distribution, you may also have the **/var/log/secure** file being filled with messages from the auth and/or authpriv syslog facilities. This log will include su and/or ssh failed login attempts. Some distributions put this in **/var/log/auth.log**, verify the syslog configuration.

```
[root@RHEL4b ~]# cat /var/log/secure
Jul 30 07:09:03 sshd[4387]: Accepted publickey for paul from ::ffff:19\
2.168.1.52 port 33188 ssh2
Jul 30 05:09:03 sshd[4388]: Accepted publickey for paul from ::ffff:19\
2.168.1.52 port 33188 ssh2
Jul 30 07:22:27 sshd[4655]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 05:22:27 sshd[4656]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 07:22:30 sshd[4655]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 05:22:30 sshd[4656]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 07:22:33 sshd[4655]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 05:22:33 sshd[4656]: Failed password for Hermione from ::ffff:1\
92.168.1.52 port 38752 ssh2
Jul 30 08:27:33 sshd[5018]: Invalid user roberto from ::ffff:192.168.1\
.52
Jul 30 06:27:33 sshd[5019]: input_userauth_request: invalid user rober\
to
Jul 30 06:27:33 sshd[5019]: Failed none for invalid user roberto from \
::ffff:192.168.1.52 port 41064 ssh2
Jul 30 06:27:33 sshd[5019]: Failed publickey for invalid user roberto \
from ::ffff:192.168.1.52 port 41064 ssh2
Jul 30 08:27:36 sshd[5018]: Failed password for invalid user roberto f\
rom ::ffff:192.168.1.52 port 41064 ssh2
Jul 30 06:27:36 sshd[5019]: Failed password for invalid user roberto f\
rom ::ffff:192.168.1.52 port 41064 ssh2
[root@RHEL4b ~]#
```

You can enable this yourself, with a custom log file by adding the following line to **syslog.conf**.

```
auth.*,authpriv.* /var/log/customsec.log
```

52.2. syslogd

52.2.1. about syslog

The standard method of logging on Linux was through the **syslogd** daemon. Syslog was developed by **Eric Allman** for sendmail, but quickly became a standard among many Unix applications and was much later written as rfc 3164. The syslog daemon can receive messages on udp **port 514** from many applications (and appliances), and can append to log files, print, display messages on terminals and forward logs to other syslogd daemons on other machines. The syslogd daemon is configured in **/etc/syslog.conf**.

52.2.2. about rsyslog

The new method is called **reliable and extended syslogd** and uses the **rsyslogd** daemon and the **/etc/rsyslogd.conf** configuration file. The syntax is backwards compatible.

Each line in the configuration file uses a **facility** to determine where the message is coming from. It also contains a **priority** for the severity of the message, and an **action** to decide on what to do with the message.

52.2.3. modules

The new **rsyslog** has many more features that can be expanded by using modules. Modules allow for example exporting of syslog logging to a database.

See the manuals for more information (when you are done with this chapter).

```
root@rhel65:/etc# man rsyslog.conf
root@rhel65:/etc# man rsyslogd
root@rhel65:/etc#
```

52.2.4. facilities

The **man rsyslog.conf** command will explain the different default facilities for certain daemons, such as mail, lpr, news and kern(el) messages. The local0 to local7 facility can be used for appliances (or any networked device that supports syslog). Here is a list of all facilities for rsyslog.conf version 1.3. The security keyword is deprecated.

```
auth (security)
authpriv
cron
daemon
ftp
kern
lpr mail
mark (internal use only)
news
syslog
user
uucp
local0-7
```

52.2.5. priorities

The worst severity a message can have is **emerg** followed by **alert** and **crit**. Lowest priority should go to **info** and **debug** messages. Specifying a severity will also log all messages with a higher severity. You can prefix the severity with = to obtain only messages that match that severity. You can also specify **.none** to prevent a specific action from any message from a certain facility.

Here is a list of all priorities, in ascending order. The keywords warn, error and panic are deprecated.

```
debug
info
notice
warning (warn)
err (error)
crit
alert
emerg (panic)
```

52.2.6. actions

The default action is to send a message to the username listed as action. When the action is prefixed with a / then rsyslog will send the message to the file (which can be a regular file, but also a printer or terminal). The @ sign prefix will send the message on to another syslog server. Here is a list of all possible actions.

```
root,user1      list of users, separated by comma's
*              message to all logged on users
/              file (can be a printer, a console, a tty, ...)
-/            file, but don't sync after every write
|              named pipe
@              other syslog hostname
```

In addition, you can prefix actions with a - to omit syncing the file after every logging.

52.2.7. configuration

Below a sample configuration of custom local4 messages in **/etc/rsyslog.conf**.

```
local4.crit      /var/log/critandabove
local4.=crit     /var/log/onlycrit
local4.*         /var/log/alllocal4
```

52.2.8. restarting rsyslogd

Don't forget to restart the server after changing its configuration.

```
root@rhel65:/etc# service rsyslog restart
Shutting down system logger:          [ OK ]
Starting system logger:               [ OK ]
root@rhel65:/etc#
```

52.3. logger

The **logger** command can be used to generate syslog test messages. You can also use it in scripts. An example of testing syslogd with the **logger** tool.

```
[root@rhel4a ~]# logger -p local4.debug "l4 debug"
[root@rhel4a ~]# logger -p local4.crit "l4 crit"
[root@rhel4a ~]# logger -p local4.emerg "l4 emerg"
[root@rhel4a ~]#
```

The results of the tests with **logger**.

```
[root@rhel4a ~]# cat /var/log/critandabove
Feb 14 19:55:19 rhel4a paul: l4 crit
Feb 14 19:55:28 rhel4a paul: l4 emerg
[root@rhel4a ~]# cat /var/log/onlycrit
Feb 14 19:55:19 rhel4a paul: l4 crit
[root@rhel4a ~]# cat /var/log/alllocal4
Feb 14 19:55:11 rhel4a paul: l4 debug
Feb 14 19:55:19 rhel4a paul: l4 crit
Feb 14 19:55:28 rhel4a paul: l4 emerg
[root@rhel4a ~]#
```

52.4. watching logs

You might want to use the **tail -f** command to look at the last lines of a log file. The **-f** option will dynamically display lines that are appended to the log.

```
paul@ubu1010:~$ tail -f /var/log/udev
SEQNUM=1741
SOUND_INITIALIZED=1
ID_VENDOR_FROM_DATABASE=nVidia Corporation
ID_MODEL_FROM_DATABASE=MCP79 High Definition Audio
ID_BUS=pci
ID_VENDOR_ID=0x10de
ID_MODEL_ID=0x0ac0
ID_PATH=pci-0000:00:08.0
SOUND_FORM_FACTOR=internal
```

You can automatically repeat commands by preceding them with the **watch** command. When executing the following:

```
[root@rhel6 ~]# watch who
```

Something similar to this, repeating the output of the **who** command every two seconds, will appear on the screen.

```
Every 2.0s: who                Sun Jul 17 15:31:03 2011

root      tty1                2011-07-17 13:28
paul      pts/0                    2011-07-17 13:31 (192.168.1.30)
paul      pts/1                    2011-07-17 15:19 (192.168.1.30)
```

52.5. rotating logs

A lot of log files are always growing in size. To keep this within bounds, you may want to use **logrotate** to rotate, compress, remove and mail log files. More info on the logrotate command in **/etc/logrotate.conf**.. Individual configurations can be found in the **/etc/logrotate.d/** directory.

Below a screenshot of the default Red Hat logrotate.conf file.

```
root@rhel65:/etc# cat logrotate.conf
# see "man logrotate" for details
# rotate log files weekly
weekly

# keep 4 weeks worth of backlogs
rotate 4

# create new (empty) log files after rotating old ones
create

# use date as a suffix of the rotated file
dateext

# uncomment this if you want your log files compressed
#compress

# RPM packages drop log rotation information into this directory
include /etc/logrotate.d

# no packages own wtmp and btmp -- we'll rotate them here
/var/log/wtmp {
    monthly
    create 0664 root utmp
        minsize 1M
    rotate 1
}

/var/log/btmp {
    missingok
    monthly
    create 0600 root utmp
    rotate 1
}

# system-specific logs may be also be configured here.
root@rhel65:/etc#
```


52.6. practice : logging

1. Display the `/var/run/utmp` file with the proper command (not with `cat` or `vi`).
2. Display the `/var/log/wtmp` file.
3. Use the `lastlog` and `lastb` commands, understand the difference.
4. Examine `syslog` to find the location of the log file containing `ssh` failed logins.
5. Configure `syslog` to put `local4.error` and above messages in `/var/log/l4e.log` and `local4.info` only `.info` in `/var/log/l4i.log`. Test that it works with the `logger` tool!
6. Configure `/var/log/Mysu.log`, all the `su` to root messages should go in that log. Test that it works!
7. Send the `local5` messages to the `syslog` server of your neighbour. Test that it works.
8. Write a script that executes `logger` to `local4` every 15 seconds (different message). Use `tail -f` and `watch` on your `local4` log files.

52.7. solution : logging

1. Display the /var/run/utmp file.

```
who
```

2. Display the /var/log/wtmp file.

```
last
```

3. Use the lastlog and lastb commands, understand the difference.

```
lastlog : when users last logged on
```

```
lastb: failed (bad) login attempts
```

4. Examine syslog to find the location of the log file containing ssh failed logins.

Answer depends on whether you machine uses **syslog** or **rsyslog** (newer).

```
[root@rhel53 ~]# grep authpriv /etc/syslog.conf
authpriv.* /var/log/secure
```

```
[root@rhel71 ~]# grep ^authpriv /etc/rsyslog.conf
authpriv.* /var/log/secure
```

```
paul@debian8:~$ grep ^auth /etc/rsyslog.conf
auth,authpriv.* /var/log/auth.log
```

5. Configure syslog to put local4.error and above messages in /var/log/l4e.log and local4.info only .info in /var/log/l4i.log. Test that it works with the logger tool!

With **syslog**:

```
echo local4.error /var/log/l4e.log >> /etc/syslog.conf
echo local4.=info /var/log/l4i.log >> /etc/syslog.conf
service syslog restart
```

With **rsyslog**:

```
echo local4.error /var/log/l4e.log >> /etc/rsyslog.conf
echo local4.=info /var/log/l4i.log >> /etc/rsyslog.conf
service rsyslog restart
```

On both:

```
logger -p local4.error "l4 error test"
logger -p local4.alert "l4 alert test"
logger -p local4.info "l4 info test"
cat /var/log/l4e.log
cat /var/log/l4i.log
```

6. Configure /var/log/Mysu.log, all the su to root messages should go in that log. Test that it works!

```
echo authpriv.* /var/log/Mysu.log >> /etc/syslog.conf
```

This will log more than just the **su** usage.

7. Send the local5 messages to the syslog server of your neighbour. Test that it works.

On RHEL5, edit **/etc/sysconfig/syslog** to enable remote listening on the server.

On RHEL7, uncomment these two lines in **/etc/rsyslog.conf** to enable 'UDP syslog reception'.

```
# Provides UDP syslog reception
$ModLoad imudp
$UDPServerRun 514
```

On Debian/Ubuntu edit **/etc/default/syslog** or **/etc/default/rsyslog**.

on the client: `logger -p local5.info "test local5 to neighbour"`

8. Write a script that executes logger to local4 every 15 seconds (different message). Use tail -f and watch on your local4 log files.

```
root@rhel53 scripts# cat logloop
#!/bin/bash

for i in `seq 1 10`
do
logger -p local4.info "local4.info test number $i"
sleep 15
done

root@rhel53 scripts# chmod +x logloop
root@rhel53 scripts# ./logloop &
[1] 8264
root@rhel53 scripts# tail -f /var/log/local4.all.log
Mar 28 13:13:36 rhel53 root: local4.info test number 1
Mar 28 13:13:51 rhel53 root: local4.info test number 2
...
```

Chapter 53. memory management

This chapter will tell you how to manage RAM memory and cache.

We start with some simple tools to display information about memory: **free -om**, **top** and **cat /proc/meminfo**.

We continue with managing swap space, using terms like **swapping**, **paging** and **virtual memory**.

The last part is about using **vmstat** to monitor swap usage.

53.1. displaying memory and cache

53.1.1. /proc/meminfo

Displaying **/proc/meminfo** will tell you a lot about the memory on your Linux computer.

```
paul@ubu1010:~$ cat /proc/meminfo
MemTotal:       3830176 kB
MemFree:        244060 kB
Buffers:        41020 kB
Cached:         2035292 kB
SwapCached:     9892 kB
...
```

The first line contains the total amount of physical RAM, the second line is the unused RAM. **Buffers** is RAM used for buffering files, **cached** is the amount of RAM used as cache and **SwapCached** is the amount of swap used as cache. The file gives us much more information outside of the scope of this course.

53.1.2. free

The **free** tool can display the information provided by **/proc/meminfo** in a more readable format. The example below displays brief memory information in megabytes.

```
paul@ubu1010:~$ free -om
              total        used        free      shared    buffers     cached
Mem:           3740         3519          221           0           42          1994
Swap:          6234           82         6152
```

53.1.3. top

The **top** tool is often used to look at processes consuming most of the cpu, but it also displays memory information on line four and five (which can be toggled by pressing **m**).

Below a screenshot of **top** on the same ubu1010 from above.

```
top - 10:44:34 up 16 days, 9:56, 6 users, load average: 0.13, 0.09, 0.12
Tasks: 166 total,  1 running, 165 sleeping,  0 stopped,  0 zombie
Cpu(s):  5.1%us,  4.6%sy,  0.6%ni, 88.7%id,  0.8%wa,  0.0%hi,  0.3%si,  0.0%st
Mem:    3830176k total, 3613720k used,  216456k free,  45452k buffers
Swap:   6384636k total,  84988k used, 6299648k free, 2050948k cached
```

53.2. managing swap space

53.2.1. about swap space

When the operating system needs more memory than physically present in RAM, it can use **swap space**. Swap space is located on slower but cheaper memory. Notice that, although hard disks are commonly used for swap space, their access times are one hundred thousand times slower.

The swap space can be a file, a partition, or a combination of files and partitions. You can see the swap space with the **free** command, or with **cat /proc/swaps**.

```
paul@ubu1010:~$ free -o | grep -v Mem
              total        used        free      shared    buffers     cached
Swap:      6384636      84988     6299648
paul@ubu1010:~$ cat /proc/swaps
Filename                                Type              Size      Used   Priority
/dev/sda3                              partition         6384636   84988   -1
```

The amount of swap space that you need depends heavily on the services that the computer provides.

53.2.2. creating a swap partition

You can activate or deactivate swap space with the **swapon** and **swapoff** commands. New swap space can be created with the **mkswap** command. The screenshot below shows the creation and activation of a swap partition.

```
root@RHELv4u4:~# fdisk -l 2> /dev/null | grep hda
Disk /dev/hda: 536 MB, 536870912 bytes
/dev/hda1            1            1040        524128+   83   Linux
root@RHELv4u4:~# mkswap /dev/hda1
Setting up swapspace version 1, size = 536702 kB
root@RHELv4u4:~# swapon /dev/hda1
```

Now you can see that **/proc/swaps** displays all swap spaces separately, whereas the **free -om** command only makes a human readable summary.

```
root@RHELv4u4:~# cat /proc/swaps
Filename                                Type              Size      Used   Priority
/dev/mapper/VolGroup00-LogVol01        partition         1048568    0      -1
/dev/hda1                              partition         524120     0      -2
root@RHELv4u4:~# free -om
              total        used        free      shared    buffers     cached
Mem:           249          245           4           0          125          54
Swap:          1535           0          1535
```

53.2.3. creating a swap file

Here is one more example showing you how to create a **swap file**. On Solaris you can use **mkfile** instead of **dd**.

```
root@RHELv4u4:~# dd if=/dev/zero of=/smallswapfile bs=1024 count=4096
4096+0 records in
4096+0 records out
root@RHELv4u4:~# mkswap /smallswapfile
Setting up swapspace version 1, size = 4190 kB
root@RHELv4u4:~# swapon /smallswapfile
root@RHELv4u4:~# cat /proc/swaps
```

Filename	Type	Size	Used	Priority
/dev/mapper/VolGroup00-LogVol01	partition	1048568	0	-1
/dev/hda1	partition	524120	0	-2
/smallswapfile	file	4088	0	-3

53.2.4. swap space in /etc/fstab

If you like these swaps to be permanent, then don't forget to add them to **/etc/fstab**. The lines in **/etc/fstab** will be similar to the following.

/dev/hda1	swap	swap	defaults	0 0
/smallswapfile	swap	swap	defaults	0 0

53.3. monitoring memory with vmstat

You can find information about **swap usage** using **vmstat**.

Below a simple **vmstat** displaying information in megabytes.

```
paul@ubu1010:~$ vmstat -S m
procs -----memory----- ---swap-- -----io---- -system- ----cpu----
 r  b  swpd  free  buff  cache  si   so   bi    bo    in   cs  us  sy  id  wa
 0   0    87   225   46  2097   0    0    2     5   14   8  6  5  89  1
```

Below a sample **vmstat** when (in another terminal) root launches a **find /**. It generates a lot of disk i/o (bi and bo are disk blocks in and out). There is no need for swapping here.

```
paul@ubu1010:~$ vmstat 2 100
procs -----memory----- ---swap-- -----io---- -system- ----cpu----
 r  b  swpd  free  buff  cache  si   so   bi    bo    in   cs  us  sy  id  wa
 0   0  84984 1999436 53416 269536   0    0    2     5    2   10  6  5  89  1
 0   0  84984 1999428 53416 269564   0    0    0     0  1713 2748  4  4  92  0
 0   0  84984 1999552 53416 269564   0    0    0     0  1672 1838  4  6  90  0
 0   0  84984 1999552 53424 269560   0    0    0    14  1587 2526  5  7  87  2
 0   0  84984 1999180 53424 269580   0    0    0   100  1748 2193  4  6  91  0
 1   0  84984 1997800 54508 269760   0    0   610    0  1836 3890 17 10  68  4
 1   0  84984 1994620 55040 269748   0    0   250   168  1724 4365 19 17  56  9
 0   1  84984 1978508 55292 269704   0    0   126    0  1957 2897 19 18  58  4
 0   0  84984 1974608 58964 269784   0    0  1826   478  2605 4355  7  7  44  41
 0   2  84984 1971260 62268 269728   0    0  1634   756  2257 3865  7  7  47  39
```

Below a sample **vmstat** when executing (on RHEL6) a simple memory leaking program. Now you see a lot of memory being swapped (si is 'swapped in').

```
[paul@rhel6c ~]$ vmstat 2 100
procs -----memory----- ---swap-- -----io---- --system-- ----cpu----
 r  b  swpd  free  buff  cache  si   so   bi    bo    in   cs  us  sy  id  wa  st
 0   3 245208  5280   232  1916  261    0    0    42   27   21  0  1  98  1  0
 0   2 263372  4800    72   908 143840 128    0  1138  462  191  2 10  0  88  0
 1   3 350672  4792    56   992 169280 256    0  1092  360  142  1 13  0  86  0
 1   4 449584  4788    56  1024 95880  64    0   606  471  191  2 13  0  85  0
 0   4 471968  4828    56  1140 44832  80    0   390  235   90  2 12  0  87  0
 3   5 505960  4764    56  1136 68008  16    0   538  286  109  1 12  0  87  0
```

The code below was used to simulate a memory leak (and force swapping). This code was found on wikipedia without author.

```
paul@mac:~$ cat memleak.c
#include <stdlib.h>

int main(void)
{
    while (malloc(50));
    return 0;
}
```


53.4. practice : memory

1. Use **dmesg** to find the total amount of memory in your computer.
2. Use **free** to display memory usage in kilobytes (then in megabytes).
3. On a virtual machine, create a swap partition (you might need an extra virtual disk for this).
4. Add a 20 megabyte swap file to the system.
5. Put all swap spaces in **/etc/fstab** and activate them. Test with a reboot that they are mounted.
6. Use **free** to verify usage of current swap.
7. (optional) Display the usage of swap with **vmstat** and **free -s** during a memory leak.

53.5. solution : memory

1. Use **dmesg** to find the total amount of memory in your computer.

```
dmesg | grep Memory
```

2. Use **free** to display memory usage in kilobytes (then in megabytes).

```
free ; free -m
```

3. On a virtual machine, create a swap partition (you might need an extra virtual disk for this).

```
mkswap /dev/sdd1 ; swapon /dev/sdd1
```

4. Add a 20 megabyte swap file to the system.

```
dd if=/dev/zero of=/swapfile20mb bs=1024 count=20000  
mkswap /swapfile20mb  
swapon /swapfile20mb
```

5. Put all swap spaces in **/etc/fstab** and activate them. Test with a reboot that they are mounted.

```
root@computer# tail -2 /etc/fstab  
/dev/sdd1      swap swap defaults 0 0  
/swapfile20mb swap swap defaults 0 0
```

6. Use **free** to verify usage of current swap.

```
free -om
```

7. (optional) Display the usage of swap with **vmstat** and **free -s** during a memory leak.

Chapter 54. resource monitoring

Monitoring is the process of obtaining information about the utilization of memory, cpu, bandwidth and storage. You should start monitoring your system as soon as possible, to be able to create a **baseline**. Make sure that you get to know your system! This baseline is important because it allows you to see a steady or sudden growth in **resource utilization** and likewise steady (or sudden) decline in **resource availability**. It will allow you to plan for scaling up or scaling out.

Let us look at some tools that go beyond **ps fax**, **df -h**, **free -om** and **du -sh**.

54.1. four basic resources

The four basic resources to monitor are:

- cpu
- network
- ram memory
- storage

54.2. top

To start monitoring, you can use **top**. This tool will monitor ram memory, cpu and swap. Top will automatically refresh. Inside **top** you can use many commands, like **k** to kill processes, or **t** and **m** to toggle displaying task and memory information, or the number **1** to have one line per cpu, or one summary line for all cpu's.

```
top - 12:23:16 up 2 days, 4:01, 2 users, load average: 0.00, 0.00, 0.00
Tasks: 61 total, 1 running, 60 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.3% us, 0.5% sy, 0.0% ni, 98.9% id, 0.2% wa, 0.0% hi, 0.0% si
Mem: 255972k total, 240952k used, 15020k free, 59024k buffers
Swap: 524280k total, 144k used, 524136k free, 112356k cached
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	16	0	2816	560	480	S	0.0	0.2	0:00.91	init
2	root	34	19	0	0	0	S	0.0	0.0	0:00.01	ksoftirqd/0
3	root	5	-10	0	0	0	S	0.0	0.0	0:00.57	events/0
4	root	5	-10	0	0	0	S	0.0	0.0	0:00.00	khelper
5	root	15	-10	0	0	0	S	0.0	0.0	0:00.00	kacpid
16	root	5	-10	0	0	0	S	0.0	0.0	0:00.08	kblockd/0
26	root	15	0	0	0	0	S	0.0	0.0	0:02.86	pdflush
...											

You can customize top to display the columns of your choice, or to display only the processes that you find interesting.

```
[paul@RHELv4u3 ~]$ top p 3456 p 8732 p 9654
```

54.3. free

The **free** command is common on Linux to monitor free memory. You can use free to display information every x seconds, but the output is not ideal.

```
[paul@RHELv4u3 gen]$ free -om -s 10
total      used      free      shared    buffers    cached
Mem:       249        222         27          0         50        109
Swap:      511          0        511
total      used      free      shared    buffers    cached
Mem:       249        222         27          0         50        109
Swap:      511          0        511

[paul@RHELv4u3 gen]$
```

54.4. watch

It might be more interesting to combine `free` with the **watch** program. This program can run commands with a delay, and can highlight changes (with the `-d` switch).

```
[paul@RHELv4u3 ~]$ watch -d -n 3 free -om
...
Every 3.0s: free -om                               Sat Jan 27 12:13:03 2007

total          used          free        shared    buffers         cached
Mem:           249           230           19           0           56           109
Swap:          511             0          511
```

54.5. vmstat

To monitor CPU, disk and memory statistics in one line there is **vmstat**. The screenshot below shows `vmstat` running every two seconds 100 times (or until the Ctrl-C). Below the `r`, you see the number of processes waiting for the CPU, sleeping processes go below `b`. Swap usage (`swpd`) stayed constant at 144 kilobytes, free memory dropped from 16.7MB to 12.9MB. See `man vmstat` for the rest.

```
[paul@RHELv4u3 ~]$ vmstat 2 100
procs -----memory----- --swap--  ---io---  --system--  ---cpu----
r  b  swpd   free   buff  cache   si   so   bi   bo   in    cs  us  sy  id  wa
0  0   144  16708  58212 111612    0    0    3    4   75    62  0   1  99   0
0  0   144  16708  58212 111612    0    0    0    0  976    22  0   0 100   0
0  0   144  16708  58212 111612    0    0    0    0  958    14  0   1  99   0
1  0   144  16528  58212 111612    0    0    0   18 1432   7417  1  32  66   0
1  0   144  16468  58212 111612    0    0    0    0 2910  20048  4  95   1   0
1  0   144  16408  58212 111612    0    0    0    0 3210  19509  4  97   0   0
1  0   144  15568  58816 111612    0    0  300 1632 2423 10189  2  62   0  36
0  1   144  13648  60324 111612    0    0  754    0 1910   2843  1  27   0  72
0  0   144  12928  60948 111612    0    0  312  418 1346   1258  0  14  57  29
0  0   144  12928  60948 111612    0    0    0    0  977    19  0   0 100   0
0  0   144  12988  60948 111612    0    0    0    0  977    15  0   0 100   0
0  0   144  12988  60948 111612    0    0    0    0  978    18  0   0 100   0

[paul@RHELv4u3 ~]$
```

54.6. iostat

The **iostat** tool can display disk and cpu statistics. The **-d** switch below makes iostat only display disk information (500 times every two seconds). The first block displays statistics since the last reboot.

```
[paul@RHELv4u3 ~]$ iostat -d 2 500
Linux 2.6.9-34.EL (RHELv4u3.localdomain)      01/27/2007

Device:            tps    Blk_read/s    Blk_wrtn/s    Blk_read    Blk_wrtn
hdc                 0.00         0.01         0.00         1080         0
sda                 0.52         5.07         7.78        941798       1445148
sda1                0.00         0.01         0.00         968          4
sda2                1.13         5.06         7.78       939862       1445144
dm-0                1.13         5.05         7.77       939034       1444856
dm-1                0.00         0.00         0.00         360         288

Device:            tps    Blk_read/s    Blk_wrtn/s    Blk_read    Blk_wrtn
hdc                 0.00         0.00         0.00          0          0
sda                 0.00         0.00         0.00          0          0
sda1                0.00         0.00         0.00          0          0
sda2                0.00         0.00         0.00          0          0
dm-0                0.00         0.00         0.00          0          0
dm-1                0.00         0.00         0.00          0          0
...
[paul@RHELv4u3 ~]$
```

You can have more statistics using **iostat -d -x**, or display only cpu statistics with **iostat -c**.

```
[paul@RHELv4u3 ~]$ iostat -c 5 500
Linux 2.6.9-34.EL (RHELv4u3.localdomain)      01/27/2007

avg-cpu:  %user   %nice    %sys %iowait    %idle
0.31    0.02    0.52    0.23   98.92

avg-cpu:  %user   %nice    %sys %iowait    %idle
0.62    0.00   52.16   47.23    0.00

avg-cpu:  %user   %nice    %sys %iowait    %idle
2.92    0.00   36.95   60.13    0.00

avg-cpu:  %user   %nice    %sys %iowait    %idle
0.63    0.00   36.63   62.32    0.42

avg-cpu:  %user   %nice    %sys %iowait    %idle
0.00    0.00    0.20    0.20   99.59

[paul@RHELv4u3 ~]$
```

54.7. mpstat

On multi-processor machines, **mpstat** can display statistics for all, or for a selected cpu.

```
paul@laika:~$ mpstat -P ALL
Linux 2.6.20-3-generic (laika) 02/09/2007

CPU %user   %nice    %sys %iowait    %irq   %soft  %steal   %idle   intr/s
all  1.77    0.03    1.37    1.03    0.02   0.39    0.00   95.40  1304.91
  0  1.73    0.02    1.47    1.93    0.04   0.77    0.00   94.04  1304.91
  1  1.81    0.03    1.27    0.13    0.00   0.00    0.00   96.76    0.00
paul@laika:~$
```

54.8. sadc and sar

The **sadc** tool writes system utilization data to **/var/log/sa/sa??**, where ?? is replaced with the current day of the month. By default, cron runs the **sa1** script every 10 minutes, the **sa1** script runs **sadc** for one second. Just before midnight every day, cron runs the **sa2** script, which in turn invokes **sar**. The **sar** tool will read the daily data generated by **sadc** and put it in **/var/log/sa/sar??**. These **sar reports** contain a lot of statistics.

You can also use **sar** to display a portion of the statistics that were gathered. Like this example for cpu statistics.

```
[paul@RHELv4u3 sa]$ sar -u | head
Linux 2.6.9-34.EL (RHELv4u3.localdomain) 01/27/2007

12:00:01 AM      CPU      %user      %nice    %system    %iowait    %idle
12:10:01 AM      all       0.48       0.01       0.60       0.04     98.87
12:20:01 AM      all       0.49       0.01       0.60       0.06     98.84
12:30:01 AM      all       0.49       0.01       0.64       0.25     98.62
12:40:02 AM      all       0.44       0.01       0.62       0.07     98.86
12:50:01 AM      all       0.42       0.01       0.60       0.10     98.87
01:00:01 AM      all       0.47       0.01       0.65       0.08     98.80
01:10:01 AM      all       0.45       0.01       0.68       0.08     98.78
[paul@RHELv4u3 sa]$
```

There are other useful **sar** options, like **sar -I PROC** to display interrupt activity per interrupt and per CPU, or **sar -r** for memory related statistics. Check the manual page of **sar** for more.

54.9. ntop

The **ntop** tool is not present in default Red Hat installs. Once run, it will generate a very extensive analysis of network traffic in html on <http://localhost:3000>.

54.10. iftop

The **iftop** tool will display bandwidth by socket statistics for a specific network device. Not available on default Red Hat servers.

```

1.91Mb      3.81Mb      5.72Mb      7.63Mb      9.54Mb
-----|-----|-----|-----|-----|-----|-----
laika.local => barry                                4.94Kb  6.65Kb  69.9Kb
              <=                                7.41Kb  16.4Kb  766Kb
laika.local => ik-in-f19.google.com                  0b    1.58Kb  14.4Kb
              <=                                0b    292b   41.0Kb
laika.local => ik-in-f99.google.com                  0b    83b    4.01Kb
              <=                                0b    83b    39.8Kb
laika.local => ug-in-f189.google.com                 0b    42b    664b
              <=                                0b    42b    406b
laika.local => 10.0.0.138                            0b    0b     149b
              <=                                0b    0b     256b
laika.local => 224.0.0.251                          0b    0b     86b
              <=                                0b    0b     0b
laika.local => ik-in-f83.google.com                  0b    0b     39b
              <=                                0b    0b     21b

```

54.11. iptraf

Use **iptraf** for a colourful display of ip traffic over the network cards.

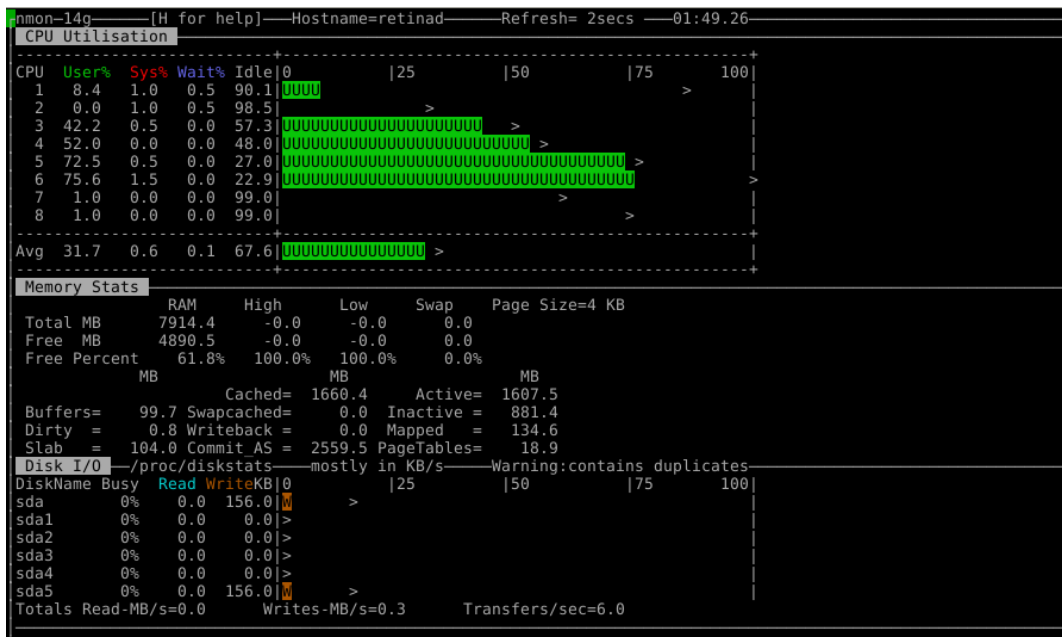
```

[root@centos65 ~]# iptraf
[root@centos65 ~]# iptraf -i eth0

```

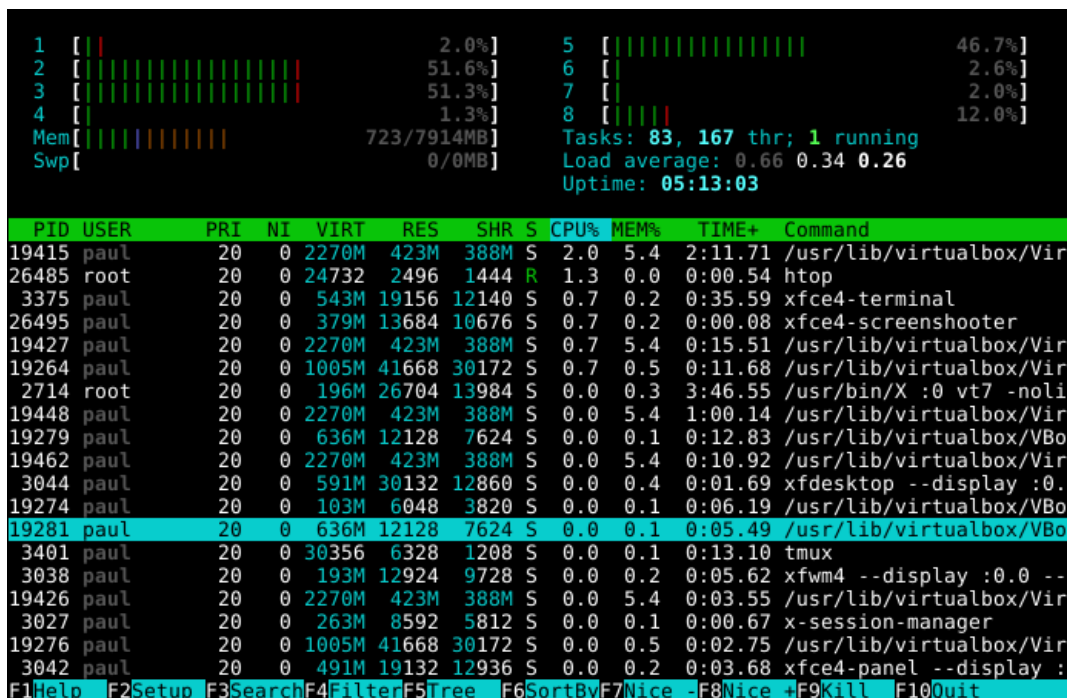

54.12. nmon

Another popular and all round tool is **nmon**.



54.13. htop

You can use **htop** instead of top.



Chapter 55. package management

Most Linux distributions have a **package management** system with online **repositories** containing thousands of packages. This makes it very easy to install and remove applications, operating system components, documentation and much more.

We first discuss the Debian package format **.deb** and its tools **dpkg**, **apt-get** and **aptitude**. This should be similar on Debian, Ubuntu, Mint and all derived distributions.

Then we look at the Red Hat package format **.rpm** and its tools **rpm** and **yum**. This should be similar on Red Hat, Fedora, CentOS and all derived distributions.

55.1. package terminology

55.1.1. repository

A lot of software and documentation for your Linux distribution is available as **packages** in one or more centrally distributed **repositories**. These **packages** in such a **repository** are tested and very easy to install (or remove) with a graphical or command line installer.

55.1.2. .deb packages

Debian, Ubuntu, Mint and all derivatives from Debian and Ubuntu use **.deb** packages. To manage software on these systems, you can use **aptitude** or **apt-get**, both these tools are a front end for **dpkg**.

55.1.3. .rpm packages

Red Hat, Fedora, CentOS, OpenSUSE, Mandriva, Red Flag and others use **.rpm** packages. The tools to manage software packages on these systems are **yum** and **rpm**.

55.1.4. dependency

Some packages need other packages to function. Tools like **apt-get**, **aptitude** and **yum** will install all **dependencies** you need. When using **dpkg** or **rpm**, or when building from **source**, you will need to install dependencies yourself.

55.1.5. open source

These repositories contain a lot of independent **open source software**. Often the source code is customized to integrate better with your distribution. Most distributions also offer this modified source code as a **package** in one or more **source repositories**.

You are free to go to the project website itself (samba.org, apache.org, github.com, ...) and download the **vanilla** (= without the custom distribution changes) source code.

55.1.6. GUI software management

End users have several graphical applications available via the desktop (look for 'add/remove software' or something similar).

Below a screenshot of Ubuntu Software Center running on Ubuntu 12.04. Graphical tools are not discussed in this book.



55.2. deb package management

55.2.1. about deb

Most people use **aptitude** or **apt-get** to manage their Debian/Ubuntu family of Linux distributions. Both are a front end for **dpkg** and are themselves a back end for **synaptic** and other graphical tools.

55.2.2. dpkg -l

The low level tool to work with **.deb** packages is **dpkg**. Here you see how to obtain a list of all installed packages on a Debian server.

```
root@debian6:~# dpkg -l | wc -l
265
```

Compare this to the same list on a Ubuntu Desktop computer.

```
root@ubu1204~# dpkg -l | wc -l
2527
```

55.2.3. dpkg -l \$package

Here is an example on how to get information on an individual package. The **ii** at the beginning means the package is installed.

```
root@debian6:~# dpkg -l rsync | tail -1 | tr -s ' '
ii rsync 3.0.7-2 fast remote file copy program (like rcp)
```

55.2.4. dpkg -S

You can find the package that installed a certain file on your computer with **dpkg -S**. This example shows how to find the package for three files on a typical Debian server.

```
root@debian6:~# dpkg -S /usr/share/doc/tmux/ /etc/ssh/ssh_config /sbin/ifconfig
tmux: /usr/share/doc/tmux/
openssh-client: /etc/ssh/ssh_config
net-tools: /sbin/ifconfig
```

55.2.5. dpkg -L

You can also get a list of all files that are installed by a certain program. Below is the list for the **tmux** package.

```
root@debian6:~# dpkg -L tmux
/.
/etc
/etc/init.d
/etc/init.d/tmux-cleanup
/usr
/usr/share
/usr/share/lintian
/usr/share/lintian/overrides
/usr/share/lintian/overrides/tmux
/usr/share/doc
```

```
/usr/share/doc/tmux
/usr/share/doc/tmux/TODO.gz
/usr/share/doc/tmux/FAQ.gz
/usr/share/doc/tmux/changelog.Debian.gz
/usr/share/doc/tmux/NEWS.Debian.gz
/usr/share/doc/tmux/changelog.gz
/usr/share/doc/tmux/copyright
/usr/share/doc/tmux/examples
/usr/share/doc/tmux/examples/tmux.vim.gz
/usr/share/doc/tmux/examples/h-boetes.conf
/usr/share/doc/tmux/examples/n-marriott.conf
/usr/share/doc/tmux/examples/screen-keys.conf
/usr/share/doc/tmux/examples/t-williams.conf
/usr/share/doc/tmux/examples/vim-keys.conf
/usr/share/doc/tmux/NOTES
/usr/share/man
/usr/share/man/man1
/usr/share/man/man1/tmux.1.gz
/usr/bin
/usr/bin/tmux
```

55.2.6. dpkg

You could use **dpkg -i** to install a package and **dpkg -r** to remove a package, but you'd have to manually keep track of dependencies. Using **apt-get** or **aptitude** is much easier.

55.3. apt-get

Debian has been using **apt-get** to manage packages since 1998. Today Debian and many Debian-based distributions still actively support **apt-get**, though some experts claim **aptitude** is better at handling dependencies than **apt-get**.

Both commands use the same configuration files and can be used alternately; whenever you see **apt-get** in documentation, feel free to type **aptitude**.

We will start with **apt-get** and discuss **aptitude** in the next section.

55.3.1. apt-get update

When typing **apt-get update** you are downloading the names, versions and short description of all packages available on all configured repositories for your system.

In the example below you can see some repositories at the url **be.archive.ubuntu.com** because this computer was installed in Belgium. This url can be different for you.

```
root@ubu1204~# apt-get update
Ign http://be.archive.ubuntu.com precise InRelease
Ign http://extras.ubuntu.com precise InRelease
Ign http://security.ubuntu.com precise-security InRelease
Ign http://archive.canonical.com precise InRelease
Ign http://be.archive.ubuntu.com precise-updates InRelease
...
Hit http://be.archive.ubuntu.com precise-backports/main Translation-en
Hit http://be.archive.ubuntu.com precise-backports/multiverse Translation-en
Hit http://be.archive.ubuntu.com precise-backports/restricted Translation-en
Hit http://be.archive.ubuntu.com precise-backports/universe Translation-en
Fetched 13.7 MB in 8s (1682 kB/s)
Reading package lists... Done
root@mac~#
```

Run **apt-get update** every time before performing other package operations.

55.3.2. apt-get upgrade

One of the nicest features of **apt-get** is that it allows for a secure update of **all software currently installed** on your computer with just **one** command.

```
root@debian6:~# apt-get upgrade
Reading package lists... Done
Building dependency tree
Reading state information... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@debian6:~#
```

The above screenshot shows that all software is updated to the latest version available for my distribution.

55.3.3. apt-get clean

apt-get keeps a copy of downloaded packages in **/var/cache/apt/archives**, as can be seen in this screenshot.

```

root@ubu1204~# ls /var/cache/apt/archives/ | head
accountsservice_0.6.15-2ubuntu9.4_i386.deb
apport_2.0.1-0ubuntu14_all.deb
apport-gtk_2.0.1-0ubuntu14_all.deb
apt_0.8.16~exp12ubuntu10.3_i386.deb
apt-transport-https_0.8.16~exp12ubuntu10.3_i386.deb
apt-utils_0.8.16~exp12ubuntu10.3_i386.deb
bind9-host_1%3a9.8.1.dfsg.P1-4ubuntu0.4_i386.deb
chromium-browser_20.0.1132.47~r144678-0ubuntu0.12.04.1_i386.deb
chromium-browser-l10n_20.0.1132.47~r144678-0ubuntu0.12.04.1_all.deb
chromium-codecs-ffmpeg_20.0.1132.47~r144678-0ubuntu0.12.04.1_i386.deb

```

Running **apt-get clean** removes all .deb files from that directory.

```

root@ubu1204~# apt-get clean
root@ubu1204~# ls /var/cache/apt/archives/*.deb
ls: cannot access /var/cache/apt/archives/*.deb: No such file or directory

```

55.3.4. apt-cache search

Use **apt-cache search** to search for availability of a package. Here we look for **rsync**.

```

root@ubu1204~# apt-cache search rsync | grep ^rsync
rsync - fast, versatile, remote (and local) file-copying tool
rsyncrypto - rsync friendly encryption

```

55.3.5. apt-get install

You can install one or more applications by appending their name behind **apt-get install**. The screenshot shows how to install the **rsync** package.

```

root@ubu1204~# apt-get install rsync
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  rsync
0 upgraded, 1 newly installed, 0 to remove and 8 not upgraded.
Need to get 299 kB of archives.
After this operation, 634 kB of additional disk space will be used.
Get:1 http://be.archive.ubuntu.com/ubuntu/ precise/main rsync i386 3.0.9-1ubuntu1 [299 kB]
Fetched 299 kB in 0s (740 kB/s)
Selecting previously unselected package rsync.
(Reading database ... 323649 files and directories currently installed.)
Unpacking rsync (from ../rsync_3.0.9-1ubuntu1_i386.deb) ...
Processing triggers for man-db ...
Processing triggers for ureadahead ...
Setting up rsync (3.0.9-1ubuntu1) ...
Removing any system startup links for /etc/init.d/rsync ...
root@ubu1204~#

```

55.3.6. apt-get remove

You can remove one or more applications by appending their name behind **apt-get remove**. The screenshot shows how to remove the **rsync** package.

```

root@ubu1204~# apt-get remove rsync
Reading package lists... Done
Building dependency tree
Reading state information... Done

```



```
The following packages will be REMOVED:
  rsync ubuntu-standard
0 upgraded, 0 newly installed, 2 to remove and 8 not upgraded.
After this operation, 692 kB disk space will be freed.
Do you want to continue [Y/n]?
(Reading database ... 323681 files and directories currently installed.)
Removing ubuntu-standard ...
Removing rsync ...
* Stopping rsync daemon rsync
Processing triggers for ureadahead ...
Processing triggers for man-db ...
root@ubu1204~#
```

Note however that some configuration information is not removed.

```
root@ubu1204~# dpkg -l rsync | tail -1 | tr -s ' '
rc rsync 3.0.9-1ubuntu1 fast, versatile, remote (and local) file-copying tool
```

55.3.7. apt-get purge

You can purge one or more applications by appending their name behind **apt-get purge**. Purging will also remove all existing configuration files related to that application. The screenshot shows how to purge the **rsync** package.

```
root@ubu1204~# apt-get purge rsync
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages will be REMOVED:
  rsync*
0 upgraded, 0 newly installed, 1 to remove and 8 not upgraded.
After this operation, 0 B of additional disk space will be used.
Do you want to continue [Y/n]?
(Reading database ... 323651 files and directories currently installed.)
Removing rsync ...
Purging configuration files for rsync ...
Processing triggers for ureadahead ...
root@ubu1204~#
```

Note that **dpkg** has no information about a purged package, except that it is uninstalled and no configuration is left on the system.

```
root@ubu1204~# dpkg -l rsync | tail -1 | tr -s ' '
un rsync <none> (no description available)
```

55.4. aptitude

Most people use **aptitude** for package management on Debian, Mint and Ubuntu systems.

To synchronize with the repositories.

```
aptitude update
```

To patch and upgrade all software to the latest version on Debian.

```
aptitude upgrade
```

To patch and upgrade all software to the latest version on Ubuntu and Mint.

```
aptitude safe-upgrade
```

To install an application with all dependencies.

```
aptitude install $package
```

To search the repositories for applications that contain a certain string in their name or description.

```
aptitude search $string
```

To remove an application.

```
aptitude remove $package
```

To remove an application and all configuration files.

```
aptitude purge $package
```

55.5. apt

Both **apt-get** and **aptitude** use the same configuration information in **/etc/apt/**. Thus adding a repository for one of them, will automatically add it for both.

55.5.1. /etc/apt/sources.list

The resource list used by **apt-get** and **aptitude** is located in **/etc/apt/sources.list**. This file contains a list of http or ftp sources where packages for the distribution can be downloaded.

This is what that list looks like on my Debian server.

```
root@debian6:~# cat /etc/apt/sources.list
deb http://ftp.be.debian.org/debian/ squeeze main
deb-src http://ftp.be.debian.org/debian/ squeeze main

deb http://security.debian.org/ squeeze/updates main
deb-src http://security.debian.org/ squeeze/updates main

# squeeze-updates, previously known as 'volatile'
deb http://ftp.be.debian.org/debian/ squeeze-updates main
deb-src http://ftp.be.debian.org/debian/ squeeze-updates main
```

On my Ubuntu there are four times as many online repositories in use.

```
root@ubu1204~# wc -l /etc/apt/sources.list
63 /etc/apt/sources.list
```

There is much more to learn about **apt**, explore commands like **add-apt-repository**, **apt-key** and **apt-get**.

55.6. rpm

55.6.1. about rpm

The **Red Hat package manager** can be used on the command line with **rpm** or in a graphical way going to Applications--System Settings--Add/Remove Applications. Type **rpm --help** to see some of the options.

Software distributed in the **rpm** format will be named **foo-version.platform.rpm** .

55.6.2. rpm -qa

To obtain a list of all installed software, use the **rpm -qa** command.

```
[root@RHEL52 ~]# rpm -qa | grep samba
system-config-samba-1.2.39-1.el5
samba-3.0.28-1.el5_2.1
samba-client-3.0.28-1.el5_2.1
samba-common-3.0.28-1.el5_2.1
```

55.6.3. rpm -q

To verify whether one package is installed, use **rpm -q**.

```
root@RHELV4u4:~# rpm -q gcc
gcc-3.4.6-3
root@RHELV4u4:~# rpm -q laika
package laika is not installed
```

55.6.4. rpm -Uvh

To install or upgrade a package, use the **-Uvh** switches. The **-U** switch is the same as **-i** for install, except that older versions of the software are removed. The **-vh** switches are for nicer output.

```
root@RHELV4u4:~# rpm -Uvh gcc-3.4.6-3
```

55.6.5. rpm -e

To remove a package, use the **-e** switch.

```
root@RHELV4u4:~# rpm -e gcc-3.4.6-3
```

rpm -e verifies dependencies, and thus will prevent you from accidentally erasing packages that are needed by other packages.

```
[root@RHEL52 ~]# rpm -e gcc-4.1.2-42.el5
error: Failed dependencies:
gcc = 4.1.2-42.el5 is needed by (installed) gcc-c++-4.1.2-42.el5.i386
gcc = 4.1.2-42.el5 is needed by (installed) gcc-gfortran-4.1.2-42.el5.i386
gcc is needed by (installed) systemtap-0.6.2-1.el5_2.2.i386
```

55.6.6. /var/lib/rpm

The **rpm** database is located at **/var/lib/rpm**. This database contains all meta information about packages that are installed (via rpm). It keeps track of all files, which enables complete removes of software.

55.6.7. rpm2cpio

We can use **rpm2cpio** to convert an **rpm** to a **cpio** archive.

```
[root@RHEL53 ~]# file kernel.src.rpm
kernel.src.rpm: RPM v3 src PowerPC kernel-2.6.18-92.1.13.el5
[root@RHEL53 ~]# rpm2cpio kernel.src.rpm > kernel.cpio
[root@RHEL53 ~]# file kernel.cpio
kernel.cpio: ASCII cpio archive (SVR4 with no CRC)
```

But why would you want to do this ?

Perhaps just to see of list of files in the **rpm** file.

```
[root@RHEL53 ~]# rpm2cpio kernel.src.rpm | cpio -t | head -5
COPYING.modules
Config.mk
Module.kabi_i686
Module.kabi_i686PAE
Module.kabi_i686xen
```

Or to extract one file from an **rpm** package.

```
[root@RHEL53 ~]# rpm2cpio kernel.src.rpm | cpio -iv Config.mk
Config.mk
246098 blocks
```

55.7. yum

55.7.1. about yum

The **Yellowdog Updater, Modified (yum)** is an easier command to work with **rpm** packages. It is installed by default on Fedora and Red Hat Enterprise Linux since version 5.2.

55.7.2. yum list

Issue **yum list available** to see a list of available packages. The **available** parameter is optional.

```
root@rhel65:/etc# yum list | wc -l
This system is receiving updates from Red Hat Subscription Management.
3935
root@rhel65:/etc#
```

Issue **yum list \$package** to get all versions (in different repositories) of one package.

```
[root@rhel55 ~]# yum list samba
Loaded plugins: rhnplugin, security
Installed Packages
samba.i386                3.0.33-3.28.el5          installed
Available Packages
samba.i386                3.0.33-3.29.el5_5        rhel-i386-server-5
```

55.7.3. yum search

To search for a package containing a certain string in the description or name use **yum search \$string**.

```
[root@rhel55 ~]# yum search gcc44
Loaded plugins: rhnplugin, security
===== Matched: gcc44 =====
gcc44.i386 : Preview of GCC version 4.4
gcc44-c++.i386 : C++ support for GCC version 4.4
gcc44-gfortran.i386 : Fortran support for GCC 4.4 previe
```

55.7.4. yum provides

To search for a package containing a certain file (you might need for compiling things) use **yum provides \$filename**.

```
root@rhel65:/etc# yum provides /usr/share/man/man5/passwd.5.gz
Loaded plugins: product-id, subscription-manager
This system is receiving updates from Red Hat Subscription Management.
rhel-6-server-cf-tools-1-rpms                | 2.8 kB      00:00
rhel-6-server-rpms                          | 3.7 kB      00:00
man-pages-3.22-12.el6.noarch : Man (manual) pages from the Linux Documenta...
Repo                : rhel-6-server-rpms
Matched from:
Filename           : /usr/share/man/man5/passwd.5.gz

man-pages-3.22-20.el6.noarch : Man (manual) pages from the Linux Documenta...
Repo                : rhel-6-server-rpms
Matched from:
Filename           : /usr/share/man/man5/passwd.5.gz

man-pages-3.22-17.el6.noarch : Man (manual) pages from the Linux Documenta...
Repo                : rhel-6-server-rpms
Matched from:
Filename           : /usr/share/man/man5/passwd.5.gz

man-pages-3.22-20.el6.noarch : Man (manual) pages from the Linux Documenta...
Repo                : installed
Matched from:
Other              : Provides-match: /usr/share/man/man5/passwd.5.gz

root@rhel65:/etc#
```

55.7.5. yum install

To install an application, use **yum install \$package**. Naturally **yum** will install all the necessary dependencies.

```
[root@rhel55 ~]# yum install sudo
Loaded plugins: rhnplugin, security
Setting up Install Process
Resolving Dependencies
--> Running transaction check
---> Package sudo.i386 0:1.7.2p1-7.el5_5 set to be updated
--> Finished Dependency Resolution

Dependencies Resolved

=====
Package      Arch      Version      Repository      Size
=====
Installing:
sudo         i386      1.7.2p1-7.el5_5  rhel-i386-server-5  230 k

Transaction Summary
=====
Install      1 Package(s)
Upgrade      0 Package(s)

Total download size: 230 k
Is this ok [y/N]: y
Downloading Packages:
sudo-1.7.2p1-7.el5_5.i386.rpm          | 230 kB      00:00
Running rpm_check_debug
Running Transaction Test
Finished Transaction Test
Transaction Test Succeeded
Running Transaction
  Installing      : sudo                                1/1

Installed:
sudo.i386 0:1.7.2p1-7.el5_5

Complete!
```

You can add more than one parameter here.

```
yum install $package1 $package2 $package3
```


55.7.6. yum update

To bring all applications up to date, by downloading and installing them, issue **yum update**. All software that was installed via **yum** will be updated to the latest version that is available in the repository.

```
yum update
```

If you only want to update one package, use **yum update \$package**.

```
[root@rhel55 ~]# yum update sudo
Loaded plugins: rhnplugin, security
Skipping security plugin, no data
Setting up Update Process
Resolving Dependencies
Skipping security plugin, no data
--> Running transaction check
---> Package sudo.i386 0:1.7.2p1-7.el5_5 set to be updated
--> Finished Dependency Resolution

Dependencies Resolved

=====
Package           Arch      Version           Repository          Size
=====
Updating:
sudo              i386      1.7.2p1-7.el5_5   rhel-i386-server-5 230 k

Transaction Summary
=====
Install           0 Package(s)
Upgrade           1 Package(s)

Total download size: 230 k
Is this ok [y/N]: y
Downloading Packages:
sudo-1.7.2p1-7.el5_5.i386.rpm | 230 kB    00:00
Running rpm_check_debug
Running Transaction Test
Finished Transaction Test
Transaction Test Succeeded
Running Transaction
  Updating       : sudo                               1/2
  Cleanup        : sudo                               2/2

Updated:
  sudo.i386 0:1.7.2p1-7.el5_5

Complete!
```

55.7.7. yum software groups

Issue **yum grouplist** to see a list of all available software groups.

```
[root@rhel55 ~]# yum grouplist
Loaded plugins: rhnplugin, security
Setting up Group Process
Installed Groups:
  Administration Tools
  Authoring and Publishing
  DNS Name Server
  Development Libraries
  Development Tools
  Editors
  GNOME Desktop Environment
  GNOME Software Development
  Graphical Internet
  Graphics
  Legacy Network Server
  Legacy Software Development
  Legacy Software Support
  Mail Server
  Network Servers
  Office/Productivity
  Printing Support
  Server Configuration Tools
  System Tools
  Text-based Internet
  Web Server
  Windows File Server
  X Software Development
  X Window System
Available Groups:
  Engineering and Scientific
  FTP Server
  Games and Entertainment
  Java Development
  KDE (K Desktop Environment)
  KDE Software Development
  MySQL Database
  News Server
  OpenFabrics Enterprise Distribution
  PostgreSQL Database
  Sound and Video
Done
```

To install a set of applications, brought together via a group, use **yum groupinstall \$groupname**.

```
[root@rhel55 ~]# yum groupinstall 'Sound and video'
Loaded plugins: rhnplugin, security
Setting up Group Process
Package alsa-utils-1.0.17-1.el5.i386 already installed and latest version
Package sox-12.18.1-1.i386 already installed and latest version
Package 9:mkisofs-2.01-10.7.el5.i386 already installed and latest version
Package 9:cdrecord-2.01-10.7.el5.i386 already installed and latest version
Package cdrdao-1.2.1-2.i386 already installed and latest version
Resolving Dependencies
--> Running transaction check
---> Package cdda2wav.i386 9:2.01-10.7.el5 set to be updated
---> Package cdparanoia.i386 0:alpha9.8-27.2 set to be updated
---> Package sound-juicer.i386 0:2.16.0-3.el5 set to be updated
--> Processing Dependency: libmusicbrainz >= 2.1.0 for package: sound-juicer
--> Processing Dependency: libmusicbrainz.so.4 for package: sound-juicer
---> Package vorbis-tools.i386 1:1.1.1-3.el5 set to be updated
--> Processing Dependency: libao >= 0.8.4 for package: vorbis-tools
--> Processing Dependency: libao.so.2 for package: vorbis-tools
--> Running transaction check
---> Package libao.i386 0:0.8.6-7 set to be updated
---> Package libmusicbrainz.i386 0:2.1.1-4.1 set to be updated
--> Finished Dependency Resolution
...
```

Read the manual page of **yum** for more information about managing groups in **yum**.

55.7.8. /etc/yum.conf and repositories

The configuration of **yum** repositories is done in **/etc/yum/yum.conf** and **/etc/yum/repos.d/**.

Configuring **yum** itself is done in **/etc/yum.conf**. This file will contain the location of a log file and a cache directory for **yum** and can also contain a list of repositories.

Recently **yum** started accepting several **repo** files with each file containing a list of **repositories**. These **repo** files are located in the **/etc/yum.repos.d/** directory.

One important flag for yum is **enablerepo**. Use this command if you want to use a repository that is not enabled by default.

```
yum $command $foo --enablerepo=$repo
```

An example of the contents of the repo file: MyRepo.repo

```
[$repo]
name=My Repository
baseurl=http://path/to/MyRepo
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-MyRep
```

55.8. alien

alien is experimental software that converts between **rpm** and **deb** package formats (and others).

Below an example of how to use **alien** to convert an **rpm** package to a **deb** package.

```
paul@barry:~$ ls -l netcat*
-rw-r--r-- 1 paul paul 123912 2009-06-04 14:58 netcat-0.7.1-1.i386.rpm
paul@barry:~$ alien --to-deb netcat-0.7.1-1.i386.rpm
netcat_0.7.1-2_i386.deb generated
paul@barry:~$ ls -l netcat*
-rw-r--r-- 1 paul paul 123912 2009-06-04 14:58 netcat-0.7.1-1.i386.rpm
-rw-r--r-- 1 root root 125236 2009-06-04 14:59 netcat_0.7.1-2_i386.deb
```

*In real life, use the **netcat** tool provided by your distribution, or use the .deb file from their website.*

55.9. downloading software outside the repository

First and most important, whenever you download software, start by reading the README file!

Normally the readme will explain what to do after download. You will probably receive a .tar.gz or a .tgz file. Read the documentation, then put the compressed file in a directory. You can use the following to find out where the package wants to install.

```
tar tvzpf $downloadedFile.tgz
```

You unpack them like with **tar xzf**, it will create a directory called applicationName-1.2.3

```
tar xzf $applicationName.tgz
```

Replace the z with a j when the file ends in .tar.bz2. The **tar**, **gzip** and **bzip2** commands are explained in detail in the Linux Fundamentals course.

If you download a **.deb** file, then you'll have to use **dpkg** to install it, **.rpm**'s can be installed with the **rpm** command.

55.10. compiling software

First and most important, whenever you download source code for installation, start by reading the README file!

Usually the steps are always the same three : running **./configure** followed by **make** (which is the actual compiling) and then by **make install** to copy the files to their proper location.

```
./configure  
make  
make install
```

55.11. practice: package management

1. Verify whether gcc, sudo and wesnoth are installed.
2. Use yum or aptitude to search for and install the scp, tmux, and man-pages packages. Did you find them all ?
3. Search the internet for 'webmin' and figure out how to install it.
4. If time permits, search for and install samba including the samba docs pdf files (thousands of pages in two pdf's).

55.12. solution: package management

1. Verify whether gcc, sudo and wesnoth are installed.

```
On Red Hat/CentOS:  
rpm -qa | grep gcc  
rpm -qa | grep sudo  
rpm -qa | grep wesnoth
```

```
On Debian/Ubuntu:  
dpkg -l | grep gcc  
dpkg -l | grep sudo  
dpkg -l | grep wesnoth
```

2. Use yum or aptitude to search for and install the scp, tmux, and man-pages packages. Did you find them all ?

```
On Red Hat/CentOS:  
yum search scp  
yum search tmux  
yum search man-pages
```

```
On Debian/Ubuntu:  
aptitude search scp  
aptitude search tmux  
aptitude search man-pages
```

3. Search the internet for 'webmin' and figure out how to install it.

Google should point you to webmin.com.

There are several formats available there choose .rpm, .deb or .tgz .

4. If time permits, search for and install samba including the samba docs pdf files (thousands of pages in two pdf's).

Part XIV. network management

Table of Contents

56. general networking	580
56.1. network layers	581
56.2. unicast, multicast, broadcast, anycast	584
56.3. lan-wan-man	586
56.4. internet - intranet - extranet	588
56.5. tcp/ip	589
57. interface configuration	590
57.1. to gui or not to gui	591
57.2. Debian nic configuration	592
57.3. RHEL nic configuration	594
57.4. ifconfig	596
57.5. ip	598
57.6. dhclient	599
57.7. hostname	599
57.8. arp	600
57.9. route	601
57.10. ping	601
57.11. optional: ethtool	602
57.12. practice: interface configuration	603
57.13. solution: interface configuration	604
58. network sniffing	606
58.1. wireshark	607
58.2. tcpdump	611
58.3. practice: network sniffing	612
58.4. solution: network sniffing	613
59. binding and bonding	614
59.1. binding on Redhat/Fedora	615
59.2. binding on Debian/Ubuntu	616
59.3. bonding on Redhat/Fedora	617
59.4. bonding on Debian/Ubuntu	619
59.5. practice: binding and bonding	621
59.6. solution: binding and bonding	622
60. ssh client and server	623
60.1. about ssh	624
60.2. log on to a remote server	626
60.3. executing a command in remote	626
60.4. scp	627
60.5. setting up passwordless ssh	628
60.6. X forwarding via ssh	629
60.7. troubleshooting ssh	630
60.8. sshd	631
60.9. sshd keys	631
60.10. ssh-agent	631
60.11. practice: ssh	632
60.12. solution: ssh	633
61. introduction to nfs	635
61.1. nfs protocol versions	636
61.2. rpcinfo	636
61.3. server configuration	637
61.4. /etc/exports	637
61.5. exports	637
61.6. client configuration	638
61.7. practice: introduction to nfs	639
62. introduction to networking	640
62.1. introduction to iptables	641

62.2. practice : iptables	642
62.3. solution : iptables	643
62.4. xinetd and inetd	644
62.5. practice : inetd and xinetd	646
62.6. network file system	647
62.7. practice : network file system	649

Chapter 56. general networking

While this chapter is not directly about **Linux**, it does contain general networking concepts that will help you in troubleshooting networks on **Linux**.

56.1. network layers

56.1.1. seven OSI layers

When talking about protocol layers, people usually mention the seven layers of the **osi** protocol (Application, Presentation, Session, Transport, Network, Data Link and Physical). We will discuss layers 2 and 3 in depth, and focus less on the other layers. The reason is that these layers are important for understanding networks. You will hear administrators use words like "this is a layer 2 device" or "this is a layer 3 broadcast", and you should be able to understand what they are talking about.

56.1.2. four DoD layers

The **DoD** (or tcp/ip) model has only four layers, roughly mapping its **network access layer** to OSI layers 1 and 2 (Physical and Datalink), its **internet** (IP) layer to the OSI **network layer**, its **host-to-host** (tcp, udp) layer to OSI layer 4 (transport) and its **application layer** to OSI layers 5, 6 and 7.

Below an attempt to put OSI and DoD layers next to some protocols and devices.

OSI Model	DoD Model	protocols		devices/apps
layer 5, 6, 7	application	dns, dhcp, ntp, snmp, https, ftp, ssh, telnet, http, pop3... others		web server, mail server, browser, mail client...
layer 4	host-to-host	tcp	udp	gateway
layer 3	internet	ip, icmp, igmp		router, firewall layer 3 switch
layer 2	network access	arp (mac), rarp		bridge layer 2 switch
layer 1		ethernet, token ring		hub

56.1.3. short introduction to the physical layer

The physical layer, or **layer 1**, is all about voltage, electrical signals and mechanical connections. Some networks might still use **coax** cables, but most will have migrated to **utp** (cat 5 or better) with **rj45** connectors.

Devices like **repeaters** and **hubs** are part of this layer. You cannot use software to 'see' a **repeater** or **hub** on the network. The only thing these devices are doing is amplifying electrical signals on cables. **Passive hubs** are multiport amplifiers that amplify an incoming electrical signal on all other connections. **Active hubs** do this by reading and retransmitting bits, without interpreting any meaning in those bits.

Network technologies like **csma/cd** and **token ring** are defined on this layer.

This is all we have to say about **layer 1** in this book.

56.1.4. short introduction to the data link layer

The data link layer, or **layer 2** is about frames. A frame has a **crc** (cyclic redundancy check). In the case of ethernet (802.3), each network card is identifiable by a unique 48-bit **mac** address (media access control address).

On this layer we find devices like bridges and switches. A bridge is more intelligent than a hub because a **bridge** can make decisions based on the mac address of computers. A **switch** also understands mac addresses.

In this book we will discuss commands like **arp** and **ifconfig** to explore this layer.

56.1.5. short introduction to the network layer

Layer 3 is about ip packets. This layer gives every host a unique 32-bit ip address. But **ip** is not the only protocol on this layer, there is also icmp, igmp, ipv6 and more. A complete list can be found in the **/etc/protocols** file.

On this layer we find devices like **routers** and layer 3 switches, devices that know (and have) an ip address.

In tcp/ip this layer is commonly referred to as the **internet layer**.

56.1.6. short introduction to the transport layer

We will discuss the **tcp** and **udp** protocols in the context of layer 4. The DoD model calls this the host-to-host layer.

56.1.7. layers 5, 6 and 7

The tcp/ip application layer includes layers 5, 6 and 7. Details on the difference between these layers are out of scope of this course.

56.1.8. network layers in this book

Stacking of layers in this book is based on the **Protocols in Frame** explanation in the **wireshark** sniffer. When sniffing a dhcp packet, we notice the following in the sniffer.

```
[Protocols in Frame: eth:ip:udp:bootp]
```

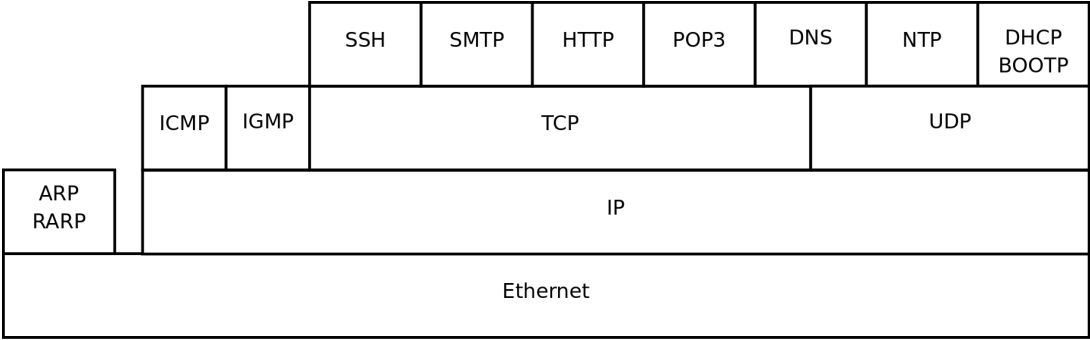
Sniffing for ntp (Network Time Protocol) packets gives us this line, which makes us conclude to put **ntp** next to **bootp** in the protocol chart below.

```
[Protocols in Frame: eth:ip:udp:ntp]
```

Sniffing an **arp** broadcast makes us put arp next to **ip**. All these protocols are explained later in this chapter.

```
[Protocols in Frame: eth:arp]
```

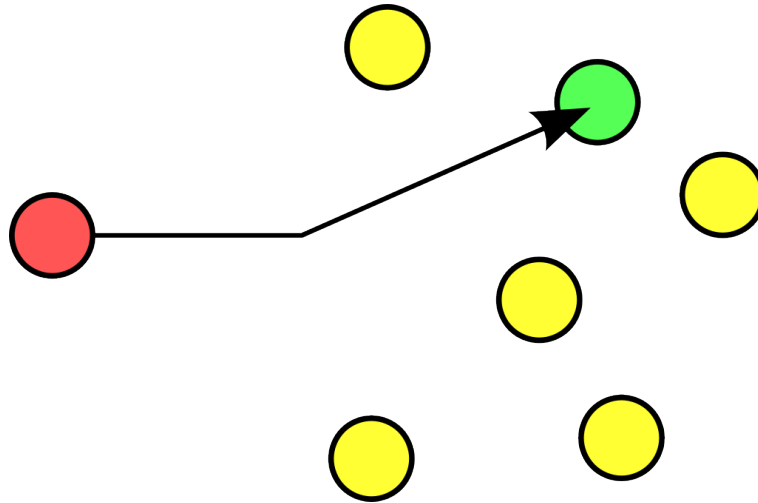
Below is a protocol chart based on wireshark's knowledge. It contains some very common protocols that are discussed in this book. The chart does not contain all protocols.



56.2. unicast, multicast, broadcast, anycast

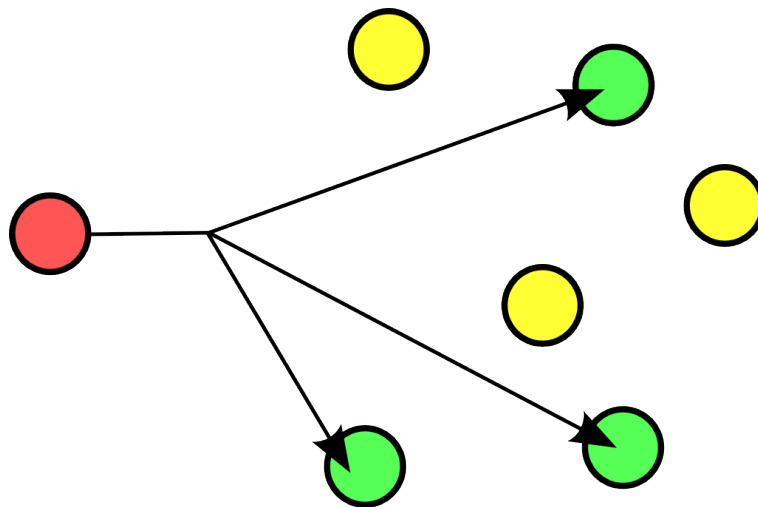
56.2.1. unicast

A **unicast** communication originates from one computer and is destined for exactly one other computer (or host). It is common for computers to have many **unicast** communications.



56.2.2. multicast

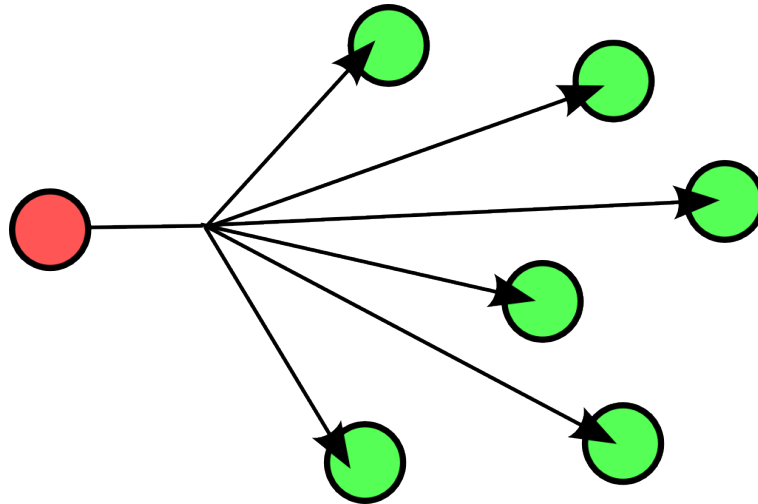
A **multicast** is destined for a group (of computers).



Some examples of **multicast** are Realplayer (.sdp files) and **ripv2** (a routing protocol).

56.2.3. broadcast

A **broadcast** is meant for everyone.

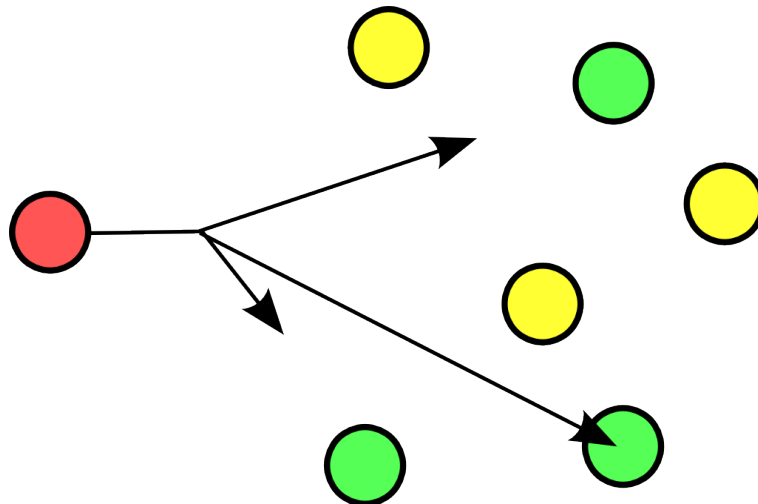


Typical example here is the BBC (British Broadcasting Corporation) broadcasting to everyone. In datacommunications a broadcast is most common confined to the **lan**.

Careful, a **layer 2 broadcast** is very different from a **layer 3 broadcast**. A layer two broadcast is received by all network cards on the same segment (it does not pass any router), whereas a layer 3 broadcast is received by all hosts in the same ip subnet.

56.2.4. anycast

The **root name servers** of the internet use **anycast**. An **anycast** signal goes to the (geographically) nearest of a well defined group.



With thanks to the nice anonymous wikipedia contributor to put these pictures in the public domain.

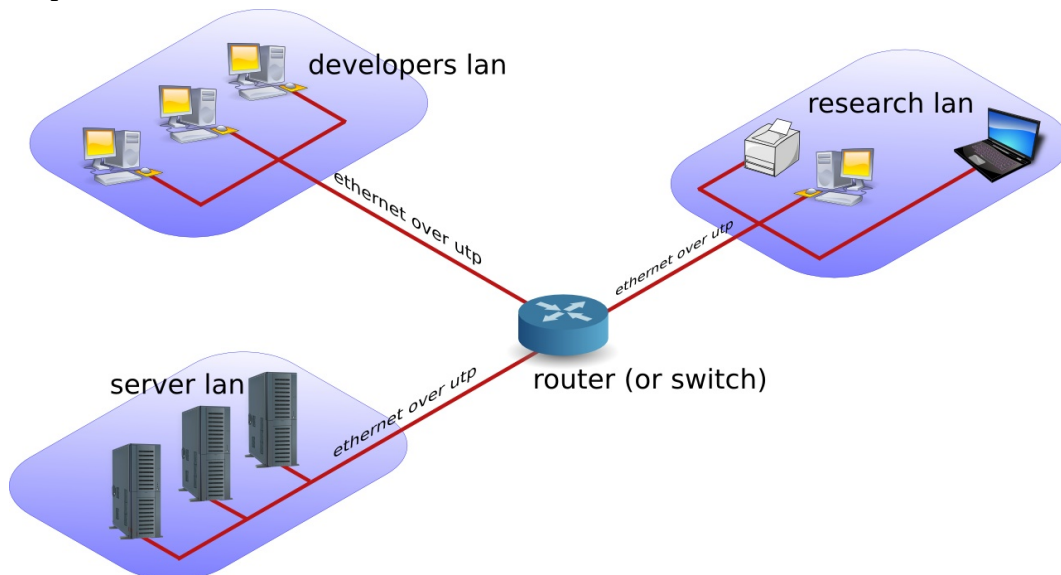
56.3. lan-wan-man

The term **lan** is used for local area networks, as opposed to a **wan** for wide area networks. The difference between the two is determined by the **distance** between the computers, and not by the number of computers in a network. Some protocols like **atm** are designed for use in a **wan**, others like **ethernet** are designed for use in a **lan**.

56.3.1. lan

A **lan** (Local Area Network) is a local network. This can be one room, or one floor, or even one big building. We say **lan** as long as computers are **close** to each other. You can also define a **lan** when all computers are **ethernet** connected.

A **lan** can contain multiple smaller **lan**'s. The picture below shows three **lan**'s that together make up one **lan**.



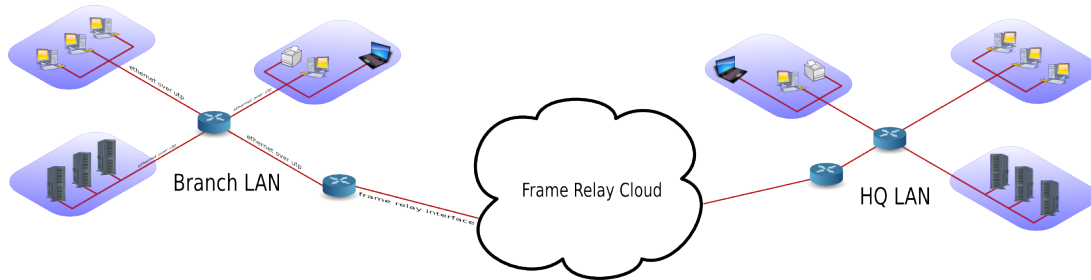
56.3.2. man

A **man** (Metropolitan Area Network) is something inbetween a **lan** and a **wan**, often comprising several buildings on the same campus or in the same city. A **man** can use **fdi** or **ethernet** or other protocols for connectivity.

56.3.3. wan

A **wan** (Wide Area Network) is a network with a lot of distance between the computers (or hosts). These hosts are often connected by **leased lines**. A **wan** does not use **ethernet**, but protocols like **fdi**, **frame relay**, **ATM** or **X.25** to connect computers (and networks).

The picture below shows a branch office that is connected through **Frame Relay** with headquarters.



The acronym **wan** is also used for large surface area networks like the **internet**.

Cisco is known for their **wan** technology. They make **routers** that connect many **lan** networks using **wan** protocols.

56.3.4. pan-wpan

Your home network is called a **pan** (Personal Area Network). A wireless **pan** is a **wpan**.

56.4. internet - intranet - extranet

The **internet** is a global network. It connects many networks using the **tcp/ip** protocol stack.

The origin of the **internet** is the **arpanet**. The **arpanet** was created in 1969, that year only four computers were connected in the network. In 1971 the first **e-mail** was sent over the **arpanet**. **E-mail** took 75 percent of all **arpanet** traffic in 1973. 1973 was also the year **ftp** was introduced, and saw the connection of the first European countries (Norway and UK). In 2009 the internet was available to 25 percent of the world population. In 2011 it is estimated that only a quarter of internet webpages are in English.

An **intranet** is a private **tcp/ip** network. An **intranet** uses the same protocols as the **internet**, but is only accessible to people from within one organization.

An **extranet** is similar to an **intranet**, but some trusted organizations (partners/clients/suppliers/...) also get access.

56.5. tcp/ip

56.5.1. history of tcp/ip

In the Sixties development of the **tcp/ip** protocol stack was started by the US Department of Defense. In the Eighties a lot of commercial enterprises developed their own protocol stack: IBM created **sna**, Novell had **ipx/spx**, Microsoft completed **netbeui** and Apple worked with **appletalk**. All the efforts from the Eighties failed to survive the Nineties. By the end of the Nineties, almost all computers in the world were able to speak tcp/ip.

In my humble opinion, the main reason for the survival of **tcp/ip** over all the other protocols is its openness. Everyone is free to develop and use the tcp/ip protocol suite.

56.5.2. rfc (request for comment)

The protocols that are used on the internet are defined in **rfc's**. An rfc or **request for comment** describes the inner working of all internet protocols. The **IETF** (Internet Engineering Task Force) is the sole publisher of these protocols since 1986.

The official website for the rfc's is <http://www.rfc-editor.org>. This website contains all rfc's in plain text, for example rfc2132 (which defines dhcp and bootp) is accessible at <http://www.rfc-editor.org/rfc/rfc2132.txt>.

56.5.3. many protocols

For reliable connections, you use **tcp**, whereas **udp** is connectionless but faster. The **icmp** error messages are used by **ping**, multicast groups are managed by **igmp**.

These protocols are visible in the protocol field of the ip header, and are listed in the **/etc/protocols** file.

```
paul@debian5:~$ grep tcp /etc/protocols
tcp      6      TCP          # transmission control protocol
```

56.5.4. many services

Network cards are uniquely identified by their **mac address**, hosts by their **ip address** and applications by their **port number**.

Common application level protocols like smtp, http, ssh, telnet and ftp have fixed **port numbers**. There is a list of **port numbers** in **/etc/services**.

```
paul@ubu1010:~$ grep ssh /etc/services
ssh      22/tcp      # SSH Remote Login Protocol
ssh      22/udp
```

Chapter 57. interface configuration

This chapter explains how to configure **network interface cards** to work with **tcp/ip**.

57.1. to gui or not to gui

Recent Linux distributions often include a graphical application to configure the network. Some people complain that these applications mess networking configurations up when used simultaneously with command line configurations. Notably **Network Manager** (often replaced by **wicd**) and **yast** are known to not care about configuration changes via the command line.

Since the goal of this course is **server** administration, we will assume our Linux servers are always administered through the command line.

This chapter only focuses on using the command line for network interface configuration!

Unfortunately there is no single combination of Linux commands and **/etc** files that works on all Linux distributions. We discuss networking on two (large but distinct) Linux distribution families.

We start with **Debian** (this should also work on Ubuntu and Mint), then continue with **RHEL** (which is identical to CentOS and Fedora).

57.2. Debian nic configuration

57.2.1. /etc/network/interfaces

The `/etc/network/interfaces` file is a core network interface card configuration file on **debian**.

dhcp client

The screenshot below shows that our computer is configured for **dhcp** on **eth0** (the first network interface card or nic).

```
paul@debian8:~$ cat /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet dhcp
```

Configuring network cards for **dhcp** is good practice for clients, but servers usually require a **fixed ip address**.

fixed ip

The screenshot below shows `/etc/network/interfaces` configured with a **fixed ip address**.

```
root@debian7~# cat /etc/network/interfaces
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
address    10.42.189.198
broadcast  10.42.189.207
netmask    255.255.255.240
gateway    10.42.189.193
```

The screenshot above also shows that you can provide more configuration than just the ip address. See **interfaces(5)** for help on setting a **gateway**, **netmask** or any of the other options.

57.2.2. /sbin/ifdown

It is advised (but not mandatory) to down an interface before changing its configuration. This can be done with the **ifdown** command.

The command will not give any output when downing an interface with a fixed ip address. However **ifconfig** will no longer show the interface.

```
root@ubu1104srv:~# ifdown eth0
root@ubu1104srv:~# ifconfig
lo    Link encap:Local Loopback
      inet addr:127.0.0.1  Mask:255.0.0.0
      inet6 addr: ::1/128 Scope:Host
      UP LOOPBACK RUNNING  MTU:16436  Metric:1
      RX packets:106 errors:0 dropped:0 overruns:0 frame:0
      TX packets:106 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:0
      RX bytes:11162 (11.1 KB)  TX bytes:11162 (11.1 KB)
```

An interface that is down cannot be used to connect to the network.

57.2.3. /sbin/ifup

Below a screenshot of **ifup** bringing the **eth0** ethernet interface up using dhcp. (Note that this is a Ubuntu 10.10 screenshot, Ubuntu 11.04 omits **ifup** output by default.)

```
root@ubu1010srv:/etc/network# ifup eth0
Internet Systems Consortium DHCP Client V3.1.3
Copyright 2004-2009 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/eth0/08:00:27:cd:7f:fc
Sending on   LPF/eth0/08:00:27:cd:7f:fc
Sending on   Socket/fallback
DHCPREQUEST of 192.168.1.34 on eth0 to 255.255.255.255 port 67
DHCPNAK from 192.168.33.100
DHCPDISCOVER on eth0 to 255.255.255.255 port 67 interval 3
DHCPOFFER of 192.168.33.77 from 192.168.33.100
DHCPREQUEST of 192.168.33.77 on eth0 to 255.255.255.255 port 67
DHCPACK of 192.168.33.77 from 192.168.33.100
bound to 192.168.33.77 -- renewal in 95 seconds.
ssh stop/waiting
ssh start/running, process 1301
root@ubu1010srv:/etc/network#
```

The details of **dhcp** are covered in a separate chapter in the **Linux Servers** course.

57.3. RHEL nic configuration

57.3.1. /etc/sysconfig/network

The `/etc/sysconfig/network` file is a global (across all network cards) configuration file. It allows us to define whether we want networking (`NETWORKING=yes|no`), what the hostname should be (`HOSTNAME=`) and which gateway to use (`GATEWAY=`).

```
[root@rhel6 ~]# cat /etc/sysconfig/network
NETWORKING=yes
HOSTNAME=rhel6
GATEWAY=192.168.1.1
```

There are a dozen more options settable in this file, details can be found in `/usr/share/doc/initscripts-*/sysconfig.txt`.

Note that this file contains no settings at all in a default RHEL7 install (with networking enabled).

```
[root@rhel71 ~]# cat /etc/sysconfig/network
# Created by anaconda
```

57.3.2. /etc/sysconfig/network-scripts/ifcfg-

Each network card can be configured individually using the `/etc/sysconfig/network-scripts/ifcfg-*` files. When you have only one network card, then this will probably be `/etc/sysconfig/network-scripts/ifcfg-eth0`.

dhcp client

Below a screenshot of `/etc/sysconfig/network-scripts/ifcfg-eth0` configured for dhcp (`BOOTPROTO="dhcp"`). Note also the `NM_CONTROLLED` parameter to disable control of this nic by **Network Manager**. This parameter is not explained (not even mentioned) in `/usr/share/doc/initscripts-*/sysconfig.txt`, but many others are.

```
[root@rhel6 ~]# cat /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE="eth0"
HWADDR="08:00:27:DD:0D:5C"
NM_CONTROLLED="no"
BOOTPROTO="dhcp"
ONBOOT="yes"
```

The `BOOTPROTO` variable can be set to either **dhcp** or **bootp**, anything else will be considered **static** meaning there should be no protocol used at boot time to set the interface values.

RHEL7 adds **ipv6** variables to this file.

```
[root@rhel71 network-scripts]# cat ifcfg-enp0s3
TYPE="Ethernet"
BOOTPROTO="dhcp"
DEFROUTE="yes"
PEERDNS="yes"
PEERROUTES="yes"
IPV4_FAILURE_FATAL="no"
```

```
IPV6INIT="yes"
IPV6_AUTOCONF="yes"
IPV6_DEFROUTE="yes"
IPV6_PEERDNS="yes"
IPV6_PEERROUTES="yes"
IPV6_FAILURE_FATAL="no"
NAME="enp0s3"
UUID="9fa6a83a-2f8e-4ecc-962c-5f614605f4ee"
DEVICE="enp0s3"
ONBOOT="yes"
[root@rhel71 network-scripts]#
```

fixed ip

Below a screenshot of a **fixed ip** configuration in `/etc/sysconfig/network-scripts/ifcfg-eth0`.

```
[root@rhel6 ~]# cat /etc/sysconfig/network-scripts/ifcfg-eth0
DEVICE="eth0"
HWADDR="08:00:27:DD:0D:5C"
NM_CONTROLLED="no"
BOOTPROTO="none"
IPADDR="192.168.1.99"
NETMASK="255.255.255.0"
GATEWAY="192.168.1.1"
ONBOOT="yes"
```

The `HWADDR` can be used to make sure that each network card gets the correct name when multiple network cards are present in the computer. It can not be used to assign a **mac address** to a network card. For this, you need to specify the `MACADDR` variable. Do not use `HWADDR` and `MACADDR` in the same **`ifcfg-ethx`** file.

The `BROADCAST=` and `NETWORK=` parameters from previous RHEL/Fedora versions are obsoleted.

57.3.3. nmcli

On RHEL7 you should run **`nmcli connection reload`** if you changed configuration files in `/etc/sysconfig/` to enable your changes.

The **`nmcli`** tool has many options to configure networking on the command line in RHEL7/CentOS7

```
man nmcli
```

57.3.4. nmtui

Another recommendation for RHEL7/CentOS7 is to use **`nmtui`**. This tool will use a 'windowed' interface in command line to manage network interfaces.

```
nmtui
```

57.3.5. /sbin/ifup and /sbin/ifdown

The **ifup** and **ifdown** commands will set an interface up or down, using the configuration discussed above. This is identical to their behaviour in Debian and Ubuntu.

```
[root@rhel6 ~]# ifdown eth0 && ifup eth0
[root@rhel6 ~]# ifconfig eth0
eth0 Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
      inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
      inet6 addr: fe80::a00:27ff:fedd:d5c/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
      RX packets:2452 errors:0 dropped:0 overruns:0 frame:0
      TX packets:1881 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:257036 (251.0 KiB) TX bytes:184767 (180.4 KiB)
```

57.4. ifconfig

The use of **/sbin/ifconfig** without any arguments will present you with a list of all active network interface cards, including wireless and the loopback interface. In the screenshot below **eth0** has no ip address.

```
root@ubu1010:~# ifconfig
eth0 Link encap:Ethernet HWaddr 00:26:bb:5d:2e:52
      UP BROADCAST MULTICAST MTU:1500 Metric:1
      RX packets:0 errors:0 dropped:0 overruns:0 frame:0
      TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
      Interrupt:43 Base address:0xe000

eth1 Link encap:Ethernet HWaddr 00:26:bb:12:7a:5e
      inet addr:192.168.1.30 Bcast:192.168.1.255 Mask:255.255.255.0
      inet6 addr: fe80::226:bbff:fe12:7a5e/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
      RX packets:11141791 errors:202 dropped:0 overruns:0 frame:11580126
      TX packets:6473056 errors:3860 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:3476531617 (3.4 GB) TX bytes:2114919475 (2.1 GB)
      Interrupt:23

lo    Link encap:Local Loopback
      inet addr:127.0.0.1 Mask:255.0.0.0
      inet6 addr: ::1/128 Scope:Host
      UP LOOPBACK RUNNING MTU:16436 Metric:1
      RX packets:2879 errors:0 dropped:0 overruns:0 frame:0
      TX packets:2879 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:0
      RX bytes:486510 (486.5 KB) TX bytes:486510 (486.5 KB)
```

You can also use **ifconfig** to obtain information about just one network card.

```
[root@rhel6 ~]# ifconfig eth0
eth0 Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
      inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
      inet6 addr: fe80::a00:27ff:fedd:d5c/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
      RX packets:2969 errors:0 dropped:0 overruns:0 frame:0
      TX packets:1918 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
```

```
RX bytes:335942 (328.0 KiB) TX bytes:190157 (185.7 KiB)
```

When **/sbin** is not in the **\$PATH** of a normal user you will have to type the full path, as seen here on Debian.

```
paul@debian5:~$ /sbin/ifconfig eth3
eth3 Link encap:Ethernet HWaddr 08:00:27:ab:67:30
      inet addr:192.168.1.29 Bcast:192.168.1.255 Mask:255.255.255.0
      inet6 addr: fe80::a00:27ff:feab:6730/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
      RX packets:27155 errors:0 dropped:0 overruns:0 frame:0
      TX packets:30527 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:13095386 (12.4 MiB) TX bytes:25767221 (24.5 MiB)
```

57.4.1. up and down

You can also use **ifconfig** to bring an interface up or down. The difference with **ifup** is that **ifconfig eth0 up** will re-activate the nic keeping its existing (current) configuration, whereas **ifup** will read the correct file that contains a (possibly new) configuration and use this config file to bring the interface up.

```
[root@rhel6 ~]# ifconfig eth0 down
[root@rhel6 ~]# ifconfig eth0 up
[root@rhel6 ~]# ifconfig eth0
eth0 Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
      inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
      inet6 addr: fe80::a00:27ff:fedd:d5c/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
      RX packets:2995 errors:0 dropped:0 overruns:0 frame:0
      TX packets:1927 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:339030 (331.0 KiB) TX bytes:191583 (187.0 KiB)
```

57.4.2. setting ip address

You can **temporary** set an ip address with **ifconfig**. This ip address is only valid until the next **ifup/ifdown** cycle or until the next **reboot**.

```
[root@rhel6 ~]# ifconfig eth0 | grep 192
      inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
[root@rhel6 ~]# ifconfig eth0 192.168.33.42 netmask 255.255.0.0
[root@rhel6 ~]# ifconfig eth0 | grep 192
      inet addr:192.168.33.42 Bcast:192.168.255.255 Mask:255.255.0.0
[root@rhel6 ~]# ifdown eth0 && ifup eth0
[root@rhel6 ~]# ifconfig eth0 | grep 192
      inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
```

57.4.3. setting mac address

You can also use **ifconfig** to set another **mac address** than the one hard coded in the network card. This screenshot shows you how.

```
[root@rhel6 ~]# ifconfig eth0 | grep HWaddr
eth0 Link encap:Ethernet HWaddr 08:00:27:DD:0D:5C
[root@rhel6 ~]# ifconfig eth0 hw ether 00:42:42:42:42:42
[root@rhel6 ~]# ifconfig eth0 | grep HWaddr
eth0 Link encap:Ethernet HWaddr 00:42:42:42:42:42
```

57.5. ip

The **ifconfig** tool is deprecated on some systems. Use the **ip** tool instead.

To see ip addresses on RHEL7 for example, use this command:

```
[root@rhel71 ~]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 08:00:27:89:22:33 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.135/24 brd 192.168.1.255 scope global dynamic enp0s3
        valid_lft 6173sec preferred_lft 6173sec
    inet6 fe80::a00:27ff:fe89:2233/64 scope link
        valid_lft forever preferred_lft forever
[root@rhel71 ~]#
```

57.6. dhclient

Home and client Linux desktops often have **/sbin/dhclient** running. This is a daemon that enables a network interface to lease an ip configuration from a **dhcp server**. When your adapter is configured for **dhcp** or **bootp**, then **/sbin/ifup** will start the **dhclient** daemon.

When a lease is renewed, **dhclient** will override your **ifconfig** set ip address!

57.7. hostname

Every host receives a **hostname**, often placed in a **DNS name space** forming the **fqdn** or Fully Qualified Domain Name.

This screenshot shows the **hostname** command and the configuration of the hostname on Red Hat/Fedora.

```
[root@rhel6 ~]# grep HOSTNAME /etc/sysconfig/network
HOSTNAME=rhel6
[root@rhel6 ~]# hostname
rhel6
```

Starting with RHEL7/CentOS7 this file is empty. The hostname is configured in the standard **/etc/hostname** file.

```
[root@rhel71 ~]# cat /etc/hostname
rhel71.linux-training.be
[root@rhel71 ~]#
```

Ubuntu/Debian uses the **/etc/hostname** file to configure the **hostname**.

```
paul@debian8:~$ cat /etc/hostname
server42
paul@debian8:~$ hostname
server42
```

On all Linux distributions you can change the **hostname** using the **hostname \$newname** command. This is not a permanent change.

```
[root@rhel6 ~]# hostname server42
[root@rhel6 ~]# hostname
server42
```

On any Linux you can use **sysctl** to display and set the hostname.

```
[root@rhel6 ~]# sysctl kernel.hostname
kernel.hostname = server42
[root@rhel6 ~]# sysctl kernel.hostname=rhel6
kernel.hostname = rhel6
[root@rhel6 ~]# sysctl kernel.hostname
kernel.hostname = rhel6
[root@rhel6 ~]# hostname
rhel6
```

57.8. arp

The **ip to mac** resolution is handled by the **layer two broadcast** protocol **arp**. The **arp table** can be displayed with the **arp tool**. The screenshot below shows the list of computers that this computer recently communicated with.

```
root@barry:~# arp -a
? (192.168.1.191) at 00:0C:29:3B:15:80 [ether] on eth1
agapi (192.168.1.73) at 00:03:BA:09:7F:D2 [ether] on eth1
anya (192.168.1.1) at 00:12:01:E2:87:FB [ether] on eth1
faith (192.168.1.41) at 00:0E:7F:41:0D:EB [ether] on eth1
kiss (192.168.1.49) at 00:D0:E0:91:79:95 [ether] on eth1
laika (192.168.1.40) at 00:90:F5:4E:AE:17 [ether] on eth1
pasha (192.168.1.71) at 00:03:BA:02:C3:82 [ether] on eth1
shaka (192.168.1.72) at 00:03:BA:09:7C:F9 [ether] on eth1
root@barry:~#
```

Anya is a Cisco Firewall, faith is a laser printer, kiss is a Kiss DP600, laika is a laptop and Agapi, Shaka and Pasha are SPARC servers. The question mark is a Red Hat Enterprise Linux server running on a virtual machine.

You can use **arp -d** to remove an entry from the **arp table**.

```
[root@rhel6 ~]# arp
Address      HWtype  HWaddress      Flags Mask    Iface
ubu1010      ether   00:26:bb:12:7a:5e  C           eth0
anya         ether   00:02:cf:aa:68:f0  C           eth0
[root@rhel6 ~]# arp -d anya
[root@rhel6 ~]# arp
Address      HWtype  HWaddress      Flags Mask    Iface
ubu1010      ether   00:26:bb:12:7a:5e  C           eth0
anya         (incomplete)
[root@rhel6 ~]# ping anya
PING anya (192.168.1.1) 56(84) bytes of data.
64 bytes from anya (192.168.1.1): icmp_seq=1 ttl=254 time=10.2 ms
...
[root@rhel6 ~]# arp
Address      HWtype  HWaddress      Flags Mask    Iface
ubu1010      ether   00:26:bb:12:7a:5e  C           eth0
anya         ether   00:02:cf:aa:68:f0  C           eth0
```


57.9. route

You can see the computer's local routing table with the **/sbin/route** command (and also with **netstat -r**).

```
root@RHEL4b ~]# netstat -r
Kernel IP routing table
Destination      Gateway         Genmask         Flags   MSS Window  irtt Iface
192.168.1.0      *              255.255.255.0   U        0  0        0 eth0
[root@RHEL4b ~]# route
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0      *              255.255.255.0   U        0      0      0 eth0
[root@RHEL4b ~]#
```

It appears this computer does not have a **gateway** configured, so we use **route add default gw** to add a **default gateway** on the fly.

```
[root@RHEL4b ~]# route add default gw 192.168.1.1
[root@RHEL4b ~]# route
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0      *              255.255.255.0   U        0      0      0 eth0
default          192.168.1.1    0.0.0.0         UG        0      0      0 eth0
[root@RHEL4b ~]#
```

Unless you configure the gateway in one of the **/etc/** file from the start of this chapter, your computer will forget this **gateway** after a reboot.

57.10. ping

If you can **ping** to another host, then **tcp/ip** is configured.

```
[root@RHEL4b ~]# ping 192.168.1.5
PING 192.168.1.5 (192.168.1.5) 56(84) bytes of data.
64 bytes from 192.168.1.5: icmp_seq=0 ttl=64 time=1004 ms
64 bytes from 192.168.1.5: icmp_seq=1 ttl=64 time=1.19 ms
64 bytes from 192.168.1.5: icmp_seq=2 ttl=64 time=0.494 ms
64 bytes from 192.168.1.5: icmp_seq=3 ttl=64 time=0.419 ms

--- 192.168.1.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 0.419/251.574/1004.186/434.520 ms, pipe 2
[root@RHEL4b ~]#
```

57.11. optional: ethtool

To display or change network card settings, use **ethtool**. The results depend on the capabilities of your network card. The example shows a network that auto-negotiates its bandwidth.

```
root@laika:~# ethtool eth0
Settings for eth0:
Supported ports: [ TP ]
Supported link modes:   10baseT/Half 10baseT/Full
                        100baseT/Half 100baseT/Full
                        1000baseT/Full
Supports auto-negotiation: Yes
Advertised link modes:  10baseT/Half 10baseT/Full
                        100baseT/Half 100baseT/Full
                        1000baseT/Full
Advertised auto-negotiation: Yes
Speed: 1000Mb/s
Duplex: Full
Port: Twisted Pair
PHYAD: 0
Transceiver: internal
Auto-negotiation: on
Supports Wake-on: pumbg
Wake-on: g
Current message level: 0x00000033 (51)
Link detected: yes
```

This example shows how to use ethtool to switch the bandwidth from 1000Mbit to 100Mbit and back. Note that some time passes before the nic is back to 1000Mbit.

```
root@laika:~# ethtool eth0 | grep Speed
Speed: 1000Mb/s
root@laika:~# ethtool -s eth0 speed 100
root@laika:~# ethtool eth0 | grep Speed
Speed: 100Mb/s
root@laika:~# ethtool -s eth0 speed 1000
root@laika:~# ethtool eth0 | grep Speed
Speed: 1000Mb/s
```

57.12. practice: interface configuration

1. Verify whether **dhclient** is running.
2. Display your current ip address(es).
3. Display the configuration file where this **ip address** is defined.
4. Follow the **nic configuration** in the book to change your ip address from **dhcp client** to **fixed**. Keep the same **ip address** to avoid conflicts!
5. Did you also configure the correct **gateway** in the previous question ? If not, then do this now.
6. Verify that you have a gateway.
7. Verify that you can connect to the gateway, that it is alive.
8. Change the last two digits of your **mac address**.
9. Which ports are used by http, pop3, ssh, telnet, nntp and ftp ?

Note that **sctp** was omitted from the screenshot.

10. Explain why e-mail and websites are sent over **tcp** and not **udp**.
11. Display the **hostname** of your computer.
12. Which ip-addresses did your computer recently have contact with ?

57.13. solution: interface configuration

1. Verify whether **dhclient** is running.

```
paul@debian5:~$ ps fax | grep dhclient
```

2. Display your current ip address(es).

```
paul@debian5:~$ /sbin/ifconfig | grep 'inet '
    inet addr:192.168.1.31 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:127.0.0.1 Mask:255.0.0.0
```

3. Display the configuration file where this **ip address** is defined.

```
Ubuntu/Debian: cat /etc/network/interfaces
Redhat/Fedora: cat /etc/sysconfig/network-scripts/ifcfg-eth*
```

4. Follow the **nic configuration** in the book to change your ip address from **dhcp client** to **fixed**. Keep the same **ip address** to avoid conflicts!

```
Ubuntu/Debian:
ifdown eth0
vi /etc/network/interfaces
ifup eth0
```

```
Redhat/Fedora:
ifdown eth0
vi /etc/sysconfig/network-scripts/ifcfg-eth0
ifup eth0
```

5. Did you also configure the correct **gateway** in the previous question ? If not, then do this now.

6. Verify that you have a gateway.

```
paul@debian5:~$ /sbin/route
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
192.168.1.0 * 255.255.255.0 U 0 0 0 eth0
default 192.168.1.1 0.0.0.0 UG 0 0 0 eth0
```

7. Verify that you can connect to the gateway, that it is alive.

```
paul@debian5:~$ ping -c3 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=254 time=2.28 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=254 time=2.94 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=254 time=2.34 ms

--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 2.283/2.524/2.941/0.296 ms
```

8. Change the last two digits of your **mac address**.

```
[root@rhel6 ~]# ifconfig eth0 hw ether 08:00:27:ab:67:XX
```

9. Which ports are used by http, pop3, ssh, telnet, nntp and ftp ?

```
root@rhel6 ~# grep '^http ' /etc/services
```

```
http      80/tcp      www www-http    # WorldWideWeb HTTP
http      80/udp      www www-http    # HyperText Transfer Protocol
root@rhel6 ~# grep '^smtp ' /etc/services
smtp      25/tcp      mail
smtp      25/udp      mail
root@rhel6 ~# grep '^ssh ' /etc/services
ssh       22/tcp      # The Secure Shell (SSH) Protocol
ssh       22/udp      # The Secure Shell (SSH) Protocol
root@rhel6 ~# grep '^telnet ' /etc/services
telnet    23/tcp
telnet    23/udp
root@rhel6 ~# grep '^nntp ' /etc/services
nntp      119/tcp      readnews untp   # USENET News Transfer Protocol
nntp      119/udp      readnews untp   # USENET News Transfer Protocol
root@rhel6 ~# grep '^ftp ' /etc/services
ftp       21/tcp
ftp       21/udp      fsp fspd
```

Note that **sctp** was omitted from the screenshot.

10. Explain why e-mail and websites are sent over **tcp** and not **udp**.

Because tcp is reliable and udp is not.

11. Display the **hostname** of your computer.

```
paul@debian5:~$ hostname
debian5
```

12. Which ip-addresses did your computer recently have contact with ?

```
root@rhel6 ~# arp -a
? (192.168.1.1) at 00:02:cf:aa:68:f0 [ether] on eth2
? (192.168.1.30) at 00:26:bb:12:7a:5e [ether] on eth2
? (192.168.1.31) at 08:00:27:8e:8a:a8 [ether] on eth2
```

Chapter 58. network sniffing

A network administrator should be able to use a sniffer like **wireshark** or **tcpdump** to troubleshoot network problems.

A student should often use a sniffer to learn about networking. This chapter introduces you to **network sniffing**.

58.1. wireshark

58.1.1. installing wireshark

This example shows how to install **wireshark** on **.deb** based distributions (including Debian, Mint, Xubuntu, and others).

```
root@debian8:~# apt-get install wireshark
Reading package lists... Done
Building dependency tree
Reading state information... Done
... (output truncated)
```

On **.rpm** based distributions like CentOS, RHEL and Fedora you can use **yum** to install **wireshark**.

```
[root@centos7 ~]# yum install wireshark
Loaded plugins: fastestmirror
Loading mirror speeds from cached hostfile
... (output truncated)
```

58.1.2. selecting interface

When you start **wireshark** for the first time, you will need to select an interface. You will see a dialog box that looks similar to this one.



It is possible that there are no interfaces available because some distributions only allow root to sniff the network. You may need to use **sudo wireshark**.

Or you can follow the general advice to sniff using **tcpdump** or any other tool, and save the capture to a file. Any saved capture can be analyzed using **wireshark** at a later time.

58.1.3. minimize traffic

Sniffing a network can generate many thousands of packets in a very short time. This can be overwhelming. Try to mitigate by isolating your sniffer on the network. Preferably sniff an isolated virtual network interface over which you control all traffic.

If you are at home to learn sniffing, then it could help to close all network programs on your computer, and disconnect other computers and devices like smartphones and tablets to minimize the traffic.

Even more important than this is the use of **filters** which will be discussed in this chapter.

58.1.4. sniffing ping

I started the sniffer and captured all packets while doing these three **ping** commands (there is no need for root to do this):

```
root@debian7:~# ping -c2 ns1.paul.local
PING ns1.paul.local (10.104.33.30) 56(84) bytes of data.
64 bytes from 10.104.33.30: icmp_req=1 ttl=64 time=0.010 ms
64 bytes from 10.104.33.30: icmp_req=2 ttl=64 time=0.023 ms

--- ns1.paul.local ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.010/0.016/0.023/0.007 ms
root@debian7:~# ping -c3 linux-training.be
PING linux-training.be (188.93.155.87) 56(84) bytes of data.
64 bytes from antares.ginsys.net (188.93.155.87): icmp_req=1 ttl=56 time=15.6 ms
64 bytes from antares.ginsys.net (188.93.155.87): icmp_req=2 ttl=56 time=17.8 ms
64 bytes from antares.ginsys.net (188.93.155.87): icmp_req=3 ttl=56 time=14.7 ms

--- linux-training.be ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 14.756/16.110/17.881/1.309 ms
root@debian7:~# ping -c1 centos7.paul.local
PING centos7.paul.local (10.104.33.31) 56(84) bytes of data.
64 bytes from 10.104.33.31: icmp_req=1 ttl=64 time=0.590 ms

--- centos7.paul.local ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.590/0.590/0.590/0.000 ms
```

In total more than 200 packets were sniffed from the network. Things become clearer when you enter **icmp** in the filter field and press the **apply** button.

Filter: icmp					Expression...	Clear	Apply	Save
No.	Source	Destination	Protocol	Info				
31	10.104.33.30	10.104.33.30	ICMP	Echo (ping) request id=0x09f6, seq=1/				
32	10.104.33.30	10.104.33.30	ICMP	Echo (ping) reply id=0x09f6, seq=1/				
47	10.104.33.30	10.104.33.30	ICMP	Echo (ping) request id=0x09f6, seq=2/				
48	10.104.33.30	10.104.33.30	ICMP	Echo (ping) reply id=0x09f6, seq=2/				
103	192.168.1.103	188.93.155.87	ICMP	Echo (ping) request id=0x09f7, seq=1/				
104	188.93.155.87	192.168.1.103	ICMP	Echo (ping) reply id=0x09f7, seq=1/				
115	192.168.1.103	188.93.155.87	ICMP	Echo (ping) request id=0x09f7, seq=2/				
116	188.93.155.87	192.168.1.103	ICMP	Echo (ping) reply id=0x09f7, seq=2/				
123	192.168.1.103	188.93.155.87	ICMP	Echo (ping) request id=0x09f7, seq=3/				
124	188.93.155.87	192.168.1.103	ICMP	Echo (ping) reply id=0x09f7, seq=3/				
170	10.104.33.30	10.104.33.31	ICMP	Echo (ping) request id=0x09f8, seq=1/				
171	10.104.33.31	10.104.33.30	ICMP	Echo (ping) reply id=0x09f8, seq=1/				

58.1.5. sniffing ping and dns

Using the same capture as before, but now with a different **filter**. We want to see both **dns** and **icmp** traffic, so we enter both in the filter field.

We put **dns or icmp** in the filter to achieve this. Putting **dns and icmp** would render nothing because there is no packet that matches both protocols.

Filter:	icmp or dns			▼	Expression...	Clear	Apply	Save
No.	Source	Destination	Protocol	Info				
25	10.104.33.30	10.104.33.30	DNS	Standard query 0xa668 A ns1.paul.local				
26	10.104.33.30	10.104.33.30	DNS	Standard query response 0xa668 A 10.104.33.30				
31	10.104.33.30	10.104.33.30	ICMP	Echo (ping) request id=0x09f6, seq=1/2!				
32	10.104.33.30	10.104.33.30	ICMP	Echo (ping) reply id=0x09f6, seq=1/2!				

In the screenshot above you can see that packets 25 and 26 both have 10.104.33.30 as **source** and **destination** ip address. That is because the dns client is the same computer as the dns server.

The same is true for packets 31 and 32, since the machine is actually pinging itself.

58.1.6. specific ip address

This is a screenshot that filters for **dns** packets that contain a certain **ip address**. The filter in use is **ip.addr==10.104.33.30 and dns**. The **and** directive forces each displayed packet to match both conditions.

Filter:		ip.addr==10.104.33.30 and dns		▼	Expression...	Clear	Apply	Save
No.	Source	Destination	Protocol	Info				
93	10.104.33.30	10.104.33.30	DNS	Standard query 0xa34a A linux-training.be				
98	10.104.33.30	10.104.33.30	DNS	Standard query response 0xa34a A 188.93.155.87				

Packet 93 is the **dns query** for the A record of linux-training.be. Packet 98 is the response from the **dns server**. What do you think happened in the packets between 93 and 98 ? Try to answer this before reading on (it always helps to try to predict what you will see, and then checking your prediction).

58.1.7. filtering by frame

The correct technical term for a **packet** as sniffed is a **frame** (because we sniff on layer two). So to display packets with certain numbers, we use **frame.number** in the filter.

Filter:	frame.number>92 and frame.number<99		Expression...	Clear	Apply	Save
No.	Source	Destination	Protocol	Info		
93	10.104.33.30	10.104.33.30	DNS	Standard query	0xa34a	A linux-training.be
94	192.168.1.103	8.8.8.8	DNS	Standard query	0xf008	A linux-training.be
95	192.168.1.103	8.8.8.8	DNS	Standard query	0x0ff5	NS <Root>
96	8.8.8.8	192.168.1.103	DNS	Standard query	response 0x0ff5	NS d.root-server
97	8.8.8.8	192.168.1.103	DNS	Standard query	response 0xf008	A 188.93.155.87
98	10.104.33.30	10.104.33.30	DNS	Standard query	response 0xa34a	A 188.93.155.87

58.1.8. looking inside packets

The middle pane can be expanded. When selecting a line in this pane, you can see the corresponding bytes in the frame in the bottom panel.

This screenshot shows the middle pane with the source address of my laptop selected.

The screenshot shows the Wireshark interface with Frame 1 selected. The packet details pane on the left shows the following structure:

- Frame 1: 77 bytes on wire (616 bits), 77 bytes captured (616 bits) on interface 0
- Ethernet II, Src: Apple_36:24:28 (b8:e8:56:36:24:28), Dst: IcpElect_c9:07:10 (00:08:9b:c9:07:10)
- Destination: IcpElect_c9:07:10 (00:08:9b:c9:07:10)
- Source: Apple_36:24:28 (b8:e8:56:36:24:28) - This line is selected.
- Type: IP (0x0800)
- Internet Protocol Version 4, Src: 192.168.1.35 (192.168.1.35), Dst: 192.168.1.42 (192.168.1.42)
- User Datagram Protocol, Src Port: 57676 (57676), Dst Port: 53 (53)
- Domain Name System (query)

The packet bytes pane on the right shows the raw data in hexadecimal and ASCII:

```

0000  00 08 9b c9 07 10 b8 e8 56 36 24 28 08 00 45 00  .... V6$(..E.
0010  00 3f 6f 73 40 00 40 11 47 9d c0 a8 01 23 c0 a8  .?os@.@. G....#..
0020  01 2a e1 4c 00 35 00 2b be 44 af c5 01 00 00 01  .*..L.5.+ .D.....
0030  00 00 00 00 00 00 0e 6c 69 6e 75 78 2d 74 72 61  ....l inux-tra
0040  69 6e 69 6e 67 02 62 65 00 00 01 00 01         ining.be .....
  
```

Note that the above works fine when sniffing one interface. When sniffing with for example **tcpdump -i any** you will end up with **Linux cooked** at this level.

The screenshot shows the Wireshark interface with Frame 25 selected. The packet details pane on the left shows the following structure:

- Frame 25: 76 bytes on wire (608 bits), 76 bytes captured (608 bits)
- Linux cooked capture
- Packet type: Unicast to us (0)
- Link-layer address type: 772
- Link-layer address length: 6
- Source: 00:00:00_00:00:00 (00:00:00:00:00:00)
- Protocol: IP (0x0800)
- Internet Protocol Version 4, Src: 10.104.33.30 (10.104.33.30), Dst: 10.104.33.30 (10.104.33.30)

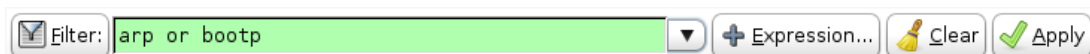
The packet bytes pane on the right shows the raw data in hexadecimal and ASCII:

```

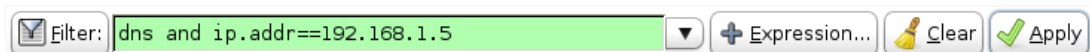
0000  00 00 03 04 00 06 00 00 00 00 00 00 00 08 00  ....
0010  45 00 00 3c 38 d6 40 00 40 11 aa cf 0a 68 21 1e  E...<8.@. @...h!.
0020  0a 68 21 1e 82 bd 00 35 00 28 57 45 a6 68 01 00  .h!....5 .(WE.h..
0030  00 01 00 00 00 00 00 00 03 6e 73 31 04 70 61 75  .... .nsl.pau
0040  6c 05 6c 6f 63 61 6c 00 00 01 00 01         l.local. ....
  
```

58.1.9. other filter examples

You can combine two protocols with a logical **or** between them. The example below shows how to filter only **arp** and **bootp** (or **dhcp**) packets.



This example shows how to filter for **dns** traffic containing a certain **ip address**.



58.2. tcpdump

Sniffing on the command line can be done with **tcpdump**. Here are some examples.

Using the **tcpdump host \$ip** command displays all traffic with one host (192.168.1.38 in this example).

```
root@ubuntu910:~# tcpdump host 192.168.1.38
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 96 bytes
```

Capturing only ssh (tcp port 22) traffic can be done with **tcpdump tcp port \$port**. This screenshot is cropped to 76 characters for readability in the pdf.

```
root@deb503:~# tcpdump tcp port 22
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), capture size 96 bytes
14:22:20.716313 IP deb503.local.37973 > rhel53.local.ssh: P 666050963:66605
14:22:20.719936 IP rhel53.local.ssh > deb503.local.37973: P 1:49(48) ack 48
14:22:20.720922 IP rhel53.local.ssh > deb503.local.37973: P 49:113(64) ack
14:22:20.721321 IP rhel53.local.ssh > deb503.local.37973: P 113:161(48) ack
14:22:20.721820 IP deb503.local.37973 > rhel53.local.ssh: . ack 161 win 200
14:22:20.722492 IP rhel53.local.ssh > deb503.local.37973: P 161:225(64) ack
14:22:20.760602 IP deb503.local.37973 > rhel53.local.ssh: . ack 225 win 200
14:22:23.108106 IP deb503.local.54424 > ubuntu910.local.ssh: P 467252637:46
14:22:23.116804 IP ubuntu910.local.ssh > deb503.local.54424: P 1:81(80) ack
14:22:23.116844 IP deb503.local.54424 > ubuntu910.local.ssh: . ack 81 win 2
^C
10 packets captured
10 packets received by filter
0 packets dropped by kernel
```

Same as above, but write the output to a file with the **tcpdump -w \$filename** command.

```
root@ubuntu910:~# tcpdump -w sshdump.tcpdump tcp port 22
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 96 bytes
^C
17 packets captured
17 packets received by filter
0 packets dropped by kernel
```

With **tcpdump -r \$filename** the file created above can be displayed.

```
root@ubuntu910:~# tcpdump -r sshdump.tcpdump
```

Many more examples can be found in the manual page of **tcpdump**.

58.3. practice: network sniffing

1. Install wireshark on your computer (not inside a virtual machine).
2. Start a ping between your computer and another computer.
3. Start sniffing the network.
4. Display only the ping echo's in the top pane using a filter.
5. Now ping to a name (like `www.linux-training.be`) and try to sniff the DNS query and response. Which DNS server was used ? Was it a tcp or udp query and response ?
6. Find an amateur/hobby/club website that features a login prompt. Attempt to login with user 'paul' and password 'hunter2' while your sniffer is running. Now find this information in the sniffer.

58.4. solution: network sniffing

1. Install wireshark on your computer (not inside a virtual machine).

Debian/Ubuntu: `aptitude install wireshark`

Red Hat/Mandriva/Fedora: `yum install wireshark`

2. Start a ping between your computer and another computer.

`ping $ip_address`

3. Start sniffing the network.

`(sudo) wireshark`

select an interface (probably eth0)

4. Display only the ping echo's in the top pane using a filter.

type 'icmp' (without quotes) in the filter box, and then click 'apply'

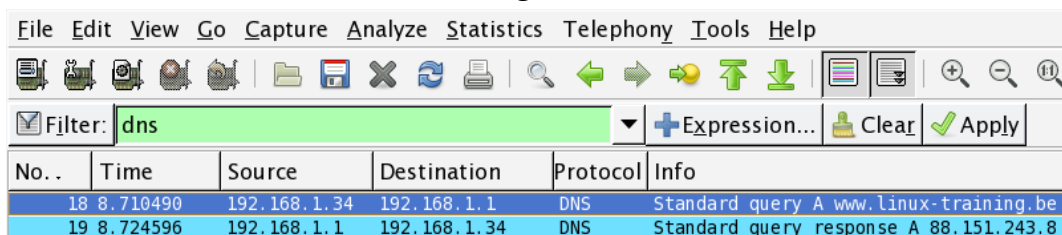
5. Now ping to a name (like `www.linux-training.be`) and try to sniff the DNS query and response. Which DNS server was used ? Was it a tcp or udp query and response ?

First start the sniffer.

Enter 'dns' in the filter box and click apply.

```
root@ubuntu910:~# ping www.linux-training.be
PING www.linux-training.be (88.151.243.8) 56(84) bytes of data.
64 bytes from fosfor.openminds.be (88.151.243.8): icmp_seq=1 ttl=58 time=14.9 ms
64 bytes from fosfor.openminds.be (88.151.243.8): icmp_seq=2 ttl=58 time=16.0 ms
^C
--- www.linux-training.be ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 14.984/15.539/16.095/0.569 ms
```

The wireshark screen should look something like this.



No..	Time	Source	Destination	Protocol	Info
18	8.710490	192.168.1.34	192.168.1.1	DNS	Standard query A www.linux-training.be
19	8.724596	192.168.1.1	192.168.1.34	DNS	Standard query response A 88.151.243.8

The details in wireshark will say the DNS query was inside a udp packet.

6. Find an amateur/hobby/club website that features a login prompt. Attempt to login with user 'paul' and password 'hunter2' while your sniffer is running. Now find this information in the sniffer.

Chapter 59. binding and bonding

Sometimes a server needs more than one **ip address** on the same network card, we call this **binding** ip addresses.

Linux can also activate multiple network cards behind the same **ip address**, this is called **bonding**.

This chapter will teach you how to configure **binding** and **bonding** on the most common Linux distributions.

59.1. binding on Redhat/Fedora

59.1.1. binding extra ip addresses

To bind more than one **ip address** to the same interface, use **ifcfg-eth0:0**, where the last zero can be anything else. Only two directives are required in the files.

```
[root@rhel6 ~]# cat /etc/sysconfig/network-scripts/ifcfg-eth0:0
DEVICE="eth0:0"
IPADDR="192.168.1.133"
[root@rhel6 ~]# cat /etc/sysconfig/network-scripts/ifcfg-eth0:1
DEVICE="eth0:0"
IPADDR="192.168.1.142"
```

59.1.2. enabling extra ip-addresses

To activate a virtual network interface, use **ifup**, to deactivate it, use **ifdown**.

```
[root@rhel6 ~]# ifup eth0:0
[root@rhel6 ~]# ifconfig | grep 'inet '
    inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:192.168.1.133 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:127.0.0.1 Mask:255.0.0.0
[root@rhel6 ~]# ifup eth0:1
[root@rhel6 ~]# ifconfig | grep 'inet '
    inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:192.168.1.133 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:192.168.1.142 Bcast:192.168.1.255 Mask:255.255.255.0
    inet addr:127.0.0.1 Mask:255.0.0.0
```

59.1.3. verifying extra ip-addresses

Use **ping** from another computer to check the activation, or use **ifconfig** like in this screenshot.

```
[root@rhel6 ~]# ifconfig
eth0  Link encap:Ethernet  HWaddr 08:00:27:DD:0D:5C
      inet addr:192.168.1.99 Bcast:192.168.1.255 Mask:255.255.255.0
      inet6 addr: fe80::a00:27ff:fedd:d5c/64 Scope:Link
      UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
      RX packets:1259 errors:0 dropped:0 overruns:0 frame:0
      TX packets:545 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:1000
      RX bytes:115260 (112.5 KiB)  TX bytes:84293 (82.3 KiB)

eth0:0 Link encap:Ethernet  HWaddr 08:00:27:DD:0D:5C
      inet addr:192.168.1.133 Bcast:192.168.1.255 Mask:255.255.255.0
      UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1

eth0:1 Link encap:Ethernet  HWaddr 08:00:27:DD:0D:5C
      inet addr:192.168.1.142 Bcast:192.168.1.255 Mask:255.255.255.0
      UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
```

59.2. binding on Debian/Ubuntu

59.2.1. binding extra ip addresses

The configuration of multiple ip addresses on the same network card is done in **/etc/network/interfaces** by adding **eth0:x** devices. Adding the **netmask** is mandatory.

```
debian5:~# cat /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
iface eth0 inet static
address 192.168.1.34
network 192.168.1.0
netmask 255.255.255.0
gateway 192.168.1.1
auto eth0

auto eth0:0
iface eth0:0 inet static
address 192.168.1.233
netmask 255.255.255.0

auto eth0:1
iface eth0:1 inet static
address 192.168.1.242
netmask 255.255.255.0
```

59.2.2. enabling extra ip-addresses

Use **ifup** to enable the extra addresses.

```
debian5:~# ifup eth0:0
debian5:~# ifup eth0:1
```

59.2.3. verifying extra ip-addresses

Use **ping** from another computer to check the activation, or use **ifconfig** like in this screenshot.

```
debian5:~# ifconfig | grep 'inet '
  inet addr:192.168.1.34  Bcast:192.168.1.255  Mask:255.255.255.0
  inet addr:192.168.1.233  Bcast:192.168.1.255  Mask:255.255.255.0
  inet addr:192.168.1.242  Bcast:192.168.1.255  Mask:255.255.255.0
  inet addr:127.0.0.1  Mask:255.0.0.0
```


59.3. bonding on Redhat/Fedora

We start with **ifconfig -a** to get a list of all the network cards on our system.

```
[root@rhel6 network-scripts]# ifconfig -a | grep Ethernet
eth0      Link encap:Ethernet  HWaddr 08:00:27:DD:0D:5C
eth1      Link encap:Ethernet  HWaddr 08:00:27:DA:C1:49
eth2      Link encap:Ethernet  HWaddr 08:00:27:40:03:3B
```

In this demo we decide to bond **eth1** and **eth2**.

We will name our bond **bond0** and add this entry to **modprobe** so the kernel can load the **bonding module** when we bring the interface up.

```
[root@rhel6 network-scripts]# cat /etc/modprobe.d/bonding.conf
alias bond0 bonding
```

Then we create **/etc/sysconfig/network-scripts/ifcfg-bond0** to configure our **bond0** interface.

```
[root@rhel6 network-scripts]# pwd
/etc/sysconfig/network-scripts
[root@rhel6 network-scripts]# cat ifcfg-bond0
DEVICE=bond0
IPADDR=192.168.1.199
NETMASK=255.255.255.0
ONBOOT=yes
BOOTPROTO=none
USERCTL=no
```

Next we create two files, one for each network card that we will use as slave in **bond0**.

```
[root@rhel6 network-scripts]# cat ifcfg-eth1
DEVICE=eth1
BOOTPROTO=none
ONBOOT=yes
MASTER=bond0
SLAVE=yes
USERCTL=no
[root@rhel6 network-scripts]# cat ifcfg-eth2
DEVICE=eth2
BOOTPROTO=none
ONBOOT=yes
MASTER=bond0
SLAVE=yes
USERCTL=no
```

Finally we bring the interface up with **ifup bond0**.

```
[root@rhel6 network-scripts]# ifup bond0
[root@rhel6 network-scripts]# ifconfig bond0
bond0      Link encap:Ethernet  HWaddr 08:00:27:DA:C1:49
            inet addr:192.168.1.199  Bcast:192.168.1.255  Mask:255.255.255.0
            inet6 addr: fe80::a00:27ff:feda:c149/64 Scope:Link
            UP BROADCAST RUNNING MASTER MULTICAST  MTU:1500  Metric:1
            RX packets:251 errors:0 dropped:0 overruns:0 frame:0
            TX packets:21 errors:0 dropped:0 overruns:0 carrier:0
            collisions:0 txqueuelen:0
            RX bytes:39852 (38.9 KiB)  TX bytes:1070 (1.0 KiB)
```

The **bond** should also be visible in **/proc/net/bonding**.

```
[root@rhel6 network-scripts]# cat /proc/net/bonding/bond0  
Ethernet Channel Bonding Driver: v3.5.0 (November 4, 2008)
```

```
Bonding Mode: load balancing (round-robin)  
MII Status: up  
MII Polling Interval (ms): 0  
Up Delay (ms): 0  
Down Delay (ms): 0
```

```
Slave Interface: eth1  
MII Status: up  
Link Failure Count: 0  
Permanent HW addr: 08:00:27:da:c1:49
```

```
Slave Interface: eth2  
MII Status: up  
Link Failure Count: 0  
Permanent HW addr: 08:00:27:40:03:3b
```

59.4. bonding on Debian/Ubuntu

We start with **ifconfig -a** to get a list of all the network cards on our system.

```
debian5:~# ifconfig -a | grep Ethernet
eth0      Link encap:Ethernet  HWaddr 08:00:27:bb:18:a4
eth1      Link encap:Ethernet  HWaddr 08:00:27:63:9a:95
eth2      Link encap:Ethernet  HWaddr 08:00:27:27:a4:92
```

In this demo we decide to bond **eth1** and **eth2**.

We also need to install the **ifenslave** package.

```
debian5:~# aptitude search ifenslave
p ifenslave      - Attach and detach slave interfaces to a bonding device
p ifenslave-2.6 - Attach and detach slave interfaces to a bonding device
debian5:~# aptitude install ifenslave
Reading package lists... Done
...
```

Next we update the **/etc/network/interfaces** file with information about the **bond0** interface.

```
debian5:~# tail -7 /etc/network/interfaces
iface bond0 inet static
    address 192.168.1.42
    netmask 255.255.255.0
    gateway 192.168.1.1
    slaves eth1 eth2
    bond-mode active-backup
    bond_primary eth1
```

On older version of Debian/Ubuntu you needed to **modprobe bonding**, but this is no longer required. Use **ifup** to bring the interface up, then test that it works.

```
debian5:~# ifup bond0
debian5:~# ifconfig bond0
bond0      Link encap:Ethernet  HWaddr 08:00:27:63:9a:95
            inet addr:192.168.1.42  Bcast:192.168.1.255  Mask:255.255.255.0
            inet6 addr: fe80::a00:27ff:fe63:9a95/64 Scope:Link
            UP BROADCAST RUNNING MASTER MULTICAST  MTU:1500  Metric:1
            RX packets:212 errors:0 dropped:0 overruns:0 frame:0
            TX packets:39 errors:0 dropped:0 overruns:0 carrier:0
            collisions:0 txqueuelen:0
            RX bytes:31978 (31.2 KiB)  TX bytes:6709 (6.5 KiB)
```

The **bond** should also be visible in **/proc/net/bonding**.

```
debian5:~# cat /proc/net/bonding/bond0
Ethernet Channel Bonding Driver: v3.2.5 (March 21, 2008)

Bonding Mode: fault-tolerance (active-backup)
Primary Slave: eth1
Currently Active Slave: eth1
MII Status: up
MII Polling Interval (ms): 0
Up Delay (ms): 0
Down Delay (ms): 0

Slave Interface: eth1
MII Status: up
Link Failure Count: 0
```

Permanent HW addr: 08:00:27:63:9a:95

Slave Interface: eth2

MII Status: up

Link Failure Count: 0

Permanent HW addr: 08:00:27:27:a4:92

59.5. practice: binding and bonding

1. Add an extra **ip address** to one of your network cards. Test that it works (have your neighbour ssh to it)!
2. Use **ifdown** to disable this extra **ip address**.
3. Make sure your neighbour also succeeded in **binding** an extra ip address before you continue.
4. Add an extra network card (or two) to your virtual machine and use the theory to **bond** two network cards.

59.6. solution: binding and bonding

1. Add an extra **ip address** to one of your network cards. Test that it works (have your neighbour ssh to it)!

```
Redhat/Fedora:  
add an /etc/sysconfig/network-scripts/ifcfg-ethX:X file  
as shown in the theory
```

```
Debian/Ubuntu:  
expand the /etc/network/interfaces file  
as shown in the theory
```

2. Use **ifdown** to disable this extra **ip address**.

```
ifdown eth0:0
```

3. Make sure your neighbour also succeeded in **binding** an extra ip address before you continue.

```
ping $extra_ip_neighbour  
or  
ssh $extra_ip_neighbour
```

4. Add an extra network card (or two) to your virtual machine and use the theory to **bond** two network cards.

```
Redhat/Fedora:  
add ifcfg-ethX and ifcfg-bondX files in /etc/sysconfig/network-scripts  
as shown in the theory  
and don't forget the modprobe.conf
```

```
Debian/Ubuntu:  
expand the /etc/network/interfaces file  
as shown in the theory  
and don't forget to install the ifenslave package
```

Chapter 60. ssh client and server

The **secure shell** or **ssh** is a collection of tools using a secure protocol for communications with remote Linux computers.

This chapter gives an overview of the most common commands related to the use of the **sshd** server and the **ssh** client.

60.1. about ssh

60.1.1. secure shell

Avoid using **telnet**, **rlogin** and **rsh** to remotely connect to your servers. These older protocols do not encrypt the login session, which means your user id and password can be sniffed by tools like **wireshark** or **tcpdump**. To securely connect to your servers, use **ssh**.

The **ssh protocol** is secure in two ways. Firstly the connection is **encrypted** and secondly the connection is **authenticated** both ways.

An ssh connection always starts with a cryptographic handshake, followed by **encryption** of the transport layer using a symmetric cypher. In other words, the tunnel is encrypted before you start typing anything.

Then **authentication** takes place (using user id/password or public/private keys) and communication can begin over the encrypted connection.

The **ssh protocol** will remember the servers it connected to (and warn you in case something suspicious happened).

The **openssh** package is maintained by the **OpenBSD** people and is distributed with a lot of operating systems (it may even be the most popular package in the world).

60.1.2. /etc/ssh/

Configuration of **ssh** client and server is done in the **/etc/ssh** directory. In the next sections we will discuss most of the files found in **/etc/ssh/**.

60.1.3. ssh protocol versions

The **ssh** protocol has two versions (1 and 2). Avoid using version 1 anywhere, since it contains some known vulnerabilities. You can control the protocol version via **/etc/ssh/ssh_config** for the client side and **/etc/ssh/sshd_config** for the openssh-server daemon.

```
paul@ubu1204:/etc/ssh$ grep Protocol ssh_config
# Protocol 2,1
paul@ubu1204:/etc/ssh$ grep Protocol sshd_config
Protocol 2
```


60.1.4. public and private keys

The **ssh** protocol uses the well known system of **public and private keys**. The below explanation is succinct, more information can be found on wikipedia.

http://en.wikipedia.org/wiki/Public-key_cryptography

Imagine Alice and Bob, two people that like to communicate with each other. Using **public and private keys** they can communicate with **encryption** and with **authentication**.

When Alice wants to send an encrypted message to Bob, she uses the **public key** of Bob. Bob shares his **public key** with Alice, but keeps his **private key** private! Since Bob is the only one to have Bob's **private key**, Alice is sure that Bob is the only one that can read the encrypted message.

When Bob wants to verify that the message came from Alice, Bob uses the **public key** of Alice to verify that Alice signed the message with her **private key**. Since Alice is the only one to have Alice's **private key**, Bob is sure the message came from Alice.

60.1.5. rsa and dsa algorithms

This chapter does not explain the technical implementation of cryptographic algorithms, it only explains how to use the ssh tools with **rsa** and **dsa**. More information about these algorithms can be found here:

[http://en.wikipedia.org/wiki/RSA_\(algorithm\)](http://en.wikipedia.org/wiki/RSA_(algorithm))
http://en.wikipedia.org/wiki/Digital_Signature_Algorithm

60.2. log on to a remote server

The following screenshot shows how to use **ssh** to log on to a remote computer running Linux. The local user is named **paul** and he is logging on as user **admin42** on the remote system.

```
paul@ubu1204:~$ ssh admin42@192.168.1.30
The authenticity of host '192.168.1.30 (192.168.1.30)' can't be established.
RSA key fingerprint is b5:fb:3c:53:50:b4:ab:81:f3:cd:2e:bb:ba:44:d3:75.
Are you sure you want to continue connecting (yes/no)?
```

As you can see, the user **paul** is presented with an **rsa** authentication fingerprint from the remote system. The user can accept this by typing **yes**. We will see later that an entry will be added to the `~/.ssh/known_hosts` file.

```
paul@ubu1204:~$ ssh admin42@192.168.1.30
The authenticity of host '192.168.1.30 (192.168.1.30)' can't be established.
RSA key fingerprint is b5:fb:3c:53:50:b4:ab:81:f3:cd:2e:bb:ba:44:d3:75.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.1.30' (RSA) to the list of known hosts.
admin42@192.168.1.30's password:
Welcome to Ubuntu 12.04 LTS (GNU/Linux 3.2.0-26-generic-pae i686)

 * Documentation:  https://help.ubuntu.com/

1 package can be updated.
0 updates are security updates.

Last login: Wed Jun  6 19:25:57 2012 from 172.28.0.131
admin42@ubuserver:~$
```

The user can get log out of the remote server by typing **exit** or by using **Ctrl-d**.

```
admin42@ubuserver:~$ exit
logout
Connection to 192.168.1.30 closed.
paul@ubu1204:~$
```

60.3. executing a command in remote

This screenshot shows how to execute the **pwd** command on the remote server. There is no need to **exit** the server manually.

```
paul@ubu1204:~$ ssh admin42@192.168.1.30 pwd
admin42@192.168.1.30's password:
/home/admin42
paul@ubu1204:~$
```

60.4. scp

The **scp** command works just like **cp**, but allows the source and destination of the copy to be behind **ssh**. Here is an example where we copy the **/etc/hosts** file from the remote server to the home directory of user paul.

```
paul@ubu1204:~$ scp admin42@192.168.1.30:/etc/hosts /home/paul/serverhosts
admin42@192.168.1.30's password:
hosts                               100% 809      0.8KB/s   00:00
```

Here is an example of the reverse, copying a local file to a remote server.

```
paul@ubu1204:~$ scp ~/serverhosts admin42@192.168.1.30:/etc/hosts.new
admin42@192.168.1.30's password:
serverhosts                         100% 809      0.8KB/s   00:00
```

60.5. setting up passwordless ssh

To set up passwordless ssh authentication through public/private keys, use **ssh-keygen** to generate a key pair without a passphrase, and then copy your public key to the destination server. Let's do this step by step.

In the example that follows, we will set up ssh without password between Alice and Bob. Alice has an account on a Red Hat Enterprise Linux server, Bob is using Ubuntu on his laptop. Bob wants to give Alice access using ssh and the public and private key system. This means that even if Bob changes his password on his laptop, Alice will still have access.

60.5.1. ssh-keygen

The example below shows how Alice uses **ssh-keygen** to generate a key pair. Alice does not enter a passphrase.

```
[alice@RHEL5 ~]$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/alice/.ssh/id_rsa):
Created directory '/home/alice/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/alice/.ssh/id_rsa.
Your public key has been saved in /home/alice/.ssh/id_rsa.pub.
The key fingerprint is:
9b:ac:ac:56:c2:98:e5:d9:18:c4:2a:51:72:bb:45:eb alice@RHEL5
[alice@RHEL5 ~]$
```

You can use **ssh-keygen -t dsa** in the same way.

60.5.2. ~/.ssh

While **ssh-keygen** generates a public and a private key, it will also create a hidden **.ssh** directory with proper permissions. If you create the **.ssh** directory manually, then you need to chmod 700 it! Otherwise ssh will refuse to use the keys (world readable private keys are not secure!).

As you can see, the **.ssh** directory is secure in Alice's home directory.

```
[alice@RHEL5 ~]$ ls -ld .ssh
drwx----- 2 alice alice 4096 May  1 07:38 .ssh
[alice@RHEL5 ~]$
```

Bob is using Ubuntu at home. He decides to manually create the **.ssh** directory, so he needs to manually secure it.

```
bob@laika:~$ mkdir .ssh
bob@laika:~$ ls -ld .ssh
drwxr-xr-x 2 bob bob 4096 2008-05-14 16:53 .ssh
bob@laika:~$ chmod 700 .ssh/
bob@laika:~$
```

60.5.3. id_rsa and id_rsa.pub

The **ssh-keygen** command generate two keys in **.ssh**. The public key is named **~/.ssh/id_rsa.pub**. The private key is named **~/.ssh/id_rsa**.

```
[alice@RHEL5 ~]$ ls -l .ssh/
total 16
-rw----- 1 alice alice 1671 May  1 07:38 id_rsa
-rw-r--r-- 1 alice alice  393 May  1 07:38 id_rsa.pub
```

The files will be named **id_dsa** and **id_dsa.pub** when using **dsa** instead of **rsa**.

60.5.4. copy the public key to the other computer

To copy the public key from Alice's server to Bob's laptop, Alice decides to use **scp**.

```
[alice@RHEL5 .ssh]$ scp id_rsa.pub bob@192.168.48.92:~/.ssh/authorized_keys
bob@192.168.48.92's password:
id_rsa.pub                                100% 393      0.4KB/s   00:00
```

Be careful when copying a second key! Do not overwrite the first key, instead append the key to the same **~/.ssh/authorized_keys** file!

```
cat id_rsa.pub >> ~/.ssh/authorized_keys
```

Alice could also have used **ssh-copy-id** like in this example.

```
ssh-copy-id -i .ssh/id_rsa.pub bob@192.168.48.92
```

60.5.5. authorized_keys

In your **~/.ssh** directory, you can create a file called **authorized_keys**. This file can contain one or more public keys from people you trust. Those trusted people can use their private keys to prove their identity and gain access to your account via **ssh** (without password). The example shows Bob's **authorized_keys** file containing the public key of Alice.

```
bob@laika:~$ cat .ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEApcQ9xzylZJes1sR+hPyqW2vyzt1D4zTLqk\
MDWBR4mMFuUZD/0583I3Lg/Q+JIq0RSksNzaL/BNLDou1jMpBe2Dmf/u22u4KmqLJBfDhe\
yTmGSBzeNYCYRSMq78CT9l9a+y6x/shucwhaILsy8A2XfJ9VCggkVtu7XlWFDL2cum08/0\
mRFwVrfc/uPsAn5XkkTscL4g21mQbnp9wJC40pGSJXXMuF0k8MgCb5ieSnpKFniAKM+tEo\
/vjDGSi3F/bxu691jscrU0VudIo0So98HUfEf7jKBRIkxGAC7I4HLA+/zX730IvRFab2hv\
tUhn6RhrBtUJjbsGgiYeFTLdfctQ== alice@RHEL5
```

60.5.6. passwordless ssh

Alice can now use **ssh** to connect passwordless to Bob's laptop. In combination with **ssh**'s capability to execute commands on the remote host, this can be useful in pipes across different machines.

```
[alice@RHEL5 ~]$ ssh bob@192.168.48.92 "ls -l .ssh"
total 4
-rw-r--r-- 1 bob bob 393 2008-05-14 17:03 authorized_keys
[alice@RHEL5 ~]$
```

60.6. X forwarding via ssh

Another popular feature of **ssh** is called **X11 forwarding** and is implemented with **ssh -X**.

Below an example of X forwarding: user paul logs in as user greet on her computer to start the graphical application mozilla-thunderbird. Although the application will run on the remote computer from greet, it will be displayed on the screen attached locally to paul's computer.

```
paul@debian5:~/PDF$ ssh -X greet@greet.dyndns.org -p 55555
Warning: Permanently added the RSA host key for IP address \
'81.240.174.161' to the list of known hosts.
Password:
Linux raika 2.6.8-2-686 #1 Tue Aug 16 13:22:48 UTC 2005 i686 GNU/Linux

Last login: Thu Jan 18 12:35:56 2007
greet@raika:~$ ps fax | grep thun
greet@raika:~$ mozilla-thunderbird &
[1] 30336
```

60.7. troubleshooting ssh

Use **ssh -v** to get debug information about the ssh connection attempt.

```
paul@debian5:~$ ssh -v bert@192.168.1.192
OpenSSH_4.3p2 Debian-8ubuntu1, OpenSSL 0.9.8c 05 Sep 2006
debug1: Reading configuration data /home/paul/.ssh/config
debug1: Reading configuration data /etc/ssh/ssh_config
debug1: Applying options for *
debug1: Connecting to 192.168.1.192 [192.168.1.192] port 22.
debug1: Connection established.
debug1: identity file /home/paul/.ssh/identity type -1
debug1: identity file /home/paul/.ssh/id_rsa type 1
debug1: identity file /home/paul/.ssh/id_dsa type -1
debug1: Remote protocol version 1.99, remote software version OpenSSH_3
debug1: match: OpenSSH_3.9p1 pat OpenSSH_3.*
debug1: Enabling compatibility mode for protocol 2.0
...
```

60.8. sshd

The ssh server is called **sshd** and is provided by the **openssh-server** package.

```
root@ubu1204~# dpkg -l openssh-server | tail -1
ii  openssh-server  1:5.9p1-5ubuntu1  secure shell (SSH) server,...
```

60.9. sshd keys

The public keys used by the sshd server are located in **/etc/ssh** and are world readable. The private keys are only readable by root.

```
root@ubu1204~# ls -l /etc/ssh/ssh_host_*
-rw----- 1 root root 668 Jun 7 2011 /etc/ssh/ssh_host_dsa_key
-rw-r--r-- 1 root root 598 Jun 7 2011 /etc/ssh/ssh_host_dsa_key.pub
-rw----- 1 root root 1679 Jun 7 2011 /etc/ssh/ssh_host_rsa_key
-rw-r--r-- 1 root root 390 Jun 7 2011 /etc/ssh/ssh_host_rsa_key.pub
```

60.10. ssh-agent

When generating keys with **ssh-keygen**, you have the option to enter a passphrase to protect access to the keys. To avoid having to type this passphrase every time, you can add the key to **ssh-agent** using **ssh-add**.

Most Linux distributions will start the **ssh-agent** automatically when you log on.

```
root@ubu1204~# ps -ef | grep ssh-agent
paul      2405   2365   0 08:13 ?        00:00:00 /usr/bin/ssh-agent...
```

This clipped screenshot shows how to use **ssh-add** to list the keys that are currently added to the **ssh-agent**

```
paul@debian5:~$ ssh-add -L
ssh-rsa AAAAB3NzaC1yc2EAAAABIwAAAQEAvgI+Vx5UrisusZPl8da8URHGsxG7yivv3/\
...
wMGqa48Kellwom8Tgb4Sgcwpp/V0/ldA5m+BGCw== paul@deb503
```

60.11. practice: ssh

0. Make sure that you have access to **two Linux computers**, or work together with a partner for this exercise. For this practice, we will name one of the machines the server.
1. Install **sshd** on the server
2. Verify in the ssh configuration files that only protocol version 2 is allowed.
3. Use **ssh** to log on to the server, show your current directory and then exit the server.
4. Use **scp** to copy a file from your computer to the server.
5. Use **scp** to copy a file from the server to your computer.
6. (optional, only works when you have a graphical install of Linux) Install the xeyes package on the server and use ssh to run xeyes on the server, but display it on your client.
7. (optional, same as previous) Create a bookmark in firefox, then quit firefox on client and server. Use **ssh -X** to run firefox on your display, but on your neighbour's computer. Do you see your neighbour's bookmark ?
8. Use **ssh-keygen** to create a key pair without passphrase. Setup passwordless ssh between you and your neighbour. (or between your client and your server)
9. Verify that the permissions on the server key files are correct; world readable for the public keys and only root access for the private keys.
10. Verify that the **ssh-agent** is running.
11. (optional) Protect your keypair with a **passphrase**, then add this key to the **ssh-agent** and test your passwordless ssh to the server.

60.12. solution: ssh

0. Make sure that you have access to **two Linux computers**, or work together with a partner for this exercise. For this practice, we will name one of the machines the server.

1. Install **sshd** on the server

```
apt-get install openssh-server (on Ubuntu/Debian)
yum -y install openssh-server (on Centos/Fedora/Red Hat)
```

2. Verify in the ssh configuration files that only protocol version 2 is allowed.

```
grep Protocol /etc/ssh/ssh*_config
```

3. Use **ssh** to log on to the server, show your current directory and then exit the server.

```
user@client$ ssh user@server-ip-address
user@server$ pwd
/home/user
user@server$ exit
```

4. Use **scp** to copy a file from your computer to the server.

```
scp localfile user@server:~
```

5. Use **scp** to copy a file from the server to your computer.

```
scp user@server:~/serverfile .
```

6. (optional, only works when you have a graphical install of Linux) Install the xeyes package on the server and use ssh to run xeyes on the server, but display it on your client.

```
on the server:
apt-get install xeyes
on the client:
ssh -X user@server-ip
xeyes
```

7. (optional, same as previous) Create a bookmark in firefox, then quit firefox on client and server. Use **ssh -X** to run firefox on your display, but on your neighbour's computer. Do you see your neighbour's bookmark ?

8. Use **ssh-keygen** to create a key pair without passphrase. Setup passwordless ssh between you and your neighbour. (or between your client and your server)

See solution in book "setting up passwordless ssh"

9. Verify that the permissions on the server key files are correct; world readable for the public keys and only root access for the private keys.

```
ls -l /etc/ssh/ssh_host_*
```

10. Verify that the **ssh-agent** is running.

```
ps fax | grep ssh-agent
```

11. (optional) Protect your keypair with a **passphrase**, then add this key to the **ssh-agent** and test your passwordless ssh to the server.

```
man ssh-keygen  
man ssh-agent  
man ssh-add
```

Chapter 61. introduction to nfs

The **network file system** (or simply **nfs**) enables us since the Eighties to share a directory with other computers on the network.

In this chapter we see how to setup an **nfs** server and an **nfs** client computer.

61.1. nfs protocol versions

The older **nfs** versions 2 and 3 are stateless (**udp**) by default (but they can use **tcp**). The more recent **nfs version 4** brings a stateful protocol with better performance and stronger security.

NFS version 4 was defined in **rfc 3010** in 2000 and **rfc 3530** in 2003 and requires **tcp** (port 2049). It also supports **Kerberos** user authentication as an option when mounting a share. NFS versions 2 and 3 authenticate only the host.

61.2. rpcinfo

Clients connect to the server using **rpc** (on Linux this can be managed by the **portmap** daemon). Look at **rpcinfo** to verify that **nfs** and its related services are running.

```
root@RHELv4u2:~# /etc/init.d/portmap status
portmap (pid 1920) is running...
root@RHELv4u2:~# rpcinfo -p
program vers proto  port
100000      2    tcp    111  portmapper
100000      2    udp    111  portmapper
100024      1    udp   32768  status
100024      1    tcp   32769  status
root@RHELv4u2:~# service nfs start
Starting NFS services:                [ OK ]
Starting NFS quotas:                  [ OK ]
Starting NFS daemon:                  [ OK ]
Starting NFS mountd:                  [ OK ]
```

The same **rpcinfo** command when **nfs** is started.

```
root@RHELv4u2:~# rpcinfo -p
program vers proto  port
100000      2    tcp    111  portmapper
100000      2    udp    111  portmapper
100024      1    udp   32768  status
100024      1    tcp   32769  status
100011      1    udp     985  rquotad
100011      2    udp     985  rquotad
100011      1    tcp     988  rquotad
100011      2    tcp     988  rquotad
100003      2    udp    2049  nfs
100003      3    udp    2049  nfs
100003      4    udp    2049  nfs
100003      2    tcp    2049  nfs
100003      3    tcp    2049  nfs
100003      4    tcp    2049  nfs
100021      1    udp   32770  nlockmgr
100021      3    udp   32770  nlockmgr
100021      4    udp   32770  nlockmgr
100021      1    tcp   32789  nlockmgr
100021      3    tcp   32789  nlockmgr
100021      4    tcp   32789  nlockmgr
100005      1    udp    1004  mountd
100005      1    tcp    1007  mountd
100005      2    udp    1004  mountd
100005      2    tcp    1007  mountd
100005      3    udp    1004  mountd
100005      3    tcp    1007  mountd
```

61.3. server configuration

nfs is configured in **/etc/exports**. You might want some way (**ldap**?) to synchronize **userid**'s across computers when using **nfs** a lot.

The **rootsquash** option will change UID 0 to the UID of a **nobody** (or similar) user account. The **sync** option will write writes to disk before completing the client request.

61.4. /etc/exports

Here is a sample **/etc/exports** to explain the syntax:

```
paul@laika:~$ cat /etc/exports
# Everyone can read this share
/mnt/data/iso *(ro)

# Only the computers named pasha and barry can readwrite this one
/var/www pasha(rw) barry(rw)

# same, but without root squashing for barry
/var/ftp pasha(rw) barry(rw,no_root_squash)

# everyone from the netsec.local domain gets access
/var/backup *.netsec.local(rw)

# ro for one network, rw for the other
/var/upload 192.168.1.0/24(ro) 192.168.5.0/24(rw)
```

More recent incarnations of **nfs** require the **subtree_check** option to be explicitly set (or unset with **no_subtree_check**). The **/etc/exports** file then looks like this:

```
root@debian6 ~# cat /etc/exports
# Everyone can read this share
/srv/iso *(ro,no_subtree_check)

# Only the computers named pasha and barry can readwrite this one
/var/www pasha(rw,no_subtree_check) barry(rw,no_subtree_check)

# same, but without root squashing for barry
/var/ftp pasha(rw,no_subtree_check) barry(rw,no_root_squash,no_subtree_check)
```

61.5. exportfs

You don't need to restart the **nfs** server to start exporting your newly created exports. You can use the **exportfs -va** command to do this. It will write the exported directories to **/var/lib/nfs/etab**, where they are immediately applied.

```
root@debian6 ~# exportfs -va
exporting pasha:/var/ftp
exporting barry:/var/ftp
exporting pasha:/var/www
exporting barry:/var/www
exporting */:/srv/iso
```

61.6. client configuration

We have seen the **mount** command and the **/etc/fstab** file before.

```
root@RHELv4u2:~# mount -t nfs barry:/mnt/data/iso /home/project55/
root@RHELv4u2:~# cat /etc/fstab | grep nfs
barry:/mnt/data/iso    /home/iso             nfs      defaults    0 0
root@RHELv4u2:~#
```

Here is another simple example. Suppose the project55 people tell you they only need a couple of CD-ROM images, and you already have them available on an **nfs** server. You could issue the following command to mount this storage on their **/home/project55** mount point.

```
root@RHELv4u2:~# mount -t nfs 192.168.1.40:/mnt/data/iso /home/project55/
root@RHELv4u2:~# ls -lh /home/project55/
total 3.6G
drwxr-xr-x  2 1000 1000 4.0K Jan 16 17:55 RHELv4u1
drwxr-xr-x  2 1000 1000 4.0K Jan 16 14:14 RHELv4u2
drwxr-xr-x  2 1000 1000 4.0K Jan 16 14:54 RHELv4u3
drwxr-xr-x  2 1000 1000 4.0K Jan 16 11:09 RHELv4u4
-rw-r--r--  1 root root 1.6G Oct 13 15:22 sled10-vmwarews5-vm.zip
root@RHELv4u2:~#
```

61.7. practice: introduction to nfs

1. Create two directories with some files. Use **nfs** to share one of them as read only, the other must be writable. Have your neighbour connect to them to test.
2. Investigate the user owner of the files created by your neighbour.
3. Protect a share by ip-address or hostname, so only your neighbour can connect.

Chapter 62. introduction to networking

62.1. introduction to iptables

62.1.1. iptables firewall

The Linux kernel has a built-in stateful firewall named **iptables**. To stop the **iptables** firewall on Red Hat, use the service command.

```
root@RHELv4u4:~# service iptables stop
Flushing firewall rules:                [ OK ]
Setting chains to policy ACCEPT: filter [ OK ]
Unloading iptables modules:            [ OK ]
root@RHELv4u4:~#
```

The easy way to configure iptables, is to use a graphical tool like KDE's **kmyfirewall** or **Security Level Configuration Tool**. You can find the latter in the graphical menu, somewhere in System Tools - Security, or you can start it by typing **system-config-securitylevel** in bash. These tools allow for some basic firewall configuration. You can decide whether to enable or disable the firewall, and what typical standard ports are allowed when the firewall is active. You can even add some custom ports. When you are done, the configuration is written to **/etc/sysconfig/iptables** on Red Hat.

```
root@RHELv4u4:~# cat /etc/sysconfig/iptables
# Firewall configuration written by system-config-securitylevel
# Manual customization of this file is not recommended.
*filter
:INPUT ACCEPT [0:0]
:FORWARD ACCEPT [0:0]
:OUTPUT ACCEPT [0:0]
:RH-Firewall-1-INPUT - [0:0]
-A INPUT -j RH-Firewall-1-INPUT
-A FORWARD -j RH-Firewall-1-INPUT
-A RH-Firewall-1-INPUT -i lo -j ACCEPT
-A RH-Firewall-1-INPUT -p icmp --icmp-type any -j ACCEPT
-A RH-Firewall-1-INPUT -p 50 -j ACCEPT
-A RH-Firewall-1-INPUT -p 51 -j ACCEPT
-A RH-Firewall-1-INPUT -p udp --dport 5353 -d 224.0.0.251 -j ACCEPT
-A RH-Firewall-1-INPUT -p udp -m udp --dport 631 -j ACCEPT
-A RH-Firewall-1-INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
-A RH-F...NPUT -m state --state NEW -m tcp -p tcp --dport 22 -j ACCEPT
-A RH-F...NPUT -m state --state NEW -m tcp -p tcp --dport 80 -j ACCEPT
-A RH-F...NPUT -m state --state NEW -m tcp -p tcp --dport 21 -j ACCEPT
-A RH-F...NPUT -m state --state NEW -m tcp -p tcp --dport 25 -j ACCEPT
-A RH-Firewall-1-INPUT -j REJECT --reject-with icmp-host-prohibited
COMMIT
root@RHELv4u4:~#
```

To start the service, issue the **service iptables start** command. You can configure iptables to start at boot time with **chkconfig**.

```
root@RHELv4u4:~# service iptables start
Applying iptables firewall rules:      [ OK ]
root@RHELv4u4:~# chkconfig iptables on
root@RHELv4u4:~#
```

One of the nice features of iptables is that it displays extensive **status** information when queried with the **service iptables status** command.

```

root@RHELV4u4:~# service iptables status
Table: filter
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
RH-Firewall-1-INPUT  all  --  0.0.0.0/0              0.0.0.0/0

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination
RH-Firewall-1-INPUT  all  --  0.0.0.0/0              0.0.0.0/0

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

Chain RH-Firewall-1-INPUT (2 references)
target     prot opt source                destination
ACCEPT     all  --  0.0.0.0/0              0.0.0.0/0
ACCEPT     icmp --  0.0.0.0/0              0.0.0.0/0      icmp type 255
ACCEPT     esp  --  0.0.0.0/0              0.0.0.0/0
ACCEPT     ah   --  0.0.0.0/0              0.0.0.0/0
ACCEPT     udp  --  0.0.0.0/0              224.0.0.251    udp dpt:5353
ACCEPT     udp  --  0.0.0.0/0              0.0.0.0/0      udp dpt:631
ACCEPT     all  --  0.0.0.0/0              0.0.0.0/0      state RELATED,ESTABLISHED
ACCEPT     tcp  --  0.0.0.0/0              0.0.0.0/0      state NEW tcp dpt:22
ACCEPT     tcp  --  0.0.0.0/0              0.0.0.0/0      state NEW tcp dpt:80
ACCEPT     tcp  --  0.0.0.0/0              0.0.0.0/0      state NEW tcp dpt:21
ACCEPT     tcp  --  0.0.0.0/0              0.0.0.0/0      state NEW tcp dpt:25
REJECT     all  --  0.0.0.0/0              0.0.0.0/0      reject-with icmp-host-prohibited

root@RHELV4u4:~#

```

Mastering firewall configuration requires a decent knowledge of tcp/ip. Good iptables tutorials can be found online here <http://iptables-tutorial.frozentux.net/iptables-tutorial.html> and here <http://tldp.org/HOWTO/IP-Masquerade-HOWTO/>.

62.2. practice : iptables

1. Verify whether the firewall is running.
2. Stop the running firewall.

62.3. solution : iptables

1. Verify whether the firewall is running.

```
root@rhel55 ~# service iptables status | head
Table: filter
Chain INPUT (policy ACCEPT)
num  target      prot opt source                destination
1    RH-Firewall-1-INPUT  all  --  0.0.0.0/0            0.0.0.0/0

Chain FORWARD (policy ACCEPT)
num  target      prot opt source                destination
1    RH-Firewall-1-INPUT  all  --  0.0.0.0/0            0.0.0.0/0

Chain OUTPUT (policy ACCEPT)
```

2. Stop the running firewall.

```
root@rhel55 ~# service iptables stop
Flushing firewall rules: [ OK ]
Setting chains to policy ACCEPT: filter [ OK ]
Unloading iptables modules: [ OK ]
root@rhel55 ~# service iptables status
Firewall is stopped.
```

62.4. xinetd and inetd

62.4.1. the superdaemon

Back when resources like RAM memory were limited, a super-server was devised to listen to all sockets and start the appropriate daemon only when needed. Services like **swat**, **telnet** and **ftp** are typically served by such a super-server. The **xinetd** superdaemon is more recent than **inetd**. We will discuss the configuration both daemons.

Recent Linux distributions like RHEL5 and Ubuntu10.04 do not activate **inetd** or **xinetd** by default, unless an application requires it.

62.4.2. inetd or xinetd

First verify whether your computer is running **inetd** or **xinetd**. This Debian 4.0 Etch is running **inetd**.

```
root@barry:~# ps fax | grep inet
3870 ?          Ss          0:00 /usr/sbin/inetd
```

This Red Hat Enterprise Linux 4 update 4 is running **xinetd**.

```
[root@RHEL4b ~]# ps fax | grep inet
3003 ?          Ss          0:00 xinetd -stayalive -pidfile /var/run/xinetd.pid
```

Both daemons have the same functionality (listening to many ports, starting other daemons when they are needed), but they have different configuration files.

62.4.3. xinetd superdaemon

The **xinetd** daemon is often called a superdaemon because it listens to a lot of incoming connections, and starts other daemons when they are needed. When a connection request is received, **xinetd** will first check TCP wrappers (/etc/hosts.allow and /etc/hosts.deny) and then give control of the connection to the other daemon. This superdaemon is configured through **/etc/xinetd.conf** and the files in the directory **/etc/xinetd.d**. Let's first take a look at **/etc/xinetd.conf**.

```
paul@RHELV4u2:~$ cat /etc/xinetd.conf
#
# Simple configuration file for xinetd
#
# Some defaults, and include /etc/xinetd.d/

defaults
{
    instances                = 60
    log_type                  = SYSLOG authpriv
    log_on_success            = HOST PID
    log_on_failure            = HOST
    cps                      = 25 30
```

```
}
includedir /etc/xinetd.d
paul@RHELv4u2:~$
```

According to the settings in this file, xinetd can handle 60 client requests at once. It uses the **authpriv** facility to log the host ip-address and pid of successful daemon spawns. When a service (aka protocol linked to daemon) gets more than 25 cps (connections per second), it holds subsequent requests for 30 seconds.

The directory **/etc/xinetd.d** contains more specific configuration files. Let's also take a look at one of them.

```
paul@RHELv4u2:~$ ls /etc/xinetd.d
amanda      chargen-udp  echo        klogin      rexec       talk
amandaidx   cups-lpd    echo-udp    krb5-telnet rlogin      telnet
amidxtape   daytime     eklogin     kshell      rsh         tftp
auth        daytime-udp finger      ktalk       rsync       time
chargen     dbuskdd-cdb gssftp      ntalk       swat        time-udp
paul@RHELv4u2:~$ cat /etc/xinetd.d/swat
# default: off
# description: SWAT is the Samba Web Admin Tool. Use swat \
#              to configure your Samba server. To use SWAT, \
#              connect to port 901 with your favorite web browser.
service swat
{
    port                = 901
    socket_type         = stream
    wait                = no
    only_from           = 127.0.0.1
    user                = root
    server              = /usr/sbin/swat
    log_on_failure      += USERID
    disable             = yes
}
paul@RHELv4u2:~$
```

The services should be listed in the **/etc/services** file. Port determines the service port, and must be the same as the port specified in **/etc/services**. The **socket_type** should be set to **stream** for tcp services (and to **dgram** for udp). The **log_on_failure** += concats the userid to the log message formatted in **/etc/xinetd.conf**. The last setting **disable** can be set to yes or no. Setting this to **no** means the service is enabled!

Check the xinetd and xinetd.conf manual pages for many more configuration options.

62.4.4. inetd superdaemon

This superdaemon has only one configuration file **/etc/inetd.conf**. Every protocol or daemon that it is listening for, gets one line in this file.

```
root@barry:~# grep ftp /etc/inetd.conf
tftp dgram udp wait nobody /usr/sbin/tcpd /usr/sbin/in.tftpd /boot/tftp
root@barry:~#
```

You can disable a service in `inetd.conf` above by putting a `#` at the start of that line. Here an example of the disabled vmware web interface (listening on tcp port 902).

```
paul@laika:~$ grep vmware /etc/inetd.conf
#902 stream tcp nowait root /usr/sbin/vmware-authd vmware-authd
```

62.5. practice : inetd and xinetd

1. Verify on all systems whether they are using xinetd or inetd.
2. Look at the configuration files.
3. (If telnet is installable, then replace swat in these questions with telnet) Is swat installed ? If not, then install swat and look at the changes in the (x)inetd configuration. Is swat enabled or disabled ?
4. Disable swat, test it. Enable swat, test it.

62.6. network file system

62.6.1. protocol versions

The older **nfs** versions 2 and 3 are stateless (udp) by default, but they can use tcp. Clients connect to the server using **rpc** (on Linux this is controlled by the **portmap** daemon. Look at **rpcinfo** to verify that **nfs** and its related services are running.

```
root@RHELv4u2:~# /etc/init.d/portmap status
portmap (pid 1920) is running...
root@RHELv4u2:~# rpcinfo -p
program vers proto  port
100000    2    tcp    111  portmapper
100000    2    udp    111  portmapper
100024    1    udp   32768  status
100024    1    tcp   32769  status
root@RHELv4u2:~# service nfs start
Starting NFS services:                [ OK ]
Starting NFS quotas:                  [ OK ]
Starting NFS daemon:                  [ OK ]
Starting NFS mountd:                  [ OK ]
```

The same **rpcinfo** command when **nfs** is started.

```
root@RHELv4u2:~# rpcinfo -p
program vers proto  port
100000    2    tcp    111  portmapper
100000    2    udp    111  portmapper
100024    1    udp   32768  status
100024    1    tcp   32769  status
100011    1    udp    985  rquotad
100011    2    udp    985  rquotad
100011    1    tcp    988  rquotad
100011    2    tcp    988  rquotad
100003    2    udp   2049  nfs
100003    3    udp   2049  nfs
100003    4    udp   2049  nfs
100003    2    tcp   2049  nfs
100003    3    tcp   2049  nfs
100003    4    tcp   2049  nfs
100021    1    udp   32770  nlockmgr
100021    3    udp   32770  nlockmgr
100021    4    udp   32770  nlockmgr
100021    1    tcp   32789  nlockmgr
100021    3    tcp   32789  nlockmgr
100021    4    tcp   32789  nlockmgr
100005    1    udp   1004  mountd
100005    1    tcp   1007  mountd
100005    2    udp   1004  mountd
100005    2    tcp   1007  mountd
100005    3    udp   1004  mountd
100005    3    tcp   1007  mountd
root@RHELv4u2:~#
```

nfs version 4 requires tcp (port 2049) and supports **Kerberos** user authentication as an option. **nfs** authentication only takes place when mounting the share. **nfs** versions 2 and 3 authenticate only the host.

62.6.2. server configuration

nfs is configured in **/etc/exports**. Here is a sample **/etc/exports** to explain the syntax. You need some way (NIS domain or LDAP) to synchronize userid's across computers when using **nfs** a lot. The **rootsquash** option will change UID 0 to the UID of the **nfsnobody** user account. The **sync** option will write writes to disk before completing the client request.

```
paul@laika:~$ cat /etc/exports
# Everyone can read this share
/mnt/data/iso *(ro)

# Only the computers barry and pasha can readwrite this one
/var/www pasha(rw) barry(rw)

# same, but without root squashing for barry
/var/ftp pasha(rw) barry(rw,no_root_squash)

# everyone from the netsec.lan domain gets access
/var/backup *.netsec.lan(rw)

# ro for one network, rw for the other
/var/upload 192.168.1.0/24(ro) 192.168.5.0/24(rw)
```

You don't need to restart the **nfs** server to start exporting your newly created exports. You can use the **exportfs -va** command to do this. It will write the exported directories to **/var/lib/nfs/etab**, where they are immediately applied.

62.6.3. client configuration

We have seen the **mount** command and the **/etc/fstab** file before.

```
root@RHELv4u2:~# mount -t nfs barry:/mnt/data/iso /home/project55/
root@RHELv4u2:~# cat /etc/fstab | grep nfs
barry:/mnt/data/iso /home/iso nfs defaults 0 0
root@RHELv4u2:~#
```

Here is another simple example. Suppose the **project55** people tell you they only need a couple of CD-ROM images, and you already have them available on an **nfs** server. You could issue the following command to mount this storage on their **/home/project55** mount point.

```
root@RHELv4u2:~# mount -t nfs 192.168.1.40:/mnt/data/iso /home/project55/
root@RHELv4u2:~# ls -lh /home/project55/
total 3.6G
drwxr-xr-x 2 1000 1000 4.0K Jan 16 17:55 RHELv4u1
drwxr-xr-x 2 1000 1000 4.0K Jan 16 14:14 RHELv4u2
drwxr-xr-x 2 1000 1000 4.0K Jan 16 14:54 RHELv4u3
drwxr-xr-x 2 1000 1000 4.0K Jan 16 11:09 RHELv4u4
-rw-r--r-- 1 root root 1.6G Oct 13 15:22 sled10-vmwarews5-vm.zip
root@RHELv4u2:~#
```


62.7. practice : network file system

1. Create two directories with some files. Use **nfs** to share one of them as read only, the other must be writable. Have your neighbour connect to them to test.
2. Investigate the user owner of the files created by your neighbour.
3. Protect a share by ip-address or hostname, so only your neighbour can connect.

Part XV. kernel management

Table of Contents

63. the Linux kernel	652
63.1. about the Linux kernel	653
63.2. Linux kernel source	656
63.3. kernel boot files	660
63.4. Linux kernel modules	662
63.5. compiling a kernel	667
63.6. compiling one module	670
64. library management	672
64.1. introduction	673
64.2. /lib and /usr/lib	673
64.3. ldd	673
64.4. ltrace	674
64.5. dpkg -S and debsums	674
64.6. rpm -qf and rpm -V	675
64.7. tracing with strace	676

Chapter 63. the Linux kernel

63.1. about the Linux kernel

63.1.1. kernel versions

In 1991 Linux Torvalds wrote (the first version of) the Linux kernel. He put it online, and other people started contributing code. Over 4000 individuals contributed source code to the latest kernel release (version 2.6.27 in November 2008).

Major Linux kernel versions used to come in even and odd numbers. Versions **2.0**, **2.2**, **2.4** and **2.6** are considered stable kernel versions. Whereas **2.1**, **2.3** and **2.5** were unstable (read development) versions. Since the release of 2.6.0 in January 2004, all development has been done in the 2.6 tree. There is currently no v2.7.x and according to Linus the even/stable vs odd/development scheme is abandoned forever.

63.1.2. `uname -r`

To see your current Linux kernel version, issue the **uname -r** command as shown below.

This first example shows Linux major version **2.6** and minor version **24**. The rest **-22-generic** is specific to the distribution (Ubuntu in this case).

```
paul@laika:~$ uname -r
2.6.24-22-generic
```

The same command on Red Hat Enterprise Linux shows an older kernel (2.6.18) with **-92.1.17.el5** being specific to the distribution.

```
[paul@RHEL52 ~]$ uname -r
2.6.18-92.1.17.el5
```

63.1.3. `/proc/cmdline`

The parameters that were passed to the kernel at boot time are in **/proc/cmdline**.

```
paul@RHELv4u4:~$ cat /proc/cmdline
ro root=/dev/VolGroup00/LogVol00 rhgb quiet
```

63.1.4. single user mode

When booting the kernel with the **single** parameter, it starts in **single user mode**. Linux can start in a bash shell with the **root** user logged on (without password).

Some distributions prevent the use of this feature (at kernel compile time).

63.1.5. init=/bin/bash

Normally the kernel invokes **init** as the first daemon process. Adding **init=/bin/bash** to the kernel parameters will instead invoke bash (again with root logged on without providing a password).

63.1.6. /var/log/messages

The kernel reports during boot to **syslog** which writes a lot of kernel actions in **/var/log/messages**. Looking at this file reveals when the kernel was started, including all the devices that were detected at boot time.

```
[root@RHEL53 ~]# grep -A16 "syslogd 1.4.1:" /var/log/messages|cut -b24-
syslogd 1.4.1: restart.
kernel: klogd 1.4.1, log source = /proc/kmsg started.
kernel: Linux version 2.6.18-128.el5 (mockbuild@hs20-bc1-5.build.red...)
kernel: BIOS-provided physical RAM map:
kernel: BIOS-e820: 0000000000000000 - 0000000000009f800 (usable)
kernel: BIOS-e820: 0000000000009f800 - 000000000000a0000 (reserved)
kernel: BIOS-e820: 000000000000ca000 - 000000000000cc000 (reserved)
kernel: BIOS-e820: 000000000000dc000 - 000000000000100000 (reserved)
kernel: BIOS-e820: 000000000000100000 - 0000000000001fef0000 (usable)
kernel: BIOS-e820: 0000000000001fef0000 - 0000000000001feff000 (ACPI data)
kernel: BIOS-e820: 0000000000001feff000 - 0000000000001fff0000 (ACPI NVS)
kernel: BIOS-e820: 0000000000001fff0000 - 00000000000020000000 (usable)
kernel: BIOS-e820: 000000000000fec00000 - 000000000000fec10000 (reserved)
kernel: BIOS-e820: 000000000000fee00000 - 000000000000fee01000 (reserved)
kernel: BIOS-e820: 000000000000ffe00000 - 000000000000100000000 (reserved)
kernel: 0MB HIGHMEM available.
kernel: 512MB LOWMEM available.
```

This example shows how to use **/var/log/messages** to see kernel information about **/dev/sda**.

```
[root@RHEL53 ~]# grep sda /var/log/messages | cut -b24-
kernel: SCSI device sda: 41943040 512-byte hdwr sectors (21475 MB)
kernel: sda: Write Protect is off
kernel: sda: cache data unavailable
kernel: sda: assuming drive cache: write through
kernel: SCSI device sda: 41943040 512-byte hdwr sectors (21475 MB)
kernel: sda: Write Protect is off
kernel: sda: cache data unavailable
kernel: sda: assuming drive cache: write through
kernel: sda: sda1 sda2
kernel: sd 0:0:0:0: Attached scsi disk sda
kernel: EXT3 FS on sda1, internal journal
```

63.1.7. dmesg

The **dmesg** command prints out all the kernel bootup messages (from the last boot).

```
[root@RHEL53 ~]# dmesg | head
Linux version 2.6.18-128.el5 (mockbuild@hs20-bc1-5.build.redhat.com)
BIOS-provided physical RAM map:
 BIOS-e820: 0000000000000000 - 0000000000009f800 (usable)
 BIOS-e820: 0000000000009f800 - 000000000000a0000 (reserved)
 BIOS-e820: 000000000000ca000 - 000000000000cc000 (reserved)
 BIOS-e820: 000000000000dc000 - 00000000000100000 (reserved)
 BIOS-e820: 00000000000100000 - 0000000001fef0000 (usable)
 BIOS-e820: 0000000001fef0000 - 0000000001feff000 (ACPI data)
 BIOS-e820: 0000000001feff000 - 0000000001ff00000 (ACPI NVS)
 BIOS-e820: 0000000001ff00000 - 00000000200000000 (usable)
```

Thus to find information about /dev/sda, using **dmesg** will yield only kernel messages from the last boot.

```
[root@RHEL53 ~]# dmesg | grep sda
SCSI device sda: 41943040 512-byte hdwr sectors (21475 MB)
sda: Write Protect is off
sda: Mode Sense: 5d 00 00 00
sda: cache data unavailable
sda: assuming drive cache: write through
SCSI device sda: 41943040 512-byte hdwr sectors (21475 MB)
sda: Write Protect is off
sda: Mode Sense: 5d 00 00 00
sda: cache data unavailable
sda: assuming drive cache: write through
sda: sda1 sda2
sd 0:0:0:0: Attached scsi disk sda
EXT3 FS on sda1, internal journal
```

63.2. Linux kernel source

63.2.1. ftp.kernel.org

The home of the Linux kernel source is **ftp.kernel.org**. It contains all official releases of the Linux kernel source code from 1991. It provides free downloads over http, ftp and rsync of all these releases, as well as changelogs and patches. More information can be obtained on the website **www.kernel.org**.

Anyone can anonymously use an ftp client to access ftp.kernel.org

```
paul@laika:~$ ftp ftp.kernel.org
Connected to pub3.kernel.org.
220 Welcome to ftp.kernel.org.
Name (ftp.kernel.org:paul): anonymous
331 Please specify the password.
Password:
230-      Welcome to the
230-
230-    LINUX KERNEL ARCHIVES
230-      ftp.kernel.org
```

All the Linux kernel versions are located in the pub/linux/kernel/ directory.

```
ftp> ls pub/linux/kernel/v*
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
drwxrwsr-x   2 536      536      4096 Mar 20  2003 v1.0
drwxrwsr-x   2 536      536     20480 Mar 20  2003 v1.1
drwxrwsr-x   2 536      536      8192 Mar 20  2003 v1.2
drwxrwsr-x   2 536      536     40960 Mar 20  2003 v1.3
drwxrwsr-x   3 536      536     16384 Feb 08  2004 v2.0
drwxrwsr-x   2 536      536     53248 Mar 20  2003 v2.1
drwxrwsr-x   3 536      536     12288 Mar 24  2004 v2.2
drwxrwsr-x   2 536      536     24576 Mar 20  2003 v2.3
drwxrwsr-x   5 536      536     28672 Dec 02 08:14 v2.4
drwxrwsr-x   4 536      536     32768 Jul 14  2003 v2.5
drwxrwsr-x   7 536      536    110592 Dec 05 22:36 v2.6
226 Directory send OK.
ftp>
```


63.2.2. /usr/src

On your local computer, the kernel source is located in **/usr/src**. Note though that the structure inside **/usr/src** might be different depending on the distribution that you are using.

First let's take a look at **/usr/src on Debian**. There appear to be two versions of the complete Linux source code there. Looking for a specific file (`e1000_main.c`) with `find` reveals it's exact location.

```
paul@barry:~$ ls -l /usr/src/
drwxr-xr-x 20 root root      4096 2006-04-04 22:12 linux-source-2.6.15
drwxr-xr-x 19 root root      4096 2006-07-15 17:32 linux-source-2.6.16
paul@barry:~$ find /usr/src -name e1000_main.c
/usr/src/linux-source-2.6.15/drivers/net/e1000/e1000_main.c
/usr/src/linux-source-2.6.16/drivers/net/e1000/e1000_main.c
```

This is very similar to **/usr/src on Ubuntu**, except there is only one kernel here (and it is newer).

```
paul@laika:~$ ls -l /usr/src/
drwxr-xr-x 23 root root      4096 2008-11-24 23:28 linux-source-2.6.24
paul@laika:~$ find /usr/src -name "e1000_main.c"
/usr/src/linux-source-2.6.24/drivers/net/e1000/e1000_main.c
```

Now take a look at **/usr/src on Red Hat Enterprise Linux**.

```
[paul@RHEL52 ~]$ ls -l /usr/src/
drwxr-xr-x 5 root root 4096 Dec  5 19:23 kernels
drwxr-xr-x 7 root root 4096 Oct 11 13:22 redhat
```

We will have to dig a little deeper to find the kernel source on Red Hat!

```
[paul@RHEL52 ~]$ cd /usr/src/redhat/BUILD/
[paul@RHEL52 BUILD]$ find . -name "e1000_main.c"
./kernel-2.6.18/linux-2.6.18.i686/drivers/net/e1000/e1000_main.c
```

63.2.3. downloading the kernel source

Debian

Installing the kernel source on Debian is really simple with **aptitude install linux-source**. You can do a search for all linux-source packages first, like in this screenshot.

```
root@barry:~# aptitude search linux-source
v   linux-source          -
v   linux-source-2.6      -
id  linux-source-2.6.15   - Linux kernel source for version 2.6.15
i   linux-source-2.6.16   - Linux kernel source for version 2.6.16
p   linux-source-2.6.18   - Linux kernel source for version 2.6.18
p   linux-source-2.6.24   - Linux kernel source for version 2.6.24
```

And then use **aptitude install** to download and install the Debian Linux kernel source code.

```
root@barry:~# aptitude install linux-source-2.6.24
```

When the aptitude is finished, you will see a new file named **/usr/src/linux-source-<version>.tar.bz2**

```
root@barry:/usr/src# ls -lh
drwxr-xr-x 20 root root 4.0K 2006-04-04 22:12 linux-source-2.6.15
drwxr-xr-x 19 root root 4.0K 2006-07-15 17:32 linux-source-2.6.16
-rw-r--r--  1 root root 45M 2008-12-02 10:56 linux-source-2.6.24.tar.bz2
```

Ubuntu

Ubuntu is based on Debian and also uses **aptitude**, so the task is very similar.

```
root@laika:~# aptitude search linux-source
i   linux-source          - Linux kernel source with Ubuntu patches
v   linux-source-2.6      -
i A linux-source-2.6.24    - Linux kernel source for version 2.6.24
root@laika:~# aptitude install linux-source
```

And when aptitude finishes, we end up with a **/usr/src/linux-source-<version>.tar.bz** file.

```
oot@laika:~# ll /usr/src
total 45M
-rw-r--r--  1 root root 45M 2008-11-24 23:30 linux-source-2.6.24.tar.bz2
```

Red Hat Enterprise Linux

The Red Hat kernel source is located on the fourth source cdrom. The file is called **kernel-2.6.9-42.EL.src.rpm** (example for RHELv4u4). It is also available online at <ftp://ftp.redhat.com/pub/redhat/linux/enterprise/5Server/en/os/SRPMS/> (example for RHEL5).

To download the kernel source on RHEL, use this long wget command (on one line, without the trailing \).

```
wget ftp://ftp.redhat.com/pub/redhat/linux/enterprise/5Server/en/os/\
SRPMS/kernel-`uname -r`.src.rpm
```

When the wget download is finished, you end up with a 60M .rpm file.

```
[root@RHEL52 src]# ll
total 60M
-rw-r--r-- 1 root root 60M Dec 5 20:54 kernel-2.6.18-92.1.17.el5.src.rpm
drwxr-xr-x 5 root root 4.0K Dec 5 19:23 kernels
drwxr-xr-x 7 root root 4.0K Oct 11 13:22 redhat
```

We will need to perform some more steps before this can be used as kernel source code.

First, we issue the **rpm -i kernel-2.6.9-42.EL.src.rpm** command to install this Red Hat package.

```
[root@RHEL52 src]# ll
total 60M
-rw-r--r-- 1 root root 60M Dec 5 20:54 kernel-2.6.18-92.1.17.el5.src.rpm
drwxr-xr-x 5 root root 4.0K Dec 5 19:23 kernels
drwxr-xr-x 7 root root 4.0K Oct 11 13:22 redhat
[root@RHEL52 src]# rpm -i kernel-2.6.18-92.1.17.el5.src.rpm
```

Then we move to the SPECS directory and perform an **rpmbuild**.

```
[root@RHEL52 ~]# cd /usr/src/redhat/SPECS
[root@RHEL52 SPECS]# rpmbuild -bp -vv --target=i686 kernel-2.6.spec
```

The rpmbuild command put the RHEL Linux kernel source code in **/usr/src/redhat/BUILD/kernel-<version>/.**

```
[root@RHEL52 kernel-2.6.18]# pwd
/usr/src/redhat/BUILD/kernel-2.6.18
[root@RHEL52 kernel-2.6.18]# ll
total 20K
drwxr-xr-x 2 root root 4.0K Dec 6 2007 config
-rw-r--r-- 1 root root 3.1K Dec 5 20:58 Config.mk
drwxr-xr-x 20 root root 4.0K Dec 5 20:58 linux-2.6.18.i686
drwxr-xr-x 19 root root 4.0K Sep 20 2006 vanilla
drwxr-xr-x 8 root root 4.0K Dec 6 2007 xen
```

63.3. kernel boot files

63.3.1. vmlinuz

The **vmlinuz** file in /boot is the compressed kernel.

```
paul@barry:~$ ls -lh /boot | grep vmlinuz
-rw-r--r-- 1 root root 1.2M 2006-03-06 16:22 vmlinuz-2.6.15-1-486
-rw-r--r-- 1 root root 1.1M 2006-03-06 16:30 vmlinuz-2.6.15-1-686
-rw-r--r-- 1 root root 1.3M 2008-02-11 00:00 vmlinuz-2.6.18-6-686
paul@barry:~$
```

63.3.2. initrd

The kernel uses **initrd** (an initial RAM disk) at boot time. The initrd is mounted before the kernel loads, and can contain additional drivers and modules. It is a **compressed cpio archive**, so you can look at the contents in this way.

```
root@RHELv4u4:/boot# mkdir /mnt/initrd
root@RHELv4u4:/boot# cp initrd-2.6.9-42.0.3.EL.img TMPinitrd.gz
root@RHELv4u4:/boot# gunzip TMPinitrd.gz
root@RHELv4u4:/boot# file TMPinitrd
TMPinitrd: ASCII cpio archive (SVR4 with no CRC)
root@RHELv4u4:/boot# cd /mnt/initrd/
root@RHELv4u4:/mnt/initrd# cpio -i | /boot/TMPinitrd
4985 blocks
root@RHELv4u4:/mnt/initrd# ls -l
total 76
drwxr-xr-x  2 root root 4096 Feb  5 08:36 bin
drwxr-xr-x  2 root root 4096 Feb  5 08:36 dev
drwxr-xr-x  4 root root 4096 Feb  5 08:36 etc
-rwxr-xr-x  1 root root 1607 Feb  5 08:36 init
drwxr-xr-x  2 root root 4096 Feb  5 08:36 lib
drwxr-xr-x  2 root root 4096 Feb  5 08:36 loopfs
drwxr-xr-x  2 root root 4096 Feb  5 08:36 proc
lrwxrwxrwx  1 root root    3 Feb  5 08:36 sbin -> bin
drwxr-xr-x  2 root root 4096 Feb  5 08:36 sys
drwxr-xr-x  2 root root 4096 Feb  5 08:36 sysroot
root@RHELv4u4:/mnt/initrd#
```

63.3.3. System.map

The **System.map** contains the symbol table and changes with every kernel compile. The symbol table is also present in **/proc/kallsyms** (pre 2.6 kernels name this file **/proc/ksyms**).

```
root@RHELv4u4:/boot# head System.map-`uname -r`
00000400 A __kernel_vsyscall
0000041a A SYSENTER_RETURN_OFFSET
00000420 A __kernel_sigreturn
00000440 A __kernel_rt_sigreturn
c0100000 A _text
c0100000 T startup_32
c01000c6 t checkCPUtype
c0100147 t is486
c010014e t is386
c010019f t L6
root@RHELv4u4:/boot# head /proc/kallsyms
c0100228 t _stext
c0100228 t calibrate_delay_direct
c0100228 t stext
c0100337 t calibrate_delay
c01004db t rest_init
c0100580 t do_pre_smp_initcalls
c0100585 t run_init_process
c01005ac t init
c0100789 t early_param_test
c01007ad t early_setup_test
root@RHELv4u4:/boot#
```

63.3.4. .config

The last file copied to the **/boot** directory is the kernel configuration used for compilation. This file is not necessary in the **/boot** directory, but it is common practice to put a copy there. It allows you to recompile a kernel, starting from the same configuration as an existing working one.

63.4. Linux kernel modules

63.4.1. about kernel modules

The Linux kernel is a monolithic kernel with loadable modules. These modules contain parts of the kernel used typically for device drivers, file systems and network protocols. Most of the time the necessary kernel modules are loaded automatically and dynamically without administrator interaction.

63.4.2. `/lib/modules`

The modules are stored in the `/lib/modules/<kernel-version>` directory. There is a separate directory for each kernel that was compiled for your system.

```
paul@laika:~$ ll /lib/modules/  
total 12K  
drwxr-xr-x 7 root root 4.0K 2008-11-10 14:32 2.6.24-16-generic  
drwxr-xr-x 8 root root 4.0K 2008-12-06 15:39 2.6.24-21-generic  
drwxr-xr-x 8 root root 4.0K 2008-12-05 12:58 2.6.24-22-generic
```

63.4.3. `<module>.ko`

The file containing the modules usually ends in `.ko`. This screenshot shows the location of the isdn module files.

```
paul@laika:~$ find /lib/modules -name isdn.ko  
/lib/modules/2.6.24-21-generic/kernel/drivers/isdn/i4l/isdn.ko  
/lib/modules/2.6.24-22-generic/kernel/drivers/isdn/i4l/isdn.ko  
/lib/modules/2.6.24-16-generic/kernel/drivers/isdn/i4l/isdn.ko
```

63.4.4. `lsmod`

To see a list of currently loaded modules, use **lsmod**. You see the name of each loaded module, the size, the use count, and the names of other modules using this one.

```
[root@RHEL52 ~]# lsmod | head -5  
Module                Size  Used by  
autofs4                24517  2  
hidp                   23105  2  
rfcomm                 42457  0  
l2cap                  29505  10 hidp, rfcomm
```

63.4.5. /proc/modules

/proc/modules lists all modules loaded by the kernel. The output would be too long to display here, so let's **grep** for the **vm** module.

We see that **vmmon** and **vmnet** are both loaded. You can display the same information with **lsmod**. Actually **lsmod** only reads and reformats the output of **/proc/modules**.

```
paul@laika:~$ cat /proc/modules | grep vm
vmnet 36896 13 - Live 0xffffffff88b21000 (P)
vmmon 194540 0 - Live 0xffffffff88af0000 (P)
paul@laika:~$ lsmod | grep vm
vmnet                36896  13
vmmon                194540  0
paul@laika:~$
```

63.4.6. module dependencies

Some modules depend on others. In the following example, you can see that the **nfsd** module is used by **exportfs**, **lockd** and **sunrpc**.

```
paul@laika:~$ cat /proc/modules | grep nfsd
nfsd 267432 17 - Live 0xffffffff88a40000
exportfs 7808 1 nfsd, Live 0xffffffff88a3d000
lockd 73520 3 nfs,nfsd, Live 0xffffffff88a2a000
sunrpc 185032 12 nfs,nfsd,lockd, Live 0xffffffff889fb000
paul@laika:~$ lsmod | grep nfsd
nfsd                267432  17
exportfs             7808  1 nfsd
lockd                73520  3 nfs,nfsd
sunrpc              185032  12 nfs,nfsd,lockd
paul@laika:~$
```

63.4.7. insmod

Kernel modules can be manually loaded with the **insmod** command. This is a very simple (and obsolete) way of loading modules. The screenshot shows **insmod** loading the fat module (for fat file system support).

```
root@barry:/lib/modules/2.6.17-2-686# lsmod | grep fat
root@barry:/lib/modules/2.6.17-2-686# insmod kernel/fs/fat/fat.ko
root@barry:/lib/modules/2.6.17-2-686# lsmod | grep fat
fat                46588  0
```

insmod is not detecting dependencies, so it fails to load the isdn module (because the isdn module depends on the slhc module).

```
[root@RHEL52 drivers]# pwd
/lib/modules/2.6.18-92.1.18.el5/kernel/drivers
[root@RHEL52 kernel]# insmod isdn/i4l/isdn.ko
insmod: error inserting 'isdn/i4l/isdn.ko': -1 Unknown symbol in module
```

63.4.8. modinfo

As you can see in the screenshot of **modinfo** below, the isdn module depends in the slhc module.

```
[root@RHEL52 drivers]# modinfo isdn/i4l/isdn.ko | head -6
filename:      isdn/i4l/isdn.ko
license:      GPL
author:       Fritz Elfert
description:   ISDN4Linux: link layer
srcversion:    99650346E708173496F6739
depends:       slhc
```

63.4.9. modprobe

The big advantage of **modprobe** over **insmod** is that modprobe will load all necessary modules, whereas insmod requires manual loading of dependencies. Another advantage is that you don't need to point to the filename with full path.

This screenshot shows how modprobe loads the isdn module, automatically loading slhc in background.

```
[root@RHEL52 kernel]# lsmod | grep isdn
[root@RHEL52 kernel]# modprobe isdn
[root@RHEL52 kernel]# lsmod | grep isdn
isdn                122433  0
slhc                 10561  1 isdn
[root@RHEL52 kernel]#
```


63.4.10. /lib/modules/<kernel>/modules.dep

Module dependencies are stored in **modules.dep**.

```
[root@RHEL52 2.6.18-92.1.18.el5]# pwd
/lib/modules/2.6.18-92.1.18.el5
[root@RHEL52 2.6.18-92.1.18.el5]# head -3 modules.dep
/lib/modules/2.6.18-92.1.18.el5/kernel/drivers/net/tokenring/3c359.ko:
/lib/modules/2.6.18-92.1.18.el5/kernel/drivers/net/pcmcia/3c574_cs.ko:
/lib/modules/2.6.18-92.1.18.el5/kernel/drivers/net/pcmcia/3c589_cs.ko:
```

63.4.11. depmod

The **modules.dep** file can be updated (recreated) with the **depmod** command. In this screenshot no modules were added, so **depmod** generates the same file.

```
root@barry:/lib/modules/2.6.17-2-686# ls -l modules.dep
-rw-r--r-- 1 root root 310676 2008-03-01 16:32 modules.dep
root@barry:/lib/modules/2.6.17-2-686# depmod
root@barry:/lib/modules/2.6.17-2-686# ls -l modules.dep
-rw-r--r-- 1 root root 310676 2008-12-07 13:54 modules.dep
```

63.4.12. rmmod

Similar to **insmod**, the **rmmod** command is rarely used anymore.

```
[root@RHELV4u3 ~]# modprobe isdn
[root@RHELV4u3 ~]# rmmod slhc
ERROR: Module slhc is in use by isdn
[root@RHELV4u3 ~]# rmmod isdn
[root@RHELV4u3 ~]# rmmod slhc
[root@RHELV4u3 ~]# lsmod | grep isdn
[root@RHELV4u3 ~]#
```

63.4.13. modprobe -r

Contrary to **rmmod**, **modprobe** will automatically remove unneeded modules.

```
[root@RHELV4u3 ~]# modprobe isdn
[root@RHELV4u3 ~]# lsmod | grep isdn
isdn          133537  0
slhc          7233  1 isdn
[root@RHELV4u3 ~]# modprobe -r isdn
[root@RHELV4u3 ~]# lsmod | grep isdn
[root@RHELV4u3 ~]# lsmod | grep slhc
[root@RHELV4u3 ~]#
```

63.4.14. /etc/modprobe.conf

The **/etc/modprobe.conf** file and the **/etc/modprobe.d** directory can contain aliases (used by humans) and options (for dependent modules) for **modprobe**.

```
[root@RHEL52 ~]# cat /etc/modprobe.conf
alias scsi_hostadapter mptbase
alias scsi_hostadapter1 mptspi
alias scsi_hostadapter2 ata_piix
alias eth0 pcnet32
alias eth2 pcnet32
```

```
alias eth1 pcnet32
```

63.5. compiling a kernel

63.5.1. extraversion

Enter into `/usr/src/redhat/BUILD/kernel-2.6.9/linux-2.6.9/` and change the **extraversion** in the Makefile.

```
[root@RHEL52 linux-2.6.18.i686]# pwd
/usr/src/redhat/BUILD/kernel-2.6.18/linux-2.6.18.i686
[root@RHEL52 linux-2.6.18.i686]# vi Makefile
[root@RHEL52 linux-2.6.18.i686]# head -4 Makefile
VERSION = 2
PATCHLEVEL = 6
SUBLEVEL = 18
EXTRAVERSION = -paul2008
```

63.5.2. make mrproper

Now clean up the source from any previous installs with **make mrproper**. If this is your first after downloading the source code, then this is not needed.

```
[root@RHEL52 linux-2.6.18.i686]# make mrproper
CLEAN    scripts/basic
CLEAN    scripts/kconfig
CLEAN    include/config
CLEAN    .config .config.old
```

63.5.3. .config

Now copy a working **.config** from `/boot` to our kernel directory. This file contains the configuration that was used for your current working kernel. It determines whether modules are included in compilation or not.

```
[root@RHEL52 linux-2.6.18.i686]# cp /boot/config-2.6.18-92.1.18.el5 .config
```

63.5.4. make menuconfig

Now run **make menuconfig** (or the graphical **make xconfig**). This tool allows you to select whether to compile stuff as a module (m), as part of the kernel (*), or not at all (smaller kernel size). If you remove too much, your kernel will not work. The configuration will be stored in the hidden **.config** file.

```
[root@RHEL52 linux-2.6.18.i686]# make menuconfig
```

63.5.5. make clean

Issue a **make clean** to prepare the kernel for compile. **make clean** will remove most generated files, but keeps your kernel configuration. Running a **make mrproper** at this point would destroy the **.config** file that you built with **make menuconfig**.

```
[root@RHEL52 linux-2.6.18.i686]# make clean
```

63.5.6. make bzImage

And then run **make bzImage**, sit back and relax while the kernel compiles. You can use **time make bzImage** to know how long it takes to compile, so next time you can go for a short walk.

```
[root@RHEL52 linux-2.6.18.i686]# time make bzImage
HOSTCC  scripts/basic/fixdep
HOSTCC  scripts/basic/docproc
HOSTCC  scripts/kconfig/conf.o
HOSTCC  scripts/kconfig/kxgettext.o
...
```

This command will end with telling you the location of the **bzImage** file (and with time info if you also specified the time command).

```
Kernel: arch/i386/boot/bzImage is ready  (#1)

real 13m59.573s
user 1m22.631s
sys 11m51.034s
[root@RHEL52 linux-2.6.18.i686]#
```

You can already copy this image to /boot with **cp arch/i386/boot/bzImage /boot/vmlinuz-<kernel-version>**.

63.5.7. make modules

Now run **make modules**. It can take 20 to 50 minutes to compile all the modules.

```
[root@RHEL52 linux-2.6.18.i686]# time make modules
CHK      include/linux/version.h
CHK      include/linux/utsrelease.h
CC [M]   arch/i386/kernel/msr.o
CC [M]   arch/i386/kernel/cpuid.o
CC [M]   arch/i386/kernel/microcode.o
```

63.5.8. make modules_install

To copy all the compiled modules to **/lib/modules** just run **make modules_install** (takes about 20 seconds). Here's a screenshot from before the command.

```
[root@RHEL52 linux-2.6.18.i686]# ls -l /lib/modules/
total 20
drwxr-xr-x 6 root root 4096 Oct 15 13:09 2.6.18-92.1.13.el5
drwxr-xr-x 6 root root 4096 Nov 11 08:51 2.6.18-92.1.17.el5
drwxr-xr-x 6 root root 4096 Dec  6 07:11 2.6.18-92.1.18.el5
[root@RHEL52 linux-2.6.18.i686]# make modules_install
```

And here is the same directory after. Notice that **make modules_install** created a new directory for the new kernel.

```
[root@RHEL52 linux-2.6.18.i686]# ls -l /lib/modules/
total 24
drwxr-xr-x 6 root root 4096 Oct 15 13:09 2.6.18-92.1.13.el5
drwxr-xr-x 6 root root 4096 Nov 11 08:51 2.6.18-92.1.17.el5
drwxr-xr-x 6 root root 4096 Dec  6 07:11 2.6.18-92.1.18.el5
drwxr-xr-x 3 root root 4096 Dec  6 08:50 2.6.18-paul2008
```

63.5.9. /boot

We still need to copy the kernel, the System.map and our configuration file to /boot. Strictly speaking the .config file is not obligatory, but it might help you in future compilations of the kernel.

```
[root@RHEL52 ]# pwd
/usr/src/redhat/BUILD/kernel-2.6.18/linux-2.6.18.i686
[root@RHEL52 ]# cp System.map /boot/System.map-2.6.18-paul2008
[root@RHEL52 ]# cp .config /boot/config-2.6.18-paul2008
[root@RHEL52 ]# cp arch/i386/boot/bzImage /boot/vmlinuz-2.6.18-paul2008
```

63.5.10. mkinitrd

The kernel often uses an initrd file at bootup. We can use **mkinitrd** to generate this file. Make sure you use the correct kernel name!

```
[root@RHEL52 ]# pwd
/usr/src/redhat/BUILD/kernel-2.6.18/linux-2.6.18.i686
[root@RHEL52 ]# mkinitrd /boot/initrd-2.6.18-paul2008 2.6.18-paul2008
```

63.5.11. bootloader

Compilation is now finished, don't forget to create an additional stanza in grub or lilo.

63.6. compiling one module

63.6.1. hello.c

A little C program that will be our module.

```
[root@rhel4a kernel_module]# cat hello.c
#include <linux/module.h>
#include <section>

int init_module(void)
{
    printk(KERN_INFO "Start Hello World...\n");
    return 0;
}

void cleanup_module(void)
{
    printk(KERN_INFO "End Hello World... \n");
}
```

63.6.2. Makefile

The make file for this module.

```
[root@rhel4a kernel_module]# cat Makefile
obj-m += hello.o
all:
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

These are the only two files needed.

```
[root@rhel4a kernel_module]# ll
total 16
-rw-rw-r-- 1 paul paul 250 Feb 15 19:14 hello.c
-rw-rw-r-- 1 paul paul 153 Feb 15 19:15 Makefile
```

63.6.3. make

The running of the **make** command.

```
[root@rhel4a kernel_module]# make
make -C /lib/modules/2.6.9-paul-2/build M=~/.kernel_module modules
make[1]: Entering dir... `/usr/src/redhat/BUILD/kernel-2.6.9/linux-2.6.9'
CC [M] /home/paul/kernel_module/hello.o
Building modules, stage 2.
MODPOST
CC      /home/paul/kernel_module/hello.mod.o
LD [M] /home/paul/kernel_module/hello.ko
make[1]: Leaving dir... `/usr/src/redhat/BUILD/kernel-2.6.9/linux-2.6.9'
[root@rhel4a kernel_module]#
```

Now we have more files.

```
[root@rhel4a kernel_module]# ll
total 172
-rw-rw-r-- 1 paul paul 250 Feb 15 19:14 hello.c
-rw-r--r-- 1 root root 64475 Feb 15 19:15 hello.ko
-rw-r--r-- 1 root root 632 Feb 15 19:15 hello.mod.c
-rw-r--r-- 1 root root 37036 Feb 15 19:15 hello.mod.o
-rw-r--r-- 1 root root 28396 Feb 15 19:15 hello.o
-rw-rw-r-- 1 paul paul 153 Feb 15 19:15 Makefile
[root@rhel4a kernel_module]#
```

63.6.4. hello.ko

Use **modinfo** to verify that it is really a module.

```
[root@rhel4a kernel_module]# modinfo hello.ko
filename:      hello.ko
vermagic:      2.6.9-paul-2 SMP 686 REGPARM 4KSTACKS gcc-3.4
depends:
[root@rhel4a kernel_module]#
```

Good, so now we can load our hello module.

```
[root@rhel4a kernel_module]# lsmod | grep hello
[root@rhel4a kernel_module]# insmod ./hello.ko
[root@rhel4a kernel_module]# lsmod | grep hello
hello                5504  0
[root@rhel4a kernel_module]# tail -1 /var/log/messages
Feb 15 19:16:07 rhel4a kernel: Start Hello World...
[root@rhel4a kernel_module]# rmmod hello
[root@rhel4a kernel_module]#
```

Finally **/var/log/messages** has a little surprise.

```
[root@rhel4a kernel_module]# tail -2 /var/log/messages
Feb 15 19:16:07 rhel4a kernel: Start Hello World...
Feb 15 19:16:35 rhel4a kernel: End Hello World...
[root@rhel4a kernel_module]#
```

Chapter 64. library management

64.1. introduction

With **libraries** we are talking about dynamically linked libraries (aka shared objects). These are binaries that contain functions and are not started themselves as programs, but are called by other binaries.

Several programs can use the same library. The name of the library file usually starts with **lib**, followed by the actual name of the library, then the characters **.so** and finally a version number.

64.2. /lib and /usr/lib

When you look at the **/lib** or the **/usr/lib** directory, you will see a lot of symbolic links. Most **libraries** have a detailed version number in their name, but receive a symbolic link from a filename which only contains the major version number.

```
root@rhel53 ~# ls -l /lib/libext*
lrwxrwxrwx 1 root root 16 Feb 18 16:36 /lib/libext2fs.so.2 -> libext2fs.so.2.4
-rwxr-xr-x 1 root root 113K Jun 30 2009 /lib/libext2fs.so.2.4
```

64.3. ldd

Many programs have dependencies on the installation of certain libraries. You can display these dependencies with **ldd**.

This example shows the dependencies of the **su** command.

```
paul@RHEL5 ~$ ldd /bin/su
linux-gate.so.1 => (0x003f7000)
libpam.so.0 => /lib/libpam.so.0 (0x00d5c000)
libpam_misc.so.0 => /lib/libpam_misc.so.0 (0x0073c000)
libcrypt.so.1 => /lib/libcrypt.so.1 (0x00aa4000)
libdl.so.2 => /lib/libdl.so.2 (0x00800000)
libc.so.6 => /lib/libc.so.6 (0x00ec1000)
libaudit.so.0 => /lib/libaudit.so.0 (0x0049f000)
/lib/ld-linux.so.2 (0x4769c000)
```

64.4. ltrace

The **ltrace** program allows to see all the calls made to library functions by a program. The example below uses the **-c** option to get only a summary count (there can be many calls), and the **-l** option to only show calls in one library file. All this to see what calls are made when executing **su - serena** as root.

```
root@deb503:~# ltrace -c -l /lib/libpam.so.0 su - serena
serena@deb503:~$ exit
logout
```

% time	seconds	usecs/call	calls	function
70.31	0.014117	14117	1	pam_start
12.36	0.002482	2482	1	pam_open_session
5.17	0.001039	1039	1	pam_acct_mgmt
4.36	0.000876	876	1	pam_end
3.36	0.000675	675	1	pam_close_session
3.22	0.000646	646	1	pam_authenticate
0.48	0.000096	48	2	pam_set_item
0.27	0.000054	54	1	pam_setcred
0.25	0.000050	50	1	pam_getenvlist
0.22	0.000044	44	1	pam_get_item
100.00	0.020079		11	total

64.5. dpkg -S and debsums

Find out on Debian/Ubuntu to which package a library belongs.

```
paul@deb503:/lib$ dpkg -S libext2fs.so.2.4
e2fslibs: /lib/libext2fs.so.2.4
```

You can then verify the integrity of all files in this package using **debsums**.

```
paul@deb503:~$ debsums e2fslibs
/usr/share/doc/e2fslibs/changelog.Debian.gz      OK
/usr/share/doc/e2fslibs/copyright                OK
/lib/libe2p.so.2.3                              OK
/lib/libext2fs.so.2.4                           OK
```

Should a library be broken, then reinstall it with **aptitude reinstall \$package**.

```
root@deb503:~# aptitude reinstall e2fslibs
Reading package lists... Done
Building dependency tree
Reading state information... Done
Reading extended state information
Initializing package states... Done
Reading task descriptions... Done
The following packages will be REINSTALLED:
  e2fslibs
...
```

64.6. rpm -qf and rpm -V

Find out on Red Hat/Fedora to which package a library belongs.

```
paul@RHEL5 ~$ rpm -qf /lib/libext2fs.so.2.4
e2fsprogs-libs-1.39-8.el5
```

You can then use **rpm -V** to verify all files in this package. In the example below the output shows that the **S**ize and the **T**ime stamp of the file have changed since installation.

```
root@rhel53 ~# rpm -V e2fsprogs-libs
prelink: /lib/libext2fs.so.2.4: prelinked file size differs
S.?....T    /lib/libext2fs.so.2.4
```

You can then use **yum reinstall \$package** to overwrite the existing library with an original version.

```
root@rhel53 lib# yum reinstall e2fsprogs-libs
Loaded plugins: rhnplugin, security
Setting up Reinstall Process
Resolving Dependencies
--> Running transaction check
---> Package e2fsprogs-libs.i386 0:1.39-23.el5 set to be erased
---> Package e2fsprogs-libs.i386 0:1.39-23.el5 set to be updated
--> Finished Dependency Resolution
...
```

The package verification now reports no problems with the library.

```
root@rhel53 lib# rpm -V e2fsprogs-libs
root@rhel53 lib#
```

64.7. tracing with strace

More detailed tracing of all function calls can be done with **strace**. We start by creating a read only file.

```
root@deb503:~# echo hello > 42.txt
root@deb503:~# chmod 400 42.txt
root@deb503:~# ls -l 42.txt
-r----- 1 root root 6 2011-09-26 12:03 42.txt
```

We open the file with **vi**, but include the **strace** command with an output file for the trace before **vi**. This will create a file with all the function calls done by **vi**.

```
root@deb503:~# strace -o strace.txt vi 42.txt
```

The file is read only, but we still change the contents, and use the **:w!** directive to write to this file. Then we close **vi** and take a look at the trace log.

```
root@deb503:~# grep chmod strace.txt
chmod("42.txt", 0100600)      = -1 ENOENT (No such file or directory)
chmod("42.txt", 0100400)      = 0
root@deb503:~# ls -l 42.txt
-r----- 1 root root 12 2011-09-26 12:04 42.txt
```

Notice that **vi** changed the permissions on the file twice. The trace log is too long to show a complete screenshot in this book.

```
root@deb503:~# wc -l strace.txt
941 strace.txt
```

Part XVI. backup management

Table of Contents

65. backup	679
65.1. About tape devices	679
65.2. Compression	680
65.3. tar	680
65.4. Backup Types	682
65.5. dump and restore	683
65.6. cpio	683
65.7. dd	684
65.8. split	685
65.9. practice: backup	685

Chapter 65. backup

65.1. About tape devices

Don't forget that the name of a device strictly speaking has no meaning since the kernel will use the major and minor number to find the hardware! See the man page of **mknod** and the `devices.txt` file in the Linux kernel source for more info.

65.1.1. SCSI tapes

On the official Linux device list (<http://www.lanana.org/docs/device-list/>) we find the names for SCSI tapes (major 9 char). SCSI tape devices are located underneath **/dev/st** and are numbered starting with 0 for the first tape device.

```
/dev/st0    First tape device
/dev/st1    Second tape device
/dev/st2    Third tape device
```

To prevent **automatic rewinding of tapes**, prefix them with the letter **n**.

```
/dev/nst0   First no rewind tape device
/dev/nst1   Second no rewind tape device
/dev/nst2   Third no rewind tape device
```

By default, SCSI tapes on Linux will use the highest hardware compression that is supported by the tape device. To lower the compression level, append one of the letters **l** (low), **m** (medium) or **a** (auto) to the tape name.

```
/dev/st0l   First low compression tape device
/dev/st0m   First medium compression tape device
/dev/nst2m  Third no rewind medium compression tape device
```

65.1.2. IDE tapes

On the official Linux device list (<http://www.lanana.org/docs/device-list/>) we find the names for IDE tapes (major 37 char). IDE tape devices are located underneath **/dev/ht** and are numbered starting with 0 for the first tape device. No rewind and compression is similar to SCSI tapes.

```
/dev/ht0    First IDE tape device
/dev/nht0    Second no rewind IDE tape device
/dev/ht0m    First medium compression IDE tape device
```

65.1.3. mt

To manage your tapes, use **mt** (Magnetic Tape). Some examples.

To receive information about the status of the tape.

```
mt -f /dev/st0 status
```

To rewind a tape...

```
mt -f /dev/st0 rewind
```

To rewind and eject a tape...

```
mt -f /dev/st0 eject
```

To erase a tape...

```
mt -f /dev/st0 erase
```

65.2. Compression

It can be beneficial to compress files before backup. The two most popular tools for compression of regular files on Linux are **gzip/gunzip** and **bzip2/bunzip2**. Below you can see gzip in action, notice that it adds the **.gz** extension to the file.

```
paul@RHELv4u4:~/test$ ls -l allfiles.tx*
-rw-rw-r-- 1 paul paul 8813553 Feb 27 05:38 allfiles.txt
paul@RHELv4u4:~/test$ gzip allfiles.txt
paul@RHELv4u4:~/test$ ls -l allfiles.tx*
-rw-rw-r-- 1 paul paul 931863 Feb 27 05:38 allfiles.txt.gz
paul@RHELv4u4:~/test$ gunzip allfiles.txt.gz
paul@RHELv4u4:~/test$ ls -l allfiles.tx*
-rw-rw-r-- 1 paul paul 8813553 Feb 27 05:38 allfiles.txt
paul@RHELv4u4:~/test$
```

In general, gzip is much faster than bzip2, but the latter one compresses a lot better. Let us compare the two.

```
paul@RHELv4u4:~/test$ cp allfiles.txt bllfiles.txt
paul@RHELv4u4:~/test$ time gzip allfiles.txt

real    0m0.050s
user    0m0.041s
sys     0m0.009s
paul@RHELv4u4:~/test$ time bzip2 bllfiles.txt

real    0m5.968s
user    0m5.794s
sys     0m0.076s
paul@RHELv4u4:~/test$ ls -l ?llfiles.tx*
-rw-rw-r-- 1 paul paul 931863 Feb 27 05:38 allfiles.txt.gz
-rw-rw-r-- 1 paul paul 708871 May 12 10:52 bllfiles.txt.bz2
paul@RHELv4u4:~/test$
```

65.3. tar

The **tar** utility gets its name from **Tape ARchive**. This tool will receive and send files to a destination (typically a tape or a regular file). The **c** option is used to create a tar archive

(or tarfile), the **f** option to name/create the **tarfile**. The example below takes a backup of /etc into the file /backup/etc.tar .

```
root@RHELv4u4:~# tar cf /backup/etc.tar /etc
root@RHELv4u4:~# ls -l /backup/etc.tar
-rw-r--r-- 1 root root 47800320 May 12 11:47 /backup/etc.tar
root@RHELv4u4:~#
```

Compression can be achieved without pipes since tar uses the **z** flag to compress with gzip, and the **j** flag to compress with bzip2.

```
root@RHELv4u4:~# tar czf /backup/etc.tar.gz /etc
root@RHELv4u4:~# tar cjf /backup/etc.tar.bz2 /etc
root@RHELv4u4:~# ls -l /backup/etc.ta*
-rw-r--r-- 1 root root 47800320 May 12 11:47 /backup/etc.tar
-rw-r--r-- 1 root root 6077340 May 12 11:48 /backup/etc.tar.bz2
-rw-r--r-- 1 root root 8496607 May 12 11:47 /backup/etc.tar.gz
root@RHELv4u4:~#
```

The **t** option is used to **list the contents of a tar file**. Verbose mode is enabled with **v** (also useful when you want to see the files being archived during archiving).

```
root@RHELv4u4:~# tar tvf /backup/etc.tar
drwxr-xr-x root/root      0 2007-05-12 09:38:21 etc/
-rw-r--r-- root/root    2657 2004-09-27 10:15:03 etc/warnquota.conf
-rw-r--r-- root/root   13136 2006-11-03 17:34:50 etc/mime.types
drwxr-xr-x root/root      0 2004-11-03 13:35:50 etc/sound/
...
```

To **list a specific file in a tar archive**, use the **t** option, added with the filename (without leading /).

```
root@RHELv4u4:~# tar tvf /backup/etc.tar etc/resolv.conf
-rw-r--r-- root/root      77 2007-05-12 08:31:32 etc/resolv.conf
root@RHELv4u4:~#
```

Use the **x** flag to **restore a tar archive**, or a single file from the archive. Remember that by default tar will restore the file in the current directory.

```
root@RHELv4u4:~# tar xvf /backup/etc.tar etc/resolv.conf
etc/resolv.conf
root@RHELv4u4:~# ls -l /etc/resolv.conf
-rw-r--r-- 2 root root 40 May 12 12:05 /etc/resolv.conf
root@RHELv4u4:~# ls -l etc/resolv.conf
-rw-r--r-- 1 root root 77 May 12 08:31 etc/resolv.conf
root@RHELv4u4:~#
```

You can **preserve file permissions** with the **p** flag. And you can exclude directories or file with **--exclude**.

```
root ~# tar cpzf /backup/etc_with_perms.tgz /etc
```

```
root ~# tar cpzf /backup/etc_no_sysconf.tgz /etc --exclude /etc/sysconfig
root ~# ls -l /backup/etc_*
-rw-r--r-- 1 root root 8434293 May 12 12:48 /backup/etc_no_sysconf.tgz
-rw-r--r-- 1 root root 8496591 May 12 12:48 /backup/etc_with_perms.tgz
root ~#
```

You can also create a text file with names of files and directories to archive, and then supply this file to tar with the -T flag.

```
root@RHELv4u4:~# find /etc -name *.conf > files_to_archive.txt
root@RHELv4u4:~# find /home -name *.pdf >> files_to_archive.txt
root@RHELv4u4:~# tar cpzf /backup/backup.tgz -T files_to_archive.txt
```

The tar utility can receive filenames from the find command, with the help of xargs.

```
find /etc -type f -name "*.conf" | xargs tar czf /backup/confs.tar.gz
```

You can also use tar to copy a directory, this is more efficient than using cp -r.

```
(cd /etc; tar -cf - . ) | (cd /backup/copy_of_etc/; tar -xpf - )
```

Another example of tar, this copies a directory securely over the network.

```
(cd /etc;tar -cf - . )|(ssh user@srv 'cd /backup/cp_of_etc/; tar -xf - ')
```

tar can be used together with gzip and copy a file to a remote server through ssh

```
cat backup.tar | gzip | ssh bashuser@192.168.1.105 "cat - > backup.tgz"
```

Compress the tar backup when it is on the network, but leave it uncompressed at the destination.

```
cat backup.tar | gzip | ssh user@192.168.1.105 "gunzip|cat - > backup.tar"
```

Same as the previous, but let ssh handle the compression

```
cat backup.tar | ssh -C bashuser@192.168.1.105 "cat - > backup.tar"
```

65.4. Backup Types

Linux uses **multilevel incremental** backups using distinct levels. A full backup is a backup at level 0. A higher level x backup will include all changes since the last level x-1 backup.

Suppose you take a full backup on Monday (level 0) and a level 1 backup on Tuesday, then the Tuesday backup will contain all changes since Monday. Taking a level 2 on Wednesday

will contain all changes since Tuesday (the last level 2-1). A level 3 backup on Thursday will contain all changes since Wednesday (the last level 3-1). Another level 3 on Friday will also contain all changes since Wednesday. A level 2 backup on Saturday would take all changes since the last level 1 from Tuesday.

65.5. dump and restore

While **dump** is similar to tar, it is also very different because it looks at the file system. Where tar receives a lists of files to backup, dump will find files to backup by itself by examining ext2. Files found by dump will be copied to a tape or regular file. In case the target is not big enough to hold the dump (end-of-media), it is broken into multiple volumes.

Restoring files that were backed up with dump is done with the **restore** command. In the example below we take a full level 0 backup of two partitions to a SCSI tape. The no rewind is mandatory to put the volumes behind each other on the tape.

```
dump 0f /dev/nst0 /boot
dump 0f /dev/nst0 /
```

Listing files in a dump archive is done with **dump -t**, and you can compare files with **dump -C**.

You can omit files from a dump by changing the dump attribute with the **chattr** command. The d attribute on ext will tell dump to skip the file, even during a full backup. In the following example, /etc/hosts is excluded from dump archives.

```
chattr +d /etc/hosts
```

To restore the complete file system with **restore**, use the -r option. This can be useful to change the size or block size of a file system. You should have a clean file system mounted and cd'd into it. Like this example shows.

```
mke2fs /dev/hda3
mount /dev/hda3 /mnt/data
cd /mnt/data
restore rf /dev/nst0
```

To extract only one file or directory from a dump, use the -x option.

```
restore -xf /dev/st0 /etc
```

65.6. cpio

Different from tar and dump is **cpio** (Copy Input and Output). It can be used to receive filenames, but copies the actual files. This makes it an easy companion with find! Some examples below.

find sends filenames to cpio, which puts the files in an archive.

```
find /etc -depth -print | cpio -oaV -O archive.cpio
```

The same, but compressed with gzip

```
find /etc -depth -print | cpio -oaV | gzip -c > archive.cpio.gz
```

Now pipe it through ssh (backup files to a compressed file on another machine)

```
find /etc -depth -print|cpio -oaV|gzip -c|ssh server "cat - > etc.cpio.gz"
```

find sends filenames to cpio | cpio sends files to ssh | ssh sends files to cpio 'cpio extracts files'

```
find /etc -depth -print | cpio -oaV | ssh user@host 'cpio -imVd'
```

the same but reversed: copy a dir from the remote host to the local machine

```
ssh user@host "find path -depth -print | cpio -oaV" | cpio -imVd
```

65.7. dd

65.7.1. About dd

Some people use **dd** to create backups. This can be very powerful, but dd backups can only be restored to very similar partitions or devices. There are however a lot of useful things possible with dd. Some examples.

65.7.2. Create a CDROM image

The easiest way to create a **.ISO file** from any CD. The if switch means Input File, of is the Output File. Any good tool can burn a copy of the CD with this .ISO file.

```
dd if=/dev/cdrom of=/path/to/cdrom.ISO
```

65.7.3. Create a floppy image

A little outdated maybe, but just in case : make an image file from a 1.44MB floppy. Blocksize is defined by bs, and count contains the number of blocks to copy.

```
dd if=/dev/floppy of=/path/to/floppy.img bs=1024 count=1440
```

65.7.4. Copy the master boot record

Use dd to copy the **MBR** (Master Boot Record) of hard disk /dev/hda to a file.

```
dd if=/dev/hda of=/MBR.img bs=512 count=1
```

65.7.5. Copy files

This example shows how dd can copy files. Copy the file summer.txt to copy_of_summer.txt .

```
dd if=~ /summer.txt of=~ /copy_of_summer.txt
```

65.7.6. Image disks or partitions

And who needs ghost when dd can create a (compressed) image of a partition.

```
dd if=/dev/hdb2 of=/image_of_hdb2.IMG
dd if=/dev/hdb2 | gzip > /image_of_hdb2.IMG.gz
```

65.7.7. Create files of a certain size

dd can be used to create a file of any size. The first example creates a one MEBIbyte file, the second a one MEGAbyte file.

```
dd if=/dev/zero of=file1MB count=1024 bs=1024
dd if=/dev/zero of=file1MB count=1000 bs=1024
```

65.7.8. CDROM server example

And there are of course endless combinations with ssh and bzip2. This example puts a bzip2 backup of a cdrom on a remote server.

```
dd if=/dev/cdrom |bzip2|ssh user@host "cat - > /backups/cd/cdrom.iso.bz2"
```

65.8. split

The **split** command is useful to split files into smaller files. This can be useful to fit the file onto multiple instances of a medium too small to contain the complete file. In the example below, a file of size 5000 bytes is split into three smaller files, with maximum 2000 bytes each.

```
paul@laika:~/test$ ls -l
total 8
-rw-r--r-- 1 paul paul 5000 2007-09-09 20:46 bigfile1
paul@laika:~/test$ split -b 2000 bigfile1 splitfile.
paul@laika:~/test$ ls -l
total 20
-rw-r--r-- 1 paul paul 5000 2007-09-09 20:46 bigfile1
-rw-r--r-- 1 paul paul 2000 2007-09-09 20:47 splitfile.aa
-rw-r--r-- 1 paul paul 2000 2007-09-09 20:47 splitfile.ab
-rw-r--r-- 1 paul paul 1000 2007-09-09 20:47 splitfile.ac
```

65.9. practice: backup

!! Careful with tar options and the position of the backup file, mistakes can destroy your system!!

1. Create a directory (or partition if you like) for backups. Link (or mount) it under /mnt/backup.

- 2a. Use tar to backup /etc in /mnt/backup/etc_date.tgz, the backup must be gzipped. (Replace date with the current date)
- 2b. Use tar to backup /bin to /mnt/backup/bin_date.tar.bz2, the backup must be bzip2'd.
- 2c. Choose a file in /etc and /bin and verify with tar that the file is indeed backed up.
- 2d. Extract those two files to your home directory.
- 3a. Create a backup directory for your neighbour, make it accessible under /mnt/neighbourName
- 3b. Combine ssh and tar to put a backup of your /boot on your neighbours computer in /mnt/YourName
- 4a. Combine find and cpio to create a cpio archive of /etc.
- 4b. Choose a file in /etc and restore it from the cpio archive into your home directory.
- 5. Use dd and ssh to put a backup of the master boot record on your neighbours computer.
- 6. (On the real computer) Create and mount an ISO image of the ubuntu cdrom.
- 7. Combine dd and gzip to create a 'ghost' image of one of your partitions on another partition.
- 8. Use dd to create a five megabyte file in ~/testsplit and name it biggest. Then split this file in smaller two megabyte parts.

```
mkdir testsplit
```

```
dd if=/dev/zero of=~/testsplit/biggest count=5000 bs=1024
```

```
split -b 2000000 biggest parts
```

Part XVII. apache and squid

Table of Contents

66. apache web server	689
66.1. introduction to apache	690
66.2. port virtual hosts on Debian	697
66.3. named virtual hosts on Debian	701
66.4. password protected website on Debian	703
66.5. port virtual hosts on CentOS	704
66.6. named virtual hosts on CentOS	708
66.7. password protected website on CentOS	710
66.8. troubleshooting apache	712
66.9. virtual hosts example	713
66.10. aliases and redirects	713
66.11. more on .htaccess	713
66.12. traffic	713
66.13. self signed cert on Debian	714
66.14. self signed cert on RHEL/CentOS	716
66.15. practice: apache	718
67. introduction to squid	719
67.1. about proxy servers	719
67.2. installing squid	720
67.3. port 3128	720
67.4. starting and stopping	720
67.5. client proxy settings	721
67.6. upside down images	723
67.7. /var/log/squid	725
67.8. access control	725
67.9. testing squid	725
67.10. name resolution	725

Chapter 66. apache web server

In this chapter we learn how to setup a web server with the **apache** software.

According to NetCraft (http://news.netcraft.com/archives/web_server_survey.html) about seventy percent of all web servers are running on Apache. The name is derived from a **patchy** web server, because of all the patches people wrote for the NCSA httpd server.

Later chapters will expand this web server into a LAMP stack (Linux, Apache, Mysql, Perl/PHP/Python).

66.1. introduction to apache

66.1.1. installing on Debian

This screenshot shows that there is no **apache** server installed, nor does the **/var/www** directory exist.

```
root@debian7:~# ls -l /var/www
ls: cannot access /var/www: No such file or directory
root@debian7:~# dpkg -l | grep apache
```

To install **apache** on Debian:

```
root@debian7:~# aptitude install apache2
The following NEW packages will be installed:
  apache2 apache2-mpm-worker{a} apache2-utils{a} apache2.2-bin{a} apache2.2-com\
mon{a} libapr1{a} libaprutil1{a} libaprutil1-dbd-sqlite3{a} libaprutil1-ldap{a}\
ssl-cert{a}
0 packages upgraded, 10 newly installed, 0 to remove and 0 not upgraded.
Need to get 1,487 kB of archives. After unpacking 5,673 kB will be used.
Do you want to continue? [Y/n/?]
```

After installation, the same two commands as above will yield a different result:

```
root@debian7:~# ls -l /var/www
total 4
-rw-r--r-- 1 root root 177 Apr 29 11:55 index.html
root@debian7:~# dpkg -l | grep apache | tr -s ' '
ii apache2 2.2.22-13+deb7u1 amd64 Apache HTTP Server metapackage
ii apache2-mpm-worker 2.2.22-13+deb7u1 amd64 Apache HTTP Server - high speed th\
readed model
ii apache2-utils 2.2.22-13+deb7u1 amd64 utility programs for webrowsers
ii apache2.2-bin 2.2.22-13+deb7u1 amd64 Apache HTTP Server common binary files
ii apache2.2-common 2.2.22-13+deb7u1 amd64 Apache HTTP Server common files
```

66.1.2. installing on RHEL/CentOS

Note that Red Hat derived distributions use **httpd** as package and process name instead of **apache**.

To verify whether **apache** is installed in CentOS/RHEL:

```
[root@centos65 ~]# rpm -q httpd
package httpd is not installed
[root@centos65 ~]# ls -l /var/www
ls: cannot access /var/www: No such file or directory
```

To install apache on CentOS:

```
[root@centos65 ~]# yum install httpd
```

After running the **yum install httpd** command, the Centos 6.5 server has apache installed and the **/var/www** directory exists.

```
[root@centos65 ~]# rpm -q httpd
httpd-2.2.15-30.el6.centos.x86_64
[root@centos65 ~]# ls -l /var/www
total 16
drwxr-xr-x. 2 root root 4096 Apr  3 23:57 cgi-bin
drwxr-xr-x. 3 root root 4096 May  6 13:08 error
drwxr-xr-x. 2 root root 4096 Apr  3 23:57 html
drwxr-xr-x. 3 root root 4096 May  6 13:08 icons
[root@centos65 ~]#
```

66.1.3. running apache on Debian

This is how you start **apache2** on Debian.

```
root@debian7:~# service apache2 status
Apache2 is NOT running.
root@debian7:~# service apache2 start
Starting web server: apache2apache2: Could not reliably determine the server's \
fully qualified domain name, using 127.0.1.1 for ServerName
.
```

To verify, run the **service apache2 status** command again or use **ps**.

```
root@debian7:~# service apache2 status
Apache2 is running (pid 3680).
root@debian7:~# ps -C apache2
  PID TTY          TIME CMD
 3680 ?            00:00:00 apache2
 3683 ?            00:00:00 apache2
 3684 ?            00:00:00 apache2
 3685 ?            00:00:00 apache2
root@debian7:~#
```

Or use **wget** and **file** to verify that your web server serves an html document.

```
root@debian7:~# wget 127.0.0.1
--2014-05-06 13:27:02-- http://127.0.0.1/
Connecting to 127.0.0.1:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 177 [text/html]
Saving to: `index.html'

100%[=====>] 177  --.-K/s  in 0s

2014-05-06 13:27:02 (15.8 MB/s) - `index.html' saved [177/177]

root@debian7:~# file index.html
index.html: HTML document, ASCII text
root@debian7:~#
```

Or verify that apache is running by opening a web browser, and browse to the ip-address of your server. An Apache test page should be shown.

You can do the following to quickly avoid the 'could not reliably determine the fqdn' message when restarting apache.

```
root@debian7:~# echo ServerName Debian7 >> /etc/apache2/apache2.conf
root@debian7:~# service apache2 restart
Restarting web server: apache2 ... waiting .
root@debian7:~#
```

66.1.4. running apache on CentOS

Starting the **httpd** on RHEL/CentOS is done with the **service** command.

```
[root@centos65 ~]# service httpd status
httpd is stopped
[root@centos65 ~]# service httpd start
Starting httpd: httpd: Could not reliably determine the server's fully qualifie\
d domain name, using 127.0.0.1 for ServerName
[ OK ]
[root@centos65 ~]#
```

To verify that **apache** is running, use **ps** or issue the **service httpd status** command again.

```
[root@centos65 ~]# service httpd status
httpd (pid 2410) is running...
[root@centos65 ~]# ps -C httpd
  PID TTY          TIME CMD
 2410 ?            00:00:00 httpd
 2412 ?            00:00:00 httpd
 2413 ?            00:00:00 httpd
 2414 ?            00:00:00 httpd
 2415 ?            00:00:00 httpd
 2416 ?            00:00:00 httpd
 2417 ?            00:00:00 httpd
 2418 ?            00:00:00 httpd
 2419 ?            00:00:00 httpd
[root@centos65 ~]#
```

To prevent the 'Could not reliably determine the fqdn' message, issue the following command.

```
[root@centos65 ~]# echo ServerName Centos65 >> /etc/httpd/conf/httpd.conf
[root@centos65 ~]# service httpd restart
Stopping httpd:          [ OK ]
Starting httpd:          [ OK ]
[root@centos65 ~]#
```

66.1.5. index file on CentOS

CentOS does not provide a standard `index.html` or `index.php` file. A simple `wget` gives an error.

```
[root@centos65 ~]# wget 127.0.0.1
--2014-05-06 15:10:22-- http://127.0.0.1/
Connecting to 127.0.0.1:80... connected.
HTTP request sent, awaiting response... 403 Forbidden
2014-05-06 15:10:22 ERROR 403: Forbidden.
```

Instead when visiting the ip-address of your server in a web browser you get a **`noindex.html`** page. You can verify this using `wget`.

```
[root@centos65 ~]# wget http://127.0.0.1/error/noindex.html
--2014-05-06 15:16:05-- http://127.0.0.1/error/noindex.html
Connecting to 127.0.0.1:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5039 (4.9K) [text/html]
Saving to: "noindex.html"

100%[=====] 5,039      --.-K/s   in 0s

2014-05-06 15:16:05 (289 MB/s) - "noindex.html" saved [5039/5039]

[root@centos65 ~]# file noindex.html
noindex.html: HTML document text
[root@centos65 ~]#
```

Any custom **`index.html`** file in **`/var/www/html`** will immediately serve as an index for this web server.

```
[root@centos65 ~]# echo 'Welcome to my website' > /var/www/html/index.html
[root@centos65 ~]# wget http://127.0.0.1
--2014-05-06 15:19:16-- http://127.0.0.1/
Connecting to 127.0.0.1:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 22 [text/html]
Saving to: "index.html"

100%[=====] 22          --.-K/s   in 0s

2014-05-06 15:19:16 (1.95 MB/s) - "index.html" saved [22/22]

[root@centos65 ~]# cat index.html
Welcome to my website
```

66.1.6. default website

Changing the default website of a freshly installed apache web server is easy. All you need to do is create (or change) an `index.html` file in the DocumentRoot directory.

To locate the DocumentRoot directory on Debian:

```
root@debian7:~# grep DocumentRoot /etc/apache2/sites-available/default
DocumentRoot /var/www
```

This means that `/var/www/index.html` is the default web site.

```
root@debian7:~# cat /var/www/index.html
<html><body><h1>It works!</h1>
<p>This is the default web page for this server.</p>
<p>The web server software is running but no content has been added, yet.</p>
</body></html>
root@debian7:~#
```

This screenshot shows how to locate the **DocumentRoot** directory on RHEL/CentOS.

```
[root@centos65 ~]# grep ^DocumentRoot /etc/httpd/conf/httpd.conf
DocumentRoot "/var/www/html"
```

RHEL/CentOS have no default web page (only the `noindex.html` error page mentioned before). But an **`index.html`** file created in `/var/www/html/` will automatically be used as default page.

```
[root@centos65 ~]# echo '<html><head><title>Default website</title></head><body>\
><p>A new web page</p></body></html>' > /var/www/html/index.html
[root@centos65 ~]# cat /var/www/html/index.html
<html><head><title>Default website</title></head><body><p>A new web page</p></b\
ody></html>
[root@centos65 ~]#
```

66.1.7. apache configuration

There are many similarities, but also a couple of differences when configuring **apache** on Debian or on CentOS. Both Linux families will get their own chapters with examples.

All configuration on RHEL/CentOS is done in **/etc/httpd**.

```
[root@centos65 ~]# ls -l /etc/httpd/
total 8
drwxr-xr-x. 2 root root 4096 May  6 13:08 conf
drwxr-xr-x. 2 root root 4096 May  6 13:08 conf.d
lrwxrwxrwx. 1 root root   19 May  6 13:08 logs -> ../../var/log/httpd
lrwxrwxrwx. 1 root root   29 May  6 13:08 modules -> ../../usr/lib64/httpd/modu\
les
lrwxrwxrwx. 1 root root   19 May  6 13:08 run -> ../../var/run/httpd
[root@centos65 ~]#
```

Debian (and ubuntu/mint/...) use **/etc/apache2**.

```
root@debian7:~# ls -l /etc/apache2/
total 72
-rw-r--r-- 1 root root 9659 May  6 14:23 apache2.conf
drwxr-xr-x 2 root root 4096 May  6 13:19 conf.d
-rw-r--r-- 1 root root 1465 Jan 31 18:35 envvars
-rw-r--r-- 1 root root 31063 Jul 20 2013 magic
drwxr-xr-x 2 root root 4096 May  6 13:19 mods-available
drwxr-xr-x 2 root root 4096 May  6 13:19 mods-enabled
-rw-r--r-- 1 root root 750 Jan 26 12:13 ports.conf
drwxr-xr-x 2 root root 4096 May  6 13:19 sites-available
drwxr-xr-x 2 root root 4096 May  6 13:19 sites-enabled
root@debian7:~#
```


66.2. port virtual hosts on Debian

66.2.1. default virtual host

Debian has a virtualhost configuration file for its default website in `/etc/apache2/sites-available/default`.

```
root@debian7:~# head -2 /etc/apache2/sites-available/default
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
```

66.2.2. three extra virtual hosts

In this scenario we create three additional websites for three customers that share a clubhouse and want to jointly hire you. They are a model train club named **Choo Choo**, a chess club named **Chess Club 42** and a hackerspace named **hunter2**.

One way to put three websites on one web server, is to put each website on a different port. This screenshot shows three newly created **virtual hosts**, one for each customer.

```
root@debian7:~# vi /etc/apache2/sites-available/choochoo
root@debian7:~# cat /etc/apache2/sites-available/choochoo
<VirtualHost *:7000>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/choochoo
</VirtualHost>
root@debian7:~# vi /etc/apache2/sites-available/chessclub42
root@debian7:~# cat /etc/apache2/sites-available/chessclub42
<VirtualHost *:8000>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/chessclub42
</VirtualHost>
root@debian7:~# vi /etc/apache2/sites-available/hunter2
root@debian7:~# cat /etc/apache2/sites-available/hunter2
<VirtualHost *:9000>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/hunter2
</VirtualHost>
```

Notice the different port numbers 7000, 8000 and 9000. Notice also that we specified a unique **DocumentRoot** for each website.

Are you using **Ubuntu** or **Mint**, then these configfiles need to end in **.conf**.

66.2.3. three extra ports

We need to enable these three ports on apache in the **ports.conf** file. Open this file with **vi** and add three lines to **listen** on three extra ports.

```
root@debian7:~# vi /etc/apache2/ports.conf
```

Verify with **grep** that the **Listen** directives are added correctly.

```
root@debian7:~# grep ^Listen /etc/apache2/ports.conf
Listen 80
Listen 7000
Listen 8000
Listen 9000
```

66.2.4. three extra websites

Next we need to create three **DocumentRoot** directories.

```
root@debian7:~# mkdir /var/www/choochoo
root@debian7:~# mkdir /var/www/chessclub42
root@debian7:~# mkdir /var/www/hunter2
```

And we have to put some really simple website in those directories.

```
root@debian7:~# echo 'Choo Choo model train Choo Choo' > /var/www/choochoo/index.html
root@debian7:~# echo 'Welcome to chess club 42' > /var/www/chessclub42/index.html
root@debian7:~# echo 'HaCkInG iS fUn At HuNtEr2' > /var/www/hunter2/index.html
```

66.2.5. enabling extra websites

The last step is to enable the websites with the **a2ensite** command. This command will create links in **sites-enabled**.

The links are not there yet...

```
root@debian7:~# cd /etc/apache2/
root@debian7:/etc/apache2# ls sites-available/
chessclub42  choochoo  default  default-ssl  hunter2
root@debian7:/etc/apache2# ls sites-enabled/
000-default
```

So we run the **a2ensite** command for all websites.

```
root@debian7:/etc/apache2# a2ensite choochoo
Enabling site choochoo.
To activate the new configuration, you need to run:
    service apache2 reload
root@debian7:/etc/apache2# a2ensite chessclub42
Enabling site chessclub42.
To activate the new configuration, you need to run:
    service apache2 reload
root@debian7:/etc/apache2# a2ensite hunter2
Enabling site hunter2.
To activate the new configuration, you need to run:
    service apache2 reload
```

The links are created, so we can tell **apache**.

```
root@debian7:/etc/apache2# ls sites-enabled/
000-default  chessclub42  choochoo  hunter2
root@debian7:/etc/apache2# service apache2 reload
Reloading web server config: apache2.
root@debian7:/etc/apache2#
```

66.2.6. testing the three websites

Testing the model train club named **Choo Choo** on port 7000.

```
root@debian7:/etc/apache2# wget 127.0.0.1:7000
--2014-05-06 21:16:03-- http://127.0.0.1:7000/
Connecting to 127.0.0.1:7000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 32 [text/html]
Saving to: `index.html'

100%[=====>] 32          --.-K/s   in 0s

2014-05-06 21:16:03 (2.92 MB/s) - `index.html' saved [32/32]

root@debian7:/etc/apache2# cat index.html
Choo Choo model train Choo Choo
```

Testing the chess club named **Chess Club 42** on port 8000.

```
root@debian7:/etc/apache2# wget 127.0.0.1:8000
--2014-05-06 21:16:20-- http://127.0.0.1:8000/
Connecting to 127.0.0.1:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 25 [text/html]
Saving to: `index.html.1'

100%[=====>] 25          --.-K/s   in 0s

2014-05-06 21:16:20 (2.16 MB/s) - `index.html.1' saved [25/25]

root@debian7:/etc/apache2# cat index.html.1
Welcome to chess club 42
```

Testing the hacker club named **hunter2** on port 9000.

```
root@debian7:/etc/apache2# wget 127.0.0.1:9000
--2014-05-06 21:16:30-- http://127.0.0.1:9000/
Connecting to 127.0.0.1:9000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 26 [text/html]
Saving to: `index.html.2'

100%[=====>] 26          --.-K/s   in 0s

2014-05-06 21:16:30 (2.01 MB/s) - `index.html.2' saved [26/26]

root@debian7:/etc/apache2# cat index.html.2
HaCkInG iS fUn At HuNtEr2
```

Cleaning up the temporary files.

```
root@debian7:/etc/apache2# rm index.html index.html.1 index.html.2
```

Try testing from another computer using the ip-address of your server.

66.3. named virtual hosts on Debian

66.3.1. named virtual hosts

The chess club and the model train club find the port numbers too hard to remember. They would prefer to have their website accessible by name.

We continue work on the same server that has three websites on three ports. We need to make sure those websites are accessible using the names **choochoo.local**, **chessclub42.local** and **hunter2.local**.

We start by creating three new virtualhosts.

```
root@debian7:/etc/apache2/sites-available# vi choochoo.local
root@debian7:/etc/apache2/sites-available# vi chessclub42.local
root@debian7:/etc/apache2/sites-available# vi hunter2.local
root@debian7:/etc/apache2/sites-available# cat choochoo.local
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    ServerName choochoo.local
    DocumentRoot /var/www/choochoo
</VirtualHost>
root@debian7:/etc/apache2/sites-available# cat chessclub42.local
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    ServerName chessclub42.local
    DocumentRoot /var/www/chessclub42
</VirtualHost>
root@debian7:/etc/apache2/sites-available# cat hunter2.local
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    ServerName hunter2.local
    DocumentRoot /var/www/hunter2
</VirtualHost>
root@debian7:/etc/apache2/sites-available#
```

Notice that they all listen on **port 80** and have an extra **ServerName** directive.

66.3.2. name resolution

We need some way to resolve names. This can be done with DNS, which is discussed in another chapter. For this demo it is also possible to quickly add the three names to the **/etc/hosts** file.

```
root@debian7:/etc/apache2/sites-available# grep ^192 /etc/hosts
192.168.42.50 choochoo.local
192.168.42.50 chessclub42.local
192.168.42.50 hunter2.local
```

Note that you may have another ip address...

66.3.3. enabling virtual hosts

Next we enable them with **a2ensite**.

```
root@debian7:/etc/apache2/sites-available# a2ensite choochoo.local
Enabling site choochoo.local.
To activate the new configuration, you need to run:
    service apache2 reload
root@debian7:/etc/apache2/sites-available# a2ensite chessclub42.local
Enabling site chessclub42.local.
To activate the new configuration, you need to run:
    service apache2 reload
root@debian7:/etc/apache2/sites-available# a2ensite hunter2.local
Enabling site hunter2.local.
To activate the new configuration, you need to run:
    service apache2 reload
```

66.3.4. reload and verify

After a **service apache2 reload** the websites should be available by name.

```
root@debian7:/etc/apache2/sites-available# service apache2 reload
Reloading web server config: apache2.
root@debian7:/etc/apache2/sites-available# wget chessclub42.local
--2014-05-06 21:37:13--  http://chessclub42.local/
Resolving chessclub42.local (chessclub42.local)... 192.168.42.50
Connecting to chessclub42.local (chessclub42.local)|192.168.42.50|:80... connec\
cted.
HTTP request sent, awaiting response... 200 OK
Length: 25 [text/html]
Saving to: `index.html'

100%[=====>] 25          --.-K/s   in 0s

2014-05-06 21:37:13 (2.06 MB/s) - `index.html' saved [25/25]

root@debian7:/etc/apache2/sites-available# cat index.html
Welcome to chess club 42
```

66.4. password protected website on Debian

You can secure files and directories in your website with a **.htaccess** file that refers to a **.htpasswd** file. The **htpasswd** command can create a **.htpasswd** file that contains a userid and an (encrypted) password.

This screenshot creates a user and password for the hacker named **cliff** and uses the **-c** flag to create the **.htpasswd** file.

```
root@debian7:~# htpasswd -c /var/www/.htpasswd cliff
New password:
Re-type new password:
Adding password for user cliff
root@debian7:~# cat /var/www/.htpasswd
cliff:$apr1$vujll0KL$./SZ4w9q0swhX93pQ0PVp.
```

Hacker **rob** also wants access, this screenshot shows how to add a second user and password to **.htpasswd**.

```
root@debian7:~# htpasswd /var/www/.htpasswd rob
New password:
Re-type new password:
Adding password for user rob
root@debian7:~# cat /var/www/.htpasswd
cliff:$apr1$vujll0KL$./SZ4w9q0swhX93pQ0PVp.
rob:$apr1$HNln1FFt$nRlpF0H.IW11/1DRq4lQo0
```

Both Cliff and Rob chose the same password (hunter2), but that is not visible in the **.htpasswd** file because of the different salts.

Next we need to create a **.htaccess** file in the **DocumentRoot** of the website we want to protect. This screenshot shows an example.

```
root@debian7:~# cd /var/www/hunter2/
root@debian7:/var/www/hunter2# cat .htaccess
AuthUserFile /var/www/.htpasswd
AuthName "Members only!"
AuthType Basic
require valid-user
```

Note that we are protecting the website on **port 9000** that we created earlier.

And because we put the website for the Hackerspace named hunter2 in a subdirectory of the default website, we will need to adjust the **AllowOverride** parameter in **/etc/apache2/sites-available/default** as this screenshot shows (with line numbers on Debian7, your may vary).

```
9      <Directory /var/www/>
10          Options Indexes FollowSymLinks MultiViews
11          AllowOverride Authconfig
12          Order allow,deny
13          allow from all
14      </Directory>
```

Now restart the apache2 server and test that it works!

66.5. port virtual hosts on CentOS

66.5.1. default virtual host

Unlike Debian, CentOS has no `virtualHost` configuration file for its default website. Instead the default configuration will throw a standard error page when no index file can be found in the default location (`/var/www/html`).

66.5.2. three extra virtual hosts

In this scenario we create three additional websites for three customers that share a clubhouse and want to jointly hire you. They are a model train club named **Choo Choo**, a chess club named **Chess Club 42** and a hackerspace named **hunter2**.

One way to put three websites on one web server, is to put each website on a different port. This screenshot shows three newly created **virtual hosts**, one for each customer.

```
[root@CentOS65 ~]# vi /etc/httpd/conf.d/choochoo.conf
[root@CentOS65 ~]# cat /etc/httpd/conf.d/choochoo.conf
<VirtualHost *:7000>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/choochoo
</VirtualHost>
[root@CentOS65 ~]# vi /etc/httpd/conf.d/chessclub42.conf
[root@CentOS65 ~]# cat /etc/httpd/conf.d/chessclub42.conf
<VirtualHost *:8000>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/chessclub42
</VirtualHost>
[root@CentOS65 ~]# vi /etc/httpd/conf.d/hunter2.conf
[root@CentOS65 ~]# cat /etc/httpd/conf.d/hunter2.conf
<VirtualHost *:9000>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/hunter2
</VirtualHost>
```

Notice the different port numbers 7000, 8000 and 9000. Notice also that we specified a unique **DocumentRoot** for each website.

66.5.3. three extra ports

We need to enable these three ports on apache in the **httpd.conf** file.

```
[root@CentOS65 ~]# vi /etc/httpd/conf/httpd.conf
root@debian7:~# grep ^Listen /etc/httpd/conf/httpd.conf
Listen 80
Listen 7000
Listen 8000
Listen 9000
```


66.5.4. SELinux guards our ports

If we try to restart our server, we will notice the following error:

```
[root@CentOS65 ~]# service httpd restart
Stopping httpd:                                [ OK ]
Starting httpd:
    (13)Permission denied: make_sock: could not bind to address 0.0.0.0:7000
no listening sockets available, shutting down
                                           [ FAILED ]
```

This is due to SELinux reserving ports 7000 and 8000 for other uses. We need to tell SELinux we want to use these ports for http traffic

```
[root@CentOS65 ~]# semanage port -m -t http_port_t -p tcp 7000
[root@CentOS65 ~]# semanage port -m -t http_port_t -p tcp 8000
[root@CentOS65 ~]# service httpd restart
Stopping httpd:                                [ OK ]
Starting httpd:                                [ OK ]
```

66.5.5. three extra websites

Next we need to create three **DocumentRoot** directories.

```
[root@CentOS65 ~]# mkdir /var/www/html/choochoo
[root@CentOS65 ~]# mkdir /var/www/html/chessclub42
[root@CentOS65 ~]# mkdir /var/www/html/hunter2
```

And we have to put some really simple website in those directories.

```
[root@CentOS65 ~]# echo 'Choo Choo model train Choo Choo' > /var/www/html/choochoo/index.html
[root@CentOS65 ~]# echo 'Welcome to chess club 42' > /var/www/html/chessclub42/index.html
[root@CentOS65 ~]# echo 'HaCkInG iS fUn At HuNtEr2' > /var/www/html/hunter2/index.html
```

66.5.6. enabling extra websites

The only way to enable or disable configurations in RHEL/CentOS is by renaming or moving the configuration files. Any file in `/etc/httpd/conf.d` ending on `.conf` will be loaded by Apache. To disable a site we can either rename the file or move it to another directory.

The files are created, so we can tell **apache**.

```
[root@CentOS65 ~]# ls /etc/httpd/conf.d/
chessclub42.conf  choochoo.conf  hunter2.conf  README  welcome.conf
[root@CentOS65 ~]# service httpd reload
Reloading httpd:
```

66.5.7. testing the three websites

Testing the model train club named **Choo Choo** on port 7000.

```
[root@CentOS65 ~]# wget 127.0.0.1:7000
--2014-05-11 11:59:36-- http://127.0.0.1:7000/
Connecting to 127.0.0.1:7000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 32 [text/html]
Saving to: `index.html'

100%[=====>] 32          --.-K/s   in 0s

2014-05-11 11:59:36 (4.47 MB/s) - `index.html' saved [32/32]

[root@CentOS65 ~]# cat index.html
Choo Choo model train Choo Choo
```

Testing the chess club named **Chess Club 42** on port 8000.

```
[root@CentOS65 ~]# wget 127.0.0.1:8000
--2014-05-11 12:01:30-- http://127.0.0.1:8000/
Connecting to 127.0.0.1:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 25 [text/html]
Saving to: `index.html.1'

100%[=====>] 25          --.-K/s   in 0s

2014-05-11 12:01:30 (4.25 MB/s) - `index.html.1' saved [25/25]

root@debian7:/etc/apache2# cat index.html.1
Welcome to chess club 42
```

Testing the hacker club named **hunter2** on port 9000.

```
[root@CentOS65 ~]# wget 127.0.0.1:9000
--2014-05-11 12:02:37-- http://127.0.0.1:9000/
Connecting to 127.0.0.1:9000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 26 [text/html]
Saving to: `index.html.2'

100%[=====>] 26          --.-K/s   in 0s

2014-05-11 12:02:37 (4.49 MB/s) - `index.html.2' saved [26/26]

root@debian7:/etc/apache2# cat index.html.2
HaCkInG iS fUn At HuNtEr2
```

Cleaning up the temporary files.

```
[root@CentOS65 ~]# rm index.html index.html.1 index.html.2
```

66.5.8. firewall rules

If we attempt to access the site from another machine however, we will not be able to view the website yet. The firewall is blocking incoming connections. We need to open these incoming ports first

```
[root@CentOS65 ~]# iptables -I INPUT -p tcp --dport 80 -j ACCEPT
[root@CentOS65 ~]# iptables -I INPUT -p tcp --dport 7000 -j ACCEPT
[root@CentOS65 ~]# iptables -I INPUT -p tcp --dport 8000 -j ACCEPT
[root@CentOS65 ~]# iptables -I INPUT -p tcp --dport 9000 -j ACCEPT
```

And if we want these rules to remain active after a reboot, we need to save them

```
[root@CentOS65 ~]# service iptables save
iptables: Saving firewall rules to /etc/sysconfig/iptables:[ OK ]
```

66.6. named virtual hosts on CentOS

66.6.1. named virtual hosts

The chess club and the model train club find the port numbers too hard to remember. They would prefer to have their website accessible by name.

We continue work on the same server that has three websites on three ports. We need to make sure those websites are accessible using the names **choochoo.local**, **chessclub42.local** and **hunter2.local**.

First, we need to enable named virtual hosts in the configuration

```
[root@CentOS65 ~]# vi /etc/httpd/conf/httpd.conf
[root@CentOS65 ~]# grep ^NameVirtualHost /etc/httpd/conf/httpd.conf
NameVirtualHost *:80
[root@CentOS65 ~]#
```

Next we need to create three new virtualhosts.

```
[root@CentOS65 ~]# vi /etc/httpd/conf.d/choochoo.local.conf
[root@CentOS65 ~]# vi /etc/httpd/conf.d/chessclub42.local.conf
[root@CentOS65 ~]# vi /etc/httpd/conf.d/hunter2.local.conf
[root@CentOS65 ~]# cat /etc/httpd/conf.d/choochoo.local.conf
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    ServerName choochoo.local
    DocumentRoot /var/www/html/choochoo
</VirtualHost>
[root@CentOS65 ~]# cat /etc/httpd/conf.d/chessclub42.local.conf
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    ServerName chessclub42.local
    DocumentRoot /var/www/html/chessclub42
</VirtualHost>
[root@CentOS65 ~]# cat /etc/httpd/conf.d/hunter2.local.conf
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    ServerName hunter2.local
    DocumentRoot /var/www/html/hunter2
</VirtualHost>
[root@CentOS65 ~]#
```

Notice that they all listen on **port 80** and have an extra **ServerName** directive.

66.6.2. name resolution

We need some way to resolve names. This can be done with DNS, which is discussed in another chapter. For this demo it is also possible to quickly add the three names to the **/etc/hosts** file.

```
[root@CentOS65 ~]# grep ^192 /etc/hosts
192.168.1.225 choochoo.local
192.168.1.225 chessclub42.local
192.168.1.225 hunter2.local
```

Note that you may have another ip address...

66.6.3. reload and verify

After a **service httpd reload** the websites should be available by name.

```
[root@CentOS65 ~]# service httpd reload
Reloading httpd:
[root@CentOS65 ~]# wget chessclub42.local
--2014-05-25 16:59:14-- http://chessclub42.local/
Resolving chessclub42.local... 192.168.1.225
Connecting to chessclub42.local|192.168.1.225|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 25 [text/html]
Saving to: âindex.htmlâ

100%[=====>] 25          --.-K/s   in 0s

2014-05-25 16:59:15 (1014 KB/s) - `index.html' saved [25/25]

[root@CentOS65 ~]# cat index.html
Welcome to chess club 42
```

66.7. password protected website on CentOS

You can secure files and directories in your website with a **.htaccess** file that refers to a **.htpasswd** file. The **htpasswd** command can create a **.htpasswd** file that contains a userid and an (encrypted) password.

This screenshot creates a user and password for the hacker named **cliff** and uses the **-c** flag to create the **.htpasswd** file.

```
[root@CentOS65 ~]# htpasswd -c /var/www/.htpasswd cliff
New password:
Re-type new password:
Adding password for user cliff
[root@CentOS65 ~]# cat /var/www/.htpasswd
cliff:QNwTryMLBctU
```

Hacker **rob** also wants access, this screenshot shows how to add a second user and password to **.htpasswd**.

```
[root@CentOS65 ~]# htpasswd /var/www/.htpasswd rob
New password:
Re-type new password:
Adding password for user rob
[root@CentOS65 ~]# cat /var/www/.htpasswd
cliff:QNwTryMLBctU
rob:EC2v0CcrMXDoM
[root@CentOS65 ~]#
```

Both Cliff and Rob chose the same password (hunter2), but that is not visible in the **.htpasswd** file because of the different salts.

Next we need to create a **.htaccess** file in the **DocumentRoot** of the website we want to protect. This screenshot shows an example.

```
[root@CentOS65 ~]# cat /var/www/html/hunter2/.htaccess
AuthUserFile /var/www/.htpasswd
AuthName "Members only!"
AuthType Basic
require valid-user
```

Note that we are protecting the website on **port 9000** that we created earlier.

And because we put the website for the Hackerspace named hunter2 in a subdirectory of the default website, we will need to adjust the **AllowOverride** parameter in **/etc/httpd/conf/httpd.conf** under the **<Directory "/var/www/html">** directive as this screenshot shows.

```
[root@CentOS65 ~]# vi /etc/httpd/conf/httpd.conf

<Directory "/var/www/html">

#
# Possible values for the Options directive are "None", "All",
# or any combination of:
#   Indexes Includes FollowSymLinks SymLinksifOwnerMatch ExecCGI MultiViews
#
# Note that "MultiViews" must be named *explicitly* --- "Options All"
# doesn't give it to you.
#
# The Options directive is both complicated and important. Please see
# http://httpd.apache.org/docs/2.2/mod/core.html#options
# for more information.
#
    Options Indexes FollowSymLinks

#
# AllowOverride controls what directives may be placed in .htaccess files.
# It can be "All", "None", or any combination of the keywords:
#   Options FileInfo AuthConfig Limit
#
    AllowOverride Authconfig

#
# Controls who can get stuff from this server.
#
    Order allow,deny
    Allow from all

</Directory>
```

Now restart the apache2 server and test that it works!

66.8. troubleshooting apache

When apache restarts, it will verify the syntax of files in the configuration folder **/etc/apache2** on debian or **/etc/httpd** on CentOS and it will tell you the name of the faulty file, the line number and an explanation of the error.

```
root@debian7:~# service apache2 restart
apache2: Syntax error on line 268 of /etc/apache2/apache2.conf: Syntax error o\
n line 1 of /etc/apache2/sites-enabled/chessclub42: /etc/apache2/sites-enabled\
/chessclub42:4: <VirtualHost> was not closed.\n/etc/apache2/sites-enabled/ches\
club42:1: <VirtualHost> was not closed.
Action 'configtest' failed.
The Apache error log may have more information.
failed!
```

Below you see the problem... a missing / before on line 4.

```
root@debian7:~# cat /etc/apache2/sites-available/chessclub42
<VirtualHost *:8000>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/chessclub42
<VirtualHost>
```

Let us force another error by renaming the directory of one of our websites:

```
root@debian7:~# mv /var/www/choochoo/ /var/www/chooshoo
root@debian7:~# !ser
service apache2 restart
Restarting web server: apache2Warning: DocumentRoot [/var/www/choochoo] does n\
ot exist
Warning: DocumentRoot [/var/www/choochoo] does not exist
... waiting Warning: DocumentRoot [/var/www/choochoo] does not exist
Warning: DocumentRoot [/var/www/choochoo] does not exist
.
```

As you can see, apache will tell you exactly what is wrong.

You can also troubleshoot by connecting to the website via a browser and then checking the apache log files in **/var/log/apache**.

66.9. virtual hosts example

Below is a sample virtual host configuration. This virtual hosts overrules the default Apache **ErrorDocument** directive.

```
<VirtualHost 83.217.76.245:80>
ServerName cobbaut.be
ServerAlias www.cobbaut.be
DocumentRoot /home/paul/public_html
ErrorLog /home/paul/logs/error_log
CustomLog /home/paul/logs/access_log common
ScriptAlias /cgi-bin/ /home/paul/cgi-bin/
<Directory /home/paul/public_html>
    Options Indexes IncludesNOEXEC FollowSymLinks
    allow from all
</Directory>
ErrorDocument 404 http://www.cobbaut.be/cobbaut.php
</VirtualHost>
```

66.10. aliases and redirects

Apache supports aliases for directories, like this example shows.

```
Alias /paul/ "/home/paul/public_html/"
```

Similarly, content can be redirected to another website or web server.

```
Redirect permanent /foo http://www.foo.com/bar
```

66.11. more on .htaccess

You can do much more with **.htaccess**. One example is to use .htaccess to prevent people from certain domains to access your website. Like in this case, where a number of referer spammers are blocked from the website.

```
paul@lounge:~/cobbaut.be$ cat .htaccess
# Options +FollowSymlinks
RewriteEngine On
RewriteCond %{HTTP_REFERER} ^http://(www\.)?buy-adipex.fw.nu.*$ [OR]
RewriteCond %{HTTP_REFERER} ^http://(www\.)?buy-levitra.asso.ws.*$ [NC,OR]
RewriteCond %{HTTP_REFERER} ^http://(www\.)?buy-tramadol.fw.nu.*$ [NC,OR]
RewriteCond %{HTTP_REFERER} ^http://(www\.)?buy-viagra.lookin.at.*$ [NC,OR]
...
RewriteCond %{HTTP_REFERER} ^http://(www\.)?www.healthinsurancehelp.net.*$ [NC]
RewriteRule .* - [F,L]
paul@lounge:~/cobbaut.be$
```

66.12. traffic

Apache keeps a log of all visitors. The **webalizer** is often used to parse this log into nice html statistics.

66.13. self signed cert on Debian

Below is a very quick guide on setting up Apache2 on Debian 7 with a self-signed certificate.

Chances are these packages are already installed.

```
root@debian7:~# aptitude install apache2 openssl
No packages will be installed, upgraded, or removed.
0 packages upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Need to get 0 B of archives. After unpacking 0 B will be used.
```

Create a directory to store the certs, and use **openssl** to create a self signed cert that is valid for 999 days.

```
root@debian7:~# mkdir /etc/ssl/localcerts
root@debian7:~# openssl req -new -x509 -days 999 -nodes -out /etc/ssl/local\
certs/apache.pem -keyout /etc/ssl/localcerts/apache.key
Generating a 2048 bit RSA private key
...
...
writing new private key to '/etc/ssl/localcerts/apache.key'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:BE
State or Province Name (full name) [Some-State]:Antwerp
Locality Name (eg, city) []:Antwerp
Organization Name (eg, company) [Internet Widgits Pty Ltd]:linux-training.be
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:Paul
Email Address []:
```

A little security never hurt anyone.

```
root@debian7:~# ls -l /etc/ssl/localcerts/
total 8
-rw-r--r-- 1 root root 1704 Sep 16 18:24 apache.key
-rw-r--r-- 1 root root 1302 Sep 16 18:24 apache.pem
root@debian7:~# chmod 600 /etc/ssl/localcerts/*
root@debian7:~# ls -l /etc/ssl/localcerts/
total 8
-rw----- 1 root root 1704 Sep 16 18:24 apache.key
-rw----- 1 root root 1302 Sep 16 18:24 apache.pem
```

Enable the **apache ssl mod**.

```
root@debian7:~# a2enmod ssl
Enabling module ssl.
See /usr/share/doc/apache2.2-common/README.Debian.gz on how to configure SSL\
and create self-signed certificates.
To activate the new configuration, you need to run:
    service apache2 restart
```

Create the website configuration.

```
root@debian7:~# vi /etc/apache2/sites-available/choochoos
```

```

root@debian7:~# cat /etc/apache2/sites-available/choochoos
<VirtualHost *:7000>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/choochoos
    SSLEngine On
    SSLCertificateFile /etc/ssl/localcerts/apache.pem
    SSLCertificateKeyFile /etc/ssl/localcerts/apache.key
</VirtualHost>
root@debian7:~#

```

And create the website itself.

```

root@debian7:/var/www/choochoos# vi index.html
root@debian7:/var/www/choochoos# cat index.html
Choo Choo HTTPS secured model train Choo Choo

```

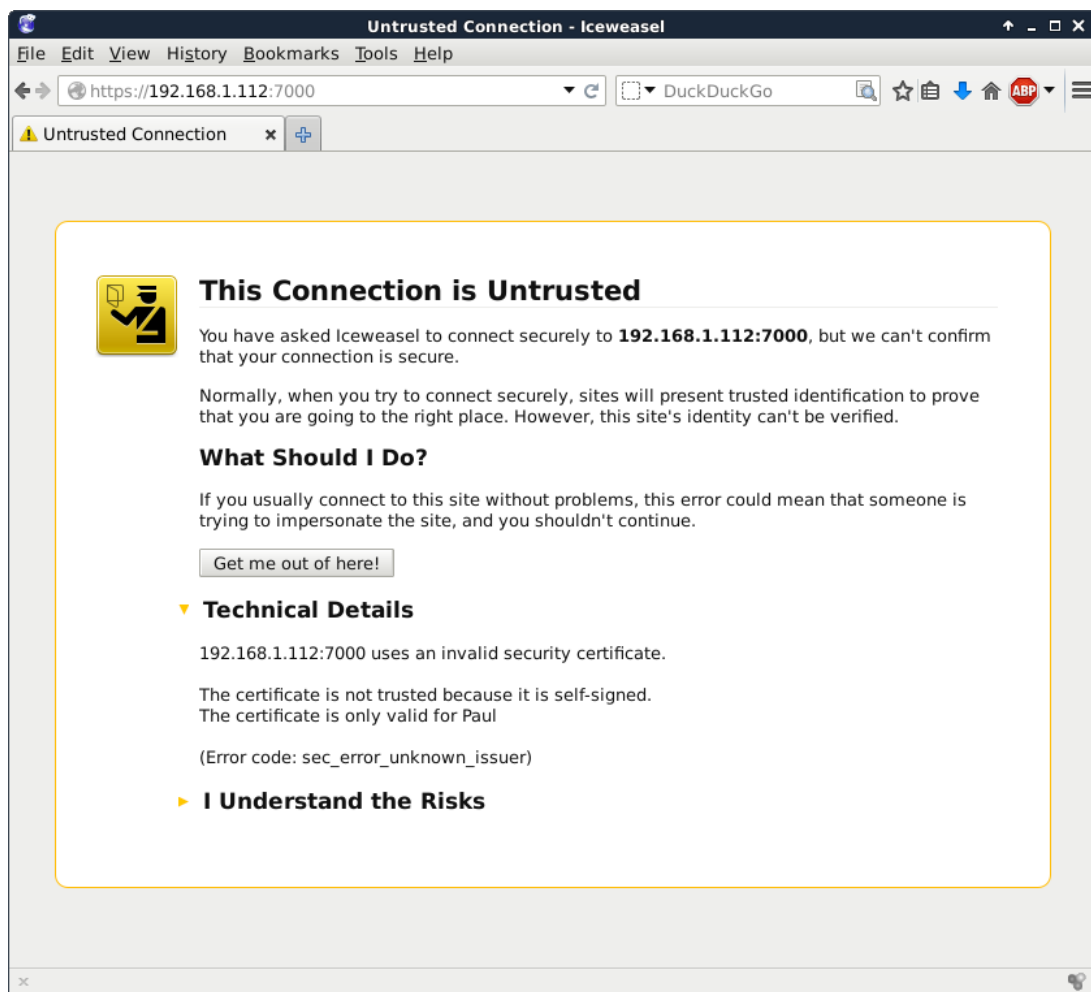
Enable the website and restart (or reload) apache2.

```

root@debian7:/var/www/choochoos# a2ensite choochoos
Enabling site choochoos.
To activate the new configuration, you need to run:
    service apache2 reload
root@debian7:/var/www/choochoos# service apache2 restart
Restarting web server: apache2 ... waiting .

```

Chances are your browser will warn you about the self signed certificate.



66.14. self signed cert on RHEL/CentOS

Below is a quick way to create a self signed cert for https on RHEL/CentOS. You may need these packages:

```
[root@paulserver ~]# yum install httpd openssl mod_ssl
Loaded plugins: fastestmirror
Loading mirror speeds from cached hostfile
* base: ftp.belnet.be
* extras: ftp.belnet.be
* updates: mirrors.vooservers.com
base | 3.7 kB 00:00
Setting up Install Process
Package httpd-2.2.15-31.el6.centos.x86_64 already installed and latest version
Package openssl-1.0.1e-16.el6_5.15.x86_64 already installed and latest version
Package 1:mod_ssl-2.2.15-31.el6.centos.x86_64 already ins... and latest version
Nothing to do
```

We use **openssl** to create the certificate.

```
[root@paulserver ~]# mkdir certs
[root@paulserver ~]# cd certs
[root@paulserver certs]# openssl genrsa -out ca.key 2048
Generating RSA private key, 2048 bit long modulus
.....+++
.....+++
e is 65537 (0x10001)
[root@paulserver certs]# openssl req -new -key ca.key -out ca.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [XX]:BE
State or Province Name (full name) []:antwerp
Locality Name (eg, city) [Default City]:antwerp
Organization Name (eg, company) [Default Company Ltd]:antwerp
Organizational Unit Name (eg, section) []:
Common Name (eg, your name or your server's hostname) []:paulserver
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
[root@paulserver certs]# openssl x509 -req -days 365 -in ca.csr -signkey ca.ke\
y -out ca.crt
Signature ok
subject=/C=BE/ST=antwerp/L=antwerp/O=antwerp/CN=paulserver
Getting Private key
```

We copy the keys to the right location (You may be missing SELinux info here).

```
[root@paulserver certs]# cp ca.crt /etc/pki/tls/certs/
[root@paulserver certs]# cp ca.key ca.csr /etc/pki/tls/private/
```

We add the location of our keys to this file, and also add the **NameVirtualHost *:443** directive.

```
[root@paulserver certs]# vi /etc/httpd/conf.d/ssl.conf
```

```
[root@paulserver certs]# grep ^SSLCerti /etc/httpd/conf.d/ssl.conf
SSLCertificateFile /etc/pki/tls/certs/ca.crt
SSLCertificateKeyFile /etc/pki/tls/private/ca.key
```

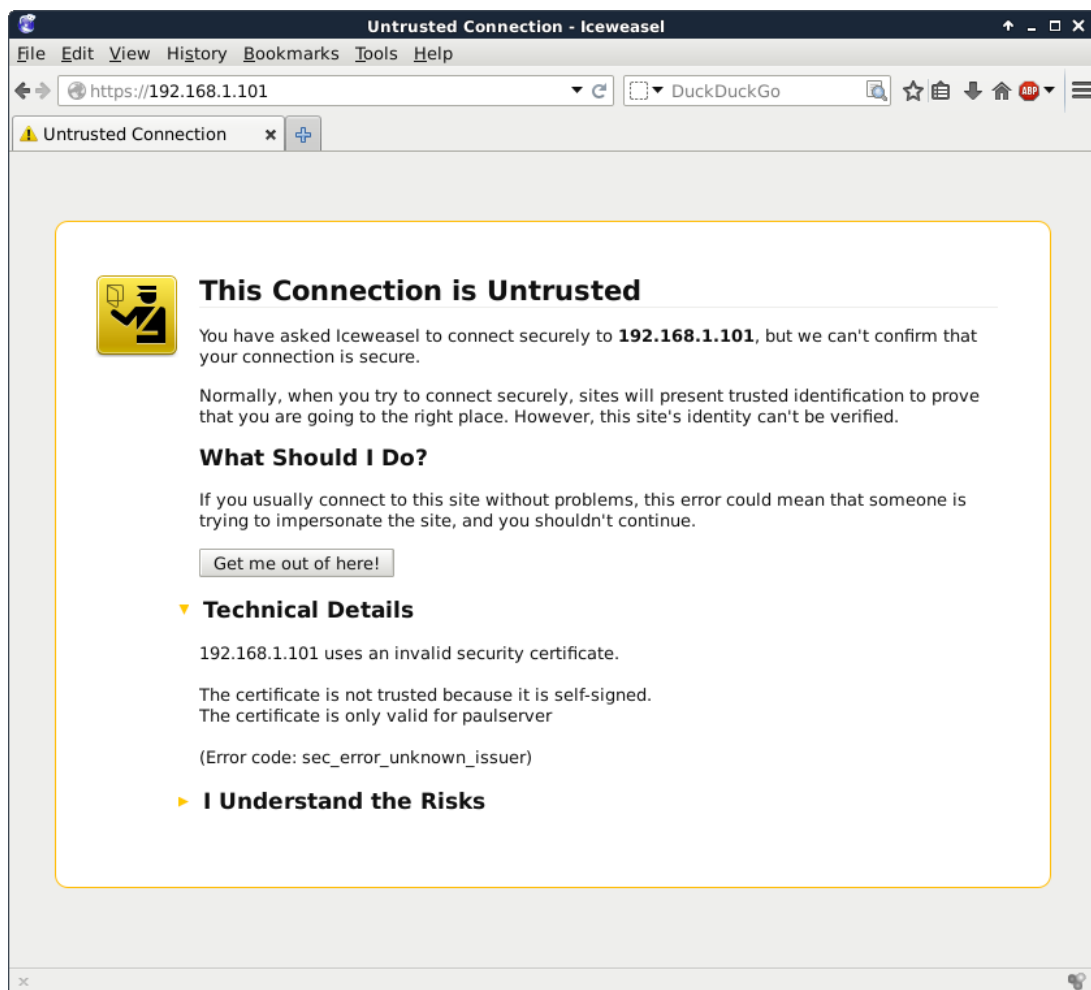
Create a website configuration.

```
[root@paulserver certs]# vi /etc/httpd/conf.d/choochoos.conf
[root@paulserver certs]# cat /etc/httpd/conf.d/choochoos.conf
<VirtualHost *:443>
    SSLEngine on
    SSLCertificateFile /etc/pki/tls/certs/ca.crt
    SSLCertificateKeyFile /etc/pki/tls/private/ca.key
    DocumentRoot /var/www/choochoos
    ServerName paulserver
</VirtualHost>
[root@paulserver certs]#
```

Create a simple website and restart apache.

```
[root@paulserver certs]# mkdir /var/www/choochoos
[root@paulserver certs]# echo HTTPS model train choochoos > /var/www/choochoos/\
index.html
[root@paulserver httpd]# service httpd restart
Stopping httpd:          [ OK ]
Starting httpd:          [ OK ]
```

And your browser will probably warn you that this certificate is self signed.



66.15. practice: apache

1. Verify that Apache is installed and running.
2. Browse to the Apache HTML manual.
3. Create three virtual hosts that listen on ports 8472, 31337 and 1201. Test that it all works.
4. Create three named virtual hosts startrek.local, starwars.local and stargate.local. Test that it all works.
5. Create a virtual hosts that listens on another ip-address.
6. Protect one of your websites with a user/password combo.

Chapter 67. introduction to squid

67.1. about proxy servers

67.1.1. usage

A **proxy server** is a server that caches the internet. Clients connect to the proxy server with a request for an internet server. The proxy server will connect to the internet server on behalf of the client. The proxy server will also cache the pages retrieved from the internet server. A proxy server may provide pages from his cache to a client, instead of connecting to the internet server to retrieve the (same) pages.

A proxy server has two main advantages. It improves web surfing speed when returning cached data to clients, and it reduces the required bandwidth (cost) to the internet.

Smaller organizations sometimes put the proxy server on the same physical computer that serves as a NAT to the internet. In larger organizations, the proxy server is one of many servers in the DMZ.

When web traffic passes via a proxy server, it is common practice to configure the proxy with extra settings for access control. Access control in a proxy server can mean user account access, but also website(url), ip-address or dns restrictions.

67.1.2. open proxy servers

You can find lists of open proxy servers on the internet that enable you to surf anonymously. This works when the proxy server connects on your behalf to a website, without logging your ip-address. But be careful, these (listed) open proxy servers could be created in order to eavesdrop upon their users.

67.1.3. squid

This module is an introduction to the **squid** proxy server (<http://www.squid-cache.org>). We will first configure squid as a normal proxy server.

67.2. installing squid

This screenshot shows how to install squid on Debian with **aptitude**. Use **yum** if you are on Red Hat/CentOS.

```
root@debian7:~# aptitude install squid
The following NEW packages will be installed:
  squid squid-common{a} squid-langpack{a}
0 packages upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 1,513 kB of archives. After unpacking 4,540 kB will be used.
Do you want to continue? [Y/n/?]
...output truncated...
Setting up squid-langpack (20120616-1) ...
Setting up squid-common (2.7.STABLE9-4.1) ...
Setting up squid (2.7.STABLE9-4.1) ...
Creating squid spool directory structure
2014/08/01 15:19:31| Creating Swap Directories
Restarting Squid HTTP proxy: squid.
```

squid's main configuration file is **/etc/squid/squid.conf**. The file explains every parameter in great detail.

```
root@debian7:~# wc -l /etc/squid/squid.conf
4948 /etc/squid/squid.conf
```

67.3. port 3128

By default the **squid proxy server** will listen to **port 3128**.

```
root@debian7:~# grep ^http_port /etc/squid/squid.conf
http_port 3128
root@debian7:~#
```

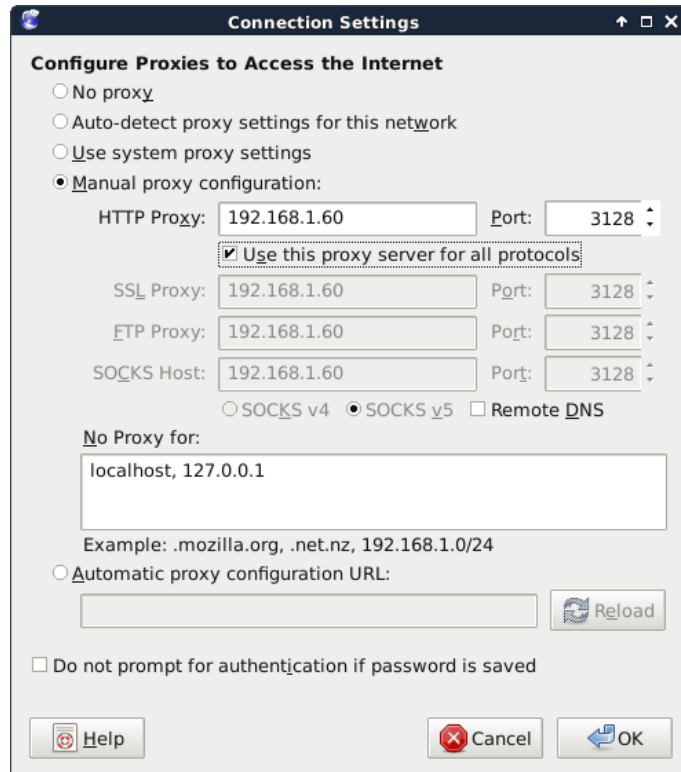
67.4. starting and stopping

You can manage **squid** with the standard **service** command as shown in this screenshot.

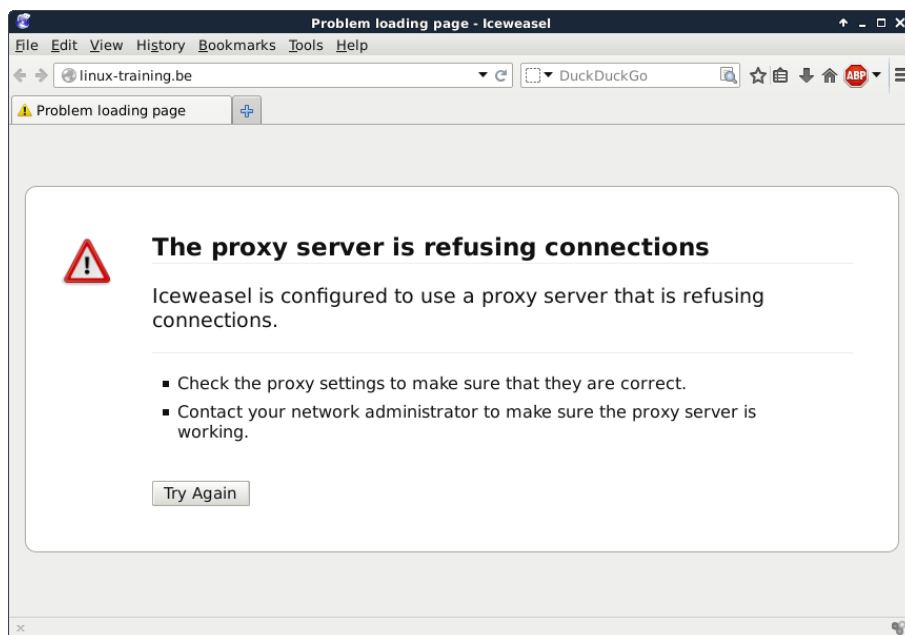
```
root@debian7:~# service squid start
Starting Squid HTTP proxy: squid.
root@debian7:~# service squid restart
Restarting Squid HTTP proxy: squid.
root@debian7:~# service squid status
squid is running.
root@debian7:~# service squid stop
Stopping Squid HTTP proxy: squid.
root@debian7:~#
```


67.5. client proxy settings

To enable a proxy server in **Firefox** or **Iceweasel** go to **Edit Preferences** and configure as shown in this screenshot (replace 192.168.1.60 with the ip address of your proxy server).



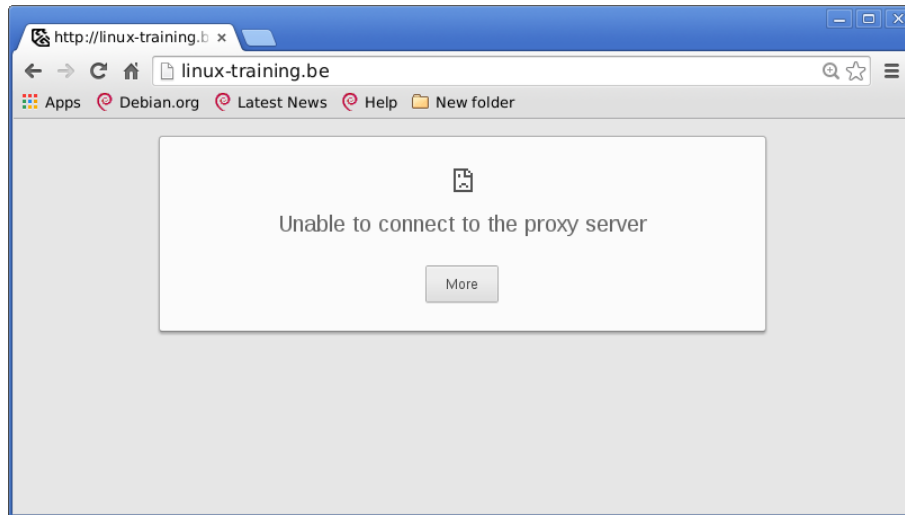
Test that your internet works with the proxy enabled. Also test that after a **service squid stop** command on your proxy server that you get a message similar to this screenshot.



To enable a proxy server with Google Chrome (or Debian Chromium) start the program from the command line like this:

```
paul@debian7:~$ chromium --proxy-server='192.168.1.60:3128'
```

Disabling the proxy with **service squid stop** should result in an error message similar to this screenshot.



67.6. upside down images

A proxy server sits inbetween your browser and the internet. So besides caching of internet data (the original function of a proxy server) and besides firewall like restrictions based on www content, a proxy server is in the perfect position to alter the webpages that you visit.

You could for instance change the advertising on a webpage (or remove certain advertisers), or like we do in this example; change all images so they are upside down.

The server needs command line tools to manipulate images and a perl script that uses these tools (and **wget** to download the images locally and serve them with **apache2**). In this example we use **imagemagick** (which provides tools like **convert** and **mogrify**).

```
root@debian7:~# aptitude install imagemagick wget perl apache2
...output truncated...
root@debian7:~# dpkg -S $(readlink -f $(which mogrify))
imagemagick: /usr/bin/mogrify.im6
root@debian7:~#
```

The perl script that is shown in the screenshot below can be found on several websites, yet I have not found the original author. It is however a very simple script that uses **wget** and **mogrify** to download images (.jpg .gif and .png), flip them and store them in **/var/www/images**.

```
root@debian7:~# cat /usr/local/bin/flip.pl
#!/usr/bin/perl
$|=1;
$count = 0;
$pid = $$;
while (<>) {
    chomp $_;
    if ($_ =~ /(.*\.jpg)/i) {
        $url = $_;
        system("/usr/bin/wget", "-q", "-O", "/var/www/images/$pid-$count.jpg", "$url");
        system("/usr/bin/mogrify", "-flip", "/var/www/images/$pid-$count.jpg");
        print "http://127.0.0.1/images/$pid-$count.jpg\n";
    }
    elsif ($_ =~ /(.*\.gif)/i) {
        $url = $_;
        system("/usr/bin/wget", "-q", "-O", "/var/www/images/$pid-$count.gif", "$url");
        system("/usr/bin/mogrify", "-flip", "/var/www/images/$pid-$count.gif");
        print "http://127.0.0.1/images/$pid-$count.gif\n";
    }
    elsif ($_ =~ /(.*\.png)/i) {
        $url = $_;
        system("/usr/bin/wget", "-q", "-O", "/var/www/images/$pid-$count.png", "$url");
        system("/usr/bin/mogrify", "-flip", "/var/www/images/$pid-$count.png");
        print "http://127.0.0.1/images/$pid-$count.png\n";
    }
    else {
        print "$_\n";
    }
    $count++;
}
```

Change (or enable) also the following line in **/etc/squid/suiqd.conf**.

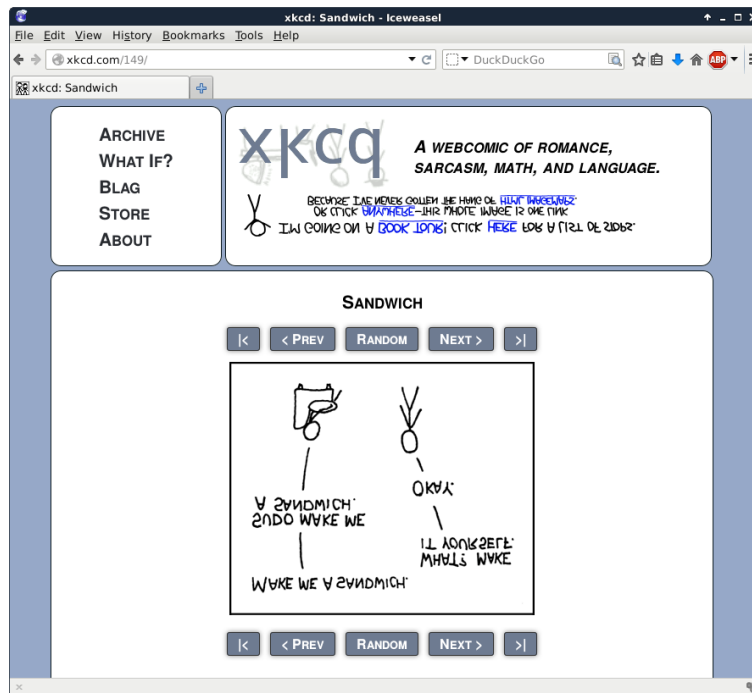
```
http_access allow localnet
http_port 3128 transparent
```

```
url_rwwrite_program /usr/local/bin/flip.pl
```

The directory this script uses is **/var/www/images** and should be accessible by both the **squid server** (which uses the user named **proxy** and by the **apache2** webserver (which uses the user **www-data**). The screenshot below shows how to create this directory, set the permissions and make the users a member of the other groups.

```
root@debian7:~# mkdir /var/www/images
root@debian7:~# chown www-data:www-data /var/www/images
root@debian7:~# chmod 755 /var/www/images
root@debian7:~# usermod -aG www-data proxy
root@debian7:~# usermod -aG proxy www-data
```

Test that it works after restarting **squid** and **apache2**.



67.7. /var/log/squid

The standard log file location for squid is **/var/log/squid**.

```
[root@RHEL4 ~]# grep "/var/log" /etc/squid/squid.conf
# cache_access_log /var/log/squid/access.log
# cache_log /var/log/squid/cache.log
# cache_store_log /var/log/squid/store.log
```

67.8. access control

The default squid setup only allows localhost access. To enable access for a private network range, look for the "INSERT YOUR OWN RULE(S) HERE..." sentence in squid.conf and add two lines similar to the screenshot below.

```
# INSERT YOUR OWN RULE(S) HERE TO ALLOW ACCESS FROM YOUR CLIENTS

acl company_network src 192.168.1.0/24
http_access allow company_network
```

67.9. testing squid

First, make sure that the server running squid has access to the internet.

```
[root@RHEL4 ~]# wget -q http://linux-training.be/index.html
[root@RHEL4 ~]# ls -l index.html
-rw-r--r-- 1 root root 2269 Sep 18 13:18 index.html
[root@RHEL4 ~]#
```

Then configure a browser on a client to use the proxy server, or you could set the HTTP_PROXY (sometimes http_proxy) variable to point command line programs to the proxy.

```
[root@fedora ~]# export HTTP_PROXY=http://192.168.1.39:8080
[root@ubuntu ~]# export http_proxy=http://192.168.1.39:8080
```

Testing a client machine can then be done with wget (wget -q is used to simplify the screenshot).

```
[root@RHEL5 ~]# > /etc/resolv.conf
[root@RHEL5 ~]# wget -q http://www.linux-training.be/index.html
[root@RHEL5 ~]# ls -l index.html
-rw-r--r-- 1 root root 2269 Sep 18 2008 index.html
[root@RHEL5 ~]#
```

67.10. name resolution

You need name resolution working on the **squid** server, but you don't need name resolution on the clients.

```
[paul@RHEL5 ~]$ wget http://grep.be
--14:35:44-- http://grep.be
Resolving grep.be... failed: Temporary failure in name resolution.
[paul@RHEL5 ~]$ export http_proxy=http://192.168.1.39:8080
[paul@RHEL5 ~]$ wget http://grep.be
--14:35:49-- http://grep.be/
```

```
Connecting to 192.168.1.39:8080... connected.  
Proxy request sent, awaiting response... 200 OK  
Length: 5390 (5.3K) [text/html]  
Saving to: `index.html.1'  
  
100%[=====>] 5,390      --.-K/s   in 0.1s  
  
14:38:29 (54.8 KB/s) - `index.html' saved [5390/5390]  
  
[paul@RHEL5 ~]$
```

Part XVIII. mysql database

Table of Contents

68. introduction to sql using mysql	729
68.1. installing mysql	730
68.2. accessing mysql	731
68.3. mysql databases	733
68.4. mysql tables	735
68.5. mysql records	737
68.6. joining two tables	740
68.7. mysql triggers	741

Chapter 68. introduction to sql using mysql

mysql is a database server that understands Structured Query Language (**SQL**). MySQL was developed by the Swedish Company **MySQL AB**. The first release was in 1995. In 2008 MySQL AB was bought by Sun Microsystems (which is now owned by Oracle).

mysql is very popular for websites in combination with **php** and **apache** (the **m** in **lamp** servers), but **mysql** is also used in organizations with huge databases like Facebook, Flickr, Google, Nokia, Wikipedia and Youtube.

This chapter will teach you **sql** by creating and using small databases, tables, queries and a simple trigger in a local **mysql** server.

68.1. installing mysql

On Debian/Ubuntu you can use **aptitude** install **mysql-server** to install the **mysql server** and **client**.

```
root@ubu1204~# aptitude install mysql-server
The following NEW packages will be installed:
  libdbd-mysql-perl{a} libdbi-perl{a} libhtml-template-perl{a}
  libnet-daemon-perl{a} libplrpc-perl{a} mysql-client-5.5{a}
  mysql-client-core-5.5{a} mysql-server mysql-server-5.5{a}
  mysql-server-core-5.5{a}
0 packages upgraded, 10 newly installed, 0 to remove and 1 not upgraded.
Need to get 25.5 MB of archives. After unpacking 88.4 MB will be used.
Do you want to continue? [Y/n/?]
```

During the installation you will be asked to provide a password for the **root mysql user**, remember this password (or use **hunter2** like i do).

To verify the installed version, use **dpkg -l** on Debian/Ubuntu. This screenshot shows version 5.0 installed.

```
root@ubu1204~# dpkg -l mysql-server | tail -1 | tr -s ' ' | cut -c-72
ii mysql-server 5.5.24-0ubuntu0.12.04.1 MySQL database server (metapacka
```

Issue **rpm -q** to get version information about MySQL on Red Hat/Fedora/CentOS.

```
[paul@RHEL52 ~]$ rpm -q mysql-server
mysql-server-5.0.45-7.el5
```

You will need at least version 5.0 to work with **triggers**.

68.2. accessing mysql

68.2.1. Linux users

The installation of **mysql** creates a user account in **/etc/passwd** and a group account in **/etc/group**.

```
kevin@ubu1204:~$ tail -1 /etc/passwd
mysql:x:120:131:MySQL Server,,,:/nonexistent:/bin/false
kevin@ubu1204:~$ tail -1 /etc/group
mysql:x:131:
```

The mysql daemon **mysqld** will run with the credentials of this user and group.

```
root@ubu1204~# ps -eo uid,user,gid,group,comm | grep mysqld
120 mysql      131 mysql      mysqld
```

68.2.2. mysql client application

You can now use mysql from the commandline by just typing **mysql -u root -p** and you'll be asked for the password (of the **mysql root** account). In the screenshot below the user typed **exit** to exit the mysql console.

```
root@ubu1204~# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 43
Server version: 5.5.24-0ubuntu0.12.04.1 (Ubuntu)

Copyright (c) 2000, 2011, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> exit
Bye
```

You could also put the password in clear text on the command line, but that would not be very secure. Anyone with access to your bash history would be able to read your mysql root password.

```
root@ubu1204~# mysql -u root -phunter2
Welcome to the MySQL monitor.  Commands end with ; or \g.
...
```

68.2.3. ~/.my.cnf

You can save configuration in your home directory in the hidden file **.my.cnf**. In the screenshot below we put the root user and password in **.my.cnf**.

```
kevin@ubu1204:~$ pwd
/home/kevin
kevin@ubu1204:~$ cat .my.cnf
[client]
user=root
password=hunter2
kevin@ubu1204:~$
```

This enables us to log on as the **root mysql** user just by typing **mysql**.

```
kevin@ubu1204:~$ mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 56
Server version: 5.5.24-0ubuntu0.12.04.1 (Ubuntu)
```

68.2.4. the mysql command line client

You can use the **mysql** command to take a look at the databases, and to execute SQL queries on them. The screenshots below show you how.

Here we execute the command **show databases**. Every command must be terminated by a delimiter. The default delimiter is **;** (the semicolon).

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| test |
+-----+
4 rows in set (0.00 sec)
```

We will use this prompt in the next sections.

68.3. mysql databases

68.3.1. listing all databases

You can use the **mysql** command to take a look at the databases, and to execute SQL queries on them. The screenshots below show you how. First, we log on to our MySQL server and execute the command **show databases** to see which databases exist on our mysql server.

```
kevin@ubu1204:~$ mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 57
Server version: 5.5.24-0ubuntu0.12.04.1 (Ubuntu)

Copyright (c) 2000, 2011, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database                |
+-----+
| information_schema       |
| mysql                   |
| performance_schema       |
| test                    |
+-----+
4 rows in set (0.00 sec)
```

68.3.2. creating a database

You can create a new database with the **create database** command.

```
mysql> create database famouspeople;
Query OK, 1 row affected (0.00 sec)

mysql> show databases;
+-----+
| Database                |
+-----+
| information_schema       |
| famouspeople             |
| mysql                   |
| performance_schema       |
| test                    |
+-----+
5 rows in set (0.00 sec)
```

68.3.3. using a database

Next we tell **mysql** to use one particular database with the **use \$database** command. This screenshot shows how to make wikidb the current database (in use).

```
mysql> use famouspeople;  
Database changed  
mysql>
```

68.3.4. access to a database

To give someone access to a mysql database, use the **grant** command.

```
mysql> grant all on famouspeople.* to kevin@localhost IDENTIFIED BY "hunter2";  
Query OK, 0 rows affected (0.00 sec)
```

68.3.5. deleting a database

When a database is no longer needed, you can permanently remove it with the **drop database** command.

```
mysql> drop database demodb;  
Query OK, 1 row affected (0.09 sec)
```

68.3.6. backup and restore a database

You can take a backup of a database, or move it to another computer using the **mysql** and **mysqldump** commands. In the screenshot below, we take a backup of the wikidb database on the computer named laika.

```
mysqldump -u root famouspeople > famouspeople.backup.20120708.sql
```

Here is a screenshot of a database restore operation from this backup.

```
mysql -u root famouspeople < famouspeople.backup.20120708.sql
```

68.4. mysql tables

68.4.1. listing tables

You can see a list of tables in the current database with the **show tables;** command. Our **famouspeople** database has no tables yet.

```
mysql> use famouspeople;
Database changed
mysql> show tables;
Empty set (0.00 sec)
```

68.4.2. creating a table

The **create table** command will create a new table.

This screenshot shows the creation of a country table. We use the **countrycode** as a **primary key** (all country codes are uniquely defined). Most country codes are two or three letters, so a **char** of three uses less space than a **varchar** of three. The **country name** and the name of the capital are both defined as **varchar**. The population can be seen as an **integer**.

```
mysql> create table country (
-> countrycode char(3) NOT NULL,
-> countryname varchar(70) NOT NULL,
-> population int,
-> countrycapital varchar(50),
-> primary key (countrycode)
-> );
```

Query OK, 0 rows affected (0.19 sec)

```
mysql> show tables;
+-----+
| Tables_in_famouspeople |
+-----+
| country                  |
+-----+
1 row in set (0.00 sec)
```

```
mysql>
```

You are allowed to type the **create table** command on one long line, but administrators often use multiple lines to improve readability.

```
mysql> create table country ( countrycode char(3) NOT NULL, countryname\
varchar(70) NOT NULL, population int, countrycapital varchar(50), prim\
ary key (countrycode) );
Query OK, 0 rows affected (0.18 sec)
```

68.4.3. describing a table

To see a description of the structure of a table, issue the **describe \$tablename** command as shown below.

```
mysql> describe country;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| countrycode    | char(3)       | NO   | PRI | NULL    |       |
| countryname    | varchar(70)   | NO   |     | NULL    |       |
| population     | int(11)       | YES  |     | NULL    |       |
| countrycapital | varchar(50)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

68.4.4. removing a table

To remove a table from a database, issue the **drop table \$tablename** command as shown below.

```
mysql> drop table country;
Query OK, 0 rows affected (0.00 sec)
```


68.5. mysql records

68.5.1. creating records

Use **insert** to enter data into the table. The screenshot shows several insert statements that insert values depending on the position of the data in the statement.

```
mysql> insert into country values ('BE','Belgium','11000000','Brussels');
Query OK, 1 row affected (0.05 sec)

mysql> insert into country values ('DE','Germany','82000000','Berlin');
Query OK, 1 row affected (0.05 sec)

mysql> insert into country values ('JP','Japan','128000000','Tokyo');
Query OK, 1 row affected (0.05 sec)
```

Some administrators prefer to use uppercase for **sql** keywords. The mysql client accepts both.

```
mysql> INSERT INTO country VALUES ('FR','France','64000000','Paris');
Query OK, 1 row affected (0.00 sec)
```

Note that you get an error when using a duplicate **primary key**.

```
mysql> insert into country values ('DE','Germany','82000000','Berlin');
ERROR 1062 (23000): Duplicate entry 'DE' for key 'PRIMARY'
```

68.5.2. viewing all records

Below an example of a simple **select** query to look at the contents of a table.

```
mysql> select * from country;
+-----+-----+-----+-----+
| countrycode | countryname | population | countrycapital |
+-----+-----+-----+-----+
| BE          | Belgium     | 11000000  | Brussels       |
| CN          | China       | 1400000000 | Beijing        |
| DE          | Germany     | 82000000  | Berlin         |
| FR          | France      | 64000000  | Paris          |
| IN          | India       | 1300000000 | New Delhi      |
| JP          | Japan       | 128000000 | Tokyo          |
| MX          | Mexico      | 113000000 | Mexico City    |
| US          | United States | 313000000 | Washington     |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

68.5.3. updating records

Consider the following **insert** statement. The capital of Spain is not Barcelona, it is Madrid.

```
mysql> insert into country values ('ES','Spain','48000000','Barcelona');
Query OK, 1 row affected (0.08 sec)
```

Using an **update** statement, the record can be updated.

```
mysql> update country set countrycapital='Madrid' where countrycode='ES';
Query OK, 1 row affected (0.07 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

We can use a **select** statement to verify this change.

```
mysql> select * from country;
+-----+-----+-----+-----+
| countrycode | countryname | population | countrycapital |
+-----+-----+-----+-----+
| BE          | Belgium    | 11000000  | Brussels       |
| CN          | China      | 1400000000| Beijing        |
| DE          | Germany    | 82000000  | Berlin         |
| ES          | Spain      | 48000000  | Madrid         |
| FR          | France     | 64000000  | Paris          |
| IN          | India      | 1300000000| New Delhi      |
| JP          | Japan      | 128000000 | Tokyo          |
| MX          | Mexico     | 113000000 | Mexico City    |
| US          | United States | 313000000 | Washington     |
+-----+-----+-----+-----+
9 rows in set (0.00 sec)
```

68.5.4. viewing selected records

Using a **where** clause in a **select** statement, you can specify which record(s) you want to see.

```
mysql> SELECT * FROM country WHERE countrycode='ES';
+-----+-----+-----+-----+
| countrycode | countryname | population | countrycapital |
+-----+-----+-----+-----+
| ES          | Spain      | 48000000  | Madrid         |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Another example of the **where** clause.

```
mysql> select * from country where countryname='Spain';
+-----+-----+-----+-----+
| countrycode | countryname | population | countrycapital |
+-----+-----+-----+-----+
| ES          | Spain      | 48000000  | Madrid         |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

68.5.5. primary key in where clause ?

The **primary key** of a table is a field that uniquely identifies every record (every row) in the table. when using another field in the **where** clause, it is possible to get multiple rows returned.

```
mysql> insert into country values ('EG','Egypt','82000000','Cairo');
```

Query OK, 1 row affected (0.33 sec)

```
mysql> select * from country where population='82000000';
```

countrycode	countryname	population	countrycapital
DE	Germany	82000000	Berlin
EG	Egypt	82000000	Cairo

2 rows in set (0.00 sec)

68.5.6. ordering records

We know that **select** allows us to see all records in a table. Consider this table.

```
mysql> select countryname,population from country;
```

countryname	population
Belgium	11000000
China	1400000000
Germany	82000000
Egypt	82000000
Spain	48000000
France	64000000
India	1300000000
Japan	128000000
Mexico	113000000
United States	313000000

10 rows in set (0.00 sec)

Using the **order by** clause, we can change the order in which the records are presented.

```
mysql> select countryname,population from country order by countryname;
```

countryname	population
Belgium	11000000
China	1400000000
Egypt	82000000
France	64000000
Germany	82000000
India	1300000000
Japan	128000000
Mexico	113000000
Spain	48000000
United States	313000000

10 rows in set (0.00 sec)

68.5.7. grouping records

Consider this table of people. The screenshot shows how to use the **avg** function to calculate an average.

```
mysql> select * from people;
```

Name	Field	birthyear	countrycode
Barack Obama	politics	1961	US
Deng Xiaoping	politics	1904	CN

```
| Guy Verhofstadt | politics | 1953 | BE |
| Justine Henin   | tennis  | 1982 | BE |
| Kim Clijsters   | tennis  | 1983 | BE |
| Li Na           | tennis  | 1982 | CN |
| Liu Yang        | astronaut | 1978 | CN |
| Serena Williams | tennis  | 1981 | US |
| Venus Williams  | tennis  | 1980 | US |
+-----+-----+-----+-----+
9 rows in set (0.00 sec)

mysql> select Field,AVG(birthyear) from people;
+-----+-----+
| Field | AVG(birthyear) |
+-----+-----+
| politics | 1967.111111111111 |
+-----+-----+
1 row in set (0.00 sec)
```

Using the **group by** clause, we can have an average per field.

```
mysql> select Field,AVG(birthyear) from people group by Field;
+-----+-----+
| Field | AVG(birthyear) |
+-----+-----+
| astronaut | 1978 |
| politics | 1939.333333333333 |
| tennis | 1981.6 |
+-----+-----+
3 rows in set (0.00 sec)
```

68.5.8. deleting records

You can use the **delete** to permanently remove a record from a table.

```
mysql> delete from country where countryname='Spain';
Query OK, 1 row affected (0.06 sec)

mysql> select * from country where countryname='Spain';
Empty set (0.00 sec)
```

68.6. joining two tables

68.6.1. inner join

With an **inner join** you can take values from two tables and combine them in one result. Consider the country and the people tables from the previous section when looking at this screenshot of an **inner join**.

```
mysql> select Name,Field,countryname
-> from country
-> inner join people on people.countrycode=country.countrycode;
+-----+-----+-----+
| Name | Field | countryname |
+-----+-----+-----+
| Barack Obama | politics | United States |
| Deng Xiaoping | politics | China |
| Guy Verhofstadt | politics | Belgium |
| Justine Henin | tennis | Belgium |
| Kim Clijsters | tennis | Belgium |
| Li Na | tennis | China |
```

```
| Liu Yang      | astronaut | China      |
| Serena Williams | tennis   | United States |
| Venus Williams | tennis   | United States |
+-----+-----+-----+
9 rows in set (0.00 sec)
```

This **inner join** will show only records with a match on **countrycode** in both tables.

68.6.2. left join

A **left join** is different from an **inner join** in that it will take all rows from the left table, regardless of a match in the right table.

```
mysql> select Name,Field,countrysname from country left join people on people.countrycode=countr
+-----+-----+-----+
| Name      | Field    | countrysname |
+-----+-----+-----+
| Guy Verhofstadt | politics | Belgium      |
| Justine Henin   | tennis  | Belgium      |
| Kim Clijsters   | tennis  | Belgium      |
| Deng Xiaoping   | politics | China        |
| Li Na           | tennis  | China        |
| Liu Yang        | astronaut | China        |
| NULL           | NULL     | Germany      |
| NULL           | NULL     | Egypt        |
| NULL           | NULL     | Spain        |
| NULL           | NULL     | France       |
| NULL           | NULL     | India        |
| NULL           | NULL     | Japan        |
| NULL           | NULL     | Mexico       |
| Barack Obama    | politics | United States |
| Serena Williams | tennis  | United States |
| Venus Williams  | tennis  | United States |
+-----+-----+-----+
16 rows in set (0.00 sec)
```

You can see that some countries are present, even when they have no matching records in the **people** table.

68.7. mysql triggers

68.7.1. using a before trigger

Consider the following **create table** command. The last field (**amount**) is the multiplication of the two fields named **unitprice** and **unitcount**.

```
mysql> create table invoices (
-> id char(8) NOT NULL,
-> customerid char(3) NOT NULL,
-> unitprice int,
-> unitcount smallint,
-> amount int );
Query OK, 0 rows affected (0.00 sec)
```

We can let mysql do the calculation for that by using a **before trigger**. The screenshot below shows the creation of a trigger that calculates the amount by multiplying two fields that are about to be inserted.

```
mysql> create trigger total_amount before INSERT on invoices
```

```
-> for each row set new.amount = new.unitprice * new.unitcount ;  
Query OK, 0 rows affected (0.02 sec)
```

Here we verify that the trigger works by inserting a new record, without providing the total amount.

```
mysql> insert into invoices values ('20090526','ABC','199','10','');  
Query OK, 1 row affected (0.02 sec)
```

Looking at the record proves that the trigger works.

```
mysql> select * from invoices;  
+-----+-----+-----+-----+-----+  
| id      | customerid | unitprice | unitcount | amount |  
+-----+-----+-----+-----+-----+  
| 20090526 | ABC       | 199      | 10       | 1990   |  
+-----+-----+-----+-----+-----+  
1 row in set (0.00 sec)
```

68.7.2. removing a trigger

When a **trigger** is no longer needed, you can delete it with the **drop trigger** command.

```
mysql> drop trigger total_amount;  
Query OK, 0 rows affected (0.00 sec)
```

Part XIX. dns server

Table of Contents

69. introduction to DNS	745
69.1. about dns	746
69.2. dns namespace	749
69.3. caching only servers	754
69.4. authoritative dns servers	757
69.5. primary and secondary	757
69.6. zone transfers	757
69.7. master and slave	759
69.8. SOA record	759
69.9. full or incremental zone transfers	760
69.10. DNS cache	761
69.11. forward lookup zone example	762
69.12. example: caching only DNS server	763
69.13. example: caching only with forwarder	765
69.14. example: primary authoritative server	767
69.15. example: a DNS slave server	771
69.16. practice: dns	773
69.17. solution: dns	774
70. advanced DNS	775
70.1. example: DNS round robin	776
70.2. DNS delegation	777
70.3. example: DNS delegation	778
70.4. example: split-horizon dns	780
70.5. old dns topics	782

Chapter 69. introduction to DNS

dns is a fundamental part of every large computer network. **dns** is used by many network services to translate names into network addresses and to locate services on the network (by name).

Whenever you visit a web site, send an e-mail, log on to Active Directory, play Minecraft, chat, or use VoIP, there will be one or (many) more queries to **dns** services.

Should **dns** fail at your organization, then the whole network will grind to a halt (unless you hardcoded the network addresses).

You will notice that even the largest of organizations benefit greatly from having one **dns** infrastructure. Thus **dns** requires all business units to work together.

Even at home, most home modems and routers have builtin **dns** functionality.

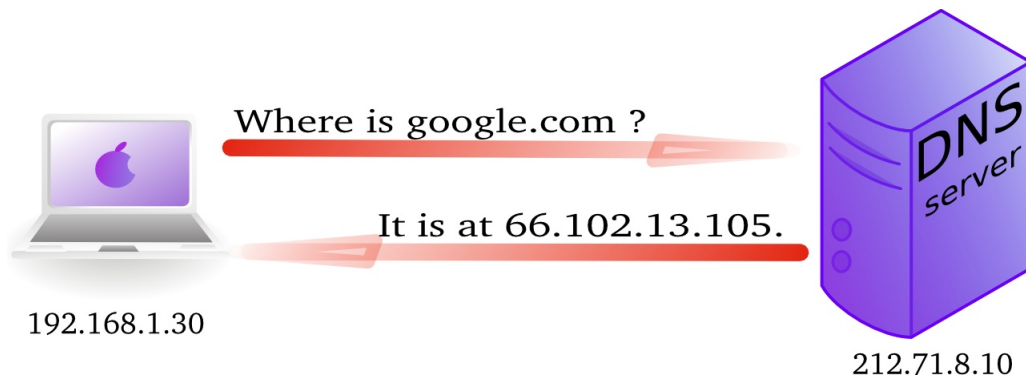
This module will explain what **dns** actually is and how to set it up using **Linux** and **bind9**.

69.1. about dns

69.1.1. name to ip address resolution

The **domain name system** or **dns** is a service on a tcp/ip network that enables clients to translate names into ip addresses. Actually **dns** is much more than that, but let's keep it simple for now.

When you use a browser to go to a website, then you type the name of that website in the url bar. But for your computer to actually communicate with the web server hosting said website, your computer needs the ip address of that web server. That is where **dns** comes in.



In wireshark you can use the **dns** filter to see this traffic.

Filter:	dns	▼	Expression...	Clear	Apply
No. -	Time	Source	Destination	Protocol	Info
4560	11.467767	192.168.1.30	212.71.8.10	DNS	Standard query A google.com
4569	11.487774	212.71.8.10	192.168.1.30	DNS	Standard query response A 66.102.13.105

69.1.2. history

In the Seventies, only a few hundred computers were connected to the internet. To resolve names, computers had a flat file that contained a table to resolve hostnames to ip addresses. This local file was downloaded from **hosts.txt** on an ftp server in Stanford.

In 1984 **Paul Mockapetris** created **dns**, a distributed treelike hierarchical database that will be explained in detail in these chapters.

Today, **dns** or **domain name system** is a worldwide distributed hierarchical database controlled by **ICANN**. Its primary function is to resolve names to ip addresses, and to point to internet servers providing **smtp** or **ldap** services.

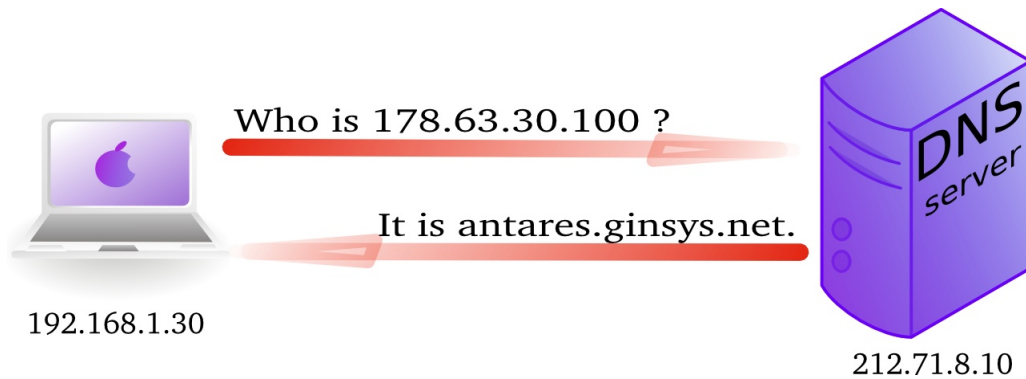
The old **hosts.txt** file is still active today on most computer systems under the name **/etc/hosts** (or C:/Windows/System32/Drivers/etc/hosts). We will discuss this file later, as it can influence name resolution.

69.1.3. forward and reverse lookup queries

The question a client asks a dns server is called a **query**. When a client queries for an ip address, this is called a **forward lookup query** (as seen in the previous drawing).

The reverse, a query for the name of a host, is called a **reverse lookup query**.

Below a picture of a **reverse lookup query**.



Here is a screenshot of a **reverse lookup query** in **nslookup**.

```
root@debian7:~# nslookup
> set type=PTR
> 188.93.155.87
Server:      192.168.1.42
Address:     192.168.1.42#53

Non-authoritative answer:
87.155.93.188.in-addr.arpa      name = antares.ginsys.net.
```

This is what a reverse lookup looks like when sniffing with **tcpdump**.

```
root@debian7:~# tcpdump udp port 53
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 65535 bytes
11:01:29.357685 IP 192.168.1.103.42041 > 192.168.1.42.domain: 14763+ PTR\
R? 87.155.93.188.in-addr.arpa. (44)
11:01:29.640093 IP 192.168.1.42.domain > 192.168.1.103.42041: 14763 1/0\
/0 PTR antares.ginsys.net. (76)
```

And here is what it looks like in **wireshark** (note this is an older screenshot).

Filter: dns						Expression...	Clear	Apply
No.	Time	Source	Destination	Protocol	Info			
280	172.307847	192.168.1.30	212.71.8.10	DNS	Standard query PTR 100.30.63.178.in-addr.arpa			
281	172.321299	212.71.8.10	192.168.1.30	DNS	Standard query response PTR antares.ginsys.net			

69.1.4. /etc/resolv.conf

A client computer needs to know the ip address of the **dns server** to be able to send queries to it. This is either provided by a **dhcp server** or manually entered.

Linux clients keep this information in the **/etc/resolv.conf** file.

```
root@debian7:~# cat /etc/resolv.conf
domain linux-training.be
search linux-training.be
nameserver 192.168.1.42
root@debian7:~#
```

You can manually change the ip address in this file to use another **dns** server. For example Google provides a public name server at 8.8.8.8 and 8.8.4.4.

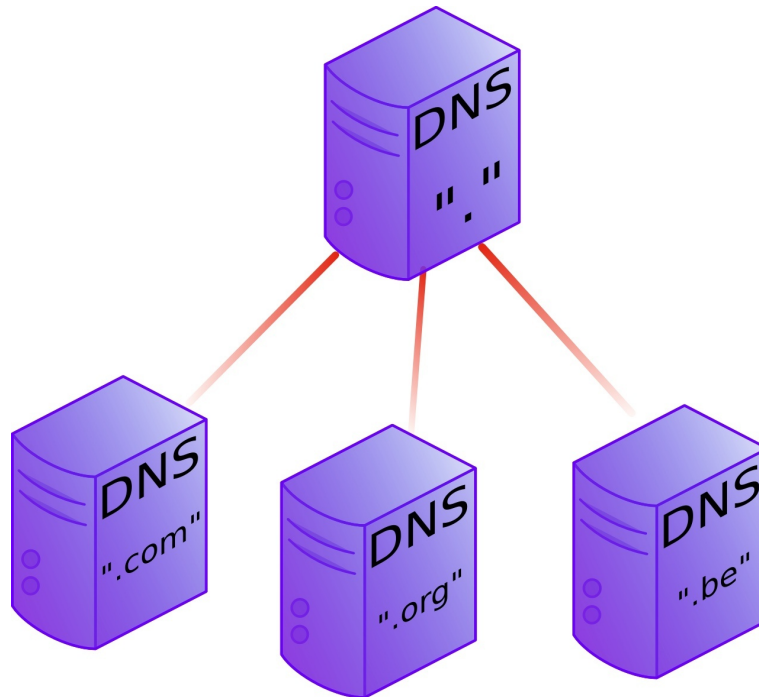
```
root@debian7:~# cat /etc/resolv.conf
nameserver 8.8.8.8
root@debian7:~#
```

Please note that on **dhcp clients** this value can be overwritten when the **dhcp lease** is renewed.

69.2. dns namespace

69.2.1. hierarchy

The **dns namespace** is hierarchical tree structure, with the **root servers** (aka dot-servers) at the top. The **root servers** are usually represented by a dot.



Below the **root-servers** are the **Top Level Domains** or **tld's**.

There are more **tld's** than shown in the picture. Currently about 200 countries have a **tld**. And there are several general **tld's** like .com, .edu, .org, .gov, .net, .mil, .int and more recently also .aero, .info, .museum, ...

69.2.2. root servers

There are thirteen **root servers** on the internet, they are named **A** to **M**. Journalists often refer to these servers as **the master servers of the internet**, because if these servers go down, then nobody can (use names to) connect to websites.

The root servers are not thirteen physical machines, they are many more. For example the **F** root server consists of 46 physical machines that all behave as one (using anycast).

<http://root-servers.org>
<http://f.root-servers.org>
http://en.wikipedia.org/wiki/Root_nameserver.

69.2.3. root hints

Every **dns server software** will come with a list of **root hints** to locate the **root servers**.

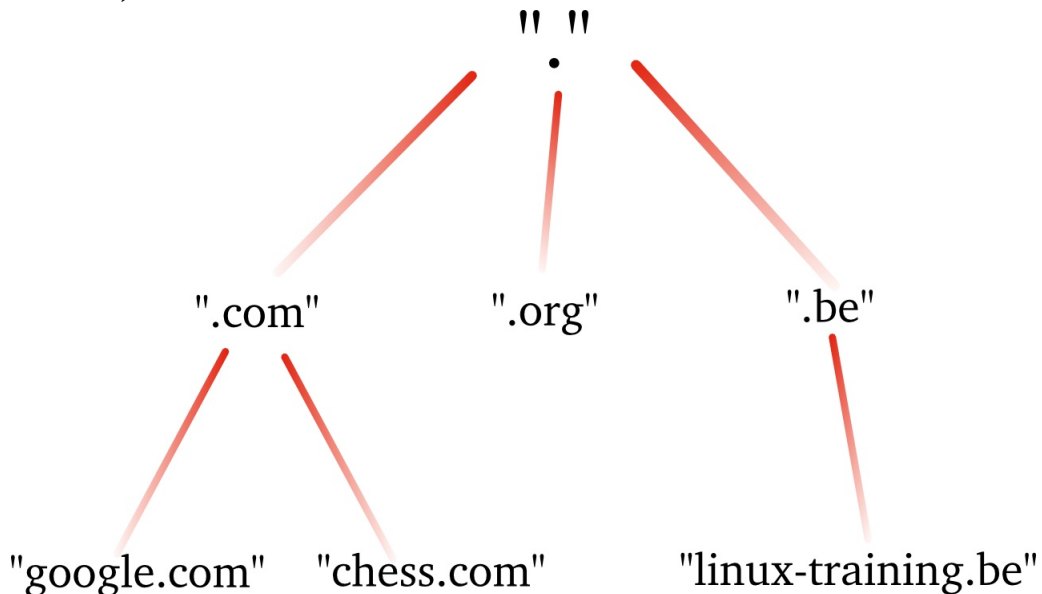
This screenshot shows a small portion of the root hints file that comes with **bind 9.8.4**.

```
root@debian7:~# grep -w 'A ' /etc/bind/db.root
A.ROOT-SERVERS.NET.      3600000      A      198.41.0.4
B.ROOT-SERVERS.NET.      3600000      A      192.228.79.201
C.ROOT-SERVERS.NET.      3600000      A      192.33.4.12
D.ROOT-SERVERS.NET.      3600000      A      199.7.91.13
E.ROOT-SERVERS.NET.      3600000      A      192.203.230.10
F.ROOT-SERVERS.NET.      3600000      A      192.5.5.241
G.ROOT-SERVERS.NET.      3600000      A      192.112.36.4
H.ROOT-SERVERS.NET.      3600000      A      128.63.2.53
I.ROOT-SERVERS.NET.      3600000      A      192.36.148.17
J.ROOT-SERVERS.NET.      3600000      A      192.58.128.30
K.ROOT-SERVERS.NET.      3600000      A      193.0.14.129
L.ROOT-SERVERS.NET.      3600000      A      199.7.83.42
M.ROOT-SERVERS.NET.      3600000      A      202.12.27.33
root@debian7:~#
```

69.2.4. domains

One level below the **top level domains** are the **domains**. Domains can have subdomains (also called child domains).

This picture shows **dns domains** like google.com, chess.com, linux-training.be (there are millions more).



DNS domains are registered at the **tld** servers, the **tld** servers are registered at the **dot servers**.

69.2.5. top level domains

Below the root level are the **top level domains** or **tld's**. Originally there were only seven defined:

Table 69.1. the first top level domains

year	TLD	purpose
1985	.arpa	Reverse lookup via in-addr.arpa
1985	.com	Commercial Organizations
1985	.edu	US Educational Institutions
1985	.gov	US Government Institutions
1985	.mil	US Military
1985	.net	Internet Service Providers, Internet Infrastructure
1985	.org	Non profit Organizations
1988	.int	International Treaties like nato.int

Country **tld's** were defined for individual countries, like **.uk** in 1985 for Great Britain (yes really), **.be** for Belgium in 1988 and **.fr** for France in 1986. See RFC 1591 for more info.

In 1998 seven new general purpose **tld's** where chosen, they became active in the 21st century.

Table 69.2. new general purpose tld's

year	TLD	purpose
2002	.aero	aviation related
2001	.biz	businesses
2001	.coop	for co-operatives
2001	.info	informative internet resources
2001	.museum	for museums
2001	.name	for all kinds of names, pseudonyms and labels...
2004	.pro	for professionals

Many people were surprised by the choices, claiming not much use for them and wanting a separate **.xxx** domain (introduced in 2011) for adult content, and **.kidz** a save haven for children. In the meantime more useless **tld's** were create like **.travel** (for travel agents) and **.tel** (for internet communications) and **.jobs** (for jobs sites).

In 2012 **ICANN** released a list of 2000 new **tld's** that would gradually become available.

69.2.6. fully qualified domain name

The **fully qualified domain name** or **fqdn** is the combination of the **hostname** of a machine appended with its **domain name**.

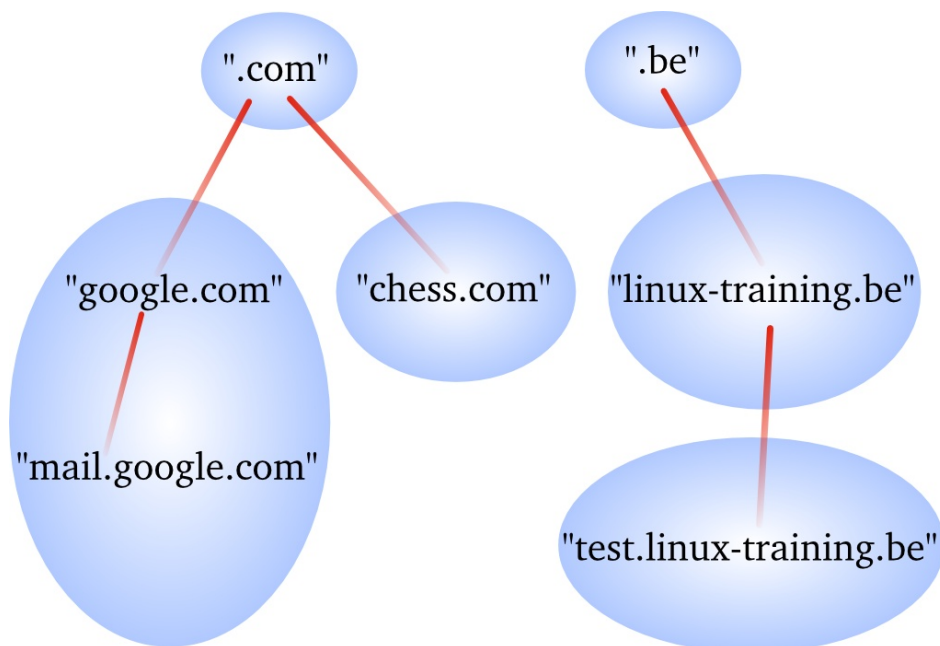
If for example a system is called **gwen** and it is in the domain **linux-training.be**, then the fqdn of this system is **gwen.linux-training.be**.

On Linux systems you can use the **hostname** and **dnsdomainname** commands to verify this information.

```
root@gwen:~# hostname
gwen
root@gwen:~# dnsdomainname
linux-training.be
root@gwen:~# hostname --fqdn
gwen.linux-training.be
root@gwen:~# cat /etc/debian_version
6.0.10
```

69.2.7. dns zones

A **zone** (aka a **zone of authority**) is a portion of the DNS tree that covers one domain name or child domain name. The picture below represents zones as blue ovals. Some zones will contain delegate authority over a child domain to another zone.



A **dns server** can be **authoritative** over 0, 1 or more **dns zones**. We will see more details later on the relation between a **dns server** and a **dns zone**.

A **dns zone** consists of **records**, also called **resource records**. We will list some of those **resource records** on the next page.

69.2.8. dns records

A record

The **A record**, which is also called a **host record** contains the ipv4-address of a computer. When a DNS client queries a DNS server for an A record, then the DNS server will resolve the hostname in the query to an ip address. An **AAAA record** is similar but contains an ipv6 address instead of ipv4.

PTR record

A **PTR record** is the reverse of an A record. It contains the name of a computer and can be used to resolve an ip address to a hostname.

NS record

A **NS record** or **nameserver record** is a record that points to a DNS name server (in this zone). You can list all your name servers for your DNS zone in distinct NS records.

glue A record

An A record that maps the name of an NS record to an ip address is said to be a **glue record**.

SOA record

The SOA record of a zone contains meta information about the zone itself. The contents of the SOA record is explained in detail in the section about zone transfers. There is exactly one SOA record for each zone.

CNAME record

A **CNAME record** maps a hostname to a hostname, creating effectively an alias for an existing hostname. The name of the mail server is often aliased to **mail** or **smtp**, and the name of a web server to **www**.

MX record

The **MX** record points to an **smtp server**. When you send an email to another domain, then your mail server will need the MX record of the target domain's mail server.

69.3. caching only servers

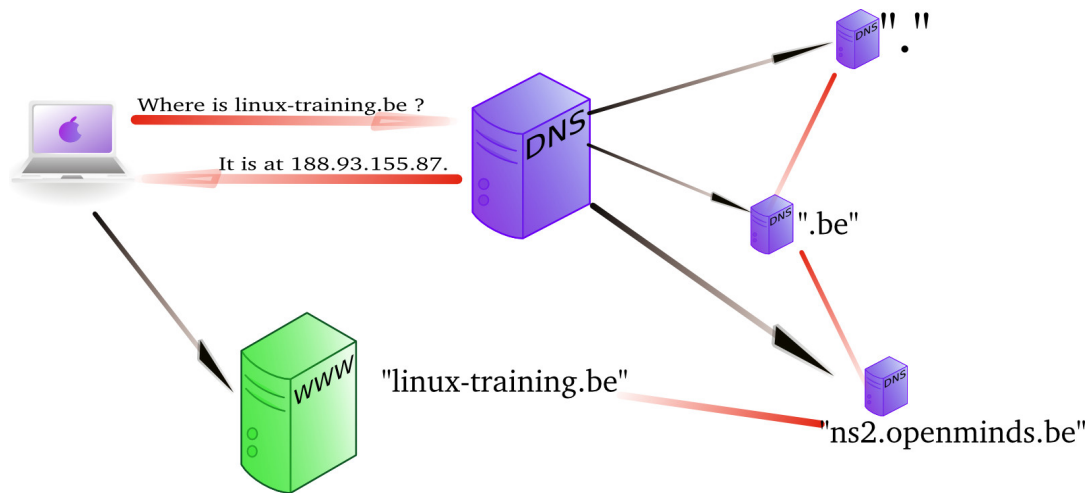
A **dns server** that is set up without **authority** over a **zone**, but that is connected to other name servers and caches the queries is called a **caching only name server**. Caching only name servers do not have a **zone database** with resource records. Instead they connect to other name servers and cache that information.

There are two kinds of caching only name servers. Those with a **forwarder**, and those that use the **root servers**.

69.3.1. caching only server without forwarder

A caching only server without forwarder will have to get information elsewhere. When it receives a query from a client, then it will consult one of the **root servers**. The **root server** will refer it to a **tld** server, which will refer it to another **dns** server. That last server might know the answer to the query, or may refer to yet another server. In the end, our hard working **dns** server will find an answer and report this back to the client.

In the picture below, the clients asks for the ip address of linux-training.be. Our caching only server will contact the root server, and be referred to the .be server. It will then contact the .be server and be referred to one of the name servers of Openminds. One of these name servers (in this cas ns1.openminds.be) will answer the query with the ip address of linux-training.be. When our caching only server reports this to the client, then the client can connect to this website.

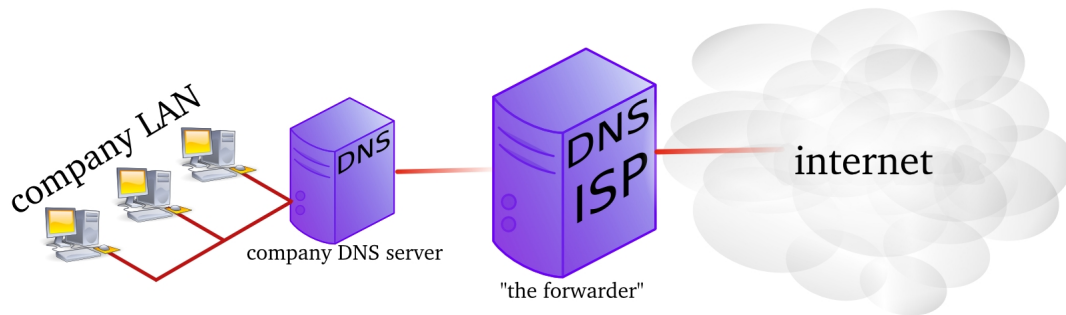


Sniffing with **tcpdump** will give you this (the first 20 characters of each line are cut).

```
192.168.1.103.41251 > M.ROOT-SERVERS.NET.domain: 37279% [1au] A? linux-tr\
aining.be. (46)
M.ROOT-SERVERS.NET.domain > 192.168.1.103.41251: 37279- 0/11/13 (740)
192.168.1.103.65268 > d.ns.dns.be.domain: 38555% [1au] A? linux-training.\
be. (46)
d.ns.dns.be.domain > 192.168.1.103.65268: 38555- 0/7/5 (737)
192.168.1.103.7514 > ns2.openminds.be.domain: 60888% [1au] A? linux-train\
ing.be. (46)
ns2.openminds.be.domain > 192.168.1.103.7514: 60888*- 1/0/1 A 188.93.155.\
87 (62)
```

69.3.2. caching only server with forwarder

A **caching only server** with a **forwarder** is a DNS server that will get all its information from the **forwarder**. The **forwarder** must be a **dns server** for example the **dns server** of an **internet service provider**.



This picture shows a **dns server** on the company LAN that has set the **dns server** from their **isp** as a **forwarder**. If the ip address of the **isp dns server** is 212.71.8.10, then the following lines would occur in the **named.conf** file of the company **dns server**:

```
forwarders {
    212.71.8.10;
};
```

You can also configure your **dns server** to work with **conditional forwarder(s)**. The definition of a conditional forwarder looks like this.

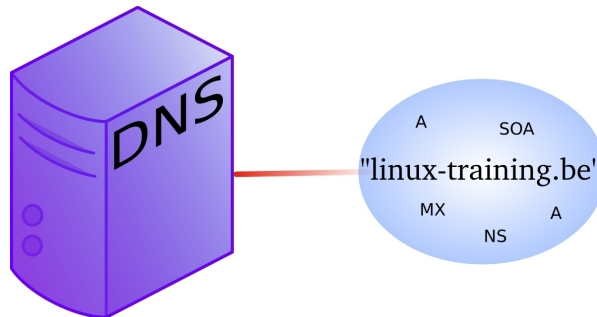
```
zone "someotherdomain.local" {
    type forward;
    forward only;
    forwarders { 10.104.42.1; };
};
```

69.3.3. iterative or recursive query

A **recursive query** is a DNS query where the client that is submitting the query expects a complete answer (Like the fat red arrow above going from the Macbook to the DNS server). An **iterative query** is a DNS query where the client does not expect a complete answer (the three black arrows originating from the DNS server in the picture above). Iterative queries usually take place between name servers. The root name servers do not respond to recursive queries.

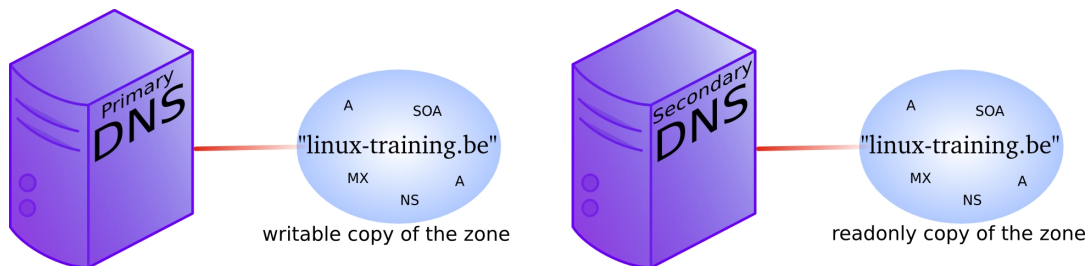
69.4. authoritative dns servers

A DNS server that is controlling a zone, is said to be the **authoritative** DNS server for that zone. Remember that a **zone** is a collection of **resource records**.



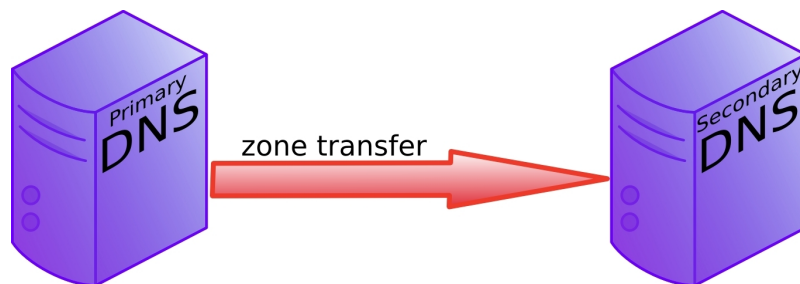
69.5. primary and secondary

When you set up the first **authoritative** dns server for a zone, then this is called the **primary dns server**. This server will have a readable and writable copy of the **zone database**. For reasons of fault tolerance, performance or load balancing you may decide to set up another **dns server** with authority over that zone. This is called a **secondary** dns server.



69.6. zone transfers

The slave server receives a copy of the zone database from the master server using a **zone transfer**. Zone transfers are requested by the slave servers at regular intervals. Those intervals are defined in the **soa record**.



You can force a refresh from a zone with **rndc**. The example below force a transfer of the **fred.local** zone, and shows the log from **/var/log/syslog**.

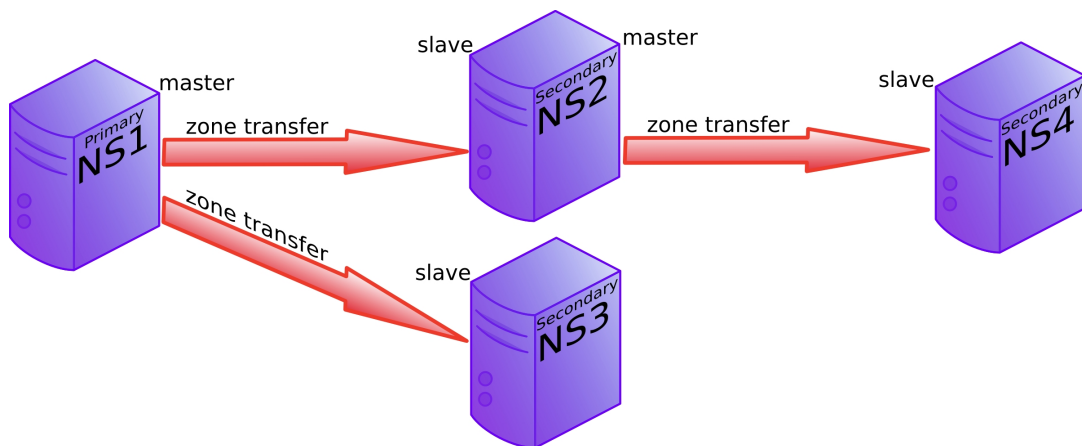
```
root@debian7:/etc/bind# rndc refresh fred.local
```

```
root@debian7:/etc/bind# grep fred /var/log/syslog | tail -7 | cut -c38-  
zone fred.local/IN: sending notifies (serial 1)  
received control channel command 'refresh fred.local'  
zone fred.local/IN: Transfer started.  
transfer of 'fred.local/IN' from 10.104.109.1#53: connected using 10.104.33.30#57367  
zone fred.local/IN: transferred serial 2  
transfer of 'fred.local/IN' from 10.104.109.1#53: Transfer completed: 1 messages, 10 records,  
zone fred.local/IN: sending notifies (serial 2)  
root@debian7:/etc/bind#
```

69.7. master and slave

When adding a **secondary dns server** to a zone, then you will configure this server as a **slave server** to the **primary server**. The primary server then becomes the **master server** of the slave server.

Often the **primary dns server** is the **master** server of all slaves. Sometimes a **slave server** is **master server** for a second line slave server. In the picture below ns1 is the primary dns server and ns2, ns3 and ns4 are secondaries. The master for slaves ns2 and ns3 is ns1, but the master for ns4 is ns2.



69.8. SOA record

The **soa record** contains a **refresh** value. If this is set to 30 minutes, then the slave server will request a copy of the zone file every 30 minutes. There is also a **retry** value. The retry value is used when the master server did not reply to the last zone transfer request. The value for **expiry time** says how long the slave server will answer to queries, without receiving a zone update.

Below an example of how to use nslookup to query the **soa record** of a zone (linux-training.be).

```

root@debian6:~# nslookup
> set type=SOA
> server ns1.openminds.be
> linux-training.be
Server:      ns1.openminds.be
Address:     195.47.215.14#53

linux-training.be
    origin = ns1.openminds.be
    mail addr = hostmaster.openminds.be
    serial = 2321001133
    refresh = 14400
    retry = 3600
    expire = 604800
    minimum = 3600
  
```

Zone transfers only occur when the zone database was updated (meaning when one or more resource records were added, removed or changed on the master server). The slave server

will compare the **serial number** of its own copy of the SOA record with the serial number of its master's SOA record. When both serial numbers are the same, then no update is needed (because no records were added, removed or deleted). When the slave has a lower serial number than its master, then a zone transfer is requested.

Below a zone transfer captured in wireshark.

Time	Source	Destination	Protocol	Info
1 0.000000	192.168.1.37	192.168.1.35	DNS	Standard query SOA cobbaut.paul
2 0.008502	192.168.1.35	192.168.1.37	DNS	Standard query response SOA ns.cobbaut.paul
3 0.014672	192.168.1.37	192.168.1.35	TCP	33713 > domain [SYN] Seq=0 Win=5840 Len=0 MS
4 0.015215	192.168.1.35	192.168.1.37	TCP	domain > 33713 [SYN, ACK] Seq=0 Ack=1 Win=57
5 0.015307	192.168.1.37	192.168.1.35	TCP	33713 > domain [ACK] Seq=1 Ack=1 Win=5856 Le
6 0.015954	192.168.1.37	192.168.1.35	TCP	[TCP segment of a reassembled PDU]
7 0.018359	192.168.1.35	192.168.1.37	TCP	domain > 33713 [ACK] Seq=1 Ack=3 Win=5792 Le
8 0.018411	192.168.1.37	192.168.1.35	DNS	Standard query IXFR cobbaut.paul
9 0.018823	192.168.1.35	192.168.1.37	TCP	domain > 33713 [ACK] Seq=1 Ack=77 Win=5792 L
10 0.019784	192.168.1.35	192.168.1.37	DNS	Standard query response SOA ns.cobbaut.paul
11 0.019821	192.168.1.37	192.168.1.35	TCP	33713 > domain [ACK] Seq=77 Ack=295 Win=6912
12 0.020618	192.168.1.37	192.168.1.35	TCP	33713 > domain [FIN, ACK] Seq=77 Ack=295 Win
13 0.021011	192.168.1.35	192.168.1.37	TCP	domain > 33713 [FIN, ACK] Seq=295 Ack=78 Win
14 0.021040	192.168.1.37	192.168.1.35	TCP	33713 > domain [ACK] Seq=78 Ack=296 Win=6912

69.9. full or incremental zone transfers

When a zone transfer occurs, this can be either a full zone transfer or an incremental zone transfer. The decision depends on the size of the transfer that is needed to completely update the zone on the slave server. An incremental zone transfer is preferred when the total size of changes is smaller than the size of the zone database. Full zone transfers use the **axfr** protocol, incremental zone transfer use the **ixfr** protocol.

69.10. DNS cache

DNS is a caching protocol.

When a client queries its local DNS server, and the local DNS server is not authoritative for the query, then this server will go looking for an authoritative name server in the DNS tree. The local name server will first query a root server, then a **tld** server and then a domain server. When the local name server resolves the query, then it will relay this information to the client that submitted the query, and it will also keep a copy of these queries in its cache. So when a(nother) client submits the same query to this name server, then it will retrieve this information from its cache.

For example, a client queries for the A record on `www.linux-training.be` to its local server. This is the first query ever received by this local server. The local server checks that it is not authoritative for the `linux-training.be` domain, nor for the **.be tld**, and it is also not a root server. So the local server will use the root hints to send an **iterative** query to a root server.

The root server will reply with a reference to the server that is authoritative for the `.be` domain (root DNS servers do not resolve fqdn's, and root servers do not respond to recursive queries).

The local server will then send an iterative query to the authoritative server for the **.be tld**. This server will respond with a reference to the name server that is authoritative for the `linux-training.be` domain.

The local server will then send the query for `www.linux-training.be` to the authoritative server (or one of its slave servers) for the `linux-training.be` domain. When the local server receives the ip address for `www.linux-training.be`, then it will provide this information to the client that submitted this query.

Besides caching the A record for `www.linux-training.be`, the local server will also cache the NS and A record for the `linux-training.be` name server and the `.be` name server.

69.11. forward lookup zone example

The way to set up zones in `/etc/bind/named.conf.local` is to create a zone entry with a reference to another file (this other file contains the **zone database**).

Here is an example of such an entry in `/etc/bind/named.conf.local`:

```
root@debian7:~# cat /etc/bind/named.conf.local
//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "paul.local" IN {
    type master;
    file "/etc/bind/db.paul.local";
    allow-update { none; };
};
root@debian7:~#
```

To create the zone file, the easy method is to copy an existing zone file (this is easier than writing from scratch).

```
root@debian7:/etc/bind# cp db.empty db.paul.local
root@debian7:/etc/bind# vi db.paul.local
```

Here is an example of a zone file.

```
root@debian7:/etc/bind# cat db.paul.local
; zone for classroom teaching
$TTL      86400
@         IN      SOA      debianpaul.paul.local. root.paul.local (
                                2014100100      ; Serial
                                1h                ; Refresh
                                1h                ; Retry
                                2h                ; Expire
                                86400 )          ; Negative Cache TTL
;
; name servers
;
    IN      NS      ns1
    IN      NS      debianpaul
    IN      NS      debian7
;
; servers
;
debianpaul    IN      A      10.104.33.30
debian7       IN      A      10.104.33.30
ns1           IN      A      10.104.33.30
;www          IN      A      10.104.33.30
```

69.12. example: caching only DNS server

1. installing DNS software on Debian

```
root@debian7:~# aptitude update && aptitude upgrade
...
root@debian7:~# aptitude install bind9
...
root@debian7:~# dpkg -l | grep bind9 | tr -s ' '
ii bind9 1:9.8.4.dfsg.P1-6+nmu2+deb7u2 amd64 Internet Domain Name Server
ii bind9-host 1:9.8.4.dfsg.P1-6+nmu2+deb7u2 amd64 Version of 'host' bundled...
ii bind9utils 1:9.8.4.dfsg.P1-6+nmu2+deb7u2 amd64 Utilities for BIND
ii libbind9-80 1:9.8.4.dfsg.P1-6+nmu2+deb7u2 amd64 BIND9 Shared Library use...
root@debian7:~#
```

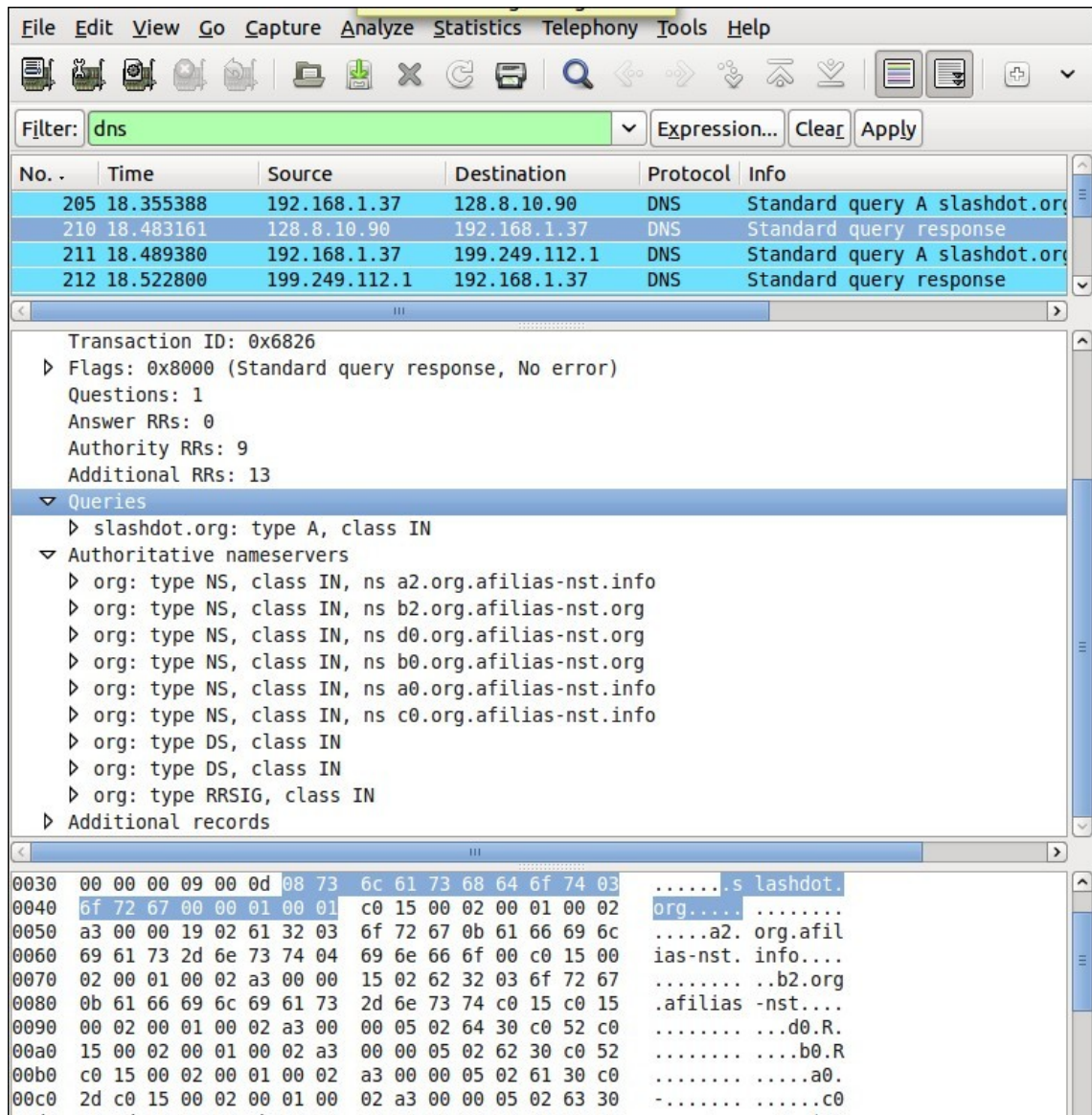
2. Discover the default configuration files. Can you define the purpose of each file ?

```
root@debian7:~# ls -l /etc/bind
total 52
-rw-r--r-- 1 root root 2389 Sep  5 20:25 bind.keys
-rw-r--r-- 1 root root 237 Sep  5 20:25 db.0
-rw-r--r-- 1 root root 271 Sep  5 20:25 db.127
-rw-r--r-- 1 root root 237 Sep  5 20:25 db.255
-rw-r--r-- 1 root root 353 Sep  5 20:25 db.empty
-rw-r--r-- 1 root root 270 Sep  5 20:25 db.local
-rw-r--r-- 1 root root 3048 Sep  5 20:25 db.root
-rw-r--r-- 1 root bind 463 Sep  5 20:25 named.conf
-rw-r--r-- 1 root bind 490 Sep  5 20:25 named.conf.default-zones
-rw-r--r-- 1 root bind 374 Oct  1 20:01 named.conf.local
-rw-r--r-- 1 root bind 913 Oct  1 13:24 named.conf.options
-rw-r----- 1 bind bind 77 Oct  1 11:14 rndc.key
-rw-r--r-- 1 root root 1317 Sep  5 20:25 zones.rfc191
```

3. Setup caching only dns server. This is normally the default setup. A caching-only name server will look up names for you and cache them. Many tutorials will tell you to add a **forwarder**, but we first try without this!

Hey this seems to work without a **forwarder**. Using a sniffer you can find out what really happens. Your freshly install dns server is not using a cache, and it is not using your local dns server (from /etc/resolv.conf). So where is this information coming from ? And what can you learn from sniffing this dns traffic ?

4. Explain in detail what happens when you enable a caching only dns server without forwarder. This wireshark screenshot can help, but you learn more by sniffing the traffic yourself.



You should see traffic to a **root name server** whenever you try a new **tld** for the first time. Remember that **dns** is a caching protocol, which means that repeating a query will generate a lot less traffic since your **dns server** will still have the answer in its memory.

69.13. example: caching only with forwarder

5. Add the public Google **dns server** as a **forwarder**. The ip address of this server is 8.8.8.8 .

Before the change:

```
root@debian7:~# grep -A2 'forwarders {' /etc/bind/named.conf.options
// forwarders {
//     0.0.0.0;
// };
```

changing:

```
root@debian7:~# vi /etc/bind/named.conf.options
```

After the change:

```
root@debian7:~# grep -A2 'forwarders {' /etc/bind/named.conf.options
forwarders {
    8.8.8.8;
};
```

Restart the server:

```
root@debian7:~# service bind9 restart
Stopping domain name service...: bind9.
Starting domain name service...: bind9.
```

6. Explain the purpose of adding the **forwarder**. What is our **dns server** doing when it receives a query ?

```
root@debian7:~# nslookup
> server
Default server: 10.104.33.30
Address: 10.104.33.30#53
> linux-training.be
Server:      10.104.33.30
Address:     10.104.33.30#53

Non-authoritative answer:
Name:   linux-training.be
Address: 188.93.155.87
>
```

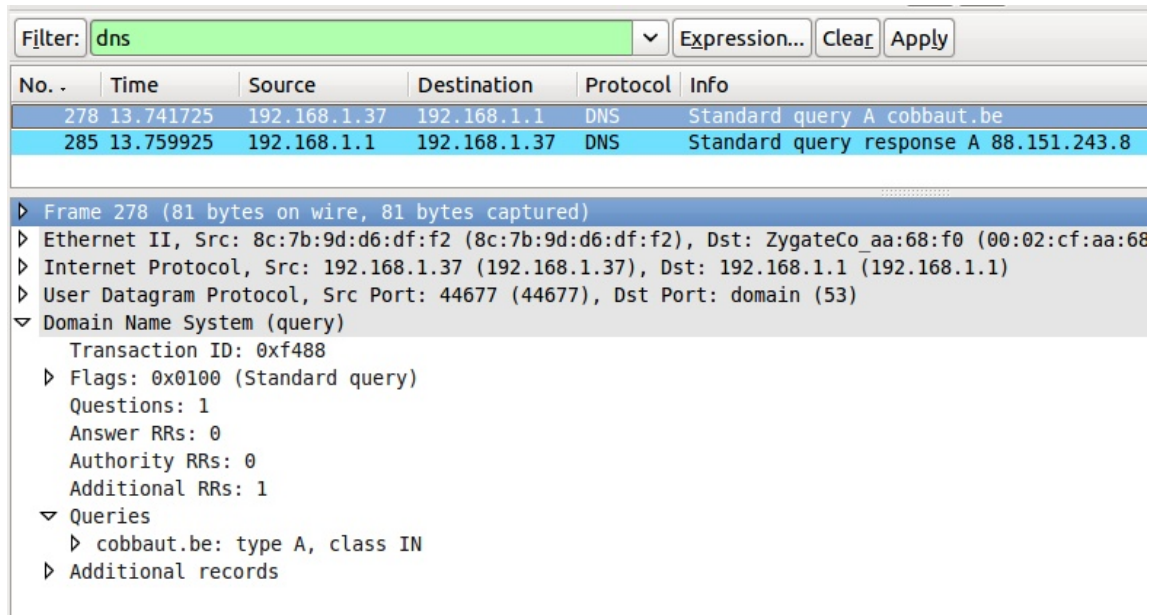
This is the output of **tcpdump udp port 53** while executing the above query for **linux-training.be** in **nslookup**.

```
root@debian7:~# tcpdump udp port 53
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 65535 bytes
```

You should find the following two lines in the output of **tcpdump**:

```
10.104.33.30.19381 > google-public-dns-a.google.com.domain: 18237+% [1au] A? \
linux-training.be. (46)
google-public-dns-a.google.com.domain > 10.104.33.30.19381: 18237 1/0/1 A 188\
.93.155.87 (62)
```

Below is an (old) wireshark screenshot that can help, you should see something similar (but with different ip addresses).



7. What happens when you query for the same domain name more than once ?

8. Why does it say "non-authoritative answer" ? When is a dns server authoritative ?

9. You can also use **dig** instead of **nslookup**.

```
root@debian7:~# dig @10.104.33.30 linux-training.be +short
188.93.155.87
root@debian7:~#
```

10. How can we avoid having to set the server in dig or nslookup ?

Change this:

```
root@debian7:~# cat /etc/resolv.conf
nameserver 10.46.101.1
root@debian7:~#
```

into this:

```
root@debian7:~# cat /etc/resolv.conf
nameserver 10.104.33.30
root@debian7:~#
```

11. When you use **dig** for the first time for a domain, where is the answer coming from ? And the second time ? How can you tell ?

69.14. example: primary authoritative server

1. Instead of only caching the information from other servers, we will now make our server authoritative for our own domain.

2. I choose the top level domain **.local** and the domain **paul.local** and put the information in **/etc/bind/named.conf.local**.

```
root@debian7:~# cat /etc/bind/named.conf.local
//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "paul.local" IN {
    type master;
    file "/etc/bind/db.paul.local";
    allow-update { none; };
};
```

3. Also add a **zone database file**, similar to this one (add some A records for testing). Set the **Refresh** and **Retry** values not too high so you can sniff this traffic (this example makes the slave server contact the master every hour).

```
root@debian7:~# cat /etc/bind/db.paul.local
; zone for classroom teaching
$TTL      86400
@         IN      SOA      debianpaul.paul.local. root.paul.local (
                                2014100101      ; Serial
                                1h                ; Refresh
                                1h                ; Retry
                                2h                ; Expire
                                900 )            ; Negative Cache TTL
;
; name servers
;
    IN      NS      ns1
    IN      NS      debianpaul
    IN      NS      debian7
;
; servers
;
debianpaul    IN      A      10.104.33.30
debian7       IN      A      10.104.33.30
ns1           IN      A      10.104.33.30
;www          IN      A      10.104.33.30
root@debian7:~#
```

Note that the **www** record is commented out, so it will not resolve.

69.14.1. using your own DNS server

If you are confident that your **dns server** works, then set it as default and only dns server in **/etc/resolv.conf**.

```
root@debian7:~# cat /etc/resolv.conf
nameserver 10.104.33.30
root@debian7:~#
```

In case you also use **dhclient**, you will need to add your dns server to **/etc/dhcp/dhclient.conf**.

```
root@debian7:~# diff /etc/dhcp/dhclient.conf /etc/dhcp/dhclient.conf.original
21c21
< prepend domain-name-servers 10.104.33.30;
---
> #prepend domain-name-servers 127.0.0.1;
23,24c23
< #      domain-name, domain-name-servers, domain-search, host-name,
<      domain-name, domain-search, host-name,
---
>      domain-name, domain-name-servers, domain-search, host-name,
root@debian7:~#
```

The above screenshot shows that 10.104.33.30 is now a default option that the **dhcp client** should no longer request from the **dhcp server**.

Adjust **/etc/hosts** to reflect your **domain name** and verify with **hostname** and **dnsdomainname**.

```
root@debian7:~# grep debian7 /etc/hosts
127.0.1.1 debian7.paul.local debian7
root@debian7:~# hostname
debian7
root@debian7:~# hostname --fqdn
debian7.paul.local
root@debian7:~# dnsdomainname
paul.local
```


69.14.2. using your own domain

Consider the following screenshot:

```
root@debian7b:~# cat /etc/resolv.conf
nameserver 10.104.33.30
root@debian7b:~# ping -c1 www
ping: unknown host www
root@debian7b:~# vi /etc/resolv.conf
root@debian7b:~# cat /etc/resolv.conf
nameserver 10.104.33.30
domain paul.local
root@debian7b:~# ping -c1 www
PING www.paul.local (10.104.33.31) 56(84) bytes of data.
64 bytes from 10.104.33.31: icmp_req=1 ttl=64 time=0.021 ms

--- www.paul.local ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.021/0.021/0.021/0.000 ms
root@debian7b:~#
```

Adding the **domain paul.local** directive to **/etc/resolv.conf** allows omitting the domain when using hostnames.

You can accomplish this feature automatically by adjusting **dhclient.conf**.

```
root@debian7:~# grep paul.local /etc/dhcp/dhclient.conf
prepend domain-name "paul.local";
prepend domain-search "paul.local";
root@debian7:~#
```

4. Restart the DNS server and check your zone in the error log.

```
root@debian7:~# service bind9 restart
Stopping domain name service...: bind9.
Starting domain name service...: bind9.
root@debian7:~# grep paul.local /var/log/syslog
Oct  6 09:22:18 debian7 named[2707]: zone paul.local/IN: loaded serial 2014100101
Oct  6 09:22:18 debian7 named[2707]: zone paul.local/IN: sending notification (serial 2014100101)
```

5. Use **dig** or **nslookup** (or even **ping**) to test your A records.

```
root@debian7:~# ping -c1 ns1.paul.local
PING ns1.paul.local (10.104.33.30) 56(84) bytes of data.
64 bytes from 10.104.33.30: icmp_req=1 ttl=64 time=0.006 ms

--- ns1.paul.local ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.006/0.006/0.006/0.000 ms
root@debian7:~# ping -c1 www.paul.local
ping: unknown host www.paul.local
```

Note that the **www** record was commented out, so it should fail.

```
root@debian7:~# dig debian7.paul.local

; <<>> DiG 9.8.4-rpz2+rl005.12-P1 <<>> debian7.paul.local
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 50491
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 3, ADDITIONAL: 2

;; QUESTION SECTION:
;debian7.paul.local.          IN      A

;; ANSWER SECTION:
debian7.paul.local.         86400   IN      A      10.104.33.30

;; AUTHORITY SECTION:
paul.local.                 86400   IN      NS      ns1.paul.local.
paul.local.                 86400   IN      NS      debian7.paul.local.
paul.local.                 86400   IN      NS      debianpaul.paul.local.

;; ADDITIONAL SECTION:
ns1.paul.local.             86400   IN      A      10.104.33.30
debianpaul.paul.local.      86400   IN      A      10.104.33.30

;; Query time: 4 msec
;; SERVER: 10.104.33.30#53(10.104.33.30)
;; WHEN: Mon Oct  6 09:35:25 2014
;; MSG SIZE rcvd: 141

root@debian7:~#
```

6. Our primary server appears to be up and running. Note the information here:

```
server os   : Debian 7
ip address  : 10.104.33.30
domain name: paul.local
server name: ns1.paul.local
```

69.15. example: a DNS slave server

1. A slave server transfers zone information over the network from a master server (a slave can also be a master). A primary server maintains zone records in its local file system. As an exercise, and to verify the work of all students, set up a slave server of all the master servers in the classroom.

2. Before configuring the slave server, we may have to allow transfers from our zone to this server. Remember that this is not very secure since transfers are in clear text and limited to an ip address. This example follows our demo from above.

Imagine a student named **Jesse** having completed the setup as shown before, with the domain name **jesse.local** and the ip address 10.104.15.20. The goal is to have a slave server of paul.local on Jesse's computer and a slave zone of jesse.local on my computer.

Below is an example of an **allow-transfer** statement. Careful, maybe the default allows transfer to any.

```
root@debian7:/etc/bind# cat named.conf.local
//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "paul.local" IN {
    type master;
    file "/etc/bind/db.paul.local";
    allow-update { none; };
    allow-transfer { 10.104.15.20; };
};
```

3. With the configuration below I can make my server a slave for the **jesse.local** zone.

```
root@debian7:/etc/bind# tail -6 named.conf.local
zone "jesse.local" IN {
    type slave;
    file "/var/cache/named/db.jesse.local";
    masters { 10.104.15.20; };
};

root@debian7:/etc/bind# mkdir /var/cache/named/
root@debian7:/etc/bind# chown bind:bind /var/cache/named/
root@debian7:/etc/bind# ls -ld /var/cache/named/
drwxr-xr-x 2 bind bind 4096 Oct  1 20:01 /var/cache/named/
```

Note that we put the **slave zones** in **/var/cache/named** and not in **/etc/bind**.

4. Restarting bind on the slave server should transfer the zone database file. Verify this in **/var/log/syslog**. (time and date are truncated from the screenshot, and Jesse did not use the current date in the serial number...)

```
root@debian7:/etc/bind# grep jesse /var/log/syslog
named[2731]: zone jesse.local/IN: Transfer started.
named[2731]: transfer of 'jesse.local/IN' from 10.104.15.20#53: connected u\
sing 10.104.33.30#44719
named[2731]: zone jesse.local/IN: transferred serial 20110516
named[2731]: transfer of 'jesse.local/IN' from 10.104.15.20#53: Transfer co\
mpleted: 1 messages, 8 records, 239 bytes, 0.001 secs (239000 bytes/sec)
```

And the contents of the **slave zone**:

```
root@debian7:/etc/bind# cat /var/cache/named/db.jesse.local
$ORIGIN .
$TTL 604800      ; 1 week
jesse.local      IN SOA  ns.jesse.local. root.jesse.local.jesse.local. (
                    20110516      ; serial
                    300           ; refresh (5 minutes)
                    200           ; retry (3 minutes 20 seconds)
                    2419200        ; expire (4 weeks)
                    604800         ; minimum (1 week)
                    )
                    NS      ns.jesse.local.
$ORIGIN jesse.local.
anya             A       10.104.15.1
mac              A       10.104.15.30
ns               A       10.104.15.20
ubu1010srv       A       10.104.15.20
www              A       10.104.15.25
root@debian7:/etc/bind#
```

69.16. practice: dns

1. Install **bind9** and verify with a sniffer how it works.
2. Add a **forwarder** and verify that it works.
3. Create a **primary forward lookup zone** named yourname.local with at least two NS records and four A records.
4. Use **dig** and **nslookup** to verify your NS and A records.
5. Create a **slave** of your primary zone (on another server) and verify the **zone transfer**.
6. Set up two primary zones on two servers and implement a **conditional forwarder** (you can use the two servers from before).

69.17. solution: dns

1. Install **bind9** and verify with a sniffer how it works.

You should see queries to the root name servers with **tcpdump** or **wireshark**.

2. Add a **forwarder** and verify that it works.

The forwarder can be added in `named.conf.options` as seen in the theory.

3. Create a **primary forward lookup zone** named `yourname.local` with at least two NS records and four A records.

This is literally explained in the theory.

4. Use **dig** and **nslookup** to verify your NS and A records.

This is literally explained in the theory.

5. Create a **slave** of your primary zone (on another server) and verify the **zone transfer**.

This is literally explained in the theory.

6. Set up two primary zones on two servers and implement a **conditional forwarder** (you can use the two servers from before).

A conditional forwarder is set in `named.conf.local` as a zone.
(see the theory on forwarder)

Chapter 70. advanced DNS

This chapter expands your DNS server with topics like **round robin dns** for load balancing servers, **dns delegation** to delegate child domains to another team and **split horizon dns** so you can provide local service locations to clients.

There is more to **dns**, content will be added **rsn**.

70.1. example: DNS round robin

When you create multiple A records for the same name, then **bind** will do a **round robin** of the order in which the records are returned. This allows the use of DNS as a load balancer between hosts, since clients will usually take the first ip-address offered.

Consider this example from the `/etc/bind/db.paul.local` zone configuration file. There are two A records for **www** pointing to two distinct ip addresses.

```
root@debian7:~# grep www /etc/bind/db.paul.local
www          IN      A      10.104.33.30
www          IN      A      10.104.33.31
```

Below a screenshot of **nslookup** querying a load balanced A record. Notice the order of ip addresses returned.

```
root@debian7:~# nslookup www.paul.local 10.104.33.30
Server:          10.104.33.30
Address:         10.104.33.30#53

Name:   www.paul.local
Address: 10.104.33.31
Name:   www.paul.local
Address: 10.104.33.30

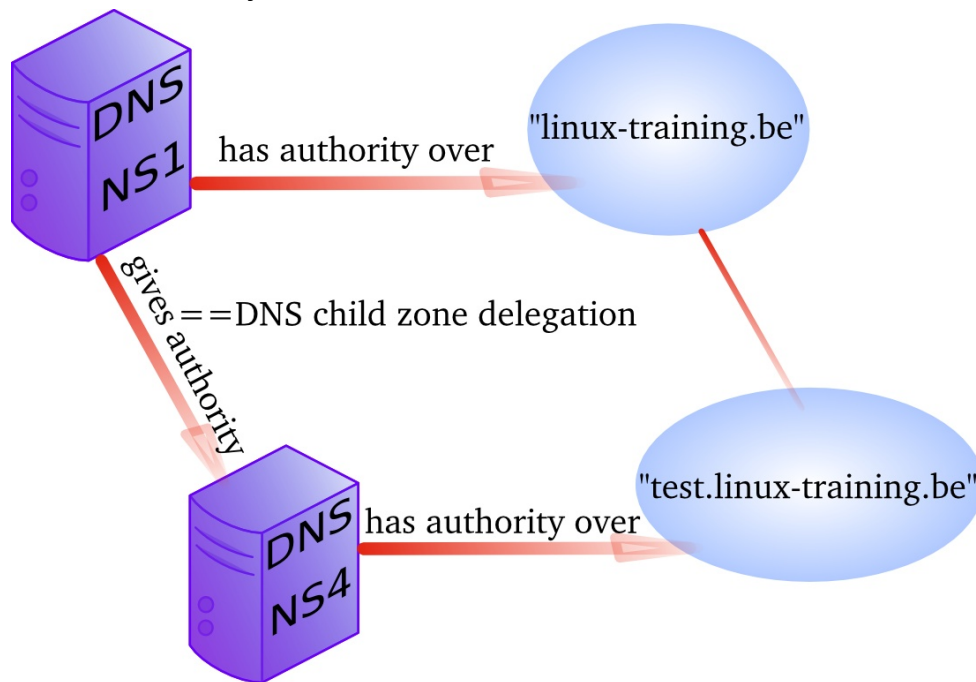
root@debian7:~# nslookup www.paul.local 10.104.33.30
Server:          10.104.33.30
Address:         10.104.33.30#53

Name:   www.paul.local
Address: 10.104.33.30
Name:   www.paul.local
Address: 10.104.33.31
```

Try to set up a website on two web servers (with a small difference so you can distinguish the websites) and test the **round robin**.

70.2. DNS delegation

You can **delegate** a child domain to another DNS server. The child domain then becomes a new zone, with authority at the new dns server.



When **delegation** is properly set up, then clients that query your parent zone will also be able to resolve the delegated child zones.

70.3. example: DNS delegation

We have another **Linux server** named **debian7b** and we want to make it responsible for the child domain **test42.paul.local**.

*Note the name of the servers in the screenshots are either **debian7** (hosting the parent domain) or **debian7b** (hosting the child domain).*

We start by adjusting the **/etc/bind/named.conf.local** file (on the server hosting the parent domain) to make sure that no forwarder will be used when resolving authoritative names.

```
root@debian7:~# grep -A4 paul.local /etc/bind/named.conf.local
zone "paul.local" IN {
    type master;
    file "/etc/bind/db.paul.local";
    allow-update { none; };
    allow-transfer { 10.104.15.20; };
    forwarders { };
};
root@debian7:~#
```

Technically, you could also set **allow-transfer** to **{ any; };** while troubleshooting and then refine it later, but this is not needed for delegation.

Then we add the delegation to our zone database:

```
root@debian7:~# tail -3 /etc/bind/db.paul.local
$ORIGIN test42.paul.local.
@      IN      NS      ns2.test42.paul.local.
ns2    IN      A       10.104.33.31    ; the glue record
root@debian7:~#
```

Don't forget to restart **bind** and verify **/var/log/syslog**.

```
root@debian7:~# service bind9 restart
Stopping domain name service...: bind9.
Starting domain name service...: bind9.
root@debian7:~# grep paul.local /var/log/syslog | cut -c28- | tail -2
named[3202]: zone paul.local/IN: loaded serial 2014100801
named[3202]: zone paul.local/IN: sending notifies (serial 2014100801)
root@debian7:~#
```

*Note that on your terminal you can type **tail -40 /var/log/syslog** because the only reason I use **grep**, **cut** and **tail -2** is to limit the size of the screenshots in this book.*

Next we create a zone database file on the second server, as seen in this screenshot:

```
root@debian7b:~# cat /etc/bind/db.test42.paul.local
; child zone for classroom teaching
$TTL      86400
$ORIGIN   test42.paul.local.
@         IN      SOA      ns2.test42.paul.local. root.test42.paul.local. (
                                2014100802      ; Serial
                                1h                ; Refresh
                                1h                ; Retry
                                2h                ; Expire
                                900 )             ; Negative Cache TTL
;
; name servers
;
;         IN      NS       ns2.test42.paul.local.
;         IN      NS       debian7b.test42.paul.local.
;
; servers
;
ns2         IN      A       10.104.33.31
debian7b    IN      A       10.104.33.31
testsrv     IN      A       10.104.33.31
root@debian7b:~#
```

The second server also needs a zone definition in **named.conf.local**, followed by a restart of **bind**.

```
root@debian7b:~# cat /etc/bind/named.conf.local
//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "test42.paul.local" IN {
    type master;
    file "/etc/bind/db.test42.paul.local";
    allow-update { none; };
    allow-transfer { any; };
};
root@debian7b:~#
```

Testing on the parent server:

```
root@debian7:~# dig ns1.paul.local +short
10.104.33.30
root@debian7:~# dig ns2.test42.paul.local +short
10.104.33.31
root@debian7:~# dig debian7b.test42.paul.local +short
10.104.33.31
```

70.4. example: split-horizon dns

Suppose you want to answer dns queries depending on who is asking. For example when someone from the 10.104.15.0/24 network (managed by Jesse) asks for the A record `www.paul.local`, then dns answers with 10.104.33.30. But when someone from the 10.104.42.0/24 network (managed by Keith) asks for the same A record of `www.paul.local`, he will get 10.104.33.31 as an answer.

A **split-horizon** setup can be used to redirect people to **local** copies of certain services.

In this example we want to decide on specific answers for two networks (Jesse's and Keith's) and prevent them from using our dns server for **recursion**, while maintaining the capability to resolve the internet and our `paul.local` zone from our own network.

We start by creating three **view** clauses in `named.conf.local`.

```
root@debian7:/etc/bind# cat named.conf.local
view "paul" {
match-clients { 10.104.33.0; localhost; };
include "/etc/bind/named.conf.default-zones";
zone "paul.local" IN {
    type master;
    file "/etc/bind/db.paul.local";
    allow-update { none; };
};
// end view internal

view "jesse" {
match-clients { 10.104.15/24; };
zone "paul.local" IN {
    type master;
    file "/etc/bind/db.paul.local.jesse";
    allow-update { none; };
};
// end view jesse

view "keith" {
match-clients { 10.104.42/24; };
zone "paul.local" IN {
    type master;
    file "/etc/bind/db.paul.local.keith";
    allow-update { none; };
};
// end view keith
```

Note that we included the **default-zones** in the internal zone. It is mandatory to put all zones inside views when using a view.

The zone files are identical copies, except for the **www** record. You can see that the **round robin** is still active for internal users, computers from 10.104.15.0/24 (Jesse) will always receive 10.104.33.30 while computers from 10.104.42.0/24 (Keith) will receive 10.104.33.31.

```
root@debian7:/etc/bind# grep www db.paul.local db.paul.local.[jk]*
db.paul.local:www          IN      A      10.104.33.30
db.paul.local:www          IN      A      10.104.33.31
db.paul.local.jesse:www    IN      A      10.104.33.30
db.paul.local.keith:www    IN      A      10.104.33.31
```


70.5. old dns topics

All the dns things below this paragraph are old and in urgent need of review.

70.5.1. old example: reverse DNS

1. We can add ip to name resolution to our dns-server using a reverse dns zone.
2. Start by adding a .arpa zone to /etc/bind/named.conf.local like this (we set notify to no to avoid sending of notify messages to other name servers):

```
root@ubu1010srv:/etc/bind# grep -A4 arpa named.conf.local
zone "1.168.192.in-addr.arpa" {
    type master;
    notify no;
    file "/etc/bind/db.192";
};
```

3. Also create a zone database file for this reverse lookup zone.

```
root@ubu1010srv:/etc/bind# cat db.192
;
; BIND reverse data file for 192.168.1.0/24 network
;
$TTL 604800
@ IN SOA ns.cobbaut.paul root.cobbaut.paul. (
    20110516 ; Serial
    604800 ; Refresh
    86400 ; Retry
    2419200 ; Expire
    604800 ) ; Negative Cache TTL
;
@ IN NS ns.
37 IN PTR ns.cobbaut.paul.
1 IN PTR anya.cobbaut.paul.
30 IN PTR mac.cobbaut.paul.
root@ubu1010srv:/etc/bind#
```

4. Test with nslookup or dig:

```
root@ubu1010srv:/etc/bind# dig 1.168.192.in-addr.arpa AXFR
```

70.5.2. old DNS load balancing

Not as above. When you have more than one DNS server authoritative for a zone, you can spread queries amongst all server. One way to do this is by creating NS records for all servers that participate in the load balancing of external queries.

You could also configure different name servers on internal clients.

70.5.3. old DNS notify

The original design of DNS in rfc 1034 and rfc 1035 implemented a **refresh** time in the **SOA** record to configure a time loop for slaves to query their master server. This can result in a lot of useless pull requests, or in a significant lag between updates.

For this reason **dns notify (rfc 1996)** was designed. The server will now notify slaves whenever there is an update. By default this feature is activated in **bind**.

Notify can be disabled as in this screenshot.

```
zone "1.168.192.in-addr.arpa" {  
    type master;  
    notify no;  
    file "/etc/bind/db.192";  
};
```

70.5.4. old testing IXFR and AXFR

Full zone transfers (AXFR) are initiated when you restart the bind server, or when you manually update the zone database file directly. With **nsupdate** you can update a zone database and initiate an incremental zone transfer.

You need DDNS allowed for **nsupdate** to work.

```
root@ubu1010srv:/etc/bind# nsupdate  
> server 127.0.0.1  
> update add mac14.linux-training.be 86400 A 192.168.1.23  
> send  
update failed: REFUSED
```

70.5.5. old DDNS integration with DHCP

Some organizations like to have all their client computers in DNS. This can be cumbersome to maintain. Luckily **rfc 2136** describes integration of DHCP servers with a DNS server. Whenever DHCP acknowledges a client ip configuration, it can notify DNS with this clients ip-address and name. This is called **dynamic updates** or DDNS.

70.5.6. old reverse is forward in-addr.arpa

Reverse lookup is actually implemented as a forward lookup in the **in-addr.arpa** domain. This domain has 256 child domains (from 0.in-addr.arpa to 255.in-addr.arpa), with each child domain having again 256 child domains. And this twice more to a structure of over four billion (2 to the power 32) domains.

70.5.7. old ipv6

With rfc 3596 came ipv6 extensions for DNS. There is the AAAA record for ipv6 hosts on the network, and there is the **ip6.int** domain for reverse lookup (having 16 child domains from 0.ip6.int to f.ip6.int, each of those having again 16 child domains...and this 16 times.

70.5.8. old DNS security: file corruption

To mitigate file corruption on the **zone files** and the **bind configuration** files protect them with Unix permissions and take regular backups.

70.5.9. old DNS security: zone transfers

Limit zone transfers to certain ip addresses instead of to **any**. Nevermind that ip-addresses can be spoofed, still use this.

70.5.10. old DNS security: zone transfers, ip spoofing

You could setup DNSSEC (which is not the easiest to maintain) and with rfc 2845(tsig?) and with rfc 2930(tkey, but this is open to brute force), or you could disable all zone transfers and use a script with ssh to copy them manually.

70.5.11. old DNS security: queries

Allow recursion only from the local network, and iterative queries from outside only when necessary. This can be configured on master and slave servers.

```
view "internal" {  
  match-clients { 192.168.42/24; };  
  recursion yes;  
  ...  
};  
  
view "external" {  
  match-clients { any; };  
  recursion no;  
  ...  
};
```

Or allow only queries from the local network.

```
options {  
  allow-query { 192.168.42.0/24; localhost; };  
};  
  
zone "cobbaut.paul" {  
  allow-query { any; };  
};
```

Or only allow recursive queries from internal clients.

```
options {  
  allow-recursion { 192.168.42.0/24; localhost; };  
};
```



```
};
```

70.5.12. old DNS security: chrooted bind

Most Linux distributions allow an easy setup of bind in a **chrooted** environment.

70.5.13. old DNS security: DNSSEC

DNSSEC uses public/private keys to secure communications, this is described in rfc's 4033, 4034 and 4035.

70.5.14. old DNS security: root

Do not run bind as root. Do not run any application daemon as root.

Part XX. dhcp server

Table of Contents

71. introduction to dhcp	788
71.1. four broadcasts	789
71.2. picturing dhcp	790
71.3. installing a dhcp server	791
71.4. dhcp server on Red Hat	791
71.5. dhcp options	791
71.6. client reservations	791
71.7. example config files	792
71.8. older example config files	792
71.9. advanced dhcp	794
71.10. Practice: dhcp	795

Chapter 71. introduction to dhcp

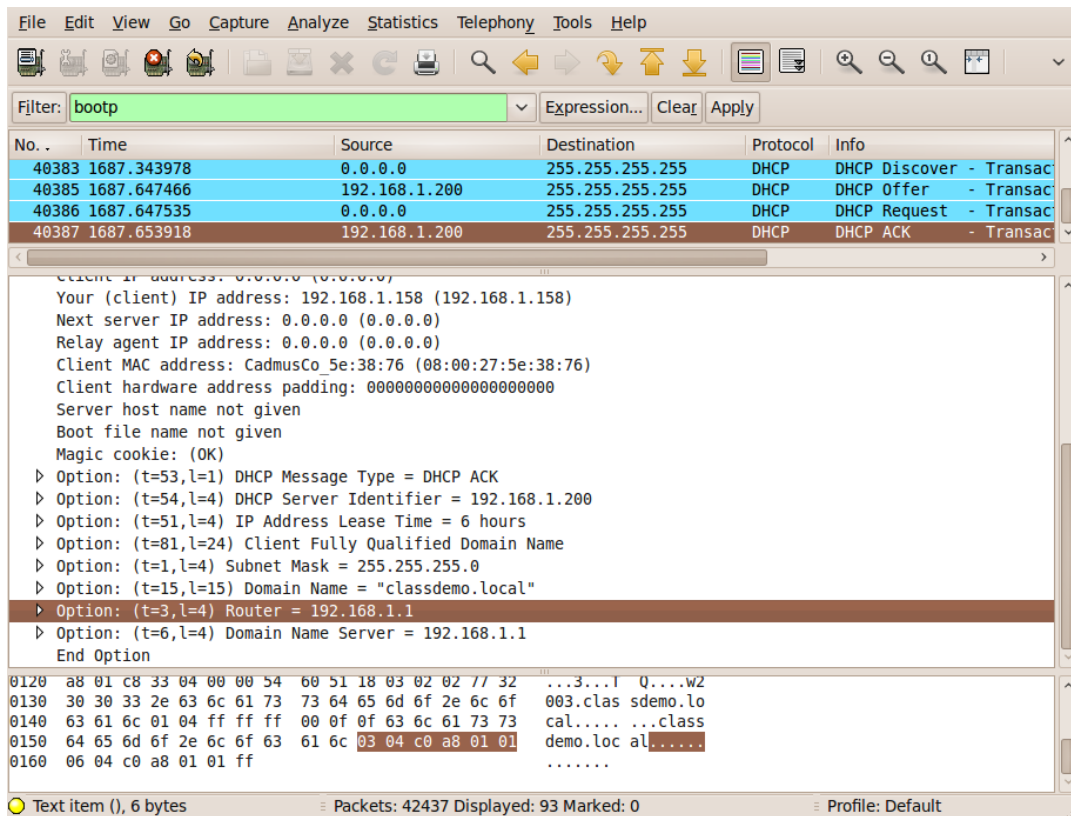
Dynamic Host Configuration Protocol (or short **dhcp**) is a standard tcp/ip protocol that distributes ip configurations to clients. **dhcp** is defined in **rfc 2131** (before that it was defined as an update to **bootp** in rfc 1531/1541).

The alternative to **dhcp** is manually entering the ip configuration on each client computer.

71.1. four broadcasts

dhcp works with layer 2 broadcasts. A **dhcp** client that starts, will send a **dhcp discover** on the network. All **dhcp servers** (that have a lease available) will respond with a **dhcp offer**. The client will choose one of those offers and will send a **dhcp request** containing the chosen offer. The **dhcp server** usually responds with a **dhcp ack**(knowledge).

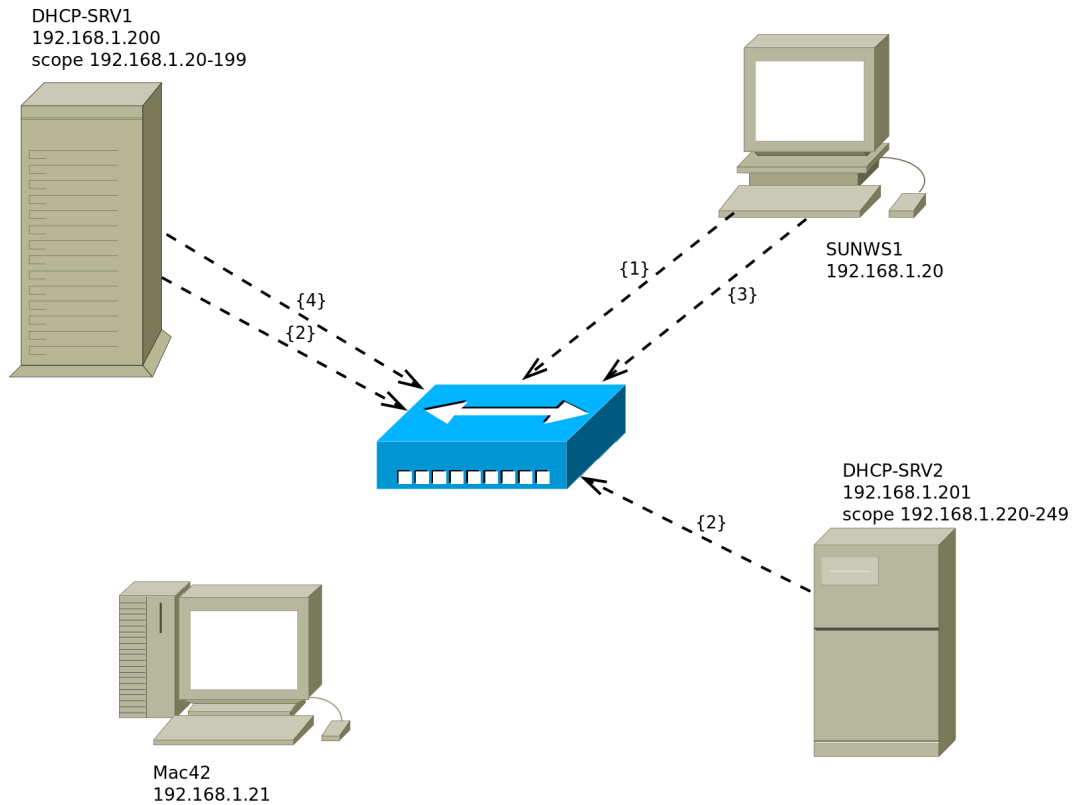
In wireshark it looks like this.



When this procedure is finished, then the client is allowed to use that ip-configuration until the end of its lease time.

71.2. picturing dhcp

Here we have a small network with two **dhcp servers** named DHCP-SRV1 and DHCP-SRV2 and two clients (SunWS1 and Mac42). All computers are connected by a hub or switch (pictured in the middle). All four computers have a cable to the hub (cables not pictured).



1. The client SunWS1 sends a **dhcp discover** on the network. All computers receive this broadcast.
2. Both **dhcp servers** answer with a **dhcp offer**. DHCP-SRV1 is a **dedicated dhcp server** and is faster in sending a **dhcp offer** than DHCP-SRV2 (who happens to also be a file server).
3. The client chooses the offer from DHCP-SRV1 and sends a **dhcp request** on the network.
4. DHCP-SRV1 answers with a **dhcp ack** (short for acknowledge).

All four broadcasts (or five when you count both offers) can be layer 2 ethernet broadcast to mac address **ff:ff:ff:ff:ff:ff** and a layer 3 ip broadcast to 255.255.255.255.

The same story can be read in **rfc 2131**.

71.3. installing a dhcp server

On Debian/Ubuntu

```
debian5:~# aptitude install dhcp3-server
Reading package lists... Done
Building dependency tree
Reading state information... Done
Reading extended state information
Initializing package states... Done
Reading task descriptions... Done
The following NEW packages will be installed:
  dhcp3-server
```

You get a configuration file with many examples.

```
debian5:~# ls -l /etc/dhcp3/dhcpd.conf
-rw-r--r-- 1 root root 3551 2011-04-10 21:23 /etc/dhcp3/dhcpd.conf
```

71.4. dhcp server on Red Hat

After installing we get a **/etc/dhcpd.conf** that points us to an example file named **dhcpd.conf.sample**.

```
[root@localhost ~]# cat /etc/dhcpd.conf
#
# DHCP Server Configuration file.
#   see /usr/share/doc/dhcp*/dhcpd.conf.sample
```

So we copy the sample and adjust it for our real situation. We name the copy **/etc/dhcpd.conf**.

```
subnet 192.168.1.0 netmask 255.255.255.0 {
  range 192.168.1.140 192.168.1.159
  option routers          192.168.1.1;
  option subnet-mask      255.255.255.0;
  option domain-name      "classdemo.local";
  option domain-name-servers 192.168.1.1;
  default-lease-time      21600;
}
```

71.5. dhcp options

Options can be set on the global, scope, client-reservation level.

```
option subnet-mask 255.255.255.0;
option domain-name "linux-training.be";
option domain-name-servers "ns1.openminds.be";
option routers 192.168.42.1;
```

71.6. client reservations

You can reserve an ip configuration for a client using the mac address.

```
host pc42 {
  hardware ethernet 11:22:33:44:55:66;
  fixed-address 192.168.42.42;
```

```
}
```

You can add individual options to this reservation.

```
host pc42 {  
  hardware ethernet 11:22:33:44:55:66;  
  fixed-address 192.168.42.42;  
  option domain-name "linux-training.be";  
  option routers 192.168.42.1;  
}
```

71.7. example config files

Below you see several sections of `/etc/dhcp/dhcpd.conf` on a **Debian 6** server.

```
# NetSec Antwerp Network  
subnet 192.168.1.0 netmask 255.255.255.0 {  
  range 192.168.1.20 192.168.1.199;  
  option domain-name-servers ns1.netsec.local;  
  option domain-name "netsec.local";  
  option routers 192.168.1.1;  
  option broadcast-address 192.168.1.255;  
  default-lease-time 7200;  
  max-lease-time 7200;  
}
```

Above the general configuration for the network, with a pool of 180 addresses.

Below two client reservations:

```
#  
# laptops  
#  
  
host mac {  
  hardware ethernet 00:26:bb:xx:xx:xx;  
  fixed-address mac.netsec.local;  
}  
  
host vmac {  
  hardware ethernet 8c:7b:9d:xx:xx:xx;  
  fixed-address vmac.netsec.local;  
}
```

71.8. older example config files

For `dhcpd.conf` on Fedora with dynamic updates for a DNS domain.

```
[root@fedora14 ~]# cat /etc/dhcp/dhcpd.conf  
authoritative;  
include "/etc/rndc.key";  
  
log-facility local6;  
  
server-identifier fedora14;  
ddns-domainname "office.linux-training.be";  
ddns-update-style interim;  
ddns-updates on;  
update-static-leases on;  
  
option domain-name "office.linux-training.be";
```



```
option domain-name-servers 192.168.42.100;
option ip-forwarding off;

default-lease-time 1800;
max-lease-time 3600;

zone office.linux-training.be {
    primary 192.168.42.100;
}

subnet 192.168.4.0 netmask 255.255.255.0 {
    range 192.168.4.24 192.168.4.40;
}
```

Allowing any updates in the zone database (part of the named.conf configuration)

```
zone "office.linux-training.be" {
    type master;
    file "/var/named/db.office.linux-training.be";
    allow-transfer { any; };
    allow-update { any; };
};
```

Allowing secure key updates in the zone database (part of the named.conf configuration)

```
zone "office.linux-training.be" {
    type master;
    file "/var/named/db.office.linux-training.be";
    allow-transfer { any; };
    allow-update { key mykey; };
};
```

Sample key file contents:

```
[root@fedora14 ~]# cat /etc/rndc.key
key "rndc-key" {
    algorithm hmac-md5;
    secret "4Ykd58uIeUr3Ve6ad1qTfQ==";
};
```

Generate your own keys with **dnssec-keygen**.

How to include a key in a config file:

```
include "/etc/bind/rndc.key";
```

Also make sure that **bind** can write to your db.zone file (using chmod/chown). For Ubuntu this can be in /etc/bind, for Fedora in /var/named.

71.9. advanced dhcp

71.9.1. 80/20 rule

DHCP servers should not be a single point of failure. Let us discuss redundant dhcp server setups.

71.9.2. relay agent

To avoid having to place a dhcp server on every segment, we can use **dhcp relay agents**.

71.9.3. rogue dhcp servers

Rogue dhcp servers are a problem without a solution. For example accidental connection of a (believed to be simple) hub/switch to a network with an internal dhcp server.

71.9.4. dhcp and ddns

DHCP can dynamically update DNS when it configures a client computer. DDNS can be used with or without secure keys.

When set up properly records can be added automaticall to the zone file:

```
root@fedora14~# tail -2 /var/named/db.office.linux-training.be
ubu1010srv      A      192.168.42.151
                TXT      "00dfbb15e144a273c3cf2d6ae933885782"
```

71.10. Practice: dhcp

1. Make sure you have a unique fixed ip address for your DNS and DHCP server (easier on the same machine).
2. Install DHCP and browse the explanation in the default configuration file `/etc/dhcp/dhcpd.conf` or `/etc/dhcp3/dhcpd.conf`.
3. Decide on a valid scope and activate it.
4. Test with a client that your DHCP server works.
5. Use wireshark to capture the four broadcasts when a client receives an ip (for the first time).
6. Use wireshark to capture a DHCPNAK and a DHCPRelease.
7. Reserve a configuration for a particular client (using mac address).
8. Configure your DHCP/DNS server(s) with a proper hostname and domainname (`/etc/hosts`, `/etc/hostname`, `/etc/sysconfig/network` on Fedora/RHEL, `/etc/resolv.conf` ...). You may need to disable NetworkManager on *buntu-desktops.
9. Make sure your DNS server still works, and is master over (at least) one domain.

There are several ways to do steps 10-11-12. Google is your friend in exploring DDNS with keys, with key-files or without keys.

10. Configure your DNS server to allow dynamic updates from your DHCP server.
11. Configure your DHCP server to send dynamic updates to your DNS server.
12. Test the working of Dynamic DNS.

Part XXI. iptables firewall

Table of Contents

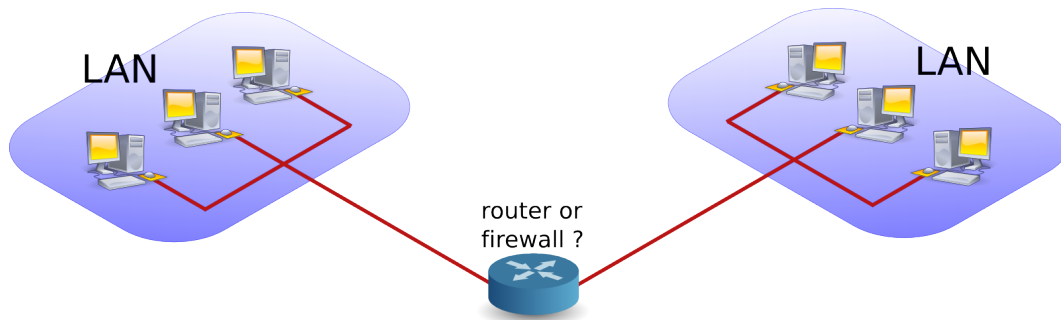
72. introduction to routers	798
72.1. router or firewall	799
72.2. packet forwarding	799
72.3. packet filtering	799
72.4. stateful	799
72.5. nat (network address translation)	800
72.6. pat (port address translation)	800
72.7. snat (source nat)	800
72.8. masquerading	800
72.9. dnat (destination nat)	800
72.10. port forwarding	800
72.11. /proc/sys/net/ipv4/ip_forward	801
72.12. /etc/sysctl.conf	801
72.13. sysctl	801
72.14. practice: packet forwarding	802
72.15. solution: packet forwarding	804
73. iptables firewall	807
73.1. iptables tables	808
73.2. starting and stopping iptables	808
73.3. the filter table	809
73.4. practice: packet filtering	814
73.5. solution: packet filtering	815
73.6. network address translation	816

Chapter 72. introduction to routers

What follows is a very brief introduction to using Linux as a router.

72.1. router or firewall

A **router** is a device that connects two networks. A **firewall** is a device that besides acting as a **router**, also contains (and implements) rules to determine whether packets are allowed to travel from one network to another. A firewall can be configured to block access based on networks, hosts, protocols and ports. Firewalls can also change the contents of packets while forwarding them.



72.2. packet forwarding

Packet forwarding means allowing packets to go from one network to another. When a multihomed host is connected to two different networks, and it allows packets to travel from one network to another through its two network interfaces, it is said to have enabled **packet forwarding**.

72.3. packet filtering

Packet filtering is very similar to packet forwarding, but every packet is individually tested against rules that decide on allowing or dropping the packet. The rules are stored by iptables.

72.4. stateful

A **stateful** firewall is an advancement over stateless firewalls that inspect every individual packet. A stateful firewall will keep a table of active connections, and is knowledgeable enough to recognise when new connections are part of an active session. Linux iptables is a stateful firewall.

72.5. nat (network address translation)

A **nat** device is a router that is also changing the source and/or target ip-address in packets. It is typically used to connect multiple computers in a private address range (rfc 1918) with the (public) internet. A **nat** can hide private addresses from the internet.

It is important to understand that people and vendors do not always use the right term when referring to a certain type of **nat**. Be sure you talk about the same thing. We can distinguish several types of **nat**.

72.6. pat (port address translation)

nat often includes **pat**. A **pat** device is a router that is also changing the source and/or target tcp/udp port in packets. **pat** is Cisco terminology and is used by **snat**, **dnat**, **masquerading** and **port forwarding** in Linux. RFC 3022 calls it **NAPT** and defines the **nat/pat** combo as "traditional nat". A device sold to you as a nat-device will probably do **nat** and **pat**.

72.7. snat (source nat)

A **snat** device is changing the source ip-address when a packet passes our **nat**. **snat** configuration with iptables includes a fixed target source address.

72.8. masquerading

Masquerading is a form of **snat** that will hide the (private) source ip-addresses of your private network using a public ip-address. Masquerading is common on dynamic internet interfaces (broadband modem/routers). Masquerade configuration with iptables uses a dynamic target source address.

72.9. dnat (destination nat)

A **dnat** device is changing the destination ip-address when a packet passes our **nat**.

72.10. port forwarding

When static **dnat** is set up in a way that allows outside connections to enter our private network, then we call it **port forwarding**.

72.11. /proc/sys/net/ipv4/ip_forward

Whether a host is forwarding packets is defined in **/proc/sys/net/ipv4/ip_forward**. The following screenshot shows how to enable packet forwarding on Linux.

```
root@router~# echo 1 > /proc/sys/net/ipv4/ip_forward
```

The next command shows how to disable packet forwarding.

```
root@router~# echo 0 > /proc/sys/net/ipv4/ip_forward
```

Use cat to check if packet forwarding is enabled.

```
root@router~# cat /proc/sys/net/ipv4/ip_forward
```

72.12. /etc/sysctl.conf

By default, most Linux computers are not configured for automatic packet forwarding. To enable packet forwarding whenever the system starts, change the **net.ipv4.ip_forward** variable in **/etc/sysctl.conf** to the value 1.

```
root@router~# grep ip_forward /etc/sysctl.conf
net.ipv4.ip_forward = 0
```

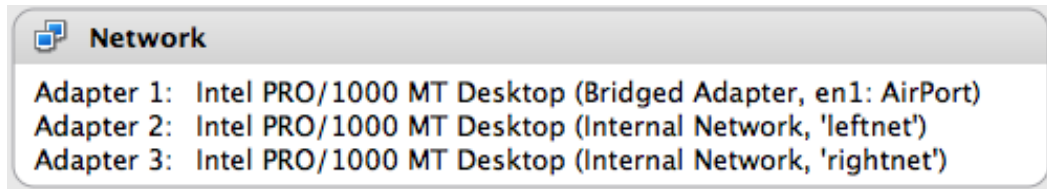
72.13. sysctl

For more information, take a look at the man page of **sysctl**.

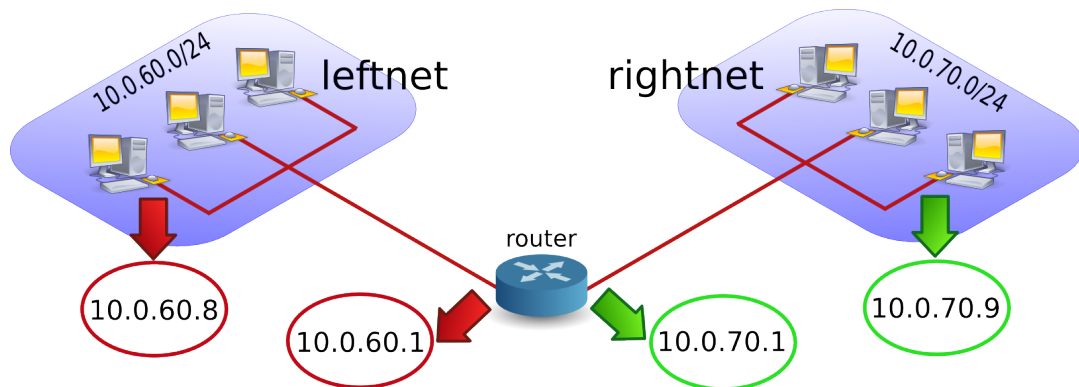
```
root@debian6~# man sysctl
root@debian6~# sysctl -a 2>/dev/null | grep ip_forward
net.ipv4.ip_forward = 0
```

72.14. practice: packet forwarding

0. You have the option to select (or create) an internal network when adding a network card in **VirtualBox** or **VMWare**. Use this option to create two internal networks. I named them **leftnet** and **rightnet**, but you can choose any other name.



1. Set up two Linux machines, one on **leftnet**, the other on **rightnet**. Make sure they both get an ip-address in the correct subnet. These two machines will be 'left' and 'right' from the 'router'.



2. Set up a third Linux computer with three network cards, one on **leftnet**, the other on **rightnet**. This computer will be the 'router'. Complete the table below with the relevant names, ip-addresses and **mac-addresses**.

Table 72.1. Packet Forwarding Exercise

	leftnet computer	the router		rightnet computer
MAC				
IP				

3. How can you verify whether the **router** will allow packet forwarding by default or not ? Test that you can **ping** from the **router** to the two other machines, and from those two machines to the **router**. Use **arp -a** to make sure you are connected with the correct **mac addresses**.

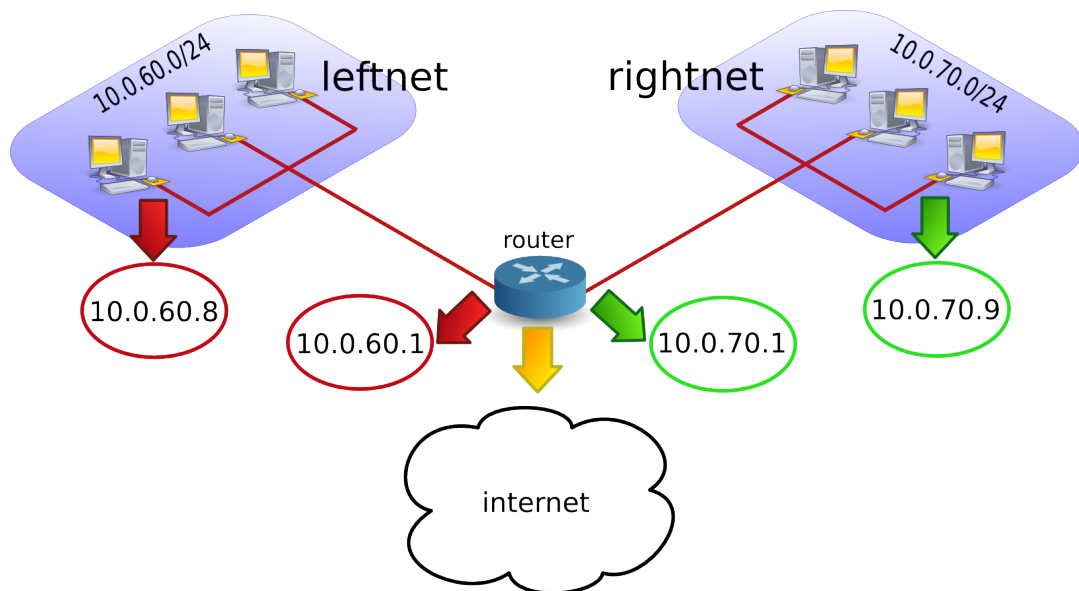
4. **Ping** from the leftnet computer to the rightnet computer. Enable and/or disable packet forwarding on the **router** and verify what happens to the ping between the two networks. If you do not succeed in pinging between the two networks (on different subnets), then use a sniffer like **wireshark** or **tcpdump** to discover the problem.

5. Use **wireshark** or **tcpdump -xx** to answer the following questions. Does the source MAC change when a packet passes through the filter ? And the destination MAC ? What about source and destination IP-addresses ?

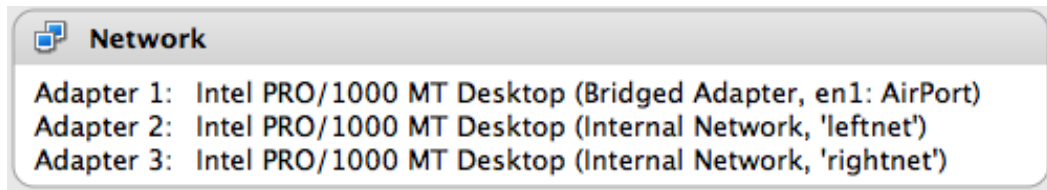
6. Remember the third network card on the router ? Connect this card to a LAN with internet connection. On many LAN's the command **dhclient eth0** just works (replace **eth0** with the correct interface).

```
root@router~# dhclient eth0
```

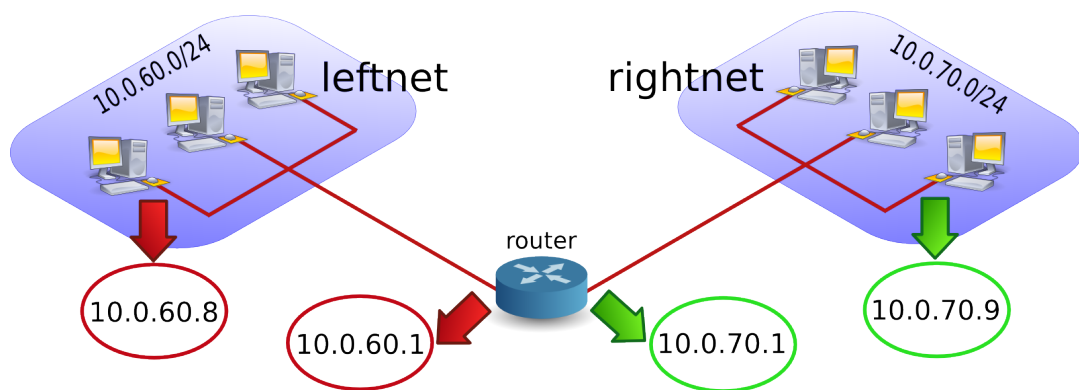
You now have a setup similar to this picture. What needs to be done to give internet access to **leftnet** and **rightnet**.



72.15. solution: packet forwarding



1. Set up two Linux machines, one on **leftnet**, the other on **rightnet**. Make sure they both get an ip-address in the correct subnet. These two machines will be 'left' and 'right' from the 'router'.



The ip configuration on your computers should be similar to the following two screenshots. Both machines must be in a different subnet (here 192.168.60.0/24 and 192.168.70.0/24). I created a little script on both machines to configure the interfaces.

```
root@left~# cat leftnet.sh
pkill dhclient
ifconfig eth0 192.168.60.8 netmask 255.255.255.0
```

```
root@right~# cat rightnet.sh
pkill dhclient
ifconfig eth0 192.168.70.9 netmask 255.255.255.0
```

2. Set up a third Linux computer with three network cards, one on **leftnet**, the other on **rightnet**. This computer will be the 'router'. Complete the table below with the relevant names, ip-addresses and mac-addresses.

```
root@router~# cat router.sh
ifconfig eth1 192.168.60.1 netmask 255.255.255.0
ifconfig eth2 192.168.70.1 netmask 255.255.255.0
#echo 1 > /proc/sys/net/ipv4/ip_forward
```

Your setup may use different ip and mac addresses than the ones in the table below.

Table 72.2. Packet Forwarding Solution

leftnet computer	the router		rightnet computer
08:00:27:f6:ab:b9	08:00:27:43:1f:5a	08:00:27:be:4a:6b	08:00:27:14:8b:17
192.168.60.8	192.168.60.1	192.168.70.1	192.168.70.9

3. How can you verify whether the **router** will allow packet forwarding by default or not ? Test that you can ping from the **router** to the two other machines, and from those two machines to the **router**. Use **arp -a** to make sure you are connected with the correct **mac addresses**.

This can be done with "**grep ip_forward /etc/sysctl.conf**" (1 is enabled, 0 is disabled) or with **sysctl -a | grep ip_for**.

```
root@router~# grep ip_for /etc/sysctl.conf
net.ipv4.ip_forward = 0
```

4. Ping from the leftnet computer to the rightnet computer. Enable and/or disable packet forwarding on the **router** and verify what happens to the ping between the two networks. If you do not succeed in pinging between the two networks (on different subnets), then use a sniffer like wireshark or tcpdump to discover the problem.

Did you forget to add a **default gateway** to the LAN machines ? Use **route add default gw 'ip-address'**.

```
root@left~# route add default gw 192.168.60.1
```

```
root@right~# route add default gw 192.168.70.1
```

You should be able to ping when packet forwarding is enabled (and both default gateways are properly configured). The ping will not work when packet forwarding is disabled or when gateways are not configured correctly.

5. Use wireshark or tcpdump -xx to answer the following questions. Does the source MAC change when a packet passes through the filter ? And the destination MAC ? What about source and destination IP-addresses ?

Both MAC addresses are changed when passing the router. Use **tcpdump -xx** like this:

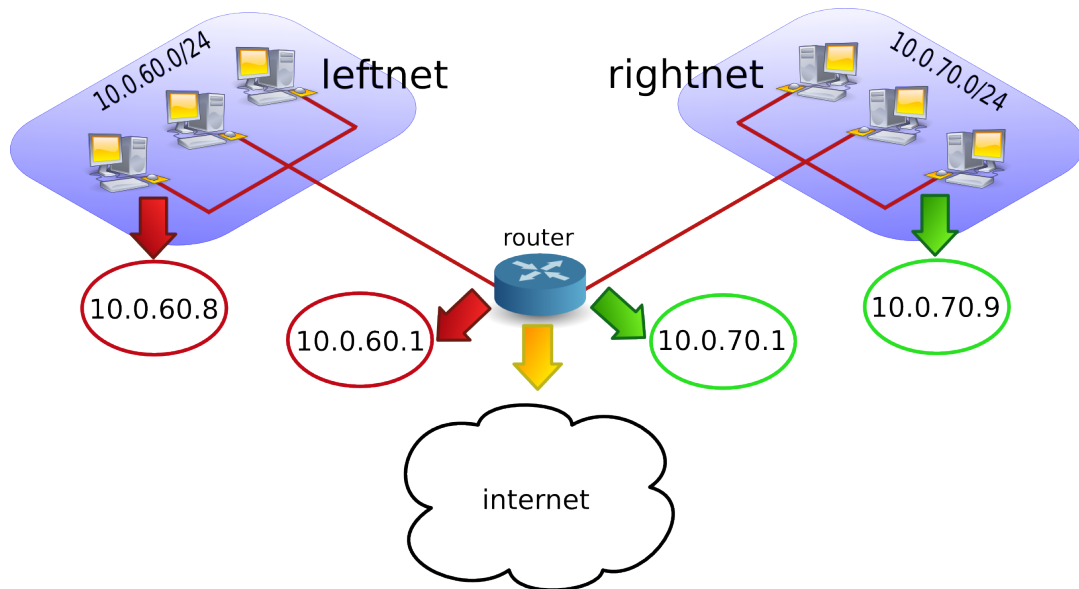
```
root@router~# tcpdump -xx -i eth1
```

```
root@router~# tcpdump -xx -i eth2
```

6. Remember the third network card on the router ? Connect this card to a LAN with internet connection. On many LAN's the command **dhclient eth0** just works (replace **eth0** with the correct interface).

```
root@router~# dhclient eth0
```

You now have a setup similar to this picture. What needs to be done to give internet access to **leftnet** and **rightnet**.



The clients on **leftnet** and **rightnet** need a working **dns server**. We use one of Google's dns servers here.

```
echo nameserver 8.8.8.8 > /etc/resolv.conf
```

Chapter 73. iptables firewall

This chapter introduces some simple firewall rules and how to configure them with **iptables**.

iptables is an application that allows a user to configure the firewall functionality built into the **Linux** kernel.

73.1. iptables tables

By default there are three **tables** in the kernel that contain sets of rules.

The **filter table** is used for packet filtering.

```
root@debian6~# iptables -t filter -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
```

The **nat table** is used for address translation.

```
root@debian6~# iptables -t nat -L
Chain PREROUTING (policy ACCEPT)
target     prot opt source                destination

Chain POSTROUTING (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
```

The **mangle table** can be used for special-purpose processing of packets.

Series of rules in each table are called a **chain**. We will discuss chains and the nat table later in this chapter.

73.2. starting and stopping iptables

The following screenshot shows how to stop and start **iptables** on Red Hat/Fedora/CentOS and compatible distributions.

```
[root@centos6 ~]# service iptables stop
[root@centos6 ~]# service iptables start
iptables: Applying firewall rules [ ok ]
[root@centos6 ~]#
```

Debian and *buntu distributions do not have this script, but allow for an uninstall.

```
root@debian6~# aptitude purge iptables
```

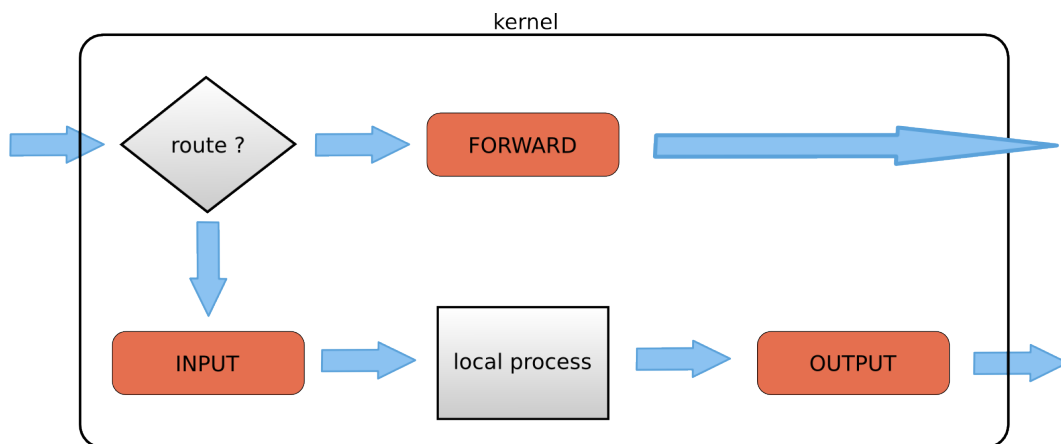

73.3. the filter table

73.3.1. about packet filtering

Packet filtering is a bit more than **packet forwarding**. While **packet forwarding** uses only a routing table to make decisions, **packet filtering** also uses a list of rules. The kernel will inspect packets and decide based on these rules what to do with each packet.

73.3.2. filter table

The filter table in **iptables** has three chains (sets of rules). The INPUT chain is used for any packet coming into the system. The OUTPUT chain is for any packet leaving the system. And the FORWARD chain is for packets that are forwarded (routed) through the system.



The screenshot below shows how to list the filter table and all its rules.

```
[root@RHEL5 ~]# iptables -t filter -nL
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
[root@RHEL5 ~]#
```

As you can see, all three chains in the filter table are set to **ACCEPT** everything. **ACCEPT** is the default behaviour.

73.3.3. setting default rules

The default for the default rule is indeed to ACCEPT everything. This is not the most secure firewall.

A more secure setup would be to DROP everything. A package that is **dropped** will not continue in any chain, and no warning or error will be sent anywhere.

The below commands lock down a computer. Do not execute these commands inside a remote ssh shell.

```
root@debianpaul~# iptables -P INPUT DROP
root@debianpaul~# iptables -P OUTPUT DROP
root@debianpaul~# iptables -P FORWARD DROP
root@debianpaul~# iptables -L
Chain INPUT (policy DROP)
target     prot opt source                destination

Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy DROP)
target     prot opt source                destination
```

73.3.4. changing policy rules

To start, let's set the default policy for all three chains to drop everything. Note that you might lose your connection when typing this over ssh ;-).

```
[root@RHEL5 ~]# iptables -P INPUT DROP
[root@RHEL5 ~]# iptables -P FORWARD DROP
[root@RHEL5 ~]# iptables -P OUTPUT DROP
```

Next, we allow the server to use its own loopback device (this allows the server to access its services running on localhost). We first append a rule to the INPUT chain to allow (ACCEPT) traffic from the lo (loopback) interface, then we do the same to allow packets to leave the system through the loopback interface.

```
[root@RHEL5 ~]# iptables -A INPUT -i lo -j ACCEPT
[root@RHEL5 ~]# iptables -A OUTPUT -o lo -j ACCEPT
```

Looking at the filter table again (omitting -t filter because it is the default table).

```
[root@RHEL5 ~]# iptables -nL
Chain INPUT (policy DROP)
target     prot opt source                destination
ACCEPT     all  --  0.0.0.0/0             0.0.0.0/0

Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy DROP)
target     prot opt source                destination
ACCEPT     all  --  0.0.0.0/0             0.0.0.0/0
```

73.3.5. Allowing ssh over eth0

This example show how to add two rules to allow ssh access to your system from outside.

```
[root@RHEL5 ~]# iptables -A INPUT -i eth0 -p tcp --dport 22 -j ACCEPT
[root@RHEL5 ~]# iptables -A OUTPUT -o eth0 -p tcp --sport 22 -j ACCEPT
```

The filter table will look something like this screenshot (note that -v is added for more verbose output).

```
[root@RHEL5 ~]# iptables -nvL
Chain INPUT (policy DROP 7 packets, 609 bytes)
  pkts bytes target prot opt in     out     source            destination
    0     0 ACCEPT all  --  lo      *       0.0.0.0/0         0.0.0.0/0
    0     0 ACCEPT tcp  --  eth0    *       0.0.0.0/0         0.0.0.0/0    tcp dpt:22

Chain FORWARD (policy DROP 0 packets, 0 bytes)
  pkts bytes target prot opt in     out     source            destination

Chain OUTPUT (policy DROP 3 packets, 228 bytes)
  pkts bytes target prot opt in     out     source            destination
    0     0 ACCEPT all  --  *       lo      0.0.0.0/0         0.0.0.0/0
    0     0 ACCEPT tcp  --  *       eth0    0.0.0.0/0         0.0.0.0/0    tcp spt:22
[root@RHEL5 ~]#
```

73.3.6. Allowing access from a subnet

This example shows how to allow access from any computer in the 10.1.1.0/24 network, but only through eth1. There is no port (application) limitation here.

```
[root@RHEL5 ~]# iptables -A INPUT -i eth1 -s 10.1.1.0/24 -p tcp -j ACCEPT
[root@RHEL5 ~]# iptables -A OUTPUT -o eth1 -d 10.1.1.0/24 -p tcp -j ACCEPT
```

Together with the previous examples, the policy is expanding.

```
[root@RHEL5 ~]# iptables -nvL
Chain INPUT (policy DROP 7 packets, 609 bytes)
  pkts bytes target prot opt in     out     source            destination
    0     0 ACCEPT all  --  lo      *       0.0.0.0/0         0.0.0.0/0
    0     0 ACCEPT tcp  --  eth0    *       0.0.0.0/0         0.0.0.0/0    tcp dpt:22
    0     0 ACCEPT tcp  --  eth1    *       10.1.1.0/24       0.0.0.0/0

Chain FORWARD (policy DROP 0 packets, 0 bytes)
  pkts bytes target prot opt in     out     source            destination

Chain OUTPUT (policy DROP 3 packets, 228 bytes)
  pkts bytes target prot opt in     out     source            destination
    0     0 ACCEPT all  --  *       lo      0.0.0.0/0         0.0.0.0/0
    0     0 ACCEPT tcp  --  *       eth0    0.0.0.0/0         0.0.0.0/0    tcp spt:22
    0     0 ACCEPT tcp  --  *       eth1    0.0.0.0/0         10.1.1.0/24
```

73.3.7. iptables save

Use **iptables save** to automatically implement these rules when the firewall is (re)started.

```
[root@RHEL5 ~]# /etc/init.d/iptables save
Saving firewall rules to /etc/sysconfig/iptables:          [ OK ]
[root@RHEL5 ~]#
```

73.3.8. scripting example

You can write a simple script for these rules. Below is an example script that implements the firewall rules that you saw before in this chapter.

```
#!/bin/bash
# first cleanup everything
iptables -t filter -F
iptables -t filter -X
iptables -t nat -F
iptables -t nat -X

# default drop
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT DROP

# allow loopback device
iptables -A INPUT -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT

# allow ssh over eth0 from outside to system
iptables -A INPUT -i eth0 -p tcp --dport 22 -j ACCEPT
iptables -A OUTPUT -o eth0 -p tcp --sport 22 -j ACCEPT

# allow any traffic from 10.1.1.0/24 to system
iptables -A INPUT -i eth1 -s 10.1.1.0/24 -p tcp -j ACCEPT
iptables -A OUTPUT -o eth1 -d 10.1.1.0/24 -p tcp -j ACCEPT
```

73.3.9. Allowing ICMP(ping)

When you enable iptables, you will get an '**Operation not permitted**' message when trying to ping other hosts.

```
[root@RHEL5 ~# ping 192.168.187.130
PING 192.168.187.130 (192.168.187.130) 56(84) bytes of data.
ping: sendmsg: Operation not permitted
ping: sendmsg: Operation not permitted
```

The screenshot below shows you how to setup iptables to allow a ping from or to your machine.

```
[root@RHEL5 ~]# iptables -A INPUT -p icmp --icmp-type any -j ACCEPT
[root@RHEL5 ~]# iptables -A OUTPUT -p icmp --icmp-type any -j ACCEPT
```

The previous two lines do not allow other computers to route ping messages through your router, because it only handles INPUT and OUTPUT. For routing of ping, you will need to enable it on the FORWARD chain. The following command enables routing of icmp messages between networks.

```
[root@RHEL5 ~]# iptables -A FORWARD -p icmp --icmp-type any -j ACCEPT
```

73.4. practice: packet filtering

1. Make sure you can ssh to your router-system when iptables is active.
2. Make sure you can ping to your router-system when iptables is active.
3. Define one of your networks as 'internal' and the other as 'external'. Configure the router to allow visits to a website (http) to go from the internal network to the external network (but not in the other direction).
4. Make sure the internal network can ssh to the external, but not the other way around.

73.5. solution: packet filtering

A possible solution, where leftnet is the internal and rightnet is the external network.

```
#!/bin/bash

# first cleanup everything
iptables -t filter -F
iptables -t filter -X
iptables -t nat -F
iptables -t nat -X

# default drop
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT DROP

# allow loopback device
iptables -A INPUT -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT

# question 1: allow ssh over eth0
iptables -A INPUT -i eth0 -p tcp --dport 22 -j ACCEPT
iptables -A OUTPUT -o eth0 -p tcp --sport 22 -j ACCEPT

# question 2: Allow icmp(ping) anywhere
iptables -A INPUT -p icmp --icmp-type any -j ACCEPT
iptables -A FORWARD -p icmp --icmp-type any -j ACCEPT
iptables -A OUTPUT -p icmp --icmp-type any -j ACCEPT

# question 3: allow http from internal(leftnet) to external(rightnet)
iptables -A FORWARD -i eth1 -o eth2 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -i eth2 -o eth1 -p tcp --sport 80 -j ACCEPT

# question 4: allow ssh from internal(leftnet) to external(rightnet)
iptables -A FORWARD -i eth1 -o eth2 -p tcp --dport 22 -j ACCEPT
iptables -A FORWARD -i eth2 -o eth1 -p tcp --sport 22 -j ACCEPT

# allow http from external(rightnet) to internal(leftnet)
# iptables -A FORWARD -i eth2 -o eth1 -p tcp --dport 80 -j ACCEPT
# iptables -A FORWARD -i eth1 -o eth2 -p tcp --sport 80 -j ACCEPT

# allow rpcinfo over eth0 from outside to system
# iptables -A INPUT -i eth2 -p tcp --dport 111 -j ACCEPT
# iptables -A OUTPUT -o eth2 -p tcp --sport 111 -j ACCEPT
```

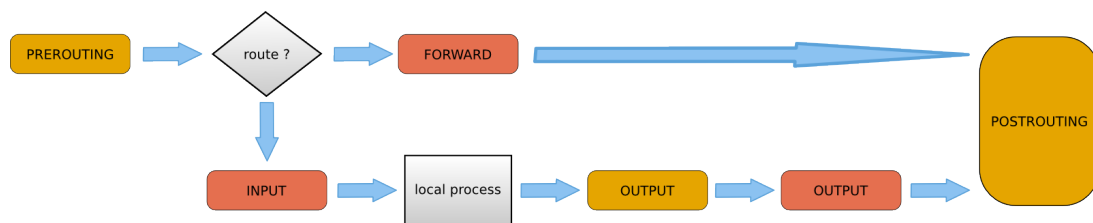
73.6. network address translation

73.6.1. about NAT

A NAT device is a router that is also changing the source and/or target ip-address in packets. It is typically used to connect multiple computers in a private address range with the (public) internet. A NAT can hide private addresses from the internet.

NAT was developed to mitigate the use of real ip addresses, to allow private address ranges to reach the internet and back, and to not disclose details about internal networks to the outside.

The nat table in iptables adds two new chains. PREROUTING allows altering of packets before they reach the INPUT chain. POSTROUTING allows altering packets after they exit the OUTPUT chain.



Use **iptables -t nat -nL** to look at the NAT table. The screenshot below shows an empty NAT table.

```

[root@RHEL5 ~]# iptables -t nat -nL
Chain PREROUTING (policy ACCEPT)
target     prot opt source               destination

Chain POSTROUTING (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
[root@RHEL5 ~]#
  
```


73.6.2. SNAT (Source NAT)

The goal of source nat is to change the source address inside a packet before it leaves the system (e.g. to the internet). The destination will return the packet to the NAT-device. This means our NAT-device will need to keep a table in memory of all the packets it changed, so it can deliver the packet to the original source (e.g. in the private network).

Because SNAT is about packets leaving the system, it uses the POSTROUTING chain.

Here is an example SNAT rule. The rule says that packets coming from 10.1.1.0/24 network and exiting via eth1 will get the source ip-address set to 11.12.13.14. (Note that this is a one line command!)

```
iptables -t nat -A POSTROUTING -o eth1 -s 10.1.1.0/24 -j SNAT \
--to-source 11.12.13.14
```

Of course there must exist a proper iptables filter setup to allow the packet to traverse from one network to the other.

73.6.3. SNAT example setup

This example script uses a typical nat setup. The internal (eth0) network has access via SNAT to external (eth1) web servers (port 80).

```
#!/bin/bash
#
# iptables script for simple classic nat websurfing
# eth0 is internal network, eth1 is internet
#
echo 0 > /proc/sys/net/ipv4/ip_forward
iptables -P INPUT ACCEPT
iptables -P OUTPUT ACCEPT
iptables -P FORWARD DROP
iptables -A FORWARD -i eth0 -o eth1 -s 10.1.1.0/24 -p tcp \
--dport 80 -j ACCEPT
iptables -A FORWARD -i eth1 -o eth0 -d 10.1.1.0/24 -p tcp \
--sport 80 -j ACCEPT
iptables -t nat -A POSTROUTING -o eth1 -s 10.1.1.0/24 -j SNAT \
--to-source 11.12.13.14
echo 1 > /proc/sys/net/ipv4/ip_forward
```

73.6.4. IP masquerading

IP masquerading is very similar to SNAT, but is meant for dynamic interfaces. Typical example are broadband 'router/modems' connected to the internet and receiving a different ip-address from the isp, each time they are cold-booted.

The only change needed to convert the SNAT script to a masquerading is one line.

```
iptables -t nat -A POSTROUTING -o eth1 -s 10.1.1.0/24 -j MASQUERADE
```

73.6.5. DNAT (Destination NAT)

DNAT is typically used to allow packets from the internet to be redirected to an internal server (in your DMZ) and in a private address range that is inaccessible directly from the internet.

This example script allows internet users to reach your internal (192.168.1.99) server via ssh (port 22).

```
#!/bin/bash
#
# iptables script for DNAT
# eth0 is internal network, eth1 is internet
#
echo 0 > /proc/sys/net/ipv4/ip_forward
iptables -P INPUT ACCEPT
iptables -P OUTPUT ACCEPT
iptables -P FORWARD DROP
iptables -A FORWARD -i eth0 -o eth1 -s 10.1.1.0/24 -j ACCEPT
iptables -A FORWARD -i eth1 -o eth0 -p tcp --dport 22 -j ACCEPT
iptables -t nat -A PREROUTING -i eth1 -p tcp --dport 22 \
-j DNAT --to-destination 10.1.1.99
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Part XXII. selinux

Table of Contents

74. introduction to SELinux	821
74.1. selinux modes	822
74.2. logging	822
74.3. activating selinux	822
74.4. getenforce	823
74.5. setenforce	823
74.6. sestatus	824
74.7. policy	824
74.8. /etc/selinux/config	824
74.9. DAC or MAC	825
74.10. ls -Z	825
74.11. -Z	825
74.12. /selinux	826
74.13. identity	826
74.14. role	826
74.15. type (or domain)	827
74.16. security context	828
74.17. transition	828
74.18. extended attributes	829
74.19. process security context	829
74.20. chcon	829
74.21. an example	830
74.22. setroubleshoot	832
74.23. booleans	834

Chapter 74. introduction to SELinux

Security Enhanced Linux or **SELinux** is a set of modifications developed by the United States National Security Agency (NSA) to provide a variety of security policies for Linux. SELinux was released as open source at the end of 2000. Since kernel version 2.6 it is an integrated part of Linux.

SELinux offers security! SELinux can control what kind of access users have to files and processes. Even when a file received **chmod 777**, SELinux can still prevent applications from accessing it (Unix file permissions are checked first!). SELinux does this by placing users in **roles** that represent a security context. Administrators have very strict control on access permissions granted to roles.

SELinux is present in the latest versions of Red Hat Enterprise Linux, Debian, CentOS, Fedora, and many other distributions..

74.1. selinux modes

selinux knows three modes: enforcing, permissive and disabled. The **enforcing** mode will enforce policies, and may deny access based on **selinux rules**. The **permissive** mode will not enforce policies, but can still log actions that would have been denied in **enforcing** mode. The **disabled** mode disables **selinux**.

74.2. logging

Verify that **syslog** is running and activated on boot to enable logging of deny messages in **/var/log/messages**.

```
[root@rhel55 ~]# chkconfig --list syslog
syslog          0:off 1:off 2:on 3:on 4:on 5:on 6:off
```

Verify that **auditd** is running and activated on boot to enable logging of easier to read messages in **/var/log/audit/audit.log**.

```
[root@rhel55 ~]# chkconfig --list auditd
auditd          0:off 1:off 2:on 3:on 4:on 5:on 6:off
```

If not activated, then run **chkconfig --levels 2345 auditd on** and **service auditd start**.

```
[root@rhel55 ~]# service auditd status
auditd (pid 1660) is running...
[root@rhel55 ~]# service syslog status
syslogd (pid 1688) is running...
klogd (pid 1691) is running...
```

The **/var/log/messages** log file will tell you that **selinux** is disabled.

```
root@deb503:~# grep -i selinux /var/log/messages
Jun 25 15:59:34 deb503 kernel: [    0.084083] SELinux:  Disabled at boot.
```

Or that it is enabled.

```
root@deb503:~# grep SELinux /var/log/messages | grep -i Init
Jun 25 15:09:52 deb503 kernel: [    0.084094] SELinux:  Initializing.
```

74.3. activating selinux

On RHEL you can use the GUI tool to activate **selinux**, on Debian there is the **selinux-activate** command. Activation requires a reboot.

```
root@deb503:~# selinux-activate
Activating SE Linux
Searching for GRUB installation directory ... found: /boot/grub
Searching for default file ... found: /boot/grub/default
Testing for an existing GRUB menu.lst file ... found: /boot/grub/menu.lst
Searching for splash image ... none found, skipping ...
Found kernel: /boot/vmlinuz-2.6.26-2-686
Updating /boot/grub/menu.lst ... done

SE Linux is activated.  You may need to reboot now.
```

74.4. getenforce

Use **getenforce** to verify whether selinux is **enforced**, **disabled** or **permissive**.

```
[root@rhel55 ~]# getenforce
Permissive
```

The **/selinux/enforce** file contains 1 when enforcing, and 0 when permissive mode is active.

```
root@fedora13 ~# cat /selinux/enforce
1root@fedora13 ~#
```

74.5. setenforce

You can use **setenforce** to switch between the **Permissive** or the **Enforcing** state once **selinux** is activated..

```
[root@rhel55 ~]# setenforce Enforcing
[root@rhel55 ~]# getenforce
Enforcing
[root@rhel55 ~]# setenforce Permissive
[root@rhel55 ~]# getenforce
Permissive
```

Or you could just use 0 and 1 as argument.

```
[root@centos65 ~]# setenforce 1
[root@centos65 ~]# getenforce
Enforcing
[root@centos65 ~]# setenforce 0
[root@centos65 ~]# getenforce
Permissive
[root@centos65 ~]#
```

74.6. sestatus

You can see the current **selinux** status and policy with the **sestatus** command.

```
[root@rhel55 ~]# sestatus
SELinux status:                enabled
SELinuxfs mount:              /selinux
Current mode:                  permissive
Mode from config file:         permissive
Policy version:                21
Policy from config file:       targeted
```

74.7. policy

Most Red Hat server will have the **targeted** policy. Only NSA/FBI/CIA/DOD/HLS use the **mls** policy.

The targeted policy will protect hundreds of processes, but lets other processes run 'unconfined' (= they can do anything).

74.8. /etc/selinux/config

The main configuration file for **selinux** is **/etc/selinux/config**. When in **permissive** mode, the file looks like this.

The targeted policy is selected in **/etc/selinux/config**.

```
[root@centos65 ~]# cat /etc/selinux/config
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#     enforcing - SELinux security policy is enforced.
#     permissive - SELinux prints warnings instead of enforcing.
#     disabled - SELinux is fully disabled.
SELINUX=permissive
# SELINUXTYPE= type of policy in use. Possible values are:
#     targeted - Only targeted network daemons are protected.
#     strict - Full SELinux protection.
SELINUXTYPE=targeted
```


74.9. DAC or MAC

Standard Unix permissions use **Discretionary Access Control** to set permissions on files. This means that a user that owns a file, can make it world readable by typing **chmod 777 \$file**.

With **selinux** the kernel will enforce **Mandatory Access Control** which strictly controls what processes or threads can do with files (superseding DAC). Processes are confined by the kernel to the minimum access they require.

SELinux MAC is about labeling and type enforcing! Files, processes, etc are all labeled with an SELinux context. For files, these are extended attributes, for processes this is managed by the kernel.

The format of the labels is as follows:

```
user:role:type:(level)
```

We only use the **type** label in the targeted policy.

74.10. ls -Z

To see the DAC permissions on a file, use **ls -l** to display user and group **owner** and permissions.

For MAC permissions there is new **-Z** option added to **ls**. The output shows that file in **/root** have a XXXtype of **admin_home_t**.

```
[root@centos65 ~]# ls -Z
-rw----- . root root system_u:object_r:admin_home_t:s0 anaconda-ks.cfg
-rw-r--r-- . root root system_u:object_r:admin_home_t:s0 install.log
-rw-r--r-- . root root system_u:object_r:admin_home_t:s0 install.log.syslog
```

```
[root@centos65 ~]# useradd -m -s /bin/bash pol
[root@centos65 ~]# ls -Z /home/pol/.bashrc
-rw-r--r-- . pol pol unconfined_u:object_r:user_home_t:s0 /home/pol/.bashrc
```

74.11. -Z

There are also some other tools with the **-Z** switch:

```
mkdir -Z
cp -Z
ps -Z
netstat -Z
...
```

74.12. /selinux

When selinux is active, there is a new virtual file system named **/selinux**. (You can compare it to /proc and /dev.)

```
[root@centos65 ~]# ls -l /selinux/
total 0
-rw-rw-rw-. 1 root root 0 Apr 12 19:40 access
dr-xr-xr-x. 2 root root 0 Apr 12 19:40 avc
dr-xr-xr-x. 2 root root 0 Apr 12 19:40 booleans
-rw-r--r--. 1 root root 0 Apr 12 19:40 checkreqprot
dr-xr-xr-x. 83 root root 0 Apr 12 19:40 class
--w-----. 1 root root 0 Apr 12 19:40 commit_pending_bools
-rw-rw-rw-. 1 root root 0 Apr 12 19:40 context
-rw-rw-rw-. 1 root root 0 Apr 12 19:40 create
-r--r--r--. 1 root root 0 Apr 12 19:40 deny_unknown
--w-----. 1 root root 0 Apr 12 19:40 disable
-rw-r--r--. 1 root root 0 Apr 12 19:40 enforce
dr-xr-xr-x. 2 root root 0 Apr 12 19:40 initial_contexts
-rw-----. 1 root root 0 Apr 12 19:40 load
-rw-rw-rw-. 1 root root 0 Apr 12 19:40 member
-r--r--r--. 1 root root 0 Apr 12 19:40 mls
crw-rw-rw-. 1 root root 1, 3 Apr 12 19:40 null
-r-----. 1 root root 0 Apr 12 19:40 policy
dr-xr-xr-x. 2 root root 0 Apr 12 19:40 policy_capabilities
-r--r--r--. 1 root root 0 Apr 12 19:40 policyvers
-r--r--r--. 1 root root 0 Apr 12 19:40 reject_unknown
-rw-rw-rw-. 1 root root 0 Apr 12 19:40 relabel
-r--r--r--. 1 root root 0 Apr 12 19:40 status
-rw-rw-rw-. 1 root root 0 Apr 12 19:40 user
```

Although some files in **/selinux** appear with size 0, they often contain a boolean value. Check **/selinux/enforce** to see if selinux is running in enforced mode.

```
[root@RHEL5 ~]# ls -l /selinux/enforce
-rw-r--r-- 1 root root 0 Apr 29 08:21 /selinux/enforce
[root@RHEL5 ~]# echo $(cat /selinux/enforce)
1
```

74.13. identity

The **SELinux Identity** of a user is distinct from the user ID. An identity is part of a security context, and (via domains) determines what you can do. The screenshot shows user **root** having identity **user_u**.

```
[root@rhel55 ~]# id -Z
user_u:system_r:unconfined_t
```

74.14. role

The **selinux role** defines the domains that can be used. A **role** is denied to enter a domain, unless the **role** is explicitly authorized to do so.

74.15. type (or domain)

The **selinux context** is the security context of a process. An **selinux type** determines what a process can do. The screenshot shows `init` running in type **init_t** and the `mingetty`'s running in type **getty_t**.

```
[root@centos65 ~]# ps fax -Z | grep /sbin/init
system_u:system_r:init_t:s0      1 ?          Ss        0:00 /sbin/init
[root@centos65 ~]# ps fax -Z | grep getty_t
system_u:system_r:getty_t:s0    1307 tty1      Ss+       0:00 /sbin/mingetty /dev/tty1
system_u:system_r:getty_t:s0    1309 tty2      Ss+       0:00 /sbin/mingetty /dev/tty2
system_u:system_r:getty_t:s0    1311 tty3      Ss+       0:00 /sbin/mingetty /dev/tty3
system_u:system_r:getty_t:s0    1313 tty4      Ss+       0:00 /sbin/mingetty /dev/tty4
system_u:system_r:getty_t:s0    1320 tty5      Ss+       0:00 /sbin/mingetty /dev/tty5
system_u:system_r:getty_t:s0    1322 tty6      Ss+       0:00 /sbin/mingetty /dev/tty6
```

The **selinux type** is similar to an **selinux domain**, but refers to directories and files instead of processes.

Hundreds of binaries also have a type:

```
[root@centos65 sbin]# ls -lZ useradd usermod userdel httpd postcat postfix
-rwxr-xr-x. root root system_u:object_r:httpd_exec_t:s0 httpd
-rwxr-xr-x. root root system_u:object_r:postfix_master_exec_t:s0 postcat
-rwxr-xr-x. root root system_u:object_r:postfix_master_exec_t:s0 postfix
-rwxr-x---. root root system_u:object_r:useradd_exec_t:s0 useradd
-rwxr-x---. root root system_u:object_r:useradd_exec_t:s0 userdel
-rwxr-x---. root root system_u:object_r:useradd_exec_t:s0 usermod
```

Ports also have a context.

```
[root@centos65 sbin]# netstat -nptlZ | tr -s ' ' | cut -d' ' -f6-

Foreign Address State PID/Program name Security Context
LISTEN 1096/rpcbind system_u:system_r:rpcbind_t:s0
LISTEN 1208/sshd system_u:system_r:sshd_t:s0-s0:c0.c1023
LISTEN 1284/master system_u:system_r:postfix_master_t:s0
LISTEN 1114/rpc.statd system_u:system_r:rpcd_t:s0
LISTEN 1096/rpcbind system_u:system_r:rpcbind_t:s0
LISTEN 1666/httpd unconfined_u:system_r:httpd_t:s0
LISTEN 1208/sshd system_u:system_r:sshd_t:s0-s0:c0.c1023
LISTEN 1114/rpc.statd system_u:system_r:rpcd_t:s0
LISTEN 1284/master system_u:system_r:postfix_master_t:s0
```

You can also get a list of ports that are managed by SELinux:

```
[root@centos65 ~]# semanage port -l | tail
xfs_port_t          tcp          7100
xserver_port_t      tcp          6000-6150
zabbix_agent_port_t tcp          10050
zabbix_port_t       tcp          10051
zaraftcp_port_t     tcp          236, 237
zebra_port_t         tcp          2600-2604, 2606
zebra_port_t         udp          2600-2604, 2606
zented_port_t       tcp          1229
zented_port_t       udp          1229
zope_port_t         tcp          8021
```

74.16. security context

The combination of identity, role and domain or type make up the **selinux security context**. The **id** will show you your security context in the form identity:role:domain.

```
[paul@RHEL5 ~]$ id | cut -d' ' -f4
context=user_u:system_r:unconfined_t
```

The **ls -Z** command shows the security context for a file in the form identity:role:type.

```
[paul@RHEL5 ~]$ ls -Z test
-rw-rw-r-- paul paul user_u:object_r:user_home_t test
```

The security context for processes visible in `/proc` defines both the type (of the file in `/proc`) and the domain (of the running process). Let's take a look at the `init` process and `/proc/1/`.

The `init` process runs in domain **init_t**.

```
[root@RHEL5 ~]# ps -ZC init
LABEL                                PID TTY          TIME CMD
system_u:system_r:init_t             1 ?           00:00:01 init
```

The `/proc/1/` directory, which identifies the **init** process, has type **init_t**.

```
[root@RHEL5 ~]# ls -Zd /proc/1/
dr-xr-xr-x root root system_u:system_r:init_t /proc/1/
```

It is not a coincidence that the domain of the **init** process and the type of `/proc/1/` are both **init_t**.

Don't try to use **chcon** on `/proc`! It will not work.

74.17. transition

An **selinux transition** (aka an selinux labelling) determines the security context that will be assigned. A transition of process domains is used when you execute a process. A transition of file type happens when you create a file.

An example of file type transition.

```
[pol@centos65 ~]$ touch test /tmp/test
[pol@centos65 ~]$ ls -Z test
-rw-rw-r-- pol pol unconfined_u:object_r:user_home_t:s0 test
[pol@centos65 ~]$ ls -Z /tmp/test
-rw-rw-r-- pol pol unconfined_u:object_r:user_tmp_t:s0 /tmp/test
```

74.18. extended attributes

Extended attributes are used by **selinux** to store security contexts. These attributes can be viewed with **ls** when **selinux** is running.

```
[root@RHEL5 home]# ls --context
drwx----- paul paul system_u:object_r:user_home_dir_t paul
drwxr-xr-x root root user_u:object_r:user_home_dir_t project42
drwxr-xr-x root root user_u:object_r:user_home_dir_t project55
[root@RHEL5 home]# ls -Z
drwx----- paul paul system_u:object_r:user_home_dir_t paul
drwxr-xr-x root root user_u:object_r:user_home_dir_t project42
drwxr-xr-x root root user_u:object_r:user_home_dir_t project55
[root@RHEL5 home]#
```

When selinux is not running, then **getfattr** is the tool to use.

```
[root@RHEL5 etc]# getfattr -m . -d hosts
# file: hosts
security.selinux="system_u:object_r:etc_t:s0\000"
```

74.19. process security context

A new option is added to **ps** to see the selinux security context of processes.

```
[root@RHEL5 etc]# ps -ZC mingetty
LABEL                                PID TTY          TIME CMD
system_u:system_r:getty_t            2941 tty1        00:00:00 mingetty
system_u:system_r:getty_t            2942 tty2        00:00:00 mingetty
```

74.20. chcon

Use **chcon** to change the selinux security context.

This example shows how to use **chcon** to change the **type** of a file.

```
[root@rhel55 ~]# ls -Z /var/www/html/test42.txt
-rw-r--r-- root root user_u:object_r:httpd_sys_content_t /var/www/html/test4\
2.txt
[root@rhel55 ~]# chcon -t samba_share_t /var/www/html/test42.txt
[root@rhel55 ~]# ls -Z /var/www/html/test42.txt
-rw-r--r-- root root user_u:object_r:samba_share_t /var/www/html/test42.txt
```

Be sure to read **man chcon**.

74.21. an example

The **Apache2 webserver** is by default targeted with **SELinux**. The next screenshot shows that any file created in **/var/www/html** will by default get the **httpd_sys_content_t** type.

```
[root@centos65 ~]# touch /var/www/html/test42.txt
[root@centos65 ~]# ls -Z /var/www/html/test42.txt
-rw-r--r--. root root unconfined_u:object_r:httpd_sys_content_t:s0 /var/www/h\
tml/test42.txt
```

Files created elsewhere do not get this type.

```
[root@centos65 ~]# touch /root/test42.txt
[root@centos65 ~]# ls -Z /root/test42.txt
-rw-r--r--. root root unconfined_u:object_r:admin_home_t:s0 /root/test42.txt
```

Make sure **Apache2** runs.

```
[root@centos65 ~]# service httpd restart
Stopping httpd:          [ OK ]
Starting httpd:          [ OK ]
```

Will this work ? Yes it does.

```
[root@centos65 ~]# wget http://localhost/test42.txt
--2014-04-12 20:56:47-- http://localhost/test42.txt
Resolving localhost... :1, 127.0.0.1
Connecting to localhost|::1|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 0 [text/plain]
Saving to: "test42.txt"
...
```

Why does this work ? Because Apache2 runs in the **httpd_t** domain and the files in **/var/www/html** have the **httpd_sys_content_t** type.

```
[root@centos65 ~]# ps -ZC httpd | head -4
LABEL                                PID TTY          TIME CMD
unconfined_u:system_r:httpd_t:s0 1666 ?           00:00:00 httpd
unconfined_u:system_r:httpd_t:s0 1668 ?           00:00:00 httpd
unconfined_u:system_r:httpd_t:s0 1669 ?           00:00:00 httpd
```

So let's set SELinux to **enforcing** and change the **type** of this file.

```
[root@centos65 ~]# chcon -t samba_share_t /var/www/html/test42.txt
[root@centos65 ~]# ls -Z /var/www/html/test42.txt
-rw-r--r--. root root unconfined_u:object_r:samba_share_t:s0 /var/www/html/t\
est42.txt
[root@centos65 ~]# setenforce 1
[root@centos65 ~]# getenforce
Enforcing
```

There are two possibilities now: either it works, or it fails. It works when **selinux** is in **permissive mode**, it fails when in **enforcing mode**.

```
[root@centos65 ~]# wget http://localhost/test42.txt
--2014-04-12 21:05:02-- http://localhost/test42.txt
Resolving localhost... ::1, 127.0.0.1
Connecting to localhost|::1|:80... connected.
HTTP request sent, awaiting response... 403 Forbidden
2014-04-12 21:05:02 ERROR 403: Forbidden.
```

The log file gives you a cryptic message...

```
[root@centos65 ~]# tail -3 /var/log/audit/audit.log
type=SYSCALL msg=audit(1398200702.803:64): arch=c0000003e syscall=4 succ\
ess=no exit=-13 a0=7f5fbc334d70 a1=7fff553b4f10 a2=7fff553b4f10 a3=0 it\
ems=0 ppid=1666 pid=1673 auid=500 uid=48 gid=48 euid=48 suid=48 fsuid=4\
8 egid=48 sgid=48 fsgid=48 tty=(none) ses=1 comm="httpd" exe="/usr/sbin\
/httpd" subj=unconfined_u:system_r:httpd_t:s0 key=(null)
type=AVC msg=audit(1398200702.804:65): avc: denied { getattr } for p\
id=1673 comm="httpd" path="/var/www/html/test42.txt" dev=dm-0 ino=26324\
1 scontext=unconfined_u:system_r:httpd_t:s0 tcontext=unconfined_u:objec\
t_r:samba_share_t:s0 tclass=file
type=SYSCALL msg=audit(1398200702.804:65): arch=c0000003e syscall=6 succ\
ess=no exit=-13 a0=7f5fbc334e40 a1=7fff553b4f10 a2=7fff553b4f10 a3=1 it\
ems=0 ppid=1666 pid=1673 auid=500 uid=48 gid=48 euid=48 suid=48 fsuid=4\
8 egid=48 sgid=48 fsgid=48 tty=(none) ses=1 comm="httpd" exe="/usr/sbin\
/httpd" subj=unconfined_u:system_r:httpd_t:s0 key=(null)
```

And **/var/log/messages** mentions nothing of the failed download.

74.22. setroubleshoot

The log file above was not very helpful, but these two packages can make your life much easier.

```
[root@centos65 ~]# yum -y install setroubleshoot setroubleshoot-server
```

You need to **reboot** for this to work...

So we reboot, restart the httpd server, reactive SELinux Enforce, and do the wget again... and it fails (because of SELinux).

```
[root@centos65 ~]# service httpd restart
Stopping httpd:                                     [FAILED]
Starting httpd:                                     [ OK ]
[root@centos65 ~]# getenforce
Permissive
[root@centos65 ~]# setenforce 1
[root@centos65 ~]# getenforce
Enforcing
[root@centos65 ~]# wget http://localhost/test42.txt
--2014-04-12 21:44:13-- http://localhost/test42.txt
Resolving localhost... :1, 127.0.0.1
Connecting to localhost[:1]:80... connected.
HTTP request sent, awaiting response... 403 Forbidden
2014-04-12 21:44:13 ERROR 403: Forbidden.
```

The **/var/log/audit/** is still not out best friend, but take a look at **/var/log/messages**.

```
[root@centos65 ~]# tail -2 /var/log/messages
Apr 12 21:44:16 centos65 setroubleshoot: SELinux is preventing /usr/sbin/h\
ttpd from getattr access on the file /var/www/html/test42.txt. For complete \
SELinux messages. run sealert -l b2a84386-54c1-4344-96fb-dcf969776696
Apr 12 21:44:16 centos65 setroubleshoot: SELinux is preventing /usr/sbin/h\
ttpd from getattr access on the file /var/www/html/test42.txt. For complete \
SELinux messages. run sealert -l b2a84386-54c1-4344-96fb-dcf969776696
```

So we run the command it suggests...

```
[root@centos65 ~]# sealert -l b2a84386-54c1-4344-96fb-dcf969776696
SELinux is preventing /usr/sbin/httpd from getattr access on the file /va\
r/www/html/test42.txt.

**** Plugin restorecon (92.2 confidence) suggests ****

If you want to fix the label.
/var/www/html/test42.txt default label should be httpd_sys_content_t.
Then you can run restorecon.
Do
# /sbin/restorecon -v /var/www/html/test42.txt
...
```


We follow the friendly advice and try again to download our file:

```
[root@centos65 ~]# /sbin/restorecon -v /var/www/html/test42.txt
/sbin/restorecon reset /var/www/html/test42.txt context unconfined_u:object\
t_r:samba_share_t:s0->unconfined_u:object_r:httpd_sys_content_t:s0
[root@centos65 ~]# wget http://localhost/test42.txt
--2014-04-12 21:54:03-- http://localhost/test42.txt
Resolving localhost... ::1, 127.0.0.1
Connecting to localhost[::1]:80... connected.
HTTP request sent, awaiting response... 200 OK
```

It works!

74.23. booleans

Booleans are on/off switches

```
[root@centos65 ~]# getsebool -a | head
abrt_anon_write --> off
abrt_handle_event --> off
allow_console_login --> on
allow_cvs_read_shadow --> off
allow_daemons_dump_core --> on
allow_daemons_use_tcp_wrapper --> off
allow_daemons_use_tty --> on
allow_domain_fd_use --> on
allow_execheap --> off
allow_execmem --> on
```

You can set and read individual booleans.

```
[root@centos65 ~]# setsebool httpd_read_user_content=1
[root@centos65 ~]# getsebool httpd_read_user_content
httpd_read_user_content --> on
[root@centos65 ~]# setsebool httpd_enable_homedirs=1
[root@centos65 ~]# getsebool httpd_enable_homedirs
httpd_enable_homedirs --> on
```

You can set these booleans permanent.

```
[root@centos65 ~]# setsebool -P httpd_enable_homedirs=1
[root@centos65 ~]# setsebool -P httpd_read_user_content=1
```

The above commands regenerate the complete `/etc/selinux/targeted` directory!

```
[root@centos65 ~]# cat /etc/selinux/targeted/modules/active/booleans.local
# This file is auto-generated by libsemanage
# Do not edit directly.

httpd_enable_homedirs=1
httpd_read_user_content=1
```

Part XXIII. Introduction to Samba

Table of Contents

75. introduction to samba	838
75.1. verify installed version	839
75.2. installing samba	840
75.3. documentation	841
75.4. starting and stopping samba	842
75.5. samba daemons	843
75.6. the SMB protocol	844
75.7. practice: introduction to samba	845
76. getting started with samba	846
76.1. /etc/samba/smb.conf	847
76.2. /usr/bin/testparm	848
76.3. /usr/bin/smbclient	849
76.4. /usr/bin/smbtree	851
76.5. server string	852
76.6. Samba Web Administration Tool (SWAT)	853
76.7. practice: getting started with samba	854
76.8. solution: getting started with samba	855
77. a read only file server	857
77.1. Setting up a directory to share	858
77.2. configure the share	858
77.3. restart the server	859
77.4. verify the share	859
77.5. a note on netcat	861
77.6. practice: read only file server	862
77.7. solution: read only file server	863
78. a writable file server	864
78.1. set up a directory to share	865
78.2. share section in smb.conf	865
78.3. configure the share	865
78.4. test connection with windows	865
78.5. test writing with windows	866
78.6. How is this possible ?	866
78.7. practice: writable file server	867
78.8. solution: writable file server	868
79. samba first user account	869
79.1. creating a samba user	870
79.2. ownership of files	870
79.3. /usr/bin/smbpasswd	870
79.4. /etc/samba/smbpasswd	870
79.5. passwd backend	871
79.6. forcing this user	871
79.7. practice: first samba user account	872
79.8. solution: first samba user account	873
80. samba authentication	874
80.1. creating the users on Linux	875
80.2. creating the users on samba	875
80.3. security = user	875
80.4. configuring the share	876
80.5. testing access with net use	876
80.6. testing access with smbclient	876
80.7. verify ownership	877
80.8. common problems	877
80.9. practice : samba authentication	879
80.10. solution: samba authentication	880
81. samba securing shares	881

81.1. security based on user name	882
81.2. security based on ip-address	882
81.3. security through obscurity	883
81.4. file system security	883
81.5. practice: securing shares	885
81.6. solution: securing shares	886
82. samba domain member	888
82.1. changes in smb.conf	889
82.2. joining an Active Directory domain	890
82.3. winbind	891
82.4. wbio	891
82.5. getent	892
82.6. file ownership	893
82.7. practice : samba domain member	894
83. samba domain controller	895
83.1. about Domain Controllers	896
83.2. About security modes	896
83.3. About password backends	897
83.4. [global] section in smb.conf	897
83.5. netlogon share	898
83.6. other [share] sections	898
83.7. Users and Groups	899
83.8. tdbsam	899
83.9. about computer accounts	900
83.10. local or roaming profiles	900
83.11. Groups in NTFS acls	901
83.12. logon scripts	902
83.13. practice: samba domain controller	903
84. a brief look at samba 4	904
84.1. Samba 4 alpha 6	906

Chapter 75. introduction to samba

This introduction to the Samba server simply explains how to install Samba 3 and briefly mentions the SMB protocol.

75.1. verify installed version

75.1.1. .rpm based distributions

To see the version of samba installed on Red Hat, Fedora or CentOS use **rpm -q samba**.

```
[root@RHEL52 ~]# rpm -q samba  
samba-3.0.28-1.el5_2.1
```

The screenshot above shows that RHEL5 has **Samba** version 3.0 installed. The last number in the Samba version counts the number of updates or patches.

Below the same command on a more recent version of CentOS with Samba version 3.5 installed.

```
[root@centos6 ~]# rpm -q samba  
samba-3.5.10-116.el6_2.i686
```

75.1.2. .deb based distributions

Use **dpkg -l** or **aptitude show** on Debian or Ubuntu. Both Debian 7.0 (Wheezy) and Ubuntu 12.04 (Precise) use version 3.6.3 of the Samba server.

```
root@debian7~# aptitude show samba | grep Version  
Version: 2:3.6.3-1
```

Ubuntu 12.04 is currently at Samba version 3.6.3.

```
root@ubu1204:~# dpkg -l samba | tail -1  
ii samba 2:3.6.3-2ubuntu2.1 SMB/CIFS file, print, and login server for Unix
```

75.2. installing samba

75.2.1. .rpm based distributions

Samba is installed by default on Red Hat Enterprise Linux. If Samba is not yet installed, then you can use the graphical menu (Applications -- System Settings -- Add/Remove Applications) and select "Windows File Server" in the Server section. The non-graphical way is to use **rpm** or **yum**.

When you downloaded the .rpm file, you can install Samba like this.

```
[paul@RHEL52 ~]$ rpm -i samba-3.0.28-1.el5_2.1.rpm
```

When you have a subscription to RHN (Red Hat Network), then **yum** is an easy tool to use. This **yum** command works by default on Fedora and CentOS.

```
[root@centos6 ~]# yum install samba
```

75.2.2. .deb based distributions

Ubuntu and Debian users can use the **aptitude** program (or use a graphical tool like Synaptic).

```
root@debian7~# aptitude install samba
The following NEW packages will be installed:
  samba samba-common{a} samba-common-bin{a} tdb-tools{a}
0 packages upgraded, 4 newly installed, 0 to remove and 1 not upgraded.
Need to get 15.1 MB of archives. After unpacking 42.9 MB will be used.
Do you want to continue? [Y/n/?]
...
```


75.3. documentation

75.3.1. samba howto

Samba comes with excellent documentation in html and pdf format (and also as a free download from samba.org and it is for sale as a printed book).

The documentation is a separate package, so install it if you want it on the server itself.

```
[root@centos6 ~]# yum install samba-doc
...
[root@centos6 ~]# ls -l /usr/share/doc/samba-doc-3.5.10/
total 10916
drwxr-xr-x. 6 root root    4096 May  6 15:50 htmldocs
-rw-r--r--. 1 root root 4605496 Jun 14 2011 Samba3-ByExample.pdf
-rw-r--r--. 1 root root 608260 Jun 14 2011 Samba3-Developers-Guide.pdf
-rw-r--r--. 1 root root 5954602 Jun 14 2011 Samba3-HOWTO.pdf
```

This action is very similar on Ubuntu and Debian except that the pdf files are in a separate package named **samba-doc-pdf**.

```
root@ubu1204:~# aptitude install samba-doc-pdf
The following NEW packages will be installed:
  samba-doc-pdf
...
```

75.3.2. samba by example

Besides the howto, there is also an excellent book called **Samba By Example** (again available as printed edition in shops, and as a free pdf and html).

75.4. starting and stopping samba

You can start the daemons by invoking **/etc/init.d/smb start** (some systems use **/etc/init.d/samba**) on any linux.

```
root@laika:~# /etc/init.d/samba stop
* Stopping Samba daemons [ OK ]
root@laika:~# /etc/init.d/samba start
* Starting Samba daemons [ OK ]
root@laika:~# /etc/init.d/samba restart
* Stopping Samba daemons [ OK ]
* Starting Samba daemons [ OK ]
root@laika:~# /etc/init.d/samba status
* SMBD is running [ OK ]
```

Red Hat derived systems are happy with **service smb start**.

```
[root@RHEL4b ~]# /etc/init.d/smb start
Starting SMB services: [ OK ]
Starting NMB services: [ OK ]
[root@RHEL4b ~]# service smb restart
Shutting down SMB services: [ OK ]
Shutting down NMB services: [ OK ]
Starting SMB services: [ OK ]
Starting NMB services: [ OK ]
[root@RHEL4b ~]#
```

75.5. samba daemons

Samba 3 consists of three daemons, they are named **nmbd**, **smbd** and **winbindd**.

75.5.1. nmbd

The **nmbd** daemon takes care of all the names and naming. It registers and resolves names, and handles browsing. According to the Samba documentation, it should be the first daemon to start.

```
[root@RHEL52 ~]# ps -C nmbd
  PID TTY          TIME CMD
 5681 ?            00:00:00 nmbd
```

75.5.2. smbd

The **smbd** daemon manages file transfers and authentication.

```
[root@RHEL52 ~]# ps -C smbd
  PID TTY          TIME CMD
 5678 ?            00:00:00 smbd
 5683 ?            00:00:00 smbd
```

75.5.3. winbindd

The **winbind daemon** (winbindd) is only started to handle Microsoft Windows domain membership.

Note that **winbindd** is started by the **/etc/init.d/winbind** script (two dd's for the daemon and only one d for the script).

```
[root@RHEL52 ~]# /etc/init.d/winbind start
Starting Winbind services: [ OK ]
[root@RHEL52 ~]# ps -C winbindd
  PID TTY          TIME CMD
 5752 ?            00:00:00 winbindd
 5754 ?            00:00:00 winbindd
```

On Debian and Ubuntu, the winbindd daemon is installed via a separate package called **winbind**.

75.6. the SMB protocol

75.6.1. brief history

Development of this protocol was started by **IBM** in the early eighties. By the end of the eighties, most development was done by **Microsoft**. SMB is an application level protocol designed to run on top of NetBIOS/NetBEUI, but can also be run on top of tcp/ip.

In 1996 Microsoft was asked to document the protocol. They submitted CIFS (Common Internet File System) as an internet draft, but it never got final rfc status.

In 2004 the European Union decided Microsoft should document the protocol to enable other developers to write compatible software. December 20th 2007 Microsoft came to an agreement. The Samba team now has access to SMB/CIFS, Windows for Workgroups and Active Directory documentation.

75.6.2. broadcasting protocol

SMB uses the NetBIOS **service location protocol**, which is a broadcasting protocol. This means that NetBIOS names have to be unique on the network (even when you have different IP-addresses). Having duplicate names on an SMB network can seriously harm communications.

75.6.3. NetBIOS names

NetBIOS names are similar to **hostnames**, but are always uppercase and only 15 characters in length. Microsoft Windows computers and Samba servers will broadcast this name on the network.

75.6.4. network bandwidth

Having many broadcasting SMB/CIFS computers on your network can cause bandwidth issues. A solution can be the use of a **NetBIOS name server** (NBNS) like **WINS** (Windows Internet Naming Service).

75.7. practice: introduction to samba

0. !! Make sure you know your student number, anything **ANYTHING** you name must include your student number!
1. Verify that you can logon to a Linux/Unix computer. Write down the name and ip address of this computer.
2. Do the same for all the other (virtual) machines available to you.
3. Verify networking by pinging the computer, edit the appropriate hosts files so you can use names. Test the names by pinging them.
4. Make sure Samba is installed, write down the version of Samba.
5. Open the Official Samba-3 howto pdf file that is installed on your computer. How many A4 pages is this file ? Then look at the same pdf on samba.org, it is updated regularly.
6. Stop the Samba server.

Chapter 76. getting started with samba

76.1. /etc/samba/smb.conf

76.1.1. smbd -b

Samba configuration is done in the **smb.conf** file. The file can be edited manually, or you can use a web based interface like webmin or swat to manage it. The file is usually located in /etc/samba. You can find the exact location with **smbd -b**.

```
[root@RHEL4b ~]# smbd -b | grep CONFIGFILE
CONFIGFILE: /etc/samba/smb.conf
```

76.1.2. the default smb.conf

The default smb.conf file contains a lot of examples with explanations.

```
[paul@RHEL4b ~]$ ls -l /etc/samba/smb.conf
-rw-r--r-- 1 root root 10836 May 30 23:08 /etc/samba/smb.conf
```

Also on Ubuntu and Debian, smb.conf is packed with samples and explanations.

```
paul@laika:~$ ls -l /etc/samba/smb.conf
-rw-r--r-- 1 root root 10515 2007-05-24 00:21 /etc/samba/smb.conf
```

76.1.3. minimal smb.conf

Below is an example of a very minimalistic **smb.conf**. It allows samba to start, and to be visible to other computers (Microsoft shows computers in Network Neighborhood or My Network Places).

```
[paul@RHEL4b ~]$ cat /etc/samba/smb.conf
[global]
workgroup = WORKGROUP
[firstshare]
path = /srv/samba/public
```

76.1.4. net view

Below is a screenshot of the **net view** command on Microsoft Windows Server 2003 sp2. It shows how a Red Hat Enterprise Linux 5.3 and a Ubuntu 9.04 Samba server, both with a minimalistic smb.conf, are visible to Microsoft computers nearby.

```
C:\Documents and Settings\Administrator>net view
Server Name          Remark
-----
\\LAIKA               Samba 3.3.2
\\RHEL53              Samba 3.0.33-3.7.el5
\\W2003
The command completed successfully.
```

76.1.5. long lines in smb.conf

Some parameters in smb.conf can get a long list of values behind them. You can continue a line (for clarity) on the next by ending the line with a backslash.

```
valid users = Serena, Venus, Lindsay \
```

Kim, Justine, Sabine \
Amelie, Marie, Suzanne

76.1.6. curious smb.conf

Curious but true: smb.conf accepts synonyms like **create mode** and **create mask**, and (sometimes) minor spelling errors like **browsable** and **browseable**. And on occasion you can even switch words, the **guest only** parameter is identical to **only guest**. And **writable** = **yes** is the same as **readonly** = **no**.

76.1.7. man smb.conf

You can access a lot of documentation when typing **man smb.conf**.

```
[root@RHEL4b samba]# apropos samba
cupsaddsmb      (8)  - export printers to samba for windows clients
lmhosts         (5)  - The Samba NetBIOS hosts file
net             (8)  - Tool for administration of Samba and remote CIFS servers
pdbedit        (8)  - manage the SAM database (Database of Samba Users)
samba          (7)  - A Windows SMB/CIFS fileserver for UNIX
smb.conf [smb]  (5)  - The configuration file for the Samba suite
smbpasswd       (5)  - The Samba encrypted password file
smbstatus       (1)  - report on current Samba connections
swat            (8)  - Samba Web Administration Tool
tdbbackup       (8)  - tool for backing up and ... of samba .tdb files
[root@RHEL4b samba]#
```

76.2. /usr/bin/testparm

76.2.1. syntax check smb.conf

To verify the syntax of the smb.conf file, you can use **testparm**.

```
[paul@RHEL4b ~]$ testparm
Load smb config files from /etc/samba/smb.conf
Processing section "[firstshare]"
Loaded services file OK.
Server role: ROLE_STANDALONE
Press enter to see a dump of your service definitions
```

76.2.2. testparm -v

An interesting option is **testparm -v**, which will output all the global options with their default value.

```
[root@RHEL52 ~]# testparm -v | head
Load smb config files from /etc/samba/smb.conf
Processing section "[pub0]"
Processing section "[global$]"
Loaded services file OK.
Server role: ROLE_STANDALONE
Press enter to see a dump of your service definitions

[global]
dos charset = CP850
unix charset = UTF-8
display charset = LOCALE
workgroup = WORKGROUP
```



```
realm =
netbios name = TEACHER0
netbios aliases =
netbios scope =
server string = Samba 3.0.28-1.el5_2.1
...
```

There were about 350 default values for smb.conf parameters in Samba 3.0.x. This number grew to almost 400 in Samba 3.5.x.

76.2.3. testparm -s

The samba daemons are constantly (once every 60 seconds) checking the smb.conf file, so it is good practice to keep this file small. But it is also good practice to document your samba configuration, and to explicitly set options that have the same default values. The **testparm -s** option allows you to do both. It will output the smallest possible samba configuration file, while retaining all your settings. The idea is to have your samba configuration in another file (like smb.conf.full) and let testparm parse this for you. The screenshot below shows you how. First the smb.conf.full file with the explicitly set option workgroup to WORKGROUP.

```
[root@RHEL4b samba]# cat smb.conf.full
[global]
workgroup = WORKGROUP

# This is a demo of a documented smb.conf
# These two lines are removed by testparm -s

server string = Public Test Server

[firstshare]
path = /srv/samba/public
```

Next, we execute testparm with the -s option, and redirect stdout to the real **smb.conf** file.

```
[root@RHEL4b samba]# testparm -s smb.conf.full > smb.conf
Load smb config files from smb.conf.full
Processing section "[firstshare]"
Loaded services file OK.
```

And below is the end result. The two comment lines and the default option are no longer there.

```
[root@RHEL4b samba]# cat smb.conf
# Global parameters
[global]
server string = Public Test Server

[firstshare]
path = /srv/samba/public
[root@RHEL4b samba]#
```

76.3. /usr/bin/smbclient

76.3.1. smbclient looking at Samba

With **smbclient** you can see browsing and share information from your smb server. It will display all your shares, your workgroup, and the name of the Master Browser. The -N switch

is added to avoid having to enter an empty password. The `-L` switch is followed by the name of the host to check.

```
[root@RHEL4b init.d]# smbclient -NL rhel4b
Anonymous login successful
Domain=[WORKGROUP] OS=[Unix] Server=[Samba 3.0.10-1.4E.9]

Sharename      Type      Comment
-----
firstshare     Disk
IPC$           IPC       IPC Service (Public Test Server)
ADMIN$         IPC       IPC Service (Public Test Server)
Anonymous login successful
Domain=[WORKGROUP] OS=[Unix] Server=[Samba 3.0.10-1.4E.9]

Server          Comment
-----
RHEL4B          Public Test Server
WINXP

Workgroup       Master
-----
WORKGROUP       WINXP
```

76.3.2. smbclient anonymous

The screenshot below uses **smbclient** to display information about a remote smb server (in this case a computer with Ubuntu 11.10).

```
root@ubu1110:/etc/samba# testparm smbclient -NL 127.0.0.1
Anonymous login successful
Domain=[LINUXTR] OS=[Unix] Server=[Samba 3.5.11]

Sharename      Type      Comment
-----
share1         Disk
IPC$           IPC       IPC Service (Samba 3.5.11)
Anonymous login successful
Domain=[LINUXTR] OS=[Unix] Server=[Samba 3.5.11]

Server          Comment
-----

Workgroup       Master
-----
LINUXTR         DEBIAN6
WORKGROUP       UBU1110
```

76.3.3. smbclient with credentials

Windows versions after xp sp2 and 2003 sp1 do not accept guest access (the `NT_STATUS_ACCESS_DENIED` error). This example shows how to provide credentials with **smbclient**.

```
[paul@RHEL53 ~]$ smbclient -L w2003 -U administrator%stargate
Domain=[W2003] OS=[Windows Server 2003 3790 Service Pack 2] Server=...

Sharename      Type      Comment
-----
C$             Disk      Default share
```

```
IPC$          IPC      Remote IPC
ADMIN$        Disk     Remote Admin
...
```

76.4. /usr/bin/smbtree

Another useful tool to troubleshoot Samba or simply to browse the SMB network is **smbtree**. In its simplest form, smbtree will do an anonymous browsing on the local subnet, displaying all SMB computers and (if authorized) their shares.

Let's take a look at two screenshots of smbtree in action (with blank password). The first one is taken immediately after booting four different computers (one MS Windows 2000, one MS Windows xp, one MS Windows 2003 and one RHEL 4 with Samba 3.0.10).

```
[paul@RHEL4b ~]$ smbtree
Password:
WORKGROUP
PEGASUS
  \\WINXP
  \\RHEL4B                      Pegasus Domain Member Server
Error connecting to 127.0.0.1 (Connection refused)
cli_full_connection: failed to connect to RHEL4B<20> (127.0.0.1)
  \\HM2003
[paul@RHEL4b ~]$
```

The information displayed in the previous screenshot looks incomplete. The browsing elections are still ongoing, the browse list is not yet distributed to all clients by the (to be elected) browser master. The next screenshot was taken about one minute later. And it shows even less.

```
[paul@RHEL4b ~]$ smbtree
Password:
WORKGROUP
  \\W2000
[paul@RHEL4b ~]$
```

So we wait a while, and then run **smbtree** again, this time it looks a lot nicer.

```
[paul@RHEL4b ~]$ smbtree
Password:
WORKGROUP
  \\W2000
PEGASUS
  \\WINXP
  \\RHEL4B                      Pegasus Domain Member Server
    \\RHEL4B\ADMIN$            IPC Service (Pegasus Domain Member Server)
    \\RHEL4B\IPC$              IPC Service (Pegasus Domain Member Server)
    \\RHEL4B\domaindata        Active Directory users only
  \\HM2003
[paul@RHEL4b ~]$ smbtree --version
Version 3.0.10-1.4E.9
[paul@RHEL4b ~]$
```

I added the version number of **smbtree** in the previous screenshot, to show you the difference when using the latest version of smbtree (below a screenshot taken from Ubuntu Feisty Fawn). The latest version shows a more complete overview of machines and shares.

```
paul@laika:~$ smbtree --version
Version 3.0.24
```

```

paul@laika:~$ smbtree
Password:
WORKGROUP
  \\W2000
    \\W2000\firstshare
    \\W2000\C$           Default share
    \\W2000\ADMIN$       Remote Admin
    \\W2000\IPC$         Remote IPC
PEGASUS
  \\WINXP
cli_rpc_pipe_open: cli_nt_create failed on pipe \srvsvc to machine WINXP.
Error was NT_STATUS_ACCESS_DENIED
  \\RHEL4B               Pegasus Domain Member Server
    \\RHEL4B\ADMIN$       IPC Service (Pegasus Domain Member Server)
    \\RHEL4B\IPC$         IPC Service (Pegasus Domain Member Server)
    \\RHEL4B\domaindata   Active Directory users only
  \\HM2003
cli_rpc_pipe_open: cli_nt_create failed on pipe \srvsvc to machine HM2003.
Error was NT_STATUS_ACCESS_DENIED
paul@laika:~$

```

The previous screenshot also provides useful errors on why we cannot see shared info on computers winxp and w2003. Let us try the old **smbtree** version on our RHEL server, but this time with Administrator credentials (which are the same on all computers).

```

[paul@RHEL4b ~]$ smbtree -UAdministrator%Stargate1
WORKGROUP
  \\W2000
PEGASUS
  \\WINXP
    \\WINXP\C$           Default share
    \\WINXP\ADMIN$       Remote Admin
    \\WINXP\share55
    \\WINXP\IPC$         Remote IPC
  \\RHEL4B               Pegasus Domain Member Server
    \\RHEL4B\ADMIN$       IPC Service (Pegasus Domain Member Server)
    \\RHEL4B\IPC$         IPC Service (Pegasus Domain Member Server)
    \\RHEL4B\domaindata   Active Directory users only
  \\HM2003
    \\HM2003\NETLOGON     Logon server share
    \\HM2003\SYSVOL       Logon server share
    \\HM2003\WSUSTemp     A network share used by Local Publishing ...
    \\HM2003\ADMIN$       Remote Admin
    \\HM2003\tools
    \\HM2003\IPC$         Remote IPC
    \\HM2003\wsusContent  A network share to be used by Local ...
    \\HM2003\C$           Default share
[paul@RHEL4b ~]$

```

As you can see, this gives a very nice overview of all SMB computers and their shares.

76.5. server string

The comment seen by the **net view** and the **smbclient** commands is the default value for the **server string** option. Simply adding this value to the global section in **smb.conf** and restarting samba will change the option.

```

[root@RHEL53 samba]# testparm -s 2>/dev/null | grep server
server string = Red Hat Server in Paris

```

After a short while, the changed option is visible on the Microsoft computers.

```
C:\Documents and Settings\Administrator>net view
Server Name                Remark
-----
\\LAIKA                    Ubuntu 9.04 server in Antwerp
\\RHEL53                   Red Hat Server in Paris
\\W2003
```

76.6. Samba Web Administration Tool (SWAT)

Samba comes with a web based tool to manage your samba configuration file. **SWAT** is accessible with a web browser on port 901 of the host system. To enable the tool, first find out whether your system is using the **inetd** or the **xinetd** superdaemon.

```
[root@RHEL4b samba]# ps fax | grep inet
15026 pts/0    S+   0:00                  \_ grep inet
2771 ?        Ss   0:00 xinetd -stayalive -pidfile /var/run/xinetd.pid
[root@RHEL4b samba]#
```

Then edit the **inetd.conf** or change the `disable = yes` line in **/etc/xinetd.d/swat** to `disable = no`.

```
[root@RHEL4b samba]# cat /etc/xinetd.d/swat
# default: off
# description: SWAT is the Samba Web Admin Tool. Use swat \
#               to configure your Samba server. To use SWAT, \
#               connect to port 901 with your favorite web browser.
service swat
{
    port                = 901
    socket_type         = stream
    wait                = no
    only_from           = 127.0.0.1
    user                = root
    server               = /usr/sbin/swat
    log_on_failure      += USERID
    disable             = no
}
[root@RHEL4b samba]# /etc/init.d/xinetd restart
Stopping xinetd:          [ OK ]
Starting xinetd:          [ OK ]
[root@RHEL4b samba]#
```

Change the **only from** value to enable swat from remote computers. This examples shows how to provide swat access to all computers in a /24 subnet.

```
[root@RHEL53 xinetd.d]# grep only /etc/xinetd.d/swat
only_from = 192.168.1.0/24
```

Be careful when using SWAT, it erases all your manually edited comments in **smb.conf**.

76.7. practice: getting started with samba

1. Take a backup copy of the original smb.conf, name it smb.conf.orig
2. Enable SWAT and take a look at it.
3. Stop the Samba server.
4. Create a minimalistic smb.conf.minimal and test it with testparm.
5. Use testparm -s to create /etc/samba/smb.conf from your smb.conf.minimal .
6. Start Samba with your minimal smb.conf.
7. Verify with smbclient that your Samba server works.
8. Verify that another (Microsoft) computer can see your Samba server.
9. Browse the network with net view, smbtree and with Windows Explorer.
10. Change the "Server String" parameter in smb.conf. How long does it take before you see the change (net view, smbclient, My Network Places,...) ?
11. Will restarting Samba after a change to smb.conf speed up the change ?
12. Which computer is the master browser master in your workgroup ? What is the master browser ?
13. If time permits (or if you are waiting for other students to finish this practice), then install a sniffer (wireshark) and watch the browser elections.

76.8. solution: getting started with samba

1. Take a backup copy of the original smb.conf, name it smb.conf.orig

```
cd /etc/samba ; cp smb.conf smb.conf.orig
```

2. Enable SWAT and take a look at it.

```
on Debian/Ubuntu: vi /etc/inetd.conf (remove # before swat)
```

```
on RHEL/Fedora: vi /etc/xinetd.d/swat (set disable to no)
```

3. Stop the Samba server.

```
/etc/init.d/smb stop (Red Hat)
```

```
/etc/init.d/samba stop (Debian)
```

4. Create a minimalistic smb.conf.minimal and test it with testparm.

```
cd /etc/samba ; mkdir my_smb_confs ; cd my_smb_confs
```

```
vi smb.conf.minimal
```

```
testparm smb.conf.minimal
```

5. Use testparm -s to create /etc/samba/smb.conf from your smb.conf.minimal .

```
testparm -s smb.conf.minimal > ../smb.conf
```

6. Start Samba with your minimal smb.conf.

```
/etc/init.d/smb restart (Red Hat)
```

```
/etc/init.d/samba restart (Debian)
```

7. Verify with smbclient that your Samba server works.

```
smbclient -NL 127.0.0.1
```

8. Verify that another computer can see your Samba server.

```
smbclient -NL 'ip-address' (on a Linux)
```

9. Browse the network with net view, smbtree and with Windows Explorer.

```
on Linux: smbtree
```

```
on Windows: net view (and WindowsKey + e)
```

10. Change the "Server String" parameter in smb.conf. How long does it take before you see the change (net view, smbclient, My Network Places,...) ?

```
vi /etc/samba/smb.conf
```

```
(should take only seconds when restarting samba)
```

11. Will restarting Samba after a change to smb.conf speed up the change ?

```
yes
```

12. Which computer is the master browser master in your workgroup ? What is the master browser ?

The computer that won the elections.

This machine will make the list of computers in the network

13. If time permits (or if you are waiting for other students to finish this practice), then install a sniffer (wireshark) and watch the browser elections.

On ubuntu: `sudo aptitude install wireshark`

then: `sudo wireshark, select interface`

Chapter 77. a read only file server

77.1. Setting up a directory to share

Let's start with setting up a very simple read only file server with Samba. Everyone (even anonymous guests) will receive read access.

The first step is to create a directory and put some test files in it.

```
[root@RHEL52 ~]# mkdir -p /srv/samba/readonly
[root@RHEL52 ~]# cd /srv/samba/readonly/
[root@RHEL52 readonly]# echo "It is cold today." > winter.txt
[root@RHEL52 readonly]# echo "It is hot today." > summer.txt
[root@RHEL52 readonly]# ls -l
total 8
-rw-r--r-- 1 root root 17 Jan 21 05:49 summer.txt
-rw-r--r-- 1 root root 18 Jan 21 05:49 winter.txt
[root@RHEL52 readonly]#
```

77.2. configure the share

77.2.1. smb.conf [global] section

In this example the samba server is a member of WORKGROUP (the default workgroup). We also set a descriptive server string, this string is visible to users browsing the network with net view, windows explorer or smbclient.

```
[root@RHEL52 samba]# head -5 smb.conf
[global]
workgroup = WORKGROUP
server string = Public Anonymous File Server
netbios name = TEACHER0
security = share
```

You might have noticed the line with **security = share**. This line sets the default security mode for our samba server. Setting the security mode to **share** will allow clients (smbclient, any windows, another Samba server, ...) to provide a password for each share. This is one way of using the SMB/CIFS protocol. The other way (called **user mode**) will allow the client to provide a username/password combination, before the server knows which share the client wants to access.

77.2.2. smb.conf [share] section

The share is called pubread and the path is set to our newly created directory. Everyone is allowed access (**guest ok = yes**) and security is set to read only.

```
[pubread]
path = /srv/samba/readonly
comment = files to read
read only = yes
guest ok = yes
```

Here is a very similar configuration on Ubuntu 11.10.

```

root@ubu1110:~# cat /etc/samba/smb.conf
[global]
workgroup = LINUXTR
netbios name = UBU1110
security = share
[roshare1]
path = /srv/samba/readonly
read only = yes
guest ok = yes

```

It doesn't really matter which Linux distribution you use. Below the same config on Debian 6, as good as identical.

```

root@debian6:~# cat /etc/samba/smb.conf
[global]
workgroup = LINUXTR
netbios name = DEBIAN6
security = share
[roshare1]
path = /srv/samba/readonly
read only = yes
guest ok = yes

```

77.3. restart the server

After testing with **testparm**, restart the samba server (so you don't have to wait).

```

[root@RHEL4b readonly]# service smb restart
Shutting down SMB services:                [ OK ]
Shutting down NMB services:                [ OK ]
Starting SMB services:                    [ OK ]
Starting NMB services:                    [ OK ]

```

77.4. verify the share

77.4.1. verify with smbclient

You can now verify the existence of the share with **smbclient**. Our **pubread** is listed as the fourth share.

```

[root@RHEL52 samba]# smbclient -NL 127.0.0.1
Domain=[WORKGROUP] OS=[Unix] Server=[Samba 3.0.33-3.7.el5]

Sharename      Type      Comment
-----
IPC$           IPC       IPC Service (Public Anonymous File Server)
global$       Disk
pub0           Disk
pubread        Disk      files to read
Domain=[WORKGROUP] OS=[Unix] Server=[Samba 3.0.33-3.7.el5]

Server          Comment
-----
TEACHER0       Samba 3.0.33-3.7.el5
W2003EE

Workgroup       Master
-----
WORKGROUP       W2003EE

```

77.4.2. verify on windows

The final test is to go to a Microsoft windows computer and read a file on the Samba server. First we use the **net use** command to mount the pubread share on the driveletter k.

```
C:\>net use K: \\teacher0\pubread
The command completed successfully.
```

Then we test looking at the contents of the share, and reading the files.

```
C:\>dir k:
Volume in drive K is pubread
Volume Serial Number is 0C82-11F2

Directory of K:\

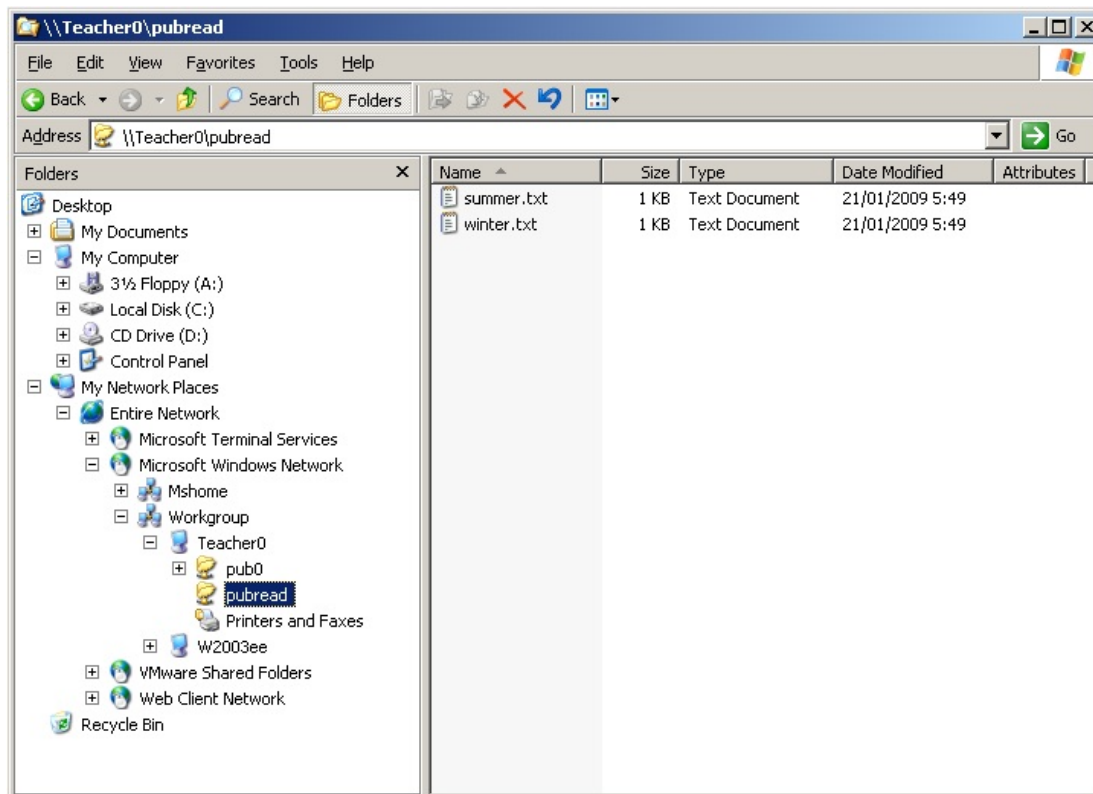
21/01/2009  05:49    <DIR>          .
21/01/2009  05:49    <DIR>          ..
21/01/2009  05:49                  17 summer.txt
21/01/2009  05:49                  18 winter.txt
                2 File(s)                35 bytes
                2 Dir(s)  13.496.242.176 bytes free
```

Just to be on the safe side, let us try writing.

```
K:\>echo very cold > winter.txt
Access is denied.
```

```
K:\>
```

Or you can use windows explorer...



77.5. a note on netcat

The Windows command line screenshot is made in a Linux console, using **netcat** as a pipe to a Windows command shell.

The way this works, is by enabling netcat to listen on the windows computer to a certain port, executing cmd.exe when a connection is received. Netcat is similar to cat, in the way that cat does nothing, only netcat does nothing over the network.

To enable this connection, type the following on the windows computer (after downloading netcat for windows).

```
nc -l -p 23 -t -e cmd.exe
```

And then connect to this machine with netcat from any Linux computer. You end up with a cmd.exe prompt inside your Linux shell.

```
paul@laika:~$ nc 192.168.1.38 23
Microsoft Windows [Version 5.2.3790]
(C) Copyright 1985-2003 Microsoft Corp.

C:\>net use k: /delete
net use k: /delete
k: was deleted successfully.
```

77.6. practice: read only file server

1. Create a directory in a good location (FHS) to share files for everyone to read.
2. Make sure the directory is owned properly and is world accessible.
3. Put a textfile in this directory.
4. Share the directory with Samba.
5. Verify from your own and from another computer (smbclient, net use, ...) that the share is accessible for reading.
6. Make a backup copy of your smb.conf, name it smb.conf.ReadOnlyFileServer.

77.7. solution: read only file server

1. Create a directory in a good location (FHS) to share files for everyone to read.

choose one of these...

```
mkdir -p /srv/samba/readonly
```

```
mkdir -p /home/samba/readonly
```

```
/home/paul/readonly is wrong!!
```

```
/etc/samba/readonly is wrong!!
```

```
/readonly is wrong!!
```

2. Make sure the directory is owned properly and is world accessible.

```
chown root:root /srv/samba/readonly
```

```
chmod 755 /srv/samba/readonly
```

3. Put a textfile in this directory.

```
echo Hello World > hello.txt
```

4. Share the directory with Samba.

You smb.conf.readonly could look like this:

```
[global]
workgroup = WORKGROUP
server string = Read Only File Server
netbios name = STUDENTx
security = share
```

```
[readonlyX]
path = /srv/samba/readonly
comment = read only file share
read only = yes
guest ok = yes
```

test with testparm before going in production!

5. Verify from your own and from another computer (smbclient, net use, ...) that the share is accessible for reading.

On Linux: `smbclient -NL 127.0.0.1`

On Windows Explorer: browse to My Network Places

On Windows cmd.exe: `net use L: //studentx/readonly`

6. Make a backup copy of your smb.conf, name it smb.conf.ReadOnlyFileServer.

```
cp smb.conf smb.conf.ReadOnlyFileServer
```

Chapter 78. a writable file server

78.1. set up a directory to share

In this second example, we will create a share where everyone can create files and write to files. Again, we start by creating a directory

```
[root@RHEL52 samba]# mkdir -p /srv/samba/writable
[root@RHEL52 samba]# chmod 777 /srv/samba/writable/
```

78.2. share section in smb.conf

There are two parameters to make a share writable. We can use **read only** or **writable**. This example shows how to use **writable** to give write access to a share.

```
writable = yes
```

And this is an example of using the **read only** parameter to give write access to a share.

```
read only = no
```

78.3. configure the share

Then we simply add a share to our file server by editing **smb.conf**. Below the check with testparm. (We could have changed the description of the server...)

```
[root@RHEL52 samba]# testparm
Load smb config files from /etc/samba/smb.conf
Processing section "[pubwrite]"
Processing section "[pubread]"
Loaded services file OK.
Server role: ROLE_STANDALONE
Press enter to see a dump of your service definitions

[global]
netbios name = TEACHER0
server string = Public Anonymous File Server
security = SHARE

[pubwrite]
comment = files to write
path = /srv/samba/writable
read only = No
guest ok = Yes

[pubread]
comment = files to read
path = /srv/samba/readonly
guest ok = Yes
```

78.4. test connection with windows

We can now test the connection on a windows 2003 computer. We use the **net use** for this.

```
C:\>net use L: \\teacher0\pubwrite
net use L: \\teacher0\pubwrite
The command completed successfully.
```

78.5. test writing with windows

We mounted the **pubwrite** share on the L: drive in windows. Below we test that we can write to this share.

```
L:\>echo hoi > hoi.txt

L:\>dir
Volume in drive L is pubwrite
Volume Serial Number is 0C82-272A

Directory of L:\

21/01/2009  06:11    <DIR>          .
21/01/2009  06:11    <DIR>          ..
21/01/2009  06:16                6 hoi.txt
               1 File(s)                6 bytes
               2 Dir(s) 13.496.238.080 bytes free
```

78.6. How is this possible ?

Linux (or any Unix) always needs a user account to gain access to a system. The windows computer did not provide the samba server with a user account or a password. Instead, the Linux owner of the files created through this writable share is the Linux guest account (usually named nobody).

```
[root@RHEL52 samba]# ls -l /srv/samba/writable/
total 4
-rwxr--r-- 1 nobody nobody 6 Jan 21 06:16 hoi.txt
```

So this is not the cleanest solution. We will need to improve this.

78.7. practice: writable file server

1. Create a directory and share it with Samba.
2. Make sure everyone can read and write files, test writing with smbclient and from a Microsoft computer.
3. Verify the ownership of files created by (various) users.

78.8. solution: writable file server

1. Create a directory and share it with Samba.

```
mkdir /srv/samba/writable
```

```
chmod 777 /srv/samba/writable
```

the share section in smb.conf can look like this:

```
[pubwrite]
path = /srv/samba/writable
comment = files to write
read only = no
guest ok = yes
```

2. Make sure everyone can read and write files, test writing with smbclient and from a Microsoft computer.

to test writing with smbclient:

```
echo one > count.txt
echo two >> count.txt
echo three >> count.txt
smbclient //localhost/pubwrite
Password:
smb: \> put count.txt
```

3. Verify the ownership of files created by (various) users.

```
ls -l /srv/samba/writable
```

Chapter 79. samba first user account

79.1. creating a samba user

We will create a user for our samba file server and make this user the owner of the directory and all of its files. This anonymous user gets a clear description, but does not get a login shell.

```
[root@RHEL52 samba]# useradd -s /bin/false sambanobody
[root@RHEL52 samba]# usermod -c "Anonymous Samba Access" sambanobody
[root@RHEL52 samba]# passwd sambanobody
Changing password for user sambanobody.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
```

79.2. ownership of files

We can use this user as owner of files and directories, instead of using the root account. This approach is clear and more secure.

```
[root@RHEL52 samba]# chown -R sambanobody:sambanobody /srv/samba/
[root@RHEL52 samba]# ls -al /srv/samba/writable/
total 12
drwxrwxrwx 2 sambanobody sambanobody 4096 Jan 21 06:11 .
drwxr-xr-x 6 sambanobody sambanobody 4096 Jan 21 06:11 ..
-rwxr--r-- 1 sambanobody sambanobody 6 Jan 21 06:16 hoi.txt
```

79.3. /usr/bin/smbpasswd

The sambanobody user account that we created in the previous examples is not yet used by samba. It just owns the files and directories that we created for our shares. The goal of this section is to force ownership of files created through the samba share to belong to our sambanobody user. Remember, our server is still accessible to everyone, nobody needs to know this user account or password. We just want a clean Linux server.

To accomplish this, we first have to tell Samba about this user. We can do this by adding the account to **smbpasswd**.

```
[root@RHEL52 samba]# smbpasswd -a sambanobody
New SMB password:
Retype new SMB password:
Added user sambanobody.
```

79.4. /etc/samba/smbpasswd

To find out where Samba keeps this information (for now), use **smbd -b**. The `PRIVATE_DIR` variable will show you where the smbpasswd database is located.

```
[root@RHEL52 samba]# smbd -b | grep PRIVATE
PRIVATE_DIR: /etc/samba
[root@RHEL52 samba]# ls -l smbpasswd
-rw----- 1 root root 110 Jan 21 06:19 smbpasswd
```

You can use a simple cat to see the contents of the **smbpasswd** database. The sambanobody user does have a password (it is secret).

```
[root@RHEL52 samba]# cat smbpasswd
```

```
sambanobody:503:AE9 ... 9DB309C528E540978:[U ]:LCT-4976B05B:
```

79.5. passdb backend

Note that recent versions of Samba have **tdbsam** as default for the **passdb backend** parameter.

```
root@ubu1110:~# testparm -v 2>/dev/null| grep 'passdb backend'

passdb backend = tdbsam
```

79.6. forcing this user

Now that Samba knows about this user, we can adjust our writable share to force the ownership of files created through it. For this we use the **force user** and **force group** options. Now we can be sure that all files in the Samba writable share are owned by the same sambanobody user.

Below is the renewed definition of our share in smb.conf.

```
[pubwrite]
path = /srv/samba/writable
comment = files to write
force user = sambanobody
force group = sambanobody
read only = no
guest ok = yes
```

When you reconnect to the share and write a file, then this sambanobody user will own the newly created file (and nobody needs to know the password).

79.7. practice: first samba user account

1. Create a user account for use with samba.
2. Add this user to samba's user database.
3. Create a writable shared directory and use the "force user" and "force group" directives to force ownership of files.
4. Test the working of force user with smbclient, net use and Windows Explorer.

79.8. solution: first samba user account

1. Create a user account for use with samba.

```
useradd -s /bin/false smbguest
```

```
usermod -c 'samba guest'
```

```
passwd smbguest
```

2. Add this user to samba's user database.

```
smbpasswd -a smbguest
```

3. Create a writable shared directory and use the "force user" and "force group" directives to force ownership of files.

```
[userwrite]
path = /srv/samba/userwrite
comment = everyone writes files owned by smbguest
read only = no
guest ok = yes
force user = smbguest
force group = smbguest
```

4. Test the working of force user with smbclient, net use and Windows Explorer.

```
ls -l /srv/samba/userwrite (and verify ownership)
```

Chapter 80. samba authentication

80.1. creating the users on Linux

The goal of this example is to set up a file share accessible to a number of different users. The users will need to authenticate with their password before access to this share is granted. We will first create three randomly named users, each with their own password. First we add these users to Linux.

```
[root@RHEL52 ~]# useradd -c "Serena Williams" serena
[root@RHEL52 ~]# useradd -c "Justine Henin" justine
[root@RHEL52 ~]# useradd -c "Martina Hingis" martina
[root@RHEL52 ~]# passwd serena
Changing password for user serena.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
[root@RHEL52 ~]# passwd justine
Changing password for user justine.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
[root@RHEL52 ~]# passwd martina
Changing password for user martina.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
```

80.2. creating the users on samba

Then we add them to the **smbpasswd** file, with the same password.

```
[root@RHEL52 ~]# smbpasswd -a serena
New SMB password:
Retype new SMB password:
Added user serena.
[root@RHEL52 ~]# smbpasswd -a justine
New SMB password:
Retype new SMB password:
Added user justine.
[root@RHEL52 ~]# smbpasswd -a martina
New SMB password:
Retype new SMB password:
Added user martina.
```

80.3. security = user

Remember that we set samba's security mode to share with the **security = share** directive in the [global] section ? Since we now require users to always provide a userid and password for access to our samba server, we will need to change this. Setting **security = user** will require the client to provide samba with a valid userid and password before giving access to a share.

Our [global] section now looks like this.

```
[global]
workgroup = WORKGROUP
netbios name = TEACHER0
server string = Samba File Server
security = user
```

80.4. configuring the share

We add the following [share] section to our smb.conf (and we do not forget to create the directory /srv/samba/authwrite).

```
[authwrite]
path = /srv/samba/authwrite
comment = authenticated users only
read only = no
guest ok = no
```

80.5. testing access with net use

After restarting samba, we test with different users from within Microsoft computers. The screenshots use the **net use** First serena from Windows XP.

```
C:\>net use m: \\teacher0\authwrite stargate /user:serena
The command completed successfully.

C:\>m:

M:\>echo greetings from Serena > serena.txt
```

The next screenshot is martina on a Windows 2000 computer, she succeeds in writing her files, but fails to overwrite the file from serena.

```
C:\>net use k: \\teacher0\authwrite stargate /user:martina
The command completed successfully.

C:\>k:

K:\>echo greetings from martina > Martina.txt

K:\>echo test overwrite > serena.txt
Access is denied.
```

80.6. testing access with smbclient

You can also test connecting with authentication with **smbclient**. First we test with a wrong password.

```
[root@RHEL52 samba]# smbclient //teacher0/authwrite -U martina wrongpass
session setup failed: NT_STATUS_LOGON_FAILURE
```

Then we test with the correct password, and verify that we can access a file on the share.

```
[root@RHEL52 samba]# smbclient //teacher0/authwrite -U martina stargate
Domain=[TEACHER0] OS=[Unix] Server=[Samba 3.0.33-3.7.el5]
smb: \> more serena.txt
getting file \serena.txt of size 14 as /tmp/smbmore.QQfmSN (6.8 kb/s)
one
two
three
smb: \> q
```

80.7. verify ownership

We now have a simple standalone samba file server with authenticated access. And the files in the shares belong to their proper owners.

```
[root@RHEL52 samba]# ls -l /srv/samba/authwrite/
total 8
-rwxr--r-- 1 martina martina  0 Jan 21 20:06 martina.txt
-rwxr--r-- 1 serena  serena  14 Jan 21 20:06 serena.txt
-rwxr--r-- 1 serena  serena   6 Jan 21 20:09 ser.txt
```

80.8. common problems

80.8.1. NT_STATUS_BAD_NETWORK_NAME

You can get `NT_STATUS_BAD_NETWORK_NAME` when you forget to create the target directory.

```
[root@RHEL52 samba]# rm -rf /srv/samba/authwrite/
[root@RHEL52 samba]# smbclient //teacher0/authwrite -U martina stargate
Domain=[TEACHER0] OS=[Unix] Server=[Samba 3.0.33-3.7.el5]
tree connect failed: NT_STATUS_BAD_NETWORK_NAME
```

80.8.2. NT_STATUS_LOGON_FAILURE

You can get `NT_STATUS_LOGON_FAILURE` when you type the wrong password or when you type an unexisting username.

```
[root@RHEL52 samba]# smbclient //teacher0/authwrite -U martina STARGATE
session setup failed: NT_STATUS_LOGON_FAILURE
```

80.8.3. usernames are (not) case sensitive

Remember that usernames on Linux are case sensitive.

```
[root@RHEL52 samba]# su - MARTINA
su: user MARTINA does not exist
```

```
[root@RHEL52 samba]# su - martina  
[martina@RHEL52 ~]$
```

But usernames on Microsoft computers are not case sensitive.

```
[root@RHEL52 samba]# smbclient //teacher0/authwrite -U martina stargate  
Domain=[TEACHER0] OS=[Unix] Server=[Samba 3.0.33-3.7.el5]  
smb: \> q  
[root@RHEL52 samba]# smbclient //teacher0/authwrite -U MARTINA stargate  
Domain=[TEACHER0] OS=[Unix] Server=[Samba 3.0.33-3.7.el5]  
smb: \> q
```

80.9. practice : samba authentication

0. Make sure you have properly named backups of your smb.conf of the previous practices.
1. Create three users (on the Linux and on the samba), remember their passwords!
2. Set up a shared directory that is only accessible to authenticated users.
3. Use smbclient and a windows computer to access your share, use more than one user account (windows requires a logoff/logon for this).
4. Verify that files created by these users belong to them.
5. Try to change or delete a file from another user.

80.10. solution: samba authentication

1. Create three users (on the Linux and on the samba), remember their passwords!

```
useradd -c 'SMB user1' userx
```

```
passwd userx
```

2. Set up a shared directory that is only accessible to authenticated users.

The shared section in smb.conf could look like this:

```
[authwrite]
path = /srv/samba/authwrite
comment = authenticated users only
read only = no
guest ok = no
```

3. Use smbclient and a windows computer to access your share, use more than one user account (windows requires a logoff/logon for this).

```
on Linux: smbclient //studentX/authwrite -U user1 password
```

```
on windows net use p: \\studentX\authwrite password /user:user2
```

4. Verify that files created by these users belong to them.

```
ls -l /srv/samba/authwrite
```

5. Try to change or delete a file from another user.

you should not be able to change or overwrite files from others.

Chapter 81. samba securing shares

81.1. security based on user name

81.1.1. valid users

To restrict users per share, you can use the **valid users** parameter. In the example below, only the users listed as valid will be able to access the tennis share.

```
[tennis]
path = /srv/samba/tennis
comment = authenticated and valid users only
read only = No
guest ok = No
valid users = serena, kim, venus, justine
```

81.1.2. invalid users

If you are paranoia, you can also use **invalid users** to explicitly deny the listed users access. When a user is in both lists, the user has no access!

```
[tennis]
path = /srv/samba/tennis
read only = No
guest ok = No
valid users = kim, serena, venus, justine
invalid users = venus
```

81.1.3. read list

On a writable share, you can set a list of read only users with the **read list** parameter.

```
[football]
path = /srv/samba/football
read only = No
guest ok = No
read list = martina, roberto
```

81.1.4. write list

Even on a read only share, you can set a list of users that can write. Use the **write list** parameter.

```
[football]
path = /srv/samba/golf
read only = Yes
guest ok = No
write list = eddy, jan
```

81.2. security based on ip-address

81.2.1. hosts allow

The **hosts allow** or **allow hosts** parameter is one of the key advantages of Samba. It allows access control of shares on the ip-address level. To allow only specific hosts to access a share, list the hosts, separated by comma's.

```
allow hosts = 192.168.1.5, 192.168.1.40
```

Allowing entire subnets is done by ending the range with a dot.

```
allow hosts = 192.168.1.
```

Subnet masks can be added in the classical way.

```
allow hosts = 10.0.0.0/255.0.0.0
```

You can also allow an entire subnet with exceptions.

```
hosts allow = 10. except 10.0.0.12
```

81.2.2. hosts deny

The **hosts deny** or **deny hosts** parameter is the logical counterpart of the previous. The syntax is the same as for hosts allow.

```
hosts deny = 192.168.1.55, 192.168.1.56
```

81.3. security through obscurity

81.3.1. hide unreadable

Setting **hide unreadable** to yes will prevent users from seeing files that cannot be read by them.

```
hide unreadable = yes
```

81.3.2. browsable

Setting the **browseable = no** directive will hide shares from My Network Places. But it will not prevent someone from accessing the share (when the name of the share is known).

Note that **browsable** and **browseable** are both correct syntax.

```
[pubread]
path = /srv/samba/readonly
comment = files to read
read only = yes
guest ok = yes
browseable = no
```

81.4. file system security

81.4.1. create mask

You can use **create mask** and **directory mask** to set the maximum allowed permissions for newly created files and directories. The mask you set is an AND mask (it takes permissions away).

```
[tennis]
path = /srv/samba/tennis
read only = No
```

```
guest ok = No
create mask = 640
directory mask = 750
```

81.4.2. force create mode

Similar to **create mask**, but different. Where the mask from above was a logical AND, the mode you set here is a logical OR (so it adds permissions). You can use the **force create mode** and **force directory mode** to set the minimal required permissions for newly created files and directories.

```
[tennis]
path = /srv/samba/tennis
read only = No
guest ok = No
force create mode = 444
force directory mode = 550
```

81.4.3. security mask

The **security mask** and **directory security mask** work in the same way as **create mask** and **directory mask**, but apply only when a windows user is changing permissions using the windows security dialog box.

81.4.4. force security mode

The **force security mode** and **force directory security mode** work in the same way as **force create mode** and **force directory mode**, but apply only when a windows user is changing permissions using the windows security dialog box.

81.4.5. inherit permissions

With **inherit permissions = yes** you can force newly created files and directories to inherit permissions from their parent directory, overriding the create mask and directory mask settings.

```
[authwrite]
path = /srv/samba/authwrite
comment = authenticated users only
read only = no
guest ok = no
create mask = 600
directory mask = 555
inherit permissions = yes
```

81.5. practice: securing shares

1. Create a writable share called sales, and a readonly share called budget. Test that it works.
2. Limit access to the sales share to ann, sandra and veronique.
3. Make sure that roberto cannot access the sales share.
4. Even though the sales share is writable, ann should only have read access.
5. Even though the budget share is read only, sandra should also have write access.
6. Limit one shared directory to the 192.168.1.0/24 subnet, and another share to the two computers with ip-addresses 192.168.1.33 and 172.17.18.19.
7. Make sure the computer with ip 192.168.1.203 cannot access the budget share.
8. Make sure (on the budget share) that users can see only files and directories to which they have access.
9. Make sure the sales share is not visible when browsing the network.
10. All files created in the sales share should have 640 permissions or less.
11. All directories created in the budget share should have 750 permissions or more.
12. Permissions for files on the sales share should never be set more than 664.
13. Permissions for files on the budget share should never be set less than 500.
14. If time permits (or if you are waiting for other students to finish this practice), then combine the "read only" and "writable" statements to check which one has priority.
15. If time permits then combine "read list", "write list", "hosts allow" and "hosts deny". Which of these has priority ?

81.6. solution: securing shares

1. Create a writable share called sales, and a readonly share called budget. Test that it works.

see previous solutions on how to do this...

2. Limit access to the sales share to ann, sandra and veronique.

```
valid users = ann, sandra, veronique
```

3. Make sure that roberto cannot access the sales share.

```
invalid users = roberto
```

4. Even though the sales share is writable, ann should only have read access.

```
read list = ann
```

5. Even though the budget share is read only, sandra should also have write access.

```
write list = sandra
```

6. Limit one shared directory to the 192.168.1.0/24 subnet, and another share to the two computers with ip-addresses 192.168.1.33 and 172.17.18.19.

```
hosts allow = 192.168.1.
```

```
hosts allow = 192.168.1.33, 172.17.18.19
```

7. Make sure the computer with ip 192.168.1.203 cannot access the budget share.

```
hosts deny = 192.168.1.203
```

8. Make sure (on the budget share) that users can see only files and directories to which they have access.

```
hide unreadable = yes
```

9. Make sure the sales share is not visible when browsing the network.

```
browsable = no
```

10. All files created in the sales share should have 640 permissions or less.

```
create mask = 640
```

11. All directories created in the budget share should have 750 permissions or more.

```
force directory mode = 750
```

12. Permissions for files on the sales share should never be set more than 664.

```
security mask = 750
```

13. Permissions for files on the budget share should never be set less than 500.

```
force security directory mask = 500
```

14. If time permits (or if you are waiting for other students to finish this practice), then combine the "read only" and "writable" statements to check which one has priority.

15. If time permits then combine "read list", "write list", "hosts allow" and "hosts deny". Which of these has priority ?

Chapter 82. samba domain member

82.1. changes in smb.conf

82.1.1. workgroup

The **workgroup** option in the global section should match the netbios name of the Active Directory domain.

```
workgroup = STARGATE
```

82.1.2. security mode

Authentication will not be handled by samba now, but by the Active Directory domain controllers, so we set the **security** option to domain.

```
security = Domain
```

82.1.3. Linux uid's

Linux requires a user account for every user accessing its file system, we need to provide Samba with a range of uid's and gid's that it can use to create these user accounts. The range is determined with the **idmap uid** and the **idmap gid** parameters. The first Active Directory user to connect will receive Linux uid 20000.

```
idmap uid = 20000-22000  
idmap gid = 20000-22000
```

82.1.4. winbind use default domain

The **winbind use default domain** parameter makes sure winbind also operates on users without a domain component in their name.

```
winbind use default domain = yes
```

82.1.5. [global] section in smb.conf

Below is our new global section in **smb.conf**.

```
[global]  
workgroup = STARGATE  
security = Domain  
server string = Stargate Domain Member Server  
idmap uid = 20000-22000  
idmap gid = 20000-22000  
winbind use default domain = yes
```

82.1.6. realm in /etc/krb5.conf

To connect to a Windows 2003 sp2 (or later) you will need to adjust the kerberos realm in **/etc/krb5.conf** and set both lookup statements to true.

```
[libdefaults]
default_realm = STARGATE.LOCAL
dns_lookup_realm = true
dns_lookup_kdc = true
```

82.1.7. [share] section in smb.conf

Nothing special is required for the share section in smb.conf. Remember that we do not manually create users in smbpasswd or on the Linux (/etc/passwd). Only Active Directory users are allowed access.

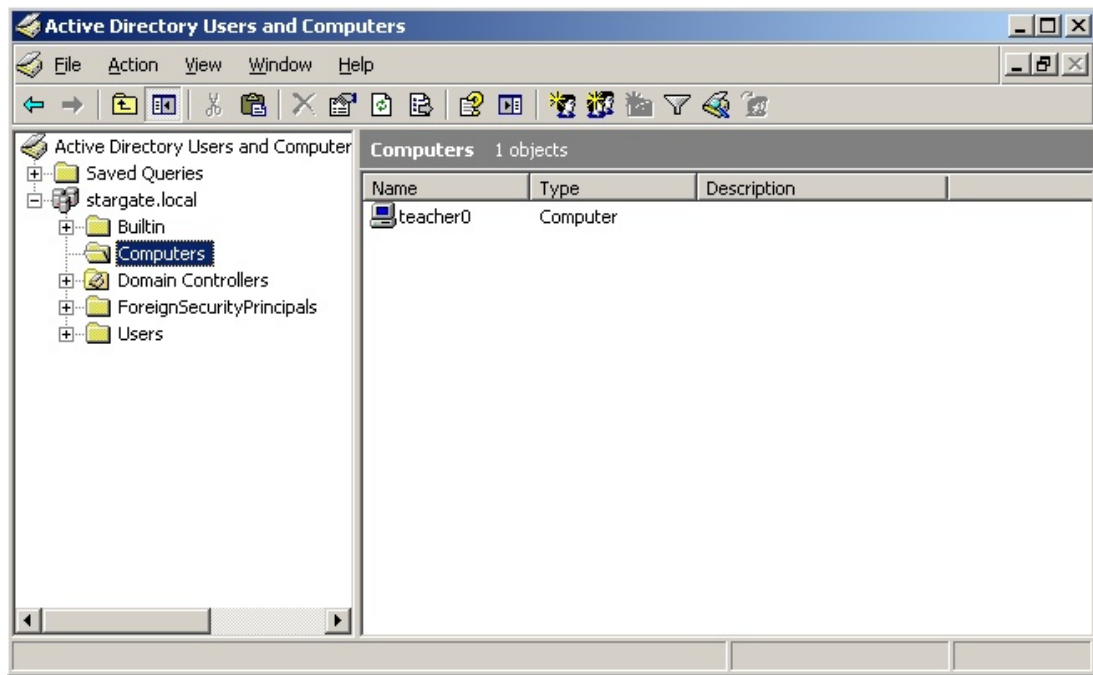
```
[domaingroup]
path = /srv/samba/domaingroup
comment = Active Directory users only
read only = No
```

82.2. joining an Active Directory domain

While the Samba server is stopped, you can use **net rpc join** to join the Active Directory domain.

```
[root@RHEL52 samba]# service smb stop
Shutting down SMB services:                [ OK ]
Shutting down NMB services:                [ OK ]
[root@RHEL52 samba]# net rpc join -U Administrator
Password:
Joined domain STARGATE.
```

We can verify in the aduc (Active Directory Users and Computers) that a computer account is created for this samba server.



82.3. winbind

82.3.1. adding winbind to nsswitch.conf

The **winbind daemon** is talking with the Active Directory domain.

We need to update the **/etc/nsswitch.conf** file now, so user group and host names can be resolved against the winbind daemon.

```
[root@RHEL52 samba]# vi /etc/nsswitch.conf
[root@RHEL52 samba]# grep winbind /etc/nsswitch.conf
passwd:      files winbind
group:       files winbind
hosts:       files dns winbind
```

82.3.2. starting samba and winbindd

Time to start Samba followed by **winbindd**.

```
[root@RHEL4b samba]# service smb start
Starting SMB services: [ OK ]
Starting NMB services: [ OK ]
[root@RHEL4b samba]# service winbind start
Starting winbindd services: [ OK ]
[root@RHEL4b samba]#
```

82.4. wbinfo

82.4.1. verify the trust

You can use **wbinfo -t** to verify the trust between your samba server and Active Directory.

```
[root@RHEL52 ~]# wbinfo -t
checking the trust secret via RPC calls succeeded
```

82.4.2. list all users

We can obtain a list of all user with the **wbinfo -u** command. The domain is not shown when the **winbind use default domain** parameter is set.

```
[root@RHEL52 ~]# wbinfo -u
TEACHER0\serena
TEACHER0\justine
TEACHER0\martina
STARGATE\administrator
STARGATE\guest
STARGATE\support_388945a0
STARGATE\pol
STARGATE\krbtgt
STARGATE\arthur
STARGATE\harry
```

82.4.3. list all groups

We can obtain a list of all domain groups with the **wbinfo -g** command. The domain is not shown when the **winbind use default domain** parameter is set.

```
[root@RHEL52 ~]# wbinfo -g
BUILTIN\administrators
BUILTIN\users
BATMAN\domain computers
BATMAN\domain controllers
BATMAN\schema admins
BATMAN\enterprise admins
BATMAN\domain admins
BATMAN\domain users
BATMAN\domain guests
BATMAN\group policy creator owners
BATMAN\dnsupdateproxy
```

82.4.4. query a user

We can use **wbinfo -a** to verify authentication of a user against Active Directory. Assuming a user account **harry** with password **stargate** is just created on the Active Directory, we get the following screenshot.

```
[root@RHEL52 ~]# wbinfo -a harry%stargate
plaintext password authentication succeeded
challenge/response password authentication succeeded
```

82.5. getent

We can use **getent** to verify that winbindd is working and actually adding the Active directory users to `/etc/passwd`.

```
[root@RHEL52 ~]# getent passwd harry
harry:*:20000:20008:harry potter:/home/BATMAN/harry:/bin/false
[root@RHEL52 ~]# getent passwd arthur
arthur:*:20001:20008:arthur dent:/home/BATMAN/arthur:/bin/false
[root@RHEL52 ~]# getent passwd bilbo
bilbo:*:20002:20008:bilbo baggins:/home/BATMAN/bilbo:/bin/false
```

If the user already exists locally, then the local user account is shown. This is because winbind is configured in **/etc/nsswitch.conf** after **files**.

```
[root@RHEL52 ~]# getent passwd paul
paul:x:500:500:Paul Cobbaut:/home/paul:/bin/bash
```

All the Active Directory users can now easily connect to the Samba share. Files created by them, belong to them.

82.6. file ownership

```
[root@RHEL4b samba]# ll /srv/samba/domaindata/
total 0
-rwxr--r--  1 justine 20000 0 Jun 22 19:54 create_by_justine_on_winxp.txt
-rwxr--r--  1 venus   20000 0 Jun 22 19:55 create_by_venus.txt
-rwxr--r--  1 maria   20000 0 Jun 22 19:57 Maria.txt
```

82.7. practice : samba domain member

1. Verify that you have a working Active Directory (AD) domain.
2. Add the domain name and domain controller to `/etc/hosts`. Set the AD-DNS in `/etc/resolv.conf`.
3. Setup Samba as a member server in the domain.
4. Verify the creation of a computer account in AD for your Samba server.
5. Verify the automatic creation of AD users in `/etc/passwd` with `wbinfo` and `getent`.
6. Connect to Samba shares with AD users, and verify ownership of their files.

Chapter 83. samba domain controller

83.1. about Domain Controllers

83.1.1. Windows NT4

Windows NT4 works with single master replication domain controllers. There is exactly one PDC (Primary Domain Controller) in the domain, and zero or more BDC's (Backup Domain Controllers). Samba 3 has all features found in Windows NT4 PDC and BDC, and more. This includes file and print serving, domain control with single logon, logon scripts, home directories and roaming profiles.

83.1.2. Windows 200x

With Windows 2000 came Active Directory. AD includes multimaster replication and group policies. Samba 3 can only be a member server in Active Directory, it cannot manage group policies. Samba 4 can do this (in beta).

83.1.3. Samba 3

Samba 3 can act as a domain controller in its own domain. In a Windows NT4 domain, with one Windows NT4 PDC and zero or more BDC's, Samba 3 can only be a member server. The same is valid for Samba 3 in an Active Directory Domain. In short, a Samba 3 domain controller can not share domain control with Windows domain controllers.

83.1.4. Samba 4

Samba 4 can be a domain controller in an Active Directory domain, including managing group policies. As of this writing, Samba 4 is not released for production!

83.2. About security modes

83.2.1. security = share

The 'Windows for Workgroups' way of working, a client requests connection to a share and provides a password for that connection. Anyone who knows a password for a share can access that share. This security model was common in Windows 3.11, Windows 95, Windows 98 and Windows ME.

83.2.2. security = user

The client will send a userid + password before the server knows which share the client wants to access. This mode should be used whenever the samba server is in control of the user database. Both for standalone and samba domain controllers.

83.2.3. security = domain

This mode will allow samba to verify user credentials using NTLM in Windows NT4 and in all Active Directory domains. This is similar to Windows NT4 BDC's joining a native Windows 2000/3 Active Directory domain.

83.2.4. security = ads

This mode will make samba use Kerberos to connect to the Active Directory domain.

83.2.5. security = server

This mode is obsolete, it can be used to forward authentication to another server.

83.3. About password backends

The previous chapters all used the **smbpasswd** user database. For domain control we opt for the **tdbsam** password backend. Another option would be to use LDAP. Larger domains will benefit from using LDAP instead of the not so scalable tdbsam. When you need more than one Domain Controller, then the Samba team advises to not use tdbsam.

83.4. [global] section in smb.conf

Now is a good time to start adding comments in your smb.conf. First we will take a look at the naming of our domain and server in the **[global]** section, and at the domain controlling parameters.

83.4.1. security

The security must be set to user (which is the default). This mode will make samba control the user accounts, so it will allow samba to act as a domain controller.

```
security = user
```

83.4.2. os level

A samba server is the most stable computer in the network, so it should win all browser elections (**os level** above 32) to become the **browser master**

```
os level = 33
```

83.4.3. passdb backend

The **passdb backend** parameter will determine whether samba uses **smbpasswd**, **tdbsam** or **ldap**.

```
passdb backend = tdbsam
```

83.4.4. preferred master

Setting the **preferred master** parameter to yes will make the nmbd daemon force an election on startup.

```
preferred master = yes
```

83.4.5. domain logons

Setting the **domain logons** parameter will make this samba server a domain controller.

```
domain logons = yes
```

83.4.6. domain master

Setting the **domain master** parameter can cause samba to claim the **domain master browser** role for its workgroup. Don't use this parameter in a workgroup with an active NT4 PDC.

```
domain master = yes
```

83.4.7. [global] section

The screenshot below shows a sample [global] section for a samba domain controller.

```
[global]
# names
workgroup = SPORTS
netbios name = DCSPORTS
server string = Sports Domain Controller
# domain control parameters
security = user
os level = 33
preferred master = Yes
domain master = Yes
domain logons = Yes
```

83.5. netlogon share

Part of the microsoft definition for a domain controller is that it should have a **netlogon share**. This is the relevant part of smb.conf to create this netlogon share on Samba.

```
[netlogon]
comment = Network Logon Service
path = /srv/samba/netlogon
admin users = root
guest ok = Yes
browseable = No
```

83.6. other [share] sections

We create some sections for file shares, to test the samba server. Users can all access the general sports file share, but only group members can access their own sports share.

```
[sports]
comment = Information about all sports
path = /srv/samba/sports
valid users = @ntsports
read only = No

[tennis]
comment = Information about tennis
path = /srv/samba/tennis
valid users = @nttennis
read only = No
```

```
[football]
comment = Information about football
path = /srv/samba/football
valid users = @ntfootball
read only = No
```

83.7. Users and Groups

To be able to use users and groups in the samba domain controller, we can first set up some groups on the Linux computer.

```
[root@RHEL52 samba]# groupadd ntadmins
[root@RHEL52 samba]# groupadd ntsports
[root@RHEL52 samba]# groupadd ntfootball
[root@RHEL52 samba]# groupadd nttennis
```

This enables us to add group membership info to some new users for our samba domain. Don't forget to give them a password.

```
[root@RHEL52 samba]# useradd -m -G ntadmins Administrator
[root@RHEL52 samba]# useradd -m -G ntsports,nttennis venus
[root@RHEL52 samba]# useradd -m -G ntsports,nttennis kim
[root@RHEL52 samba]# useradd -m -G ntsports,nttennis jelena
[root@RHEL52 samba]# useradd -m -G ntsports,ntfootball figo
[root@RHEL52 samba]# useradd -m -G ntsports,ntfootball ronaldo
[root@RHEL52 samba]# useradd -m -G ntsports,ntfootball pfaff
```

It is always safe to verify creation of users, groups and passwords in `/etc/passwd`, `/etc/shadow` and `/etc/group`.

```
[root@RHEL52 samba]# tail -11 /etc/group
ntadmins:x:507:Administrator
ntsports:x:508:venus,kim,jelena,figo,ronaldo,pfaff
ntfootball:x:509:figo,ronaldo,pfaff
nttennis:x:510:venus,kim,jelena
Administrator:x:511:
venus:x:512:
kim:x:513:
jelena:x:514:
figo:x:515:
ronaldo:x:516:
pfaff:x:517:
```

83.8. tdbsam

Next we must make these users known to samba with the `smbpasswd` tool. When you add the first user to **tdbsam**, the file `/etc/samba/passdb.tdb` will be created.

```
[root@RHEL52 samba]# smbpasswd -a root
New SMB password:
```

```
Retype new SMB password:
tdbsam_open: Converting version 0 database to version 3.
Added user root.
```

Adding all the other users generates less output, because tdbsam is already created.

```
[root@RHEL4b samba]# smbpasswd -a root
New SMB password:
Retype new SMB password:
Added user root.
```

83.9. about computer accounts

Every NT computer (Windows NT, 2000, XP, Vista) can become a member of a domain. Joining the domain (by right-clicking on My Computer) means that a computer account will be created in the domain. This computer account also has a password (but you cannot know it) to prevent other computers with the same name from accidentally becoming member of the domain. The computer account created by Samba is visible in the **/etc/passwd** file on Linux. Computer accounts appear as a normal user account, but end their name with a dollar sign. Below a screenshot of the windows 2003 computer account, created by Samba 3.

```
[root@RHEL52 samba]# tail -5 /etc/passwd
jelena:x:510:514::/home/jelena:/bin/bash
figo:x:511:515::/home/figo:/bin/bash
ronaldo:x:512:516::/home/ronaldo:/bin/bash
pfaff:x:513:517::/home/pfaff:/bin/bash
w2003ee$:x:514:518::/home/nobody:/bin/false
```

To be able to create the account, you will need to provide credentials of an account with the permission to create accounts (by default only root can do this on Linux). And we will have to tell Samba how to do this, by adding an **add machine script** to the global section of smb.conf.

```
add machine script = /usr/sbin/useradd -s /bin/false -d /home/nobody %u
```

You can now join a Microsoft computer to the sports domain (with the root user). After reboot of the Microsoft computer, you will be able to logon with Administrator (password Stargate1), but you will get an error about your roaming profile. We will fix this in the next section.

When joining the samba domain, you have to enter the credentials of a Linux account that can create users (usually only root can do this). If the Microsoft computer complains with **The parameter is incorrect**, then you possibly forgot to add the **add machine script**.

83.10. local or roaming profiles

For your information, if you want to force local profiles instead of roaming profiles, then simply add the following two lines to the global section in smb.conf.

```
logon home =
logon path =
```

Microsoft computers store a lot of User Metadata and application data in a user profile. Making this profile available on the network will enable users to keep their Desktop and Application settings across computers. User profiles on the network are called **roaming profiles** or **roving profiles**. The Samba domain controller can manage these profiles. First we need to add the relevant section in smb.conf.

```
[Profiles]
comment = User Profiles
path = /srv/samba/profiles
readonly = No
profile acls = Yes
```

Besides the share section, we also need to set the location of the profiles share (this can be another Samba server) in the global section.

```
logon path = \\%L\Profiles\%U
```

The **%L** variable is the name of this Samba server, the **%U** variable translates to the username. After adding a user to smbpasswd and letting the user log on and off, the profile of the user will look like this.

```
[root@RHEL4b samba]# ll /srv/samba/profiles/Venus/
total 568
drwxr-xr-x  4 Venus Venus   4096 Jul  5 10:03 Application Data
drwxr-xr-x  2 Venus Venus   4096 Jul  5 10:03 Cookies
drwxr-xr-x  3 Venus Venus   4096 Jul  5 10:03 Desktop
drwxr-xr-x  3 Venus Venus   4096 Jul  5 10:03 Favorites
drwxr-xr-x  4 Venus Venus   4096 Jul  5 10:03 My Documents
drwxr-xr-x  2 Venus Venus   4096 Jul  5 10:03 NetHood
-rwxr--r--  1 Venus Venus 524288 Jul  5  2007 NTUSER.DAT
-rwxr--r--  1 Venus Venus  1024 Jul  5  2007 NTUSER.DAT.LOG
-rw-r--r--  1 Venus Venus   268 Jul  5 10:03 ntuser.ini
drwxr-xr-x  2 Venus Venus   4096 Jul  5 10:03 PrintHood
drwxr-xr-x  2 Venus Venus   4096 Jul  5 10:03 Recent
drwxr-xr-x  2 Venus Venus   4096 Jul  5 10:03 SendTo
drwxr-xr-x  3 Venus Venus   4096 Jul  5 10:03 Start Menu
drwxr-xr-x  2 Venus Venus   4096 Jul  5 10:03 Templates
```

83.11. Groups in NTFS acls

We have users on Unix, we have groups on Unix that contain those users.

```
[root@RHEL4b samba]# grep nt /etc/group
...
ntadmins:x:506:Administrator
ntsports:x:507:Venus,Serena,Kim,Figo,Pfaff
nttennis:x:508:Venus,Serena,Kim
ntfootball:x:509:Figo,Pfaff
```

```
[root@RHEL4b samba]#
```

We already added Venus to the **tdbsam** with **smbpasswd**.

```
smbpasswd -a Venus
```

Does this mean that Venus can access the tennis and the sports shares ? Yes, all access works fine on the Samba server. But the nttennis group is not available on the windows machines. To make the groups available on windows (like in the ntfs security tab of files and folders), we have to map unix groups to windows groups. To do this, we use the **net groupmap** command.

```
[root@RHEL4b samba]# net groupmap add ntgroup="tennis" unixgroup=nttennis type=d
No rid or sid specified, choosing algorithmic mapping
Successfully added group tennis to the mapping db
[root@RHEL4b samba]# net groupmap add ntgroup="football" unixgroup=ntfootball type=d
No rid or sid specified, choosing algorithmic mapping
Successfully added group football to the mapping db
[root@RHEL4b samba]# net groupmap add ntgroup="sports" unixgroup=ntsports type=d
No rid or sid specified, choosing algorithmic mapping
Successfully added group sports to the mapping db
[root@RHEL4b samba]#
```

Now you can use the Samba groups on all NTFS volumes on members of the domain.

83.12. logon scripts

Before testing a logon script, make sure it has the proper carriage returns that DOS files have.

```
[root@RHEL4b netlogon]# cat start.bat
net use Z: \\DCSPORTS0\SPORTS
[root@RHEL4b netlogon]# unix2dos start.bat
unix2dos: converting file start.bat to DOS format ...
[root@RHEL4b netlogon]#
```

Then copy the scripts to the netlogon share, and add the following parameter to smb.conf.

```
logon script = start.bat
```

83.13. practice: samba domain controller

1. Setup Samba as a domain controller.
2. Create the shares salesdata, salespresentations and meetings. Salesdata must be accessible to all sales people and to all managers. SalesPresentations is only for all sales people. Meetings is only accessible to all managers. Use groups to accomplish this.
3. Join a Microsoft computer to your domain. Verify the creation of a computer account in /etc/passwd.
4. Setup and verify the proper working of roaming profiles.
5. Find information about home directories for users, set them up and verify that users receive their home directory mapped under the H:-drive in MS Windows Explorer.
6. Use a couple of samba domain groups with members to set acls on ntfs. Verify that it works!
7. Knowing that the %m variable contains the computername, create a separate log file for every computer(account).
8. Knowing that %s contains the client operating system, include a smb.%s.conf file that contains a share. (The share will only be visible to clients with that OS).
9. If time permits (or if you are waiting for other students to finish this practice), then combine "valid users" and "invalid users" with groups and usernames with "hosts allow" and "hosts deny" and make a table of which get priority over which.

Chapter 84. a brief look at samba 4

84.1. Samba 4 alpha 6

A quick view on Samba 4 alpha 6 (January 2009). You can also follow this guide <http://wiki.samba.org/index.php/Samba4/HOWTO>

Remove old Samba from Red Hat

```
yum remove samba
```

set a fix ip address (Red Hat has an easy GUI)

download and untar

```
samba.org, click 'download info', choose mirror, dl samba4 latest alpha
```

once untarred, enter the directory and read the howto4.txt

```
cd samba-4.0.0alpha6/
```

```
more howto4.txt
```

first we have to configure, compile and install samba4

```
cd source4/
```

```
./configure
```

```
make
```

```
make install
```

Then we can use the provision script to setup our realm. I used booi.schot as domain name (instead of example.com).

```
./setup/provision --realm=B00I.SCHOT --domain=B00I --adminpass=stargate \  
--server-role='domain controller'
```

i added a simple share for testing

```
vi /usr/local/samba/etc/smb.conf
```

then i started samba

```
cd /usr/local/samba/sbin/
```

```
./samba
```

I tested with smbclient, it works

```
smbclient //localhost/test -Uadministrator%stargate
```

I checked that bind (and bind-chroot) were installed (yes), so copied the srv records

```
cp booi.schot.zone /var/named/chroot/etc/
```

then appended to named.conf

```
cat named.conf >> /var/named/chroot/etc/named.conf
```

I followed these steps in the howto4.txt

```
vi /etc/init.d/named [added two export lines right after start()]
chmod a+r /usr/local/samba/private/dns.keytab
cp krb5.conf /etc/
vi /var/named/chroot/etc/named.conf
--> remove a lot, but keep allow-update { any; };
```

restart bind (named!), then tested dns with dig, this works (stripped screenshot!)

```
[root@RHEL52 private]# dig _ldap._tcp.dc._msdcs.booi.schot SRV @localhost

; (1 server found)
;; global options: printcmd
;; Got answer:
;; -HEADER- opcode: QUERY, status: NXDOMAIN, id: 58186
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 0

;; QUESTION SECTION:
;_ldap._tcp.dc._msdcs.booi.schot. IN SRV

;; AUTHORITY SECTION:
.      10800 IN SOA A.ROOT-SERVERS.NET....

;; Query time: 54 msec
;; SERVER: 127.0.0.1#53(127.0.0.1)
;; WHEN: Tue Jan 27 20:57:05 2009
;; MSG SIZE rcvd: 124

[root@RHEL52 private]#
```

made sure /etc/resolv.conf points to himself

```
[root@RHEL52 private]# cat /etc/resolv.conf
search booi.schot
nameserver 127.0.0.1
```

start windows 2003 server, enter the samba4 as DNS!

ping the domain, if it doesn't work, then add your redhats hostname and your realm to windows/system32/drivers/etc/hosts

join the windows computer to the domain

reboot the windows

log on with administrator stargate

start run dsa.msc to manage samba4

create an OU, a user and a GPO, test that it works

Part XXIV. introducing git

Table of Contents

85. git	910
85.1. git	911
85.2. installing git	912
85.3. starting a project	912
85.4. git branches	915
85.5. to be continued... ..	916
85.6. github.com	917
85.7. add your public key to github	917
85.8. practice: git	918

Chapter 85. git

This chapter is an introduction to using **git** on the command line. The **git repository** is hosted by **github**, but you are free to choose another server (or create your own).

There are many excellent online tutorials for **git**. This list can save you one Google query:

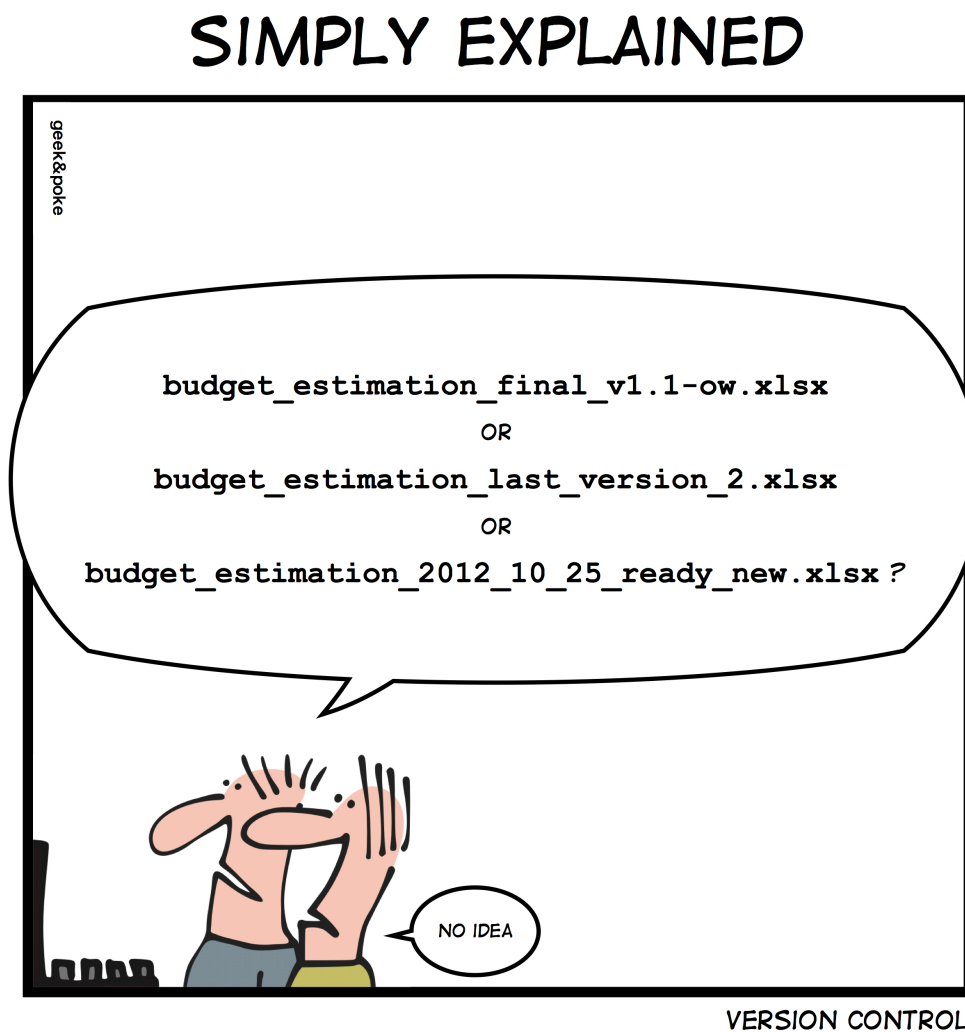
<http://gitimmersion.com/>
<http://git-scm.com/book>

85.1. git

Linus Torvalds created **git** back in 2005 when Bitkeeper changed its license and the Linux kernel developers where no longer able to use it for free.

git quickly became popular and is now the most widely used **distributed version control** system in the world.

Geek and Poke demonstrates why we need version control (image property of Geek and Poke CCA 3.0).



Besides **source code** for software, you can also find German and Icelandic **law** on github (and probably much more by the time you are reading this).

85.2. installing git

We install **git** with **aptitude install git** as seen in this screenshot on Debian 6.

```
root@debian6:~# aptitude install git
The following NEW packages will be installed:
  git libcurl3-gnutls{a} liberror-perl{a}
0 packages upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
...
Processing triggers for man-db ...
Setting up libcurl3-gnutls (7.21.0-2.1+squeeze2) ...
Setting up liberror-perl (0.17-1) ...
Setting up git (1:1.7.2.5-3) ...
```

85.3. starting a project

First we create a project directory, with a simple file in it.

```
paul@debian6-$ mkdir project42
paul@debian6-$ cd project42/
paul@debian6~/project42$ echo "echo The answer is 42." >> question.sh
```

85.3.1. git init

Then we tell **git** to create an empty git repository in this directory.

```
paul@debian6~/project42$ ls -la
total 12
drwxrwxr-x  2 paul paul 4096 Dec  8 16:41 .
drwxr-xr-x 46 paul paul 4096 Dec  8 16:41 ..
-rw-rw-r--  1 paul paul  23 Dec  8 16:41 question.sh
paul@debian6~/project42$ git init
Initialized empty Git repository in /home/paul/project42/.git/
paul@debian6~/project42$ ls -la
total 16
drwxrwxr-x  3 paul paul 4096 Dec  8 16:44 .
drwxr-xr-x 46 paul paul 4096 Dec  8 16:41 ..
drwxrwxr-x  7 paul paul 4096 Dec  8 16:44 .git
-rw-rw-r--  1 paul paul  23 Dec  8 16:41 question.sh
```

85.3.2. git config

Next we use **git config** to set some global options.

```
paul@debian6$ git config --global user.name Paul
paul@debian6$ git config --global user.email "paul.cobbaut@gmail.com"
paul@debian6$ git config --global core.editor vi
```

We can verify this config in **~/.gitconfig**:

```
paul@debian6~/project42$ cat ~/.gitconfig
[user]
  name = Paul
  email = paul.cobbaut@gmail.com
[core]
  editor = vi
```

85.3.3. git add

Time now to add file to our project with **git add**, and verify that it is added with **git status**.


```
paul@debian6~/project42$ git add question.sh
paul@debian6~/project42$ git status
# On branch master
#
# Initial commit
#
# Changes to be committed:
#   (use "git rm --cached <file>..." to unstage)
#
# new file:   question.sh
#
```

The **git status** tells us there is a new file ready to be committed.

85.3.4. git commit

With **git commit** you force git to record all added files (and all changes to those files) permanently.

```
paul@debian6~/project42$ git commit -m "starting a project"
[master (root-commit) 5c10768] starting a project
 1 file changed, 1 insertion(+)
 create mode 100644 question.sh
paul@debian6~/project42$ git status
# On branch master
nothing to commit (working directory clean)
```

85.3.5. changing a committed file

The screenshots below show several steps. First we change a file:

```
paul@debian6~/project42$ git status
# On branch master
nothing to commit (working directory clean)
paul@debian6~/project42$ vi question.sh
```

Then we verify the status and see that it is modified:

```
paul@debian6~/project42$ git status
# On branch master
# Changes not staged for commit:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
# modified:   question.sh
#
no changes added to commit (use "git add" and/or "git commit -a")
```

Next we add it to the git repository.

```
paul@debian6~/project42$ git add question.sh
paul@debian6~/project42$ git commit -m "adding a she-bang to the main script"
[master 86b8347] adding a she-bang to the main script
 1 file changed, 1 insertion(+)
paul@debian6~/project42$ git status
# On branch master
nothing to commit (working directory clean)
```

85.3.6. git log

We can see all our commits again using **git log**.

```
paul@debian6~/project42$ git log
commit 86b8347192ea025815df7a8e628d99474b41fb6c
Author: Paul <paul.cobbaut@gmail.com>
Date: Sat Dec 8 17:12:24 2012 +0100
```

```
    adding a she-bang to the main script
```

```
commit 5c10768f29aecc16161fb197765e0f14383f7bca
Author: Paul <paul.cobbaut@gmail.com>
Date: Sat Dec 8 17:09:29 2012 +0100
```

```
    starting a project
```

The log format can be changed.

```
paul@debian6~/project42$ git log --pretty=oneline
86b8347192ea025815df7a8e628d99474b41fb6c adding a she-bang to the main script
5c10768f29aecc16161fb197765e0f14383f7bca starting a project
```

The log format can be customized a lot.

```
paul@debian6~/project42$ git log --pretty=format:"%an: %ar :%s"
Paul: 8 minutes ago :adding a she-bang to the main script
Paul: 11 minutes ago :starting a project
```

85.3.7. git mv

Renaming a file can be done with **mv** followed by a **git remove** and a **git add** of the new filename. But it can be done easier and in one command using **git mv**.

```
paul@debian6~/project42$ git mv question.sh thequestion.sh
paul@debian6~/project42$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
# renamed:    question.sh -> thequestion.sh
#
paul@debian6~/project42$ git commit -m "improved naming scheme"
[master 69b2c8b] improved naming scheme
1 file changed, 0 insertions(+), 0 deletions(-)
rename question.sh => thequestion.sh (100%)
```

85.4. git branches

Working on the project can be done in one or more **git branches**. Here we create a new branch that will make changes to the script. We will **merge** this branch with the **master branch** when we are sure the script works. (It can be useful to add **git status** commands when practicing).

```
paul@debian6~/project42$ git branch
* master
paul@debian6~/project42$ git checkout -b newheader
Switched to a new branch 'newheader'
paul@debian6~/project42$ vi thequestion.sh
paul@debian6~/project42$ git add thequestion.sh
paul@debian6~/project42$ source thequestion.sh
The answer is 42.
```

It seems to work, so we commit in this branch.

```
paul@debian6~/project42$ git commit -m "adding a new company header"
[newheader 730a22b] adding a new company header
1 file changed, 4 insertions(+)
paul@debian6~/project42$ git branch
  master
* newheader
paul@debian6~/project42$ cat thequestion.sh
#!/bin/bash
#
# copyright linux-training.be
#
echo The answer is 42.
```

Let us go back to the master branch and see what happened there.

```
paul@debian6~/project42$ git checkout master
Switched to branch 'master'
paul@debian6~/project42$ cat thequestion.sh
#!/bin/bash
echo The answer is 42.
```

Nothing happened in the master branch, because we worked in another branch.

When we are sure the branch is ready for production, then we merge it into the master branch.

```
paul@debian6~/project42$ cat thequestion.sh
#!/bin/bash
echo The answer is 42.
paul@debian6~/project42$ git merge newheader
Updating 69b2c8b..730a22b
Fast-forward
 thequestion.sh | 4 ++++
1 file changed, 4 insertions(+)
paul@debian6~/project42$ cat thequestion.sh
#!/bin/bash
#
# copyright linux-training.be
#
echo The answer is 42.
```

The newheader branch can now be deleted.

```
paul@debian6~/project42$ git branch
* master
  newheader
paul@debian6~/project42$ git branch -d newheader
Deleted branch newheader (was 730a22b).
paul@debian6~/project42$ git branch
* master
```

85.5. to be continued...

The **git** story is not finished.

There are many excellent online tutorials for **git**. This list can save you one Google query:

```
http://gitimmersion.com/
http://git-scm.com/book
```

85.6. github.com

Create an account on **github.com**. This website is a frontend for an immense git server with over two and a half million users and almost five million projects (including Fedora, Linux kernel, Android, Ruby on Rails, Wine, X.org, VLC...)

<https://github.com/signup/free>

This account is free of charge, we will use it in the examples below.

85.7. add your public key to github

I prefer to use github with a **public key**, so it probably is a good idea that you also upload your public key to github.com.

You can upload your own key via the web interface:

<https://github.com/settings/ssh>

Please do not forget to protect your **private key**!

85.8. practice: git

1. Create a project on github to host a script that you wrote. Have at least two other people improve the script.

Part XXV. ipv6

Table of Contents

86. Introduction to ipv6	921
86.1. about ipv6	922
86.2. network id and host id	922
86.3. host part generation	922
86.4. ipv4 mapped ipv6 address	923
86.5. link local addresses	923
86.6. unique local addresses	923
86.7. globally unique unicast addresses	923
86.8. 6to4	923
86.9. ISP	924
86.10. non routable addresses	924
86.11. ping6	924
86.12. Belgium and ipv6	925
86.13. other websites	925
86.14. 6to4 gateways	927
86.15. ping6 and dns	927
86.16. ipv6 and tcp/http	927
86.17. ipv6 PTR record	927
86.18. 6to4 setup on Linux	927

Chapter 86. Introduction to ipv6

86.1. about ipv6

The **ipv6** protocol is designed to replace **ipv4**. Where **ip version 4** supports a maximum of four billion unique addresses, **ip version 6** expands this to **four billion times four billion times four billion times four billion** unique addresses. This is more than 100.000.000.000.000.000.000 ipv6 addresses per square cm on our planet. That should be enough, even if every cell phone, every coffee machine and every pair of socks gets an address.

Technically speaking ipv6 uses 128-bit addresses (instead of the 32-bit from ipv4). 128-bit addresses are **huge** numbers. In decimal it would amount up to 39 digits, in hexadecimal it looks like this:

```
fe80:0000:0000:0000:0a00:27ff:fe8e:8aa8
```

Luckily ipv6 allows us to omit leading zeroes. Our address from above then becomes:

```
fe80:0:0:0:a00:27ff:fe8e:8aa8
```

When a 16-bit block is zero, it can be written as **::**. Consecutive 16-bit blocks that are zero can also be written as **::**. So our address from above can be shortened to:

```
fe80::a00:27ff:fe8e:8aa8
```

This **::** can only occur once! The following is not a valid ipv6 address:

```
fe80::20:2e4f::39ac
```

The ipv6 **localhost** address is **0000:0000:0000:0000:0000:0000:0000:0001**, which can be abbreviated to **::1**.

```
paul@debian5:~/github/lt/images$ /sbin/ifconfig lo | grep inet6
inet6 addr: ::1/128 Scope:Host
```

86.2. network id and host id

One of the few similarities between ipv4 and ipv6 is that addresses have a host part and a network part determined by a subnet mask. Using the **cidr** notation this looks like this:

```
fe80::a00:27ff:fe8e:8aa8/64
```

The above address has 64 bits for the host id, theoretically allowing for 4 billion times four billion hosts.

The localhost address looks like this with cidr:

```
::1/128
```

86.3. host part generation

The host part of an automatically generated (stateless) ipv6 address contains part of the hosts mac address:

```
paul@debian5:~$ /sbin/ifconfig | head -3
```

```
eth3      Link encap:Ethernet  HWaddr 08:00:27:ab:67:30
          inet addr:192.168.1.29  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:feab:6730/64 Scope:Link
```

Some people are concerned about privacy here...

86.4. ipv4 mapped ipv6 address

Some applications use ipv4 addresses embedded in an ipv6 address. (Yes there will be an era of migration with both ipv4 and ipv6 in use.) The ipv6 address then looks like this:

```
::ffff:192.168.1.42/96
```

Indeed a mix of decimal and hexadecimal characters...

86.5. link local addresses

ipv6 addresses starting with **fe8.** can only be used on the local segment (replace the dot with an hexadecimal digit). This is the reason you see **Scope:Link** behind the address in this screenshot. This address serves only the **local link**.

```
paul@deb503:~$ /sbin/ifconfig | grep inet6
inet6 addr: fe80::a00:27ff:fe8e:8aa8/64 Scope:Link
inet6 addr: ::1/128 Scope:Host
```

These **link local** addresses all begin with **fe8.**

Every ipv6 enabled nic will get an address in this range.

86.6. unique local addresses

The now obsolete system of **site local addresses** similar to ipv4 private ranges is replaced with a system of globally unique local ipv6 addresses. This to prevent duplicates when joining of networks within **site local** ranges.

All **unique local** addresses start with **fd...**

86.7. globally unique unicast addresses

Since **ipv6** was designed to have multiple ip addresses per interface, the **global ipv6 address** can be used next to the **link local address**.

These **globally unique** addresses all begin with **2...** or **3...** as the first 16-bits.

86.8. 6to4

6to4 is defined in rfc's 2893 and 3056 as one possible way to transition between ipv4 and ipv6 by creating an ipv6 tunnel.

It encodes an ipv4 address in an ipv6 address that starts with **2002.** For example 192.168.1.42/24 will be encoded as:

```
2002:c0a8:12a:18::1
```

You can use the command below to convert any ipv4 address to this range.

```
paul@ubu1010:~$ printf "2002:%02x%02x:%02x%02x:%04x::1\n" `echo 192.168.1.42/24 \
|tr "./" " "`
2002:c0a8:012a:0018::1
```

86.9. ISP

Should you be so lucky to get an ipv6 address from an **isp**, then it will start with **2001::**.

86.10. non routable addresses

Comparable to **example.com** for DNS, the following ipv6 address ranges are reserved for examples, and not routable on the internet.

```
3fff:ffff::/32
2001:0db8::/32
```

86.11. ping6

Use **ping6** to test connectivity between ipv6 hosts. You need to specify the interface (there is no routing table for 'random' generated ipv6 link local addresses).

```
[root@fedora14 ~]# ping6 -I eth0 fe80::a00:27ff:fe3c:4346 eth0: 56
PING fe80::a00:27ff:fe3c:4346 (fe80::a00:27ff:fe3c:4346) from fe80::a00:27ff:fe3c:4346 eth0: 56
64 bytes from fe80::a00:27ff:fe3c:4346: icmp_seq=1 ttl=64 time=0.586 ms
64 bytes from fe80::a00:27ff:fe3c:4346: icmp_seq=2 ttl=64 time=3.95 ms
64 bytes from fe80::a00:27ff:fe3c:4346: icmp_seq=3 ttl=64 time=1.53 ms
```

Below a multicast ping6 that receives replies from three ip6 hosts on the same network.

```
[root@fedora14 ~]# ping6 -I eth0 ff02::1
PING ff02::1(ff02::1) from fe80::a00:27ff:fe3c:4346 eth0: 56 data bytes
64 bytes from fe80::a00:27ff:fe3c:4346: icmp_seq=1 ttl=64 time=0.598 ms
64 bytes from fe80::a00:27ff:fe3c:4346: icmp_seq=1 ttl=64 time=1.87 ms (DUP!)
64 bytes from fe80::8e7b:9dff:fed6:dff2: icmp_seq=1 ttl=64 time=535 ms (DUP!)
64 bytes from fe80::a00:27ff:fe3c:4346: icmp_seq=2 ttl=64 time=0.106 ms
64 bytes from fe80::8e7b:9dff:fed6:dff2: icmp_seq=2 ttl=64 time=1.79 ms (DUP!)
64 bytes from fe80::a00:27ff:fe3c:4346: icmp_seq=2 ttl=64 time=2.48 ms (DUP!)
```

86.12. Belgium and ipv6

A lot of information on ipv6 in Belgium can be found at www.ipv6council.be.

Sites like ipv6.belgium.be, www.bipt.be and www.bricozone.be are enabled for ipv6. Some Universities also: fundp.ac.be (Namur) and ulg.ac.be (Liege).

86.13. other websites

Other useful websites for testing ipv6 are:

test-ipv6.com
ipv6-test.com

Going to the ipv6-test.com website will test whether you have a valid accessible ipv6 address.



Going to the test-ipv6.com website will also test whether you have a valid accessible ipv6 address.

86.14. 6to4 gateways

To access ipv4 only websites when on ipv6 you can use sixxs.net (more specifically <http://www.sixxs.net/tools/gateway/>) as a gateway.

For example use <http://www.slashdot.org.sixxs.org/> instead of <http://slashdot.org>

86.15. ping6 and dns

Below a screenshot of a **ping6** from behind a 6to4 connection.

81.165.101.125	195.130.131.4	DNS	Standard query AAAA ipv6-test.com
195.130.131.4	81.165.101.125	DNS	Standard query response AAAA 2001:41d0:2:67d1::7e57:1
2002:51a5:657d::1	2001:41d0:2:67d1::7e57:1	ICMPv6	Echo request
2001:41d0:2:67d1::7e57:1	2002:51a5:657d::1	ICMPv6	Echo reply
2002:51a5:657d::1	2001:41d0:2:67d1::7e57:1	ICMPv6	Echo request
2001:41d0:2:67d1::7e57:1	2002:51a5:657d::1	ICMPv6	Echo reply

86.16. ipv6 and tcp/http

Below a screenshot of a tcp handshake and http connection over ipv6.

Source	Destination	Protocol	Info
2002:51a5:657d::1	2001:41d0:2:67d1::7e57:1	TCP	38036 > http [SYN] Seq=0 Win=5648 L
2001:41d0:2:67d1::7e57:1	2002:51a5:657d::1	TCP	http > 38036 [SYN, ACK] Seq=0 Ack=1
2002:51a5:657d::1	2001:41d0:2:67d1::7e57:1	TCP	38036 > http [ACK] Seq=1 Ack=1 Win=
2002:51a5:657d::1	2001:41d0:2:67d1::7e57:1	HTTP	GET /json/addrinfor.php?PHPSESSID=19
2001:41d0:2:67d1::7e57:1	2002:51a5:657d::1	TCP	http > 38036 [ACK] Seq=1 Ack=708 Wi
2001:41d0:2:67d1::7e57:1	2002:51a5:657d::1	HTTP	HTTP/1.1 200 OK (text/javascript)

86.17. ipv6 PTR record

As seen in the DNS chapter, ipv6 PTR records are in the ip6.net domain, and have 32 generations of child domains.

```

▶ Frame 46 (132 bytes on wire, 132 bytes captured)
▶ Ethernet II, Src: Apple_5d:2e:52 (00:26:bb:5d:2e:52), Dst: Riverdel_cf:6a:10 (00:30:b8:cf:6a:10)
▶ Internet Protocol, Src: 81.165.101.125 (81.165.101.125), Dst: 195.130.131.4 (195.130.131.4)
▶ User Datagram Protocol, Src Port: 34361 (34361), Dst Port: domain (53)
▼ Domain Name System (query)
  [Response In: 47]
  Transaction ID: 0xcfe3
  ▶ Flags: 0x0100 (Standard query)
  Questions: 1
  Answer RRs: 0
  Authority RRs: 0
  Additional RRs: 0
  ▼ Queries
    ▶ 1.0.0.0.7.5.e.7.0.0.0.0.0.0.0.0.1.d.7.6.2.0.0.0.0.d.1.4.1.0.0.2.ip6.arpa: type PTR, class IN

```

86.18. 6to4 setup on Linux

Below a transcript of a 6to4 setup on Linux.

Thanks to <http://www.anyweb.co.nz/tutorial/v6Linux6to4> and <http://mirrors.bieringer.de/Linux+IPv6-HOWTO/> and tldp.org/

```

root@mac:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:26:bb:5d:2e:52
          inet addr:81.165.101.125  Bcast:255.255.255.255  Mask:255.255.248.0

```



```

inet6 addr: fe80::226:bbff:fe5d:2e52/64 Scope:Link
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:5926044 errors:0 dropped:0 overruns:0 frame:0
TX packets:2985892 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:4274849823 (4.2 GB) TX bytes:237002019 (237.0 MB)
Interrupt:43 Base address:0x8000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1 Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING MTU:16436 Metric:1
          RX packets:598 errors:0 dropped:0 overruns:0 frame:0
          TX packets:598 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:61737 (61.7 KB) TX bytes:61737 (61.7 KB)

root@mac:~# sysctl -w net.ipv6.conf.default.forwarding=1
net.ipv6.conf.default.forwarding = 1
root@mac:~# ip tunnel add tun6to4 mode sit remote any local 81.165.101.125
root@mac:~# ip link set dev tun6to4 mtu 1472 up
root@mac:~# ip link show dev tun6to4
10: tun6to4: <NOARP,UP,LOWER_UP> mtu 1472 qdisc noqueue state UNKNOWN
    link/sit 81.165.101.125 brd 0.0.0.0
root@mac:~# ip -6 addr add dev tun6to4 2002:51a5:657d::1/64
root@mac:~# ip -6 addr add dev eth0 2002:51a5:657d:1::1/64
root@mac:~# ip -6 addr add dev eth0 fdcb:43c1:9c18:1::1/64
root@mac:~# ifconfig
eth0      Link encap:Ethernet HWaddr 00:26:bb:5d:2e:52
          inet addr:81.165.101.125 Bcast:255.255.255.255 Mask:255.255.248.0
          inet6 addr: fe80::226:bbff:fe5d:2e52/64 Scope:Link
          inet6 addr: fdcb:43c1:9c18:1::1/64 Scope:Global
          inet6 addr: 2002:51a5:657d:1::1/64 Scope:Global
          UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
          RX packets:5927436 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2986025 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:4274948430 (4.2 GB) TX bytes:237014619 (237.0 MB)
          Interrupt:43 Base address:0x8000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1 Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING MTU:16436 Metric:1
          RX packets:598 errors:0 dropped:0 overruns:0 frame:0
          TX packets:598 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:61737 (61.7 KB) TX bytes:61737 (61.7 KB)

tun6to4   Link encap:IPv6-in-IPv4
          inet6 addr: ::81.165.101.125/128 Scope:Compat
          inet6 addr: 2002:51a5:657d::1/64 Scope:Global
          UP RUNNING NOARP MTU:1472 Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

root@mac:~# ip -6 route add 2002::/16 dev tun6to4
root@mac:~# ip -6 route add ::/0 via ::192.88.99.1 dev tun6to4 metric 1
root@mac:~# ip -6 route show
::/96 via :: dev tun6to4 metric 256 mtu 1472 advmss 1412 hoplimit 0
2002:51a5:657d::/64 dev tun6to4 proto kernel metric 256 mtu 1472 advmss 1412 hoplimit 0
2002:51a5:657d:1::/64 dev eth0 proto kernel metric 256 mtu 1500 advmss 1440 hoplimit 0

```



```
2002::/16 dev tun6to4 metric 1024 mtu 1472 advmss 1412 hoplimit 0
fdb:43c1:9c18:1::/64 dev eth0 proto kernel metric 256 mtu 1500 advmss 1440 hoplimit 0
fe80::/64 dev eth0 proto kernel metric 256 mtu 1500 advmss 1440 hoplimit 0
fe80::/64 dev tun6to4 proto kernel metric 256 mtu 1472 advmss 1412 hoplimit 0
default via ::192.88.99.1 dev tun6to4 metric 1 mtu 1472 advmss 1412 hoplimit 0
root@mac:~# ping6 ipv6-test.com
PING ipv6-test.com(ipv6-test.com) 56 data bytes
64 bytes from ipv6-test.com: icmp_seq=1 ttl=57 time=42.4 ms
64 bytes from ipv6-test.com: icmp_seq=2 ttl=57 time=43.0 ms
64 bytes from ipv6-test.com: icmp_seq=3 ttl=57 time=43.5 ms
64 bytes from ipv6-test.com: icmp_seq=4 ttl=57 time=43.9 ms
64 bytes from ipv6-test.com: icmp_seq=5 ttl=57 time=45.6 ms
^C
--- ipv6-test.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 42.485/43.717/45.632/1.091 ms
```

Part XXVI. Appendices

Table of Contents

A. certifications	932
A.1. Certification	932
B. Raspberry Pi Linux	934
B.1. buying the hardware	935
B.2. downloading the images	936
B.3. writing the SD cards	937
B.4. adjusting network settings	938
B.5. connecting the hardware	939
B.6. connecting with ssh	940
B.7. camera module	941
B.8.	942
B.9. building a stack of pi's	943
B.10. sending mail from the pi	943
B.11. disable ipv6	944
B.12.	945
B.13.	946
B.14.	947
B.15. raspi camera	947
C. keyboard settings	948
C.1. about keyboard layout	948
C.2. X Keyboard Layout	948
C.3. shell keyboard layout	948
D. hardware	950
D.1. buses	950
D.2. interrupts	951
D.3. io ports	952
D.4. dma	952
E. installing Linux	954
E.1. about	954
E.2. legacy content	954
F. disk quotas	957
F.1. About Disk Quotas	957
F.2. Practice Disk quotas	957
G. introduction to vnc	958
G.1. About VNC	958
G.2. VNC Server	958
G.3. VNC Client	958
G.4. Practice VNC	959
H. cloning	960
H.1. About cloning	960
H.2. About offline cloning	960
H.3. Offline cloning example	960
I. License	962

Appendix A. certifications

A.1. Certification

A.1.1. LPI: Linux Professional Institute

LPIC Level 1

This is the junior level certification. You need to pass exams 101 and 102 to achieve **LPIC 1 certification**. To pass level one, you will need Linux command line, user management, backup and restore, installation, networking, and basic system administration skills.

LPIC Level 2

This is the advanced level certification. You need to be LPIC 1 certified and pass exams 201 and 202 to achieve **LPIC 2 certification**. To pass level two, you will need to be able to administer medium sized Linux networks, including Samba, mail, news, proxy, firewall, web, and ftp servers.

LPIC Level 3

This is the senior level certification. It contains one core exam (301) which tests advanced skills mainly about ldap. To achieve this level you also need LPIC Level 2 and pass a specialty exam (302 or 303). Exam 302 mainly focuses on Samba, and 303 on advanced security. More info on <http://www.lpi.org>.

Ubuntu

When you are LPIC Level 1 certified, you can take a LPI Ubuntu exam (199) and become Ubuntu certified.

A.1.2. Red Hat Certified Engineer

The big difference with most other certifications is that there are no multiple choice questions for **RHCE**. Red Hat Certified Engineers have to take a live exam consisting of two parts. First, they have to troubleshoot and maintain an existing but broken setup (scoring at least 80 percent), and second they have to install and configure a machine (scoring at least 70 percent).

A.1.3. MySQL

There are two tracks for MySQL certification; Certified MySQL 5.0 Developer (CMDEV) and Certified MySQL 5.0 DBA (CMDDBA). The **CMDEV** is focused towards database application developers, and the **CMDDBA** towards database administrators. Both tracks require two exams each. The MySQL cluster DBA certification requires CMDDBA certification and passing the CMCDDBA exam.

A.1.4. Novell CLP/CLE

To become a **Novell Certified Linux Professional**, you have to take a live practicum. This is a VNC session to a set of real SLES servers. You have to perform several tasks and are free to choose your method (commandline or YaST or ...). No multiple choice involved.

A.1.5. Sun Solaris

Sun uses the classical formula of multiple choice exams for certification. Passing two exams for an operating system gets you the Solaris Certified Administrator for Solaris X title.

A.1.6. Other certifications

There are many other lesser known certifications like EC council's Certified Ethical Hacker, CompTIA's Linux+, and Sair's Linux GNU.

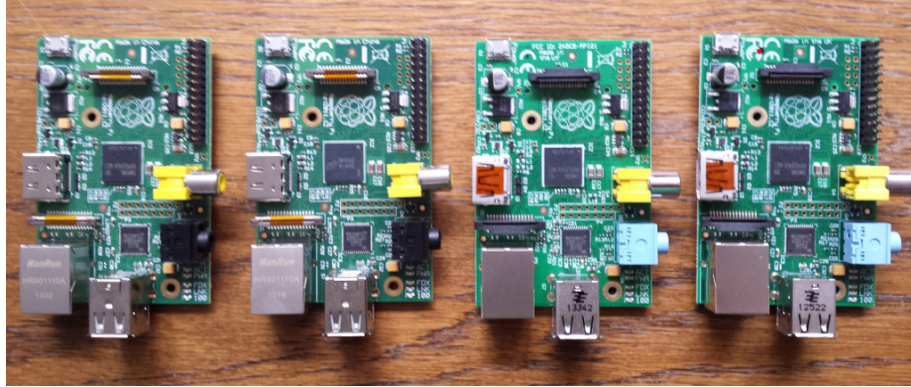
Appendix B. Raspberry Pi Linux

This module will introduce you to using Linux on the **Raspberry Pi**. We will start with two **Pi**'s and two empty **SD cards** and put **Raspbian** on one and **Pidora** on the other.

After this module you will have a networked **Raspberry Pi** with **Debian** (Raspbian) and one with **Red Hat** (Pidora).

B.1. buying the hardware

This picture shows four **Raspberry Pi**'s. The two on the left are made in UK, those on the right are made in China. I noticed no difference in using them.



The **Raspberry Pi** is advertised as a '30 euro computer'. This is not a lie, but many people will end up spending around 50 euro for each fully working and connected **Raspberry Pi**.

Besides the obvious **Raspberry Pi board** you will also need an **sd card**, a **utp ethernet cable** and a **micro usb cable** connected to a **power adapter** or powered usb port.

I recommend buying **class 10 sd cards** because they are faster than cheap sd cards. Below a picture that shows four class 10 cards; one micro sd (left) and three regular ones.



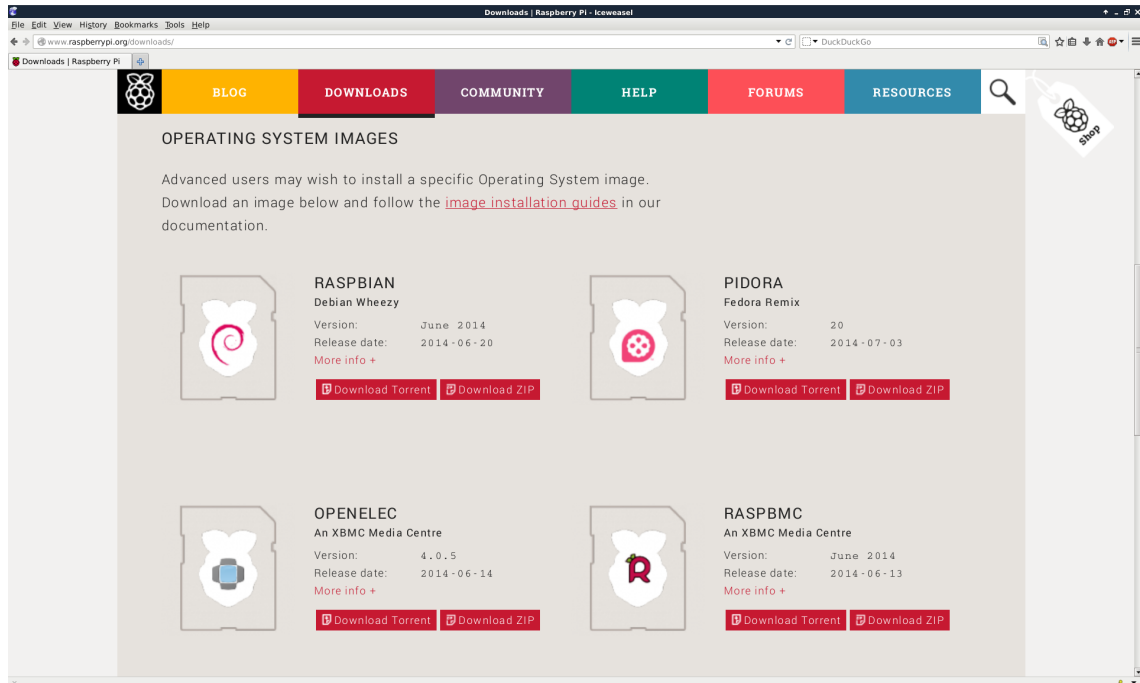
For this project I will demonstrate two **Linux distributions** so I chose to label the cards.



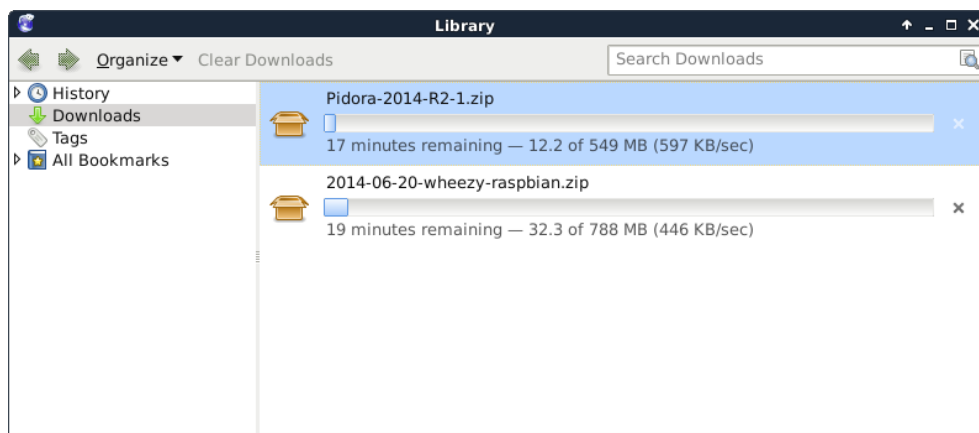
I will use the usb cable and power adapter from my old HTC Hero smartphone to power the **Raspberry Pi**.

B.2. downloading the images

Download two images from <http://raspberrypi.org/downloads>. In this module I continue with the **raspbian** and the **pidora** images.



This download may take a couple minutes to complete...



After the download, you will have two zipfiles in **/tmp**.

```
paul@debian7:~$ ls -lh /tmp/*.zip
-rw-r--r-- 1 paul paul 788M Jul 26 20:53 /tmp/2014-06-20-wheezy-raspbian.zip
-rw-r--r-- 1 paul paul 549M Jul 26 20:51 /tmp/Pidora-2014-R2-1.zip
paul@debian7:~$
```


B.3. writing the SD cards

Find out the devicename of your SD card.

```
root@debian7:~# dmesg | tail -2 | cut -c9-
30238] sd 1:0:0:0: [sdb] 15644672 512-byte logical blocks: (8.01 GB/7.45 GiB)
33843] sdb: sdb1
root@debian7:~#
```

Be very careful not to overwrite the wrong device! In my case it says **sdb**. This can be different for you!

I choose to **unzip** the downloaded file with the **-p** switch to send the unzipped data to **stdout** so it can be captured by **dd** and written to the SD card device (without creating the intermediate unzipped file).

```
root@debian7:~# unzip -p /tmp/2014-06-20-wheezy-raspbian.zip | dd of=/dev/sdb
```

This command will take a couple of minutes. Here is the output from **top -b** with relation to the unzipping and writing to the device (taken on a 2013 Macbook Pro Retina).

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
123	root	20	0	0	0	0	D	6.6	0.0	0:16.40	usb-storage
8877	root	20	0	8992	1644	572	S	6.6	0.0	0:33.09	unzip
8878	root	20	0	5816	648	556	D	0.0	0.0	0:17.85	dd

This is the output from **dd**.

```
root@debian7:~# unzip -p /tmp/2014-06-20-wheezy-raspbian.zip | dd of=/dev/sdb
5785600+0 records in
5785600+0 records out
2962227200 bytes (3.0 GB) copied, 615.552 s, 4.8 MB/s
```

Using **fdisk -l /dev/sdb** gives a quick check whether the image was written to the SD card.

```
root@debian7:~# fdisk -l /dev/sdb

Disk /dev/sdb: 8010 MB, 8010072064 bytes
247 heads, 62 sectors/track, 1021 cylinders, total 15644672 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x000b5098
```

Device	Boot	Start	End	Blocks	Id	System
/dev/sdb1		8192	122879	57344	c	W95 FAT32 (LBA)
/dev/sdb2		122880	5785599	2831360	83	Linux

B.4. adjusting network settings

We will configure the network before inserting the **sd card** in the **Raspberry Pi**. To do this we will mount the Linux partition and edit **/etc/network/interfaces**.

```
root@debian7:~# mount /dev/sdb2 /mnt
root@debian7:~# vi /mnt/etc/network/interfaces
```

Check the chapters in Debian network configuration if you don't know what to do with this file. I added four lines (in bold below) to give this **Pi** a unique address in my ghost network (when connected via ethernet cable).

```
root@debian7:~# cat /mnt/etc/network/interfaces
auto lo

iface lo inet loopback
iface eth0 inet dhcp

auto eth0:0
iface eth0:0 inet static
address 192.168.42.54
netmask 255.255.255.0

allow-hotplug wlan0
iface wlan0 inet manual
wpa-roam /etc/wpa_supplicant/wpa_supplicant.conf
iface default inet dhcp
```

B.5. connecting the hardware

Besides a **Raspberry Pi**, we also use a network utp cable, the sd card labeled **raspbian** and a usb power source (I use the adapter and usb cable from my old HTC Hero phone).

Connect the usb power last. The end result will look something like this picture.

B.6. connecting with ssh

First we try to ping the device. I use my ghost network address. The **Pi** also received an address from the DHCP server, so if you have access to the DHCP leases you can also use that address.

```
paul@debian7:~$ ping 192.168.42.54
PING 192.168.42.54 (192.168.42.54) 56(84) bytes of data.
64 bytes from 192.168.42.54: icmp_seq=1 ttl=64 time=0.577 ms
64 bytes from 192.168.42.54: icmp_seq=2 ttl=64 time=0.627 ms
^C
--- 192.168.42.54 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.577/0.602/0.627/0.025 ms
```

Success, the **Pi** replies to the ping request.

Now we login with the user **raspberry** and the password **pi**.

```
paul@debian7:~$ ssh pi@192.168.42.54
The authenticity of host '192.168.42.54 (192.168.42.54)' can't be established.
ECDSA key fingerprint is a7:9b:bf:7f:00:12:b0:f0:93:f8:09:ad:32:1d:29:57.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.42.54' (ECDSA) to the list of known hosts.
pi@192.168.42.54's password:
Linux raspberrypi 3.12.22+ #691 PREEMPT Wed Jun 18 18:29:58 BST 2014 armv6l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Jul 27 15:47:20 2014 from 192.168.1.35

NOTICE: the software on this Raspberry Pi has not been fully configured. Plea\
se run 'sudo raspi-config'

pi@raspberrypi ~ $
```

B.7. camera module

There is a cheap HD camera module for the raspberry pi. In this picture the module is connected.

To disble the camera led:

```
disable_camera_led=1 in /boot/config.txt
```

B.8.

B.9. building a stack of pi's

Here are some pictures of a **Raspberry Pi** stack held together by Lego Technic.

B.10. sending mail from the pi

install exim4

enter a smarthost in update-exim4.conf.conf

dpkg-reconfigure exim4-config (smarthost)

```
root@raspicam1:~# cat /etc/exim4/update-exim4.conf.conf
# /etc/exim4/update-exim4.conf.conf
#
# Edit this file and /etc/mailname by hand and execute update-exim4.conf
# yourself or use 'dpkg-reconfigure exim4-config'
#
# Please note that this is _not_ a dpkg-conf file and that automatic changes
# to this file might happen. The code handling this will honor your local
# changes, so this is usually fine, but will break local schemes that mess
# around with multiple versions of the file.
#
# update-exim4.conf uses this file to determine variable values to generate
# exim configuration macros for the configuration file.
#
# Most settings found in here do have corresponding questions in the
# Debconf configuration, but not all of them.
#
# This is a Debian specific file

dc_eximconfig_configtype='satellite'
dc_other_hostnames='raspberrypi'
dc_local_interfaces='127.0.0.1 ;'
dc_readhost='netsec.be'
dc_relay_domains=''
dc_minimaldns='true'
dc_relay_nets=''
dc_smarthost='smtp.telenet.be'
CFILEMODE='644'
dc_use_split_config='false'
dc_hide_mailname='true'
dc_mailname_in_oh='true'
dc_localdelivery='mail_spool'
root@raspicam1:~#
```

```
mail -s "test2" paul.cobbaut@gmail.com
```

B.11. disable ipv6

```
root@raspberrypi:~# tail -3 /etc/sysctl.conf  
  
# Disable IPv6  
net.ipv6.conf.all.disable_ipv6 = 1  
root@raspberrypi:~#
```


B.12.

B.13.

B.14.

B.15. raspi camera

We start with a raspbian image (september 2014)

```
dd if=2014-09-09-wheezy-raspbian.img of=/dev/sdb
```

Appendix C. keyboard settings

C.1. about keyboard layout

Many people (like US-Americans) prefer the default US-qwerty keyboard layout. So when you are not from the USA and want a local keyboard layout on your system, then the best practice is to select this keyboard at installation time. Then the keyboard layout will always be correct. Also, whenever you use ssh to remotely manage a Linux system, your local keyboard layout will be used, independent of the server keyboard configuration. So you will not find much information on changing keyboard layout on the fly on linux, because not many people need it. Below are some tips to help you.

C.2. X Keyboard Layout

This is the relevant portion in /etc/X11/xorg.conf, first for Belgian azerty, then for US-qwerty.

```
[paul@RHEL5 ~]$ grep -i xkb /etc/X11/xorg.conf
        Option      "XkbModel" "pc105"
        Option      "XkbLayout" "be"
```

```
[paul@RHEL5 ~]$ grep -i xkb /etc/X11/xorg.conf
        Option      "XkbModel" "pc105"
        Option      "XkbLayout" "us"
```

When in Gnome or KDE or any other graphical environment, look in the graphical menu in preferences, there will be a keyboard section to choose your layout. Use the graphical menu instead of editing xorg.conf.

C.3. shell keyboard layout

When in bash, take a look in the /etc/sysconfig/keyboard file. Below a sample US-qwerty configuration, followed by a Belgian azerty configuration.

```
[paul@RHEL5 ~]$ cat /etc/sysconfig/keyboard
KEYBOARDTYPE="pc"
KEYTABLE="us"
```

```
[paul@RHEL5 ~]$ cat /etc/sysconfig/keyboard
KEYBOARDTYPE="pc"
KEYTABLE="be-latin1"
```

The keymaps themselves can be found in /usr/share/keymaps or /lib/kbd/keymaps.

```
[paul@RHEL5 ~]$ ls -l /lib/kbd/keymaps/
total 52
drwxr-xr-x 2 root root 4096 Apr  1 00:14 amiga
```

```
drwxr-xr-x 2 root root 4096 Apr 1 00:14 atari
drwxr-xr-x 8 root root 4096 Apr 1 00:14 i386
drwxr-xr-x 2 root root 4096 Apr 1 00:14 include
drwxr-xr-x 4 root root 4096 Apr 1 00:14 mac
lrwxrwxrwx 1 root root 3 Apr 1 00:14 ppc -> mac
drwxr-xr-x 2 root root 4096 Apr 1 00:14 sun
```

Appendix D. hardware

D.1. buses

D.1.1. about buses

Hardware components communicate with the **Central Processing Unit** or **cpu** over a **bus**. The most common buses today are **usb**, **pci**, **agp**, **pci-express** and **pcmcia** aka **pc-card**. These are all **Plug and Play** buses.

Older **x86** computers often had **isa** buses, which can be configured using **jumper**s or **dip switches**.

D.1.2. /proc/bus

To list the buses recognised by the Linux kernel on your computer, look at the contents of the **/proc/bus/** directory (screenshot from Ubuntu 7.04 and RHEL4u4 below).

```
root@laika:~# ls /proc/bus/
input  pccard  pci  usb
```

```
[root@RHEL4b ~]# ls /proc/bus/
input  pci  usb
```

Can you guess which of these two screenshots was taken on a laptop ?

D.1.3. /usr/sbin/lssusb

To list all the usb devices connected to your system, you could read the contents of **/proc/bus/usb/devices** (if it exists) or you could use the more readable output of **lssusb**, which is executed here on a SPARC system with Ubuntu.

```
root@shaka:~# lssusb
Bus 001 Device 002: ID 0430:0100 Sun Microsystems, Inc. 3-button Mouse
Bus 001 Device 003: ID 0430:0005 Sun Microsystems, Inc. Type 6 Keyboard
Bus 001 Device 001: ID 04b0:0136 Nikon Corp. Coolpix 7900 (storage)
root@shaka:~#
```

D.1.4. /var/lib/usbutils/usb.ids

The **/var/lib/usbutils/usb.ids** file contains a gzipped list of all known usb devices.

```
paul@barry:~$ zmore /var/lib/usbutils/usb.ids | head
-----> /var/lib/usbutils/usb.ids <-----
#
# List of USB ID's
#
# Maintained by Vojtech Pavlik <vojtech@suse.cz>
```

```
# If you have any new entries, send them to the maintainer.
# The latest version can be obtained from
# http://www.linux-usb.org/usb.ids
#
# $Id: usb.ids,v 1.225 2006/07/13 04:18:02 dbrownell Exp $
```

D.1.5. /usr/sbin/lspci

To get a list of all pci devices connected, you could take a look at **/proc/bus/pci** or run **lspci** (partial output below).

```
paul@laika:~$ lspci
...
00:06.0 FireWire (IEEE 1394): Texas Instruments TSB43AB22/A IEEE-139...
00:08.0 Ethernet controller: Realtek Semiconductor Co., Ltd. RTL-816...
00:09.0 Multimedia controller: Philips Semiconductors SAA7133/SAA713...
00:0a.0 Network controller: RaLink RT2500 802.11g Cardbus/mini-PCI
00:0f.0 RAID bus controller: VIA Technologies, Inc. VIA VT6420 SATA ...
00:0f.1 IDE interface: VIA Technologies, Inc. VT82C586A/B/VT82C686/A...
00:10.0 USB Controller: VIA Technologies, Inc. VT82xxxxx UHCI USB 1...
00:10.1 USB Controller: VIA Technologies, Inc. VT82xxxxx UHCI USB 1...
...
```

D.2. interrupts

D.2.1. about interrupts

An **interrupt request** or **IRQ** is a request from a device to the CPU. A device raises an interrupt when it requires the attention of the CPU (could be because the device has data ready to be read by the CPU).

Since the introduction of pci, irq's can be shared among devices.

Interrupt 0 is always reserved for the timer, interrupt 1 for the keyboard. IRQ 2 is used as a channel for IRQ's 8 to 15, and thus is the same as IRQ 9.

D.2.2. /proc/interrupts

You can see a listing of interrupts on your system in **/proc/interrupts**.

```
paul@laika:~$ cat /proc/interrupts
      CPU0      CPU1
0:   1320048      555  IO-APIC-edge    timer
1:    10224        7  IO-APIC-edge    i8042
7:         0         0  IO-APIC-edge    parport0
8:         2         1  IO-APIC-edge    rtc
10:    3062       21  IO-APIC-fasteoi  acpi
12:     131        2  IO-APIC-edge    i8042
15:   47073        0  IO-APIC-edge    ide1
18:         0         1  IO-APIC-fasteoi  yenta
19:   31056        1  IO-APIC-fasteoi  libata, ohci1394
20:   19042        1  IO-APIC-fasteoi  eth0
21:   44052        1  IO-APIC-fasteoi  uhci_hcd:usb1, uhci_hcd:usb2,...
22:  188352        1  IO-APIC-fasteoi  ra0
```

```
23: 632444      1 IO-APIC-fastecoi  nvidia
24: 1585        1 IO-APIC-fastecoi  VIA82XX-MODEM, VIA8237
```

D.2.3. dmesg

You can also use **dmesg** to find irq's allocated at boot time.

```
paul@laika:~$ dmesg | grep "irq 1[45]"
[ 28.930069] ata3: PATA max UDMA/133 cmd 0x1f0 ctl 0x3f6 bmdma 0x2090 irq 14
[ 28.930071] ata4: PATA max UDMA/133 cmd 0x170 ctl 0x376 bmdma 0x2098 irq 15
```

D.3. io ports

D.3.1. about io ports

Communication in the other direction, from CPU to device, happens through **IO ports**. The CPU writes data or control codes to the IO port of the device. But this is not only a one way communication, the CPU can also use a device's IO port to read status information about the device. Unlike interrupts, ports cannot be shared!

D.3.2. /proc/ioports

You can see a listing of your system's IO ports via **/proc/ioports**.

```
[root@RHEL4b ~]# cat /proc/ioports
0000-001f : dma1
0020-0021 : pic1
0040-0043 : timer0
0050-0053 : timer1
0060-006f : keyboard
0070-0077 : rtc
0080-008f : dma page reg
00a0-00a1 : pic2
00c0-00df : dma2
00f0-00ff : fpu
0170-0177 : ide1
02f8-02ff : serial
...
```

D.4. dma

D.4.1. about dma

A device that needs a lot of data, interrupts and ports can pose a heavy load on the cpu. With **dma** or **Direct Memory Access** a device can gain (temporary) access to a specific range of the **ram** memory.

D.4.2. /proc/dma

Looking at **/proc/dma** might not give you the information that you want, since it only contains currently assigned **dma** channels for **isa** devices.


```
root@laika:~# cat /proc/dma
1: parport0
4: cascade
```

pci devices that are using dma are not listed in **/proc/dma**, in this case **dmesg** can be useful. The screenshot below shows that during boot the parallel port received dma channel 1, and the Infrared port received dma channel 3.

```
root@laika:~# dmesg | egrep -C 1 'dma 1|dma 3'
[ 20.576000] parport: PnPBIOS parport detected.
[ 20.580000] parport0: PC-style at 0x378 (0x778), irq 7, dma 1...
[ 20.764000] irda_init()
--
[ 21.204000] pnp: Device 00:0b activated.
[ 21.204000] nsc_ircc_pnp_probe() : From PnP, found firbase 0x2F8...
[ 21.204000] nsc-ircc, chip->init
```

Appendix E. installing Linux

E.1. about

The past couple of years the installation of Linux has become a lot easier than before, at least for end users installing a distro like Ubuntu, Fedora, Debian or Mandrake on their home computer. Servers usually come pre-installed, and if not pre-installed, then setup of a Linux server today is very easy.

E.2. legacy content

Everything below this paragraph was written between 2003 and 2009 or so. I will rewrite this when I find the time.

E.2.1. about

Linux can be installed in many different ways. End users most commonly use cdrom's or dvd's for installation, most of the time with a working internet connection to receive updates. Administrators might prefer network installations using protocols like **tftp**, **bootp**, **rarp** and/or **nfs** or response file solutions like **Red Hat Kickstart** or **Solaris Jumpstart**.

E.2.2. installation by cdrom

Installation of Linux from cdrom is easy! Most distributions ask very few questions during install (keyboard type, language, username) and detect all the hardware themselves. There is usually no need to retrieve third-party drivers from the internet. The GUI installation gives options like Desktop (for end users), Workstation (for developers), Server or minimal (usually without graphical interface).

E.2.3. installation with rarp and tftp

Installing over the network involves powering on the machine, have it find a rarpd server to get an ip-address, then let it find an tftps server to get an installation image copied to the machine. This image can then boot. The procedure below demonstrates how to setup three Sun SPARC servers with Ubuntu Linux, using a Debian Linux machine to host the tftp, bootp and nfs daemons.

First we need to configure the mac to ip resolution in the **/etc/ethers** configuration file. Each server will receive a unique ip-address during installation.

```
root@laika:~# cat /etc/ethers
00:03:ba:02:c3:82      192.168.1.71
00:03:ba:09:7c:f9      192.168.1.72
00:03:ba:09:7f:d2      192.168.1.73
```

We need to install the rarpd and tftpd daemons on the (Debian) machine that will be hosting the install image.

```
root@laika:~# aptitude install rarpd
```

```
root@laika:~# aptitude install tftpd
```

The tftp services must be activated in inetd or xinetd.

```
root@laika:~# cat /etc/inetd.conf | tail -1
tftp dgram udp wait nobody /usr/sbin/tcpd /usr/sbin/in.tftpd /srv/tftp
```

And finally the Linux install image must be present in the tftp served directory. The filename of the image must be the hex ip-address, this is accomplished with symbolic links.

```
root@laika:~# ll /srv/tftp/
total 7.5M
lrwxrwxrwx 1 root root 13 2007-03-02 21:49 C0A80147 -> ubuntu610.img
lrwxrwxrwx 1 root root 13 2007-03-03 14:13 C0A80148 -> ubuntu610.img
lrwxrwxrwx 1 root root 13 2007-03-02 21:49 C0A80149 -> ubuntu610.img
-rw-r--r-- 1 paul paul 7.5M 2007-03-02 21:42 ubuntu610.img
```

Time to enter **boot net** now in the openboot prompt. Twenty minutes later the three servers where humming with Linux.

E.2.4. about Red Hat kickstart

Automating Linux installations with response files can be done with **Red Hat kickstart**. One way to set it up is by using the graphical tool **/usr/sbin/system-config-kickstart**. If you prefer to set it up manually, read on.

You can modify the sample kickstart file **RH-DOCS/sample.ks** (can be found on the documentation dvd). Put this file so **anaconda** can read it.

*Anaconda is the Red Hat installer written in **python**. The name is chose because anacondas are lizard-eating pythons. Lizard is the name of the Caldera Linux installation program.*

Another option is to start with the **/root/anaconda-ks.cfg** file. This is a sample kickstart file that contains all the settings from your current installation.

Do not change the order of the sections inside your kickstart file! The Red Hat System Administration Guide contains about 25 pages describing all the options, most of them are easy ti understand if you already performed a couple of installations.

E.2.5. using kickstart

To use kickstart, name your kickstart file **ks.cfg** and put it in the root directory of your installation cdrom (or on a usb stick or a floppy). For network based installations, name the file **\$ip-address-kickstart** and place the following in **dhcpd.conf**.

```
filename "/export/kickstart"
next-server remote.installation.server
```

Leaving out the **next-server** line will result in the client looking for the file on the dhcp server itself.

Booting from cdrom with kickstart requires the following command at the **boot:** prompt.

```
linux ks=cdrom:/ks.cfg
```

When the kickstart file is on the network, use nfs or http like in these examples.

```
linux ks=nfs:servername:/path/to/ks.cfg
```

```
linux ks=http://servername/path/to/ks.cfg
```

Appendix F. disk quotas

F.1. About Disk Quotas

To limit the disk space used by user, you can set up **disk quotas**. This requires adding **usrquota** and/or **grpquota** to one or more of the file systems in **/etc/fstab**.

```
root@RHELV4u4:~# cat /etc/fstab | grep usrquota
/dev/VolGroup00/LogVol02      /home      ext3      usrquota,grpquota    0 0
```

Next you need to remount the file system.

```
root@RHELV4u4:~# mount -o remount /home
```

The next step is to build the **quota.user** and/or **quota.group** files. These files (called the **quota files**) contain the table of the disk usage on that file system. Use the **quotacheck** command to accomplish this.

```
root@RHELV4u4:~# quotacheck -cug /home
root@RHELV4u4:~# quotacheck -avug
```

The **-c** is for create, **u** for user quota, **g** for group, **a** for checking all quota enabled file systems in **/etc/fstab** and **v** for verbose information. The next step is to edit individual user quotas with **edquota** or set a general quota on the file system with **edquota -t**. The tool will enable you to put **hard** (this is the real limit) and **soft** (allows a grace period) limits on **blocks** and **inodes**. The **quota** command will verify that quota for a user is set. You can have a nice overview with **repquota**.

The final step (before your users start complaining about lack of disk space) is to enable quotas with **quotaon(1)**.

```
root@RHELV4u4:~# quotaon -vaug
```

Issue the **quotaoff** command to stop all complaints.

```
root@RHELV4u4:~# quotaoff -vaug
```

F.2. Practice Disk quotas

1. Implement disk quotas on one of your new partitions. Limit one of your users to 10 megabyte.
2. Test that they work by copying many files to the quota'd partition.

Appendix G. introduction to vnc

G.1. About VNC

VNC can be configured in gnome or KDE using the **Remote Desktop Preferences**. VNC can be used to run your desktop on another computer, and you can also use it to see and take over the Desktop of another user. The last part can be useful for help desks to show users how to do things. VNC has the added advantage of being operating system independent, a lot of products (realvnc, tightvnc, xvnc, ...) use the same protocol on Solaris, Linux, BSD and more.

G.2. VNC Server

Starting the vnc server for the first time.

```
[root@RHELv4u3 conf]# rpm -qa | grep -i vnc
vnc-server-4.0-8.1
vnc-4.0-8.1
[root@RHELv4u3 conf]# vncserver :2

You will require a password to access your desktops.

Password:
Verify:
xauth:  creating new authority file /root/.Xauthority

New 'RHELv4u3.localdomain:2 (root)' desktop is RHELv4u3.localdomain:2

Creating default startup script /root/.vnc/xstartup
Starting applications specified in /root/.vnc/xstartup
Log file is /root/.vnc/RHELv4u3.localdomain:2.log

[root@RHELv4u3 conf]#
```

G.3. VNC Client

You can now use the **vncviewer** from another machine to connect to your vnc server. It will default to a very simple graphical interface...

```
paul@laika:~$ vncviewer 192.168.1.49:2
VNC viewer version 3.3.7 - built Nov 20 2006 13:05:04
Copyright (C) 2002-2003 RealVNC Ltd.
Copyright (C) 1994-2000 AT&T Laboratories Cambridge.
See http://www.realvnc.com for information on VNC.
VNC server supports protocol version 3.8 (viewer 3.3)
Password:
VNC authentication succeeded
Desktop name "RHELv4u3.localdomain:2 (root)"
Connected to VNC server, using protocol version 3.3
...
```

If you don't like the simple twm window manager, you can comment out the last two lines of `~/.vnc/xstartup` and add a **gnome-session &** line to have vnc default to gnome instead.

```
[root@RHELv4u3 ~]# cat .vnc/xstartup
#!/bin/sh

# Uncomment the following two lines for normal desktop:
# unset SESSION_MANAGER
# exec /etc/X11/xinit/xinitrc

[ -x /etc/vnc/xstartup ] && exec /etc/vnc/xstartup
[ -r $HOME/.Xresources ] && xrdb $HOME/.Xresources
xsetroot -solid grey
vncconfig -iconic &
# xterm -geometry 80x24+10+10 -ls -title "$VNCDESKTOP Desktop" &
# twm &
gnome-session &
[root@RHELv4u3 ~]#
```

Don't forget to restart your vnc server after changing this file.

```
[root@RHELv4u3 ~]# vncserver -kill :2
Killing Xvnc process ID 5785
[root@RHELv4u3 ~]# vncserver :2

New 'RHELv4u3.localdomain:2 (root)' desktop is RHELv4u3.localdomain:2

Starting applications specified in /root/.vnc/xstartup
Log file is /root/.vnc/RHELv4u3.localdomain:2.log
```

G.4. Practice VNC

1. Use VNC to connect from one machine to another.

Appendix H. cloning

H.1. About cloning

You can have distinct goals for cloning a server. For instance a clone can be a cold iron backup system used for manual disaster recovery of a service. Or a clone can be created to serve in a test environment. Or you might want to make an almost identical server. Let's take a look at some offline and online ways to create a clone of a Linux server.

H.2. About offline cloning

The term offline cloning is used when you power off the running Linux server to create the clone. This method is easy since we don't have to consider open files and we don't have to skip virtual file systems like **/dev** or **/sys**. The offline cloning method can be broken down into these steps:

1. Boot source and target server with a bootable CD
2. Partition, format and mount volumes on the target server
3. Copy files/partitions from source to target over the network

The first step is trivial. The second step is explained in the Disk Management chapter. For the third step, you can use a combination of **ssh** or **netcat** with **cp**, **dd**, **dump** and **restore**, **tar**, **cpio**, **rsync** or even **cat**.

H.3. Offline cloning example

We have a working Red Hat Enterprise Linux 5 server, and we want a perfect copy of it on newer hardware. First thing to do is discover the disk layout.

```
[root@RHEL5 ~]# df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2       15G   4.5G   9.3G  33% /
/dev/sda1       99M    31M   64M  33% /boot
```

The **/boot** partition is small but big enough. If we create an identical partition, then **dd** should be a good cloning option. Suppose the **/** partition needs to be enlarged on the target system. The best option then is to use a combination of **dump** and **restore**. Remember that **dd** copies blocks, whereas **dump/restore** copies files.

The first step to do is to boot the target server with a live CD and partition the target disk. To do this we use the Red Hat Enterprise Linux 5 install CD. At the CD boot prompt we type "linux rescue". The cd boots into a root console where we can use **fdisk** to discover and prepare the attached disks.

When the partitions are created and have their filesystem, then we can use **dd** to copy the **/** boot partition.


```
ssh root@192.168.1.40 "dd if=/dev/sda1 | dd of=/dev/sda1"
```

Then we use a dump and restore combo to copy the / partition.

```
mkdir /mnt/x  
mount /dev/sda2 /mnt/x  
cd /mnt/x  
ssh root@192.168.1.40 "dump -0 -f - /" | restore -r -f -
```

Appendix I. License

GNU Free Documentation License

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc.

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document "free" in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or noncommercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of "copyleft", which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The "Document", below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as "you". You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A "Modified Version" of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A "Secondary Section" is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document's overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The "Invariant Sections" are certain Secondary Sections whose titles

are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The "Cover Texts" are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A "Transparent" copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not "Transparent" is called "Opaque".

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The "Title Page" means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, "Title Page" means the text near the most prominent appearance of the work's title, preceding the beginning of the body of the text.

The "publisher" means any person or entity that distributes copies of the Document to the public.

A section "Entitled XYZ" means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as "Acknowledgements", "Dedications", "Endorsements", or "History".) To "Preserve the Title" of such a section when you modify the Document means that it remains a section "Entitled XYZ" according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either

commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- * A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.

* B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.

* C. State on the Title page the name of the publisher of the Modified Version, as the publisher.

* D. Preserve all the copyright notices of the Document.

* E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.

* F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.

* G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.

* H. Include an unaltered copy of this License.

* I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.

* J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.

* K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.

* L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.

* M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.

* N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.

* O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled "Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties—for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of,

you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled "Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled "Endorsements".

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an "aggregate" if the copyright resulting from the compilation is not used to limit the legal rights of the compilation's users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document's Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled "Acknowledgements", "Dedications", or "History", the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <http://www.gnu.org/copyleft/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License "or any later version" applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies

that a proxy can decide which future versions of this License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Document.

11. RELICENSING

"Massive Multiauthor Collaboration Site" (or "MMC Site") means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A "Massive Multiauthor Collaboration" (or "MMC") contained in the site means any set of copyrightable works thus published on the MMC site.

"CC-BY-SA" means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

"Incorporate" means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is "eligible for relicensing" if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.

Index

Symbols

; (shell), 135
!! (shell), 155
! (bash history), 155
! (file globbing), 162
? (file globbing), 161
/, 75, 101
/bin, 102, 127
/bin/bash, 124, 291
/bin/cat, 102
/bin/csh, 124
/bin/date, 102
/bin/dmesg, 370
/bin/ksh, 124, 291
/bin/login, 501
/bin/rm, 128
/bin/sh, 124
/boot, 104
/boot/grub, 104
/boot/grub/grub.cfg, 104
/boot/grub/grub.conf, 104, 488
/boot/grub/menu.lst, 488
/dev, 84, 108, 381
/dev/hdX, 368
/dev/ht, 679
/dev/nst, 679
/dev/null, 108, 174
/dev/pts/1, 108
/dev/random, 119
/dev/sdb, 422
/dev/sdX, 368
/dev/st, 679
/dev/tty1, 108
/dev/urandom, 118, 120
/dev/zero, 119
/etc, 104
/etc/apache2, 696
/etc/apt/sources.list, 563
/etc/at.allow, 523
/etc/at.deny, 523
/etc/bashrc, 292
/etc/bind/named.conf.local, 762
/etc/cron.allow, 524
/etc/cron.d, 525
/etc/cron.deny, 524
/etc/crontab, 525
/etc/default/grub, 493
/etc/default/useradd, 275
/etc/ethers, 954
/etc/exports, 637, 648
/etc/filesystems, 394, 401
/etc/fstab, 325, 397, 405, 422, 543, 638, 648, 957
/etc/group, 298, 307, 731
/etc/grub.conf, 488
/etc/grub.d/40_custom, 493
/etc/gshadow, 300
/etc/hostname, 599
/etc/hosts, 119
/etc/httpd, 696
/etc/inetd.conf, 645, 853
/etc/init.d/, 104, 503, 505
/etc/init.d/rc, 500
/etc/init.d/rcS, 499
/etc/init.d/samba, 842
/etc/init.d/smb, 842
/etc/init.d/winbind, 843
/etc/inittab, 498, 500, 501
/etc/inputrc, 291
/etc/login.defs, 285
/etc/lvm/.cache, 447
/etc/modprobe.conf, 665
/etc/modprobe.d/, 665
/etc/mtab, 402, 499
/etc/network/interfaces, 592, 616, 619
/etc/nsswitch.conf, 891, 893
/etc/passwd, 190, 274, 277, 286, 286, 307, 501, 731, 900
/etc/profile, 291
/etc/protocols, 589
/etc/raidtab, 430
/etc/rc.d/rc, 500
/etc/rc.d/rc.sysinit, 499
/etc/rcS.d/, 499
/etc/rcX.d/, 500
/etc/resolv.conf, 119, 748
/etc/rsyslog.conf, 534
/etc/samba/passdb.tdb, 899
/etc/samba/smb.conf, 847, 848, 849, 865, 889
/etc/samba/smbpasswd, 870, 897
/etc/selinux/config, 824
/etc/services, 589, 645
/etc/shadow, 282, 284, 320
/etc/shells, 234, 277
/etc/shutdown.allow, 509
/etc/skel, 104, 276
/etc/squid/squid.conf, 720
/etc/ssh, 624
/etc/ssh/ssh_config, 624
/etc/ssh/sshd_config, 624
/etc/sudoers, 268, 269
/etc/sysconfig, 104
/etc/sysconfig/firstboot, 105
/etc/sysconfig/harddisks, 105
/etc/sysconfig/hwconf, 105
/etc/sysconfig/iptables, 641
/etc/sysconfig/keyboard, 105
/etc/sysconfig/network, 594
/etc/sysconfig/network-scripts/, 594
/etc/sysconfig/network-scripts/ifcfg-bond0, 617
/etc/sysctl.conf, 801
/etc/syslog.conf, 532
/etc/X11/xorg.conf, 104
/etc/xinetd.conf, 644

/etc/xinetd.d, 644
/etc/xinetd.d/swat, 853
/etc/yum.conf, 572
/etc/yum.repos.d/, 572
/export, 106
/home, 106
/lib, 103, 673
/lib/kbd/keymaps/, 105
/lib/modules, 103, 662, 668
/lib/modules/<kernel-version>/modules.dep, 665
/lib32, 103
/lib64, 103
/media, 106
/opt, 103
/proc, 84, 108
/proc/bus, 950
/proc/bus/pci, 951
/proc/bus/usb/devices, 950
/proc/cmdline, 490, 653
/proc/cpuinfo, 109
/proc/devices, 381, 381
/proc/dma, 952
/proc/filesystems, 394, 401
/proc/interrupts, 111, 951
/proc/ioports, 952
/proc/kallsyms, 661
/proc/kcore, 111
/proc/mdstat, 430
/proc/meminfo, 540, 541
/proc/modules, 663
/proc/mounts, 402
/proc/net/bonding, 617, 619
/proc/partitions, 381
/proc/scsi/scsi, 373
/proc/swaps, 542
/proc/sys, 110
/proc/sys/net/ipv4/ip_forward, 801
/root, 106
/root/anaconda-ks.cfg, 955
/run, 116
/sbin, 102, 127, 597
/sbin/init, 498
/sbin/mingetty, 501
/sbin/telinit, 508
/selinux, 826
/selinux/enforce, 826
/srv, 106
/sys, 112
/tmp, 107, 319
/usr, 113
/usr/bin, 113
/usr/bin/getfacl, 325
/usr/bin/passwd, 320
/usr/bin/setfacl, 325
/usr/include, 113
/usr/lib, 113, 673
/usr/local, 113
/usr/sbin/system-config-kickstart, 955
/usr/share, 113
/usr/share/doc, 594
/usr/share/games, 114
/usr/share/man, 114
/usr/src, 114, 657
/var, 115
/var/cache, 115
/var/lib, 116
/var/lib/nfs/etab, 637, 648
/var/lib/rpm, 116, 565
/var/lib/usbutils/usb.ids, 950
/var/lock, 116
/var/log, 115
/var/log/audit/audit.log, 822
/var/log/auth.log, 531
/var/log/btmp, 529, 530
/var/log/lastlog, 529
/var/log/messages, 115, 654, 671
/var/log/sa, 551
/var/log/secure, 531
/var/log/squid, 725
/var/log/syslog, 115
/var/log/wtmp, 509, 529
/var/run, 116
/var/run/utmp, 529
/var/spool, 115
/var/tmp, 116
., 74
./configure, 574
.., 74
.. (directory), 330
. (directory), 330
. (shell), 235
.bash_history, 156
.bash_login, 292
.bash_logout, 293
.bash_profile, 291
.bashrc, 291, 292
.deb, 554
.exrc, 228
.htaccess, 713
.htpasswd, 703, 710
.my.cnf, 732
.rpm, 554
.ssh, 628
.vimrc, 228
` (backtick), 150
~, 74
~/.ssh/authorized_keys, 629
' (single quote), 150
" (double quotes), 126
(((shell), 255
-- (shell), 236
[(file globbing), 162
[(shell), 240
\$? (shell variables), 135
\$(embedded shell, 150
\$ (shell variables), 141

\$\$, 339
\$HISTFILE, 156
\$HISTFILESIZE, 156
\$HISTSIZ, 156
\$LANG, 163
\$PATH, 127, 144
\$PPID, 339
\$PS1, 75
* (file globbing), 161
\ (backslash), 137
&, 135
&&, 136
#!/bin/bash, 234
#! (shell), 234
(pound sign), 137
>, 172
>>, 173
>|, 173
||, 136
1>, 174
2>, 174
2>&1, 174
777, 313

A

A (DNS record), 753
AAAA (DNS record), 753
access control list, 325
access time, 366
acl, 327
acls, 325
active partition, 491
agp, 950
AIX, 4
alias(bash), 128
alias(shell), 128
Alicia and Bob, 625
allow hosts (Samba), 882
anycast, 585
apache2, 692
apropos, 71
apt-get(8), 555, 559
aptitude, 674, 839, 840, 912
aptitude(1), 658
aptitude(8), 555, 562, 730
arguments(shell), 125
arp(1), 600
arp table, 600
at(1), 521, 522
ata, 366
atapi, 366
atm, 587
atq(1), 522
atrm(1), 523
auditd, 822
authoritative (dns), 757
authoritative zone, 752
axfr, 760

B

backticks, 150
badblocks(8), 374
base64, 176
bash, 218, 247
bash history, 155
bash -x, 236
bg(1), 358
Bill Callkins, 486
binaries, 102
bind, 750
bind(DNS), 776
binding, 615
binding(ip), 614
BIOS, 485
block device, 367
bonding(ip), 614
boot(grub), 490
bootloader, 487
bootp, 594, 610, 954
Bourne again shell, 124
broadcast, 585
browsable (Samba), 883
browseable (Samba), 883
browser master, 897
BSD, 4, 485
btrfs, 393
bum(8), 507
bunzip2, 200
bus, 950
bzipcat, 200
bzImage, 490
bzip2, 198, 200, 200, 490
bzip2(1), 680
bzipmore, 200

C

cable select, 366
caching only name server, 754
cal, 197
Canonical, 497
case, 257
case sensitive, 84
cat, 95, 181
cd, 74
cd -, 75
CentOS, 6
chage, 285
chain (iptables), 808
chainloader, 491
chainloading, 491
char(mysql), 735
character device, 367
chattr(1), 683
chcon(1), 828, 829
chgrp(1), 308
chkconfig, 105, 503, 822

chkconfig(8), 504
chmod, 276, 313, 825
chmod(1), 225, 312
chmod +x, 234, 314
chown, 276
chown(1), 308
chroot, 414
CHS, 367
chsh(1), 277
CIFS, 844
Cisco, 587
CMDDBA, 932
CMDEV, 932
CNAME (DNS record), 753
comm(1), 187
command line scan, 125
command mode(vi), 222
context type(selinux), 827
copyleft, 10
copyright, 9, 9
cp, 87
cp(1), 87
cpio(1), 565, 683
cpu, 950
create(mysql), 733, 735, 741
create mask (Samba), 883
cron(8), 521
crontab(1), 524
crontab(5), 524
crypt, 283
csh, 234
Ctrl-Alt-Delete, 509, 509
Ctrl d, 95
ctrl-r, 156
Ctrl-Z, 357
current directory, 74
cut, 190
cut(1), 183
cylinder, 366

D

daemon, 71, 338, 503
date, 196
dd(1), 385, 487, 543, 684
deb(5), 555
Debian, 6
debsums, 674
default(grub), 489, 491
default gateway, 601
delete(mysql), 740
Dennis Ritchie, 4
deny hosts (Samba), 883
depmod(1), 665
describe(mysql), 736
devfs, 112
device driver, 381
devices.txt, 381
df, 547

df(1), 403, 403
df -i, 329
dhclient, 803
dhclient(1), 599
dhcp, 594, 610
dhcp client, 592, 599
dhcpd.conf, 955
dhcp server, 748
directory, 330, 391
directory mask (Samba), 883
directory security mask(samba), 884
disk platters, 366
distribution, 5
distributions, 101
dma, 952
dmesg(1), 370, 952, 953
dmesg(8), 655
DNAT, 800
dns, 610, 746, 746
dnsdomainname, 752
dns namespace, 749
dns server, 748
domain (dns), 750
domain name system, 746, 746
DOS, 491
dpkg, 839
dpkg(1), 730
dpkg(8), 555, 557
dpkg -S, 674
drop(mysql), 734, 736, 742
dsa, 625
du, 547
du(1), 403
dump(1), 683
dumpkeys(1), 105

E

e2fsck(1), 397
echo, 125
echo(1), 124, 126, 339
echo \$-, 151
echo *, 164
edquota(1), 957
Edubuntu, 6
egrep, 499
eiciel, 327
ELF, 103
elif, 241
elilo, 487
el torito, 393
embedding(shell), 150
env(1), 145, 145
environment variable, 141
EOF, 95, 176
Eric Allman, 532
escaping (shell), 164
eth0, 592
ethtool(1), 602

eval, 255
Evi Nemeth, 503
exec, 340
executables, 102
exit (bash), 156
export, 145
exportfs(1), 637, 648
ext2, 392, 395
ext3, 392
extended partition, 380

F

fallback(grub), 489
fat16, 393
fat32, 393
fd (partition type), 429
fddi, 587
fdisk, 469
fdisk(1), 381, 383, 384, 429
fdisk(8), 369
fdisk limitations, 386
Fedora, 6
fg(1), 358
FHS, 101
file, 84
file(1), 103
file globbing, 160
file ownership, 307
file system, 390
Filesystem Hierarchy Standard, 101
filters, 180
filter table (iptables), 808
find(1), 195, 319, 320, 331
firewall, 799
FireWire, 112
fixed ip, 595
fixed ip address, 592
for (bash), 241
force create mode(samba), 884
force directory mode(samba), 884
force directory security mode(samba), 884
force group(samba), 871
force security mode(samba), 884
force user(samba), 871
fork, 340
forwarder (dns), 756
forward lookup query, 747
FOSS, 9
four freedoms, 10
FQDN, 599
fqdn, 752
frame relay, 587
free, 547, 548
free(1), 540, 541
Free Software, 9
free software, 9
freeware, 9
fsck(1), 397

ftp, 644
ftp://ftp.kernel.org, 656
fully qualified domain name, 752
function (shell), 258
fuser, 413, 413

G

gateway, 601
gcc(1), 284
getenforce, 823
getent(1), 892
getfacl, 325
getfattr(1), 829
getopts, 250
GID, 298
git, 911
github, 917
glob(7), 161
glue record (dns), 753
gnome-session, 958
GNU, 4
gpasswd, 300
GPL, 10
GPLv3, 10
gpt, 386
grant(mysql), 734
grep, 205, 206, 209, 499, 663
grep(1), 181
grep -i, 181
grep -v, 182
groupadd(1), 298
group by(mysql), 740
groupdel(1), 299
groupmod(1), 299
groups, 297
groups(1), 298
grpquota, 957
grub, 484, 487, 491
grub2, 488
grub-install, 492
guest ok (Samba), 858
gunzip(1), 199
gzip, 199
gzip(1), 199, 490, 680

H

halt(8), 509
hard link, 331
hdparm(8), 375
head(1), 94
head (hard disk device), 366
here directive, 96
here document, 176
here string, 176
hidden files, 76
hiddenmenu(grub), 489
hide unreadable (Samba), 883

host (DNS record), 753
hostname, 599, 752, 844
hostname(1), 599
hosts.txt, 746
hosts allow (Samba), 882
hosts deny (Samba), 883
HP, 4
HP-UX, 4
htpasswd(1), 703, 710
<http://www.kernel.org>, 656
<http://www.pathname.com/fhs/>, 101
httpd, 693

I

IBM, 4, 844
icmp, 589
id, 266
id_dsa, 629
id_dsa.pub, 629
id_rsa, 628
id_rsa.pub, 628
id(1), 828
ide, 381
identity(selinux), 826
idmap gid(samba), 889
idmap uid(samba), 889
IEEE 1394, 112
ifcfg(1), 615
ifcfg-eth0, 595
ifconfig(1), 596, 597, 615, 616, 617, 619
ifdown(1), 593, 596, 597, 615
ifenslave, 619
if then else (bash), 241
iftop(1), 552
ifup(1), 593, 596, 597, 615, 616, 617
igmp, 589
inetd, 644
inetd(8), 853
init, 338, 498, 509
init=/bin/bash, 654
initiator(iSCSI), 461
initng, 497
initrd, 660
initrd(grub), 490
inode, 328, 331
inode table, 329
insert(mysql), 737
insert mode(vi), 222
insmod(1), 664, 664
integer(mysql), 735
Intel, 485
interrupt, 951
invalid users (Samba), 882
IO Ports, 952
iostat, 415
iostat(1), 550
iotop, 416
iptables, 641, 807, 808

iptables save, 812
IRQ, 951
isa, 950
iSCSI, 461
iscsiadm, 468
iso9660, 393, 684
iterative query, 756
ixfr, 760

J

jbod, 427
jobs, 357
joliet, 393
journaling, 392
Jumpstart, 954

K

Ken Thompson, 4
Kerberos, 636, 647
kernel, 103
kernel(grub), 490
keymaps(5), 105
kickstart, 954, 955
kill(1), 338, 343, 343, 501, 502
killall(1), 345
kmyfirewall, 641
Korn shell, 157
Korn Shell, 277
ks.cfg, 955
ksh, 157, 234
kudzu, 105

L

LAMP, 729
LAN, 586
last(1), 509, 529
lastb(1), 530
lastlog(1), 530
LBA, 367
ldap, 637
ldd, 673
less(1), 97
let, 256
libraries, 673
lilo, 487, 494
lilo.conf, 494
Linus Torvalds, 4
Linux Mint, 6
ln, 332
ln(1), 331
loadkeys(1), 105
locate(1), 196
logger(1), 535
logical AND, 136
logical drive, 380
logical drives, 385
logical OR, 136

Logiciel Libre, 9
login, 529
logrotate(1), 536
LPIC 1 Certification, 932
LPIC 2 Certification, 932
ls, 76, 310, 329, 825
ls(1), 76, 329, 330, 829
ls -l, 309
lsmod, 663
lsmod(1), 662
lsnf, 412
lspci, 951
lsscsi(1), 372
lsusb, 950
ltrace, 674
lvcreate(1), 437, 439, 453
lvdisplay(1), 440, 448
lvextend(1), 440, 454
lvm, 414
LVM, 434
lvmdiskscan(1), 445
lvof, 453
lvremove(1), 453
lvrename(1), 454
lvs(1), 448
lvscan(1), 448

M

mac address, 597, 802
magic, 84
major number, 381
make, 671
make(1), 574
make bzImage, 668
make clean, 667
make menuconfig, 667
make modules, 668
make mrproper, 667
makewhatis, 72
make xconfig, 667
MAN, 586
man(1), 71, 71, 72
mandb(1), 72
mangle table (iptables), 808
man hier, 101
man -k, 71
masquerading, 800
master (hard disk device), 366
master boot record, 385, 487
master server (DNS), 759
mbr, 385, 385, 386, 487
MBR, 684
md5, 284
mdadm(1), 430
mingetty, 501
minor number, 381
mirror, 427
mkdir, 276, 401

mkdir(1), 78, 314
mkdir -p, 78
mke2fs(1), 392, 395, 439
mkfifo, 350
mkfile(1), 543
mkfs, 329
mkfs(1), 392, 395
mkinitrd(1), 392, 669
mknod(1), 679
mkswap(1), 542
modinfo, 671
modinfo(1), 664
modprobe(1), 664, 665
more(1), 97
mount, 401
mount(1), 400, 402, 638, 648
mounting, 400
mount point, 400
mpstat(1), 551
mt(1), 679
multicast, 584
multipath, 474
mv, 88
MX (DNS record), 753
mysql, 729, 731, 732, 733
mysql(group), 731
mysql(user), 731
mysql-client, 730
mysqld, 731
mysql-server, 730

N

NAPT, 800
NAT, 800
nat table (iptables), 808
NetBIOS names, 844
netcat, 861
net groupmap, 902
net rpc join(samba), 890
netstat(1), 601
net use(microsoft), 860, 865, 876
net view(microsoft), 847, 852
network file system, 635
nfs, 635, 636, 954
NFS, 647
nice, 352
nice(1), 350
nmbd(8), 843
no_subtree_check(nfs), 637
noacl(mount), 406
noclobber, 173
nodev, 394, 401
noexec(mount), 406
nosuid(mount), 406
nounset(shell), 146
Novell Certified Linux Professional, 933
NS (DNS record), 753
nslookup, 747

NT_STATUS_BAD_NETWORK_NAME, 877
NT_STATUS_LOGON_FAILURE, 877
ntop(1), 552

O

octal permissions, 313
od(1), 188, 487
OEL, 6
OpenBoot(Sun), 486
OpenBSD, 624
open source, 9
open source definition, 10
open source software, 9
openssh, 624
openssh-server, 631
openssl, 283
Oracle Enterprise Linux, 6
order by(mysql), 739
OS/2, 491
owner, 310

P

package management, 554
packet filtering, 799, 809
packet forwarding, 799
paging, 540
PAN, 587
Parallel ATA, 366
parent directory, 74
parity(raid), 427
parted, 386, 387
parted(1), 383
partition, 380
partition table, 385, 385
partprobe(1), 385
passdb backend (Samba), 871
passwd, 282, 282, 283, 285
passwd(1), 72, 320
passwd(5), 72
password(grub), 489
PAT, 800
path, 75, 76
Paul Mockapetris, 746
pc-card, 950
pci, 950
pci-express, 950
pcmcia, 950
perl, 211
perldoc, 211
pgrep(1), 342
php, 729
PID, 338
pidof(1), 339
ping, 589, 601, 802, 803
pipes, 350
pkill(1), 345
popd, 82

port forwarding, 800
portmap, 636, 647
POST, 485
poweroff(8), 509
Power On Self Test, 485
PPID, 338
prename, 211
primary dns server, 757
primary group, 275
primary partition, 380, 487, 491
primary server (DNS), 759
private key, 625
process, 338
process id, 338
proprietary, 9
proxy server, 719
ps, 341, 547
ps(1), 829
ps -ef, 341
ps fax, 341
PTR (DNS record), 753
public domain, 9
public key, 625, 917
pushd, 82
pvchange(1), 450
pvcreate(1), 437, 439, 449
pvdisplay(1), 439, 446
pvmove(1), 450
pvremove(1), 449
pvresize(1), 449
pvs(1), 445
pvscan(1), 445
pwd, 74
pwd(1), 75

Q

query (dns), 747
quota.group, 957
quota.user, 957
quota's, 957
quota(1), 957
quotacheck(1), 957
quotaoff(1), 957
quotaon(1), 957

R

RAID, 426
raid 1, 427
random number generator, 119
rarp, 954
read, 248
read list (Samba), 882
read only (Samba), 865
reboot, 156
reboot(8), 509
recursive query, 756
Red Hat, 6

regular expressions, 157
reiserfs, 393
Remote Desktop, 958
rename, 89, 211, 212, 213
renice, 351
renice(1), 350
repository, 5, 554, 555
repquota(1), 957
resize2fs(1), 440
respawn(init), 501, 501
restore(1), 683
reverse lookup query, 747
rfc 3010, 636
rfc 3530, 636
RHCE, 932
Richard Stallman, 4
rlogin, 624
rm, 86
rm(1), 332
rmdir(1), 78
rmdir -p, 79
rmmod(1), 665
rm -rf, 86
roaming profiles(samba), 901
rock ridge, 393
role(selinux), 826
root, 102, 267, 268, 269, 274
root(DNS), 749
root(grub), 490
root(mysql), 730
root directory, 101
root hints, 750
root server (dns), 755
root servers(DNS), 585
root servers (dns), 749
rootsquash, 637, 648
rotational latency, 366
route(1), 601, 601
router, 587, 799
rpc, 636
RPC, 647
rpcinfo(1), 636, 647
rpm, 116, 564, 839
rpm(1), 730
rpm(8), 555, 840
rpm2cpio(8), 565
rpm -qf, 675
rpm -V, 675
rsa, 625
rsh, 624
rsyslog, 412
runlevel, 498
runlevel(1), 508

S

sa2(1), 551
sadc(1), 551
sal, 551

salt (encryption), 284
samba, 839
sample.ks, 955
sar(1), 551, 551
sata, 366
savedefault(grub), 491
Scientific, 6
scp(1), 629
scsi, 366
scsi id, 366
secondary dns server, 757
secondary server (DNS), 759
sector, 366
security(Samba), 858
security mask(samba), 884
security mode(samba), 875
sed, 189, 214, 215
seek time, 366
select(mysql), 737, 738, 738
SELinux, 821
selinux, 824
selinux-activate, 822
service(1), 503, 641
service(8), 842
sestatus, 824
set, 151
set(shell), 142
set +x, 129
setenforce, 823
setfacl, 325
setgid, 319, 319
setuid, 236, 320, 320, 320, 406
set -x, 129
sfdisk(1), 385
she-bang (shell), 234
shell, 290
shell comment, 137
shell embedding, 150
shell escaping, 137
shell expansion, 125, 125
shell functions, 258
shift, 248
shopt, 251
show(mysql), 733, 735
shutdown(8), 508
SIGHUP, 343
SIGKILL, 508
SIGTERM, 345, 508
silo, 487
single user mode, 654
skeleton, 104
slave (hard disk device), 366
slave server (DNS), 759
sleep, 197
SMB, 844
smbclient, 850, 859
smbclient(1), 849, 876
smbd(8), 843, 847, 870

smbpasswd(1), 902
smbpasswd(8), 870, 875
smbtree, 852
smbtree(1), 851
SMF, 497
smtp, 753
SNAT, 800
soa (dns record), 757
soft link, 332
Solaris, 4, 485
solid state drive, 367
sort, 190
sort(1), 185
source, 235, 249
SPARC, 486
split(1), 685
SQL, 729, 737
squid, 719
ssd, 367
ssh, 624
ssh_host_dsa_key, 631
ssh_host_dsa_key.pub, 631
ssh_host_rsa_key, 631
ssh_host_rsa_key.pub, 631
sshd, 631
ssh-keygen, 628
ssh-keygen(1), 628
ssh -X, 629
standard input, 95
standard output, 95
stanza(grub), 490
stateful firewall, 799
stderr, 171
stdin, 171, 181
stdout, 171, 181
sticky bit, 319
strace, 676
strings(1), 97
striped disk, 427
su, 267, 267, 286, 300, 673
su -, 144
subtree_check(nfs), 637
sudo, 268, 269, 286
sudo su -, 269
Sun, 4, 485, 497
SunOS, 4
superuser, 274
swapoff(1), 542
swapon(1), 542
swap partition, 393
swap partition(s), 544
swapping, 540
swap space, 542
swat, 644
swat(8), 853
symbolic link, 332
sysctl, 801
sysctl(1), 599

sysfs, 112
syslog, 654
syslogd, 532, 532
System.map, 661
system-config-securitylevel, 641
System V, 103, 497

T

tab key(bash), 76
tac, 96
tail(1), 94, 535
tar(1), 574, 680, 681
tcp, 589, 636
tcpdump, 606, 611, 611, 747, 803
tdbsam, 871, 897, 899
tee(1), 181
telinit(8), 508
telnet, 624, 644
test, 240
testparm(1), 848, 848, 849
tftp, 954
time, 198
time(1), 668
timeout(grub), 489
title(grub), 489
tld, 751
TLD (dns), 751
top, 345
top(1), 342, 540, 541, 548
top level domain, 751
touch(1), 85
tr, 184
tr(1), 183
track, 366
transition(selinux), 828
trigger(mysql), 741
triggers(mysql), 730
tune2fs(1), 392, 396, 421
type(selinux), 827
type(shell), 127

U

Ubuntu, 6
udf, 393
udp, 589, 636
uefi, 386
umask(1), 314
unalias(bash), 129
uname(1), 653
uniq, 190
uniq(1), 186
universally unique identifier, 420
Unix, 4
unset, 151
unset(shell), 142
until (bash), 242
update(mysql), 738

updatedb(1), 196
update-grub, 493
update-rc.d, 503
update-rc.d(8), 506
upstart, 497
usb, 112, 950
use(mysql), 734
useradd, 275, 276, 283
useradd(1), 276
useradd -D, 275
userdel(1), 275
usermod, 286, 286, 299
usermod(1), 275
usrquota, 957
uuid, 420

V

valid users (Samba), 882
vanilla, 555
varchar(mysql), 735
vfat, 393
vgchange(1), 452
vgcreate(1), 437, 439, 451
vgdisplay(1), 447
vgextend(1), 451
vgmerge(1), 452
vgreduce(1), 451
vgremove(1), 451
vgs(1), 447
vgscan(1), 447
vi, 301, 676
vi(1), 221
vigr(1), 301
vim(1), 221
vimtutor(1), 221
vipw, 286
virtualbox, 802
virtual memory, 540
visudo, 268
vmlinuz, 660
vmstat, 417, 544
vmstat(1), 549
vmware, 802
vnc, 958
vncviewer(1), 958
vol_id(1), 421
vrije software, 9

W

w, 266
WAN, 587
watch(1), 535, 549
wbinfo(1), 891, 892
wc(1), 184
webalizer, 713
whatis(1), 71
whereis(1), 71

which(1), 127
while (bash), 242
white space(shell), 125
who, 190, 266
who(1), 508, 529
whoami, 266
who am i, 266
wild cards, 162
winbind(8), 891
winbind(samba), 889
winbindd(8), 843, 843, 891
wireshark, 606, 624, 803
workgroup, 858
WPAN, 587
writable (Samba), 865
write list (Samba), 882

X

X, 104
X.25, 587
x86, 485
xinetd, 644, 644
xinetd(8), 853
xstartup(vnc), 958
X Window System, 104

Y

yaboot, 487
yum, 675, 840
yum(8), 566

Z

z/IPL, 487
zcat, 199
zfs, 393
zImage, 490
zmore, 199
zombie, 338
zone (dns), 752, 757
zone transfer (dns), 757